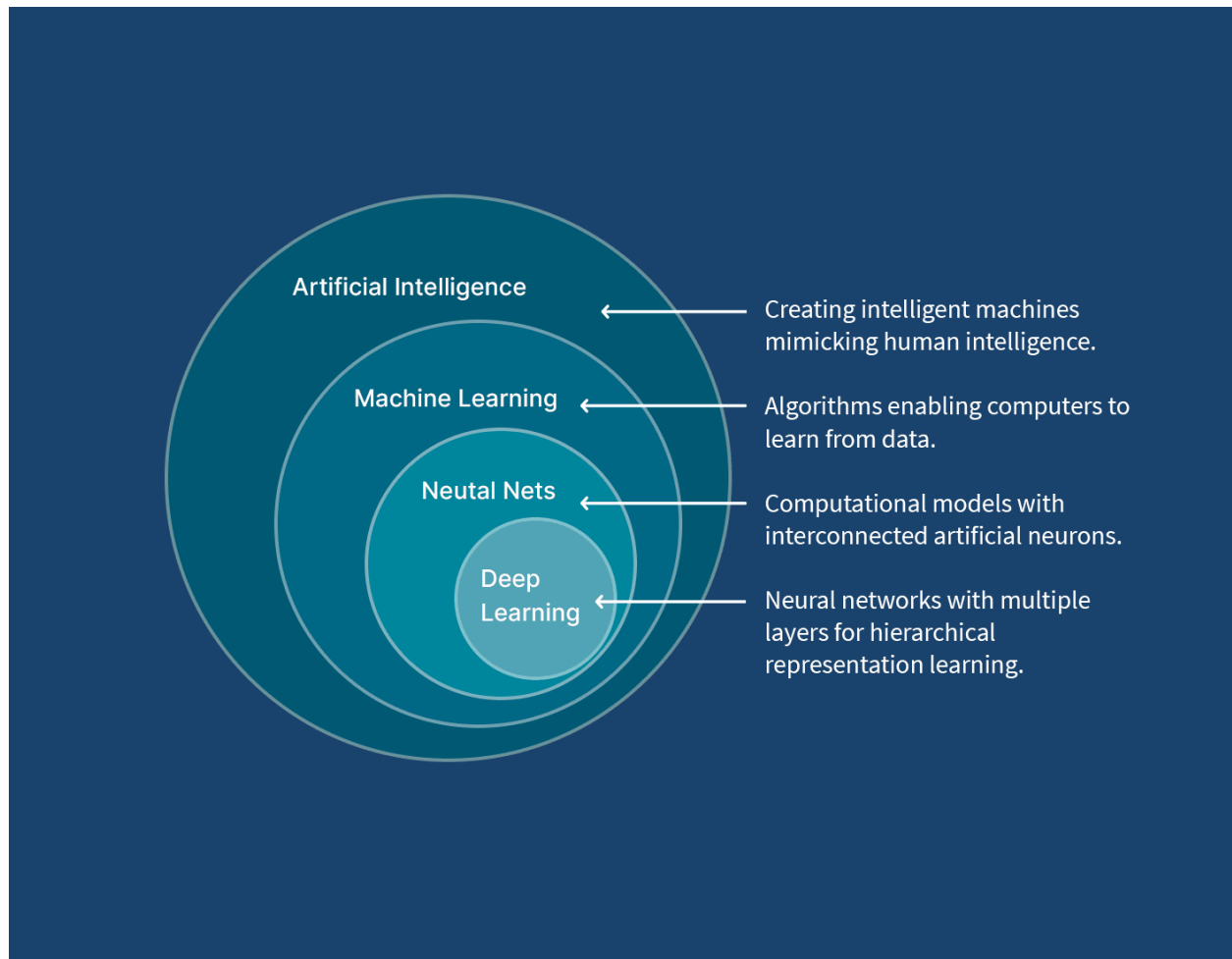


Machine Learning Models



Md. Shaon Khan

B.Sc. in Information Technology — IIT, Jahangirnagar University

সূচিপত্র (Table of Contents)

Contents

Simple Linear Regression	3
Multiple Linear Regression.....	10
Logistic Regression	29
K-Nearest Neighbors (KNN).....	45
GridSearchCV in KNN	66
Decision Tree	81
Random Forest	128
Support Vector Machine	186
আনসুপারভাইজড লার্নিং	223
K-Means Clustering	228
DBSCAN Clustering	258
Hierarchical DBSCAN (HDBSCAN).....	285
Principal Component Analysis.....	297
Naive Bayes	331
L1 (Lasso) এবং L2 (Ridge) Regularization-এর মধ্যে পার্থক্য	343
Linear Regression, Ridge Regression, Lasso Regression এবং Elastic Net-এর মধ্যে পার্থক্য	346
Gradient Boosting Algorithm	351
Agglomerative Clustering	367
t-SNE (t-Distributed Stochastic Neighbor Embedding)	375
Isolation Forest	383
One-Class SVM	390

Simple Linear Regression

১. মূল সমীকরণ (The Core Formula)

সিম্পল লিনিয়ার রিগ্রেশন মূলত একটি ইনপুট ফিচার (X) এবং একটি আউটপুট টার্গেটের (y) মধ্যে সরলরৈখিক সম্পর্ক তৈরি করে। এর গাণিতিক সূত্রটি হলো:

$$\hat{y} = wX + b$$

$\hat{y}$ (y-hat): মডেলের প্রেডিক্ট করা মান (Predicted Value)

X: ইনপুট ফিচার (Independent Variable)

w (Weight/Slope): রেখাটির ঢাল। X এক ইউনিট বাড়লে y কতটুকু বাড়বে বা কমবে, তা w নির্ধারণ করে

b (Bias/Intercept): ইন্টারসেপ্ট। X-এর মান 0 হলে y-এর মান কত হবে, তা এটি নির্দেশ করে

২. কস্ট ফাংশন (Cost Function)

মডেলের প্রেডিকশন আসল ডেটার তুলনায় কতটা ভুল করছে, তা পরিমাপ করার জন্য Mean Squared Error (MSE) কস্ট ফাংশন ব্যবহার করা হয়। এর গাণিতিক রূপ:

$$J(w, b) = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

যেহেতু

$$\hat{y}_i = wX_i + b$$

তাই সমীকরণটি দাঁড়ায়:

$$J(w, b) = \frac{1}{n} \sum_{i=1}^n (wX_i + b - y_i)^2$$

n: মোট ডেটা স্যাম্পলের সংখ্যা

y_i : আসল মান (Actual Value)

$\hat{y}_i$: মডেলের প্রেডিক্ট করা মান

৩. কস্ট ফাংশনের ডেরিভেটিভ (Partial Derivatives)

গ্রাডিয়েন্ট ডিসেন্ট পদ্ধতিতে কস্ট ফাংশন (J) এর মান সর্বনিম্ন করার জন্য আমাদের জানতে হবে w এবং b পরিবর্তনের সাথে সাথে কস্ট কতটা পরিবর্তিত হচ্ছে। এর জন্য ক্যালকুলাসের চেইন রুল (Chain Rule) ব্যবহার করে আংশিক অন্তরীকরণ (Partial Derivative) করা হয়:

w-এর সাপেক্ষে ডেরিভেটিভ (dJ/dw):

$$\frac{\partial J}{\partial w} = \frac{2}{n} \sum_{i=1}^n (w X_i + b - y_i) \cdot X_i$$

b-এর সাপেক্ষে ডেরিভেটিভ (dJ/db):

$$\frac{\partial J}{\partial b} = \frac{2}{n} \sum_{i=1}^n (w X_i + b - y_i)$$

(এখানে $wX_i + b - y_i$ অংশটি হলো মূলত প্রেডিকশন এবং আসল মানের মধ্যকার ভুল বা Error)

৪. প্যারামিটার আপডেট সূত্র (Weight & Bias Update)

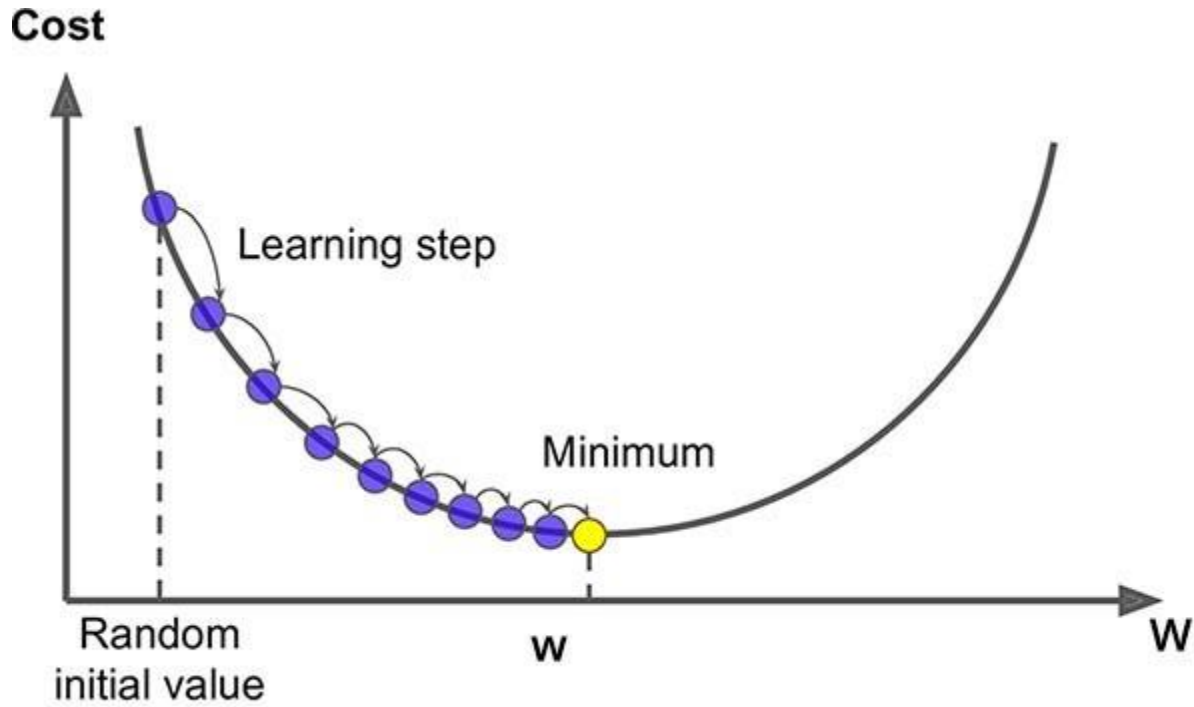
প্রতিটি ইটারেশনে বা লুপে প্রাপ্ত গ্রাডিয়েন্ট বা ডেরিভেটিভের ওপর ভিত্তি করে লার্নিং রেট (α) গুণ করে w এবং b-কে আপডেট করা হয়, যাতে তারা আস্তে আস্তে সঠিক মানের দিকে এগোতে পারে:

$$w = w - \alpha \cdot \frac{\partial J}{\partial w}$$
$$b = b - \alpha \cdot \frac{\partial J}{\partial b}$$

α (Alpha): লার্নিং রেট (Learning Rate)। এটি নির্ধারণ করে মডেল কতটা বড় বা ছোট স্টেপ ফেলে গ্লোবাল মিনিমামের দিকে নামবে

৫. কস্ট ফাংশন গ্রাফ (Cost Function Graph)

লিনিয়ার রিগ্রেশনের কস্ট ফাংশনের গ্রাফটি দেখতে একটি বাটি বা বোলের (Bowl) মতো হয়, যাকে গাণিতিক ভাষায় Convex Function বলা হয়। গ্রাডিয়েন্ট ডিসেন্ট পাহাড়ের চূড়া থেকে আস্তে আস্তে পা ফেলে বাটির একদম তলানিতে পৌঁছানোর চেষ্টা করে, যেখানে ভুলের পরিমাণ সবচেয়ে কম।



ব্যাকএন্ডে কীভাবে কাজ করে? (Full Backend Workflow)

ধরা যাক, আমাদের কাছে একটি ডেটাসেট আছে। ব্যাকএন্ডে পুরো প্রসেসটি নিচের ধাপে ধাপে সম্পন্ন হয়:

- Initialization: শুরুতেই কম্পিউটার আন্দাজে $w = 0$ এবং $b = 0$ ধরে নেয়
- Forward Pass (Prediction): এই প্রাথমিক w এবং b দিয়ে পুরো ডেটাসেটের জন্য $\hat{y} = wX + b$ সূত্র ব্যবহার করে প্রেডিকশন হিসাব করা হয়
- Compute Cost: প্রেডিকশন বের করার পর আসল মানের সাথে তুলনা করে MSE সূত্রের মাধ্যমে বর্তমান কস্ট বা ভুলের পরিমাণ (J) মাপা হয়
- Backward Pass (Gradient Computation): আংশিক অন্তরীকরণের সূত্রের মাধ্যমে বর্তমান ভুলের জন্য w এবং b কতটা দায়ী (অর্থাৎ গ্রাডিয়েন্ট $\partial J/\partial w$ এবং $\partial J/\partial b$) তা ক্যালকুলেট করা হয়
- Update Parameters: এই গ্রাডিয়েন্ট ব্যবহার করে w এবং b -এর পুরানো মান থেকে লার্নিং রেট গুণ করে নতুন মান আপডেট করা হয়
- Iteration: এই ২ থেকে ৫ নম্বর ধাপটি লুপের মাধ্যমে হাজার হাজার বার (যেমন 1,500 বার) চলতে থাকে। প্রতিবারে বাটির তলানির দিকে নামতে নামতে কস্ট কমতে থাকে এবং একপর্যায়ে স্থির হয়ে যায়

From Scratch (No Scikit-Learn)

```
1 import numpy as np
2
3 def make_prediction(X, w, b):
4     return w * X + b
5
6 def cost_function(X, y, w, b):
7     n = len(y)
8     y_pred = make_prediction(X, w, b)
9     return (1 / n) * np.sum((y_pred - y) ** 2)
10
11 def calculate_gradient(X, y, w, b):
12     n = len(y)
13     y_pred = make_prediction(X, w, b)
14     dw = (2 / n) * np.sum((y_pred - y) * X)
15     db = (2 / n) * np.sum((y_pred - y))
16     return dw, db
17
18 def gradient_descent(X, y, w, b, lr, epochs):
19     for i in range(epochs):
20         dw, db = calculate_gradient(X, y, w, b)
21         w = w - lr * dw
22         b = b - lr * db
23         if i % 100 == 0:
24             print("epoch:", i, "cost:", cost_function(X, y, w, b))
25     return w, b
26
27 X = np.array([1, 2, 3, 4, 5])
28 y = np.array([2, 4, 6, 8, 10])
29
30 w, b = gradient_descent(X, y, 0, 0, 0.01, 1000)
31
32 print("Final w:", w)
33 print("Final b:", b)
```

Linear Regression using Scikit Learn

```

1 import numpy as np
2 from sklearn.linear_model import LinearRegression, SGDRegressor
3 from sklearn.preprocessing import StandardScaler
4
5 X = np.array([1, 2, 3, 4, 5]).reshape(-1, 1)
6 y = np.array([2, 4, 6, 8, 10])
7
8 lr_model = LinearRegression()
9 lr_model.fit(X, y)
10
11 print("Linear Regression")
12 print("w:", lr_model.coef_[0])
13 print("b:", lr_model.intercept_)
14
15 print("Pred:", lr_model.predict(X))

```

SDG Regression using Scikit Learn

```

18 scaler = StandardScaler()
19 X_scaled = scaler.fit_transform(X)
20
21 sgd_model = SGDRegressor(
22     learning_rate="constant",
23     eta0=0.01,
24     max_iter=1000,
25     tol=1e-3
26 )
27
28 sgd_model.fit(X_scaled, y)
29
30 print("\nSGD Regression")
31 print("w:", sgd_model.coef_[0])
32 print("b:", sgd_model.intercept_[0])
33 print("Pred:", sgd_model.predict(X_scaled))

```

কেন এটি প্রয়োজন (Why We Need It)

বাস্তব জীবনে এমন অনেক সমস্যা আছে যেখানে দুটি ভেরিয়েবলের মধ্যে একটি Approximate Linear সম্পর্ক বিদ্যমান — যেমন পড়াশোনার সময় বনাম পরীক্ষার নম্বর, বাসার আয়তন বনাম

মূল্য, বিজ্ঞাপনে ব্যয় বনাম বিক্রয়। Simple Linear Regression এই সম্পর্কটি গাণিতিকভাবে মডেল করে ভবিষ্যদ্বাণী করতে সাহায্য করে।

Assumptions

- Linearity: X ও y-এর মধ্যে সম্পর্ক সরলরৈখিক হতে হবে
- Independence: Observation-গুলো একে অপরের থেকে স্বাধীন (independent) হতে হবে
- Homoscedasticity: Residual-এর Variance সব জায়গায় প্রায় সমান থাকতে হবে
- Normality of Residuals: Error term-গুলো Normal Distribution অনুসরণ করা উচিত

Advantages ও Disadvantages

Advantages	Disadvantages / Limitations
সহজ, দ্রুত এবং Interpretable	শুধু Linear সম্পর্কের জন্য কার্যকর
কম কম্পিউটেশনাল রিসোর্সে চলে	Outlier-এর প্রতি অত্যন্ত sensitive
Closed-form (OLS) সমাধান বিদ্যমান	Multicollinearity হলে সমস্যা হয় (Multiple Feature-এ প্রযোজ্য)
IMPORTANT NOTE MSE সবসময় Non-negative এবং একটি Convex Function — তাই Gradient Descent ব্যবহার করে এর একটি Unique Global Minimum খুঁজে পাওয়া সম্ভব।	
PRACTICAL INSIGHT Forward Pass → Compute Cost → Backward Pass → Update — এই চক্রটিই প্রায় সকল Gradient-Based Machine Learning ও Deep Learning Model-এর মূল ভিত্তি। এই ৪টি ধাপ ভালোভাবে আত্মস্থ করলে Neural Network-এর Backpropagation বুঝতেও সুবিধা হয়।	
REAL-WORLD EXAMPLE হাউজিং মার্কেটে শুধুমাত্র বাসার আয়তন (sq ft) দিয়ে আনুমানিক মূল্য নির্ধারণ, কোনো প্রতিষ্ঠানের বিজ্ঞাপন ব্যয় থেকে বিক্রয় পরিমাণ পূর্বাভাস, কিংবা পড়াশোনার ঘণ্টা থেকে পরীক্ষার নম্বর অনুমান করা — এসব ক্ষেত্রে Simple Linear Regression ব্যাপকভাবে ব্যবহৃত হয়।	
COMMON MISTAKE	

Outlier বাদ না দিয়ে সরাসরি Model Train করা — এতে Slope (w) মারাত্মকভাবে প্রভাবিত হতে পারে।

Feature ও Target-এর মধ্যে সম্পর্ক Non-linear হলেও জোর করে Linear Model ব্যবহার করা।

Learning Rate (α) অত্যধিক বড় নেওয়া, যার ফলে Gradient Descent Diverge করে।

INTERVIEW TIP

Q: কেন আমরা MSE-তে error-কে square করি, absolute value নিই না কেন?

A: Square করলে ফাংশনটি Differentiable এবং Convex হয়, ফলে Gradient Descent সহজে কাজ করতে পারে; এছাড়া বড় error-কে বেশি penalize করা যায়।

Q: OLS এবং Gradient Descent-এর মধ্যে পার্থক্য কী?

A: OLS একটি Closed-form, একবারেই সমাধানকারী পদ্ধতি; Gradient Descent ধাপে ধাপে Iterative ভাবে Optimal মান খুঁজে বের করে।

Quick Revision Notes — Simple Linear Regression

Category	Key Points
Key Formula	$\hat{y} = wX + b$; $J(w,b) = (1/n)\sum(\hat{y}_i - y_i)^2$
Gradient	$\partial J / \partial w = (2/n)\sum(\text{error} \cdot X)$; $\partial J / \partial b = (2/n)\sum(\text{error})$
Update Rule	$w := w - \alpha \cdot \partial J / \partial w$; $b := b - \alpha \cdot \partial J / \partial b$
Cost Surface	Convex (bowl-shaped) → guarantees a global minimum
Exam Focus	Derive gradients via chain rule; explain why MSE is convex
Interview Focus	Assumptions of linear regression; OLS vs Gradient Descent

Multiple Linear Regression

মাল্টিপল লিনিয়ার রিগ্রেশন (Multiple Linear Regression)

মাল্টিপল লিনিয়ার রিগ্রেশন মূলত সিম্পল লিনিয়ার রিগ্রেশনেরই সম্প্রসারিত রূপ, যেখানে একাধিক ইনপুট ফিচার $X_1, X_2, \dots, X_m$ ব্যবহার করা হয়।

১. মূল সমীকরণ (The Core Formula)

যখন একাধিক ফিচার থাকে, তখন প্রেডিকশন হয়:

$$\hat{y} = w_1X_1 + w_2X_2 + \dots + w_mX_m + b$$

এটিকে ভেক্টরাইজড ফর্মে লেখা হয়:

$$\hat{y} = XW + b$$

যেখানে:

- X : ইনপুট ফিচার ম্যাট্রিক্স (সাইজ $n \times m$)
- W : ওয়েট ভেক্টর $[w_1, w_2, \dots, w_m]^T$
- b : বায়াস (স্কেলার)
- $\hat{y}$: প্রেডিক্টেড আউটপুট

২. কস্ট ফাংশন (Cost Function)

Mean Squared Error (MSE):

$$J(W, b) = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2$$

ভেক্টর ফর্মে:

$$J(W, b) = \frac{1}{n} \| XW + b - y \|^2$$

৩. কস্ট ফাংশনের ডেরিভেটিভ (Partial Derivatives)

ওয়েটের জন্য গ্রাডিয়েন্ট:

$$\frac{\partial J}{\partial W} = \frac{2}{n} X^T (XW + b - y)$$

বায়াসের জন্য গ্রাডিয়েন্ট:

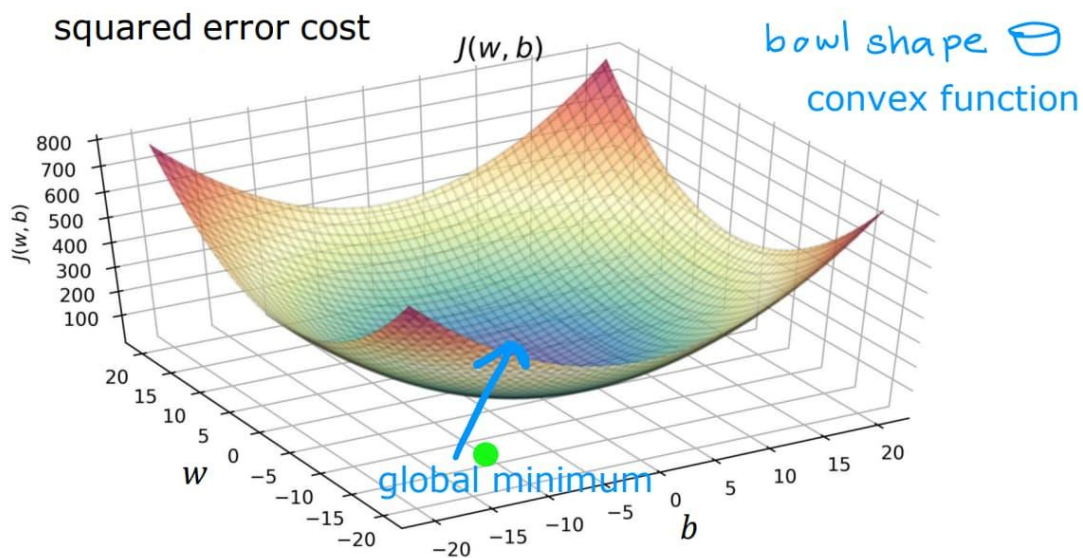
$$\frac{\partial J}{\partial b} = \frac{2}{n} \sum_{i=1}^n (XW + b - y)$$

৪. প্যারামিটার আপডেট (Weight & Bias Update)

$$W = W - \alpha \cdot \frac{\partial J}{\partial W}$$
$$b = b - \alpha \cdot \frac{\partial J}{\partial b}$$

৫. কস্ট ফাংশন গ্রাফ (Cost Function Graph)

মাল্টিপল ফিচারের ক্ষেত্রে কস্ট ফাংশন একটি **high-dimensional convex surface** তৈরি করে।
এটি দৃশ্যমানভাবে 2D বা 3D বাটি না হলেও গাণিতিকভাবে একটি মাত্র গ্লোবাল মিনিমাম থাকে।



ব্যাকএন্ড ওয়ার্কফ্লো (Full Backend Workflow)

- Initialization: $W = 0, b = 0$
- Forward Pass:

$$\hat{y} = XW + b$$

- Compute Cost: MSE দিয়ে error হিসাব
- Backward Pass:

$$X^T \cdot error$$

- Update Parameters: W এবং b আপডেট
- Convergence: লস মিনিমাম না হওয়া পর্যন্ত লুপ

Python Code

```
1 import numpy as np
2
3 X = np.array([
4     [1.0, 2100.0],
5     [2.0, 1500.0],
6     [3.0, 2800.0],
7     [4.0, 1200.0],
8     [5.0, 3200.0]
9 ])
10
11 y = np.array([45.0, 35.0, 60.0, 28.0, 70.0])
12
13 def predict(X, W, b):
14     return np.dot(X, W) + b
15
16 def compute_cost(X, y, W, b):
17     n = len(X)
18     return (1/n) * np.sum((predict(X, W, b) - y) ** 2)
19
20 def compute_gradient(X, y, W, b):
21     n = len(X)
22     error = predict(X, W, b) - y
23     dW = (2/n) * np.dot(X.T, error)
24     db = (2/n) * np.sum(error)
25     return dW, db
```



```

27 def gradient_descent(X, y, W, b, alpha, iters):
28     for i in range(iters):
29         dw, db = compute_gradient(X, y, W, b)
30
31         W = W - alpha * dw
32         b = b - alpha * db
33
34     return W, b
35
36 X_mean = np.mean(X, axis=0)
37 X_std = np.std(X, axis=0)
38 X_scaled = (X - X_mean) / X_std
39
40 W, b = gradient_descent(X_scaled, y, np.zeros(X.shape[1]), 0.0, 0.01, 50000)
41
42 print(W, b)

```

Using Scikit Learn

```

1 from sklearn.linear_model import LinearRegression, SGDRegressor
2 from sklearn.preprocessing import StandardScaler
3
4 lr = LinearRegression()
5 lr.fit(X, y)
6
7 print(lr.coef_, lr.intercept_)
8
9 scaler = StandardScaler()
10 X_scaled = scaler.fit_transform(X)
11
12 sgd = SGDRegressor(max_iter=10000, eta0=0.01, random_state=42)
13 sgd.fit(X_scaled, y)
14
15 print(sgd.coef_)
16 print(sgd.predict(X_scaled[:1]))

```

লিনিয়ার রিগ্রেশনের ক্ষেত্রে StandardScaler ব্যবহার না করলেও সাধারণত সমস্যা হয় না, কারণ LinearRegression মডেলটি একটি ডাইরেক্ট ম্যাথমেটিক্যাল সলিউশন (OLS) ব্যবহার করে একবারেই ওজন (w) এবং বায়াস (b) এর সঠিক মান বের করে ফেলে। এই পদ্ধতিতে কোনো ধাপে ধাপে ইটারেশন বা লার্নিং প্রসেস নেই, বরং সরাসরি একটি সূত্র ব্যবহার করা হয়: $(X^T X)^{-1} X^T y$ । ফলে ফিচারের মান বড় বা ছোট যাই হোক না কেন, এটি গ্লোবাল মিনিমাম সঠিকভাবে বের করতে পারে এবং স্কেলিং না করলেও এর আউটপুট সাধারণত পরিবর্তিত হয় না।

অন্যদিকে SGD (Stochastic Gradient Descent) সম্পূর্ণ ভিন্নভাবে কাজ করে। এটি ধাপে ধাপে ইটারেশনের মাধ্যমে ধীরে ধীরে সঠিক মানের দিকে এগিয়ে যায়। এখানে ফিচারের স্কেল বড় হলে গ্রাডিয়েন্ট হিসাবের সময় অত্যন্ত বড় মান তৈরি হতে পারে, যার ফলে ওয়েট হঠাৎ অনেক বেশি বেড়ে গিয়ে মডেল অস্থিতিশীল হয়ে যেতে পারে বা কস্ট ফাংশন ডাইভার্জ করে NaN হয়ে যেতে পারে। এছাড়া স্কেল না করা ডেটায় কস্ট ফাংশনের সারফেস অসম বা লম্বাটে হয়ে যায়, ফলে গ্রাডিয়েন্ট ডিসেন্ট সোজা পথে নামতে না পেরে জিগজ্যাগ পথে ধীরে ধীরে কনভার্জ করে। এই কারণে SGD ব্যবহার করার আগে StandardScaler দিয়ে ফিচারগুলোকে একই স্কেলে আনা একদম গুরুত্বপূর্ণ, যাতে ট্রেনিং স্থিতিশীল, দ্রুত এবং নির্ভুল হয়।

COMMON MISTAKE

SGDRegressor ব্যবহারের আগে StandardScaler প্রয়োগ না করা — এটি Training-কে অস্থিতিশীল করে দিতে পারে বা Cost Function NaN হয়ে যেতে পারে।

লিনিয়ার রিগ্রেশন (LinearRegression) এবং SGD কীভাবে কাজ করে

১. Linear Regression (OLS) কীভাবে কাজ করে

লিনিয়ার রিগ্রেশন একটি **direct mathematical solution** ব্যবহার করে। এখানে মডেল কোনো লুপ চালিয়ে শেখে না। বরং পুরো ডেটাসেট একবারে দেখে একটি নির্দিষ্ট গাণিতিক সূত্র ব্যবহার করে সরাসরি সেরা ওজন (w) এবং বায়াস (b) বের করে ফেলে।

এই সূত্রটি হলো:

$$\theta = (X^T X)^{-1} X^T y$$

এখানে মডেল আসলে এমনভাবে কাজ করে যেন সে বলছে:

“আমি পুরো ডেটা একসাথে দেখে সবচেয়ে সঠিক রেখাটা একবারেই হিসাব করে ফেলছি।”

- ◆ তাই এটি দ্রুত
- ◆ তাই এটি স্থিতিশীল
- ◆ এবং ছোট-মাঝারি ডেটাসেটের জন্য খুব ভালো

২. SGD (Stochastic Gradient Descent) কীভাবে কাজ করে

SGD সম্পূর্ণ ভিন্নভাবে কাজ করে। এটি কোনো সরাসরি সূত্র ব্যবহার করে না।

বরং এটি:

- প্রথমে w এবং b র্যান্ডমভাবে নেয়
- তারপর প্রতিটি ডেটা পয়েন্ট বা ছোট ব্যাচ দেখে

- ধাপে ধাপে ভুল কমানোর চেষ্টা করে
- প্রতিবার একটু একটু করে w এবং b আপডেট করে

এটা অনেকটা এমন:

“আমি অন্ধভাবে ধাপে ধাপে হাঁটছি, ভুল হলে দিক ঠিক করছি।”

এই আপডেট হয় এইভাবে:

$$w = w - \alpha \frac{\partial J}{\partial w}$$

৩. কখন কোনটা ব্যবহার করা উচিত

Linear Regression ব্যবহার করা ভালো যখন:

- ডেটাসেট ছোট বা মাঝারি (few thousand rows পর্যন্ত)
- তুমি দ্রুত এবং exact solution চাও
- কম্পিউটেশনাল রিসোর্স সমস্যা না
- ব্যাচ ট্রেনিং দরকার নেই

উদাহরণ:

- house price prediction
- simple business forecasting
- academic projects

SGD ব্যবহার করা ভালো যখন:

- ডেটাসেট অনেক বড় (millions of rows)
- online learning দরকার (data আসতে থাকে নতুন নতুন করে)
- memory সীমিত
- deep learning বা large-scale ML system

উদাহরণ:

- recommendation system
- real-time prediction system
- large-scale industrial ML

OLS (Ordinary Least Squares) কীভাবে কাজ করে

OLS হলো Linear Regression-এর একটি **closed-form solution** যেখানে মডেল কোনো iteration ছাড়াই সরাসরি best parameters (W, b) বের করে ফেলে।

১. Model Equation

Linear Regression-এর মূল সমীকরণ:

$$\hat{y} = XW + b$$

২. Cost Function (MSE)

OLS যেটা মিনিমাইজ করে:

$$J(W, b) = \frac{1}{n} \|XW + b - y\|^2$$

এটি মূলত prediction error-এর square mean।

৩. Goal of OLS

OLS-এর লক্ষ্য:

$$\min_{W, b} J(W, b)$$

অর্থাৎ এমন W এবং b খুঁজে বের করা যাতে error সবচেয়ে কম হয়।

৪. Bias Handling (Important Step)

গণনা সহজ করার জন্য b-কে X-এর সাথে যোগ করা হয়:

$$X' = [X \ 1]$$

এখন সমীকরণ হয়:

$$\hat{y} = X'\theta$$

যেখানে:

- θ = সব parameters (W এবং b একসাথে)

৫. Closed-form Solution (Main Formula)

OLS সরাসরি এই formula ব্যবহার করে solution বের করে:

$$\theta = (X^T X)^{-1} X^T y$$

৬. Step-by-step Mechanism

Step 1: Design Matrix তৈরি

$$X \rightarrow X'$$

Step 2: Matrix Multiplication

$$X^T X$$

Step 3: Inverse নেওয়া

$$(X^T X)^{-1}$$

Step 4: Final multiplication

$$\theta = (X^T X)^{-1} X^T y$$

IMPORTANT NOTE

OLS তখনই কাজ করে যখন $(X^T X)$ Invertible (Non-singular) হয়। যদি Feature-গুলোর মধ্যে Perfect Multicollinearity থাকে, তাহলে এই Matrix Singular হয়ে যায় এবং Inverse বের করা সম্ভব হয় না — এক্ষেত্রে Ridge Regression (L2 Regularization) ব্যবহার করে সমস্যাটি সমাধান করা হয়।

EXAM TIP

এই ছোট, হাতে-গণনাযোগ্য Example ($w=2, b=0$) পরীক্ষায় OLS Derivation বোঝাতে প্রায়ই আসে — Design Matrix তৈরি থেকে Final θ পর্যন্ত প্রতিটি ধাপ আলাদাভাবে দেখাতে পারলে পূর্ণ নম্বর পাওয়া যায়।

OLS (Ordinary Least Squares) — Example

ধরা যাক আমাদের কাছে একটি ছোট dataset আছে:

X (Study Hours)	y (Marks)
1	2
2	4
3	6

আমরা চাই relationship বের করতে:

$$\hat{y} = wX + b$$

১. Design Matrix তৈরি (Bias সহ)

OLS-এ bias যোগ করার জন্য X-কে expand করা হয়:

$$X = \begin{bmatrix} 1 & 1 \\ 2 & 1 \\ 3 & 1 \end{bmatrix}$$

এখানে:

- প্রথম column = feature (X)
- দ্বিতীয় column = bias (1)

২. Target Vector

$$y = \begin{bmatrix} 2 \\ 4 \\ 6 \end{bmatrix}$$

৩. OLS Formula

$$\theta = (X^T X)^{-1} X^T y$$

এখানে:

$$\theta = \begin{bmatrix} w \\ b \end{bmatrix}$$

8. Step-by-step Calculation

Step 1: Transpose

$$X^T = \begin{bmatrix} 1 & 2 & 3 \\ 1 & 1 & 1 \end{bmatrix}$$

Step 2: Multiply $X^T X$

$$X^T X = \begin{bmatrix} 14 & 6 \\ 6 & 3 \end{bmatrix}$$

Step 3: Inverse

$$\left((X^T X)^{-1} = \begin{bmatrix} 1.5 & -3 \\ -3 & 7 \end{bmatrix} \right)$$

Step 4: Multiply with $X^T y$

$$X^T y = \begin{bmatrix} 28 \\ 12 \end{bmatrix}$$

Final Result

$$\theta = \begin{bmatrix} w \\ b \end{bmatrix} = \begin{bmatrix} 2 \\ 0 \end{bmatrix}$$

৫. Final Model

$$\hat{y} = 2X + 0$$

LinearRegression (sklearn.linear_model.LinearRegression)

এটি OLS (Ordinary Least Squares) এর closed-form solution ব্যবহার করে, তাই এখানে regularization বা learning-related কোনো parameter নেই। টিউন করার মতো parameter খুবই কম।

fit_intercept

Default: True

fit_intercept নির্ধারণ করে Model Bias Term (b বা intercept) শিখবে কিনা। True রাখলে Model $\hat{y} = wX + b$ সূত্রে b-এর মান হিসাব করে। False করলে Model ধরে নেয় Data ইতোমধ্যে Centered, অর্থাৎ $b = 0$ হয়ে যায়। বাস্তবে প্রায় সব ক্ষেত্রেই True রাখা উচিত।

copy_X

Default: True

copy_X নির্ধারণ করে Training-এর সময় Input Data (X) এর একটি Copy তৈরি করে কাজ করবে কিনা। True রাখলে Original Data অপরিবর্তিত থাকে এবং Model Copy-র উপর কাজ করে। False করলে Memory সাশ্রয় হয় কিন্তু Original X Overwrite হয়ে যেতে পারে, তাই সাধারণত True রাখাই নিরাপদ।

n_jobs

Default: None

n_jobs নির্ধারণ করে Computation-এর সময় কতগুলো CPU Core ব্যবহার করা হবে। n_jobs=-1 দিলে System-এর সব Available Core ব্যবহার হয়। এটি Model-এর Accuracy পরিবর্তন করে না, শুধুমাত্র Speed বাড়ায়। বিশেষত Multi-output Regression-এ এটি কার্যকর।

positive

Default: False

positive=True করলে Model সব Coefficient (w)-কে Non-negative অর্থাৎ শূন্য বা ধনাত্মক মানে সীমাবদ্ধ রাখে। যখন জানা থাকে যে Feature এবং Target-এর মধ্যে সম্পর্ক সবসময় ধনাত্মক (যেমন বিজ্ঞাপন খরচ বাড়লে বিক্রয় কমতে পারে না), তখন এটি ব্যবহার করা হয়।

SGDRegressor (sklearn.linear_model.SGDRegressor)

এটি Gradient Descent ভিত্তিক মডেল। এখানেই আসল hyperparameter tuning হয় এবং model performance-এর উপর সবচেয়ে বেশি প্রভাব ফেলে।

SGDRegressor (Stochastic Gradient Descent Regressor) হলো Scikit-Learn-এর একটি Linear Regression Model, যা Normal Equation ব্যবহার না করে Gradient Descent-এর মাধ্যমে Model-এর Weight শিখে। এটি বিশেষভাবে Large Dataset, Online Learning এবং এমন পরিস্থিতির জন্য উপযোগী যেখানে পুরো Dataset একসাথে Memory-তে রাখা সম্ভব নয়। প্রতিটি Training Example অথবা ছোট Batch ব্যবহার করে Weight Update করা হয়, ফলে Training অনেক দ্রুত হয়।

loss

Default: 'squared_error'

loss Parameter নির্ধারণ করে Model কোন Cost Function Minimize করবে। এটি Training-এর সময় Prediction Error কীভাবে পরিমাপ করা হবে তা নিয়ন্ত্রণ করে।

সাধারণ Linear Regression-এর ক্ষেত্রে squared_error সবচেয়ে বেশি ব্যবহৃত হয়। এখানে প্রতিটি Error Square করা হয়:

$$Loss = (y - \hat{y})^2$$

এর ফলে বড় Error-এর জন্য অনেক বেশি Penalty দেওয়া হয়। তাই Dataset-এ যদি Outlier না থাকে, তাহলে squared_error সাধারণত সর্বোত্তম ফলাফল দেয়।

কিন্তু Dataset-এ যদি অনেক Outlier থাকে, তাহলে huber Loss ব্যবহার করা ভালো। Huber Loss ছোট Error-এর জন্য Squared Loss এবং বড় Error-এর জন্য Linear Loss ব্যবহার করে। ফলে কয়েকটি অস্বাভাবিক Data Point Model-কে অতিরিক্ত প্রভাবিত করতে পারে না এবং Model আরও Robust হয়।

penalty

Default: 'l2'

penalty Parameter নির্ধারণ করে Training-এর সময় Regularization ব্যবহার করা হবে কি না এবং হলে কোন ধরনের Regularization ব্যবহার হবে।

Machine Learning-এ একটি সাধারণ সমস্যা হলো Overfitting, যেখানে Model Training Data খুব ভালোভাবে শিখে ফেলে কিন্তু নতুন Data-তে খারাপ Performance দেয়। Regularization এই সমস্যা কমাতে সাহায্য করে।

l2 Regularization Weight-এর বড় মানকে Penalize করে এবং Model-কে Smooth রাখে। এটি সবচেয়ে বেশি ব্যবহৃত Regularization।

$$Loss + \alpha \sum w^2$$

l1 Regularization অনেক Weight-কে শূন্য করে দিতে পারে, ফলে Feature Selection স্বয়ংক্রিয়ভাবে হয়ে যায়।

$$Loss + \alpha \sum |w|$$

আর elasticnet L1 এবং L2 দুটোর সুবিধা একসাথে দেয়। যখন Feature Selection এবং Stable Weight দুটোই দরকার হয়, তখন ElasticNet ব্যবহার করা হয়।

alpha

Default: 0.0001

alpha হলো Regularization-এর Strength। এটি নির্ধারণ করে Regularization Term কতটা শক্তিশালী হবে।

যদি Model Overfit করে, তাহলে alpha বাড়ালে Weight-এর মান ছোট হবে এবং Model সহজ হবে। অন্যদিকে alpha খুব বেশি বড় হলে Model গুরুত্বপূর্ণ Pattern-ও শিখতে পারবে না, ফলে Underfitting দেখা দিতে পারে।

সহজভাবে বলতে গেলে, alpha Model Complexity-এর উপর ব্রেকের মতো কাজ করে। বেশি Alpha মানে বেশি নিয়ন্ত্রণ, কম Alpha মানে Model-কে বেশি স্বাধীনতা দেওয়া।

l1_ratio

Default: 0.15

l1_ratio শুধুমাত্র penalty='elasticnet' হলে কার্যকর হয়। এটি নির্ধারণ করে ElasticNet-এর মধ্যে L1 এবং L2 কত শতাংশ করে থাকবে।

যদি l1_ratio=0 হয়, তাহলে পুরোপুরি L2 Regularization ব্যবহৃত হবে। যদি l1_ratio=1 হয়, তাহলে পুরোপুরি L1 Regularization ব্যবহৃত হবে।

মারামাতি মান ব্যবহার করলে Feature Selection এবং Weight Stabilization দুটোর সুবিধাই পাওয়া যায়।

learning_rate

Default: 'invscaling'

Gradient Descent-এর সবচেয়ে গুরুত্বপূর্ণ বিষয়গুলোর একটি হলো Learning Rate। Learning Rate নির্ধারণ করে প্রতিটি Step-এ Weight কতটুকু পরিবর্তিত হবে।

learning_rate Parameter বলে দেয় Training চলাকালে Learning Rate কীভাবে পরিবর্তিত হবে।

constant ব্যবহার করলে Learning Rate পুরো Training জুড়ে একই থাকে। invscaling ব্যবহার করলে Epoch বাড়ার সাথে সাথে Learning Rate ধীরে ধীরে কমে যায়।

$$\eta_t = \frac{\eta_0}{t^{power_t}}$$

adaptive Strategy ব্যবহার করলে Learning Rate শুরুতে বড় থাকে, কিন্তু Validation Performance উন্নত না হলে ধীরে ধীরে কমে যায়। বাস্তবে অনেক ক্ষেত্রে adaptive ভালো ফলাফল দেয় কারণ এটি দ্রুত Converge করতে সাহায্য করে।

eta0

Default: 0.01

eta0 হলো Initial Learning Rate। Training-এর শুরুতে Weight Update করার জন্য এই মান ব্যবহার করা হয়।

Gradient Descent-এর Update Equation হলো:

$$w_{new} = w_{old} - \eta \frac{\partial J}{\partial w}$$

যদি eta0 খুব বড় হয়, তাহলে Model Optimal Point-এর চারপাশে লাফাতে থাকবে এবং Diverge করতে পারে। আবার যদি খুব ছোট হয়, তাহলে Training অত্যন্ত ধীর হয়ে যাবে।

সাধারণত Convergence Problem দেখা দিলে প্রথমে eta0 পরিবর্তন করা হয়।

max_iter

Default: 1000

max_iter নির্ধারণ করে সর্বোচ্চ কত Epoch Training চলবে।

একটি Epoch বলতে পুরো Dataset-এর উপর একবার সম্পূর্ণ Training বোঝায়। যদি Model Converge করার আগেই Training বন্ধ হয়ে যায়, তাহলে max_iter বাড়ানো প্রয়োজন।

Convergence Warning দেখলে সাধারণত 1000 থেকে 3000 বা 5000 করা হয়।

tol

Default: 1e-3

tol হলো Convergence Threshold। Training চলাকালে যদি Loss-এর উন্নতি এই মানের চেয়ে কম হয়ে যায়, তাহলে Algorithm ধরে নেয় যে Model প্রায় Converge করে ফেলেছে এবং Training বন্ধ করে দেয়।

বড় tol দিলে Training দ্রুত শেষ হয়, কিন্তু Precision কিছুটা কমতে পারে। ছোট tol দিলে Training বেশি সময় চলে এবং আরও ভালো Solution খুঁজে বের করার চেষ্টা করে।

early_stopping

Default: False

early_stopping=True করলে Model Training-এর সময় একটি Validation Set ব্যবহার করে।

যদি Validation Score ধারাবাহিকভাবে উন্নতি না করে, তাহলে Training আগেই বন্ধ হয়ে যায়।

এটি Overfitting প্রতিরোধের একটি অত্যন্ত কার্যকর Technique। কারণ Training Data-তে Performance বাড়লেও Validation Data-তে Performance কমতে শুরু করলে বোঝা যায় Model মুখস্থ করা শুরু করেছে।

validation_fraction

Default: 0.1

যখন early_stopping=True থাকে, তখন Dataset-এর একটি অংশ Validation Set হিসেবে আলাদা করা হয়।

validation_fraction=0.1 মানে Dataset-এর 10% Validation-এর জন্য এবং 90% Training-এর জন্য ব্যবহৃত হবে।

Validation Set ব্যবহার করে Early Stopping সিদ্ধান্ত নেওয়া হয়।

shuffle

Default: True

প্রতিটি Epoch শুরু হওয়ার আগে Data Shuffle করা হবে কি না তা shuffle Parameter নিয়ন্ত্রণ করে।

SGD-তে Data-এর Order খুব গুরুত্বপূর্ণ। যদি Data সবসময় একই ক্রমে Model-এর কাছে যায়, তাহলে Model ভুল Pattern শিখে ফেলতে পারে।

তাই বাস্তবে প্রায় সব ক্ষেত্রেই `shuffle=True` রাখা হয়।

random_state

`random_state` Training-এর Random Operations-কে Reproducible করে।

উদাহরণস্বরূপ, Data Shuffle, Validation Split ইত্যাদি Random Process-এর জন্য এটি ব্যবহার করা হয়।

`random_state=42`

একই Seed ব্যবহার করলে প্রতিবার একই Result পাওয়া যায়, যা Research এবং Experiment Tracking-এর জন্য অত্যন্ত গুরুত্বপূর্ণ।

average

Default: False

SGD-এর Training Process অনেক সময় Noise-এর কারণে Weight-কে বিভিন্ন দিকে নাড়াচাড়া করতে পারে।

`average=True` করলে Training-এর শেষ দিকে পাওয়া বিভিন্ন Weight-এর Average নেওয়া হয়।

ফলে Final Model আরও Stable হয় এবং অনেক ক্ষেত্রে Generalization Performance উন্নত হয়।

বিশেষ করে Large Dataset এবং Noisy Dataset-এ এটি উপকারী হতে পারে।

warm_start

Default: False

সাধারণত `.fit()` কল করলে Model নতুন করে Training শুরু করে।

কিন্তু `warm_start=True` করলে আগের Training-এর Weight সংরক্ষণ করা হয় এবং পরবর্তী `.fit()` সেখান থেকেই শুরু হয়।

এটি Incremental Learning, Online Learning এবং বড় Dataset Chunk-by-Chunk Training-এর জন্য উপকারী।

epsilon

Default: 0.1

`epsilon` শুধুমাত্র `loss='epsilon_insensitive'` বা `loss='huber'` ব্যবহার করার সময় কার্যকর হয়। এটি নির্ধারণ করে কতটুকু Error পর্যন্ত Penalty দেওয়া হবে না। অর্থাৎ যদি Prediction এবং Actual Value-এর পার্থক্য `epsilon`-এর মধ্যে থাকে, তাহলে সেটাকে Error হিসেবে গণ্য করা হয় না। `squared_error` loss ব্যবহার করলে এই parameter কোনো প্রভাব ফেলে না।

n_iter_no_change

Default: 5

n_iter_no_change নির্ধারণ করে কত Epoch ধরে Loss উল্লেখযোগ্যভাবে না কমলে Early Stopping বা Convergence ধরা হবে। early_stopping=True থাকলে এই সংখ্যক Epoch-এ Validation Score না বাড়লে Training বন্ধ হয়ে যায়। এটি বাড়ালে Model বেশি সময় ধৈর্য ধরে Train হয়।

power_t

Default: 0.5

power_t শুধুমাত্র learning_rate='invscaling' হলে ব্যবহৃত হয়। এটি নির্ধারণ করে Learning Rate কত দ্রুত কমবে। এর সূত্র হলো: $\eta = \eta_0 / t^{\text{power_t}}$ । power_t বড় হলে Learning Rate দ্রুত কমে, ছোট হলে ধীরে কমে। সাধারণত Default 0.5 বেশিরভাগ ক্ষেত্রে যথেষ্ট।

LinearRegression() Attributes

coef_

Default: Created after fit()

প্রতিটি Feature-এর Weight (Coefficient) সংরক্ষণ করে। এটি দেখায় একটি Feature ১ ইউনিট পরিবর্তন হলে Prediction কত পরিবর্তন হবে (অন্য সব Feature অপরিবর্তিত থাকলে)। এটি Linear Regression-এর সবচেয়ে গুরুত্বপূর্ণ Output।

intercept_

Default: Created after fit()

মডেলের Bias (Intercept) সংরক্ষণ করে। সব Feature-এর মান 0 হলে Model যে Prediction দেয় সেটিই Intercept।

rank_

Default: Created after fit() (*Dense input only*)

Training Data-এর Design Matrix-এর Rank সংরক্ষণ করে। এটি বোঝায় কতগুলো Feature একে অপরের থেকে স্বাধীন। যদি Rank Feature সংখ্যার চেয়ে কম হয়, তাহলে Multicollinearity বা Linear Dependency থাকতে পারে।

singular_

Default: Created after fit() (*Dense input only*)

Design Matrix-এর Singular Values সংরক্ষণ করে। এগুলো Matrix-এর Numerical Stability বোঝাতে সাহায্য করে। খুব ছোট Singular Value থাকলে Feature-গুলোর মধ্যে শক্তিশালী Correlation থাকতে পারে।

n_features_in_

Default: Created after fit()

Model Training-এর সময় মোট কতটি Input Feature ব্যবহার করা হয়েছে তা সংরক্ষণ করে।

feature_names_in_

Default: Created after fit() (*DataFrame input only*)

Training-এর সময় যদি Pandas DataFrame ব্যবহার করা হয়, তাহলে Feature-গুলোর নাম সংরক্ষণ করে।

SGDRegressor() Attributes

coef_

Default: Created after fit()

প্রতিটি Feature-এর Learned Weight সংরক্ষণ করে। SGD Training শেষে এই Coefficient-গুলোর সাহায্যে Prediction করা হয়।

intercept_

Default: Created after fit()

Model-এর Learned Bias (Intercept) সংরক্ষণ করে।

n_iter_

Default: Created after fit()

Training শেষ হতে Model কতটি Epoch (Iteration) সম্পন্ন করেছে তা সংরক্ষণ করে। এটি max_iter থেকে কমও হতে পারে যদি Early Stopping বা Convergence আগে হয়ে যায়।

t_

Default: Created after fit()

Training-এর সময় মোট কতবার Weight Update (Parameter Update) হয়েছে তা সংরক্ষণ করে। এটি Learning Rate Schedule-এর হিসাবেও ব্যবহৃত হয়।

loss_function_

Default: Created after fit()

Model যে Internal Loss Function ব্যবহার করেছে তার Object সংরক্ষণ করে। এটি scikit-learn-এর Internal Attribute এবং সাধারণত Debugging বা Advanced Analysis-এর জন্য ব্যবহৃত হয়।

n_features_in_

Default: Created after fit()

Training-এর সময় ব্যবহৃত মোট Feature-এর সংখ্যা সংরক্ষণ করে।

feature_names_in_

Default: Created after fit() (*DataFrame input only*)

Training Data যদি Pandas DataFrame হয়, তাহলে Feature-গুলোর নাম সংরক্ষণ করে।

average_coef_

Default: Created only when average=True

যদি average=True ব্যবহার করা হয়, তাহলে Training চলাকালীন সব Weight-এর Average মান সংরক্ষণ করে। এটি অনেক সময় আরও Stable এবং Generalized Model তৈরি করতে সাহায্য করে।

average_intercept_

Default: Created only when average=True

average=True হলে Training চলাকালীন সব Intercept-এর Average মান সংরক্ষণ করে।

classes_

Default: Not available

SGDRegressor Regression Model হওয়ায় এর কোনো Target Class থাকে না। তাই এই Attribute থাকে না।

Logistic Regression

লজিস্টিক রিগ্রেশন মূলত একটি **ক্লাসিফিকেশন অ্যালগরিদম** (Classification Algorithm), যা বাইনারি ক্লাস (যেমন: 0 অথবা 1, হ্যাঁ অথবা না) প্রেডিক্ট করতে ব্যবহৃত হয়। লিনিয়ার রিগ্রেশন যেখানে $(-\infty$ থেকে $\infty)$ পর্যন্ত যেকোনো মান দিতে পারে, লজিস্টিক রিগ্রেশন সেখানে আউটপুটকে 0 থেকে 1 এর মধ্যকার একটি প্রোবাবিলিটি (Probability) বা সম্ভাবনায় রূপান্তর করে।

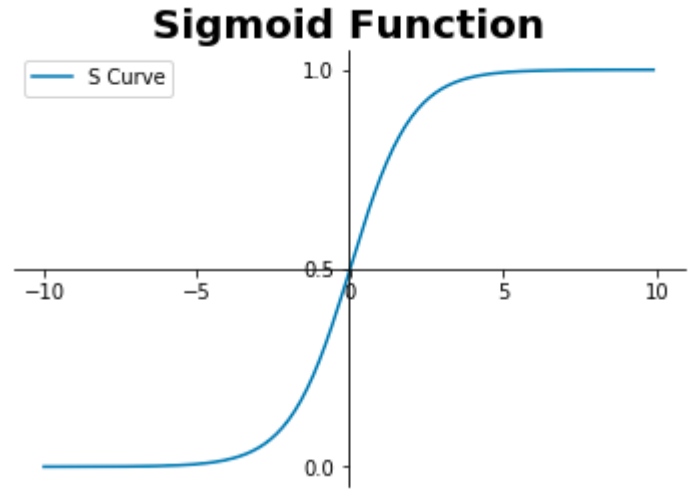
১. মূল সমীকরণ ও সিগময়েড ফাংশন (Core Equation & Sigmoid)

প্রথমে linear combination হিসাব করা হয়:

$$z = XW + b$$

এই z -কে sigmoid function-এর মাধ্যমে probability-তে রূপান্তর করা হয়:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$



Final prediction:

$$\hat{y} = \sigma(XW + b) = \frac{1}{1 + e^{-(XW+b)}}$$

Sigmoid-এর গুরুত্বপূর্ণ বৈশিষ্ট্য:

$$\begin{aligned} z = 0 & \quad \text{তখন } \sigma(z) = 0.5 \\ z \rightarrow +\infty & \quad \text{তখন } \sigma(z) \rightarrow 1 \\ z \rightarrow -\infty & \quad \text{তখন } \sigma(z) \rightarrow 0 \end{aligned}$$

Odds $\rightarrow$ Sigmoid Function (Full Derivation)

১. Odds কী?

ধরি কোনো event-এর probability হলো p

তাহলে:

$$\text{odds} = \frac{p}{1 - p}$$

মানে: success vs failure ratio

২. Log-Odds (Logit)

Odds-এর log নিলে:

$$\log(\text{odds}) = \log\left(\frac{p}{1-p}\right)$$

৩. Logistic Regression-এর assumption

$$z = \log\left(\frac{p}{1-p}\right)$$

অর্থাৎ:

- $z = \log\text{-odds}$

৪. Log undo করা (Exponential step)

আমরা জানি:

$$z = \log(\text{odds})$$

দুই পাশে exponential নিলে:

$$e^z = \text{odds}$$

৫. Odds বসানো

$$\frac{p}{1-p} = e^z$$

৬. এখন solve করি probability

$$\begin{aligned} p &= e^z(1-p) \\ p &= e^z - pe^z \\ p + pe^z &= e^z \\ p(1 + e^z) &= e^z \end{aligned}$$

৭. Final probability form

$$p = \frac{e^z}{1 + e^z}$$

৮. Sigmoid form এ রূপান্তর

উপরের expression কে rearrange করলে:

$$p = \frac{1}{1 + e^{-z}}$$

৯. Final Result (Sigmoid Function)

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

২. কস্ট ফাংশন (Binary Cross-Entropy / Log Loss)

Logistic Regression-এ Mean Squared Error ব্যবহার করা হয় না, কারণ এটি non-convex সমস্যা তৈরি করে।

তাই ব্যবহার করা হয়:

$$J(W, b) = -\frac{1}{n} \sum_{i=1}^n [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

Loss Intuition

যখন $y = 1$:

$$\text{Loss} = -\log(\hat{y})$$

- $\hat{y} \approx 1 \rightarrow \text{loss} \approx 0$
- $\hat{y} \approx 0 \rightarrow \text{loss} \rightarrow \infty$

যখন $y = 0$:

$$\text{Loss} = -\log(1 - \hat{y})$$

- $\hat{y} \approx 0 \rightarrow \text{loss} \approx 0$
- $\hat{y} \approx 1 \rightarrow \text{loss} \rightarrow \infty$

৩. গ্রাডিয়েন্ট (Partial Derivatives)

Log loss + sigmoid combine করলে খুব সুন্দরভাবে gradient simplify হয়।

Linear part:

$$z = XW + b$$

$$\hat{y} = \sigma(z)$$

Weight gradient:

$$\frac{\partial J}{\partial w_j} = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i) x_{ij}$$

Bias gradient:

$$\frac{\partial J}{\partial b} = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)$$

Important Note:

এখানে:

- $\hat{y}$ = sigmoid output
- error = $\hat{y} - y$

৪. প্যারামিটার আপডেট (Gradient Descent)

$$W = W - \alpha \cdot \frac{\partial J}{\partial W}$$

$$b = b - \alpha \cdot \frac{\partial J}{\partial b}$$

ব্যাকএন্ড ওয়ার্কফ্লো (Full Process)

Step 1: Initialization

- $W = 0$ বা ছোট random value
- $b = 0$

Step 2: Linear computation

$$z = XW + b$$

Step 3: Activation (Sigmoid)

$$\hat{y} = \frac{1}{1 + e^{-z}}$$

Step 4: Loss computation

$$J(W, b)$$

Step 5: Gradient computation

$$\frac{\partial J}{\partial w_j} = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i) x_{ij}$$

Step 6: Update parameters

$$W, b \text{ update}$$

Step 7: Prediction rule

$$\begin{aligned} \hat{y} &\geq 0.5 \Rightarrow 1 \\ \hat{y} &< 0.5 \Rightarrow 0 \end{aligned}$$

Python Code

```
1 import numpy as np
2
3 X = np.array([
4     [0.5, 1.5],
5     [1.0, 1.0],
6     [1.5, 0.5],
7     [3.0, 3.5],
8     [2.5, 4.5],
9     [4.0, 3.0]
10 ])
11
12 y = np.array([0, 0, 0, 1, 1, 1])
13
14 def sigmoid(z):
15     return 1 / (1 + np.exp(-z))
16
17 def predict_proba(X, W, b):
18     return sigmoid(np.dot(X, W) + b)
19
20 def predict(X, W, b, threshold=0.5):
21     return (predict_proba(X, W, b) >= threshold).astype(int)
22
23 def compute_cost(X, y, W, b):
24     y_pred = predict_proba(X, W, b)
25     return -np.mean(y*np.log(y_pred) + (1-y)*np.log(1-y_pred))
```

```
27 def compute_gradient(X, y, W, b):
28     error = predict_proba(X, W, b) - y
29     dW = (1/len(X)) * np.dot(X.T, error)
30     db = (1/len(X)) * np.sum(error)
31     return dW, db
32
33 def fit(X, y, lr, iters):
34     W = np.zeros(X.shape[1])
35     b = 0.0
36
37     for i in range(iters):
38         dW, db = compute_gradient(X, y, W, b)
39         W -= lr * dW
40         b -= lr * db
41
42     return W, b
43
44 W, b = fit(X, y, 0.1, 5000)
45
46 print(W, b)
47 print(predict(X, W, b))
```

Using Scikit Learn

```
1 from sklearn.linear_model import LogisticRegression, SGDClassifier
2 from sklearn.preprocessing import StandardScaler
3
4 lr = LogisticRegression(penalty=None)
5 lr.fit(X, y)
6
7 print(lr.coef_, lr.intercept_)
8 print(lr.predict(X))
9
10 scaler = StandardScaler()
11 X_scaled = scaler.fit_transform(X)
12
13 sgd = SGDClassifier(loss="log_loss", max_iter=5000, eta0=0.1, learning_rate
    ="constant")
14 sgd.fit(X_scaled, y)
15
16 print(sgd.coef_, sgd.intercept_)
17 print(sgd.predict(X_scaled))
```

LogisticRegression (sklearn.linear_model.LogisticRegression)

Logistic Regression classification সমস্যার জন্য ব্যবহৃত হয় এবং regularization-এর উপর অনেক বেশি নির্ভরশীল।

Logistic Regression হলো একটি Supervised Machine Learning Algorithm যা মূলত **Classification Problem** সমাধানের জন্য ব্যবহৃত হয়। নামের মধ্যে "Regression" থাকলেও এটি সাধারণত Class Predict করে। Binary Classification (Yes/No, Pass/Fail, Spam/Not Spam) থেকে শুরু করে Multiclass Classification পর্যন্ত এটি ব্যবহার করা যায়।

Linear Regression যেখানে সরাসরি একটি Continuous Value Predict করে, Logistic Regression সেখানে একটি Probability Predict করে এবং সেই Probability-এর ভিত্তিতে Final Class নির্ধারণ করে।

$$P(y = 1) = \frac{1}{1 + e^{-z}}$$

যেখানে,

$$z = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$

এই Sigmoid Function-এর কারণে Output সবসময় 0 এবং 1-এর মধ্যে থাকে।

C

Default: 1.0

C হলো Regularization Strength-এর Inverse।

অনেক শিক্ষার্থী প্রথমে বিভ্রান্ত হয় কারণ SGDRegressor বা Ridge Regression-এ Regularization বাড়তে Parameter-এর মান বাড়তে হয়, কিন্তু Logistic Regression-এ ঠিক উল্টো।

$$C = \frac{1}{\lambda}$$

এখানে λ হলো Regularization Strength।

অর্থাৎ,

- ছোট C $\rightarrow$ বেশি Regularization
- বড় C $\rightarrow$ কম Regularization

যদি Model Training Data খুব ভালো শিখে ফেলে কিন্তু Test Data-তে খারাপ করে, তাহলে Overfitting হচ্ছে। এমন পরিস্থিতিতে C কমানো উচিত।

উল্টোভাবে, যদি Model খুবই Simple হয়ে যায় এবং Training Data-ও ঠিকমতো শিখতে না পারে, তাহলে C বাড়ানো যেতে পারে।

বাস্তবে Logistic Regression-এর সবচেয়ে গুরুত্বপূর্ণ Hyperparameter সাধারণত C।

penalty

Default: 'l2'

penalty নির্ধারণ করে কোন ধরনের Regularization ব্যবহার করা হবে।

Machine Learning Model-এর একটি বড় সমস্যা হলো Overfitting। Regularization এই সমস্যা কমাতে সাহায্য করে।

l2 Regularization Weight-এর বড় মানকে Penalize করে এবং Coefficient-গুলোকে ছোট রাখে।

$$Loss + \lambda \sum w^2$$

এটি সবচেয়ে বেশি ব্যবহৃত Regularization।

l1 Regularization Weight-এর Absolute Value Penalize করে।

$$Loss + \lambda \sum |w|$$

এর ফলে অনেক Coefficient সরাসরি শূন্য হয়ে যায়। তাই Feature Selection-এর জন্য এটি খুব উপকারী।

elasticnet ব্যবহার করলে L1 এবং L2 দুটোর সুবিধা একসাথে পাওয়া যায়।

যদি Feature অনেক বেশি থাকে এবং কোনগুলো গুরুত্বপূর্ণ তা খুঁজতে চান, তাহলে 11 একটি ভালো Choice হতে পারে।

solver

Default: 'lbfgs'

Logistic Regression Training-এর সময় Optimal Weight বের করতে Optimization Algorithm ব্যবহার করে। solver Parameter সেই Algorithm নির্বাচন করে।

সব Solver একই কাজ করলেও তাদের Speed, Memory Usage এবং Supported Penalty আলাদা।

lbfgs

সবচেয়ে জনপ্রিয় এবং Default Solver।

- L2 Support করে
- Small ও Medium Dataset-এর জন্য ভালো
- Multiclass Support করে

সাধারণ ব্যবহারের জন্য এটি যথেষ্ট।

liblinear

- L1 এবং L2 দুটোই Support করে
- Small Dataset-এর জন্য উপযুক্ত
- Binary Classification-এ বেশি ব্যবহৃত

যদি penalty='l1' ব্যবহার করতে চান, তাহলে প্রায়ই liblinear অথবা saga ব্যবহার করতে হয়।

newton-cg

দ্বিতীয়-ধাপ (Second Order) Optimization ব্যবহার করে।

- Smooth Convergence
- Medium Dataset-এর জন্য ভালো

sag

- Large Dataset-এর জন্য দ্রুত
- শুধুমাত্র L2 Support করে

Dataset বড় হলে এটি Training অনেক দ্রুত করতে পারে।

saga

সবচেয়ে Flexible Solver।

- L1 Support করে

- L2 Support করে
- ElasticNet Support করে
- Large Dataset-এ কার্যকর

ElasticNet ব্যবহার করতে চাইলে সাধারণত saga Solver ব্যবহার করা হয়।

class_weight

Default: None

বাস্তব Dataset-এ অনেক সময় Class Distribution সমান থাকে না।

ধরুন,

Class Count

No 950

Yes 50

এখানে Model যদি সবকিছুকে "No" Predict করে, তবুও Accuracy হবে 95%।

কিন্তু বাস্তবে Model কোনো কাজে আসবে না।

এই সমস্যা সমাধানের জন্য class_weight='balanced' ব্যবহার করা হয়।

class_weight='balanced'

এটি Minority Class-কে বেশি Weight দেয়।

ফলে Model শুধু Majority Class শেখার বদলে Minority Class-কেও গুরুত্ব দেয়।

Imbalanced Dataset-এ এটি অত্যন্ত গুরুত্বপূর্ণ Parameter।

max_iter

Default: 100

Training চলাকালে Solver কত Iteration পর্যন্ত চেষ্টা করবে, সেটি max_iter নিয়ন্ত্রণ করে।

যদি Training-এর সময় নিচের Warning আসে:

ConvergenceWarning

তাহলে বুঝতে হবে Model এখনো Converge করতে পারেনি।

এক্ষেত্রে সাধারণত:

max_iter=1000

অথবা

max_iter=5000

ব্যবহার করা হয়।

fit_intercept

Default: True

Logistic Regression-এর Equation হলো:

$$z = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$

এখানে,

b

হলো Bias বা Intercept।

fit_intercept=True হলে Model Bias Term শিখবে।

বাস্তবে অধিকাংশ ক্ষেত্রেই এটি True রাখা উচিত।

শুধুমাত্র Data যদি ইতোমধ্যে Perfectly Centered হয়, তখন False বিবেচনা করা যেতে পারে।

l1_ratio

l1_ratio শুধুমাত্র নিচের ক্ষেত্রে কাজ করে:

penalty='elasticnet'

এবং

solver='saga'

এটি নির্ধারণ করে L1 এবং L2 কত শতাংশ ব্যবহার হবে।

$$0 \leq l1_ratio \leq 1$$

যদি:

l1_ratio=0

হয়, তাহলে Pure L2।

আর যদি:

l1_ratio=1

হয়, তাহলে Pure L1।

মাঝামাঝি মান দিলে L1 এবং L2-এর মিশ্রণ পাওয়া যায়।

multi_class

Default: 'auto'

যখন Target Variable-এ দুইটির বেশি Class থাকে, তখন Logistic Regression Multiclass Problem কীভাবে Handle করবে তা multi_class নির্ধারণ করে।

One-vs-Rest (OVR)

multi_class='ovr'

এখানে প্রতিটি Class-এর জন্য আলাদা Binary Classifier তৈরি হয়।

ধরুন Class:

- Cat
- Dog
- Bird

তাহলে Model তিনটি আলাদা Binary Problem তৈরি করবে।

- Cat vs Others
- Dog vs Others
- Bird vs Others

Multinomial

multi_class='multinomial'

এখানে সরাসরি Softmax ব্যবহার করা হয়।

$$P(y = i) = \frac{e^{z_i}}{\sum e^{z_j}}$$

সাধারণভাবে Multiclass Problem-এ এটি বেশি Accurate হয়।

বর্তমান Scikit-Learn Version-এ auto সাধারণত Multinomial Approach বেছে নেয় যখন Solver সেটি Support করে।

tol

Default: 1e-4

tol হলো Convergence Threshold।

Training চলাকালে যদি Loss Improvement এই মানের নিচে নেমে যায়, তাহলে Solver ধরে নেয় যে আর উল্লেখযোগ্য উন্নতি সম্ভব নয়।

ফলে Training বন্ধ হয়ে যায়।

বড় tol:

- দ্রুত Training
- কম Precision

ছোট tol:

- ধীর Training
- বেশি Precise Solution

random_state

Training-এর সময় Random Operation-এর জন্য Seed নির্ধারণ করে।

random_state=42

একই Code বারবার চালালে একই Result পাওয়ার জন্য এটি ব্যবহার করা হয়।

Research, Assignment এবং Experiment Tracking-এর ক্ষেত্রে এটি অত্যন্ত গুরুত্বপূর্ণ।

warm_start

Default: False

সাধারণত .fit() কল করলে Logistic Regression শুরু থেকে Training শুরু করে।

কিন্তু:

warm_start=True

দিলে আগের Training-এর Weight ব্যবহার করে নতুন Training শুরু হয়।

এটি Incremental Training অথবা বারবার Model Re-training-এর ক্ষেত্রে উপকারী হতে পারে।

n_jobs

n_jobs নির্ধারণ করে Training-এর সময় কতগুলো CPU Core ব্যবহার করা হবে।

n_jobs=-1

দিলে সিস্টেমের সব Available CPU Core ব্যবহার করা হয়।

এটি Model-এর Accuracy পরিবর্তন করে না।

শুধুমাত্র Training Speed বাড়ায়।

বিশেষ করে Large Dataset এবং Cross Validation-এর সময় এটি কার্যকর।

verbose

Default: 0

verbose নির্ধারণ করে Training-এর সময় কতটুকু Log বা Progress Information Console-এ দেখাবে।

verbose=0 মানে কোনো Output দেখাবে না। verbose=1 বা তার বেশি দিলে প্রতিটি Iteration-এর Progress দেখা যায়, যা Debugging বা দীর্ঘ Training-এর সময় কাজে লাগে।

dual

Default: 'auto'

dual Parameter নির্ধারণ করে Optimization Problem-এর Dual নাকি Primal Formulation ব্যবহার হবে। শুধুমাত্র solver='liblinear' এবং penalty='l2' হলে এটি প্রযোজ্য। সাধারণত $n_samples > n_features$ হলে dual=False (Primal) এবং $n_samples < n_features$ হলে dual=True (Dual) বেশি কার্যকর। 'auto' দিলে Scikit-Learn নিজেই সঠিকটি বেছে নেয়।

LogisticRegression() Attributes

coef_

Default: Created after fit()

প্রতিটি Feature-এর Learned Weight (Coefficient) সংরক্ষণ করে। Positive Coefficient হলে Feature বাড়ার সাথে Positive Class-এর Probability বাড়ে এবং Negative Coefficient হলে Probability কমে। এটি Logistic Regression-এর সবচেয়ে গুরুত্বপূর্ণ Output।

intercept_

Default: Created after fit()

Model-এর Learned Bias (Intercept) সংরক্ষণ করে। সব Feature-এর মান 0 হলে Decision Function-এর প্রাথমিক মান নির্ধারণ করে।

classes_

Default: Created after fit()

Training Data-তে থাকা সকল Target Class-এর List সংরক্ষণ করে। Binary Classification-এ সাধারণত [0, 1] এবং Multi-class Classification-এ সব Unique Class থাকে।

n_iter_

Default: Created after fit()

Solver Converge হতে প্রতিটি Class-এর জন্য কতটি Iteration লেগেছে তা সংরক্ষণ করে। যদি max_iter-এ পৌঁছেও Converge না হয়, তাহলে Warning দেখা যেতে পারে।

n_features_in_

Default: Created after fit()

Training-এর সময় Model মোট কতটি Input Feature ব্যবহার করেছে তা সংরক্ষণ করে।

feature_names_in_

Default: Created after fit() (*DataFrame input only*)

Training Data যদি Pandas DataFrame হয়, তাহলে Feature-গুলোর নাম সংরক্ষণ করে।

n_classes_

Default: Created after fit() (*Some scikit-learn versions*)

Training Data-তে মোট কতটি Target Class রয়েছে তা সংরক্ষণ করে। এটি len(classes_-এর সমান।

LogisticRegression-এর সবচেয়ে গুরুত্বপূর্ণ Parameter

IMPORTANT NOTE

Binary Cross-Entropy ভুল প্রেডিকশনকে exponentially বেশি penalize করে — একদম আত্মবিশ্বাসী কিন্তু ভুল প্রেডিকশনের loss প্রায় ∞ পর্যন্ত যেতে পারে। এটিই Log Loss-কে Classification-এর জন্য কার্যকর করে তোলে।

EXAM TIP

Odds $\rightarrow$ Log-Odds $\rightarrow$ Exponential $\rightarrow$ Algebraic rearrangement — এই ৪-ধাপের Derivation পরীক্ষায় প্রায়ই চাওয়া হয়। প্রতিটি ধাপ মুখস্থ না করে যুক্তি দিয়ে বোঝা ভালো, কারণ এতে ভুল করার সম্ভাবনা কমে যায়।

Advantages, Disadvantages ও Real-World Applications

Advantages	Disadvantages
আউটপুট সরাসরি probability হিসেবে interpretable	শুধুমাত্র linearly (বা near-linearly) separable ডেটায় ভালো কাজ করে
কম্পিউটেশনালি সাশ্রয়ী ও দ্রুত train হয়	Outlier-এর প্রতি sensitive
L1/L2 Regularization-এর মাধ্যমে Overfitting নিয়ন্ত্রণযোগ্য	জটিল Non-linear সম্পর্ক ক্যাপচার করতে পারে না
REAL-WORLD EXAMPLE Email Spam Detection, মেডিকেল ডায়াগনোসিসে রোগী আক্রান্ত/সুস্থ নির্ণয়, ব্যাংকিং-এ Credit Default Prediction, এবং Marketing-এ Customer Churn Prediction — এসব ক্ষেত্রে Logistic Regression একটি জনপ্রিয় Baseline Model।	
COMMON MISTAKE Multiclass সমস্যায় ভুলভাবে Binary Logistic Regression প্রয়োগ করা — <code>multi_class='multinomial'</code> বা One-vs-Rest ব্যবহার না করা। Imbalanced Dataset-এ <code>class_weight</code> প্যারামিটার উপেক্ষা করা, যার ফলে Minority Class-এর Recall খুব কম থাকে।	

INTERVIEW TIP

Q: কেন Logistic Regression-এ MSE ব্যবহার করা হয় না?

A: Sigmoid-এর সাথে MSE ব্যবহার করলে Cost Function Non-convex হয়ে যায়, ফলে Gradient Descent Local Minima-তে আটকে যেতে পারে। Binary Cross-Entropy ব্যবহার করলে ফাংশনটি Convex থাকে।

Q: Logistic Regression নামে 'Regression' থাকলেও এটি Classification Algorithm কেন?

A: কারণ এটি প্রথমে একটি Continuous Probability মান (Regression-এর মতো) প্রেডিক্ট করে, তারপর একটি Threshold (সাধারণত 0.5) ব্যবহার করে সেটিকে Discrete Class-এ রূপান্তর করে।

K-Nearest Neighbors (KNN)

1. Introduction to K-Nearest Neighbors

K-Nearest Neighbors (KNN) হলো Machine Learning-এর একটি জনপ্রিয় Supervised Learning Algorithm, যা Classification এবং Regression উভয় ধরনের সমস্যার সমাধান করতে ব্যবহৃত হয়। এটি একটি Non-Parametric এবং Lazy Learning Algorithm। অন্যান্য অনেক Machine Learning Algorithm-এর মতো KNN Training Phase-এ কোনো Mathematical Model তৈরি করে না। বরং সম্পূর্ণ Training Dataset মেমোরিতে সংরক্ষণ করে রাখে এবং Prediction করার সময় নতুন Data Point-এর সবচেয়ে কাছের প্রতিবেশীদের (Neighbors) খুঁজে বের করে সিদ্ধান্ত নেয়।

KNN-এর মূল ধারণা হলো:

Similar data points tend to exist close to each other in feature space.

অর্থাৎ, যেসব Data Point-এর বৈশিষ্ট্য একরকম, তারা সাধারণত Feature Space-এ কাছাকাছি অবস্থান করে।

2. Why Do We Need KNN?

বাস্তব জীবনে অনেক সমস্যা আছে যেখানে আমাদের একটি নতুন Observation-এর Category বা Value নির্ধারণ করতে হয়।

উদাহরণস্বরূপ:

- Email Spam নাকি Not Spam?
- Customer Product কিনবে নাকি কিনবে না?
- একজন রোগী রোগে আক্রান্ত নাকি সুস্থ?
- একটি বাড়ির সম্ভাব্য মূল্য কত?

এই ধরনের সমস্যার সমাধানে KNN ব্যবহার করা যায়।

KNN-এর বিশেষত্ব হলো এটি Data Distribution সম্পর্কে আগে থেকে কোনো Assumption করে না। ফলে জটিল Non-Linear Data-এর ক্ষেত্রেও এটি অনেক সময় ভালো কাজ করে।

3. Types of Problems Solved by KNN

Classification

Classification সমস্যায় Target Variable একটি Category হয়।

উদাহরণ:

Input	Output
Patient Data	Disease / No Disease
Email	Spam / Not Spam
Student Info	Pass / Fail

Classification-এর ক্ষেত্রে KNN Majority Voting ব্যবহার করে।

Regression

Regression সমস্যায় Target Variable Continuous Numeric Value হয়।

উদাহরণ:

Input	Output
House Features	House Price
Weather Data	Temperature
Employee Data	Salary

Regression-এর ক্ষেত্রে KNN Neighbors-এর Mean বা Median ব্যবহার করে।

4. Working Principle of KNN

KNN-এর সম্পূর্ণ কাজ Distance-এর উপর নির্ভরশীল।

ধরা যাক একটি নতুন Data Point এসেছে।

KNN নিচের ধাপগুলো অনুসরণ করে:

Step 1

একটি K Value নির্বাচন করা হয়।

$$K = 3$$

Step 2

নতুন Data Point থেকে Training Dataset-এর প্রতিটি Observation-এর Distance বের করা হয়।

Step 3

Distance অনুযায়ী ছোট থেকে বড় ক্রমে Sort করা হয়।

Step 4

সবচেয়ে কাছের K সংখ্যক Neighbor নির্বাচন করা হয়।

Step 5

Classification হলে Majority Voting এবং Regression হলে Average নেওয়া হয়।

5. Mathematical Foundation of KNN

KNN-এর সম্পূর্ণ Decision Making Distance Metrics-এর উপর ভিত্তি করে তৈরি।

দুটি Data Point-এর মধ্যকার Distance যত কম হবে, তারা তত বেশি Similar বলে বিবেচিত হবে।

5.1 Euclidean Distance

এটি KNN-এর সবচেয়ে ব্যবহৃত Distance Metric।

জ্যামিতিতে দুটি বিন্দুর সরলরেখার দূরত্ব নির্ণয়ের জন্য এটি ব্যবহার করা হয়।

যদি দুটি Point হয়:

$$P = (x_1, y_1)$$

এবং

$$Q = (x_2, y_2)$$

তাহলে তাদের Euclidean Distance:

$$d(P, Q) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

N-Dimensional Form

$$d(x, y) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$

এই Formula-ই Scikit-Learn-এর KNN-এর Default Distance Metric।

5.2 Manhattan Distance

Euclidean Distance সরলরেখার দূরত্ব মাপে।

কিন্তু Manhattan Distance Grid Path ধরে Distance মাপে।

Formula:

$$d(x, y) = \sum_{i=1}^n |x_i - y_i|$$

High Dimensional Dataset-এর ক্ষেত্রে অনেক সময় Manhattan Distance Euclidean Distance-এর চেয়ে ভালো ফলাফল দেয়।

5.3 Minkowski Distance

Euclidean এবং Manhattan Distance আসলে Minkowski Distance-এর Special Case।

$$d(x, y) = (\sum_{i=1}^n |x_i - y_i|^p)^{1/p}$$

যেখানে,

$$p = 1$$

হলে Manhattan Distance

এবং

$$p = 2$$

হলে Euclidean Distance পাওয়া যায়।

6. Complete Numerical Example

ধরা যাক নিচের Dataset দেওয়া আছে।

Customer	Age	Income	Buy
C1	2	4	No
C2	4	6	Yes
C3	4	2	No
C4	6	8	Yes

এখন নতুন Customer:

$$(3, 5)$$

এবং

$$K = 3$$

প্রথম Customer-এর সাথে Distance:

$$d_1 = \sqrt{(3 - 2)^2 + (5 - 4)^2} = 1.41$$

দ্বিতীয় Customer-এর সাথে:

$$d_2 = 1.41$$

তৃতীয় Customer-এর সাথে:

$$d_3 = 3.16$$

চতুর্থ Customer-এর সাথে:

$$d_4 = 4.24$$

Distance অনুযায়ী Sort করলে:

Customer	Distance	Class
C1	1.41	No
C2	1.41	Yes
C3	3.16	No
C4	4.24	Yes

Nearest 3 Neighbor:

- No
- Yes
- No

Majority Vote:

$$\begin{aligned} No &= 2 \\ Yes &= 1 \end{aligned}$$

Final Prediction:

No

7. Choosing the Optimal K Value

KNN-এর Performance-এর সবচেয়ে গুরুত্বপূর্ণ অংশ হলো K নির্বাচন করা।

K খুব ছোট হলে

$$K = 1$$

Model Noise Follow করতে শুরু করে।

ফলাফল:

- High Variance
- Overfitting

K খুব বড় হলে

$$K = 100$$

Model Local Pattern ভুলে যায়।

ফলাফল:

- High Bias
- Underfitting

সাধারণভাবে:

$$K = \sqrt{N}$$

দিয়ে শুরু করা হয়।

যেখানে N হলো Training Sample-এর সংখ্যা।

8. Feature Scaling in KNN

KNN Distance-Based Algorithm হওয়ার কারণে Feature Scaling অত্যন্ত গুরুত্বপূর্ণ।

ধরা যাক:

Feature	Range
Age	0-100
Salary	0-1,000,000

Distance Calculation-এর সময় Salary পুরো Distance-কে Dominant করে ফেলবে।

ফলে Age-এর Effect প্রায় হারিয়ে যাবে।

এই সমস্যার সমাধানের জন্য:

Standardization

$$z = \frac{x - \mu}{\sigma}$$

Min-Max Scaling

$$x' = \frac{x - x_{min}}{x_{max} - x_{min}}$$

প্রয়োগ করা হয়।

9. Curse of Dimensionality

KNN-এর সবচেয়ে বড় সীমাবদ্ধতাগুলোর একটি হলো Curse of Dimensionality।

Feature সংখ্যা বাড়তে থাকলে Data Point-গুলোর মধ্যে Distance-এর পার্থক্য কমতে শুরু করে।

ফলে:

- Nearest Neighbor খুঁজে পাওয়া কঠিন হয়
- Prediction Accuracy কমে যায়
- Computation Cost বৃদ্ধি পায়

এই কারণে High-Dimensional Dataset-এ KNN অনেক সময় দুর্বল পারফর্ম করে।

10. Computational Complexity

Operation	Complexity
Training	$O(1)$
Memory	$O(N)$
Prediction	$O(ND)$

যেখানে,

- N = Number of Samples
- D = Number of Features

এই কারণেই KNN-এর Training Fast হলেও Prediction তুলনামূলক Slow হয়।

KNN ব্যবহার করার সময় নিচের Parameters সবচেয়ে গুরুত্বপূর্ণ।

KNeighborsClassifier(

 n_neighbors=5,

 weights='distance',

 metric='minkowski',

 p=2,

 algorithm='auto'

)

```
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

wine = load_wine()

X = wine.data
y = wine.target

print("Shape of X:", X.shape)
print("Shape of y:", y.shape)

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
```



```

knn_pipeline = Pipeline([
    ("scaler", StandardScaler()),
    ("model", KNeighborsClassifier(
        n_neighbors=5,
        weights="uniform",
        metric="minkowski",
        p=2
    ))
])

knn_pipeline.fit(X_train, y_train)
y_pred = knn_pipeline.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)

print("\nAccuracy:")
print(accuracy)

print("\nClassification Report:")
print(classification_report(y_test, y_pred))

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

```

Distance Metric Comparison

Distance Metric	Formula (সংক্ষেপে)	ব্যবহারের প্রসঙ্গ
Euclidean (p=2)	$\sqrt{\sum (x_i - y_i)^2}$	সাধারণ, continuous low-dimensional data-তে default
Manhattan (p=1)	$\sum x_i - y_i $	High-dimensional বা grid-like data-তে কার্যকর
Minkowski (general)	$(\sum x_i - y_i ^p)^{1/p}$	p tune করে Euclidean/Manhattan-এর মধ্যে নমনীয়তা পাওয়া যায়

EXAM TIP

K নির্বাচনের rule of thumb $K \approx \sqrt{N}$ মুখস্থ রাখুন এবং Manual Distance Calculation (Euclidean) করে Majority Voting দেখানোর Numerical Example পরীক্ষায় প্রায়ই আসে।

COMMON MISTAKE

K = 1 ব্যবহার করে Model-কে অত্যন্ত Noise-sensitive করে ফেলা — এটি Training Accuracy-তে চমৎকার মনে হলেও Test Set-এ খারাপ Generalize করে।

Even K ব্যবহার করা, যা Binary Classification-এ Tie (সমান ভোট) তৈরি করতে পারে — সাধারণত Odd K ব্যবহার করাই নিরাপদ।

Advantages ও Disadvantages

Advantages	Disadvantages
বোঝা এবং বাস্তবায়ন করা সহজ	Prediction Phase ধীরগতির (Computationally Expensive)
Data Distribution সম্পর্কে কোনো Assumption করে না	Curse of Dimensionality-তে আক্রান্ত হয়
Classification ও Regression উভয় ক্ষেত্রে ব্যবহারযোগ্য	Feature Scaling বাধ্যতামূলক
Multi-class সমস্যায় স্বাভাবিকভাবেই কাজ করে	Memory-ভিত্তিক (পুরো ডেটাসেট সংরক্ষণ করতে হয়)

REAL-WORLD EXAMPLE

Recommendation System-এ Similar Users/Items খুঁজে বের করা, Image Recognition-এ Pattern Matching, এবং Medical Diagnosis-এ Similar Patient History অনুসন্ধান — এসব ক্ষেত্রে KNN ব্যাপকভাবে ব্যবহৃত হয়।

INTERVIEW TIP

Q: KNN-কে 'Lazy Learner' বলা হয় কেন?

A: কারণ এটি Training Phase-এ কোনো Model তৈরি করে না, শুধু ডেটা সংরক্ষণ করে রাখে; প্রকৃত Computation ঘটে শুধু Prediction Phase-এ।

Q: KNN-এ Feature Scaling কেন গুরুত্বপূর্ণ?

A: কারণ KNN সম্পূর্ণভাবে Distance-এর উপর নির্ভরশীল; বড় Scale-এর Feature ছোট Scale-এর Feature-কে Dominate করে ফেলে।

K-Nearest Neighbors (KNN) হলো একটি **Instance-Based** এবং **Lazy Learning Algorithm**। অন্যান্য Machine Learning Algorithm-এর মতো এটি Training-এর সময় কোনো Mathematical Model তৈরি করে না। বরং Training Data-কে Memory-তে সংরক্ষণ করে রাখে এবং নতুন Data আসলে তার কাছাকাছি থাকা Data Point-গুলোর Class দেখে Prediction করে।

ধরুন একটি নতুন Student-এর তথ্য দেওয়া হলো এবং জানতে হবে সে Pass করবে নাকি Fail। KNN প্রথমে Training Data-তে সবচেয়ে কাছের K জন Student খুঁজে বের করবে, তারপর তাদের Majority Vote-এর ভিত্তিতে Final Prediction দেবে।

এই কারণেই KNN-কে অনেক সময় "Similarity Based Learning" Algorithm বলা হয়।

n_neighbors

Default: 5

n_neighbors বা K হলো KNN-এর সবচেয়ে গুরুত্বপূর্ণ Hyperparameter। এটি নির্ধারণ করে Prediction করার সময় কতগুলো Nearest Neighbor বিবেচনা করা হবে।

ধরুন,

$$K = 5$$

তাহলে নতুন Data Point-এর সবচেয়ে কাছের 5টি Neighbor খুঁজে বের করা হবে এবং তাদের Vote গণনা করা হবে।

যদি K খুব ছোট হয়, যেমন:

$$n_neighbors = 1$$

তাহলে Model Training Data-এর Noise পর্যন্ত শিখে ফেলতে পারে। ফলে Overfitting দেখা দেয়।

অন্যদিকে K খুব বড় হলে অনেক দূরের Data Point-ও Decision-এ অংশ নেয়। তখন Model অত্যন্ত Smooth হয়ে যায় এবং Underfitting হতে পারে।

সাধারণভাবে:

- Small Dataset $\rightarrow K = 3$ বা 5
- Medium Dataset $\rightarrow K = 5$ বা 7
- Large Dataset $\rightarrow K = 11, 15, 21$ ইত্যাদি

ব্যবহার করা হয়।

KNN-এর Performance-এর উপর সবচেয়ে বেশি প্রভাব ফেলে `n_neighbors`।

weights

Default: 'uniform'

KNN-এ Neighbor-এর Vote কীভাবে গণনা করা হবে তা `weights` Parameter নিয়ন্ত্রণ করে।

Default Mode:

`weights='uniform'`

এখানে প্রতিটি Neighbor-এর Vote সমান গুরুত্ব পায়।

ধরুন $K=5$ ।

তাহলে ৫ জন Neighbor-এর প্রত্যেকের Vote-এর Weight হবে সমান।

অন্য Option:

`weights='distance'`

এখানে কাছের Neighbor-এর Vote বেশি গুরুত্বপূর্ণ হয়।

Weight গণনা হয়:

$$Weight = \frac{1}{Distance}$$

ফলে খুব কাছের Neighbor Prediction-এ বেশি প্রভাব ফেলে এবং দূরের Neighbor-এর প্রভাব কমে যায়।

যখন Dataset-এর Density সমান নয় অথবা Local Pattern বেশি গুরুত্বপূর্ণ, তখন `distance` সাধারণত ভালো Result দেয়।

metric

Default: 'minkowski'

KNN-এর মূল ভিত্তি হলো Distance Calculation।

কোন Formula ব্যবহার করে Distance বের করা হবে, সেটি metric Parameter নির্ধারণ করে।

ভুল Distance Metric নির্বাচন করলে "Nearest Neighbor" ভুলভাবে নির্ধারিত হতে পারে এবং Model-এর Accuracy কমে যেতে পারে।

Euclidean Distance

metric='euclidean'

Formula:

$$d = \sqrt{\sum (x_i - y_i)^2}$$

এটি Straight-Line Distance।

Continuous Numerical Data-এর জন্য এটি সবচেয়ে বেশি ব্যবহৃত Metric।

Manhattan Distance

metric='manhattan'

Formula:

$$d = \sum |x_i - y_i|$$

এটি Grid-Based Distance।

যদি Movement শুধুমাত্র Horizontal এবং Vertical Direction-এ সম্ভব হয়, তাহলে Manhattan Distance বেশি উপযুক্ত।

High-Dimensional Dataset-এ অনেক সময় এটি Euclidean-এর চেয়ে ভালো কাজ করে।

Minkowski Distance

metric='minkowski'

Formula:

$$d = (\sum |x_i - y_i|^p)^{1/p}$$

এটি Euclidean এবং Manhattan-এর General Form।

Scikit-Learn Default হিসেবে এটিই ব্যবহার করে।

Chebyshev Distance

metric='chebyshev'

Formula:

$$d = \max(|x_i - y_i|)$$

এখানে শুধুমাত্র Maximum Difference বিবেচনা করা হয়।

Cosine Distance

metric='cosine'

Text Mining, NLP এবং Document Similarity Problem-এ বেশি ব্যবহৃত হয়।

এটি Distance-এর পরিবর্তে Vector-এর Angle Measure করে।

p

Default: 2

p শুধুমাত্র তখন কাজ করে যখন:

metric='minkowski'

হয়।

এটি Minkowski Distance-এর Power Parameter।

যদি

$p=1$

হয়, তাহলে Minkowski Distance Manhattan Distance-এ রূপান্তরিত হয়।

$$d = \sum |x_i - y_i|$$

যদি

$p=2$

হয়, তাহলে এটি Euclidean Distance হয়ে যায়।

$$d = \sqrt{\sum (x_i - y_i)^2}$$

এই কারণে অনেক সময় আলাদা করে `metric='euclidean'` না লিখেও `metric='minkowski', p=2` ব্যবহার করা হয়।

algorithm

Default: 'auto'

KNN Prediction-এর সময় Nearest Neighbor Search করার জন্য কোন Internal Search Algorithm ব্যবহার হবে তা নির্ধারণ করে।

এটি Accuracy পরিবর্তন করে না।

শুধুমাত্র Computation Speed পরিবর্তন করে।

Auto

`algorithm='auto'`

Scikit-Learn Dataset দেখে নিজেই Best Algorithm নির্বাচন করে।

বেশিরভাগ ক্ষেত্রে এটিই যথেষ্ট।

Brute Force

algorithm='brute'

প্রতিটি Data Point-এর সাথে Distance Calculate করে।

Small Dataset-এর জন্য ভালো।

KD Tree

algorithm='kd_tree'

কম Dimension-এর Dataset-এ দ্রুত Search করতে পারে।

সাধারণত:

$$\text{Dimensions} < 20$$

হলে কার্যকর।

Ball Tree

algorithm='ball_tree'

High-Dimensional Dataset-এ KD Tree-এর চেয়ে ভালো কাজ করতে পারে।

leaf_size

Default: 30

KD Tree এবং Ball Tree-এর Leaf Node-এর Size নির্ধারণ করে।

এটি শুধুমাত্র Performance Optimization-এর জন্য ব্যবহৃত হয়।

ছোট Leaf Size:

- Tree গভীর হয়
- Memory বেশি লাগে

- Query দ্রুত হতে পারে

বড় Leaf Size:

- Tree কম গভীর হয়
- Memory কম লাগে
- Query ধীর হতে পারে

সাধারণ Dataset-এ Default Value পরিবর্তনের প্রয়োজন হয় না।

n_jobs

Default: None

Prediction বা Neighbor Search-এর সময় কতগুলো CPU Core ব্যবহার হবে তা নির্ধারণ করে।

n_jobs=-1

দিলে System-এর সব CPU Core ব্যবহার করা হয়।

এটি Accuracy পরিবর্তন করে না।

শুধুমাত্র Computation Time কমায়।

বিশেষ করে Large Dataset-এ এটি উপকারী।

KNN-এর আরও গুরুত্বপূর্ণ Parameter

অনেক Note-এ এগুলো বাদ যায়, কিন্তু বাস্তবে এগুলোও গুরুত্বপূর্ণ।

radius

Radius Neighbors Algorithm-এর ক্ষেত্রে ব্যবহৃত হয়।

radius=1.0

একটি নির্দিষ্ট Radius-এর ভেতরের Neighbor-গুলো বিবেচনা করা হয়।

তবে সাধারণ KNeighborsClassifier-এ এটি খুব বেশি ব্যবহৃত হয় না।

metric_params

Distance Metric-এর জন্য অতিরিক্ত Parameter পাঠাতে ব্যবহৃত হয়।

যখন Custom Distance Function ব্যবহার করা হয়, তখন এটি কাজে লাগে।

outlier_label

Radius-Based Neighbor Model-এ Outlier Handle করার জন্য ব্যবহৃত হয়।

effective_metric_

Training-এর পরে Model আসলে কোন Metric ব্যবহার করেছে তা জানায়।

effective_metric_params_

Metric-এর Final Parameter Value দেখায়।

KNeighborsClassifier() Attributes

classes_

Default: Created after fit()

Training Data-তে থাকা সকল Target Class-এর List সংরক্ষণ করে। Prediction করার সময় Model এই Class-গুলোর মধ্য থেকেই একটি Class নির্বাচন করে।

n_samples_fit_

Default: Created after fit()

Training-এর সময় মোট কতটি Sample ব্যবহার করা হয়েছে তা সংরক্ষণ করে। KNN পুরো Training Data-ই সংরক্ষণ করে, তাই এটি গুরুত্বপূর্ণ Attribute।

effective_metric_

Default: Created after fit()

Training শেষে আসলে কোন Distance Metric ব্যবহৃত হয়েছে তা সংরক্ষণ করে। যেমন euclidean, manhattan, minkowski ইত্যাদি।

effective_metric_params_

Default: Created after fit()

ব্যবহৃত Distance Metric-এর Parameter-গুলো সংরক্ষণ করে। উদাহরণস্বরূপ, metric='minkowski' এবং p=2 হলে এখানে সেই Parameter থাকবে।

outputs_2d_

Default: Created after fit()

Target Variable Single Output নাকি Multi-output তা নির্দেশ করে। False হলে Single Output এবং True হলে Multi-output Classification বোঝায়।

n_features_in_

Default: Created after fit()

Training-এর সময় Model মোট কতটি Input Feature ব্যবহার করেছে তা সংরক্ষণ করে।

feature_names_in_

Default: Created after fit() (*DataFrame input only*)

Training Data যদি Pandas DataFrame হয়, তাহলে Feature-গুলোর নাম সংরক্ষণ করে।

KNeighborsRegressor() Attributes

n_samples_fit_

Default: Created after fit()

Training-এর সময় মোট কতটি Sample ব্যবহার করা হয়েছে তা সংরক্ষণ করে।
Prediction করার সময় এই Sample-গুলোর মধ্য থেকেই Nearest Neighbor খুঁজে বের করা হয়।

effective_metric_

Default: Created after fit()

Training শেষে ব্যবহৃত প্রকৃত Distance Metric সংরক্ষণ করে।

effective_metric_params_

Default: Created after fit()

Distance Metric-এর Parameter-গুলো সংরক্ষণ করে। যেমন $p=1$, $p=2$ ইত্যাদি।

outputs_2d_

Default: Created after fit()

Target Variable Single Output নাকি Multi-output Regression তা নির্দেশ করে।

n_features_in_

Default: Created after fit()

Training-এর সময় ব্যবহৃত মোট Input Feature-এর সংখ্যা সংরক্ষণ করে।

feature_names_in_

Default: Created after fit() (*DataFrame input only*)

Training Data যদি Pandas DataFrame হয়, তাহলে Feature-গুলোর নাম সংরক্ষণ করে।

Quick Revision Notes — KNN

Category	Key Points
Key Formula	Euclidean: $\sqrt{\sum (x_i - y_i)^2}$; Minkowski: $(\sum x_i - y_i ^p)^{1/p}$
Type	Non-parametric, Lazy Learner, Instance-based
Complexity	Training $O(1)$, Prediction $O(N \cdot D)$
K too small	Overfitting (High Variance)
K too large	Underfitting (High Bias)
Must-do	Feature scaling before fitting
Exam Focus	Manual distance computation & majority voting
Interview Focus	Curse of dimensionality; lazy vs eager learning

GridSearchCV in KNN

1. Why Do We Need GridSearchCV?

KNN-এর সবচেয়ে গুরুত্বপূর্ণ Hyperparameter হলো:

$$K = n_neighbors$$

কিন্তু প্রশ্ন হলো:

K কত নেব?

3?

5?

7?

11?

21?

যদি ভুল K নির্বাচন করি তাহলে Model-এর Performance অনেক কমে যেতে পারে।

ধরো:

K	Accuracy
3	88%
5	92%
7	95%
9	93%
11	90%

এখানে দেখা যাচ্ছে:

$$K = 7$$

সবচেয়ে ভালো।

কিন্তু Dataset দেখে আগে থেকে এটা জানা সম্ভব না।

তাই আমাদের বিভিন্ন K পরীক্ষা করতে হবে।

এই কাজটিই GridSearchCV স্বয়ংক্রিয়ভাবে করে।

2. What is GridSearchCV?

GridSearchCV হলো Scikit-Learn-এর একটি Hyperparameter Tuning Technique।

এটি বিভিন্ন Parameter Combination পরীক্ষা করে এবং Best Combination খুঁজে বের করে।

ধরো তুমি KNN-এর জন্য নিচের Parameters পরীক্ষা করতে চাও:

```
n_neighbors = [3,5,7,9]
weights = ['uniform','distance']
```

তাহলে মোট Combination হবে:

n_neighbors	weights
3	uniform
3	distance
5	uniform
5	distance
7	uniform
7	distance
9	uniform
9	distance

মোট:

$$4 \times 2 = 8$$

টি Model Train হবে।

3. Why Is It Called Grid Search?

ধরো Parameter Space হলো:

Parameter	Values
K	3,5,7
Weight	uniform,distance
p	1,2

তাহলে Grid তৈরি হবে:

K	Weight	p
3	uniform	1
3	uniform	2
3	distance	1
3	distance	2
5	uniform	1
5	uniform	2
5	distance	1
5	distance	2
7	uniform	1
7	uniform	2
7	distance	1
7	distance	2

মোট:

$$3 \times 2 \times 2 = 12$$

Combination I

GridSearchCV এই পুরো Grid Search করে।

তাই নাম:

Grid Search

4. What Does CV Mean?

CV = Cross Validation

Grid Search শুধু Train/Test Split ব্যবহার করে না।

বরং Cross Validation ব্যবহার করে।

5. Cross Validation Intuition

ধরো Dataset-এ:

1000

Samples আছে।

আমরা

$cv = 5$

দিলাম।

তাহলে Dataset 5 ভাগে ভাগ হবে।

Fold-1

Fold-2

Fold-3

Fold-4

Fold-5

Iteration 1:

Train:

2 3 4 5

Validation:

1

Iteration 2:

Train:

1 3 4 5

Validation:

2

Iteration 3:

Train:

1 2 4 5

Validation:

3

Iteration 4:

Train:

1 2 3 5

Validation:

4

Iteration 5:

Train:

1 2 3 4

Validation:

5

এরপর Average Accuracy বের করা হয়।

$$CV\ Score = \frac{Score_1 + Score_2 + Score_3 + Score_4 + Score_5}{5}$$

6. Backend Working of GridSearchCV

ধরি:

n_neighbors=[3,5,7]

weights=['uniform','distance']

Combination:

(3,uniform)

(3,distance)

(5,uniform)

(5,distance)

(7,uniform)

(7,distance)

মোট:

6

Combination I

এবং

cv=5

তাহলে:

$$6 \times 5 = 30$$

বার Model Train হবে।

GridSearchCV:

1. First Combination নেয়
2. 5 Fold CV চালায়
3. Average Score বের করে
4. Next Combination নেয়
5. Repeat করে

শেষে Highest Score যেটা দেয় সেটাকে Best Model ঘোষণা করে।

7. Important KNN Hyperparameters

GridSearchCV সাধারণত KNN-এর নিচের Parameters Tune করে।

n_neighbors

সবচেয়ে গুরুত্বপূর্ণ Parameter।

`n_neighbors = [3,5,7,9,11]`

ছোট K

→ Overfitting

বড় K

→ Underfitting

weights

Uniform

সব Neighbor-এর Vote সমান।

`weights='uniform'`

Distance

কাছের Neighbor-এর Vote বেশি।

weights='distance'

Weight:

$$w = \frac{1}{d}$$

p

Minkowski Distance Power।

p = 1

Manhattan

Distance

p = 2

Euclidean

Distance

metric

metric='minkowski'

Default।

অন্যান্য:

euclidean
manhattan
cosine
chebyshev

8. Example of Full Search Space

```
param_grid = {  
    "model__n_neighbors":[  
        3,5,7,9,11,15  
    ],  
    "model__weights":[  
        "uniform",  
        "distance"  
    ],  
    "model__p":[  
        1,  
        2  
    ]  
}
```

Total Models:

$$6 \times 2 \times 2 = 24$$

If

cv=5

Then

$$24 \times 5 = 120$$

Model Fits

9. Important Outputs

After Training:

Best Parameters

grid.best_params_

Output:

```
{  
'n_neighbors':7,  
'weights':'distance',
```

```
'p':2  
}
```

Best Score

grid.best_score_

Output:

0.972

Meaning:

97.2%

Cross Validation Accuracy

Best Model

grid.best_estimator_

Returns:

Best Trained Pipeline

10. Advantages of GridSearchCV

Automatic Hyperparameter Tuning

নিজে নিজে Combination পরীক্ষা করে।

Uses Cross Validation

Result বেশি Reliable।

Finds Optimal Model

Best Parameters খুঁজে দেয়।

Easy to Use

Scikit-Learn-এ কয়েক লাইনের Code।

11. Disadvantages

Slow

Combination বেশি হলে সময় বেশি লাগে।

Computationally Expensive

Large Dataset-এ Costly।

Exhaustive Search

সব Combination পরীক্ষা করে।

কখনও কখনও অপ্রয়োজনীয়ও হতে পারে।

```
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.model_selection import GridSearchCV

from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier

wine = load_wine()

X = wine.data
y = wine.target

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

pipe = Pipeline([
    ("scaler", StandardScaler()),
    ("model", KNeighborsClassifier())
])
```

```
param_grid = {  
    "model__n_neighbors": [3,5,7,9,11,15],  
    "model__weights": [  
        "uniform",  
        "distance"  
    ],  
    "model__p": [  
        1,  
        2  
    ]  
}
```

```
grid = GridSearchCV(  
    estimator=pipe,  
    param_grid=param_grid,  
    cv=5,  
    scoring="accuracy",  
    n_jobs=-1  
)
```

```
grid.fit(X_train, y_train)  
  
print("Best Parameters:")  
print(grid.best_params_)
```

```
print("Best Parameters:")  
print(grid.best_params_)
```

```
print("\nBest CV Score:")  
print(grid.best_score_)
```

```
print("\nBest Pipeline:")  
print(grid.best_estimator_)
```

```
test_accuracy = grid.score(X_test, y_test)
```

```
print("\nTest Accuracy:")  
print(test_accuracy)
```


refit

Default: True

refit নির্ধারণ করে GridSearchCV Best Parameters খুঁজে পাওয়ার পরে সম্পূর্ণ Training Data দিয়ে আবার Model Refit করবে কিনা। True রাখলে grid.best_estimator_ থেকে সরাসরি Prediction করা যায়। False করলে শুধু Best Parameters জানা যাবে কিন্তু সরাসরি Predict করা যাবে না।

verbose

Default: 0

verbose নির্ধারণ করে GridSearchCV-এর Search চলাকালে কতটুকু Progress দেখাবে। verbose=0 মানে কোনো Log নেই। verbose=2 বা 3 দিলে প্রতিটি Fold এবং Parameter Combination-এর Score Console-এ দেখা যায়, যা বড় Search Space-এ কতটুকু এগোলো তা বোঝার জন্য কার্যকর।

pre_dispatch

Default: '2*n_jobs'

pre_dispatch নির্ধারণ করে Parallel Execution-এর সময় একসাথে কতগুলো Job Memory-তে রাখা হবে। এটি Memory Usage নিয়ন্ত্রণ করে। অনেক বড় Dataset বা বেশি Combination-এ Memory সমস্যা হলে এই মান কমিয়ে দেওয়া যায়, যেমন pre_dispatch=8।

error_score

Default: np.nan

error_score নির্ধারণ করে কোনো Combination Train করার সময় Error বা Exception আসলে সেই Combination-এর Score হিসেবে কী মান বসানো হবে। Default nan মানে Error হলে সেই Combination-এর Score NaN হয়ে যাবে। 'raise' দিলে Error সরাসরি Raise হবে এবং পুরো Training বন্ধ হয়ে যাবে, যা Debugging-এর জন্য সুবিধাজনক।

return_train_score

Default: False

return_train_score=True করলে GridSearchCV প্রতিটি Fold-এ Validation Score-এর পাশাপাশি Training Score-ও cv_results_-এ সংরক্ষণ করে। Train এবং Validation Score একসাথে তুলনা করলে Overfitting বা Underfitting সহজে সনাক্ত করা যায়। তবে এটি True করলে Computation সময় কিছুটা বাড়ে।

GridSearchCV() Attributes

best_estimator_

Default: Created after fit()

সব Parameter Combination পরীক্ষা করার পর যে Model সবচেয়ে ভালো Performance করেছে, সেটিকে Best Parameter দিয়ে পুরো Training Data-তে পুনরায় (Refit) Train করা হয়। সেই Final Trained Model-টি এই Attribute-এ সংরক্ষণ থাকে।

best_params_

Default: Created after fit()

সব Parameter Combination-এর মধ্যে যেটি সর্বোচ্চ Cross-Validation Score পেয়েছে, সেই Parameter-এর Dictionary সংরক্ষণ করে।

best_score_

Default: Created after fit()

Best Parameter Combination-এর Mean Cross-Validation Score সংরক্ষণ করে। এটি বিভিন্ন Fold-এর Average Score।

best_index_

Default: Created after fit()

cv_results_-এর মধ্যে Best Parameter Combination কোন Index-এ রয়েছে তা সংরক্ষণ করে।

cv_results_

Default: Created after fit()

প্রতিটি Parameter Combination-এর বিস্তারিত ফলাফল Dictionary আকারে সংরক্ষণ করে। এর মধ্যে Mean Score, Standard Deviation, Rank, Fit Time, Score Time এবং প্রতিটি Fold-এর Score থাকে। এটি Grid Search-এর সবচেয়ে বিস্তারিত Output।

scorer_

Default: Created after fit()

Training-এর সময় ব্যবহৃত Scoring Function সংরক্ষণ করে। যেমন accuracy, f1, precision, recall, neg_mean_squared_error ইত্যাদি।

n_splits_

Default: Created after fit()

Cross-Validation-এর সময় মোট কতটি Fold ব্যবহার করা হয়েছে তা সংরক্ষণ করে।

refit_time_

Default: Created after fit() (*Only when refit=True*)

Best Parameter পাওয়ার পর পুরো Training Data-তে Final Model পুনরায় Train (Refit) করতে কত সময় লেগেছে তা সেকেন্ডে সংরক্ষণ করে।

multimetric_

Default: Created after fit()

একাধিক Scoring Metric ব্যবহার করা হয়েছে কিনা তা নির্দেশ করে। True হলে Multiple Metrics এবং False হলে একটি মাত্র Metric ব্যবহার হয়েছে।

classes_

Default: Created after fit() (*Only if the estimator is a classifier*)

যদি ভিতরের Estimator একটি Classifier হয়, তাহলে Training Data-তে থাকা সকল Target Class-এর List সংরক্ষণ করে।

n_features_in_

Default: Created after fit()

Training-এর সময় Best Estimator মোট কতটি Input Feature ব্যবহার করেছে তা সংরক্ষণ করে।

feature_names_in_

Default: Created after fit() (*DataFrame input only*)

Training Data যদি Pandas DataFrame হয়, তাহলে Feature-গুলোর নাম সংরক্ষণ করে।

refit

Default: Available after initialization

এই Attribute নির্দেশ করে Best Parameter পাওয়ার পর Model-টিকে পুরো Training Data-তে পুনরায় Train করা হয়েছে কিনা। True হলে Refit করা হয়, False হলে করা হয় না।

multimetric_

Default: Created after fit()

Grid Search-এ একাধিক Scoring Function ব্যবহার করা হয়েছে কিনা তা নির্দেশ করে। এটি Boolean (True বা False) মান ধারণ করে।

PRACTICAL INSIGHT

Combination সংখ্যা বেশি হলে GridSearchCV-এর বদলে RandomizedSearchCV ব্যবহার করা ভালো, যা সব Combination না পরীক্ষা করে র্যান্ডমভাবে কিছু Combination Sample করে দ্রুত একটি ভালো Result দেয়।

INTERVIEW TIP

Q: GridSearchCV-তে cv=5 মানে কী?

A: ডেটাসেটকে ৫টি Fold-এ ভাগ করে প্রতিবার ৪টি Fold দিয়ে Train এবং ১টি Fold দিয়ে Validate করা হয়; এই প্রক্রিয়া ৫ বার পুনরাবৃত্তি হয়ে Average Score বের করা হয়।

Q: GridSearchCV এবং RandomizedSearchCV-এর মধ্যে পার্থক্য কী?

A: GridSearchCV সব Combination Exhaustively পরীক্ষা করে, যেখানে RandomizedSearchCV একটি নির্দিষ্ট সংখ্যক Combination র্যান্ডমভাবে Sample করে — তাই বড় Search Space-এ এটি অনেক দ্রুত।

Quick Revision Notes — GridSearchCV

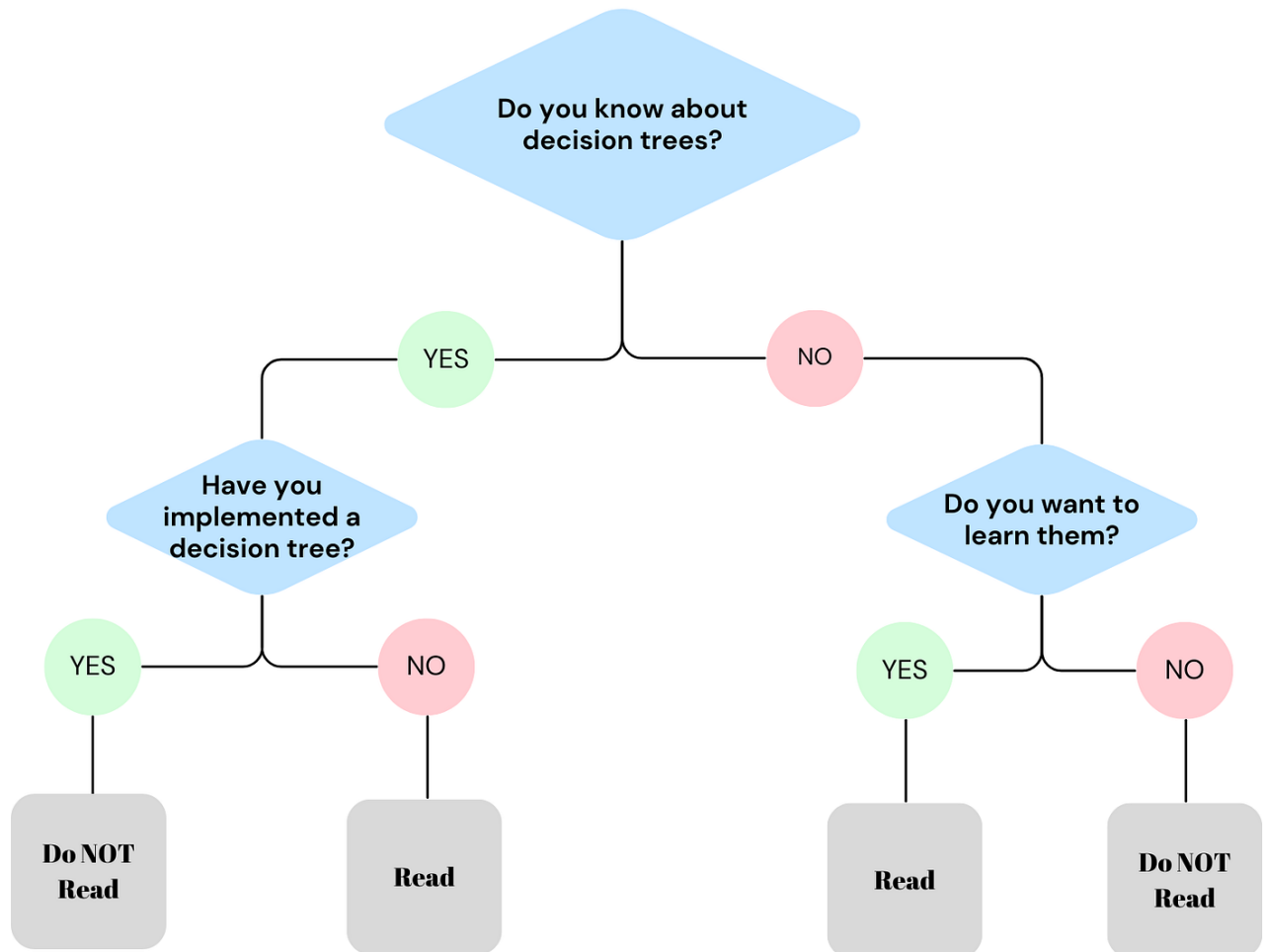
Category	Key Points
Key Formula	Total Fits = (Combinations) × (cv folds)
Purpose	Exhaustive hyperparameter search with cross-validation
Key Outputs	best_params_, best_score_, best_estimator_
Trade-off	Reliability (CV) vs Computation cost (exhaustive)
Alternative	RandomizedSearchCV for large search spaces
Exam Focus	Compute total model fits given grid size and cv
Interview Focus	GridSearchCV vs RandomizedSearchCV vs Bayesian Optimization

Decision Tree

Decision Tree (ডিসিশন ট্রি) হলো মেশিন লার্নিংয়ের একটি অত্যন্ত জনপ্রিয়, সহজ এবং শক্তিশালী Supervised Learning Algorithm, যা ক্লাসিফিকেশন (Classification) এবং রিগ্রেশন (Regression) উভয় ধরনের সমস্যার সমাধানে ব্যবহৃত হয়।

সহজ কথায়, এটি দেখতে একটি উল্টানো গাছের মতো, যার ওপরের দিকে থাকে মূল রুট বা কাণ্ড এবং নিচের দিকে ডালপালা বেয়ে চূড়ান্ত পাতা বা সিদ্ধান্তে পৌঁছানো যায়।

DECISION TREE



উদাহরণ (Intuition)

ধরুন, ছুটির দিনে আপনি বন্ধুদের সাথে বাইরে খেলতে যাবেন কি না, তা নিয়ে সিদ্ধান্ত নিতে চান। আপনার মনের ভেতর ঠিক নিচের মতো একটি চিন্তা কাজ করবে:

- প্রথম প্রশ্ন (রুট):** বাইরে কি আজ বৃষ্টি হচ্ছে?
 - যদি হ্যাঁ হয় → আপনি বাইরে যাবেন না (চূড়ান্ত সিদ্ধান্ত)।
 - যদি না হয় → আপনি পরবর্তী প্রশ্নে যাবেন।
- দ্বিতীয় প্রশ্ন:** বাইরে কি প্রচণ্ড গরম বা রোদ?
 - যদি হ্যাঁ হয় → আপনি বাইরে যাবেন না।
 - যদি না হয় → আপনি বাইরে খেলতে যাবেন (চূড়ান্ত সিদ্ধান্ত)।

মেশিন লার্নিংয়ে ডিসিশন ট্রি ঠিক এইভাবেই ডেটাসেটের ফিচারগুলোর ওপর ভিত্তি করে একটার পর একটা "হ্যাঁ/না" (Yes/No) বা "সত্য/মিথ্যা" (True/False) প্রশ্ন তৈরি করে ডেটাকে ছোট ছোট ভাগে ভাগ করে ফেলে।

ডিসিশন ট্রি-এর মূল গাঠনিক উপাদান (Core Structure)

একটি ডিসিশন ট্রি মূলত ৩টি প্রধান অংশ নিয়ে গঠিত হয়:

- Root Node (মূল নোড):** এটি গাছের একদম শুরুর বা শীর্ষের নোড। পুরো ডেটাসেটটি এখান থেকেই প্রথম বিভাজন (Split) শুরু করে। আমাদের উদাহরণে "বৃষ্টি হচ্ছে কি না" ছিল রুট নোড।
- Internal/Decision Node (অভ্যন্তরীণ নোড):** এগুলো হলো মাঝখানের ডালপালা বা প্রশ্নগুলোর নোড, যেখানে ডেটার কোনো একটি বৈশিষ্ট্যের ওপর ভিত্তি করে নতুন সিদ্ধান্ত নেওয়া হয় এবং ট্রি-টি আরও নিচে নেমে যায়।
- Leaf Node (পাতা বা চূড়ান্ত নোড):** এগুলো হলো গাছের একদম শেষ প্রান্তের নোড। এর নিচে আর কোনো ডালপালা গজায় না। এটিই মূলত আমাদের চূড়ান্ত প্রেডিকশন বা আউটপুট (যেমন: বাইরে যাব নাকি যাব না) নির্দেশ করে।

কেন এটি এত জনপ্রিয়? (Advantages)

- **সহজে বোঝা যায় (Interpretable):** এটি কীভাবে সিদ্ধান্ত নিচ্ছে তা মানুষ খুব সহজেই বুঝতে পারে (অনেকটা হোয়াইট-বক্স মডেলের মতো)।
- **ডেটা প্রিপারেশন কম লাগে:** এর জন্য খুব বেশি ফিচার স্কেলিং (যেমন: StandardScaler বা MinMaxScaler) করার প্রয়োজন হয় না, কারণ এটি ডেটার মানের আকারের চেয়ে থ্রেশহোল্ড স্প্লিটের ওপর বেশি নির্ভর করে।
- **Non-linear ডেটায় দারুণ কাজ করে:** ডেটার ভেতরে যদি সরলরৈখিক সম্পর্ক না থেকে জটিল বা আঁকাবাঁকা সম্পর্ক থাকে, তবুও এটি চমৎকার প্যাটার্ন খুঁজে বের করতে পারে।

তবে ডিসিশন ট্রি-এর একটি বড় সীমাবদ্ধতা হলো এটি খুব দ্রুত ডেটা মুখস্থ করে ফেলে, যাকে মেশিন লার্নিংয়ের ভাষায় Overfitting (High Variance) বলা হয়। এই ওভারফিটিং দূর করার জন্য পরবর্তীতে গাছের ডালপালা কেটে ছোট করা হয় (Pruning) অথবা একাধিক ডিসিশন ট্রি একসাথে মিলিয়ে Random Forest-এর মতো উন্নত অ্যালগরিদম ব্যবহার করা হয়।

ডিসিশন ট্রি (Decision Tree) Classification (ক্লাসিফিকেশন) এবং Regression (রিগ্রেশন) উভয় সমস্যার জন্যই ব্যবহার করা যায়। টার্গেট ভ্যারিয়েবলের বা আউটপুটের ধরনের ওপর ভিত্তি করে এটিকে দুই ভাগে ভাগ করা হয়:

১. Decision Tree Classifier (শ্রেণিবিভাগকারী)

- কখন ব্যবহার হয়: যখন আপনার আউটপুট বা টার্গেট ভ্যারিয়েবলটি কোনো ক্যাটাগরি, শ্রেণি বা ডিসক্রিট লেবেল (Discrete Label) হয়।
- কীভাবে প্রেডিক্ট করে: এটি গাছের শেষ প্রান্তে বা লিফ নোডে (Leaf Node) থাকা স্যাম্পলগুলোর মেজরিটি ভোটিং (Majority Voting) বা সংখ্যাগরিষ্ঠের ভোটের ভিত্তিতে ফাইনাল ক্লাস প্রেডিক্ট করে।
- বাস্তব উদাহরণ: কোনো ইমেইল 'Spam' নাকি 'Not Spam'।
 - একজন যাত্রী টাইটানিকে 'Survived' (বঁচে গেছেন) নাকি 'Not Survived' (মারা গেছেন)।

২. Decision Tree Regressor (মান অনুমানকারী)

- কখন ব্যবহার হয়: যখন আপনার আউটপুট বা টার্গেট ভ্যারিয়েবলটি কোনো কন্টিনিউয়াস বা ধারাবাহিক সংখ্যা (Continuous Numeric Value) হয়।
- কীভাবে প্রেডিক্ট করে: এটি লিফ নোডে (Leaf Node) পৌঁছানো ডেটা পয়েন্টগুলোর মানের গড় (Mean) বা মিডিয়ান (Median) হিসাব করে চূড়ান্ত সংখ্যাটি প্রেডিক্ট করে।
- বাস্তব উদাহরণ:
 - একটি বাড়ির আয়তন ও লোকেশন দেখে তার 'দাম' বা 'মূল্য' (Price) কত হবে তা অনুমান করা।
 - আবহাওয়ার ডেটা বিশ্লেষণ করে আগামীকালের 'তাপমাত্রা' (Temperature) কত ডিগ্রি হবে তা প্রেডিক্ট করা।

The Mathematical Split

গাছটি কীভাবে ডালপালা মেলবে বা ডেটাকে কোন পয়েন্টে কাটবে (Split), তার গাণিতিক পরিমাপ দুই ক্ষেত্রে ভিন্ন:

- **Classifier-এর ক্ষেত্রে:** ডেটার বিশুদ্ধতা পরিমাপ করার জন্য Gini Impurity (জিনি ইম্পিউরিটি) অথবা Information Gain / Entropy (এনট্রপি) ব্যবহার করা হয়। লক্ষ্য থাকে প্রতিটা স্প্লিটে ডেটাকে যতটা সম্ভব নিখুঁতভাবে আলাদা করা।
- **Regressor-এর ক্ষেত্রে:** ডেটার বিস্তৃতি বা ভুল পরিমাপের জন্য Variance Reduction (ভ্যারিয়েন্স রিডাকশন) অথবা Mean Squared Error (MSE) ব্যবহার করা হয়। লক্ষ্য থাকে এমনভাবে ডেটা ভাগ করা যাতে প্রতিটা ভাগের ভেতরের সংখ্যাগুলোর নিজেদের মধ্যকার দূরত্ব বা এরর সর্বনিম্ন হয়।

কোন ক্ষেত্রে সবচেয়ে বেশি ব্যবহার হয় এবং কেন?

উত্তর: Classification

- **কারণ ১ (সহজ সিদ্ধান্ত):** বাস্তব জীবনের বেশিরভাগ ব্যবসায়িক সিদ্ধান্ত বা শ্রেণিবিভাগ "হ্যাঁ/না" (Yes/No) বা নির্দিষ্ট কিছু ক্যাটাগরির ওপর ভিত্তি করে হয় (যেমন: কাস্টমার প্রোডাক্ট কিনবে নাকি কিনবে না)। ডিসিশন ট্রি-এর "IF-ELSE" লজিক এই ধরনের সমস্যা খুব নিখুঁতভাবে সমাধান করতে পারে।

- **রিগ্রেশনে সীমাবদ্ধতা:** রিগ্রেশনের ক্ষেত্রে (যেমন: বাড়ির দাম অনুমান করা) ডিসিশন ট্রি ডেটাকে ছোট ছোট বক্সে ভাগ করে প্রতিটা বক্সের গড় মান দেয়। এর ফলে আউটপুটের গ্রাফটি মসৃণ (*Smooth*) না হয়ে সিঁড়ির মতো ধাপযুক্ত (*Step – like*) হয়, যা অনেক সময় নিখুঁত সংখ্যাচক প্রেডিকশন দিতে পারে না। তাই রিগ্রেশনের জন্য মানুষ লিনিয়ার রিগ্রেশন বা অন্যান্য অ্যালগরিদম বেশি পছন্দ করে।

কোন কোন সেক্টরে সবচেয়ে বেশি ব্যবহার হয়?

ডিসিশন ট্রি এবং এর উন্নত রূপ (যেমন: Random Forest, XGBoost) মূলত নিচের সেক্টরগুলোতে ব্যাপকভাবে ব্যবহৃত হয়:

১. ব্যাংকিং ও ফাইন্যান্স সেক্টর (Banking & Finance)

- **ক্রেডিট স্কোরিং (Credit Scoring):** একজন কাস্টমারকে লোন বা ক্রেডিট কার্ড দেওয়া নিরাপদ কি না (ল্যোan ডিফল্টার হবে কি না), তা ক্লাসিফাই করতে।
- **জালিয়াতি শনাক্তকরণ (Fraud Detection):** কোনো ক্রেডিট কার্ডের ট্রানজেকশন আসল নাকি জালিয়াতি (*Fraud*), তা তাৎক্ষণিকভাবে ধরে ফেলতে।

২. স্বাস্থ্যসেবা ও চিকিৎসা বিজ্ঞান (Healthcare & Medicine)

- **রোগ নির্ণয় (Disease Diagnosis):** রোগীর লক্ষণ (রক্তচাপ, সুগার লেভেল, বয়স) দেখে তার টিউমারটি ম্যালিগন্যান্ট (ক্যান্সার) নাকি বিনাইন (সাধারণ), তা শ্রেণিবদ্ধ করতে।
- **ঝুঁকি মূল্যায়ন (Risk Assessment):** রোগীর মেডিকেল হিস্ট্রি দেখে ভবিষ্যতে তার হার্ট অ্যাটাকের ঝুঁকি কতটা, তা বের করতে।

৩. ই-কমার্স ও মার্কেটিং (E-commerce & Marketing)

- **কাস্টমার চার্ন প্রেডিকশন (Customer Churn):** কোনো কাস্টমার আপনার সার্ভিস বা সাবস্ক্রিপশন ছেড়ে চলে যাবে কি না, তা আগে থেকে অনুমান করতে।
- **টার্গেটেড মার্কেটিং:** কাস্টমারের বয়স, লিঙ্গ ও ব্রাউজিং হিস্ট্রি দেখে সে কোন ক্যাটাগরির প্রোডাক্ট কিনতে পারে, তা ক্লাসিফাই করে সঠিক অফার পাঠানো।

৪. মানব সম্পদ বিভাগ (Human Resources - HR)

- **প্রার্থী বাছাই (Candidate Screening):** শত শত জীবনবৃত্তান্ত (CV) থেকে কোন প্রার্থী ইন্টারভিউয়ের জন্য যোগ্য (Shortlisted) আর কে যোগ্য নয়, তা ডিসিশন ট্রি ফিল্টারিংয়ের মাধ্যমে দ্রুত আলাদা করতে।

Why we need Decision Tree instead of Linear Regression, Logistic Regression and KNN?

বাস্তব প্রজেক্টের কাজের অভিজ্ঞতার ওপর ভিত্তি করে ডিসিশন ট্রি-এর প্রয়োজনীয়তাকে ৪টি প্রধান সুবিধায় ভাগ করা যায়:

১. জটিল এবং নন-লিনিয়ার ডেটা হ্যান্ডলিং (Handling Complex Non-linear Relations)

- **লিনিয়ার ও লজিস্টিক রিগ্রেশনের সীমাবদ্ধতা:** এই দুটি মডেল ধরে নেয় যে ইনপুট ফিচার এবং টার্গেট ভ্যারিয়েবলের মধ্যে একটি সরলরৈখিক সম্পর্ক (Linear Relationship) বিদ্যমান। কিন্তু বাস্তব দুনিয়ার ডেটা (যেমন: টাইটানিক ডেটাসেট বা বাড়ির দাম) সাধারণত অত্যন্ত আঁকাবাঁকা বা নন-লিনিয়ার হয়। রিগ্রেশন মডেল দিয়ে জোর করে এই ডেটা ফিট করতে গেলে মডেলটির Bias হাই হয়ে যায় (Underfitting ঘটে)।
- **ডিসিশন ট্রি-এর সুবিধা:** ডিসিশন ট্রি ডেটার মধ্যে কোনো সরলরৈখিক সম্পর্ক খোঁজে না। এটি ডেটাসেটকে টুকরো টুকরো বক্সে ভাগ (Feature Space Partitioning) করে কাজ করে। ফলে ডেটার সম্পর্ক যত জটিল বা আঁকাবাঁকাই হোক না কেন, এটি খুব সহজেই সেই প্যাটার্ন ক্যাপচার করতে পারে।

২. ফিচার স্কেলিং এবং আউটলায়ার থেকে মুক্তি (No Need for Feature Scaling & Outlier Robustness)

- **KNN এবং রিগ্রেশনের সীমাবদ্ধতা:** KNN সম্পূর্ণভাবে দূরত্বের (Distance Metrics যেমন: Euclidean Distance) ওপর ভিত্তি করে কাজ করে। ফলে কোনো ফিচারের স্কেল যদি বড় হয় (যেমন: Salary ০-১,০০০,০০০ বনাম Age ০-১০০), তবে বড় ফিচারটি পুরো দূরত্বকে ডমিনেন্ট করে। তাই KNN এবং রিগ্রেশন ব্যবহারের আগে StandardScaler বা MinMaxScaler দিয়ে ডেটা স্কেল করা বাধ্যতামূলক। এছাড়া এগুলো আউটলায়ারের প্রতি অত্যন্ত সংবেদনশীল (Sensitive)।
- **ডিসিশন ট্রি-এর সুবিধা:** ডিসিশন ট্রি একটি Scale-Independent অ্যালগরিদম। এটি দূরত্বের হিসাব করে না, বরং প্রতিটা ফিচারের জন্য কাস্টম থ্রেশহোল্ড (যেমন: Age > 30 কিনা) চেক করে। তাই আপনার ডেটাতে ফিচার স্কেলিং না করলেও এটি নিখুঁত কাজ করে।

এবং চরম আউটলায়ার থাকলেও গাছের স্প্লিটিং মেকানিজমে কোনো নেতিবাচক প্রভাব পড়ে না।

৩. চরম ব্যাখ্যাযোগ্যতা বা হোয়াইট-বক্স মেকানিজম (Extreme Interpretability)

- **লজিস্টিক রিগ্রেশন ও KNN-এর সীমাবদ্ধতা:** KNN-কে "Lazy Learner" বলা হয়, কারণ এটি ব্যাকএন্ডে কোনো গাণিতিক মডেল বা সূত্র তৈরি করে না, শুধু ডেটা মেমোরিতে জমিয়ে রাখে। বড় প্রজেক্টে বা ইন্টারভিউতে মডেল কীভাবে প্রেডিক্ট করছে তা KNN বা লজিস্টিক রিগ্রেশনের জটিল সহগ (Coefficients) থেকে সাধারণ মানুষের পক্ষে বোঝা কঠিন।
- **ডিসিশন ট্রি-এর সুবিধা:** ডিসিশন ট্রি-কে White-Box Model বলা হয়। এটি একটি স্পষ্ট ভিজ্যুয়াল ফ্লোচার্ট বা "IF-ELSE" নিয়মের সেট তৈরি করে সিদ্ধান্ত নেয়। কোনো ব্যাংকের ম্যানেজার বা ডাক্তার খুব সহজেই গাছের ডালপালা দেখে বুঝতে পারেন যে কেন একজন কাস্টমারকে লোন দেওয়া হলো না কিংবা কেন একজন রোগীকে উচ্চ ঝুঁকিপূর্ণ বলা হলো।

৪. ক্যাটাগরিক্যাল ফিচারের স্বাভাবিক হ্যান্ডলিং (Natural Handling of Categorical Data)

- **অন্যান্য মডেলের সীমাবদ্ধতা:** লিনিয়ার/লজিস্টিক রিগ্রেশন এবং KNN-এ ক্যাটাগরিক্যাল ডেটা (যেমন: Sex বা Embarked) সরাসরি পাস করলে মডেল এরর দেয়। এদের জন্য ওয়ান-হট এনকোডিং (One-Hot Encoding) করা বাধ্যতামূলক, যা হাই-কার্ডিনালিটি ডেটার ক্ষেত্রে কলামের সংখ্যা বহুগুণ বাড়িয়ে দেয় (Column Explosion ঘটে)।
- **ডিসিশন ট্রি-এর সুবিধা:** ডিসিশন ট্রি ক্যাটাগরিক্যাল এবং নিউমেরিক্যাল উভয় ধরনের ডেটা একসাথে খুব স্বাভাবিকভাবে সামলাতে পারে। এটি সরাসরি টেক্সট বা ক্যাটাগরির ওপর ভিত্তি করে ডালপালা মেলতে পারে (যেমন: Sex == 'male' হলে বামে যাও, female হলে ডানে যাও), ফলে অতিরিক্ত ওয়ান-হট এনকোডিংয়ের ওপর এর নির্ভরশীলতা অনেক কম।

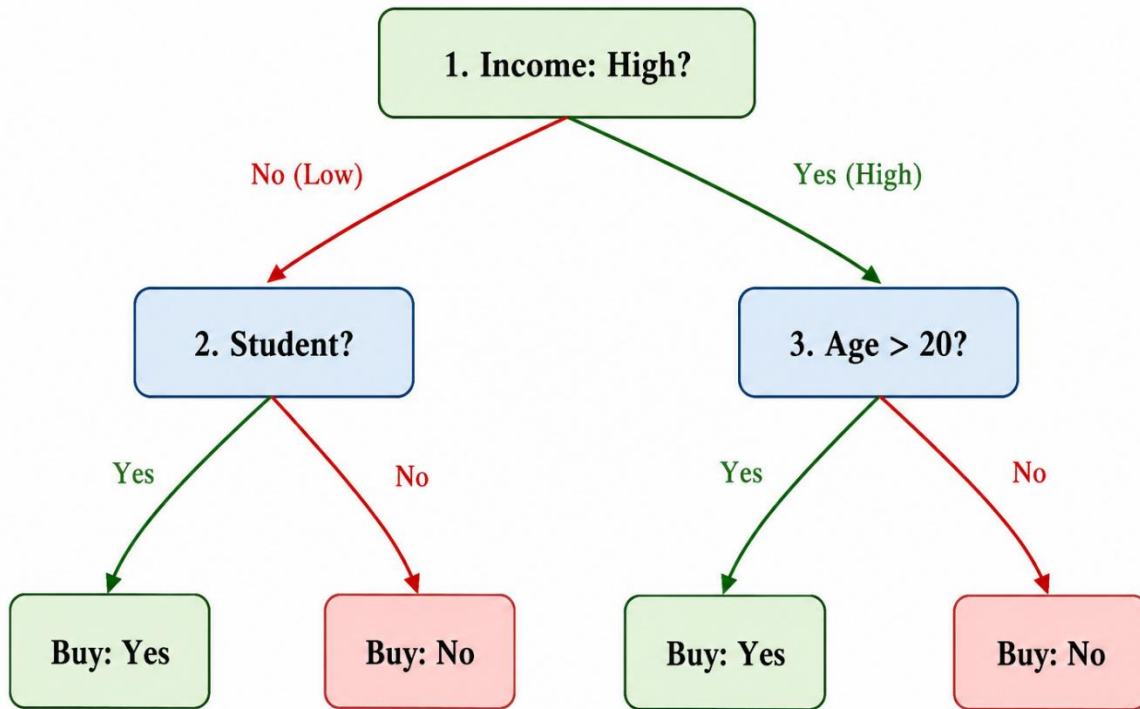
Example over a small dataset:

Customer	Age	Income	Student	Buy (Target)
C1	22	Low	Yes	Yes
C2	45	Low	No	No
C3	28	High	No	Yes
C4	35	High	Yes	Yes
C5	19	High	No	No

ডিসিশন ট্রি-এর লজিক্যাল গঠন (The Tree Structure)

আমরা যদি এই ডেটার ওপর একটি ডিসিশন ট্রি ট্রেন করি, তবে সে ব্যাকঅ্যান্ডে ইনফরমেশন গেইন (Information Gain) হিসাব করে নিচের মতো একটি গাছের কাঠামো তৈরি করবে:

1. **রুট নোড (Income):** আয় কি High?
 - যদি **না (Low)** হয় → সে ২ নম্বর প্রশ্ন বা সাব-নোডে যাবে।
 - যদি **হ্যাঁ (High)** হয় → সে ৩ নম্বর প্রশ্ন বা সাব-নোডে যাবে।
2. **সাব-নোড ১ (Student):** সে কি একজন শিক্ষার্থী?
 - হ্যাঁ → **Buy: Yes**
 - না → **Buy: No**
3. **সাব-নোড ২ (Age):** বয়স কি ২০-এর ওপরে?
 - হ্যাঁ → **Buy: Yes**
 - না → **Buy: No**



ডিসিশন ট্রির ব্যাকগ্রাউন্ডে গাছ তৈরির গাণিতিক প্রক্রিয়া

১. মূল লক্ষ্য: বিশুদ্ধতা (Purity) অর্জন

ডিসিশন ট্রি (Decision Tree) অ্যালগরিদমের প্রধান উদ্দেশ্য হলো ডেটাসেটকে এমনভাবে ধারাবাহিকভাবে বিভক্ত করা, যাতে প্রতিটি চূড়ান্ত নোডে (Leaf Node) একই শ্রেণির (Class) ডেটা থাকে। অর্থাৎ, প্রতিটি বিভাজনের মাধ্যমে ডেটার বিশুদ্ধতা (Impurity) কমিয়ে আনা এবং যতটা সম্ভব বিশুদ্ধ (Pure) নোড তৈরি করাই এই অ্যালগরিদমের মূল লক্ষ্য।

অবিশুদ্ধ নোড (Impure Node)

যদি কোনো নোডে একাধিক শ্রেণির ডেটা একসঙ্গে উপস্থিত থাকে, তবে সেটিকে অবিশুদ্ধ নোড বলা হয়। উদাহরণস্বরূপ, একটি নোডে যদি সমান সংখ্যক **Yes** এবং **No** শ্রেণির ডেটা থাকে, তবে সেই নোডে অনিশ্চয়তা সর্বাধিক থাকে। ফলে শুধুমাত্র সেই নোডের তথ্যের ভিত্তিতে সঠিক সিদ্ধান্ত গ্রহণ করা সম্ভব হয় না।

বিশুদ্ধ নোড (Pure Node)

অন্যদিকে, যদি কোনো নোডে কেবলমাত্র একটি শ্রেণির ডেটা উপস্থিত থাকে, যেমন সবগুলোই **Yes** অথবা সবগুলোই **No**, তাহলে সেই নোডকে সম্পূর্ণ বিশুদ্ধ (Pure Node) বলা হয়। এ ধরনের

নোডে আর কোনো বিভাজনের প্রয়োজন হয় না এবং এটিই ডিসিশন ট্রির চূড়ান্ত পাতা বা **Leaf Node** হিসেবে বিবেচিত হয়।

২. অবিশুদ্ধতা পরিমাপের গাণিতিক পদ্ধতি (Mathematical Metrics)

ডিসিশন ট্রি কোন ফিচারের ভিত্তিতে ডেটা ভাগ করবে তা নির্ধারণ করার জন্য বিভিন্ন গাণিতিক পরিমাপক (Metric) ব্যবহার করে। এই পরিমাপকগুলোর মাধ্যমে প্রতিটি সম্ভাব্য বিভাজনের গুণগত মান মূল্যায়ন করা হয়। সর্বাধিক ব্যবহৃত তিনটি পদ্ধতি হলো:

- Entropy
- Information Gain
- Gini Impurity

২.১ Entropy (এনট্রপি): বিশৃঙ্খলার পরিমাপ

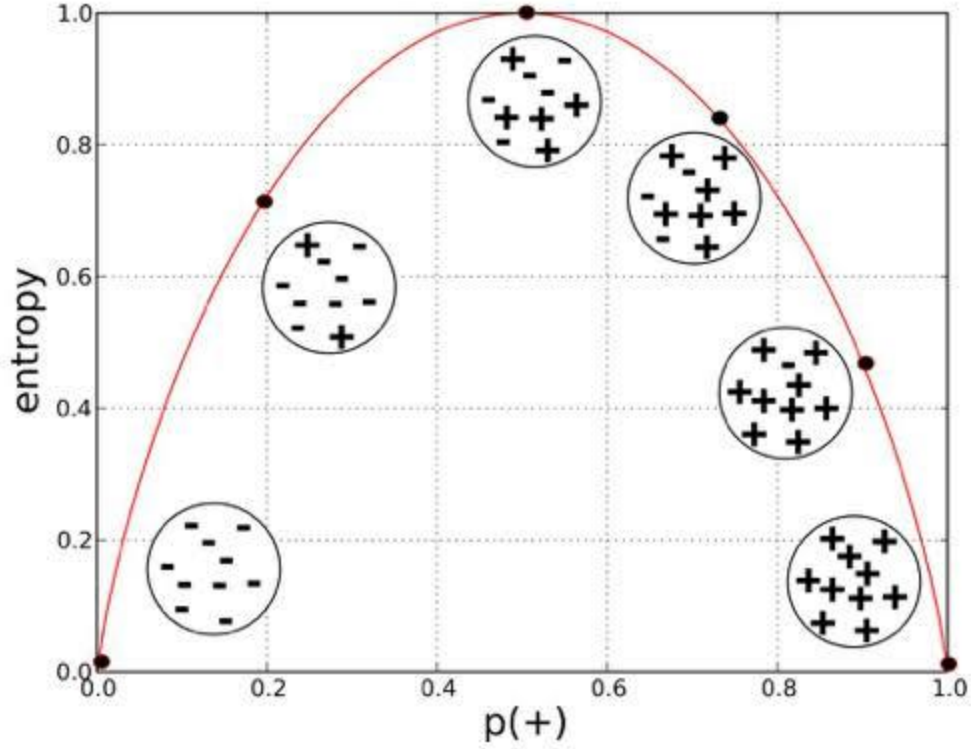
Entropy হলো কোনো নোডের অনিশ্চয়তা বা বিশৃঙ্খলার (Disorder) পরিমাপ। একটি নোডে বিভিন্ন শ্রেণির ডেটা যত বেশি মিশ্রিত থাকবে, এনট্রপির মান তত বেশি হবে। বিপরীতভাবে, নোড যত বেশি বিশুদ্ধ হবে, এনট্রপির মান তত কম হবে।

এর গাণিতিক সূত্র হলো,

$$Entropy(S) = - \sum_{i=1}^c p_i \log_2(p_i)$$

এখানে,

- S হলো বর্তমান নোডের ডেটাসেট।
- c হলো মোট শ্রেণির সংখ্যা।
- p_i হলো i -তম শ্রেণির ডেটার অনুপাত (Probability)।



এনট্রপির মানের ব্যাখ্যা

- যদি নোডে ৫০% Yes এবং ৫০% No থাকে, তাহলে অনিশ্চয়তা সর্বাধিক হয় এবং Entropy-এর মান হয় 1.0।
- যদি নোডে ১০০% একই শ্রেণির ডেটা থাকে, তাহলে কোনো অনিশ্চয়তা থাকে না এবং Entropy-এর মান হয় 0।

অতএব, ডিসিশন ট্রির লক্ষ্য হলো প্রতিটি বিভাজনের মাধ্যমে Entropy-এর মান ক্রমান্বয়ে কমিয়ে আনা।

২.২ Information Gain (ইনফরমেশন গেইন): তথ্য অর্জনের পরিমাণ

Entropy শুধুমাত্র বর্তমান নোডের বিশৃঙ্খলা পরিমাপ করে। কিন্তু কোন ফিচার ব্যবহার করলে সেই বিশৃঙ্খলা সবচেয়ে বেশি কমবে, তা নির্ধারণ করার জন্য Information Gain ব্যবহার করা হয়।

প্রথমে প্রতিটি সম্ভাব্য ফিচার ব্যবহার করে ডেটাকে বিভক্ত করা হয়। এরপর বিভাজনের আগে ও পরে Entropy-এর পরিবর্তন নির্ণয় করা হয়। এই পরিবর্তনের পরিমাণই Information Gain নামে পরিচিত।

এর গাণিতিক সূত্র হলো,

$$Information\ Gain = Entropy(Parent) - Weighted\ Average\ Entropy(Children)$$

এখানে,

- **Parent Entropy** হলো বিভাজনের পূর্বের নোডের Entropy।
- **Children Entropy** হলো বিভাজনের পর সৃষ্ট সকল সাব-নোডের ওজনযুক্ত (Weighted) গড় Entropy।

Information Gain-এর সিদ্ধান্ত

ডিসিশন ট্রি প্রতিটি ফিচারের জন্য Information Gain গণনা করে।

যে ফিচারের Information Gain সর্বাধিক হয়, অর্থাৎ যে ফিচারটি ডেটার বিশৃঙ্খলা সবচেয়ে বেশি কমাতে সক্ষম হয়, সেই ফিচারকেই পরবর্তী Decision Node অথবা Root Node হিসেবে নির্বাচন করা হয়।

২.৩ Gini Impurity (জিনি ইম্পিউরিটি): দ্রুত বিশুদ্ধতা পরিমাপ

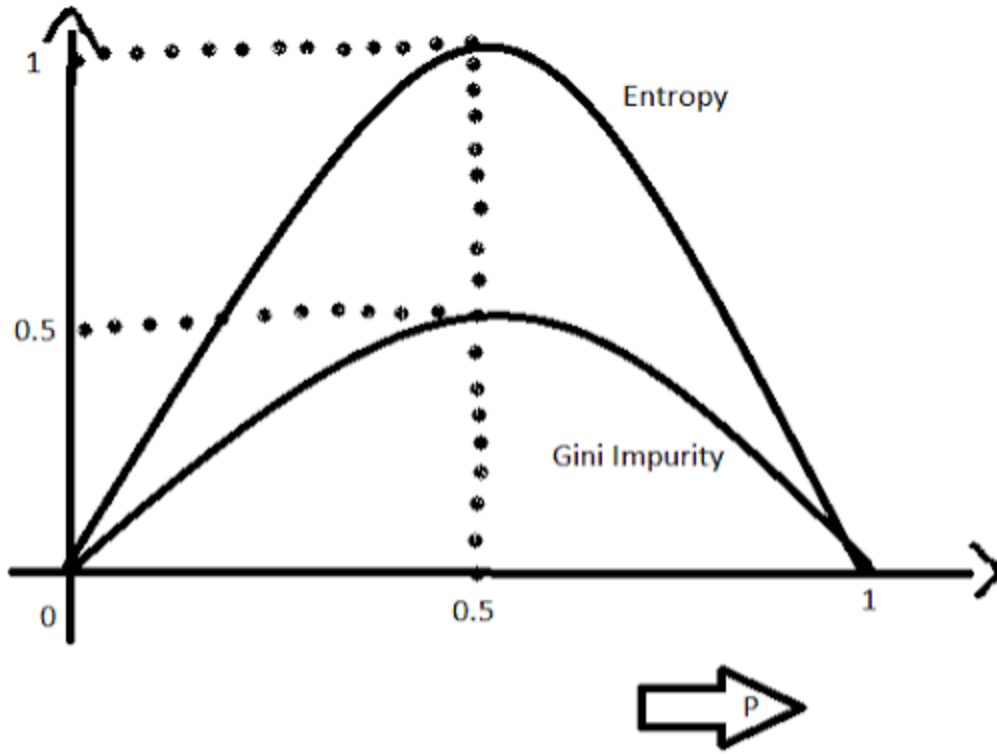
Entropy গণনার সময় লগারিদমিক ($\log_2$) অপারেশন ব্যবহৃত হয়, যা তুলনামূলকভাবে বেশি কম্পিউটেশনাল সময় গ্রহণ করে। এই কারণে বাস্তব প্রয়োগে অনেক লাইব্রেরি, বিশেষ করে **Scikit-Learn**, ডিফল্টভাবে Entropy-এর পরিবর্তে **Gini Impurity** ব্যবহার করে।

Gini Impurity-এর সূত্র হলো,

$$Gini = 1 - \sum_{i=1}^c (p_i)^2$$

এখানে,

- p_i হলো প্রতিটি শ্রেণির Probability।



$$E(S) = \sum_{i=1}^c -p_i \log_2 p_i$$

$$Gini(E) = 1 - \sum_{j=1}^c p_j^2$$

Gini Impurity-এর মানের ব্যাখ্যা

- সম্পূর্ণ বিশুদ্ধ নোডের Gini মান 0।
- দুটি শ্রেণির সমান অনুপাত থাকলে Gini-এর মান 0.5, যা সর্বোচ্চ অবিশুদ্ধতার নির্দেশক।

ডিসিশন ট্রি সর্বদা এমন বিভাজন নির্বাচন করে যাতে সাব-নোডগুলোর Gini Impurity যত সম্ভব কম হয়।

৩. ডিসিশন ট্রির ব্যাকএন্ড ওয়ার্কফ্লো

model.fit() মেথড কল করার পর ডিসিশন ট্রি একটি ধারাবাহিক গাণিতিক প্রক্রিয়ার মাধ্যমে নিজস্ব কাঠামো তৈরি করে। এই প্রক্রিয়াটি নিম্নলিখিত ধাপগুলো অনুসরণ করে।

ধাপ ১: Parent Node-এর বিশুদ্ধতা নির্ণয়

প্রথমে সম্পূর্ণ Training Dataset-কে একটি Parent Node হিসেবে বিবেচনা করা হয়। এরপর ওই নোডের Entropy অথবা Gini Impurity গণনা করা হয়।

ধাপ ২: সমস্ত ফিচার ও সম্ভাব্য Threshold পরীক্ষা করা

এরপর প্রতিটি Feature একে একে বিশ্লেষণ করা হয়।

যদি Feature সংখ্যাসূচক (Numerical) হয়, তাহলে সম্ভাব্য বিভিন্ন Threshold পরীক্ষা করা হয়।

যেমন,

- Age > 20
- Age > 25
- Age > 30

আবার যদি Feature শ্রেণিভিত্তিক (Categorical) হয়, তাহলে প্রতিটি Category অনুযায়ী সম্ভাব্য বিভাজন পরীক্ষা করা হয়।

প্রতিটি ক্ষেত্রেই ডেটাকে সাময়িকভাবে (Temporarily) দুই ভাগে বিভক্ত করা হয়।

ধাপ ৩: প্রতিটি বিভাজনের গুণগত মান মূল্যায়ন

প্রতিটি সম্ভাব্য বিভাজনের পরে সৃষ্ট সাব-নোডগুলোর Entropy অথবা Gini Impurity পুনরায় গণনা করা হয়।

এরপর Information Gain নির্ণয় করা হয় এবং দেখা হয়, কোন বিভাজন Parent Node-এর তুলনায় সবচেয়ে বেশি বিশুদ্ধতা কমাতে সক্ষম হয়েছে।

ধাপ ৪: সর্বোত্তম বিভাজন নির্বাচন

যে Feature এবং Threshold সর্বোচ্চ Information Gain প্রদান করে অথবা সর্বনিম্ন Gini Impurity সৃষ্টি করে, সেটিকে চূড়ান্ত Split হিসেবে নির্বাচন করা হয়।

এরপর Parent Node-কে স্থায়ীভাবে (Permanently) সেই নিয়ম অনুযায়ী দুই বা ততোধিক সাব-নোডে বিভক্ত করা হয়।

ধাপ ৫: পুনরাবৃত্তিমূলক (Recursive) প্রক্রিয়া

একবার প্রথম বিভাজন সম্পন্ন হলে একই প্রক্রিয়া প্রতিটি নতুন সাব-নোডের জন্য পুনরায় চালানো হয়।

অর্থাৎ,

- পুনরায় Entropy বা Gini গণনা করা হয়।
- সম্ভাব্য সকল Feature পরীক্ষা করা হয়।
- সর্বোত্তম Split নির্বাচন করা হয়।
- নতুন Sub-node তৈরি করা হয়।

এই Recursive প্রক্রিয়া বারবার চলতে থাকে যতক্ষণ না অ্যালগরিদমের থামার কোনো শর্ত পূরণ হয়।

৪. ডিসিশন ট্রি কখন বৃদ্ধি বন্ধ করে? (Stopping Criteria)

যদি ডিসিশন ট্রিকে কোনো সীমাবদ্ধতা ছাড়া প্রশিক্ষণ দেওয়া হয়, তাহলে এটি প্রতিটি Leaf Node সম্পূর্ণ বিশুদ্ধ না হওয়া পর্যন্ত বিভাজন চালিয়ে যেতে পারে। এর ফলে মডেলটি শুধুমাত্র প্রকৃত প্যাটার্ন নয়, বরং Training Dataset-এর Noise এবং ব্যতিক্রমী তথ্যও শিখে ফেলে। এই সমস্যাকে **Overfitting** বলা হয়।

Overfitting প্রতিরোধ করার জন্য বিভিন্ন Hyperparameter ব্যবহার করা হয়।

max_depth

গাছ সর্বোচ্চ কত স্তর পর্যন্ত বৃদ্ধি পাবে তা নির্ধারণ করে।

উদাহরণস্বরূপ,

`max_depth = 3`

দেওয়া হলে গাছ তিন স্তরের বেশি বৃদ্ধি পাবে না।

min_samples_split

কোনো নোডকে নতুনভাবে বিভক্ত করার জন্য সেখানে ন্যূনতম কতটি Sample থাকতে হবে, তা নির্ধারণ করে।

min_samples_leaf

প্রতিটি Leaf Node-এ সর্বনিম্ন কতটি Sample থাকতে হবে, তা নির্ধারণ করে। এর ফলে খুব ছোট বা অপ্রয়োজনীয় Leaf তৈরি হওয়ার সম্ভাবনা কমে যায়।

Example of Decision Tree over a Categorical Dataset:

ধরা যাক, একটি দোকান জানতে চায় কোনো গ্রাহক পণ্য কিনবে কিনা। এজন্য নিচের ডেটাসেটটি সংগ্রহ করা হয়েছে।

ID	Income	Student	Buy
1	High	Yes	Yes
2	High	Yes	Yes
3	High	No	No
4	High	No	No
5	Low	Yes	Yes
6	Low	Yes	Yes
7	Low	No	No
8	Low	No	Yes

এখানে,

- **Features**
 - Income
 - Student
- **Target**
 - Buy

ধাপ ১: Parent Node-এর Entropy গণনা

প্রথমে সম্পূর্ণ ডেটাসেটকে Parent Node ধরা হবে।

মোট ডেটা = 8

Buy = Yes = 5

Buy = No = 3

অতএব,

$$P(Yes) = \frac{5}{8} = 0.625$$
$$P(No) = \frac{3}{8} = 0.375$$

Entropy-এর সূত্র,

$$Entropy(S) = -\sum p_i \log_2(p_i)$$

সুতরাং,

$$Entropy(S) = -\left(\frac{5}{8} \log_2 \frac{5}{8} + \frac{3}{8} \log_2 \frac{3}{8}\right)$$

গণনা করলে,

$$\log_2(0.625) = -0.678$$
$$\log_2(0.375) = -1.415$$

অতএব,

$$Entropy = -(0.625 \times -0.678 + 0.375 \times -1.415)$$
$$= 0.954$$

অতএব,

$$Entropy(Parent) = 0.954$$

ধাপ ২: Feature = Income পরীক্ষা করা

Income-এর দুটি মান আছে।

- High
- Low

Income = High

Buy
Yes
Yes
No
No

Yes = 2

No = 2

অতএব,

$$\begin{aligned} Entropy(High) &= -\left(\frac{2}{4}\log_2\frac{2}{4} + \frac{2}{4}\log_2\frac{2}{4}\right) \\ &= 1.0 \end{aligned}$$

Income = Low

Buy
Yes
Yes
No
Yes

Yes = 3

No = 1

$$\begin{aligned} Entropy(Low) &= -\left(\frac{3}{4}\log_2\frac{3}{4} + \frac{1}{4}\log_2\frac{1}{4}\right) \\ &= 0.811 \end{aligned}$$

Weighted Entropy

$$\begin{aligned} &= \frac{4}{8} \times 1 + \frac{4}{8} \times 0.811 \\ &= 0.9055 \end{aligned}$$

Information Gain (Income)

$$\begin{aligned} IG &= 0.954 - 0.9055 \\ &= 0.0485 \end{aligned}$$

ধাপ ৩: Feature = Student পরীক্ষা করা

Student-এর দুটি মান।

- Yes
- No

Student = Yes

Buy
Yes
Yes
Yes
Yes

Yes = 4

No = 0

$$Entropy = 0$$

Student = No

Buy
No
No
No
Yes

Yes = 1

No = 3

$$Entropy = - \left(\frac{1}{4} \log_2 \frac{1}{4} + \frac{3}{4} \log_2 \frac{3}{4} \right)$$

$$= 0.811$$

Weighted Entropy

$$= \frac{4}{8} \times 0 + \frac{4}{8} \times 0.811$$

$$= 0.4055$$

Information Gain

$$IG = 0.954 - 0.4055$$

$$= 0.5485$$

ধাপ ৪: Information Gain তুলনা

Feature Information Gain

Income 0.0485

Student 0.5485

স্পষ্টভাবে দেখা যাচ্ছে,

$$0.5485 > 0.0485$$

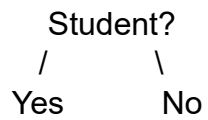
অতএব,

Student Feature সর্বাধিক Information Gain প্রদান করেছে।

সুতরাং,

Student হবে Root Node।

ধাপ ৫: প্রথম Split



Student = Yes

ডেটা

Buy
Yes
Yes
Yes
Yes

সবগুলো একই ক্লাস।

Entropy = 0

এটি একটি **Leaf Node**।

Student = Yes



Buy = Yes

Student = No

ডেটা

Income	Buy
High	No
High	No
Low	No
Low	Yes

এখানে

Yes = 1

No = 3

এখন এই নোডটি বিশুদ্ধ নয়।

অতএব, আবার Feature নির্বাচন করতে হবে।

এখানে অবশিষ্ট Feature হলো

Income

ধাপ ৬: Student = No নোডে Income দিয়ে Split

Income = High

Buy
No
No

Entropy = 0

Income = Low

Buy
No
Yes

Entropy

$$= -\left(\frac{1}{2}\log_2\frac{1}{2} + \frac{1}{2}\log_2\frac{1}{2}\right) = 1$$

Weighted Entropy

$$= \frac{2}{4} \times 0 + \frac{2}{4} \times 1 = 0.5$$

বর্তমান Parent Entropy

$$0.811$$

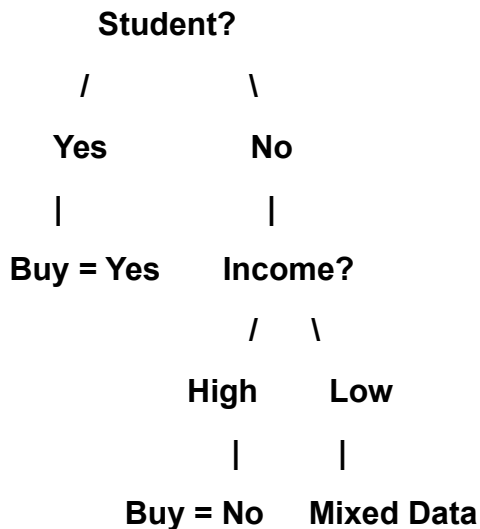
Information Gain

$$= 0.811 - 0.5 = 0.311$$

অতএব,

Income দিয়ে আবার Split করা হবে।

Final Decision Tree



এখানে Income = Low শাখায় এখনও দুটি ভিন্ন ক্লাস (Yes এবং No) রয়েছে। যেহেতু আর কোনো Feature অবশিষ্ট নেই, তাই বাস্তব অ্যালগরিদম সাধারণত Majority Class নির্বাচন করে অথবা নির্ধারিত Stopping Criteria অনুযায়ী সেই নোডকে Leaf Node বানিয়ে দেয়।

Dataset

ID	Income	Student	Buy
1	High	Yes	Yes
2	High	Yes	Yes
3	High	No	No
4	High	No	No
5	Low	Yes	Yes
6	Low	Yes	Yes
7	Low	No	No
8	Low	No	Yes

ধাপ ১: Parent Gini

মোট ৮টি ডেটা

- Yes = 5
- No = 3

$$\begin{aligned}Gini &= 1 - (P_{Yes}^2 + P_{No}^2) \\&= 1 - \left(\left(\frac{5}{8} \right)^2 + \left(\frac{3}{8} \right)^2 \right) \\&= 1 - (0.3906 + 0.1406) \\&= \boxed{0.4688}\end{aligned}$$

ধাপ ২: Income Feature

Income = High

Yes = 2, No = 2

$$Gini = 1 - (0.5^2 + 0.5^2) = 0.5$$

Income = Low

Yes = 3, No = 1

$$Gini = 1 - \left(\left(\frac{3}{4} \right)^2 + \left(\frac{1}{4} \right)^2 \right) = 0.375$$

Weighted Gini

$$= \frac{4}{8}(0.5) + \frac{4}{8}(0.375) = 0.4375$$

Gini Gain (Reduction)

$$0.4688 - 0.4375 = \boxed{0.0313}$$

ধাপ ৩: Student Feature

Student = Yes

Yes = 4, No = 0

$$Gini = 0$$

Student = No

Yes = 1, No = 3

$$Gini = 1 - \left(\left(\frac{1}{4} \right)^2 + \left(\frac{3}{4} \right)^2 \right) = 0.375$$

Weighted Gini

$$= \frac{4}{8}(0) + \frac{4}{8}(0.375) = 0.1875$$

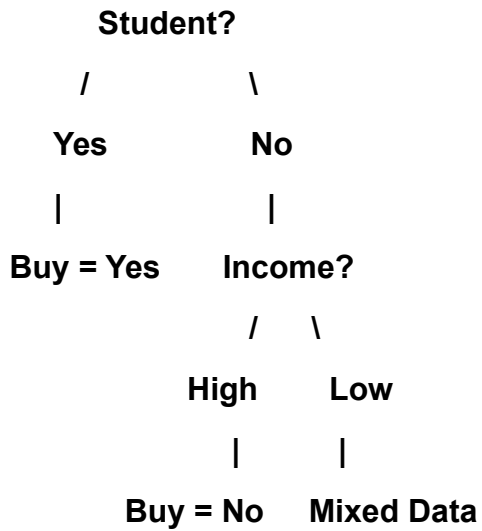
Gini Gain (Reduction)

$$0.4688 - 0.1875 = \boxed{0.2813}$$

ধাপ ৪: সিদ্ধান্ত

Feature	Weighted Gini	Gini Reduction
Income	0.4375	0.0313
Student	0.1875	0.2813

যেহেতু Student Feature-এর Weighted Gini সবচেয়ে কম (0.1875) এবং Gini Reduction সবচেয়ে বেশি (0.2813), তাই Student-কেই Root Node হিসেবে নির্বাচন করা হবে।



Classification-এ আমরা Entropy বা Gini ব্যবহার করি, কারণ সেখানে লক্ষ্য থাকে ক্লাস (Yes/No) আলাদা করা। কিন্তু Regression Tree-তে লক্ষ্য থাকে Continuous Value (যেমন: Price, Salary, House Price) পূর্বাভাস দেওয়া। তাই এখানে Variance বা Mean Squared Error (MSE) ব্যবহার করা হয়।

Dataset

ধরা যাক, আমরা পড়ার সময় (Study Hours) দেখে পরীক্ষার নম্বর (Marks) পূর্বাভাস দিতে চাই।

ID	Study Hours	Marks
1	1	40
2	2	45
3	3	50
4	4	80
5	5	85
6	6	90

Target = **Marks**

ধাপ ১: সম্ভাব্য Split নির্বাচন

ধরি,

Split = Study Hours $\leq$ 3

তাহলে Dataset দুই ভাগে বিভক্ত হবে।

Left Node

Hours	Marks
1	40
2	45
3	50

Mean

$$= \frac{40 + 45 + 50}{3} = 45$$

Variance

$$\begin{aligned} &= \frac{(40 - 45)^2 + (45 - 45)^2 + (50 - 45)^2}{3} \\ &= \frac{25 + 0 + 25}{3} = 16.67 \end{aligned}$$

Right Node

Hours	Marks
4	80
5	85
6	90

Mean

$$= \frac{80 + 85 + 90}{3} = 85$$

Variance

$$\begin{aligned} &= \frac{(80 - 85)^2 + (85 - 85)^2 + (90 - 85)^2}{3} \\ &= \frac{25 + 0 + 25}{3} = 16.67 \end{aligned}$$

ধাপ ২: Weighted Variance

Left-এ ৩টি Sample

Right-এ ৩টি Sample

$$Weighted\ Variance = \frac{3}{6}(16.67) + \frac{3}{6}(16.67) = 16.67$$

ধাপ ৩: অন্য একটি Split পরীক্ষা

ধরি,

Study Hours ≤ 4

Left Node

Marks

40, 45, 50, 80

Mean

$$= \frac{40 + 45 + 50 + 80}{4} = 53.75$$

Variance

$$= 242.19$$

Right Node

Marks

85, 90

Mean

$$= 87.5$$

Variance

$$= 6.25$$

Weighted Variance

$$\begin{aligned} &= \frac{4}{6}(242.19) + \frac{2}{6}(6.25) \\ &= 163.54 \end{aligned}$$

ধাপ ৪: সিদ্ধান্ত

Split Weighted Variance

Study Hours ≤ 3 **16.67**

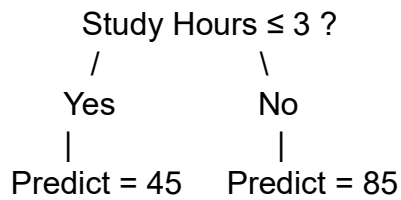
Study Hours ≤ 4 163.54

যেহেতু

16.67 < 163.54

তাই **Study Hours ≤ 3** হবে **Best Split**।

Decision Tree



Prediction

Study Hours	Prediction
2	45
3	45
5	85
6	85

এখানে প্রতিটি Leaf Node কোনো Class Label সংরক্ষণ করে না। বরং সেই Leaf-এর সকল Target Value-এর Mean (Average) সংরক্ষণ করে।

ডিসিশন ট্রি ক্লাসিফায়ার এবং ডিসিশন ট্রি রিগ্রেসরের মধ্যে পার্থক্য

বিষয়	Decision Tree Classifier	Decision Tree Regressor
পূর্বাভাসের ধরন	শ্রেণিভিত্তিক (Categorical) মান পূর্বাভাস দেয়	ধারাবাহিক সংখ্যাসূচক (Continuous Numerical) মান পূর্বাভাস দেয়
Target Variable-এর ধরন	Categorical (যেমন: Yes/No, Spam/Not Spam)	Numerical (যেমন: Price, Salary, Temperature)
মূল উদ্দেশ্য	ডেটাকে বিভিন্ন শ্রেণিতে (Class) ভাগ করা	কোনো সংখ্যাসূচক মান (Value) পূর্বাভাস দেওয়া
Split করার মানদণ্ড	Entropy, Information Gain অথবা Gini Impurity	Variance Reduction অথবা Mean Squared Error (MSE) Reduction
Impurity Measure	বিভিন্ন Class কতটা মিশ্রিত আছে তা পরিমাপ করে	সংখ্যাগুলোর বিচ্ছুরণ (Variation) কতটা তা পরিমাপ করে
Leaf Node-এর আউটপুট	একটি Class Label (যেমন: Yes বা No) সংরক্ষণ করে	Target Value-এর গড় (Mean/Average) সংরক্ষণ করে
সেরা Split নির্বাচন	সর্বোচ্চ Information Gain অথবা সর্বনিম্ন Gini বিশিষ্ট Split নির্বাচন করে	সর্বনিম্ন Variance অথবা সর্বনিম্ন MSE বিশিষ্ট Split নির্বাচন করে
Decision Rule	ভিন্ন ভিন্ন Class আলাদা করার চেষ্টা করে	কাছাকাছি সংখ্যাসূচক মানগুলোকে একই দলে রাখার চেষ্টা করে
আউটপুটের উদাহরণ	Buy = Yes	House Price = 25,00,000 টাকা
Evaluation Metrics	Accuracy, Precision, Recall, F1-score, ROC-AUC	MAE, MSE, RMSE, R ² Score
Scikit-Learn Class	DecisionTreeClassifier	DecisionTreeRegressor
সাধারণ ব্যবহার	Spam Detection, Disease Classification, Customer Churn Prediction	House Price Prediction, Salary Prediction, Sales Forecasting

Scikit-Learn-এর DecisionTreeClassifier Hyperparameter:

১. মোস্ট ইম্পোর্টেন্ট প্যারামিটারসমূহ (The Heavy Hitters)

গাছের সাইজ নিয়ন্ত্রণ করতে, ওভারফিটিং রোধ করতে এবং ইন্টারভিউতে সবচেয়ে বেশি ফোকাস করা হয় এই প্যারামিটারগুলোর ওপর।

max_depth: None (default)

- **কাজ কী:** এটি আপনার তৈরি হওয়া গাছের সর্বোচ্চ গভীরতা বা লেভেল (Level) নির্ধারণ করে।
- **কখন 'None' (Default) রাখব:** ডেটাসেট যদি খুব ছোট হয় এবং আপনি যদি দেখতে চান গাছটি সর্বোচ্চ কতদূর পর্যন্ত যেতে পারে, তখন এটি ডিফল্ট রাখা যায়। None থাকলে গাছটি বাড়তেই থাকবে যতক্ষণ না সব পাতা ১০০% বিশুদ্ধ হচ্ছে বা min_samples_split-এর চেয়ে কম স্যাম্পল থাকছে।
- **ঝুঁকি কী:** এটি ডিফল্ট রেখে দিলে গাছটি একদম শেষ সীমানা পর্যন্ত মুখস্থ করতে থাকবে এবং মারাত্মক **Overfitting (High Variance)** ঘটবে।
- **অন্যান্য অপশন ও কখন ব্যবহার করব:** এখানে যেকোনো পূর্ণসংখ্যা (যেমন: 3, 5, 10) দেওয়া যায়। বাস্তব প্রজেক্টে ওভারফিটিং বন্ধ করার সবচেয়ে প্রথম হাতিয়ার হলো এটি। সবসময় ছোট মান (যেমন: max_depth=3 বা 5) দিয়ে শুরু করা ভালো, যাতে মডেলটি জেনারেলাইজড বা সুষম থাকে।

criterion: "gini" (default)

- **কাজ কী:** প্রতিটা ধাপে ডালপালা মেলানোর জন্য বা ডেটা ভাগ (Split) করার কোয়ালিটি মাপার জন্য ব্যাকএন্ডে কোন গাণিতিক সূত্র ব্যবহার করা হবে, তা এটি নির্ধারণ করে।
- **কখন 'gini' (Default) ব্যবহার করব:** বাস্তব প্রজেক্টে দ্রুত ট্রেনিং শেষ করার জন্য এটি ব্যবহার করা হয়। জিনি ইম্পিউরিটির গাণিতিক সূত্রে কোনো লগারিদম ($\log_2$) হিসাব করতে হয় না, তাই এটি কম্পিউটেশনালি অত্যন্ত ফাস্ট এবং দ্রুত রেজাল্ট দেয়।
- **অন্যান্য অপশন:** "entropy" অথবা "log_loss" (উভয়ই শ্যানন ইনফরমেশন গেইনের ওপর ভিত্তি করে কাজ করে)।
- **কখন এবং কেন 'entropy' ব্যবহার করব:** যখন আপনি অত্যন্ত সূক্ষ্ম এবং সুষমভাবে ডেটা ভাগ করতে চান। এনট্রপি নোডের ভেতরের বিশৃঙ্খলাকে আরও নিখুঁতভাবে মাপে, তবে এর সূত্রে $\log_2$ থাকায় কম্পিউটারের প্রসেসিং টাইম সামান্য বেশি লাগে। বড় ডেটাসেটে জিনি আর এনট্রপির এক্যুরেসির মধ্যে খুব বেশি তফাৎ পাওয়া যায় না, তাই ডিফল্ট 'gini' রাখাই স্ট্যান্ডার্ড।

min_samples_split: 2 (default)

- **কাজ কী:** মাঝখানের কোনো একটি নোডকে (Internal Node) ভেঙে নতুন ডালপালা তৈরি করার জন্য ওই নোডে নূন্যতম কয়টি ডেটা স্যাম্পল থাকতে হবে, তা এটি ঠিক করে।
- **কখন '2' (Default) রাখব:** যখন আপনি গাছের স্প্লিটিং প্রক্রিয়ায় কোনো বাধা দিতে চান না। তবে ২ রাখা মানে নোডে মাত্র ২টি স্যাম্পল থাকলেও গাছ সেটিকে ভেঙে আরও নিচে নামবে, যা ওভারফিটিংয়ের ঝুঁকি বাড়ায়।
- **অন্যান্য অপশন ও কখন ব্যবহার করব:** এখানে পূর্ণসংখ্যা (যেমন: 10, 50) অথবা ফ্লোট ভগ্নাংশ (যেমন: 0.05 অর্থাৎ মোট ডেটার ৫%) দেওয়া যায়।
- **কেন পরিবর্তন করব:** ওভারফিটিং কমানোর জন্য। যদি আপনি min_samples_split=50 দেন, তবে কোনো নোডে ৫০টির কম ডেটা থাকলে গাছ সেটিকে আর ভাঙবে না, সরাসরি ওখানেই থামিয়ে দেবে।

min_samples_leaf: 1 (default)

- **কাজ কী:** একটি চূড়ান্ত পাতা বা লিফ নোডে (Leaf Node) নূন্যতম কয়টি স্যাম্পল থাকতেই হবে, তা এটি ফিক্স করে।
- **কখন '1' (Default) রাখব:** ডিফল্ট ১ রাখা মানে গাছের শেষ পাতায় মাত্র ১টি স্যাম্পল থাকলেও মডেল খুশি। এটি মডেলকে চরম মুখস্থবিদ্যার দিকে ঠেলে দেয়।
- **অন্যান্য অপশন ও কখন ব্যবহার করব:** এখানেও পূর্ণসংখ্যা বা ফ্লোট ভগ্নাংশ দেওয়া যায় (যেমন: 5, 10 বা 0.02)।
- **কেন পরিবর্তন করব:** এটি মডেলকে স্মুথ (Smooth) করতে সাহায্য করে। যদি min_samples_leaf=10 দেওয়া হয়, তবে কোনো স্প্লিট করার কারণে যদি বাম বা ডান ডালের কোনো একটিতে ১০টির কম স্যাম্পল চলে যাওয়ার পরিস্থিতি তৈরি হয়, তবে ডিসিশন ট্রি ওই স্প্লিটটি বাতিল করে দেবে। এটি ওভারফিটিং কমানোর অত্যন্ত শক্তিশালী একটি প্যারামিটার।

২. মিডিয়াম ইম্পর্টেন্ট প্যারামিটারসমূহ (Advanced Tuning)

মডেলের পারফরম্যান্স আরও নিখুঁত করতে এবং ডেটার বিশেষ পরিস্থিতি সামলাতে এগুলো ব্যবহার করা হয়।

class_weight: None (default)

- **কাজ কী:** ইমব্যালেন্সড ডেটাসেটের (Imbalanced Dataset) ক্ষেত্রে মাইনোরিটি ক্লাসকে বা ছোট ক্লাসকে অতিরিক্ত গুরুত্ব বা ওজন দেওয়ার জন্য এটি ব্যবহৃত হয়।
- **কখন 'None' (Default) রাখব:** যখন আপনার ডেটাসেটে সব ক্লাসের অনুপাত প্রায় সমান থাকে (যেমন: ৫০% Yes, ৫০% No)।
- **অন্যান্য অপশন:** "balanced" অথবা কাস্টম ডিকশনারি {0: 1, 1: 5}।
- **কখন এবং কেন 'balanced' ব্যবহার করব:** যখন ডেটা প্রবলভাবে ইমব্যালেন্সড (যেমন: টাইটানিকে মারা গেছে ৮০%, বেঁচেছে মাত্র ২০%)। class_weight='balanced' দিলে মডেলটি স্বয়ংক্রিয়ভাবে ছোট ক্লাসকে বেশি গুরুত্ব দেয়, ফলে মডেলের রিকল (Recall) এবং এফ১-স্কোর (F1-score) উন্নত হয়।

max_features: None (default)

- **কাজ কী:** প্রতিটা স্প্লিটে সেরা শর্তটি খোঁজার সময় ডিসিশন ট্রি সর্বোচ্চ কয়টি ফিচার স্ক্যান করবে, তা এটি নিয়ন্ত্রণ করে।
- **কখন 'None' (Default) রাখব:** যখন ডেটাসেটে ফিচারের সংখ্যা কম (যেমন: ১০-১৫টি কলাম) এবং আপনি চান মডেলটি প্রতিবার সব ফিচার দেখে সেরা সিদ্ধান্তটি নিক।
- **অন্যান্য অপশন:** "sqrt" (ফিচার সংখ্যার বর্গমূল), "log2" (ফিচার সংখ্যার লগ বেস ২) অথবা সরাসরি কোনো সংখ্যা বা ভগ্নাংশ।
- **কখন এবং কেন পরিবর্তন করব:** যখন ডেটাসেটে ফিচারের সংখ্যা বিশাল (যেমন: ৫০০ বা ১০০০ কলাম)। তখন প্রতিটা নোডে সব ফিচার চেক করতে গেলে মডেল স্লো হয়ে যায়। max_features='sqrt' দিলে সে প্রতি স্প্লিটে র্যান্ডমলি কিছু ফিচার বেছে নিয়ে তার মধ্য থেকে সেরাটি খোঁজে। এটি মূলত **Random Forest** অ্যালগরিদমের মূল ভিত্তি।

ccp_alpha: 0.0 (default)

- **কাজ কী:** এটি গাছের পোস্ট-প্রুনিং (Post-Pruning) বা বড় হয়ে যাওয়া গাছের অপ্রয়োজনীয় ডালপালা কেটে ছোট ফেলার জটিল গাণিতিক প্যারামিটার।

- **কখন '0.0' (Default) রাখব:** ডিফল্ট শূন্য রাখা মানে মডেলটি কোনো ডালপালা কাটবে না, গাছ যেভাবে বাড়ে সেভাবেই থাকবে।
- **অন্যান্য অপশন ও কখন ব্যবহার করব:** এখানে যেকোনো অ-ঋণাত্মক ফ্লোট মান (যেমন: 0.01, 0.015) দেওয়া যায়। যখন গাছটি অলরেডি সম্পূর্ণ বড় হয়ে ওভারফিট করে ফেলেছে, তখন কস্ট-কমপ্লেক্সিটি অ্যালগরিদম দিয়ে দুর্বল ডালগুলো কেটে মডেলকে জেনারেলাইজড করতে এটি টিউন করা হয়।

random_state: None (default)

- **কাজ কী:** মডেলের শাফলিং এবং র্যান্ডম অপারেশনগুলোকে নিয়ন্ত্রণ করে কোডের আউটপুটকে প্রতিবার একই রকম বা রিপ্ৰোডিউসিবল (*Reproducible*) রাখে।
- **কখন 'None' (Default) রাখব:** যদি আপনি কোড প্রতিবার রান করার সময় সামান্য ভিন্ন ভিন্ন গাছ বা আউটপুট দেখতে চান (রিসার্চের সময় সাধারণত রাখা হয় না)।
- **অন্যান্য অপশন ও কখন ব্যবহার করব:** এখানে যেকোনো ফিক্সড পূর্ণসংখ্যা (কনভেনশন হিসেবে 42 বা 0) দেওয়া উচিত। অ্যাসাইনমেন্ট, পরীক্ষা বা টিম প্রজেক্টে যেন আপনার কোড অন্য কম্পিউটারে রান করলেও অবিকল একই গাছ এবং একই এক্যুরেসির রেজাল্ট দেয়, তা নিশ্চিত করতে এটি ফিক্স করা বাধ্যতামূলক।

৩. লেস ইম্পোর্টেন্ট প্যারামিটারসমূহ (Rarely Used)

সাধারণ মডেলিংয়ে এগুলো খুব একটা পরিবর্তন করার প্রয়োজন হয় না, ডিফল্ট মানই চমৎকার কাজ করে।

splitter: "best" (default)

- **কাজ কী:** নোড ভাগ করার সময় ফিচারগুলোর মধ্য থেকে থ্রেশহোল্ড বেছে নেওয়ার স্ট্র্যাটেজি।
- **কখন 'best' (Default) রাখব:** সবসময়। এটি গাণিতিকভাবে সবচেয়ে নিখুঁত এবং সর্বোত্তম স্প্লিটিং পয়েন্টটি খুঁজে বের করে।
- **অন্যান্য অপশন:** "random"।
- **কখন এবং কেন 'random' ব্যবহার করব:** যখন ডেটাসেট অনেক বিশাল এবং আপনি চান মডেলটি বেস্ট পয়েন্ট না খুঁজে র্যান্ডমলি একটি ভালো পয়েন্ট নিয়ে দ্রুত ট্রেনিং শেষ করুক (কম্পিউটেশনাল টাইম বাঁচাতে)।

max_leaf_nodes: None (default)

- **কাজ কী:** একটি গাছে সর্বোচ্চ কয়টি শেষ পাতা বা লিফ নোড থাকতে পারবে তার সংখ্যা বেঁধে দেওয়া।
- **কখন 'None' (Default) রাখব:** যখন আপনি max_depth বা অন্যান্য প্যারামিটার দিয়ে অলরেডি গাছ নিয়ন্ত্রণ করছেন। None মানে অসীম সংখ্যক লিফ নোড হতে পারে।
- **অন্যান্য অপশন ও কখন ব্যবহার করব:** যেকোনো পূর্ণসংখ্যা। এটিও ওভারফিটিং কমানোর আরেকটি বিকল্প রাস্তা, তবে সাধারণত max_depth টিউন করলে এটির আলাদা প্রয়োজন পড়ে না।

min_impurity_decrease: 0.0 (default)

- **কাজ কী:** একটি নোডকে তখনই ভাঙা হবে, যদি সেই স্প্লিটের কারণে ইম্পিউরিটি বা অবিশুদ্ধতার হ্রাস পাওয়ার মান এই থ্রেশহোল্ডের চেয়ে সমান বা বড় হয়।
- **কখন '0.0' (Default) রাখব:** ডিফল্ট ০.০ রাখা মানে ইম্পিউরিটি সামান্য কমলেও (অর্থাৎ সামান্য তথ্য লাভ হলেও) গাছটি স্প্লিট করবে। সাধারণত এটি ডিফল্ট রেখেই বাকি প্যারামিটার টিউন করা হয়।

monotonic_cst: None (default)

- **কাজ কী:** ফিচারের ওপর মোনোটোনিক বা একমুখী শর্তারোপ করা (যেমন: বয়স বাড়লে ইন্সুরেন্সের প্রিমিয়াম শুধু বাড়বেই, কখনো কমবে না—এমন লজিক এনফোর্স করা)। এটি বাইনারি ক্লাসিফিকেশনের ক্ষেত্রে পজিটিভ ক্লাসের প্রোবাবিলিটির ওপর কাজ করে। সাধারণ প্রজেক্টে এটি None রাখা হয়।

min_weight_fraction_leaf: 0.0 (default)

- **কাজ কী:** একটি লিফ নোডে থাকার জন্য স্যাম্পলগুলোর ওজনের নূন্যতম যোগফলের অনুপাত। যদি স্যাম্পল ওয়েট (sample_weight) ব্যবহার না করা হয়, তবে সব স্যাম্পলের ওজন সমান থাকে এবং এটি min_samples_leaf-এর মতোই কাজ করে। সাধারণত এটি পরিবর্তন করার প্রয়োজন হয় না।

বাকি থাকা কম ব্যবহৃত প্যারামিটারসমূহ (Remaining Parameters)

এগুলো সাধারণত অ্যাডভান্সড রিসার্চ বা কাস্টম প্রজেক্ট ছাড়া খুব একটা পরিবর্তন করতে হয় না।

min_weight_fraction_leaf: 0.0 (default)

- **কাজ কী:** এটি min_samples_leaf-এর মতোই কাজ করে, তবে এটি সরাসরি স্যাম্পলের সংখ্যার বদলে স্যাম্পলগুলোর ওজনের (Weights) নূন্যতম অনুপাত হিসাব করে।
- **কখন এবং কেন ব্যবহার করব:** যদি আপনি মডেল ডট ফিট (model.fit) করার সময় sample_weight প্যারামিটার দিয়ে কিছু ডেটা রো-কে বেশি বা কম গুরুত্ব দিয়ে থাকেন। তখন এই নোডের ওজন সামগ্রিক ওজনের নির্দিষ্ট শতাংশ (যেমন: 0.05 বা ৫%) এর নিচে নেমে গেলে গাছ সেখানে ডালপালা মেলানো বন্ধ করে দেবে। যদি কোনো ওজনের তারতম্য না থাকে, তবে সব স্যাম্পলের ওজন সমান (1) ধরা হয় এবং এটি ডিফল্ট 0.0 রাখাই স্ট্যান্ডার্ড।

min_impurity_decrease: 0.0 (default)

- **কাজ কী:** একটি নোডকে তখনই স্প্লিট বা ভাগ করা হবে, যদি সেই ভাগের কারণে ইম্পিউরিটি (Gini বা Entropy) হ্রাস পাওয়ার মান এই থ্রেশহোল্ডের চেয়ে সমান বা বড় হয়।
- **গাণিতিক ব্যাকএন্ড সূত্র:**

$$\Delta i = \frac{N_t}{N} \times \left(\text{impurity} - \frac{N_{tR}}{N_t} \times \text{right_impurity} - \frac{N_{tL}}{N_t} \times \text{left_impurity} \right)$$

(এখানে N হলো মোট স্যাম্পল, N_t হলো বর্তমান নোডের স্যাম্পল, এবং L/R হলো বাম ও ডান চাইল্ড নোডের স্যাম্পল)

- **কখন এবং কেন পরিবর্তন করব:** যখন আপনি চান গাছটি ছোটখাটো বা অর্থহীন ফিচারের জন্য ডালপালা না মেলুক। যদি আপনি min_impurity_decrease=0.01 সেট করেন, তবে যে স্প্লিটটি ডেটার বিশুদ্ধতা অন্তত 0.01 পরিমাণ উন্নত করতে পারবে না, ডিসিশন ট্রি সেই স্প্লিটটি বাতিল করে দেবে। এটিও ওভারফিটিং নিয়ন্ত্রণে সাহায্য করে।

monotonic_cst: None (default)

- **কাজ কী:** এটি ফিচারের ওপর মোনোটোনিক বা একমুখী গাণিতিক শর্ত (Monotonicity Constraint) জোরপূর্বক চাপিয়ে দিতে ব্যবহৃত হয় (Scikit-Learn ভার্সন ১.৪-এ নতুন যুক্ত হয়েছে)।
- **অপশনসমূহ:** প্রতিটা ফিচারের জন্য একটি অ্যারে আকারে 1 (Monotonic Increase), -1 (Monotonic Decrease) অথবা 0 (No Constraint) দেওয়া যায়।

- **কখন এবং কেন ব্যবহার করব:** ধরুন আপনি একটি ক্রেডিট কার্ড লোন প্রজেক্ট করছেন, যেখানে আপনি নিশ্চিত করতে চান যে "কাস্টমারের আয় বাড়লে লোন পাওয়ার প্রোবাবিলিটি সবসময় বাড়বেই, কখনো কমবে না"। তখন আয়ের কলামের জন্য 1 সেট করা হয়। মনে রাখুন, এটি মাল্টি-ক্লাস ক্লাসিফিকেশন (Classes > 2) সাপোর্ট করে না, শুধু বাইনারি ক্লাসিফিকেশনের পজিটিভ ক্লাসের প্রোবাবিলিটির ওপর কাজ করে।

ডিসিশন ট্রির মূল অ্যাট্রিবিউটসমূহ (Important Attributes)

মডেল ডট ফিট (model.fit(X_train, y_train)) করার পর, মডেলটি ব্যাকএন্ডের লার্নিং থেকে কী কী তথ্য মেমোরিতে জমা করে রেখেছে, তা আমরা এই অ্যাট্রিবিউটগুলো দিয়ে দেখতে পারি। এদের শেষে সবসময় একটি আন্ডারস্কোর () থাকে।

feature_importances_

- **কাজ কী:** এটি একটি অ্যারে ব্যাক করে যা দেখায় যে, আপনার দেওয়া ফিচারগুলোর মধ্যে কোন কলামটি টার্গেট ভ্যারিয়েবল প্রেডিক্ট করতে সবচেয়ে বেশি ভূমিকা পালন করেছে।
- **ব্যাকএন্ড মেকানিজম:** যে ফিচারটি গাছের যত ওপরের দিকে (রুট নোডের কাছাকাছি) স্প্লিট করতে ব্যবহৃত হয়েছে এবং যার কারণে জিনি বা এনট্রপি সবচেয়ে বেশি কমেছে, তার ইম্পোর্টেন্স তত বেশি হয়। সব ফিচারের গুরুত্বের যোগফল সবসময় 1.0 হয়। এটি **Feature Selection**-এর জন্য সবচেয়ে জনপ্রিয় একটি টুল।

tree_

- **কাজ কী:** এটি ডিসিশন ট্রির আসল ব্যাকএন্ড অবজেক্ট (Underlying Tree Object)।
- **কেন প্রয়োজন:** গাছটি ভেতরে ভেতরে কীভাবে তৈরি হলো—তার নোড সংখ্যা কত, কোন নোডের বামে বা ডানে কী আছে, তা যদি আপনি পাইথন কোড দিয়ে স্ক্র্যাপ বা কাস্টমাইজ করতে চান, তবে model.tree_ ব্যবহার করে একদম র লেভেলের আর্কিটেকচার অ্যাক্সেস করা যায়।

classes_

- **কাজ কী:** আপনার টার্গেট ভ্যারিয়েবলে (y) আসলে কী কী শ্রেণি বা লেবেল ছিল, তার একটি ইউনিক অ্যারে দেখায়।

- **উদাহরণ:** টাইটানিক ডেটাসেটে `model.classes_` রান করলে আউটপুট আসবে `array([0, 1])`।

`n_features_in_`

- **কাজ কী:** মডেলটি ট্রেন করার সময় ইনপুট ম্যাট্রিক্সে (`X_train`) সর্বমোট কতগুলো ফিচার বা কলাম দেখেছিল, তার সুনির্দিষ্ট সংখ্যা জানায়।

`feature_names_in_`

- **কাজ কী:** ট্রেনিং ডেটায় থাকা সব কলামের অফিশিয়াল নামের একটি অ্যারে দেখায় (এটি তখনই কাজ করে যখন আপনার ইনপুট ডেটা একটি `pandas DataFrame` বা স্ট্রিং কলামযুক্ত হয়)।

`n_classes_`

- **কাজ কী:** টার্গেট কলামে মোট কয়টি ক্লাস বা ক্যাটাগরি আছে তার সংখ্যা দেয়। বাইনারি ক্লাসিফিকেশনে এর মান হবে 2।

`n_outputs_`

- **কাজ কী:** মডেলটি ফিট করার সময় কয়টি আউটপুট ভ্যারিয়েবল প্রেডিক্ট করার জন্য ট্রেন করা হয়েছিল তার সংখ্যা। সাধারণ সিঙ্গেল-আউটপুট প্রজেক্টে এর মান সবসময় 1 হয়।

DecisionTreeRegressor() Attributes

`feature_importances_`

Default: Created after `fit()`

প্রতিটি Feature-এর Importance Score সংরক্ষণ করে। যেসব Feature Prediction-এর জন্য বেশি গুরুত্বপূর্ণ, তাদের Score বেশি হয়।

`tree_`

Default: Created after `fit()`

Decision Tree-এর সম্পূর্ণ Internal Structure সংরক্ষণ করে। এর মধ্যে প্রতিটি Node-এর Threshold, Split Feature, Child Node, Sample সংখ্যা এবং Prediction Value থাকে।

`max_features_`

Default: Created after `fit()`

প্রতিটি Split-এর সময় সর্বোচ্চ কতটি Feature বিবেচনা করা হয়েছে তা সংরক্ষণ করে।

n_features_in_

Default: Created after fit()

Training-এর সময় ব্যবহৃত মোট Input Feature-এর সংখ্যা সংরক্ষণ করে।

feature_names_in_

Default: Created after fit() (*DataFrame input only*)

Training Data যদি Pandas DataFrame হয়, তাহলে Feature-গুলোর নাম সংরক্ষণ করে।

n_outputs_

Default: Created after fit()

Model মোট কতটি Target Output Predict করছে তা সংরক্ষণ করে। সাধারণ Regression-এ এটি সাধারণত 1 হয়।

Decision Tree Code using Sci-Kit Learn:

```
import numpy as np
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, classification_report

iris = load_iris()
X = pd.DataFrame(iris.data, columns=iris.feature_names)
y = pd.Series(iris.target)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

default_pipeline = Pipeline(
    steps=[
        ('model', DecisionTreeClassifier(random_state=42))
    ]
)

default_pipeline.fit(X_train, y_train)

y_pred_default = default_pipeline.predict(X_test)
print("--- Default Model Performance ---")
print(f"Test Accuracy: {accuracy_score(y_test, y_pred_default):.4f}")
print(classification_report(y_test, y_pred_default, target_names=iris.target_names))
```

Using Grid Search CV:

```
from sklearn.model_selection import GridSearchCV

tuning_pipeline = Pipeline(
    steps=[
        ('model', DecisionTreeClassifier(random_state=42))
    ]
)

param_grid = {
    'model__criterion': ['gini', 'entropy'],
    'model__max_depth': [3, 4, 5, None],
    'model__min_samples_split': [2, 5, 10],
    'model__min_samples_leaf': [1, 2, 4]
}

grid_search = GridSearchCV(
    estimator=tuning_pipeline,
    param_grid=param_grid,
    cv=5,
    scoring='accuracy',
    n_jobs=-1
)

grid_search.fit(X_train, y_train)

print("--- GridSearchCV Tuning Results (No Scaling) ---")
print("Best Parameters Found:", grid_search.best_params_)
print(f"Best CV Accuracy: {grid_search.best_score_:.4f}")

best_model_pipeline = grid_search.best_estimator_
y_pred_tuned = best_model_pipeline.predict(X_test)

print("\n--- Tuned Model Performance on Test Set ---")
print(f"Test Accuracy: {accuracy_score(y_test, y_pred_tuned):.4f}")
print(classification_report(y_test, y_pred_tuned, target_names=iris.target_names))

importances = best_model_pipeline.named_steps['model'].feature_importances_
for name, importance in zip(iris.feature_names, importances):
    print(f"{name:20s}: {importance:.4f}")
```

প্রশ্ন ১: Decision Tree-তে Gini Impurity এবং Entropy-এর মধ্যে প্রধান পার্থক্য কী? Scikit-Learn কেন ডিফল্ট হিসেবে criterion="gini" ব্যবহার করে?

উত্তর:

Gini Impurity এবং Entropy উভয়ই Decision Tree-এর প্রতিটি নোডের বিশুদ্ধতা (Purity) বা অবিশুদ্ধতা (Impurity) পরিমাপের জন্য ব্যবহৃত হয়। Entropy তথ্য তত্ত্ব (Information Theory)-এর উপর ভিত্তি করে কাজ করে এবং Information Gain ব্যবহার করে সর্বোত্তম Split নির্বাচন করে। এর গণনায় লগারিদম ($\log_2$) ব্যবহৃত হওয়ায় এটি তুলনামূলকভাবে বেশি কম্পিউটেশনাল সময় প্রয়োজন করে।

অন্যদিকে, Gini Impurity-এর সূত্র তুলনামূলকভাবে সহজ এবং এতে কোনো লগারিদমিক গণনা প্রয়োজন হয় না। ফলে এটি দ্রুত গণনা করা যায়। এই কারণেই Scikit-Learn ডিফল্টভাবে criterion="gini" ব্যবহার করে। তবে বাস্তবে Gini এবং Entropy উভয়ের Performance সাধারণত খুব কাছাকাছি হয়।

প্রশ্ন ২: Decision Tree-তে Overfitting কেন ঘটে এবং এটি প্রতিরোধ করার প্রধান উপায়গুলো কী?

উত্তর:

Decision Tree-কে যদি কোনো সীমাবদ্ধতা ছাড়া প্রশিক্ষণ দেওয়া হয়, তাহলে এটি যতক্ষণ সম্ভব Data Split করতে থাকে এবং প্রতিটি Leaf Node-কে যতটা সম্ভব বিশুদ্ধ করার চেষ্টা করে। এর ফলে মডেলটি শুধুমাত্র প্রকৃত Pattern নয়, বরং Training Data-এর Noise এবং ব্যতিক্রমী তথ্যও শিখে ফেলে। এই অবস্থাকে Overfitting বলা হয়।

Overfitting প্রতিরোধের জন্য সাধারণত নিচের Hyperparameter-গুলো ব্যবহার করা হয়:

- max_depth : Tree-এর সর্বোচ্চ গভীরতা নির্ধারণ করে।
- min_samples_split : একটি Node Split করার জন্য ন্যূনতম Sample সংখ্যা নির্ধারণ করে।
- min_samples_leaf : প্রতিটি Leaf Node-এ ন্যূনতম কতটি Sample থাকতে হবে তা নির্ধারণ করে।
- ccp_alpha : Cost-Complexity Pruning-এর মাধ্যমে অপ্রয়োজনীয় Branch অপসারণ করে।

প্রশ্ন ৩: যদি max_depth=None এবং min_samples_split=2 একসাথে ব্যবহার করা হয়, তাহলে Decision Tree-এর আচরণ কেমন হবে?

উত্তর:

যখন max_depth=None এবং min_samples_split=2 একসাথে ব্যবহার করা হয়, তখন Decision Tree-এর গভীরতার ওপর কোনো সীমা থাকে না। মডেলটি যতক্ষণ Node বিশুদ্ধ না হয়, Split করার জন্য পর্যাপ্ত Sample থাকে এবং একটি বৈধ Split পাওয়া যায়, ততক্ষণ পর্যন্ত Tree বৃদ্ধি পেতে থাকে।

ফলে Tree অত্যন্ত গভীর হয়ে যেতে পারে এবং Training Data-এর Noise পর্যন্ত শিখে ফেলতে পারে। এর ফলে Training Accuracy অনেক বেশি হলেও Test Data বা নতুন Data-এর উপর Performance কমে যেতে পারে, যা Overfitting-এর একটি সাধারণ লক্ষণ।

প্রশ্ন ৪: feature_importances_ অ্যাট্রিবিউট কীভাবে কাজ করে এবং এটি কেন গুরুত্বপূর্ণ?

উত্তর:

feature_importances_ প্রতিটি Feature Decision Tree তৈরির সময় মোট কতটুকু Impurity কমাতে সাহায্য করেছে, তার একটি পরিমাপ প্রদান করে। কোনো Feature যদি বারবার গুরুত্বপূর্ণ Split তৈরি করে এবং প্রতিবার উল্লেখযোগ্য পরিমাণে Gini Impurity বা Entropy কমায়, তাহলে তার Importance বেশি হয়।

সমস্ত Feature Importance-এর যোগফল সর্বদা 1.0 হয়।

এই Attribute ব্যবহার করে গুরুত্বপূর্ণ Feature শনাক্ত করা, অপয়োজনীয় Feature বাদ দেওয়া এবং Feature Selection সম্পাদন করা সহজ হয়। ফলে Model আরও কার্যকর এবং ব্যাখ্যাযোগ্য (Interpretable) হয়।

প্রশ্ন ৫: Decision Tree-কে কেন একটি White-Box Model বলা হয়? কোন পরিস্থিতিতে এটি Logistic Regression-এর তুলনায় বেশি কার্যকর?

উত্তর:

Decision Tree-কে একটি **White-Box Model** বলা হয়, কারণ এর সিদ্ধান্ত গ্রহণের প্রতিটি ধাপ সহজেই দেখা, বোঝা এবং ব্যাখ্যা করা যায়। এটি মূলত ধারাবাহিক **IF-ELSE** নিয়মের মাধ্যমে সিদ্ধান্ত গ্রহণ করে, ফলে কোনো Prediction কীভাবে এসেছে তা সহজেই অনুসরণ করা সম্ভব।

যখন Feature এবং Target Variable-এর মধ্যে জটিল (Complex) এবং Non-linear সম্পর্ক থাকে, তখন Decision Tree সাধারণত Logistic Regression-এর তুলনায় ভালো ফলাফল প্রদান করতে পারে। কারণ Decision Tree সহজেই Non-linear Decision Boundary শিখতে পারে, যেখানে Logistic Regression মূলত Linear Decision Boundary-এর উপর নির্ভর করে।

Quick Revision

১. মূল ধারণা (Core Intuition)

- **Type:** এটি একটি অত্যন্ত জনপ্রিয় Supervised Machine Learning Algorithm, যা ক্লাসিফিকেশন ও রিগ্রেশন উভয় ক্ষেত্রে কাজ করে।
- **Mechanism:** এটি মানুষের চিন্তাভাবনার মতো ডেটার বিভিন্ন বৈশিষ্ট্যের ওপর ভিত্তি করে একটার পর একটা "IF-ELSE" বা "Yes/No" কন্ডিশন তৈরি করে পুরো ডেটাসেটকে ছোট ছোট ভাগে ভাগ করে ফেলে।
- **White-Box Model:** এর সিদ্ধান্ত নেওয়ার পুরো প্রক্রিয়াটি একটি ফ্লোচার্ট বা গাছের ডালপালার মতো দৃশ্যমান হওয়ায় একে হোয়াইট-বক্স মডেল বলা হয় (Highly Interpretable)।

২. গাঠনিক উপাদান (Tree Anatomy)

- **Root Node (মূল নোড):** গাছের একদম শীর্ষের নোড, যেখান থেকে প্রথম স্প্লিট বা বিভাজন শুরু হয়।
- **Internal / Decision Node:** মাঝখানের নোডগুলো, যা কোনো ফিচারের শর্ত নির্দেশ করে এবং ডেটাকে আরও নিচে ভাগ করে দেয়।
- **Leaf Node (চূড়ান্ত পাতা):** গাছের একদম শেষ প্রান্তের নোড। এর নিচে আর কোনো স্প্লিট হয় না, এটিই মূলত চূড়ান্ত প্রেডিকশন বা আউটপুট।

৩. ব্যাকএন্ডের গাণিতিক পরিমাপ (Mathematical Formulation)

মডেলটি কোন ফিচারের কোন মান দিয়ে নোড ভাগ করবে, তা নির্ধারণ করতে নিচের ৩টি ডিফল্ট ম্যাট্রিক্স ব্যবহার করে:

- **Entropy (এনট্রপি):** ডেটার মধ্যকার এলোমেলো অবস্থা বা বিশৃঙ্খলার পরিমাপ। সম্পূর্ণ অবিশুদ্ধ নোডে এর মান 1.0 এবং সম্পূর্ণ বিশুদ্ধ নোডে এর মান 0 হয়।

$$Entropy(S) = - \sum p_i \log_2 p_i$$

- **Information Gain (তথ্য লাভ):** কোনো ফিচার দিয়ে ডেটা ভাগ করলে এনট্রপি বা বিশৃঙ্খলা কতটুকু কমছে তার পরিমাপ। ডিসিশন ট্রি সবসময় সর্বোচ্চ Information Gain থাকা ফিচারটিকে স্প্লিট করার জন্য বেছে নেয়।
- **Gini Impurity (জিনি ইম্পিউরিটি):** Scikit-Learn-এর ডিফল্ট মেকানিজম। এটি কম্পিউটেশনালি অত্যন্ত ফাস্ট কারণ এতে কোনো লগারিদম হিসাব করতে হয় না। সম্পূর্ণ বিশুদ্ধ নোডের জিনি ইম্পিউরিটি হয় 0।

$$Gini = 1 - \sum (p_i)^2$$

৪. প্রধান হাইপারপ্যারামিটার ও ওভারফিটিং নিয়ন্ত্রণ (Top Hyperparameters)

ডিসিশন ট্রিকে বাধা না দিলে সে শেষ পর্যন্ত ডেটা মুখস্থ করে Overfitting (High Variance) ঘটায়। এটি বন্ধ করার মূল ব্রেকগুলো হলো:

- **max_depth:** গাছের সর্বোচ্চ গভীরতা বা লেভেল ফিক্সড করে দেওয়া। ওভারফিটিং কমানোর প্রধান হাতিয়ার।
- **min_samples_split:** একটি নোডকে ভাগ করার জন্য নূন্যতম কয়টি স্যাম্পল থাকতে হবে (ডিফল্ট: ২)। এর মান বাড়ালে ওভারফিটিং কমে।
- **min_samples_leaf:** একটি চূড়ান্ত পাতায় নূন্যতম কয়টি স্যাম্পল থাকতেই হবে (ডিফল্ট: ১)। এর মান বাড়ালে মডেল স্মুথ হয়।
- **max_features:** প্রতি স্প্লিটে সেরা শর্ত খোঁজার জন্য সর্বোচ্চ কয়টি ফিচার স্ক্যান করা হবে (যেমন: "sqrt", "log2")।
- **class_weight:** ইমব্যালেন্সড ডেটাসেটে ছোট ক্লাসকে বেশি গুরুত্ব দিতে "balanced" মোড ব্যবহার করা হয়।

৫. সুবিধা ও সীমাবদ্ধতা (Pros & Cons)

Advantages (সুবিধাসমূহ)	Disadvantages / Limitations
সহজে বোঝা এবং ব্যাখ্যা করা যায় (Interpretability)	খুব দ্রুত Overfitting বা ডেটা মুখস্থ করার প্রবণতা দেখায়
ফিচার স্কেলিং বাধ্যতামূলক নয় (Scale-Independent)	ডেটার সামান্য পরিবর্তনে পুরো গাছের গঠন বদলে যায় (Unstable)
আউটলায়ারের প্রতি অত্যন্ত Robust (প্রভাব পড়ে না)	রিগ্রেশনের ক্ষেত্রে আউটপুট মসৃণ না হয়ে সিঁড়ির মতো ধাপযুক্ত হয়

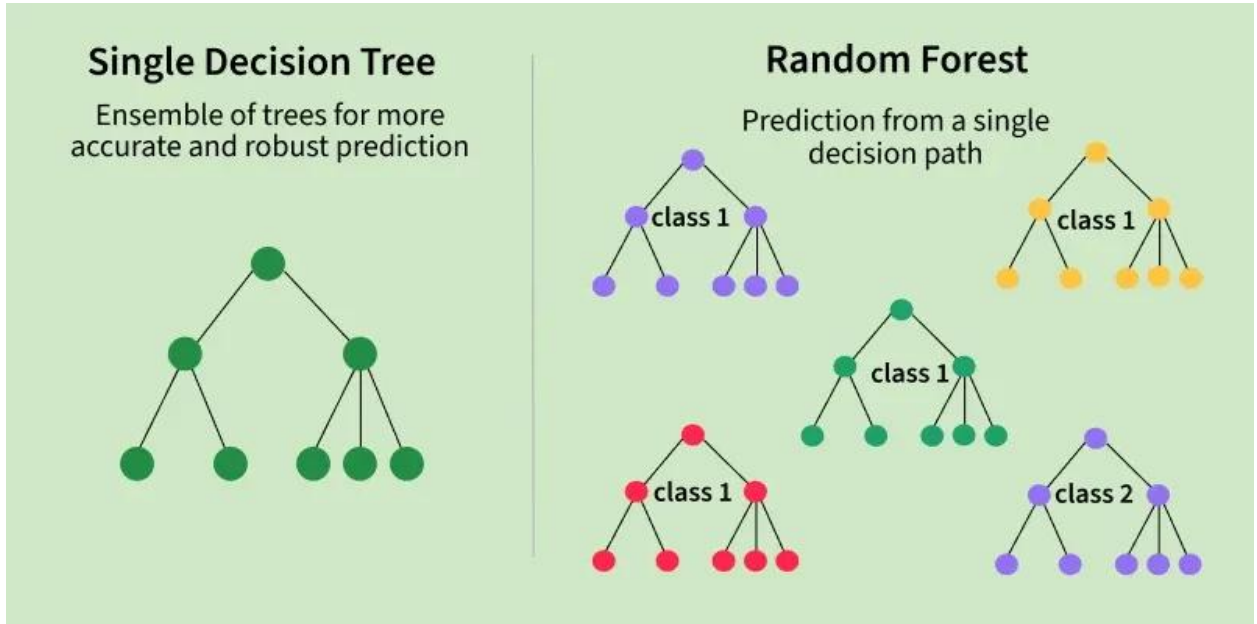
ক্যাটাগরিক্যাল ও নিউমেরিক্যাল ডেটা
একসাথে সামলাতে পারে

ক্যাটাগরির সংখ্যা অনেক বেশি হলে (High
Cardinality) গাছ জটিল হয়

Random Forest

Random Forest (র্যান্ডম ফরেস্ট) হলো মেশিন লার্নিংয়ের অত্যন্ত শক্তিশালী, জনপ্রিয় এবং বহুমুখী একটি Supervised Learning Algorithm, যা ক্লাসিফিকেশন (Classification) এবং রিগ্রেশন (Regression) দুই ধরনের সমস্যার সমাধানই নিখুঁতভাবে করতে পারে।

সহজ কথায়, অনেকগুলো ডিসিশন ট্রি (Decision Tree) বা সিদ্ধান্ত-গাছ একসাথে মিলে যে জঙ্গল বা ফরেস্ট তৈরি করে, সেটিই হলো র্যান্ডম ফরেস্ট। এটি একটি Ensemble Learning মেথড, যার মূল মন্ত্র হলো—"একটি একক গাছের সিদ্ধান্তের চেয়ে পুরো জঙ্গলের সম্মিলিত সিদ্ধান্ত অনেক বেশি নিখুঁত ও শক্তিশালী।"



ধরুন, আপনি ছুটিতে কোথাও ঘুরতে যাবেন এবং একটি সুন্দর জায়গা সিলেক্ট করতে চান।

- **ডিসিশন ট্রির পদ্ধতি:** আপনি আপনার একজন বন্ধুকে জিজ্ঞেস করলেন। সে তার ব্যক্তিগত অভিজ্ঞতা অনুযায়ী আপনাকে কক্সবাজার যাওয়ার পরামর্শ দিল। এখানে ভুল হওয়ার সম্ভাবনা বা বায়াসড (Biased) হওয়ার চান্স বেশি, কারণ এটি মাত্র একজন মানুষের মত।
- **র্যান্ডম ফরেস্টের পদ্ধতি:** আপনি ১ জনের কাছে না গিয়ে আপনার ২০ জন বন্ধুকে আলাদা আলাদাভাবে জিজ্ঞেস করলেন। কেউ বলল সিলেট, কেউ বলল সাজেক, আবার বেশিরভাগ বন্ধু বলল কক্সবাজার। এবার আপনি সবার মতামত নিয়ে ভোটিং (Majority Voting) করলেন এবং দেখলেন যে ২০ জনের মধ্যে ১৪ জনই কক্সবাজারের পক্ষে। আপনি

মেজরিটি ভোটের ভিত্তিতে সিদ্ধান্ত নিলেন কক্সবাজার যাবেন। র্যান্ডম ফরেস্ট ঠিক এইভাবেই কাজ করে।

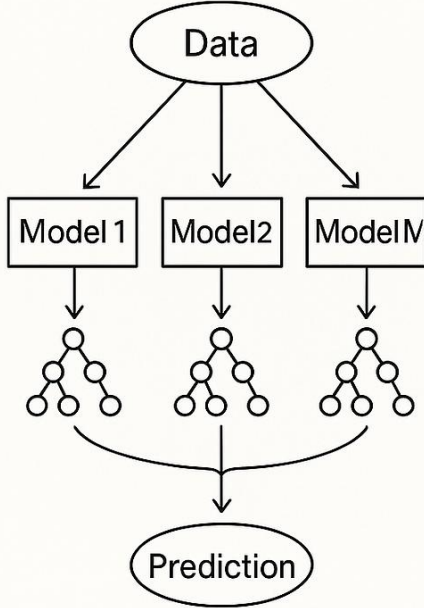
র্যান্ডম ফরেস্টের ব্যাকএন্ড মেকানিজম

র্যান্ডম ফরেস্ট মূলত দুটি প্রধান স্তরের ওপর দাঁড়িয়ে কাজ করে:

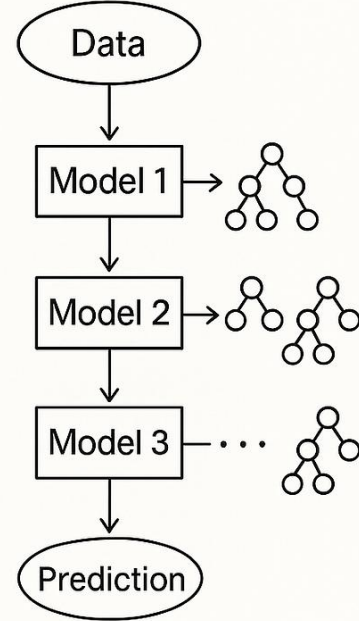
১. Bagging বা Bootstrap Aggregating (বুটস্ট্রাপ অ্যাগ্রিগেটিং)

আপনার কাছে যদি একটি মূল ডেটাসেট থাকে, র্যান্ডম ফরেস্ট সব গাছকে ছবছ এক ডেটা দেয় না। সে মূল ডেটাসেট থেকে এলোমেলোভাবে (Row Sampling with replacement) ডেটা তুলে ছোট ছোট সাব-ডেটাসেট তৈরি করে এবং এক একটি ডিসিশন ট্রি-কে ট্রেন করার জন্য দেয়। একে বলে Bootstrapping।

Bagging



Boosting



২. Feature Randomness (ফিচারের র্যান্ডমনেস)

শুধু ডেটা রো (Row) নয়, প্রতিটা ডিসিশন ট্রি নোড স্প্লিট করার সময় ডেটাসেটের সব কলাম বা ফিচার দেখতে পায় না। সে সম্পূর্ণ র্যান্ডমলি কিছু ফিচার (যেমন: মোট ১০টি ফিচারের মধ্যে ৩টি)

বেছে নিয়ে তার মধ্য থেকে সেরা স্প্লিট খুঁজে নেয় (`max_features='sqrt'`)। এর ফলে জঙ্গলের প্রতিটা গাছ একে অপরের থেকে সম্পূর্ণ আলাদা ও স্বাধীনভাবে গড়ে ওঠে।

৩. Aggregation (একত্রীকরণ বা চূড়ান্ত সিদ্ধান্ত)

সবগুলো গাছ যখন ট্রেন হয়ে যায়, তখন নতুন কোনো ডেটা প্রেডিক্ট করার জন্য সেটিকে জঙ্গলের সব গাছের মধ্য দিয়ে পাস করানো হয়:

- **Classification-এর ক্ষেত্রে:** সব গাছের আউটপুটের মধ্যে Majority Voting বা সংখ্যাগরিষ্ঠের ভোটের ভিত্তিতে ফাইনাল ক্লাস নির্ধারণ করা হয়।
- **Regression-এর ক্ষেত্রে:** প্রতিটি গাছের দেওয়া প্রেডিকশনের গড় বা গড় মান (Average /Mean) হিসাব করে চূড়ান্ত সংখ্যাটি আউটপুট দেওয়া হয়।

র্যান্ডম ফরেস্টের প্রধান সুবিধাসমূহ (Advantages)

1. **ওভারফিটিং মুক্ত (Robust to Overfitting):** ডিসিশন ট্রির সবচেয়ে বড় দুর্বলতা হলো ওভারফিটিং। র্যান্ডম ফরেস্ট অনেকগুলো গাছের ফলাফলের গড় বা ভোট নেয় বলে এতে ওভারফিটিংয়ের সম্ভাবনা প্রায় থাকে না বললেই চলে।
2. **মিসিং ভ্যালু হ্যান্ডলিং:** ডেটাসেটে যদি বড় অঙ্কের মিসিং ভ্যালু থাকে, তবুও এটি তার ব্যাকঅ্যান্ড প্রক্সিমিটি অ্যালগরিদম দিয়ে চমৎকার একুরেসী ধরে রাখতে পারে।
3. **ফিচার ইম্পোর্টেন্স (Feature Importance):** ডিসিশন ট্রির মতো এটিও ট্রেনিং শেষে `feature_importances_` প্রপার্টি ব্যবহার করে ডেটাসেটের কোন কলামটি প্রেডিকশনের জন্য সবচেয়ে বেশি গুরুত্বপূর্ণ, তা নিখুঁতভাবে জানিয়ে দেয়।
4. **স্কেলিংয়ের প্রয়োজন নেই:** ডিসিশন ট্রির মতো র্যান্ডম ফরেস্টও কোনো প্রকার ফিচার স্কেলিং (StandardScaler বা MinMaxScaler) করার প্রয়োজন পড়ে না।

সীমাবদ্ধতা (Limitations)

- **ব্ল্যাক-বক্স মডেল (Black-box Nature):** একক ডিসিশন ট্রিকে খুব সহজে `plot_tree` দিয়ে ভিজ্যুয়ালাইজ করে সাধারণ মানুষকে বোঝানো যায় (White-box)। কিন্তু র্যান্ডম ফরেস্টে যখন ৫০০টি গাছ একসাথে কাজ করে, তখন ঠিক কোন লজিকে সিদ্ধান্ত এসেছে তা ট্র্যাক করা অসম্ভব হয়ে পড়ে। তাই এটি একটি ব্ল্যাক-বক্স মডেল।

- **ধীরগতি (Slow to Predict):** র্যান্ডম ফরেস্ট ট্রেনিংয়ের সময় সমান্তরালভাবে (Parallely) কাজ করলেও, রিয়েল-টাইম প্রোডাকশনে প্রতিটি গাছের মধ্য দিয়ে ডেটা পাস করে ভোট গণনা করতে লিনিয়ার মডেলের চেয়ে সামান্য বেশি সময় নিতে পারে।

র্যান্ডম ফরেস্ট (Random Forest) ক্লাসিফিকেশন এবং রিগ্রেশন দুই ধরনের সমস্যার সমাধান করে।

১. Classification (শ্রেণিবিভাগ) এর ক্ষেত্রে:

- **মেকানিজম:** ক্লাসিফিকেশন সমস্যার জন্য র্যান্ডম ফরেস্ট তার জঙ্গলের প্রতিটি ডিসিশন ট্রি-কে আলাদা আলাদাভাবে ডেটা প্রেডিক্ট করতে দেয়।
- **চূড়ান্ত সিদ্ধান্ত:** সব গাছ থেকে প্রেডিকশন পাওয়ার পর, মডেলটি **Majority Voting (সংখ্যার ভোট)** গণনা করে। যে ক্লাস বা ক্যাটাগরিটি সবচেয়ে বেশি গাছ প্রেডিক্ট করেছে (যেমন: ১০০টি গাছের মধ্যে ৭০টি গাছ বলল 'Yes' আর ৩০টি গাছ বলল 'No'), মেজরিটি ভোটের ভিত্তিতে চূড়ান্ত আউটপুট হবে 'Yes'।

২. Regression (মান অনুমান) এর ক্ষেত্রে:

- **মেকানিজম:** রিগ্রেশন সমস্যার ক্ষেত্রে (যেখানে আউটপুট কোনো সংখ্যা হয়), জঙ্গলের প্রতিটি ডিসিশন ট্রি স্বাধীনভাবে একটি সংখ্যা বা মান প্রেডিক্ট করে।
- **চূড়ান্ত সিদ্ধান্ত:** সব গাছের দেওয়া সংখ্যাচক প্রেডিকশনগুলোর গড় বা গড় মান (Average/Mean) হিসাব করা হয়। এই গড় মানটিই হয় মডেলের চূড়ান্ত প্রেডিকশন (যেমন: ৫টি গাছ একটি বাড়ির দাম বলল যথাক্রমে ৫০, ৫২, ৪৮, ৫১ এবং ৪৯ লাখ টাকা; তাহলে র্যান্ডম ফরেস্ট এদের গড় করে চূড়ান্ত দাম বলবে ৫০.২ লাখ টাকা)।

Decision Tree থেকে Random Forest-এ রূপান্তর এবং এর প্রয়োজনীয়তা

Bias (বায়াস)

- Core Concept: একটি মেশিন লার্নিং মডেলের সরল অনুমানের কারণে যে ভুলের সৃষ্টি হয়, তাকে বায়াস বলে। মডেল যদি ডেটার ভেতরের আসল এবং জটিল প্যাটার্ন ধরতে না পারে, তবে তাকে High Bias বলা হয়। এর ফলে মডেলটি ট্রেনিং ডেটা এবং টেস্ট ডেটা-উভয় ক্ষেত্রেই খুব খারাপ পারফর্ম করে, যাকে মেশিন লার্নিংয়ের ভাষায় Underfitting বলা হয় (যেমন: একটি জটিল লিনিয়ার-নয় এমন ডেটার ওপর সাধারণ Linear Regression ব্যবহার করা)। অন্যদিকে, মডেল যদি ডেটার প্যাটার্ন খুব ভালোভাবে ধরতে পারে, তবে তাকে Low Bias বলা হয়।

Variance (ভ্যারিয়েন্স)

- Core Concept: ট্রেনিং ডেটার সামান্য পরিবর্তনের কারণে মডেলের প্রেডিকশনে যে পরিমাণ ওলটপালট বা বিচ্যুতির সৃষ্টি হয়, তাকে ভ্যারিয়েন্স বলে। মডেল যদি ট্রেনিং ডেটার প্রতিটি নয়েজ (Noise) ও ব্যতিক্রমী পয়েন্ট মুখস্থ করে ফেলে, তবে তাকে High Variance বলা হয়। এর ফলে মডেলটি ট্রেনিং সেটে ১০০% এক্যুরেসী দিলেও নতুন কোনো টেস্ট ডেটার সামনে গেলেই সম্পূর্ণ ফেইল করে, যাকে Overfitting বলা হয় (যেমন: গভীর ডালপালা বিশিষ্ট একটি আনফ্রন্ড Decision Tree)। আর মডেল যদি নতুন ডেটাতেও স্থিতিশীল পারফর্ম করে, তবে তাকে Low Variance বলা হয়।

2. The 4 Quadrants Matrix

মেশিন লার্নিং মডেলের কার্যক্ষমতা বুঝতে একটি জনপ্রিয় বুলস-আই (Bulls-eye) বা লক্ষ্যভেদের ডার্টবোর্ডের উদাহরণ ব্যবহার করা হয়, যেখানে বোর্ডের একদম কেন্দ্রটি (Center) হলো আমাদের আসল বা সঠিক উত্তর:

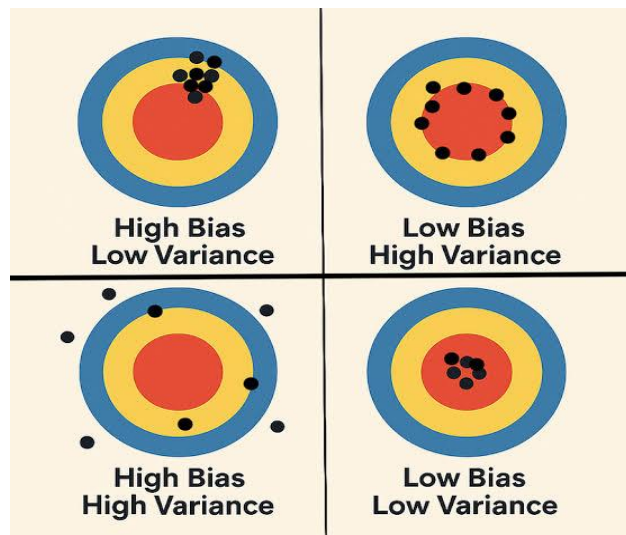
Low Bias, Low Variance (আদর্শ মডেল)

- ব্যাখ্যা: এই মডেলের প্রেডিকশনগুলো একদম নিখুঁত এবং কেন্দ্রের খুব কাছাকাছি থাকে (Low Bias)। একই সাথে ডেটা পরিবর্তন করলেও এর সিদ্ধান্ত বদলে যায় না,

সবগুলো শট এক জায়গাতেই পড়ে (Low Variance)। এটিই মেশিন লার্নিংয়ের সবচেয়ে কাঙ্ক্ষিত ও সুষম মডেল।

Low Bias, High Variance (ওভারফিটিং)

- ব্যাখ্যা: এই মডেলটি গড়ে সঠিক উত্তরের কাছাকাছি পৌঁছাতে পারে (Low Bias)। কিন্তু এর প্রেডিকশনগুলোর মধ্যে ছড়ানো-ছিটানো ভাব বা অস্থিরতা অনেক বেশি (High Variance)। অর্থাৎ, ট্রেনিং ডেটা সামান্য পরিবর্তন হলেই এর প্রেডিকশনগুলো চারদিকে ছড়িয়ে পড়ে। এটি Overfitting-এর স্পষ্ট লক্ষণ।



High Bias, Low Variance (আন্ডারফিটিং)

- ব্যাখ্যা: এই মডেলের সবগুলো প্রেডিকশন খুব কাছাকাছি এক জায়গাতেই পড়ে (Low Variance), কিন্তু সেগুলো আসল লক্ষ্য বা কেন্দ্র থেকে অনেক দূরে অবস্থান করে (High Bias)। মডেলটি ভুল লজিকে অনড় থাকে। এটি Underfitting-এর উদাহরণ।

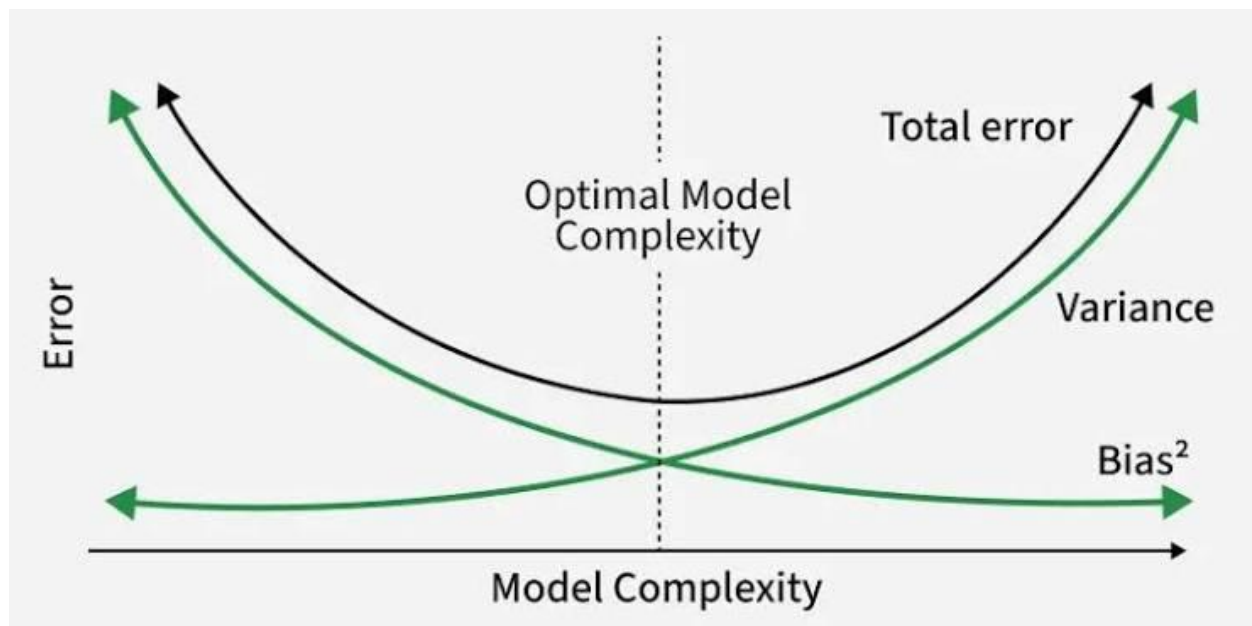
High Bias, High Variance (সবচেয়ে বাজে মডেল)

- ব্যাখ্যা: এই মডেলটির প্রেডিকশনগুলো যেমন লক্ষ্যভ্রষ্ট বা কেন্দ্র থেকে দূরে (High Bias), তেমনই সেগুলো একে অপরের থেকে চরমভাবে ছড়ানো-ছিটানো (High Variance)। এটি সবচেয়ে অস্থিতিশীল এবং অকেজো মডেল।

3. The Bias-Variance Trade-off

- The Conflict: গাণিতিক ও বাস্তবিকভাবে একই সাথে বায়াস এবং ভ্যারিয়েন্স-উভয়কেই একসাথে শূন্যে নামিয়ে আনা অসম্ভব। আপনি যদি মডেলকে খুব জটিল (Complex) বানাতে যান (যেমন: Deep Neural Networks বা Deep Decision Trees), তবে বায়াস কমবে কিন্তু ভ্যারিয়েন্স আশঙ্কাজনকভাবে বেড়ে যাবে। আবার

মডেলকে খুব সরল (Simple) রাখলে ভ্যারিয়েন্স কম থাকবে ঠিকই, কিন্তু বায়াস বেড়ে যাবে।



- The Goal: মেশিন লার্নিং ইঞ্জিনিয়ারদের মূল লক্ষ্য হলো মডেলের জটিলতার এমন একটি সর্বোত্তম বিন্দু বা সুইট স্পট (Sweet Spot) খুঁজে বের করা, যেখানে Total Error (যা বায়াস এবং ভ্যারিয়েন্সের বর্গের যোগফল) সর্বনিম্ন হয়। র্যান্ডম ফরেস্ট (Random Forest) অ্যালগরিদমটি মূলত প্রতিটি গাছের লো-বায়াস ক্ষমতাকে ধরে রেখে, ব্যাগিং প্রক্রিয়ার মাধ্যমে হাই-ভ্যারিয়েন্সকে টেনে নিচে নামিয়ে এনে এই চমৎকার ভারসাম্যটি বজায় রাখে।

The Evolution

- **মূল সমস্যা:** একটি একক ডিসিশন ট্রি ট্রেনিং ডেটাসেটের শেষ সীমা পর্যন্ত ডালপালা গজাতে থাকে এবং ডেটাকে ১০০% বিশুদ্ধ (Pure) করার চেষ্টা করে। এর ফলে এটি ডেটার ভেতরের নয়েজ (Noise) বা ছোটখাটো ভুল প্যাটার্নও মুখস্থ করে ফেলে এবং মডেলটিতে হাই ভ্যারিয়েন্স বা ওভারফিটিং (Overfitting) দেখা দেয়। এমন মডেল নতুন কোনো টেস্ট ডেটার মুখোমুখি হলে সঠিক সিদ্ধান্ত দিতে পারে না।
- **সমাধান:** ডিসিশন ট্রির জটিল নন-লিনিয়ার সম্পর্ক ধরার ক্ষমতাকে অক্ষুণ্ণ রেখে ওভারফিটিং দূর করার জন্য র্যান্ডম ফরেস্টের আগমন ঘটে। এখানে একটিমাত্র গভীর ও জটিল গাছের ওপর নির্ভর না করে, শত শত ভিন্ন ভিন্ন স্বাধীন ডিসিশন ট্রি (একটি জঙ্গল) তৈরি করা হয়। প্রতিটি গাছের নিজস্ব ভুল বা ত্রুটিগুলো যখন একত্রিত করে গড় করা হয়, তখন সামগ্রিক জঙ্গলের সিদ্ধান্তটি অত্যন্ত সুষম ও নিখুঁত হয়ে ওঠে।

Why We Moved to Random Forest

- **ওভারফিটিং ও ভ্যারিয়েন্স নিয়ন্ত্রণ (Reduction in Variance):** একটি একক ডিসিশন ট্রিতে লো বায়াস (Low Bias) কিন্তু হাই ভ্যারিয়েন্স (High Variance) থাকে। র্যান্ডম ফরেস্ট শত শত গাছের ফলাফলের গড় বা ভোট নেয় বলে এই হাই ভ্যারিয়েন্স বা ওভারফিটিংয়ের ঝুঁকি স্বয়ংক্রিয়ভাবে কমে যায়। কোনো একটি গাছের ভুল সিদ্ধান্ত বাকি গাছগুলোর সঠিক সিদ্ধান্তের ভিড়ে ঢাকা পড়ে যায়।
- **ফিচারের র্যান্ডমনেস (Feature Randomness):** একটি সাধারণ ডিসিশন ট্রি প্রতিটি নোড স্প্লিট করার সময় ডেটাসেটের উপলব্ধ সবকটি কলাম স্ক্যান করে সেরা কলামটি বেছে নেয়, যার ফলে শক্তিশালী ফিচারগুলোই বারবার ঘুরেফিরে আসে। কিন্তু র্যান্ডম ফরেস্ট প্রতি স্প্লিটে সম্পূর্ণ র্যান্ডমলি নির্দিষ্ট সংখ্যক ফিচারের একটি সাবসেট (*max_features*) বেছে নেয়। এর ফলে জঙ্গলের গাছগুলোর মধ্যে চরম গাঠনিক বৈচিত্র্য তৈরি হয় এবং ডেটার ভেতরে লুকিয়ে থাকা দুর্বল ফিচারের গুরুত্বপূর্ণ প্যাটার্নও উন্মোচিত হয়।
- **গাঠনিক স্থিতিশীলতা (Structural Stability):** একটি একক ডিসিশন ট্রি অত্যন্ত সংবেদনশীল বা আনস্টেবল। ট্রেনিং ডেটার সামান্য পরিবর্তন বা দু-একটি নতুন রো

যুক্ত হলে এর রুট নোডের স্প্লিট বদলে যেতে পারে, যা পুরো গাছের গঠন ও সিদ্ধান্ত ওলটপালট করে দেয়। র্যান্ডম ফরেস্ট বুটস্ট্রাপ স্যাম্পলিং ব্যবহারের মাধ্যমে প্রতিটি গাছকে ডেটার ভিন্ন ভিন্ন অংশ দিয়ে ট্রেন করে, ফলে ডেটার আংশিক পরিবর্তনে পুরো ফরেস্টের ওপর কোনো নেতিবাচক প্রভাব পড়ে না।

- **ব্যাখ্যাযোগ্যতা হ্রাস পাওয়া (Loss of Full Interpretability):** একটি একক গাছ থেকে যখন আমরা শত শত গাছের জঙ্গলে স্থানান্তরিত হই, তখন মডেলটি একটি স্পষ্ট হোয়াইট-বক্স (White-Box) থেকে জটিল ব্ল্যাক-বক্স (Black-Box) সিস্টেমে পরিণত হয়। একসাথে ৫০০টি গাছের হাজার হাজার কন্ডিশনাল পাথ (if-else রুলস) ট্র্যাক করা মানুষের পক্ষে অসম্ভব হয়ে পড়ে। উচ্চমাত্রার এক্যুরেসী অর্জনের জন্য এটিই হলো র্যান্ডম ফরেস্টের প্রধান ট্রেড-অফ (Trade-off)।

ডিসিশন ট্রি বনাম র্যান্ডম ফরেস্ট (Decision Tree vs Random Forest)

বৈশিষ্ট্য	Decision Tree	Random Forest
গঠন	মাত্র একটি একক গাছ দিয়ে গঠিত হয়।	শত শত ডিসিশন ট্রির সমন্বয়ে গঠিত জঙ্গল।
ওভারফিটিং ঝুঁকি	অত্যন্ত বেশি (High Variance)। খুব দ্রুত ডেটা মুখস্থ করে ফেলে।	অত্যন্ত কম। অনেক গাছের গড় নেওয়ার ফলে ওভারফিটিং স্বয়ংক্রিয়ভাবে দূর হয়।
এক্যুরেসী	মঝঝি বা তুলনামূলক কম।	সাধারণত অত্যন্ত নিখুঁত এবং উচ্চ এক্যুরেসী দেয়।
কম্পিউটেশন টাইম	খুব দ্রুত ট্রেন হয়।	ট্রেনিং হতে কিছুটা বেশি সময় ও মেমোরি লাগে।

এনসেম্বল লার্নিংয়ের প্রধান দুটি পদ্ধতি

১. Bagging (Bootstrap Aggregating) বা সমান্তরাল পদ্ধতি

- **কাজের প্রক্রিয়া:** এই পদ্ধতিতে মূল ডেটাসেট থেকে এলোমেলোভাবে প্রতিস্থাপনের মাধ্যমে (With replacement) একাধিক স্বাধীন সাব-ডেটাসেট বা বুটস্ট্র্যাপ স্যাম্পল তৈরি করা হয়। এরপর প্রতিটি স্যাম্পলের ওপর আলাদা আলাদা মডেলকে (যেমন: ডিসিশন ট্রি) সম্পূর্ণ স্বাধীন এবং সমান্তরালভাবে (Parallely) ট্রেন করা হয়।
- **চূড়ান্ত সিদ্ধান্ত:** সবকটি মডেলের ট্রেনিং শেষে তাদের আউটপুটগুলোকে একত্রিত করা হয়। ক্লাসিফিকেশনের ক্ষেত্রে মেজরিটি ভোটিং (Majority Voting) এবং রিগ্রেশনের ক্ষেত্রে সব মডেলের প্রেডিকশনের গড় (Averaging) হিসাব করে চূড়ান্ত সিদ্ধান্ত নেওয়া হয়।
- **প্রধান লক্ষ্য:** এই পদ্ধতির মূল লক্ষ্য হলো মডেলের ভ্যারিয়েন্স (Variance) বা ওভারফিটিংয়ের ঝুঁকি কমিয়ে আনা।
- **উদাহরণ:** Random Forest।

২. Boosting বা ধারাবাহিক পদ্ধতি

- **কাজের প্রক্রিয়া:** বুস্টিং পদ্ধতিতে মডেলগুলো সমান্তরালভাবে কাজ করে না, বরং সিকোয়েন্সিয়াল বা ধারাবাহিকভাবে (One after another) কাজ করে। এখানে প্রথমে একটি দুর্বল বেস মডেল (Weak Learner) দিয়ে ট্রেনিং শুরু করা হয়। প্রথম মডেলটি ট্রেন করার পর যে ডেটা পয়েন্টগুলোতে ভুল প্রেডিকশন বা ত্রুটি (Errors) থেকে যায়, পরবর্তী মডেলটি তৈরি করার সময় সেই ভুল ডেটাগুলোর ওপর বেশি গুরুত্ব বা ওয়েট (Weight) দেওয়া হয়।
- **ভুল থেকে শিক্ষা:** এভাবে প্রতিটি নতুন মডেল তার আগের মডেলের করা ভুলগুলো থেকে শিক্ষা নিয়ে নিজে থেকে ক্রমান্বয়ে উন্নত করতে থাকে। অর্থাৎ, আগের মডেলের দুর্বলতাকে পরের মডেলটি এসে সংশোধন করে।
- **চূড়ান্ত সিদ্ধান্ত:** সবকটি মডেল ধারাবাহিকভাবে ট্রেন হওয়ার পর, তাদের ওজনের ওপর ভিত্তি করে একটি ভরযুক্ত যোগফল (Weighted Average/Sum) তৈরি করে চূড়ান্ত প্রেডিকশন দেওয়া হয়।
- **প্রধান লক্ষ্য:** এই পদ্ধতির মূল লক্ষ্য হলো মডেলের বায়াস (Bias) বা ভুলের পরিমাণ কমিয়ে আনা এবং মডেলের শক্তি বৃদ্ধি করা। তবে এটি ব্যাগিংয়ের চেয়ে ওভারফিট করার ক্ষেত্রে কিছুটা বেশি সংবেদনশীল।
- **উদাহরণ:** XGBoost, AdaBoost, Gradient Boosting (GBM)।

Mathematical Intuition

- **Core Philosophy:** র্যান্ডম ফরেস্ট গাণিতিকভাবে বৈচিত্র্যময় ও স্বাধীন ডিসিশন ট্রি তৈরির জন্য Law of Large Numbers-এর নীতি ব্যবহার করে। প্রতিটি একক গাছের ভুল বা নয়েজ একে অপরের থেকে আলাদা হওয়ায়, যখন শত শত গাছের ফলাফলকে একত্রিত করা হয়, তখন সেই ত্রুটিগুলো পরস্পরকে নাকচ (Cancel out) করে দেয় এবং সামগ্রিক ভ্যারিয়েন্স শূন্যের কাছাকাছি নেমে আসে।

Gini Index / Entropy

- **Impurity Measurement:** জঙ্গলের ভেতরের প্রতিটি ডিসিশন ট্রি নোড স্প্লিট করার সময় নিচের যেকোনো একটি গাণিতিক সূত্র দিয়ে তথ্যের বিশুদ্ধতা পরিমাপ করে:
- **Gini Impurity:**

$$Gini = 1 - \sum_{i=1}^c (p_i)^2$$

(এখানে p_i হলো কোনো নোডে একটি নির্দিষ্ট ক্লাসের ডেটার অনুপাত। সম্পূর্ণ বিশুদ্ধ নোডের জিনি মান হয় 0 এবং সম্পূর্ণ অবিশুদ্ধ বা সমান অনুপাতের নোডের মান হয় 0.5)

- **Entropy:**

$$Entropy = - \sum_{i=1}^c p_i \log_2 p_i$$

(বিশুদ্ধতার গাণিতিক পরিমাপ। সম্পূর্ণ বিশুদ্ধ নোডে এর মান 0 এবং সর্বোচ্চ অবিশুদ্ধ নোডে এর মান 1.0 হয়)

Information Gain

- **Optimization Metric:** একটি নোডকে ভাঙার পর জিনি ইম্পিউরিটি বা এনট্রপি কতটা হ্রাস পেল, তা ইনফরমেশন গেইন দিয়ে পরিমাপ করা হয়। প্রতিটি গাছ এমন ফিচার এবং থ্রেশহোল্ড বেছে নেয় যার জন্য নিচের এই সমীকরণের মান সর্বোচ্চ হয়:

$$Information\ Gain = Impurity(Parent) - \sum \left(\frac{N_{Child}}{N_{Parent}} \times Impurity(Child) \right)$$

Random Feature Selection

- **Feature Subsetting:** সাধারণ ডিসিশন ট্রি নোড ভাঙার সময় সব ফিচার (d) স্ক্যান করে। কিন্তু র্যান্ডম ফরেস্ট প্রতিটি স্প্লিটে র্যান্ডমলি m সংখ্যক ফিচারের একটি সাবসেট বেছে নেয়।

- **Standard Convention:** ক্লাসিফিকেশনের ক্ষেত্রে সাধারণত $m = \sqrt{d}$ এবং রিগ্রেশনের ক্ষেত্রে $m = d/3$ কলাম নেওয়া হয়। গাণিতিকভাবে এটি গাছগুলোর মধ্যকার কোরিলেশন বা পারস্পরিক মিল কমিয়ে দেয়, যা জঙ্গলকে আরও শক্তিশালী করে।

Training Algorithm

- **Step 1 (Bootstrapping):** মূল ডেটাসেট থেকে প্রতিস্থাপনের নীতি মেনে (With replacement) $n_{estimators}$ সংখ্যক ভিন্ন ভিন্ন বুটস্ট্র্যাপ স্যাম্পল তৈরি করা হয়।
- **Step 2 (Tree Growth):** প্রতিটি স্যাম্পল ডেটার ওপর একটি করে ডিসিশন ট্রি ট্রেন করা হয়। গাছ বৃদ্ধির সময় প্রতিটি নোডে শুধুমাত্র র্যান্ডমলি সিলেক্টেড $max_features$ কলামের মধ্য থেকে সেরা স্প্লিটটি বেছে নেওয়া হয়। কোনো প্রকার প্রুনিং (Pruning) ছাড়াই গাছগুলোকে সর্বোচ্চ গভীরতা পর্যন্ত বাড়তে দেওয়া হয়।
- **Step 3 (Ensemble):** এই প্রক্রিয়াটি লুপের মতো চলতে থাকে যতক্ষণ না নির্ধারিত সংখ্যক (T) স্বাধীন গাছ তৈরি সম্পন্ন হচ্ছে।

Prediction Algorithm

- **Step 1 (Individual Prediction):** নতুন কোনো টেস্ট ডেটা রো (x) ইনপুট হিসেবে আসলে, সেটিকে জঙ্গলের প্রতিটি ট্রেনড গাছের মধ্য দিয়ে পাস করানো হয়। প্রতিটি গাছ সম্পূর্ণ স্বাধীনভাবে নিজের একটি আউটপুট ($h_t(x)$) প্রদান করে।
- **Step 2 (Classification Aggregation):** ক্লাসিফিকেশনের ক্ষেত্রে ফাইনাল প্রেডিকশন হলো মেজরিটি ভোট:

$$\hat{Y} = \text{mode}\{h_1(x), h_2(x), \dots, h_T(x)\}$$

- **Step 3 (Regression Aggregation):** রিগ্রেশনের ক্ষেত্রে ফাইনাল প্রেডিকশন হলো সব গাছের আউটপুটের গাণিতিক গড়:

$$\hat{Y} = \frac{1}{T} \sum_{t=1}^T h_t(x)$$

Time & Space Complexity

- **Training Time Complexity:**

$$O(T \cdot m \cdot N \log N)$$

(এখানে T = number of trees, N = number of samples, এবং m = number of random features যেহেতু প্রতিটি গাছ সমান্তরালভাবে বা মাল্টি-থ্রেডিং ব্যবহার করে ট্রেন করা যায়, তাই বড় ডেটাসেটেও এটি বেশ দ্রুত কাজ করে)

- **Prediction Time Complexity:**

$$O(T \cdot \text{depth})$$

(টেস্ট টাইমে প্রতিটি গাছের গভীরতা বা depth বরাবর ডেটা ট্রান্সার করতে হয়, যা রিয়েল-টাইম প্রেডিকশনের জন্য অত্যন্ত ফাস্ট)

- **Space Complexity:**

$$O(T \cdot \text{number of nodes})$$

(যেহেতু শত শত গভীর ডিসিশন ট্রির প্রতিটি নোডের শর্ত ও মান মেমোরিতে ধরে রাখতে হয়, তাই র্যান্ডম ফরেস্টের মডেল সাইজ বা মেমোরি কস্ট লিনিয়ার মডেলের চেয়ে অনেক বেশি হয়)

Number of Trees (n_estimators): 3

Maximum Features (max_features): 2

ID	Study Hours	Attendance	Assignment	Pass (Target)
1	2	Low	No	No
2	3	Medium	Yes	No
3	5	High	Yes	Yes
4	6	High	Yes	Yes
5	4	Medium	No	Yes

Total instances (N) = 5

Class distribution: Pass = Yes → {3, 4, 5} (3টি) | Pass = No → {1, 2} (2টি)

Bootstrap Sampling

Random Forest এ প্রতিটি Decision Tree কে আলাদা Bootstrap Sample দিয়ে train করা হয়। Bootstrap Sampling হলো sampling with replacement — অর্থাৎ একটি instance একাধিকবার সিলেক্ট হতে পারে। প্রতিটি sample এ মোট N = 5টি row থাকবে, কিন্তু কিছু row repeat হবে এবং কিছু বাদ পড়বে (Out-of-Bag)।

Bootstrap Sample Size = N = 5 (original dataset এর সমান)

Sampling: with replacement → প্রতিবার random index ∈ {1, 2, 3, 4, 5} থেকে বাছাই

BOOTSTRAP SAMPLE 1 — TREE 1

Row	ID	Pass
1	1	No
2	3	Yes
3	3	Yes
4	4	Yes
5	5	Yes

Yes=4, No=1 | OOB: ID {2}

BOOTSTRAP SAMPLE 2 — TREE 2

Row	ID	Pass
1	2	No
2	2	No
3	3	Yes
4	4	Yes
5	5	Yes

Yes=3, No=2 | OOB: ID {1}

BOOTSTRAP SAMPLE 3 — TREE 3

Row	ID	Pass
1	1	No
2	2	No
3	4	Yes
4	4	Yes
5	5	Yes

Yes=3, No=2 | OOB: ID {3}

Random Feature Selection ($\text{max_features} = 2$)

Random Forest এর একটি মূল বৈশিষ্ট্য হলো – প্রতিটি split এর সময় সব feature ব্যবহার না করে randomly একটি subset বাছাই করা হয়। এখানে মোট 3টি feature আছে (Study Hours, Attendance, Assignment) এবং $\text{max_features} = 2$ । অর্থাৎ প্রতি split এ যেকোনো 2টি feature থেকে সেরাটি বেছে নেওয়া হবে।

All features: {Study Hours, Attendance, Assignment} $\rightarrow$ total = 3
max_features = 2 $\rightarrow$ প্রতিটি split এ randomly 2টি বাছাই করতে হবে

Tree 1 — Feature Selection

✓ Study Hours ✓ Attendance ✗ Assignment

Random selection: **Study Hours** এবং **Attendance** chosen $\rightarrow$ **Assignment** এই split এ বিবেচিত হবে না।

Tree 2 — Feature Selection

✗ Study Hours ✓ Attendance ✓ Assignment

Random selection: **Attendance** এবং **Assignment** chosen $\rightarrow$ **Study Hours** এই split এ বিবেচিত হবে না।

Tree 3 — Feature Selection

✓ Study Hours ✗ Attendance ✓ Assignment

Random selection: **Study Hours** এবং **Assignment** chosen $\rightarrow$ **Attendance** এই split এ বিবেচিত হবে না।

Gini Impurity – সূত্র ও ধারণা

Decision Tree এ সেরা split খুঁজে বের করতে Gini Impurity ব্যবহার করা হয়। Gini Impurity একটি node এর অশুদ্ধতা (impurity) পরিমাপ করে। $\text{Gini} = 0$ মানে node সম্পূর্ণ pure (সব একই class), $\text{Gini} = 0.5$ মানে সর্বোচ্চ impurity (binary case)।

Gini(node) = 1 - Σ p_i²

যেখানে p_i = node এ class i এর proportion

Weighted Gini (split এর জন্য):

Gini_split = (|Left| / |Total|) × Gini(Left) + (|Right| / |Total|) × Gini(Right)

Information Gain = Gini(Parent) - Gini_split

→ যে split এ Information Gain সবচেয়ে বেশি, সেটিই সেরা split।

Tree 1 — সম্পূর্ণ Gini Calculation

Bootstrap Sample 1 (features: Study Hours, Attendance)

Row	ID	Study Hours	Attendance	Pass
1	1	2	Low	No
2	3	5	High	Yes
3	3	5	High	Yes
4	4	6	High	Yes
5	5	4	Medium	Yes

Step 1 — Root Node Gini Impurity

Total = 5 | Yes = 4 | No = 1

p(Yes) = 4/5 = 0.80 | p(No) = 1/5 = 0.20

Gini(Root) = 1 - (0.80² + 0.20²) = 1 - (0.64 + 0.04) = 1 - 0.68 = 0.3200

Step 2 — Feature: Attendance (Low/Medium vs High)

Split: Attendance = High (Right) vs Non-High (Left)

Left (Low or Medium): Row {1, 5} → Yes=1 (ID5), No=1 (ID1) → n=2

$$p(\text{Yes})=1/2=0.5, p(\text{No})=1/2=0.5$$

$$\text{Gini}(\text{Left}) = 1 - (0.5^2 + 0.5^2) = 1 - 0.5 = \mathbf{0.5000}$$

Right (High): Row {2,3,4} → Yes=3, No=0 → n=3

$$p(\text{Yes})=3/3=1.0, p(\text{No})=0$$

$$\text{Gini}(\text{Right}) = 1 - (1^2 + 0^2) = 1 - 1 = \mathbf{0.0000} \text{ (pure!)}$$

$$\text{Weighted Gini} = (2/5) \times 0.5 + (3/5) \times 0.0 = 0.20 + 0.0 = \mathbf{0.2000}$$

$$\text{Information Gain} = 0.3200 - 0.2000 = \mathbf{0.1200}$$

Step 3 — Feature: Study Hours (threshold = 3.5 → ≤ 3.5 vs > 3.5)

Split: Study Hours ≤ 3.5 (Left) vs Study Hours > 3.5 (Right)

Left (≤ 3.5): Row {1} (Study=2) → Yes=0, No=1 → n=1

$$\text{Gini}(\text{Left}) = 1 - (0^2 + 1^2) = 0 \text{ (pure)}$$

Right (> 3.5): Row {2,3,4,5} → Yes=4, No=0 → n=4

$$\text{Gini}(\text{Right}) = 1 - (1^2 + 0^2) = 0 \text{ (pure)}$$

$$\text{Weighted Gini} = (1/5) \times 0 + (4/5) \times 0 = \mathbf{0.0000}$$

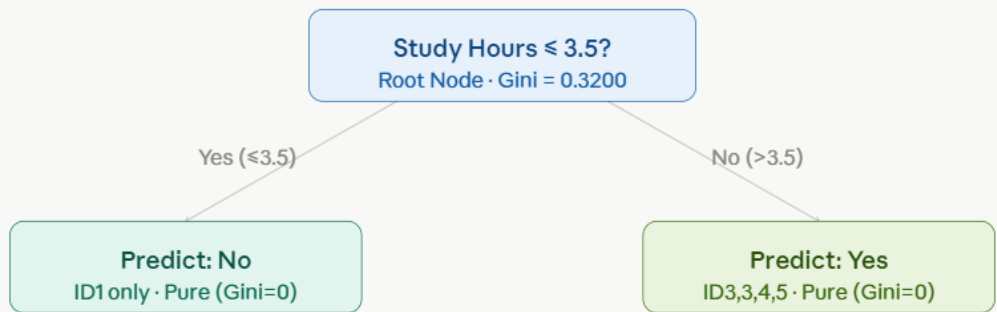
$$\text{Information Gain} = 0.3200 - 0.0000 = \mathbf{0.3200}$$

Tree 1 Root Split Decision:

Study Hours (IG = 0.3200) > Attendance (IG = 0.1200)

→ **Best Split: Study Hours ≤ 3.5**

Tree 1 — Structure



Tree 1 — max depth reached, both leaves are pure

Tree 2 — সম্পূর্ণ Gini Calculation

Bootstrap Sample 2 (features: Attendance, Assignment)

Row	ID	Attendance	Assignment	Pass
1	2	Medium	Yes	No
2	2	Medium	Yes	No
3	3	High	Yes	Yes
4	4	High	Yes	Yes
5	5	Medium	No	Yes

Step 1 — Root Node Gini Impurity

Total=5 | Yes=3 | No=2

$p(\text{Yes})=3/5=0.60$ | $p(\text{No})=2/5=0.40$

$\text{Gini}(\text{Root}) = 1 - (0.60^2 + 0.40^2) = 1 - (0.36 + 0.16) = 1 - 0.52 = 0.4800$

Step 2 — Feature: Attendance (High vs Non-High)

Left (Non-High = Medium): Row {1,2,5} → Yes=1(ID5), No=2(ID2,2) → n=3

$$p(\text{Yes})=1/3\approx 0.333, p(\text{No})=2/3\approx 0.667$$

$$\text{Gini}(\text{Left}) = 1 - (0.333^2 + 0.667^2) = 1 - (0.111 + 0.444) = 1 - 0.556 = \mathbf{0.4444}$$

Right (High): Row {3,4} → Yes=2, No=0 → n=2

$$\text{Gini}(\text{Right}) = 1 - (1^2 + 0^2) = \mathbf{0.0000} \text{ (pure)}$$

$$\text{Weighted Gini} = (3/5) \times 0.4444 + (2/5) \times 0.0 = 0.2667 + 0 = \mathbf{0.2667}$$

$$\text{Information Gain} = 0.4800 - 0.2667 = \mathbf{0.2133}$$

Step 3 — Feature: Assignment (Yes vs No)

Left (Assignment=No): Row {5} → Yes=1, No=0 → n=1

$$\text{Gini}(\text{Left}) = 0 \text{ (pure)}$$

Right (Assignment=Yes): Row {1,2,3,4} → Yes=2(ID3,4), No=2(ID2,2) → n=4

$$p(\text{Yes})=2/4=0.5, p(\text{No})=2/4=0.5$$

$$\text{Gini}(\text{Right}) = 1 - (0.5^2 + 0.5^2) = 1 - 0.5 = \mathbf{0.5000}$$

$$\text{Weighted Gini} = (1/5) \times 0 + (4/5) \times 0.5 = 0 + 0.4 = \mathbf{0.4000}$$

$$\text{Information Gain} = 0.4800 - 0.4000 = \mathbf{0.0800}$$

Tree 2 Root Split Decision:

Attendance (IG = 0.2133) > Assignment (IG = 0.0800)

→ **Best Split: Attendance = High**

Tree 2 — Second Level Split (Left child: Non-High Attendance)

Left node এ 3টি row আছে (ID2, ID2, ID5) — Yes=1, No=2। এটি impure। এখন Assignment দিয়ে split করব।

Step 4 — Left Child: Assignment split (only feature left)

$Gini(\text{Left-node}) = 0.4444$

Assignment=Yes: Row {1,2} (ID2,ID2) $\rightarrow$ Yes=0, No=2 $\rightarrow n=2$

$Gini = 1 - (0^2 + 1^2) = 0$ (pure)

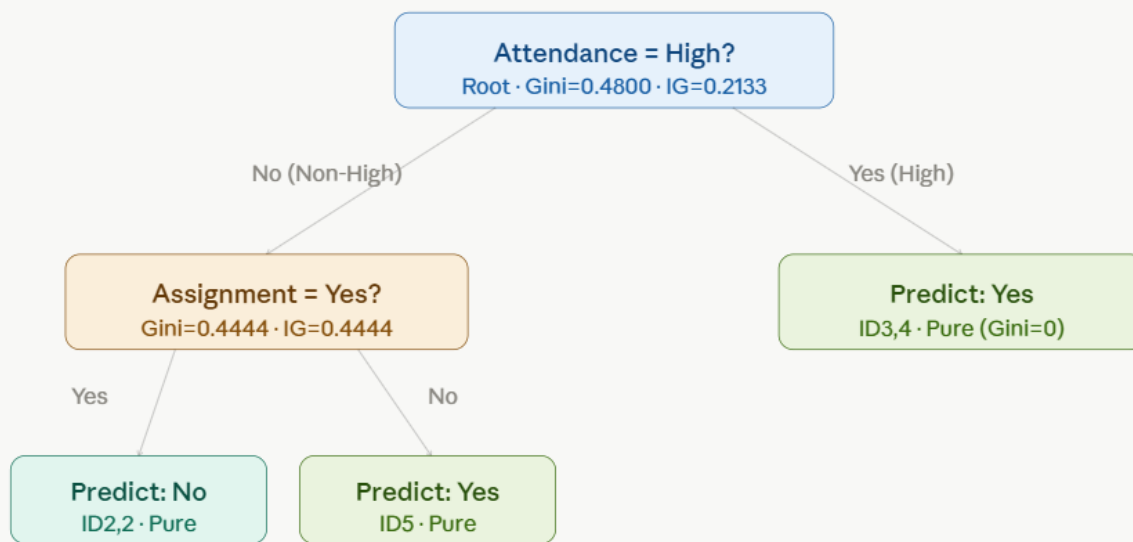
Assignment=No: Row {5} $\rightarrow$ Yes=1, No=0 $\rightarrow n=1$

$Gini = 0$ (pure)

Weighted Gini = $(2/3) \times 0 + (1/3) \times 0 = 0.0000$

IG = $0.4444 - 0 = 0.4444 \rightarrow$ Split করো!

Tree 2 — Structure



Tree 2 — depth 2, all leaves pure

Tree 3 — সম্পূর্ণ Gini Calculation

Bootstrap Sample 3 (features: Study Hours, Assignment)

Row	ID	Study Hours	Assignment	Pass
1	1	2	No	No
2	2	3	Yes	No
3	4	6	Yes	Yes
4	4	6	Yes	Yes
5	5	4	No	Yes

Step 1 — Root Node Gini Impurity

Total=5 | Yes=3 | No=2

$p(\text{Yes})=0.60$, $p(\text{No})=0.40$

Gini(Root) = $1 - (0.36 + 0.16) = 0.4800$

Step 2 — Feature: Study Hours (threshold = 3.5)

Left (≤ 3.5): Row {1,2} $\rightarrow$ Yes=0, No=2 $\rightarrow$ n=2

Gini(Left) = $1 - (0^2 + 1^2) = 0$ (pure)

Right (> 3.5): Row {3,4,5} $\rightarrow$ Yes=3 (ID 4,4,5), No=0 $\rightarrow$ n=3

Gini(Right) = $1 - (1^2 + 0^2) = 0$ (pure)

Weighted Gini = 0

Information Gain = $0.4800 - 0 = 0.4800$

Step 3 — Feature: Assignment (Yes vs No)

Left (No): Row {1,5} → Yes=1(ID5), No=1(ID1) → n=2

$$\text{Gini(Left)} = 1 - (0.5^2 + 0.5^2) = \mathbf{0.5000}$$

Right (Yes): Row {2,3,4} → Yes=2(ID4,4), No=1(ID2) → n=3

$$p(\text{Yes}) = 2/3 \approx 0.667, p(\text{No}) = 1/3 \approx 0.333$$

$$\text{Gini(Right)} = 1 - (0.667^2 + 0.333^2) = 1 - (0.444 + 0.111) = \mathbf{0.4444}$$

$$\text{Weighted Gini} = (2/5) \times 0.5 + (3/5) \times 0.4444 = 0.2 + 0.2667 = \mathbf{0.4667}$$

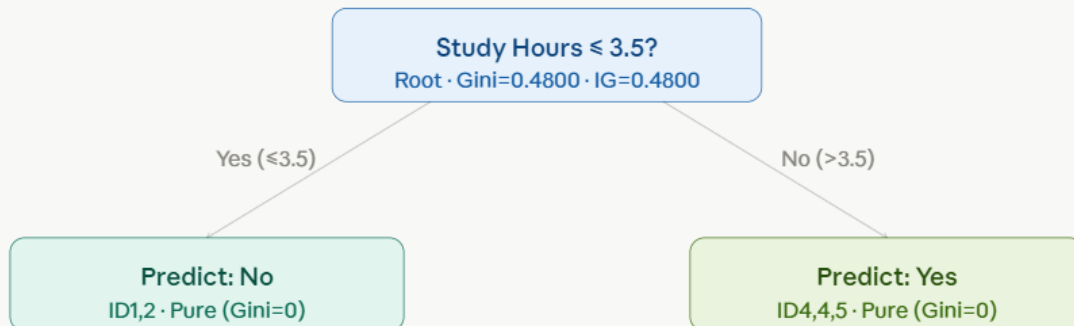
$$\text{Information Gain} = 0.4800 - 0.4667 = \mathbf{0.0133}$$

Tree 3 Root Split Decision:

Study Hours (IG = 0.4800) >> Assignment (IG = 0.0133)

→ **Best Split: Study Hours ≤ 3.5**

Tree 3 — Structure



Tree 3 — depth 1, both leaves pure (perfect separation)

নতুন Instance — Prediction

নতুন data point:

Study Hours = 4 | Attendance = High | Assignment = Yes

Tree 1 এ Traverse

Root: Study Hours ≤ 3.5 ? $\rightarrow 4 > 3.5 \rightarrow$ Right branch

Right leaf: Predict = **Yes**

Tree 1 Prediction $\rightarrow$ Yes

Tree 2 এ Traverse

Root: Attendance = High? $\rightarrow$ High = Yes $\rightarrow$ Right branch

Right leaf: Predict = **Yes**

Tree 2 Prediction $\rightarrow$ Yes

Tree 3 এ Traverse

Root: Study Hours ≤ 3.5 ? $\rightarrow 4 > 3.5 \rightarrow$ Right branch

Right leaf: Predict = **Yes**

Tree 3 Prediction $\rightarrow$ Yes

§ 9

Majority Voting — চূড়ান্ত সিদ্ধান্ত

Random Forest এ সমস্ত tree এর individual prediction collect করে **Majority Vote** নেওয়া হয়। Classification task এ যে class বেশি vote পায়, সেটিই final prediction।

Tree 1 $\rightarrow$ Yes

+

Tree 2 $\rightarrow$ Yes

+

Tree 3 $\rightarrow$ Yes

=

Yes: 3 votes

Vote Count:

Yes = 3 votes | No = 0 votes

Majority Class = Yes (3 out of 3 trees agree)

FINAL PREDICTION

✓ **Pass = Yes**

Study Hours=4, Attendance=High, Assignment=Yes → এই student টি Pass করবে।

Confidence: 3/3 trees → 100% vote unanimity

মানদণ্ড: Squared Error (MSE) | n_estimators = 3 | max_features = 2

ধাপ ১: মূল ডেটাসেট

ID	Study Hours	Sleep Hours	Attendance	Marks (Target)
1	2	8	Low	40
2	3	7	Medium	50
3	4	7	Medium	60
4	5	6	High	75
5	6	6	High	90

ধাপ ২: Bootstrap Sampling

র‍্যান্ডম ফরেস্টে প্রতিটি ট্রিকে একটি Bootstrap Sample দিয়ে প্রশিক্ষণ দেওয়া হয়। এটি হলো মূল ডেটাসেট থেকে পুনরাবৃত্তিসহ (with replacement) র‍্যান্ডমভাবে n সংখ্যক নমুনা নেওয়া (এখানে n = 5)।

Bootstrap Sample 1 (Tree 1 এর জন্য)

নির্বাচিত সারি (পুনরাবৃত্তিসহ): ID → 1, 2, 2, 4, 5

ID	Study Hours	Sleep Hours	Attendance	Marks
1	2	8	Low	40
2	3	7	Medium	50
2	3	7	Medium	50
4	5	6	High	75
5	6	6	High	90

Bootstrap Sample 2 (Tree 2 এর জন্য)

নির্বাচিত সারি (পুনরাবৃত্তিসহ): ID → 1, 3, 3, 4, 5

ID	Study Hours	Sleep Hours	Attendance	Marks
1	2	8	Low	40
3	4	7	Medium	60
3	4	7	Medium	60
4	5	6	High	75
5	6	6	High	90

Bootstrap Sample 3 (Tree 3 এর জন্য)

নির্বাচিত সারি (পুনরাবৃত্তিসহ): ID → 2, 3, 4, 4, 5

ID	Study Hours	Sleep Hours	Attendance	Marks
2	3	7	Medium	50
3	4	7	Medium	60
4	5	6	High	75
4	5	6	High	75
5	6	6	High	90

ধাপ ৩: র্যান্ডম ফিচার নির্বাচন ($\text{max_features} = 2$)

প্রতিটি নোড বিভাজনের সময় ৩টি উপলব্ধ ফিচার থেকে মাত্র ২টি ফিচার র্যান্ডমভাবে বেছে নেওয়া হয়।

- Tree 1: Study Hours, Attendance
- Tree 2: Study Hours, Sleep Hours
- Tree 3: Study Hours, Attendance

ধাপ ৪: মূল সূত্র — Squared Error (MSE)

একটি নোডে $y_1, y_2, \dots, y_n$ মান থাকলে Mean Squared Error হবে:

$$MSE = (1/n) \times \sum (y_i - \bar{y})^2$$

যেখানে $\bar{y}$ = সেই নোডের সকল টার্গেট মানের গড়।

একটি বিভাজনের Weighted MSE:

$$MSE_split = (n_left / n) \times MSE_left + (n_right / n) \times MSE_right$$

MSE হ্রাস (Information Gain):

$$Gain = MSE_parent - MSE_split$$

যে বিভাজনে Gain সর্বোচ্চ (অর্থাৎ Weighted MSE সর্বনিম্ন), সেটিই নির্বাচন করা হয়।

ধাপ ৫: Tree 1 – সম্পূর্ণ গণনা

Bootstrap Sample 1 এর ডেটা:

Study Hours	Attendance	Marks
2	Low	40
3	Medium	50
3	Medium	50
5	High	75
6	High	90

নির্বাচিত ফিচার: Study Hours, Attendance

৫.১ – Root Node MSE (Parent)

$$\bar{y}_{parent} = (40 + 50 + 50 + 75 + 90) / 5 = 305 / 5 = 61$$

$$MSE_parent = [(40-61)^2 + (50-61)^2 + (50-61)^2 + (75-61)^2 + (90-61)^2] / 5$$

$$= [(-21)^2 + (-11)^2 + (-11)^2 + (14)^2 + (29)^2] / 5$$

$$= [441 + 121 + 121 + 196 + 841] / 5$$

$$= 1720 / 5 = 344$$

$$\text{MSE}_{\text{parent}} = 344$$

৫.২ – ফিচার: Study Hours এর উপর বিভাজন পরীক্ষা

সম্ভাব্য Split Points (পরপর unique মানের মধ্যবিন্দু): 2.5, 4.0, 5.5

Split A: Study Hours ≤ 2.5

বাম দিক (Left): {40} $\rightarrow \bar{y}_L = 40$, $\text{MSE}_L = 0$

ডান দিক (Right): {50, 50, 75, 90} $\rightarrow \bar{y}_R = (50+50+75+90)/4 = 265/4 = 66.25$

$$\text{MSE}_R = [(50-66.25)^2 + (50-66.25)^2 + (75-66.25)^2 + (90-66.25)^2] / 4$$

$$= [(-16.25)^2 + (-16.25)^2 + (8.75)^2 + (23.75)^2] / 4$$

$$= [264.0625 + 264.0625 + 76.5625 + 564.0625] / 4$$

$$= 1168.75 / 4 = 292.1875$$

$$\text{MSE}_{\text{split}} = (1/5) \times 0 + (4/5) \times 292.1875 = 233.75$$

$$\text{Gain} = 344 - 233.75 = 110.25$$

Split B: Study Hours ≤ 4.0

বাম দিক: {40, 50, 50} $\rightarrow \bar{y}_L = 140/3 = 46.67$

$$\text{MSE}_L = [(40-46.67)^2 + (50-46.67)^2 + (50-46.67)^2] / 3$$

$$= [(-6.67)^2 + (3.33)^2 + (3.33)^2] / 3$$

$$= [44.49 + 11.09 + 11.09] / 3$$

$$= 66.67 / 3 = 22.22$$

ডান দিক: {75, 90} $\rightarrow \bar{y}_R = 165/2 = 82.5$

$$MSE_R = [(75-82.5)^2 + (90-82.5)^2] / 2$$

$$= [(-7.5)^2 + (7.5)^2] / 2$$

$$= [56.25 + 56.25] / 2 = \mathbf{56.25}$$

$$MSE_{split} = (3/5) \times 22.22 + (2/5) \times 56.25$$

$$= 13.33 + 22.50 = \mathbf{35.83}$$

$$\mathbf{Gain = 344 - 35.83 = 308.17} \leftarrow \text{এখন পর্যন্ত সর্বোচ্চ}$$

Split C: Study Hours ≤ 5.5

বাম দিক: {40, 50, 50, 75} $\rightarrow \bar{y}_L = 215/4 = 53.75$

$$MSE_L = [(40-53.75)^2 + (50-53.75)^2 + (50-53.75)^2 + (75-53.75)^2] / 4$$

$$= [(-13.75)^2 + (-3.75)^2 + (-3.75)^2 + (21.25)^2] / 4$$

$$= [189.0625 + 14.0625 + 14.0625 + 451.5625] / 4$$

$$= 668.75 / 4 = \mathbf{167.1875}$$

ডান দিক: {90} $\rightarrow \bar{y}_R = 90, MSE_R = 0$

$$MSE_{split} = (4/5) \times 167.1875 + (1/5) \times 0 = \mathbf{133.75}$$

$$\mathbf{Gain = 344 - 133.75 = 210.25}$$

৫.৩ – ফিচার: Attendance এর উপর বিভাজন পরীক্ষা

Split D: Attendance = Low বনাম {Medium, High}

বাম দিক (Low): {40} $\rightarrow \bar{y}_L = 40, MSE_L = 0$

ডান দিক (Medium, High): {50, 50, 75, 90} $\rightarrow \bar{y}_R = 66.25, MSE_R = 292.1875$

$$MSE_{split} = (1/5) \times 0 + (4/5) \times 292.1875 = \mathbf{233.75}$$

$$\text{Gain} = 344 - 233.75 = 110.25$$

Split E: Attendance $\in \{\text{Low, Medium}\}$ বনাম $\{\text{High}\}$

বাম দিক (Low, Medium): $\{40, 50, 50\} \rightarrow \bar{y}_L = 46.67, \text{MSE}_L = 22.22$

ডান দিক (High): $\{75, 90\} \rightarrow \bar{y}_R = 82.5, \text{MSE}_R = 56.25$

$$\text{MSE}_{\text{split}} = (3/5) \times 22.22 + (2/5) \times 56.25 = 13.33 + 22.50 = 35.83$$

$$\text{Gain} = 344 - 35.83 = 308.17 \leftarrow \text{Split B এর সমান}$$

৫.৪ – Tree 1 এর সেরা বিভাজন

Split	ফিচার	শর্ত	Gain
A	Study Hours	≤ 2.5	110.25
B	Study Hours	≤ 4.0	308.17
C	Study Hours	≤ 5.5	210.25
D	Attendance	Low বনাম বাকি	110.25
E	Attendance	$\{\text{Low, Med}\}$ বনাম High	308.17

Split B ও Split E উভয়ের Gain = 308.17 সমান। আমরা **Study Hours ≤ 4.0** নির্বাচন করি।

Root Split: Study Hours ≤ 4.0

৫.৫ – Tree 1 দ্বিতীয় স্তর: বাম নোড (Study Hours ≤ 4.0)

নমুনা: $\{40, 50, 50\} \rightarrow \bar{y} = 46.67, \text{MSE} = 22.22$

Study Hours ≤ 2.5 বিভাজন চেষ্টা:

বাম: {40} $\rightarrow$ MSE = 0

ডান: {50, 50} $\rightarrow \bar{y} = 50$, MSE = 0

MSE_split = $(1/3) \times 0 + (2/3) \times 0 = 0$

Gain = $22.22 - 0 = 22.22 \rightarrow$ নিখুঁত বিভাজন

সেরা বিভাজন: Study Hours ≤ 2.5

৫.৬ – Tree 1 দ্বিতীয় স্তর: ডান নোড (Study Hours > 4.0)

নমুনা: {75, 90} $\rightarrow \bar{y} = 82.5$, MSE = 56.25

Study Hours ≤ 5.5 বিভাজন চেষ্টা:

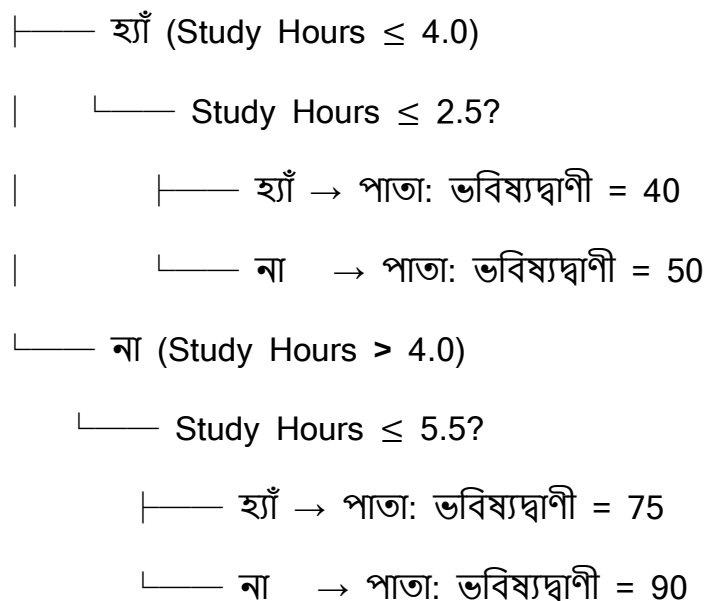
বাম: {75} $\rightarrow$ MSE = 0

ডান: {90} $\rightarrow$ MSE = 0

MSE_split = 0, Gain = **56.25** $\rightarrow$ নিখুঁত বিভাজন

৫.৭ – Tree 1: চূড়ান্ত কাঠামো

Root: Study Hours ≤ 4.0 ?



ধাপ ৬: Tree 2 – সম্পূর্ণ গণনা

Bootstrap Sample 2 এর ডেটা:

Study Hours	Sleep Hours	Marks
2	8	40
4	7	60
4	7	60
5	6	75
6	6	90

নির্বাচিত ফিচার: Study Hours, Sleep Hours

৬.১ – Root Node MSE (Parent)

$$\bar{y}_{\text{parent}} = (40 + 60 + 60 + 75 + 90) / 5 = 325 / 5 = 65$$

$$\begin{aligned}\text{MSE}_{\text{parent}} &= [(40-65)^2 + (60-65)^2 + (60-65)^2 + (75-65)^2 + (90-65)^2] / 5 \\ &= [(-25)^2 + (-5)^2 + (-5)^2 + (10)^2 + (25)^2] / 5 \\ &= [625 + 25 + 25 + 100 + 625] / 5 \\ &= 1400 / 5 = 280\end{aligned}$$

৬.২ – ফিচার: Study Hours এর উপর বিভাজন পরীক্ষা

Split Points: 3.0, 4.5, 5.5

Split A: Study Hours ≤ 3.0

বাম: {40} $\rightarrow \text{MSE}_L = 0$

ডান: {60, 60, 75, 90} $\rightarrow \bar{y}_R = 285/4 = 71.25$

$$\begin{aligned}\text{MSE}_R &= [(60-71.25)^2 + (60-71.25)^2 + (75-71.25)^2 + (90-71.25)^2] / 4 \\ &= [(-11.25)^2 + (-11.25)^2 + (3.75)^2 + (18.75)^2] / 4 \\ &= [126.5625 + 126.5625 + 14.0625 + 351.5625] / 4 \\ &= 618.75 / 4 = 154.6875\end{aligned}$$

$$\text{MSE}_{\text{split}} = (1/5) \times 0 + (4/5) \times 154.6875 = 123.75$$

$$\text{Gain} = 280 - 123.75 = 156.25$$

Split B: Study Hours ≤ 4.5

বাম: {40, 60, 60} $\rightarrow \bar{y}_L = 160/3 = 53.33$

$$\begin{aligned}\text{MSE}_L &= [(40-53.33)^2 + (60-53.33)^2 + (60-53.33)^2] / 3 \\ &= [(-13.33)^2 + (6.67)^2 + (6.67)^2] / 3\end{aligned}$$

$$= [177.69 + 44.49 + 44.49] / 3$$

$$= 266.67 / 3 = \mathbf{88.89}$$

$$\text{ডান: } \{75, 90\} \rightarrow \bar{y}_R = 82.5, \text{MSE}_R = 56.25$$

$$\text{MSE}_{\text{split}} = (3/5) \times 88.89 + (2/5) \times 56.25$$

$$= 53.33 + 22.50 = \mathbf{75.83}$$

$$\mathbf{Gain = 280 - 75.83 = 204.17} \leftarrow \text{এখন পর্যন্ত সর্বোচ্চ}$$

Split C: Study Hours ≤ 5.5

$$\text{বাম: } \{40, 60, 60, 75\} \rightarrow \bar{y}_L = 235/4 = 58.75$$

$$\text{MSE}_L = [(40-58.75)^2 + (60-58.75)^2 + (60-58.75)^2 + (75-58.75)^2] / 4$$

$$= [(-18.75)^2 + (1.25)^2 + (1.25)^2 + (16.25)^2] / 4$$

$$= [351.5625 + 1.5625 + 1.5625 + 264.0625] / 4$$

$$= 618.75 / 4 = \mathbf{154.6875}$$

$$\text{ডান: } \{90\} \rightarrow \text{MSE}_R = 0$$

$$\text{MSE}_{\text{split}} = (4/5) \times 154.6875 + (1/5) \times 0 = \mathbf{123.75}$$

$$\mathbf{Gain = 280 - 123.75 = 156.25}$$

৬.৩ – ফিচার: Sleep Hours এর উপর বিভাজন পরীক্ষা

Split Points: 6.5, 7.5

Split D: Sleep Hours ≤ 6.5

$$\text{বাম (Sleep } \leq 6.5): \{75, 90\} \rightarrow \bar{y}_L = 82.5, \text{MSE}_L = 56.25$$

$$\text{ডান (Sleep } > 6.5): \{40, 60, 60\} \rightarrow \bar{y}_R = 53.33, \text{MSE}_R = 88.89$$

$$\begin{aligned}\text{MSE}_{\text{split}} &= (2/5) \times 56.25 + (3/5) \times 88.89 \\ &= 22.50 + 53.33 = \mathbf{75.83}\end{aligned}$$

$$\text{Gain} = 280 - 75.83 = 204.17 \leftarrow \text{Split B এর সমান}$$

Split E: Sleep Hours ≤ 7.5

বাম (Sleep ≤ 7.5): {60, 60, 75, 90} $\rightarrow \bar{y}_L = 71.25$, $\text{MSE}_L = 154.6875$

ডান (Sleep > 7.5): {40} $\rightarrow \text{MSE}_R = 0$

$$\text{MSE}_{\text{split}} = (4/5) \times 154.6875 + (1/5) \times 0 = \mathbf{123.75}$$

$$\text{Gain} = 280 - 123.75 = 156.25$$

৬.৪ – Tree 2 এর সেরা বিভাজন

Split	ফিচার	শর্ত	Gain
A	Study Hours	≤ 3.0	156.25
B	Study Hours	≤ 4.5	204.17
C	Study Hours	≤ 5.5	156.25
D	Sleep Hours	≤ 6.5	204.17
E	Sleep Hours	≤ 7.5	156.25

নির্বাচিত বিভাজন: Study Hours ≤ 4.5

৬.৫ – Tree 2 দ্বিতীয় স্তর: বাম নোড (Study Hours $\leq$ 4.5)

নমুনা: {40, 60, 60} $\rightarrow \bar{y} = 53.33$, MSE = 88.89

Study Hours $\leq$ 3.0 বিভাজন:

বাম: {40} $\rightarrow$ MSE = 0 | ডান: {60, 60} $\rightarrow$ MSE = 0

MSE_split = 0, Gain = 88.89 $\rightarrow$ নিখুঁত বিভাজন

৬.৬ – Tree 2 দ্বিতীয় স্তর: ডান নোড (Study Hours $>$ 4.5)

নমুনা: {75, 90} $\rightarrow \bar{y} = 82.5$, MSE = 56.25

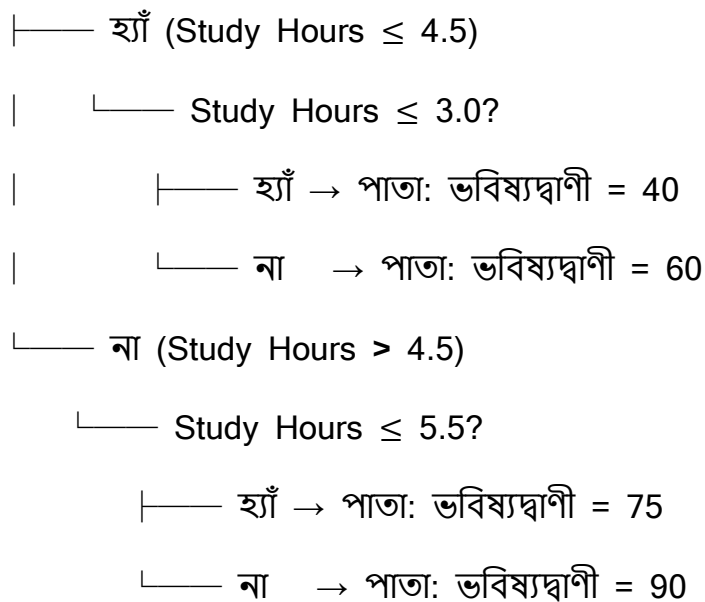
Study Hours $\leq$ 5.5 বিভাজন:

বাম: {75} $\rightarrow$ MSE = 0 | ডান: {90} $\rightarrow$ MSE = 0

MSE_split = 0, Gain = 56.25 $\rightarrow$ নিখুঁত বিভাজন

৬.৭ – Tree 2: চূড়ান্ত কাঠামো

Root: Study Hours $\leq$ 4.5?



ধাপ ৭: Tree 3 – সম্পূর্ণ গণনা

Bootstrap Sample 3 এর ডেটা:

Study Hours	Attendance	Marks
3	Medium	50
4	Medium	60
5	High	75
5	High	75
6	High	90

নির্বাচিত ফিচার: Study Hours, Attendance

৭.১ – Root Node MSE (Parent)

$$\bar{y}_{\text{parent}} = (50 + 60 + 75 + 75 + 90) / 5 = 350 / 5 = 70$$

$$\begin{aligned}\text{MSE}_{\text{parent}} &= [(50-70)^2 + (60-70)^2 + (75-70)^2 + (75-70)^2 + (90-70)^2] / 5 \\ &= [(-20)^2 + (-10)^2 + (5)^2 + (5)^2 + (20)^2] / 5 \\ &= [400 + 100 + 25 + 25 + 400] / 5 \\ &= 950 / 5 = 190\end{aligned}$$

৭.২ – ফিচার: Study Hours এর উপর বিভাজন পরীক্ষা

Split Points: 3.5, 4.5, 5.5

Split A: Study Hours $\leq$ 3.5

বাম: {50} $\rightarrow$ MSE_L = 0

ডান: {60, 75, 75, 90} $\rightarrow \bar{y}_R = 300/4 = 75$

$$\begin{aligned} \text{MSE}_R &= [(60-75)^2 + (75-75)^2 + (75-75)^2 + (90-75)^2] / 4 \\ &= [225 + 0 + 0 + 225] / 4 \\ &= 450 / 4 = \mathbf{112.5} \end{aligned}$$

$$\text{MSE}_{\text{split}} = (1/5) \times 0 + (4/5) \times 112.5 = \mathbf{90}$$

$$\mathbf{\text{Gain} = 190 - 90 = 100}$$

Split B: Study Hours ≤ 4.5

$$\text{বাম: } \{50, 60\} \rightarrow \bar{y}_L = 55$$

$$\text{MSE}_L = [(50-55)^2 + (60-55)^2] / 2 = [25 + 25] / 2 = \mathbf{25}$$

$$\text{ডান: } \{75, 75, 90\} \rightarrow \bar{y}_R = 240/3 = 80$$

$$\begin{aligned} \text{MSE}_R &= [(75-80)^2 + (75-80)^2 + (90-80)^2] / 3 \\ &= [25 + 25 + 100] / 3 \\ &= 150 / 3 = \mathbf{50} \end{aligned}$$

$$\text{MSE}_{\text{split}} = (2/5) \times 25 + (3/5) \times 50 = 10 + 30 = \mathbf{40}$$

$$\mathbf{\text{Gain} = 190 - 40 = 150} \leftarrow \text{এখন পর্যন্ত সর্বোচ্চ}$$

Split C: Study Hours ≤ 5.5

$$\text{বাম: } \{50, 60, 75, 75\} \rightarrow \bar{y}_L = 260/4 = 65$$

$$\begin{aligned} \text{MSE}_L &= [(50-65)^2 + (60-65)^2 + (75-65)^2 + (75-65)^2] / 4 \\ &= [225 + 25 + 100 + 100] / 4 \\ &= 450 / 4 = \mathbf{112.5} \end{aligned}$$

$$\text{ডান: } \{90\} \rightarrow \text{MSE}_R = 0$$

$$\text{MSE}_{\text{split}} = (4/5) \times 112.5 + (1/5) \times 0 = \mathbf{90}$$

$$\text{Gain} = 190 - 90 = 100$$

৭.৩ – ফিচার: Attendance এর উপর বিভাজন পরীক্ষা

Split D: Attendance = Medium বনাম High

বাম (Medium): {50, 60} $\rightarrow \bar{y}_L = 55$, $\text{MSE}_L = 25$

ডান (High): {75, 75, 90} $\rightarrow \bar{y}_R = 80$, $\text{MSE}_R = 50$

$$\text{MSE}_{\text{split}} = (2/5) \times 25 + (3/5) \times 50 = 10 + 30 = 40$$

$$\text{Gain} = 190 - 40 = 150 \leftarrow \text{Split B এর সমান}$$

৭.৪ – Tree 3 এর সেরা বিভাজন

Split	ফিচার	শর্ত	Gain
A	Study Hours	≤ 3.5	100
B	Study Hours	≤ 4.5	150
C	Study Hours	≤ 5.5	100
D	Attendance	Medium বনাম High	150

নির্বাচিত বিভাজন: Study Hours ≤ 4.5

৭.৫ – Tree 3 দ্বিতীয় স্তর: বাম নোড (Study Hours ≤ 4.5)

নমুনা: {50, 60} $\rightarrow \bar{y} = 55$, $\text{MSE} = 25$

Study Hours ≤ 3.5 বিভাজন:

বাম: {50} $\rightarrow \text{MSE} = 0$ | ডান: {60} $\rightarrow \text{MSE} = 0$

$\text{MSE}_{\text{split}} = 0$, $\text{Gain} = 25 \rightarrow$ নিখুঁত বিভাজন

৭.৬ – Tree 3 দ্বিতীয় স্তর: ডান নোড (Study Hours > 4.5)

নমুনা: {75, 75, 90} → $\bar{y} = 80$, MSE = 50

Study Hours ≤ 5.5 বিভাজন:

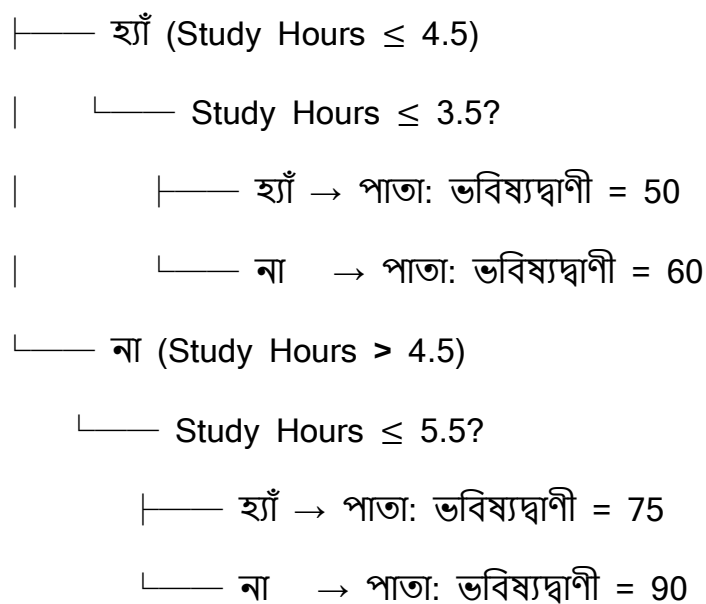
বাম: {75, 75} → $\bar{y} = 75$, MSE = 0

ডান: {90} → MSE = 0

MSE_split = 0, Gain = 50 → নিখুঁত বিভাজন

৭.৭ – Tree 3: চূড়ান্ত কাঠামো

Root: Study Hours ≤ 4.5 ?



ধাপ ৮: নতুন ডেটার জন্য ভবিষ্যদ্বাণী

নতুন ইনপুট: Study Hours = 4, Sleep Hours = 7 (Tree 2 এর জন্য ধরা হয়েছে),
Attendance = High

Tree 1 এর ভবিষ্যদ্বাণী (ফিচার: Study Hours, Attendance)

- Root: Study Hours ≤ 4.0 ? $\rightarrow 4 \leq 4.0 \rightarrow$ হ্যাঁ, বাম দিকে যাও
- নোড: Study Hours ≤ 2.5 ? $\rightarrow 4 \leq 2.5 \rightarrow$ না, ডান দিকে যাও
- পাতা: ভবিষ্যদ্বাণী = 50

Tree 2 এর ভবিষ্যদ্বাণী (ফিচার: Study Hours, Sleep Hours)

- Root: Study Hours ≤ 4.5 ? $\rightarrow 4 \leq 4.5 \rightarrow$ হ্যাঁ, বাম দিকে যাও
- নোড: Study Hours ≤ 3.0 ? $\rightarrow 4 \leq 3.0 \rightarrow$ না, ডান দিকে যাও
- পাতা: ভবিষ্যদ্বাণী = 60

Tree 3 এর ভবিষ্যদ্বাণী (ফিচার: Study Hours, Attendance)

- Root: Study Hours ≤ 4.5 ? $\rightarrow 4 \leq 4.5 \rightarrow$ হ্যাঁ, বাম দিকে যাও
- নোড: Study Hours ≤ 3.5 ? $\rightarrow 4 \leq 3.5 \rightarrow$ না, ডান দিকে যাও
- পাতা: ভবিষ্যদ্বাণী = 60

ধাপ ৯: চূড়ান্ত ভবিষ্যদ্বাণী – গড় গণনা (Averaging)

গুরুত্বপূর্ণ নোট: Regressor এর ক্ষেত্রে Random Forest Majority Voting ব্যবহার করে না (সেটি Classifier এর জন্য)। Regression এ সকল ট্রির ভবিষ্যদ্বাণীর গড় নেওয়া হয়।

ট্রি	ভবিষ্যদ্বাণী
Tree 1	50
Tree 2	60

Tree 3	60
--------	----

চূড়ান্ত ভবিষ্যদ্বাণী = $(50 + 60 + 60) / 3 = 170 / 3 \approx 56.67$

নতুন শিক্ষার্থীর প্রাপ্ত নম্বরের ভবিষ্যদ্বাণী ≈ 56.67

Classifier:

n_estimators

- **Default:** ১০০
- **কাজ কী:** র্যান্ডম ফরেস্ট মডেলের ভেতরে মোট কতগুলো ডিসিশন ট্রি (Decision Tree) বা সিদ্ধান্ত-গাছ তৈরি হবে, তা এটি ঠিক করে।
- **অন্যান্য অপশন:** যেকোনো পজিটিভ পূর্ণসংখ্যা (যেমন: ৫০, ২০০, ৫০০)।
- **কখন কোনটি ব্যবহার করব:** যদি ডেটাসেট অনেক ছোট হয় বা কম্পিউটারের র‍্যাম/প্রসেসর কম শক্তির হয়, তবে এটি কমিয়ে ৫০ করা যায়। আর যদি ডেটাসেট অনেক বড় হয় এবং আপনার সর্বোচ্চ এক্যুরেসী বা স্থায়িত্বের প্রয়োজন পড়ে, তবে এটি বাড়িয়ে ৩০০ বা ৫০০ করা উচিত।

max_features

- **Default:** "sqrt" (মোট ফিচারের সংখ্যার বর্গমূল)
- **কাজ কী:** গাছের প্রতিটা নোড স্প্লিট বা ভাগ করার সময় র্যান্ডমলি সর্বোচ্চ কয়টি ফিচার (কলাম) স্ক্যান করা হবে, তা এটি নিয়ন্ত্রণ করে।
- **অন্যান্য অপশন:** "log2" (লগ বেস ২), কোনো নির্দিষ্ট সংখ্যা (যেমন: ৩, ৫) বা ভগ্নাংশ (যেমন: ০.৫ অর্থাৎ মোট ফিচারের ৫০%)। এছাড়া None দিলে প্রতিবার সব ফিচার স্ক্যান করে।
- **কখন কোনটি ব্যবহার করব:** ডেটাসেটে যদি শত শত কলাম (High Dimensionality) থাকে, তবে ডিফল্ট "sqrt" বা "log2" রাখাই সবচেয়ে বেস্ট, কারণ এটি গাছগুলোর মধ্যে বৈচিত্র্য বাড়ায়। কিন্তু যদি কলামের সংখ্যা খুব কম হয় (যেমন: মাত্র ৩-৪টি কলাম), তখন এটি None রাখা ভালো যাতে মডেল সব কলাম দেখার সুযোগ পায়।

max_depth

- **Default:** None
- **কাজ কী:** জঙ্গলের প্রতিটি একক গাছের সর্বোচ্চ গভীরতা বা লেভেল কতটুকু হতে পারবে, তা এটি বেঁধে দেয়। None থাকলে গাছগুলো একদম শেষ সীমা পর্যন্ত বাড়তে থাকে।
- **অন্যান্য অপশন:** যেকোনো পূর্ণসংখ্যা (যেমন: ৫, ১০, ১৫)।
- **কখন কোনটি ব্যবহার করব:** যদি দেখেন মডেলটি ট্রেনিং ডেটায় চমৎকার রেজাল্ট দিলেও টেস্ট ডেটায় ফেইল করছে (ওভারফিটিং হচ্ছে), তখন গাছের এই অনিয়ন্ত্রিত বৃদ্ধি থামাতে এখানে একটি নির্দিষ্ট গভীরতা (যেমন: max_depth=10 বা 12) সেট করতে হবে।

min_samples_split

- **Default:** ২
- **কাজ কী:** একটি নোডকে ভেঙে নতুন দুটি ডাল তৈরি করার জন্য ওই নোডে নূন্যতম কয়টি ডেটা স্যাম্পল থাকতে হবে, তা এটি ঠিক করে।
- **অন্যান্য অপশন:** যেকোনো পূর্ণসংখ্যা (যেমন: ৫, ১০, ৫০) বা ভগ্নাংশ।
- **কখন কোনটি ব্যবহার করব:** যখন মডেলের ওভারফিটিং বন্ধ করার প্রয়োজন পড়ে। যদি আপনি এখানে min_samples_split=10 দেন, তবে কোনো নোডে ১০টির কম ডেটা থাকলে গাছ সেটিকে আর ভাঙবে না, ওখানেই থামিয়ে দেবে। বড় এবং নয়েজি ডেটাসেটে এর মান বাড়িয়ে দেওয়া উচিত।

min_samples_leaf

- **Default:** ১
- **কাজ কী:** একটি গাছের একদম শেষ পাতা বা লিফ নোডে (Leaf Node) নূন্যতম কয়টি ডেটা স্যাম্পল থাকতেই হবে, তা ফিক্স করে।
- **অন্যান্য অপশন:** যেকোনো পূর্ণসংখ্যা (যেমন: ২, ৫, ১০) বা ভগ্নাংশ।

- **কখন কোনটি ব্যবহার করব:** এটিও ওভারফিটিং কমানোর এবং মডেলের আউটপুটকে মসৃণ বা স্মুথ করার জন্য ব্যবহৃত হয়। যদি ডেটাসেটে অনেক আউটলায়ার বা ভুল ডেটা থাকে, তবে এর মান বাড়িয়ে ৫ বা ১০ করে দিলে ক্ষুদ্রাতিক্ষুদ্র ও অর্থহীন প্যাটার্নের জন্য নতুন কোনো পাতা তৈরি হওয়া বন্ধ হয়ে যায়।

bootstrap

- **Default:** True
- **কাজ কী:** প্রতিটি গাছ ট্রেন করার সময় বুটস্ট্রাপ স্যাম্পলিং (Row sampling with replacement) পদ্ধতি ব্যবহার করা হবে কি না, তা এটি নির্ধারণ করে।
- **অন্যান্য অপশন:** False
- **কখন কোনটি ব্যবহার করব:** এটি সবসময় True রাখাই র্যান্ডম ফরেস্টের মূল নিয়ম। তবে যদি আপনার ডেটাসেট অত্যন্ত নিখুঁত হয় এবং আপনি চান প্রতিটি গাছ পুরো ডেটাসেটটি দেখেই তৈরি হোক (কোনো রো বাদ না পড়ুক), শুধুমাত্র তখনই এটি False করা হয়।

oob_score

- **Default:** False
- **কাজ কী:** বুটস্ট্রাপ স্যাম্পলিংয়ের সময় যে ৩৬.৮% ডেটা ট্রেনিং থেকে বাদ পড়ে যায়, সেগুলো ব্যবহার করে মডেলের ট্রেনিং চলাকালীনই একটি ইন্টারনাল ভ্যালিডেশন স্কোর হিসাব করা হবে কি না, তা এটি ঠিক করে।
- **অন্যান্য অপশন:** True
- **কখন কোনটি ব্যবহার করব:** যখন আপনার ডেটাসেট ছোট এবং আপনি ভ্যালিডেশনের জন্য আলাদা করে মূল্যবান ডেটা নষ্ট করতে চান না (যেমন: train_test_split বা K-Fold না করে)। তখন oob_score=True করে দিলে আলাদা টেস্ট সেট ছাড়াই মডেলের রিয়েল এক্যুরেসীর ধারণা পাওয়া যায়।

n_jobs

- **Default:** None (কম্পিউটারের মাত্র ১টি প্রসেসর কোর ব্যবহার করে)
- **কাজ কী:** মডেল ট্রেনিং ও প্রেডিকশনের সময় ব্যাকগ্রাউন্ডে সিপিইউ-এর কয়টি কোর সমান্তরালভাবে কাজ করবে, তা এটি নিয়ন্ত্রণ করে।
- **অন্যান্য অপশন:** যেকোনো পূর্ণসংখ্যা বা -1।
- **কখন কোনটি ব্যবহার করব:** বাস্তব জীবনের বড় প্রজেক্টে সবসময় এটি -1 ব্যবহার করা উচিত। n_jobs=-1 দিলে এটি কম্পিউটারের সবগুলো প্রসেসর কোরকে একসাথে কাজে লাগিয়ে শত শত গাছ ট্রেন করার প্রক্রিয়াটিকে সুপার-ফাস্ট করে তোলে।

random_state

- **Default:** None
- **কাজ কী:** বুটস্ট্রাপ স্যাম্পলিং এবং ফিচার সিলেকশনের র্যান্ডম বা এলোমেলো প্রক্রিয়াটিকে একটি নির্দিষ্ট গাণিতিক বীজ দিয়ে লক করে দেয়।
- **অন্যান্য অপশন:** যেকোনো ফিক্সড পূর্ণসংখ্যা (যেমন: 0, 42)।
- **কখন কোনটি ব্যবহার করব:** টিম প্রজেক্ট, অ্যাসাইনমেন্ট বা কোড পরীক্ষার সময় এটি যেকোনো একটি ফিক্সড সংখ্যায় সেট করা বাধ্যতামূলক। এটি নিশ্চিত করে যে কোডটি আপনার কম্পিউটারে বা অন্য কারো কম্পিউটারে যতবারই রান করা হোক না কেন, প্রতিবার অবিকল একই গাছ এবং একই এক্যুরেসী স্কোর পাওয়া যাবে।

class_weight

- **Default:** None (সব ক্লাসকে সমান গুরুত্ব দেয়)
- **কাজ কী:** ইমব্যালেন্সড ডেটাসেটের ক্ষেত্রে ছোট বা মাইনোরিটি ক্লাসকে অতিরিক্ত গাণিতিক গুরুত্ব বা ওজন দেওয়ার জন্য এটি ব্যবহৃত হয়।
- **অন্যান্য অপশন:** "balanced" অথবা কাস্টম ডিকশনারি।
- **কখন কোনটি ব্যবহার করব:** যখন ডেটা প্রবলভাবে ইমব্যালেন্সড (যেমন: ক্রেডিট কার্ড জালিয়াতির ডেটা মাত্র ১% আর নরমাল ডেটা ৯৯%)। তখন

`class_weight='balanced'` দিলে মডেল স্বয়ংক্রিয়ভাবে ওই ১% ছোট ক্লাসকে বেশি প্রাধান্য দেয়, যার ফলে মডেলের প্রেডিকশন কোয়ালিটি বা রিকল (*Recall*) উন্নত হয়।

Regressor:

criterion

- **Default:** "squared_error"
- **কাজ কী:** রিগ্রেশন সমস্যার ক্ষেত্রে প্রতিটি ডিসিশন ট্রির নোড ভাগ বা স্প্লিট করার কোয়ালিটি মাপার জন্য ব্যাকএন্ডে কোন গাণিতিক লস ফাংশন (Loss Function) বা পরিমাপক ব্যবহার করা হবে, তা এটি নির্ধারণ করে।
- **অন্যান্য অপশন:** "absolute_error", "friedman_mse", "poisson"
- **কখন কোনটি ব্যবহার করব:**
 - **"squared_error" (Mean Squared Error - MSE):** এটি আপনার ডেটাসেটে থাকা বড় বড় ভুলগুলোকে (Large Errors) বেশি গাণিতিক শাস্তি বা গুরুত্ব দেয়। ডেটা যদি সাধারণ বা নরমাল ডিস্ট্রিবিউশন মেনে চলে, তবে এটি সবচেয়ে দ্রুত এবং সেরা ফলাফল দেয়।
 - **"absolute_error" (Mean Absolute Error - MAE):** এটি প্রতিটি ভুলের পরম মান হিসাব করে। আপনার ডেটাসেটে যদি প্রচুর আউটলায়ার (Outliers) বা অস্বাভাবিক বড়/ছোট মান থাকে, তখন এই অপশনটি ব্যবহার করা উচিত। কারণ এটি আউটলায়ারের প্রতি অত্যন্ত প্রতিরোধী (Robust)। তবে ব্যাকএন্ডে এটি প্রসেস হতে MSE-এর চেয়ে অনেক বেশি সময় নেয়।
 - **"friedman_mse":** এটি মূলত জিবিএম (Gradient Boosting) অ্যালগরিদমের উদ্ভাবক জেরোম ফ্রিডম্যানের তৈরি একটি বিশেষ পরিমাপক। এটি সাধারণ MSE-এর ওপর ভিত্তি করেই কাজ করে, তবে কিছু বিশেষ ক্ষেত্রে এটি ডালপালা গজানোর জন্য আরও নিখুঁত থ্রেশহোল্ড খুঁজে বের করতে পারে।

- **"poisson":** যখন আপনার টার্গেট কলামের ডেটা কোনো কাউন্ট বা গণনা নির্দেশ করে (যেমন: "আজকে দোকানে কতজন কাস্টমার আসবে" বা "এই রাস্তায় প্রতি ঘণ্টায় কয়টি গাড়ি চলে") এবং ডেটাটি পয়সঁ ডিস্ট্রিবিউশন (Poisson Distribution) মেনে চলে, তখন এই ক্রাইটেরিয়নটি ব্যবহার করলে লস সবচেয়ে কম হয়।

max_features

- **Default:** 1.0 (অর্থাৎ, মোট ফিচারের ১০০% বা সবগুলোই স্ক্যান করবে)
- **কাজ কী:** রিগ্রেশন গাছের প্রতিটা নোড স্প্লিট বা ভাগ করার সময় র্যান্ডমলি সর্বোচ্চ কয়টি কলাম বা ফিচার বিবেচনা করা হবে, তা এটি নিয়ন্ত্রণ করে।
- **অন্যান্য অপশন:** "sqrt", "log2", কোনো নির্দিষ্ট সংখ্যা (int) বা ভগ্নাংশ (float)।
- **কখন কোনটি ব্যবহার করব:** ক্লাসিফায়ারের ডিফল্ট ছিল "sqrt", কিন্তু রিগ্রেশনের ডিফল্ট হলো 1.0। এর মানে হলো ডিফল্ট অবস্থায় রিগ্রেশনের প্রতিটি গাছ নোড ভাগের সময় সব ফিচার চেক করার সুযোগ পায়। তবে আপনার ডেটাসেটে যদি ফিচারের সংখ্যা অনেক বেশি হয় এবং গাছগুলোর মধ্যে বৈচিত্র্য (Diversity) বাড়িয়ে ওভারফিটিং কমাতে চান, তবে এটি পরিবর্তন করে কাস্টম ফ্লোট মান (যেমন: 0.3 বা ৩০%) অথবা ক্লাসিফায়ারের মতো "sqrt" সেট করা অত্যন্ত কার্যকর।

RandomForestClassifier() Attributes

feature_importances_

Default: Created after fit()

সবগুলো Decision Tree-এর Feature Importance-এর Average মান সংরক্ষণ করে। এটি দেখায় কোন Feature Prediction করার ক্ষেত্রে সবচেয়ে বেশি গুরুত্বপূর্ণ। সব Importance Score-এর যোগফল 1 হয়।

estimators_

Default: Created after fit()

Forest-এর ভিতরে থাকা প্রতিটি Trained Decision Tree-এর List সংরক্ষণ করে। প্রতিটি Element একটি আলাদা DecisionTreeClassifier Object।

oob_score_

Default: Created after fit() *(Only when oob_score=True)*

Out-of-Bag (OOB) Sample ব্যবহার করে Model-এর Accuracy Score সংরক্ষণ করে। এটি আলাদা Validation Set ছাড়াই Model-এর Performance মূল্যায়ন করতে সাহায্য করে।

oob_decision_function_

Default: Created after fit() *(Only when oob_score=True)*

প্রতিটি Training Sample-এর জন্য Out-of-Bag Sample ব্যবহার করে Predicted Probability সংরক্ষণ করে। এটি শুধুমাত্র Classification-এর ক্ষেত্রে পাওয়া যায়।

classes_

Default: Created after fit()

Training Data-তে থাকা সকল Target Class-এর List সংরক্ষণ করে।

n_classes_

Default: Created after fit()

Training Data-তে মোট কতটি Target Class রয়েছে তা সংরক্ষণ করে।

estimators_samples_

Default: Created after fit()

প্রতিটি Decision Tree কোন কোন Training Sample (Bootstrap Sample) ব্যবহার করে Train হয়েছে, তাদের Index সংরক্ষণ করে।

n_features_in_

Default: Created after fit()

Training-এর সময় Model মোট কতটি Input Feature ব্যবহার করেছে তা সংরক্ষণ করে।

feature_names_in_

Default: Created after fit() (*DataFrame input only*)

Training Data যদি Pandas DataFrame হয়, তাহলে Feature-গুলোর নাম সংরক্ষণ করে।

n_outputs_

Default: Created after fit()

Model মোট কতটি Target Output Predict করেছে তা সংরক্ষণ করে। সাধারণ Classification-এ এটি সাধারণত 1 হয়।

RandomForestRegressor() Attributes

feature_importances_

Default: Created after fit()

সবগুলো Decision Tree-এর Feature Importance-এর Average মান সংরক্ষণ করে। এটি দেখায় কোন Feature Regression Prediction-এর জন্য সবচেয়ে বেশি গুরুত্বপূর্ণ।

estimators_

Default: Created after fit()

Forest-এর ভিতরে থাকা প্রতিটি Trained Decision Tree-এর List সংরক্ষণ করে। প্রতিটি Element একটি DecisionTreeRegressor Object।

oob_score_

Default: Created after fit() (*Only when oob_score=True*)

Out-of-Bag (OOB) Sample ব্যবহার করে Model-এর R^2 Score সংরক্ষণ করে। এটি Model-এর Generalization Performance মূল্যায়ন করতে সাহায্য করে।

estimators_samples_

Default: Created after fit()

প্রতিটি Decision Tree কোন কোন Bootstrap Sample ব্যবহার করে Train হয়েছে, তাদের Index সংরক্ষণ করে।

n_features_in_

Default: Created after fit()

Training-এর সময় ব্যবহৃত মোট Input Feature-এর সংখ্যা সংরক্ষণ করে।

feature_names_in_

Default: Created after fit() (*DataFrame input only*)

Training Data যদি Pandas DataFrame হয়, তাহলে Feature-গুলোর নাম সংরক্ষণ করে।

n_outputs_

Default: Created after fit()

Model মোট কতটি Target Output Predict করছে তা সংরক্ষণ করে। সাধারণ Regression-এ এটি সাধারণত 1 হয়।

Code:

Classification:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.pipeline import Pipeline
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix, ConfusionMatrixDisplay

digits = load_digits()
X = pd.DataFrame(digits.data)
y = pd.Series(digits.target)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

rf_pipeline = Pipeline(
    steps=[
        ('model', RandomForestClassifier(random_state=42, n_jobs=-1))
    ]
)
```

```

param_grid = {
    'model__n_estimators': [50, 100, 200],
    'model__max_features': ['sqrt', 'log2'],
    'model__max_depth': [10, 20, None],
    'model__min_samples_split': [2, 5]
}

grid_search = GridSearchCV(
    estimator=rf_pipeline,
    param_grid=param_grid,
    cv=5,
    scoring='accuracy',
    n_jobs=-1
)

grid_search.fit(X_train, y_train)

print("--- GridSearchCV Tuning Results ---")
print("Best Parameters Found:", grid_search.best_params_)
print(f"Best CV Accuracy: {grid_search.best_score_:.4f}\n")

best_rf_pipeline = grid_search.best_estimator_
y_pred = best_rf_pipeline.predict(X_test)

print("--- Final Model Performance on Test Set ---")
print(f"Test Accuracy: {accuracy_score(y_test, y_pred):.4f}\n")
print(classification_report(y_test, y_pred))

cm = confusion_matrix(y_test, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=digits.target_names)
disp.plot(cmap=plt.cm.Blues)
plt.title("Random Forest Confusion Matrix - Digits Dataset")
plt.show()

```


Code Regressor:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.pipeline import Pipeline
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error, r2_score, mean_absolute_error

housing = fetch_california_housing()
X = pd.DataFrame(housing.data, columns=housing.feature_names)
y = pd.Series(housing.target)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

rf_reg_pipeline = Pipeline(
    steps=[
        ('model', RandomForestRegressor(random_state=42, n_jobs=-1))
    ]
)
```

```

param_grid = {
    'model__n_estimators': [50, 100],
    'model__criterion': ['squared_error', 'absolute_error'],
    'model__max_features': [0.5, 'sqrt'],
    'model__max_depth': [10, None]
}

grid_search = GridSearchCV(
    estimator=rf_reg_pipeline,
    param_grid=param_grid,
    cv=3,
    scoring='neg_mean_squared_error',
    n_jobs=-1
)

grid_search.fit(X_train, y_train)

print("--- GridSearchCV Tuning Results ---")
print("Best Parameters Found:", grid_search.best_params_)

best_rf_reg_pipeline = grid_search.best_estimator_
y_pred = best_rf_reg_pipeline.predict(X_test)

mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
mae = mean_absolute_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print("--- Final Model Performance on Test Set ---")
print(f"Mean Absolute Error (MAE): {mae:.4f}")
print(f"Mean Squared Error (MSE): {mse:.4f}")
print(f"Root Mean Squared Error (RMSE): {rmse:.4f}")
print(f"R-squared Score (R2 Score): {r2:.4f}\n")

plt.figure(figsize=(8, 6))
plt.scatter(y_test, y_pred, alpha=0.3, color='crimson')
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'k--', lw=2)
plt.xlabel('Actual Price')
plt.ylabel('Predicted Price')
plt.title('Actual vs Predicted Prices - California Housing')
plt.show()

```

Interview Questions

Q1: Out-of-Bag (OOB) কী এবং এটি ব্যবহারের সুবিধা কী?

- **Answer:** র্যান্ডম ফরেস্টে বুটস্ট্রাপ স্যাম্পলিং করার সময় প্রতিস্থাপনের নীতির কারণে প্রতিটি গাছের ট্রেনিং সেট থেকে গড়ে প্রায় ৩৬.৮% ডেটা বাদ পড়ে যায়। এই বাদ পড়া ডেটাপয়েন্টগুলোকে **Out-of-Bag (OOB)** স্যাম্পল বলা হয়। মডেল ট্রেনিং শেষ হওয়ার পর, প্রতিটি গাছকে তার নিজস্ব ওওবি স্যাম্পল দিয়ে টেস্ট করে যে গড় স্কোর পাওয়া যায়, সেটিই ওওবি স্কোর। এর বড় সুবিধা হলো—এটি ব্যবহার করলে আলাদা করে কোনো টেস্ট সেট বা ক্রস-ভ্যালিডেশন (K-Fold CV) ছাড়াই মডেলের জেনারেলাইজেশন ক্ষমতা বা রিয়েল-টাইম এক্যুরেসী নিখুঁতভাবে পরিমাপ করা যায়।

Q2: Random Forest-কে কেন একটি 'Black-Box' মডেল বলা হয়?

- **Answer:** একটি একক ডিসিশন ট্রিকে খুব সহজে গ্রাফ বা ডায়াগ্রামের সাহায্যে ভিজ্যুয়ালাইজ করা যায় এবং এর প্রতিটি শর্ত (if-else রুলস) সহজে ট্র্যাকিং করা যায় (White-Box)। কিন্তু র্যান্ডম ফরেস্ট যখন ১০০ থেকে ৫০০টি গভীর গাছ একসাথে কাজ করে, তখন প্রতিটি গাছের নিজস্ব র্যান্ডম ফিচার ও র্যান্ডম ডেটা স্যাম্পল থাকে। কোনো একটি নির্দিষ্ট প্রেডিকশনের জন্য ব্যাকএন্ডে ৫০০টি গাছের হাজার হাজার জটিল শর্ত ও ভোটিং মেকানিজম একসাথে ট্র্যাক করা মানুষের পক্ষে অসম্ভব হয়ে পড়ে। গাণিতিক লজিক স্পষ্টভাবে ব্যাখ্যা করা যায় না বলেই একে **Black-Box** মডেল বলা হয়।

Q3: Random Forest-এর দুটি গাছ কি কখনো হুবহু এক হতে পারে? যদি না হয়, তবে কেন?

- **Answer:** তাত্ত্বিকভাবে বা বাস্তবে শত শত গাছের মধ্যে দুটি গাছ হুবহু এক হওয়া প্রায় অসম্ভব। এর পেছনে দুটি মূল কারণ রয়েছে: প্রথমত, **Bootstrap Sampling**-এর কারণে প্রতিটি গাছ সম্পূর্ণ আলাদা ও এলোমেলো ডেটার সাবসেট পায়। দ্বিতীয়ত এবং সবচেয়ে বড় কারণ হলো **Feature Randomness**; প্রতিটি নোড ভাগার সময় গাছগুলো সব কলাম দেখতে পায় না, বরং সম্পূর্ণ র্যান্ডমলি সিলেক্টেড কিছু কলাম (যেমন: $max_features = \sqrt{d}$) থেকে সেরা স্প্লিট বেছে নেয়। এই দ্বিমুখী র্যান্ডমনেসের কারণে জঙ্গলের প্রতিটি গাছের গঠন একে অপরের থেকে সম্পূর্ণ ভিন্ন ও স্বাধীন হয়।

Q4: আপনার ডেটাসেটে যদি প্রচুর আউটলায়ার (Outliers) থাকে, তবে রিগ্রেশনের ক্ষেত্রে আপনি কোন Criterion ব্যবহার করবেন এবং কেন?

- **Answer:** ডেটাসেটে প্রচুর আউটলায়ার থাকলে আমি criterion='absolute_error' (Mean Absolute Error - MAE) ব্যবহার করব। ডিফল্ট ক্রাইটেরিয়ন squared_error (MSE) ভুলের মানকে বর্গ (Square) করে হিসাব করে, যার ফলে একটি বড় আউটলায়ারের ভুল পুরো মডেলের ওপর বিশাল নেতিবাচক প্রভাব ফেলে। অন্যদিকে, MAE ভুলের পরম মান (Absolute value) নেয়, যা আউটলায়ারের প্রতি অত্যন্ত প্রতিরোধী (**Robust to outliers**) এবং মডেলের চূড়ান্ত গড় সিদ্ধান্তকে সহজে বিচ্যুত হতে দেয় না।

Q5: Random Forest কি ওভারফিট করতে পারে? যদি করে, তবে তা কীভাবে সমাধান করবেন?

- **Answer:** সাধারণ ডিসিশন ট্রির মতো র্যান্ডম ফরেস্ট সহজে ওভারফিট করে না, কারণ ব্যাগিং বা এভারেজিং প্রক্রিয়াটি ভ্যারিয়েন্স কমিয়ে আনে। তবে ডেটাসেটে যদি চরম মাত্রায় নয়েজ থাকে এবং হাইপারপ্যারামিটার নিয়ন্ত্রণ না করা হয়, তবে এটিও ওভারফিট করতে পারে। এর প্রধান সমাধানগুলো হলো:
- max_depth-এর মান একটি নির্দিষ্ট সংখ্যায় (যেমন: ১০ বা ১৫) বেঁধে দেওয়া।
- min_samples_leaf-এর মান বাড়িয়ে (যেমন: ৫ বা ১০) শেষ পাতার নূন্যতম ডেটা ফিল্প করা।
- n_estimators বা জঙ্গলে গাছের সংখ্যা বাড়িয়ে মডেলকে আরও সুষম করা।

Summary

- **Core Concept:** র্যান্ডম ফরেস্ট হলো একটি অত্যন্ত শক্তিশালী এবং স্থিতিশীল এনসেম্বল লার্নিং অ্যালগরিদম, যা বুটস্ট্র্যাপ স্যাম্পলিং এবং ফিচারের র্যান্ডমনেস ব্যবহার করে শত শত ডিসিশন ট্রির সমন্বয়ে জঙ্গল তৈরি করে।
- **Mechanism:** এটি ক্লাসিফিকেশনের জন্য মেজরিটি ভোটিং (Majority Voting) এবং রিগ্রেশনের জন্য গড় (Averaging) মেথড ব্যবহার করে চূড়ান্ত প্রেডিকশন দেয়।

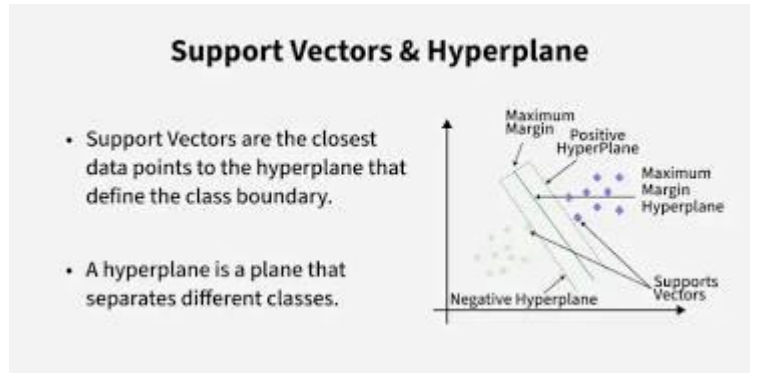
- **Strength:** এর সবচেয়ে বড় শক্তি হলো এটি কোনো প্রকার ফিচার স্কেলিং ছাড়াই র-ডেটাসেটে চমৎকার কাজ করে এবং ওভারফিটিংয়ের হাত থেকে মডেলকে স্বয়ংক্রিয়ভাবে রক্ষা করে।
- **Trade-off:** উচ্চমাত্রার এক্যুরেসী এবং স্ট্যাবিলিটি দিলেও, এটি মেমোরিতে অনেক বেশি জায়গা নেয় এবং এর ব্ল্যাক-বক্স প্রকৃতির কারণে মডেলের ভেতরের লজিক পুরোপুরি ব্যাখ্যা করা যায় না।

Support Vector Machine

সাপোর্ট ভেক্টর মেশিন (SVM) হলো একটি অত্যন্ত শক্তিশালী ও গাণিতিকভাবে সুসংহত সুপারভাইজড মেশিন লার্নিং অ্যালগরিদম। এর প্রধান উদ্দেশ্য হলো উচ্চ-মাত্রা বিশিষ্ট উপাত্ত স্পেসে (High-Dimensional Space) এমন একটি সর্বোত্তম সিদ্ধান্ত সীমানা বা Hyperplane নির্ধারণ করা, যা বিভিন্ন শ্রেণীভুক্ত উপাত্ত বিন্দুগুলোকে (Data Points) পরস্পরের থেকে সর্বোচ্চ দূরত্বে বা Maximum Margin-এ পৃথক করতে পারে।

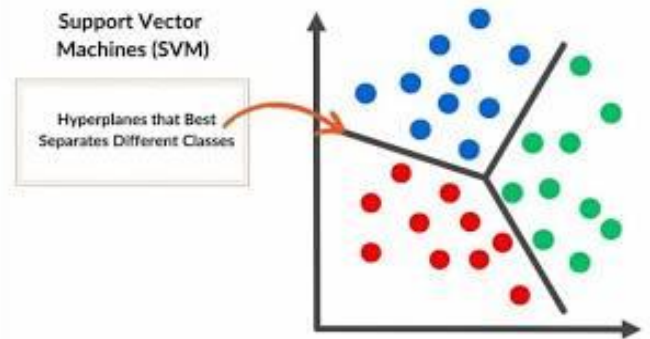
সাপোর্ট ভেক্টর (Support Vectors):

হাইপারপ্লেনের অবস্থান ও অভিযোজন (Orientation) নির্ধারণের জন্য যে নির্দিষ্ট উপাত্ত বিন্দুগুলো সরাসরি দায়ী, সেগুলোকে সাপোর্ট ভেক্টর বলা হয়। এই বিন্দুগুলো সিদ্ধান্ত সীমানার সবচেয়ে নিকটে অবস্থান করে। যদি এই বিশেষ বিন্দুগুলোকে ডেটাসেট থেকে অপসারণ করা হয়, তবে সমগ্র হাইপারপ্লেনের অবস্থান পরিবর্তিত হয়ে যাবে।



মার্জিন (Margin): হাইপারপ্লেন থেকে উভয় পাশের নিকটবর্তী সাপোর্ট ভেক্টরগুলোর লম্ব দূরত্বের সমষ্টিকে মার্জিন বলা হয়। SVM-এর গাণিতিক লক্ষ্য হলো এই মার্জিনের মান সর্বোচ্চ (Maximize) করা, যাতে মডেলটির সাধারণীকরণ ক্ষমতা (Generalization Power) বৃদ্ধি পায় এবং টেস্ট ডেটায় ভুলের পরিমাণ সর্বনিম্ন হয়।

- **শ্রেণীবিভাগ (Classification):** এটি দ্বিমাত্রিক (Binary) এবং বহুমাত্রিক (Multi-class) উভয় প্রকার জটিল শ্রেণীবিভাগ মেলাতে সক্ষম।
- **রিগ্রেশন (Regression):** সংখ্যাবাচক মান অনুমানের জন্য এর একটি বিশেষ সংস্করণ রয়েছে, যা **Support Vector Regression (SVR)** নামে পরিচিত।



- **উচ্চ-মাত্রিক উপাত্ত (High-Dimensional Data):** যখন উপাত্ত সেটে নমুনার সংখ্যার (Rows) তুলনায় বৈশিষ্ট্যের সংখ্যা (Columns/Features) অনেক বেশি থাকে (যেমন: বায়োইনফরমেটিক্স, জিনোম সিকোয়েন্সিং বা টেক্সট ক্লাসিফিকেশন), তখন SVM অনন্য কর্মদক্ষতা প্রদর্শন করে।
- **অরৈখিক উপাত্ত বিন্যাস (Non-Linear Data):** উপাত্ত যখন সরলরেখায় বিভাজনযোগ্য হয় না, তখন SVM তার বিখ্যাত **Kernel Trick** ব্যবহার করে উপাত্তগুলোকে একটি উচ্চতর মাত্রার ক্ষেত্রে (Higher Dimensional Space) রূপান্তরিত করে এবং একটি রৈখিক হাইপারপ্লেন দ্বারা পৃথক করে।

মেশিন লার্নিং শাস্ত্রে SLR, MLR, Logistic Regression, Decision Tree, Random Forest এবং KNN-এর মতো প্রতিষ্ঠিত মডেল থাকা সত্ত্বেও নিম্নলিখিত কাঠামোগত সুবিধার কারণে SVM অপরিহার্য:

- **Logistic Regression বনাম SVM:** লজিস্টিক রিগ্রেশন ক্লাসগুলোকে পৃথক করার জন্য যেকোনো একটি সাধারণ রৈখিক রেখা টানে, যা মূলত ট্রেনিং লস কমাতে সাহায্য করে। বিপরীতে, SVM খোঁজে Optimal Hyperplane, যা দুই ক্লাসের মধ্যকার দূরত্ব সর্বোচ্চ রাখে। এর ফলে নতুন ও অজানা উপাত্তের ওপর SVM-এর নির্ভুলতা লজিস্টিক রিগ্রেশনের চেয়ে অনেক বেশি স্থিতিশীল হয়।
- **KNN বনাম SVM:** KNN একটি অলস শিক্ষণ পদ্ধতি (Lazy Learner)। এটি টেস্ট টাইমে প্রতিটি নতুন নমুনার জন্য সম্পূর্ণ ডেটাসেটের দূরত্বের গাণিতিক হিসাব করে, যা বৃহৎ ডেটাসেটে অত্যন্ত ধীরগতির এবং মেমোরি সাপেক্ষ। পক্ষান্তরে, SVM ট্রেনিং শেষে শুধুমাত্র সাপোর্ট ভেক্টরগুলোকে মেমোরিতে ধারণ করে। ফলে এর প্রেডিকশন স্পিড অত্যন্ত দ্রুত এবং মেমোরি সাশ্রয়ী।
- **Decision Tree ও Random Forest বনাম SVM:** বৃক্ষ-ভিত্তিক মডেলগুলো উপাত্ত ক্ষেত্রকে অক্ষ-বরাবর লম্বালম্বিভাবে (Axis-aligned splitting) খণ্ড বিখণ্ড করে। উপাত্তের বিন্যাস যদি বৃত্তাকার বা জটিল প্যাটার্নের হয়, তবে এই মডেলগুলো অতিরিক্ত ডালপালা গজিয়ে ওভারফিটিং (Overfitting) তৈরি করে। কিন্তু SVM তার RBF (Radial Basis Function) কার্নেল প্রয়োগ করে অতি সহজেই যেকোনো জটিল ও আঁকাবাঁকা বৃত্তাকার সীমানা নিখুঁতভাবে চিহ্নিত করতে পারে।
- **SLR/MLR বনাম SVM (SVR):** সাধারণ রৈখিক রিগ্রেশন মডেলগুলো আউটলায়ার (Outliers) বা অস্বাভাবিক মান দ্বারা তীব্রভাবে প্রভাবিত ও বিচ্যুত হয়। কিন্তু SVM রিগ্রেশন (SVR) তার মার্জিনাল বাউন্ডারির ভেতরের সুনির্দিষ্ট ত্রুটিকে (epsilon) সম্পূর্ণ উপেক্ষা করে, যা মডেলটিকে আউটলায়ারের বিরুদ্ধে অত্যন্ত প্রতিরোধী (Robust) করে তোলে।

সাপোর্ট ভেক্টর মেশিন (SVM)-কে কি লজিস্টিক রিগ্রেশন (Logistic Regression)-এর একটি উন্নত সংস্করণ বা বর্ধিত রূপ (Enhanced Version) বলা চলে? উপযুক্ত গাণিতিক ও জ্যামিতিক যুক্তিসহ আলোচনা করো।

তাত্ত্বিক এবং গাণিতিক দৃষ্টিকোণ থেকে সাপোর্ট ভেক্টর মেশিন (SVM) এবং লজিস্টিক রিগ্রেশন (Logistic Regression) দুটি ভিন্ন নীতিমালার ওপর ভিত্তি করে প্রতিষ্ঠিত পৃথক ক্লাসিফিকেশন অ্যালগরিদম। লজিস্টিক রিগ্রেশন যেখানে সম্ভাব্যতা (Probability) এবং সর্বোচ্চ সম্ভাব্যতা অনুমানের (Maximum Likelihood Estimation) ওপর ভিত্তি করে কাজ করে, সেখানে SVM জ্যামিতিক মার্জিন এবং অপ্টিমাইজেশন তত্ত্বের ওপর ভিত্তি করে গড়ে উঠেছে। তবে, লজিস্টিক রিগ্রেশনের কিছু মৌলিক সীমাবদ্ধতা দূরীকরণ এবং ক্লাসিফিকেশন বাউন্ডারিকে জ্যামিতিকভাবে অধিক শক্তিশালী করার কারণে, লিনিয়ারলি সেপারেবল ও নন-লিনিয়ার ডেটা ক্লাসিফিকেশনের ক্ষেত্রে SVM-কে লজিস্টিক রিগ্রেশনের একটি "উন্নত বা বর্ধিত রূপ" (Enhanced Version) হিসেবে বিবেচনা করা যৌক্তিক।

নিচে প্রধান ৫টি গাঠনিক ও জ্যামিতিক রূপান্তরের মাধ্যমে এই উন্নয়ন প্রক্রিয়াটি ব্যাখ্যা করা হলো:

লস ফাংশনের উন্নয়ন (Loss Function Enhancement)

লজিস্টিক রিগ্রেশন এবং SVM-এর কার্যপদ্ধতির মূল পার্থক্য নিহিত রয়েছে এদের লস ফাংশনের (Loss Function) মধ্যে:

- **লজিস্টিক রিগ্রেশন (Cross-Entropy Loss):** এটি লিনিয়ার কম্বিনেশন $z = wx + b$ কে সিগময়েড ফাংশনে রূপান্তর করে সম্ভাবনা হিসাব করে। এর লস ফাংশন (Log Loss) প্রতিটি ডেটা পয়েন্টের দূরত্বের ওপর ভিত্তি করে ক্রমাগত পরিবর্তিত হয়। ফলে এটি সঠিক ও ভুল—উভয় প্রকার ক্লাসিফাইড পয়েন্টের আত্মবিশ্বাসের মাত্রা নিয়ে কাজ করে এবং প্রতিটি বিন্দুর জন্য কম-বেশি লস গণনা করে।
- **SVM (Hinge Loss):** SVM-এ লগ-লসকে প্রতিস্থাপন করা হয় হিঞ্জ লস (Hinge Loss) দ্বারা, যার গাণিতিক রূপ:

$$\text{Hinge Loss} = \max\{0, 1 - y_i(wx + b)\}$$

এই লস ফাংশনের অন্যতম বৈশিষ্ট্য হলো, যদি কোনো ডেটা পয়েন্ট সঠিকভাবে ক্লাসিফাইড হয় এবং মার্জিনের বাইরে থাকে ($y_i(wx + b) \geq 1$), তবে তার জন্য লসের মান সরাসরি **০ (Zero)** হয়ে যায়। এটি মডেলটিকে কেবল বাউন্ডারির নিকটবর্তী জটিল পয়েন্টগুলোর ওপর ফোকাস করতে বাধ্য করে।

মার্জিন ম্যাক্সিমাইজেশন এবং জ্যামিতিক বাউন্ডারি (Margin Maximization)

- **লজিস্টিক রিগ্রেশন:** লজিস্টিক রিগ্রেশনের মূল লক্ষ্য হলো এমন একটি ডিসিশন বাউন্ডারি ($wx + b = 0$) খুঁজে বের করা যা ডেটাসেটকে আলাদা করতে পারে। কিন্তু জ্যামিতিকভাবে এই বাউন্ডারিটি ডেটা পয়েন্ট থেকে কতটা দূরে থাকবে, তার কোনো সুনির্দিষ্ট নিয়ন্ত্রণ বা লক্ষ্য এতে থাকে না। এর ফলে বাউন্ডারিটি কোনো নির্দিষ্ট ক্লাসের অতি নিকটে চলে যেতে পারে, যা নতুন টেস্ট ডেটার ক্ষেত্রে ভুল সিদ্ধান্তের (Overfitting) ঝুঁকি বাড়ায়।
- **SVM (Maximum Margin Classifier):** SVM এই ধারণাকে উন্নত করে ডিসিশন বাউন্ডারির দুই পাশে একটি 'নিরাপদ অঞ্চল' বা **মার্জিন (Margin)** তৈরি করে। এর উদ্দেশ্য হলো মার্জিনের দূরত্ব $M = \frac{2}{|w|}$ কে সর্বোচ্চ (Maximize) করা। এই ম্যাক্সিমাম মার্জিন বৈশিষ্ট্যের কারণে মডেলটির সাধারণীকরণ ক্ষমতা (Generalization Ability) বহুগুণ বৃদ্ধি পায়।

আউটলায়ার প্রতিরোধ ক্ষমতা (Robustness to Outliers)

- **লজিস্টিক রিগ্রেশন:** লজিস্টিক রিগ্রেশনের লস ফাংশন পুরো ডেটাসেটের সব বিন্দুর ওপর নির্ভরশীল। ফলে ডিসিশন বাউন্ডারি থেকে অনেক দূরে অবস্থিত কোনো আউটলায়ার (Outlier) বা ভুল ডেটার সামান্য পরিবর্তনেও সম্পূর্ণ ডিসিশন বাউন্ডারিটি স্থানচ্যুত বা প্রভাবিত হতে পারে।
- **SVM:** SVM-এর ডিসিশন বাউন্ডারি কেবলমাত্র মার্জিনের ওপর বা মার্জিনের ভেতরে অবস্থিত নির্দিষ্ট কিছু বিন্দুর ওপর নির্ভর করে নির্ধারিত হয়, যাদের সাপোর্ট ভেক্টর (Support Vectors) বলা হয়। মার্জিনের বাইরে অবস্থিত অন্য হাজারটি বিন্দু পরিবর্তন বা অপসারণ করলেও ডিসিশন বাউন্ডারির কোনো পরিবর্তন হয় না। এই কারণে SVM আউটলায়ারের বিরুদ্ধে অত্যন্ত শক্তিশালী (Robust to Outliers)।

কার্নেল ট্রিক এবং নন-লিনিয়ার ডেটা ব্যবস্থাপন (The Kernel Trick)

- **লজিস্টিক রিগ্রেশন:** জটিল এবং বাস্তব জীবনের নন-লিনিয়ার ডেটা ক্লাসিফাই করার জন্য লজিস্টিক রিগ্রেশনে ম্যানুয়ালি উচ্চ-মাত্রার পলিনোমিয়াল ফিচার তৈরি করতে হয়, যা অত্যন্ত জটিল এবং কম্পিউটেশনালি ব্যয়বহুল।
- **SVM:** SVM-এ **কার্নেল ট্রিক (Kernel Trick)** নামক একটি উন্নত গাণিতিক কৌশল ব্যবহার করা হয়। এর মাধ্যমে মূল ইনপুট স্পেসের (Input Space) নন-লিনিয়ার ডেটাকে কোনো রূপান্তর ছাড়াই উচ্চ মাত্রার ফিচার স্পেসে (Feature Space) রূপান্তর করা যায় এবং সেখানে একটি লিনিয়ার হাইপারপ্লেন দ্বারা ডেটা আলাদা করা সম্ভব হয়। Linear,

Polynomial, এবং RBF (Gaussian) কার্নেল ব্যবহারের এই সুবিধা ক্লাসিফিকেশনের দক্ষতাকে লজিস্টিক রিগ্রেশনের তুলনায় বহুগুণ বাড়িয়ে দেয়।

সফট মার্জিন ও হাইপারপ্যারামিটার নিয়ন্ত্রণ (C)

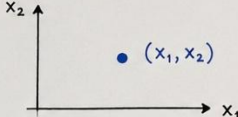
লজিস্টিক রিগ্রেশনে ভুল ক্লাসিফিকেশনের ওপর জ্যামিতিক কঠোরতা নিয়ন্ত্রণের সরাসরি কোনো প্যারামিটার নেই। পক্ষান্তরে, SVM-এ হাইপারপ্যারামিটার C (Penalty Parameter) ব্যবহারের মাধ্যমে 'হার্ড মার্জিন' (যেখানে কোনো ভুল গ্রহণযোগ্য নয়) এবং 'সফট মার্জিন' (যেখানে কিছু ভুল বা ওভারল্যাপিং অনুমোদন করা হয়)-এর মধ্যে একটি চমৎকার ভারসাম্য রক্ষা করা যায়। এটি মডেলের ওভারফিটিং এবং আন্ডারফিটিং সমস্যা খুব নিখুঁতভাবে নিয়ন্ত্রণ করতে পারে।

পরিশেষে বলা যায় যে, যদিও লজিস্টিক রিগ্রেশন এবং SVM-এর তাত্ত্বিক সূত্রপাত ভিন্ন, তবুও লজিস্টিক রিগ্রেশনের ডিসিশন বাউন্ডারিকে জ্যামিতিক মার্জিন দ্বারা সুসংহত করা, হিঞ্জ লসের মাধ্যমে অপ্ৰয়োজনীয় বিন্দুর প্রভাব মুক্ত করা, এবং কার্নেল ট্রিকের সাহায্যে জটিল নন-লিনিয়ার ডেটা প্রক্রিয়াকরণের ক্ষমতা যোগ করার মাধ্যমে SVM ক্লাসিফিকেশন প্রক্রিয়াকে এক নতুন উচ্চতায় নিয়ে গেছে। অতএব, ব্যবহারিক ও কার্যকারিতার দিক থেকে **SVM-কে লজিস্টিক রিগ্রেশনের একটি অত্যন্ত শক্তিশালী এবং উন্নত সংস্করণ (Enhanced Version) বলা সম্পূর্ণ যুক্তিযুক্ত।**

মেশিন লার্নিং এবং সাপোর্ট ভেক্টর মেশিন (SVM) বোঝার জন্য ডেটার জ্যামিতিক রূপ জানা অত্যন্ত জরুরি। নিচে মাত্রার (Dimension) ওপর ভিত্তি করে প্রধান ৫টি জ্যামিতিক উপাদান সংক্ষেপে আলোচনা করা হলো:

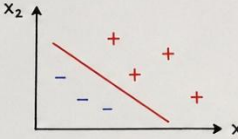
Geometric Concepts in SVM

- 1. Point
A point represents a single data instance.
- 2. Line
A line in 2D space separates the two classes.
- 3. Plane
A plane in 3D space separates the two classes.
- 4. Hyperplane
A hyperplane in n-dimensional space separates the two classes.
- 5. Linear Equation (General)
The general form of linear equation in n-dimensional space.



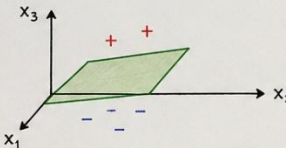
(x_1, x_2)

In n-dimensional space:
 $x = [x_1, x_2, \dots, x_n]^T$



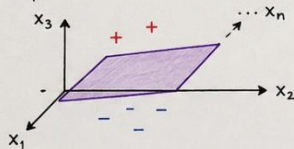
Linear Equation:
 $w_1 x_1 + w_2 x_2 + b = 0$

This is a line.



Linear Equation:
 $w_1 x_1 + w_2 x_2 + w_3 x_3 + b = 0$

This is a plane.



Linear Equation:
 $w^T x + b = 0$

This is a hyperplane.

$w_1 x_1 + w_2 x_2 + \dots + w_n x_n + b = 0$

This equation represents a hyperplane.

or $w^T x + b = 0$

১. বিন্দু (Point)

- বহুমাত্রিক স্থানে একটি বিন্দু মূলত একটি একক ডেটা ইনস্ট্যান্স বা রেকর্ডকে নির্দেশ করে।
- গাণিতিক রূপ: n -ডাইমেনশনাল স্পেসে একে একটি ভেক্টর হিসেবে প্রকাশ করা হয়:

$$x = [x_1, x_2, \dots, x_n]^T$$

২. রেখা (Line)

- দ্বিমাত্রিক (2D) স্থানে দুটি ভিন্ন ক্লাস বা ডেটা দলকে আলাদা করার জন্য ডিসিশন বাউন্ডারি হিসেবে একটি সরলরেখা ব্যবহার করা হয়।
- লিনিয়ার সমীকরণ (Linear Equation):

$$w_1 x_1 + w_2 x_2 + b = 0$$

৩. সমতল (Plane)

- ত্রিমাত্রিক (3D) স্থানে দুটি ক্লাসকে পৃথক করার বাউন্ডারি হিসেবে একটি চ্যাপ্টা সমতল পৃষ্ঠ বা প্লেন ব্যবহৃত হয়।

- লিনিয়ার সমীকরণ (Linear Equation):

$$w_1x_1 + w_2x_2 + w_3x_3 + b = 0$$

৪. হাইপারপ্লেন (Hyperplane)

- যখন ডেটার মাত্রা n -ডাইমেনশনে রূপ নেয়, তখন ক্লাস দুটিকে আলাদা করার বাউন্ডারিকে হাইপারপ্লেন বলা হয়।

- লিনিয়ার সমীকরণ (Linear Equation):

$$w^T x + b = 0$$

৫. সাধারণ লিনিয়ার সমীকরণ (Linear Equation - General)

- এটি হলো n -ডাইমেনশনাল স্পেসে যেকোনো ডিসিশন বাউন্ডারি প্রকাশের একটি সাধারণ গাণিতিক রূপ। এই সমীকরণটি মূলত একটি হাইপারপ্লেন-কে নির্দেশ করে।

- সাধারণ রূপ:

$$w_1x_1 + w_2x_2 + \dots + w_nx_n + b = 0 \quad \text{অথবা} \quad w^T x + b = 0$$

ম্যাক্সিমাম মার্জিন ক্লাসিফায়ার (Maximum Margin Classifier)

SVM-কে একটি ম্যাক্সিমাম মার্জিন ক্লাসিফায়ার বলা হয়। এটি কেবল দুই ক্লাসের ডেটাকে আলাদাই করে না, বরং উভয় ক্লাসের নিকটবর্তী ডেটা পয়েন্ট থেকে সমান দূরত্ব বজায় রেখে সম্ভাব্য সবচেয়ে চওড়া রাস্তা বা সর্বোচ্চ মার্জিন (Maximum Margin) তৈরি করে সিদ্ধান্ত রেখা নির্ধারণ করে।

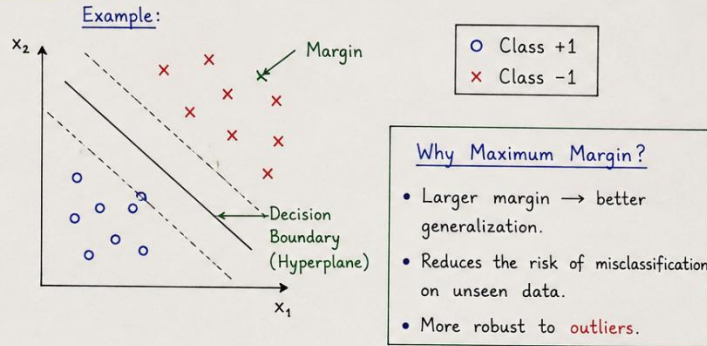
ম্যাক্সিমাম মার্জিন কেন গুরুত্বপূর্ণ (Why Maximum Margin is important)

- Better Generalization:** মার্জিনের মান যত বড় হবে ($M = \frac{2}{|w|}$), মডেলটির নতুন ও অজানা ডেটার ওপর সঠিক প্রেডিকশন দেওয়ার ক্ষমতা (Generalization Ability) তত বেশি বৃদ্ধি পায়।

Overfitting রোধ: মার্জিন ছোট হলে ডিসিশন বাউন্ডারি কোনো একটি ক্লাসের খুব কাছাকাছি চলে যায়, যা ওভারফিটিংয়ের ঝুঁকি বাড়ায়। ম্যাক্সিমাম মার্জিন মডেলকে এই ঝুঁকি থেকে মুক্ত রাখে।

3.1 What is Support Vector Machine (SVM)?

- SVM is a supervised learning algorithm used for classification (mainly binary classification).
- The main idea of SVM is to find the optimal decision boundary (hyperplane) that separates the classes with the maximum margin.
- It is a Maximum Margin Classifier.
- SVM works well even when there are outliers in the data.



3.2 Components of SVM

① Decision Boundary / Hyperplane

The boundary that separates the classes.

② Margin

The distance between the hyperplane and the nearest data points from both classes.

③ Support Vectors

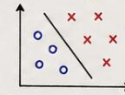
The data points that lie closest to the decision boundary. These points define the position of the boundary.

④ Generalized Model

SVM finds the best boundary that works well on new, unseen data.

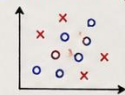
3.3 Types of Dataset

① Linearly Separable Data



Classes can be separated perfectly by a straight line / hyperplane.

② Non-Linearly Separable Data



Classes cannot be separated by a straight line / hyperplane. (Need Kernel Trick)

আউটলায়ারের বিরুদ্ধে প্রতিরোধ ক্ষমতা (Robustness against Outliers)

SVM আউটলায়ারের বিরুদ্ধে অত্যন্ত শক্তিশালী বা প্রতিরোধী (Robust to Outliers)। এর ডিসিশন বাউন্ডারি ডেটাসেটের সমস্ত বিন্দুর ওপর নির্ভর করে না; বরং এটি কেবল মার্জিনের প্রান্তে থাকা নির্দিষ্ট কিছু পয়েন্টের ওপর ভিত্তি করে তৈরি হয়। ফলে বাউন্ডারি থেকে দূরে থাকা কোনো আউটলায়ারের সামান্য পরিবর্তনে মূল সিদ্ধান্ত রেখার কোনো পরিবর্তন হয় listen না।

SVM-এর উপাদানসমূহ (Components of SVM)

ডিসিশন বাউন্ডারি (Decision Boundary)

দুই বা বহুমাত্রিক স্থানে বিভিন্ন ক্লাসের ডেটাকে পৃথক করার জন্য যে জ্যামিতিক রেখা, সমতল বা হাইপারপ্লেন ব্যবহার করা হয়, তাকে ডিসিশন বাউন্ডারি বলে। গাণিতিকভাবে একে $\pi = wx + b = 0$ সমীকরণ দ্বারা প্রকাশ করা হয়।

মার্জিন (Margin)

ডিসিশন বাউন্ডারি বা কেন্দ্রীয় হাইপারপ্লেন থেকে উভয় পাশের ক্লাসের নিকটবর্তী ডেটা পয়েন্টের মধ্যবর্তী মোট দূরত্বকে মার্জিন বলে ($M = d_1 + d_2$)। এটি মূলত একটি সুরক্ষিত অঞ্চল (Safe Zone) হিসেবে কাজ করে।

সাপোর্ট ভেক্টরস (Support Vectors)

ডেটাসেটের যে নির্দিষ্ট বিন্দু বা ডেটা ইনস্ট্যান্সগুলো মার্জিনের সীমানার ওপর অবস্থান করে এবং ডিসিশন বাউন্ডারির অবস্থান ও আকৃতি নির্ধারণে সরাসরি ভূমিকা রাখে, সেগুলোকে সাপোর্ট ভেক্টর বলে। এই ভেক্টরগুলো সরিয়ে নিলে পুরো মডেলের বাউন্ডারি পরিবর্তন হয়ে যায়।

জেনারেলাইজড মডেল (Generalized Model)

সাপোর্ট ভেক্টর ও ম্যাক্সিমাম মার্জিন ব্যবহারের মাধ্যমে তৈরি ডিসিশন বাউন্ডারিটি একটি জেনারেলাইজড বা সাধারণীকরণ মডেল তৈরি করে। এটি ট্রেনিং ডেটার পাশাপাশি সম্পূর্ণ নতুন বা টেস্ট ডেটার ওপরেও সমানভাবে নির্ভুল ফলাফল দিতে সক্ষম।

ডেটাসেটের প্রকারভেদ (Types of Dataset)

লিনিয়ারলি সেপারেবল ডেটা (Linearly Separable Data)

যেসব ডেটাসেটের ক্লাসগুলোকে একটিমাত্র সোজা সরলরেখা (2D-তে), সমতল (3D-তে) বা লিনিয়ার হাইপারপ্লেন দ্বারা সম্পূর্ণ আলাদা করা সম্ভব, সেগুলোকে লিনিয়ারলি সেপারেবল ডেটা বলে। এই ক্ষেত্রে কোনো ভুল বা ওভারল্যাপিং ছাড়া "Hard Margin" ব্যবহার করে নিখুঁত ক্লাসিফিকেশন করা যায়।

নন-লিনিয়ারলি সেপারেবল ডেটা (Non-Linearly Separable Data)

বাস্তব জীবনের যেসব জটিল ডেটাসেটকে কোনো সোজা লাইন বা সমতল হাইপারপ্লেন দিয়ে আলাদা করা যায় না, সেগুলোকে নন-লিনিয়ার ডেটা বলে। এই ধরনের ডেটা ক্লাসিফাই করার জন্য SVM-এ "Kernel Trick" ব্যবহার করে ডেটাকে উচ্চ মাত্রার ফিচার স্পেসে রূপান্তর করে আলাদা করা হয়।

হাইপারপ্লেনের প্রাথমিক সমীকরণসমূহ

ধরা যাক, একটি বহুমাত্রিক স্পেসে আমাদের কাছে দুই ধরনের ক্লাস রয়েছে—পজিটিভ (+1) এবং নেগেটিভ (-1)। এই দুই ক্লাসের মধ্যবর্তী স্থানে আমরা তিনটি সমান্তরাল হাইপারপ্লেন বা ডিসিশন বাউন্ডারি বিবেচনা করি:

1. কেন্দ্রীয় সিদ্ধান্ত রেখা (Optimal Hyperplane):

$$w^T x + b = 0$$

2. পজিটিভ মার্জিনাল হাইপারপ্লেন (π_+):

$$w^T x + b = +1$$

3. নেগেটিভ মার্জিনাল হাইপারপ্লেন (π_-):

$$w^T x + b = -1$$

এখানে, w হলো লম্ব ভেক্টর বা ওয়েট ভেক্টর (Weight Vector), x হলো ইনপুট ফিচার ভেক্টর এবং b হলো বায়াস (Bias)।

এখানে, $w = [w_1, w_2, w_3, \dots, w_n]$ হলো ওয়েট ম্যাট্রিক্স বা ভেক্টর এবং এর মান বা ম্যাগনিটিউড বের করার সূত্রটি:

$$|w| = \sqrt{w_1^2 + w_2^2 + \dots + w_n^2}$$

7. HYPERPLANE EQUATION

$$\pi = w^T x + b \quad \text{or} \quad wx + b = 0$$

where,

w = weight vector (normal to the hyperplane)

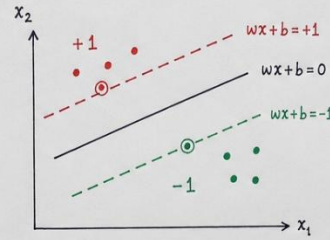
x = input feature vector

b = bias

This equation represents the decision boundary that separates two classes.

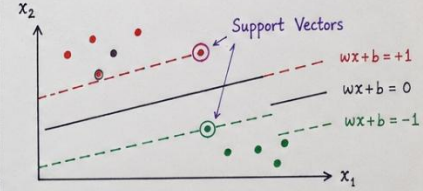
8. PARALLEL HYPERPLANES

- Positive Hyperplane : $wx + b = +1$
- Decision Boundary : $wx + b = 0$
- Negative Hyperplane : $wx + b = -1$



9. SUPPORT VECTORS

- The data points that are closest to the decision boundary.
- They lie on the margin boundaries ($wx+b=+1$ or $wx+b=-1$).
- These points are called Support Vectors.
- They play a crucial role in defining the position and orientation of the hyperplane.



10. MARGIN CALCULATION

- Distance from a point x to the hyperplane $wx+b=0$ is

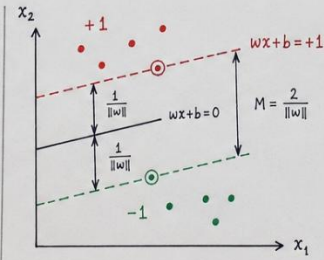
$$d = \frac{|wx+b|}{\|w\|}$$

For support vectors (on the margin boundaries),

$$|wx+b|=1 \Rightarrow d = \frac{1}{\|w\|}$$

Total Margin (distance between the two margin boundaries) is $\uparrow$

$$M = \frac{2}{\|w\|}$$



Larger margin = better generalization

11. WEIGHT VECTOR MAGNITUDE

The magnitude (or Euclidean norm) of the weight vector w is

$$\|w\| = \sqrt{w_1^2 + w_2^2 + \dots + w_n^2}$$

where,

$$w = [w_1, w_2, \dots, w_n]^T \in \mathbb{R}^n$$

A smaller $\|w\|$ leads to a larger margin.

So, SVM tries to minimize $\|w\|$ to maximize the margin.

ডিসিশন বাউন্ডারি থেকে দুই পাশের মার্জিনাল লাইনের দূরত্ব পরস্পর সমান, অর্থাৎ:

$$d_1 = d_2$$

এখানে, কেন্দ্রীয় লাইন থেকে যেকোনো একপাশের দূরত্বের জ্যামিতিক সূত্রটি হলো:

$$\text{distance} = \frac{wx + b}{\|w\|}$$

এখন, পজিটিভ মার্জিনাল লাইনের সীমানার ওপর থাকা বিন্দুর জন্য স্কের $wx + b = +1$ । সুতরাং, কেন্দ্রীয় লাইন থেকে পজিটিভ লাইনের দূরত্ব:

$$d_1 = \frac{1}{\|w\|}$$

যেহেতু দুই পাশের দূরত্ব সমান ($d_1 = d_2$), তাই অপর পাশের দূরত্বটিও হবে:

$$d_2 = \frac{1}{\|w\|}$$

নোটের চিত্র ও সমীকরণ অনুযায়ী, মোট মার্জিন (M) হলো এই দুই দূরত্বের সমষ্টি:

$$\text{Margin}(M) = d_1 + d_2$$

$$M = 2 \times d_1$$

এখন d_1 -এর মান বসিয়ে আমরা পাই:

$$M = 2 \times \frac{1}{|w|} = \frac{2}{|w|}$$

- " $|w|$ যত ছোট হবে, M তত বড় হবে"
- "Target Margin maximize করা"

গাণিতিক নিয়ম অনুযায়ী, ভগ্নাংশের হর ($|w|$) যত ছোট বা ডিক্রিজ করা যাবে, সম্পূর্ণ ভগ্নাংশের মান অর্থাৎ মার্জিন (M) তত বড় বা ম্যাক্সিমাইজ হবে। যেহেতু SVM-এর প্রধান লক্ষ্যই হলো দুই ক্লাসের মধ্যকার দূরত্ব বা মার্জিন সর্বোচ্চ করা (Target Margin Maximize), তাই মডেলটি সর্বদা $|w|$ -এর মানকে সর্বনিম্ন (Minimize) করার চেষ্টা করে।

মার্জিন সর্বোচ্চকরণ (Maximize Margin)

সাপোর্ট ভেক্টর মেশিনের প্রধান গাণিতিক লক্ষ্য হলো পজিটিভ এবং নেগেটিভ ক্লাসের মধ্যবর্তী দূরত্ব বা মার্জিন (M) সর্বোচ্চ করা। জ্যামিতিক প্রতিপাদন থেকে প্রাপ্ত মার্জিনের গাণিতিক রূপটি হলো:

$$M = \frac{2}{|w|}$$

সমতুল্য সর্বনিম্নকরণ সমস্যা (Equivalent Minimization Problem)

বীজগণিতীয় নিয়ম অনুযায়ী, কোনো ভগ্নাংশের মানকে সর্বোচ্চ (Maximize) করতে হলে তার হর-এর মানকে সর্বনিম্ন করতে হয়। অর্থাৎ, মার্জিন $M = \frac{2}{|w|}$ কে সর্বোচ্চ করার অর্থ হলো ওয়েট ভেক্টরের ম্যাগনিটিউড $|w|$ কে সর্বনিম্ন (Minimize) করা।

ক্যালকুলাস এবং ডিফারেনশিয়েশনের সুবিধার্থে (বিশেষ করে বর্গমূলের জটিলতা এড়াতে) এই সর্বোচ্চকরণের সমস্যাটিকে একটি সমতুল্য সর্বনিম্নকরণ সমস্যা হিসেবে নিচে রূপান্তর করা হয়:

$$\min \frac{1}{2} |w|^2$$

হিঞ্জ লস (Hinge Loss)

বাস্তব জীবনের ডেটাসেটে সম্পূর্ণ নিখুঁতভাবে ডেটা আলাদা করা সবসময় সম্ভব হয় না। মডেলের এই ত্রুটি বা ভুলের পরিমাণ গাণিতিকভাবে গণনা করার জন্য Hinge Loss ফাংশন ব্যবহার করা হয়। এর গাণিতিক সূত্রটি নিম্নরূপ:

$$\text{Hinge Loss} = \max\{0, 1 - y_i(wx + b)\}$$

এখানে, y_i হলো ডেটার প্রকৃত ক্লাস বা লেবেল (+1 অথবা -1) এবং $(wx + b)$ হলো মডেল দ্বারা প্রাপ্ত প্রেডিকশন স্কোর।

বিভিন্ন ক্ষেত্রে হিঞ্জ লসের মান (Different Hinge Loss Cases)

ডিসিশন বাউন্ডারির সাপেক্ষে কোনো ডেটা পয়েন্ট কোথায় অবস্থান করছে, তার ওপর ভিত্তি করে হিঞ্জ লসের মানকে ৩টি সুনির্দিষ্ট ক্ষেত্রে বিভক্ত করা যায়:

ক. সঠিক ক্লাসিফিকেশন (Correct Classification)

যখন কোনো ডেটা পয়েন্ট সম্পূর্ণ সঠিকভাবে মার্জিনের সুরক্ষিত অঞ্চলের বাইরে ক্লাসিফাই হয় (অর্থাৎ, $y_i(wx + b) \geq 1$), তখন সমীকরণের ভেতরের মানটি ঋণাত্মক বা শূন্য হয়ে যায়।

- **গাণিতিক উদাহরণ:** যদি $y_i(wx + b) = 2$ হয়, তবে $1 - 2 = -1$ ।
- **লস গণনা:** $\max(0, -1) = 0$ ।
- **সিদ্ধান্ত:** সঠিকভাবে ক্লাসিফাইড এবং মার্জিনের বাইরের বিন্দুর জন্য মডেলে কোনো পেনাল্টি বা লস যুক্ত হয় না।

খ. সঠিক কিন্তু মার্জিনের ভেতরে (Correct but Inside Margin)

যখন কোনো পয়েন্ট সঠিকভাবে সঠিক ক্লাসে অবস্থান করে, কিন্তু সেটি মার্জিনের সীমানার ভেতরে ঢুকে পড়ে (অর্থাৎ, $0 \leq y_i(wx + b) < 1$), তখন মডেলের জন্য পেনাল্টি তৈরি হয়।

- **গাণিতিক উদাহরণ:** যদি $y_i(wx + b) = 0.5$ হয়, তবে $1 - 0.5 = 0.5$ ।
- **লস গণনা:** $\max(0, 0.5) = 0.5$ ।
- **সিদ্ধান্ত:** যদিও পয়েন্টটি সঠিক ক্লাসেই আছে, তবুও মার্জিন লঙ্ঘন করার অপরাধে মডেলটিকে কিছুটা লস বা পেনাল্টি গুনতে হয়।

গ. ভুল ক্লাসিফিকেশন (Misclassified)

যখন কোনো ডেটা পয়েন্ট সম্পূর্ণ ভুল ক্লাসে চলে যায় এবং ডিসিশন বাউন্ডারি পার হয়ে অন্য পাশে অবস্থান করে (অর্থাৎ, $y_i(wx + b) < 0$), তখন লসের মান ধনাত্মকভাবে বৃদ্ধি পায়।

- **গাণিতিক উদাহরণ:** যদি $y_i(wx + b) = -0.5$ হয়, তবে $1 - (-0.5) = 1 + 0.5 = 1.5$ ।
- **লস গণনা:** $\max(0, 1.5) = 1.5$ ।
- **সিদ্ধান্ত:** ভুল ক্লাসিফিকেশনের জন্য মডেলকে সর্বোচ্চ পেনাল্টি দেওয়া হয়। ডিসিশন বাউন্ডারি থেকে পয়েন্টটি যত দূরে ভুল দিকে যাবে, লসের পরিমাণও তত রৈখিকভাবে বাড়তে থাকবে।

চূড়ান্ত কস্ট ফাংশন (Final Cost Function)

SVM-এর মূল উদ্দেশ্য (মার্জিন সর্বোচ্চ করা) এবং হিঞ্জ লস (ভুলের পেনাল্টি সর্বনিম্ন করা)—এই দুটি ভিন্ন লক্ষ্যকে একটি একক সমীকরণে যুক্ত করে চূড়ান্ত কস্ট ফাংশন J গঠন করা হয়:

$$J = \frac{1}{2} \|w\|^2 + \sum \text{Hinge Loss}$$

এই সমীকরণের প্রথম অংশ ($\frac{1}{2} \|w\|^2$) মার্জিনকে বড় করার চেষ্টা করে (যা রেগুলারাইজেশন হিসেবে কাজ করে) এবং দ্বিতীয় অংশ ($\sum \text{Hinge Loss}$) মডেলের ক্লাসিফিকেশন ভুলগুলোকে দমন করার চেষ্টা করে। একটি আদর্শ মডেল এই দুটি বিষয়ের মধ্যে একটি নিখুঁত গাণিতিক ভারসাম্য বজায় রাখে।

কোয়াদ্রেটিক প্রোগ্রামিং (Quadratic Programming - QP)

অপটিমাইজেশন টেকনিক (Optimization Technique)

চূড়ান্ত কস্ট ফাংশন J -এর মান সর্বনিম্ন করার জন্য এবং নির্দিষ্ট শর্তসাপেক্ষে (Constraints) অপটিমাল প্যারামিটার খোঁজার জন্য যে গাণিতিক ও কম্পিউটেশনাল পদ্ধতি ব্যবহার করা হয়, তাকে কোয়াদ্রেটিক প্রোগ্রামিং (QP) বলা হয়।

উত্তল অপটিমাইজেশন (Convex Optimization)

যেহেতু SVM-এর এই কস্ট ফাংশনটি একটি **উত্তল ফাংশন বা Convex Function**, সেহেতু এর জ্যামিতিক গ্রাফে কেবল একটিমাত্র সর্বনিম্ন বিন্দু বা **Global Minima** থাকে। কোয়াদ্রেটিক প্রোগ্রামিং টেকনিক ব্যবহারের মাধ্যমে কোনো লোকাল মিনিমামে (Local Minima) না আটকে সরাসরি বৈশ্বিক বা নিখুঁত অপটিমাল সলিউশন খুঁজে পাওয়া নিশ্চিত করা সম্ভব হয়।

হার্ড মার্জিন ও সফট মার্জিন (Hard Margin & Soft Margin)

বাস্তব জীবনের ডেটাসেটগুলোর গঠন এবং জটিলতার ওপর ভিত্তি করে সাপোর্ট ভেক্টর মেশিনের ডিসিশন বাউন্ডারি নির্ধারণের পদ্ধতিকে প্রধানত দুটি ভাগে ভাগ করা যায়। নিচে এই দুটি মার্জিন পদ্ধতি এবং তাদের নিয়ন্ত্রণকারী প্যারামিটার বিস্তারিতভাবে আলোচনা করা হলো:

হার্ড মার্জিন এসভিএম (Hard Margin SVM)

কোনো ভুল ক্লাসিফিকেশন নয় (No Misclassification)

হার্ড মার্জিন এসভিএম-এর প্রধান বৈশিষ্ট্য হলো এটি ডিসিশন বাউন্ডারি বা মার্জিনের ভেতরে কোনো ধরনের ভুল ক্লাসিফিকেশন বা ডেটা পয়েন্টের অনুপ্রবেশ অনুমোদন করে না। অর্থাৎ, প্রতিটি ডেটা পয়েন্টকে অবশ্যই মার্জিনের সঠিক পাশে এবং বাইরে অবস্থান করতে হবে।

নিখুঁতভাবে পৃথকীকরণযোগ্য ডেটা (Perfectly Separable Data)

এই পদ্ধতিটি কেবল তখনই সফলভাবে কাজ করে, যখন ডেটাসেটটি লিনিয়ারলি নিখুঁতভাবে পৃথকীকরণযোগ্য (Perfectly Separable Data) হয়। যদি ডেটার মধ্যে কোনো নয়েজ (Noise) বা ওভারল্যাপিং থাকে, তবে হার্ড মার্জিন মডেল কোনো গাণিতিক সমাধানে পৌঁছাতে পারে না এবং ব্যর্থ হয়।

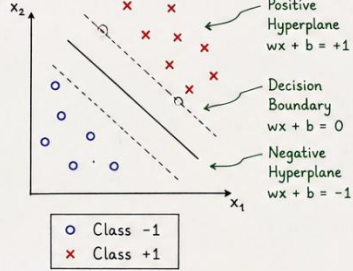
সফট মার্জিন এসভিএম (Soft Margin SVM)

ভুল ক্লাসিফিকেশন অনুমোদন (Allows Errors)

বাস্তবসম্মত সমাধানের জন্য সফট মার্জিন এসভিএম ডিসিশন বাউন্ডারি নির্ধারণের ক্ষেত্রে কিছুটা শিথিলতা প্রদর্শন করে। এই পদ্ধতিতে মডেলটি মার্জিনের ভেতরে বা ভুল পাশে কিছু সীমিত সংখ্যক ডেটা পয়েন্টের উপস্থিতি বা ত্রুটি অনুমোদন (Allows Errors) করে।

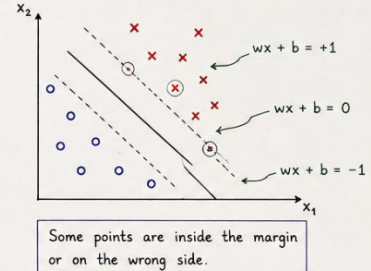
5.1 Hard Margin SVM

- Used when data is **linearly separable**.
- No misclassification allowed.
- Finds the maximum margin hyperplane that separates the two classes perfectly.



5.2 Soft Margin SVM

- Used when data is **not perfectly linearly separable**.
- Allows some misclassifications.
- More flexible and better generalization to real-world data.



5.3 Penalty Parameter (C)

In soft margin SVM, we use a penalty parameter C to control the **trade-off** between margin size and classification error.

Soft Margin SVM Objective:

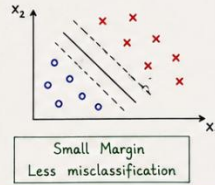
$$\text{Minimize } J = \frac{1}{2} \|w\|^2 + C \sum_{i=1}^n \xi_i$$

- $\frac{1}{2} \|w\|^2 \rightarrow$ Maximizes the margin
 $C \sum_{i=1}^n \xi_i \rightarrow$ Penalty for misclassification
 $\xi_i \rightarrow$ Slack variable ($\xi_i \geq 0$)

5.4 Effect of C (Penalty Parameter)

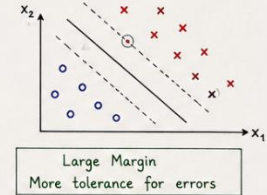
Large C ($C \uparrow$)

- High penalty on misclassification
- Tries to classify all points correctly
- Margin becomes smaller
- Behaves like **Hard Margin**



Small C ($C \downarrow$)

- Low penalty on misclassification
- Allows more errors
- Margin becomes larger
- Behaves like **Soft Margin**



Summary

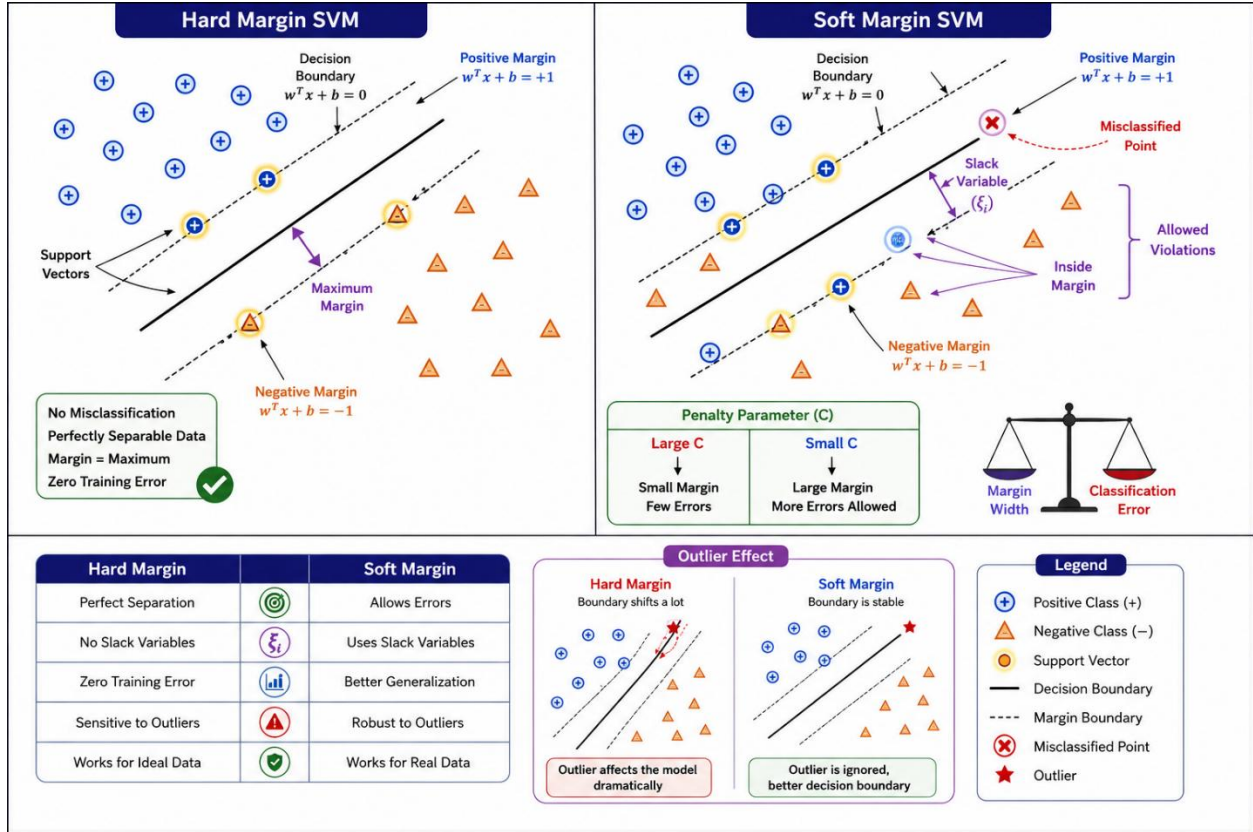
- Hard Margin $\rightarrow$ No error, smaller flexibility.
- Soft Margin $\rightarrow$ Allows error, better generalization.
- C controls the balance between margin size and training error.

Key Points

- C is a very important hyperparameter in SVM.
- Proper choice of C improves the performance of the model.
- Use **cross-validation** to select the best C .

উন্নত সাধারণীকরণ ক্ষমতা (Better Generalization)

কিছু ভুল বা ওভারল্যাপিং মেনে নেওয়ার কারণে সফট মার্জিন এসভিএম-এর ডিসিশন বাউন্ডারিটি খুব বেশি জটিল বা ওভারফিটেড হয় না। ফলে, এটি নতুন এবং অজানা টেস্ট ডেটার ওপর অনেক উন্নত সাধারণীকরণ ক্ষমতা (Better Generalization) বা নির্ভুল ফলাফল প্রদর্শন করতে পারে।



বাস্তব জীবনের ডেটা (Real World Data)

বাস্তব জীবনের প্রায় প্রতিটি ডেটাসেটই নয়াজ, আউটলায়ার এবং জটিল ওভারল্যাপিংয়ে পূর্ণ থাকে। এই ধরনের বাস্তবসম্মত ডেটা (Real World Data) অত্যন্ত সফলভাবে পরিচালনা করার জন্য সফট মার্জিন এসভিএম একটি আদর্শ ও অপরিহার্য পদ্ধতি।

পেনাল্টি প্যারামিটার (Penalty Parameter - C)

সফট মার্জিন এসভিএম-এ মডেলটি কতটা ভুল ক্লাসিফিকেশন মেনে নেবে, তা একটি নির্দিষ্ট হাইপারপ্যারামিটার দ্বারা নিয়ন্ত্রিত হয়, যাকে পেনাল্টি প্যারামিটার বা C বোলা হয়।

বড় সি-এর প্রভাব (Large C)

- কম ভুল ক্লাসিফিকেশন (Less Misclassification):** C -এর মান অনেক বড় হলে মডেলটি ভুলের জন্য উচ্চ পেনাল্টি বা শাস্তি নির্ধারণ করে। ফলে মডেলটি যেকোনো মূল্যে ভুল ক্লাসিফিকেশন এড়ানোর চেষ্টা করে এবং হার্ড মার্জিনের মতো আচরণ করে।

- **ছোট মার্জিন (Small Margin):** প্রতিটি বিন্দুকে নিখুঁতভাবে ক্লাসিফাই করার প্রয়াসে ডিসিশন বাউন্ডারির দুই পাশের মার্জিনটি অত্যন্ত সংকুচিত বা ছোট (Small Margin) হয়ে যায়।

ছোট সি-এর প্রভাব (Small C)

- **অধিক সহনশীলতা (More Tolerance):** C -এর মান ছোট হলে ভুল ক্লাসিফিকেশনের ওপর পেনাল্টির কঠোরতা অনেক কমে যায়। এর ফলে মডেলটি মার্জিনের ভেতরে বা ভুল পাশে ডেটা পয়েন্টের উপস্থিতির প্রতি অধিক সহনশীলতা (More Tolerance) প্রদর্শন করে।

বড় মার্জিন (Large Margin): ব্যক্তিগত বা বিচ্ছিন্ন কিছু ভুলকে উপেক্ষা করার কারণে মডেলটি দুই ক্লাসের মাঝে একটি সুপারিসর এবং বড় মার্জিন (Large Margin) তৈরি করতে সক্ষম হয়, যা মডেলের সাধারণীকরণ ক্ষমতাকে আরও মজবুত করে।

কার্নেল ট্রিক (Kernel Trick)

কার্নেলের প্রয়োজনীয়তা (Need for Kernel)

নন-লিনিয়ার ডেটা (Non-linear Data)

বাস্তব ক্ষেত্রের অধিকাংশ ডেটাসেটই সরলরৈখিকভাবে বিন্যস্ত থাকে না। যখন বিভিন্ন ক্লাসের ডেটা পয়েন্টগুলো একে অপরের সাথে এমনভাবে মিশ্রিত বা আবর্তিত অবস্থায় থাকে যে তাদেরকে কোনো সোজা সরলরেখা বা সমতল হাইপারপ্লেন দ্বারা আলাদা করা অসম্ভব হয়ে পড়ে, তখন তাকে নন-লিনিয়ার ডেটা বলা হয়। এই ধরনের ডেটা ক্লাসিফাই করার জন্য সাধারণ লিনিয়ার এসভিএম (Linear SVM) কার্যকর হয় না।

উচ্চ-মাত্রার ম্যাপিং (High-dimensional Mapping)

নন-লিনিয়ার ডেটার এই জটিলতা দূর করার একটি জ্যামিতিক উপায় হলো ডেটা পয়েন্টগুলোকে বর্তমান মাত্রা থেকে আরও উচ্চ মাত্রার (Higher Dimension) কোনো স্পেসে স্থানান্তরিত বা ম্যাপিং করা। কম মাত্রার স্পেসে যে ডেটা লিনিয়ারলি সেপারেবল নয়, উচ্চ মাত্রার স্পেসে ম্যাপিং করার পর সেখানে একটি সরলরৈখিক হাইপারপ্লেন দ্বারা তাদেরকে খুব সহজেই আলাদা করা সম্ভব হয়।

ফিচার ম্যাপিং (Feature Mapping)

ফিচার ম্যাপিং হলো নিম্ন-মাত্রার ইনপুট স্পেস থেকে উচ্চ-মাত্রার ফিচার স্পেসে ডেটা রূপান্তরের একটি গাণিতিক প্রক্রিয়া।

[ইনপুট স্পেস] ----- (ফিচার ম্যাপিং: $\phi(x)$) -----> [ফিচার স্পেস]

(নন-লিনিয়ার ডেটা)

(লিনিয়ারলি সেপারেবল ডেটা)

ইনপুট স্পেস (Input Space)

এটি হলো ডেটার মূল বা প্রাথমিক ডাইমেনশন, যেখানে ফিচারগুলো সরাসরি ইনপুট হিসেবে অবস্থান করে। এই স্পেসে ডেটা পয়েন্টগুলো সাধারণত নন-লিনিয়ার বা গোলকধাঁধার মতো বিন্যাসে থাকে।

ফিচার স্পেস (Feature Space)

ইনপুট স্পেসের ডেটাকে গাণিতিক রূপান্তরের মাধ্যমে যখন একটি নতুন এবং উচ্চ-মাত্রার ডাইমেনশনে নিয়ে যাওয়া হয়, তখন সেই নতুন স্থানটিকে ফিচার স্পেস বলা হয়। এই স্পেসে জটিল ডেটাও লিনিয়ার ডিসিশন বাউন্ডারি দ্বারা বিভাজনযোগ্য হয়ে ওঠে।

$\phi(x)$ ফাংশন

ইনপুট স্পেসের একটি সাধারণ ভেক্টর x -কে উচ্চ-মাত্রার ফিচার স্পেসে রূপান্তরিত করার জন্য যে ম্যাপিং ফাংশন ব্যবহার করা হয়, তাকে $\phi(x)$ দ্বারা প্রকাশ করা হয়। তবে সরাসরি উচ্চ-মাত্রার ফিচার গণনা করা কম্পিউটেশনালি অত্যন্ত ব্যয়বহুল। এসভিএম-এ এই উচ্চ-মাত্রার রূপান্তরের ডট গুণফলকে সরাসরি এবং সহজে গণনা করার গাণিতিক কৌশলকেই মূলত **কার্নেল ট্রিক (Kernel Trick)** বলা হয়।

বিভিন্ন প্রকার কার্নেল (Types of Kernels)

লিনিয়ার কার্নেল (Linear Kernel)

যখন ডেটা পয়েন্টগুলো ইতিমধ্যে সরলরৈখিকভাবে পৃথকীকরণযোগ্য অবস্থায় থাকে, তখন লিনিয়ার কার্নেল ব্যবহার করা হয়। এটি ডেটার ওপর কোনো উচ্চ-মাত্রার রূপান্তর প্রয়োগ না করে সরাসরি মূল ইনপুট স্পেসেই ডিসিশন বাউন্ডারি তৈরি করে।

পলিনোমিয়াল কার্নেল (Polynomial Kernel)

পলিনোমিয়াল কার্নেল ইনপুট ফিচারের বিভিন্ন ঘাত বা শক্তির সংমিশ্রণ তৈরি করে ডেটাকে উচ্চ মাত্রায় নিয়ে যায়। এটি একটি নির্দিষ্ট সীমার মধ্যে ফিচারগুলোর মধ্যকার পারস্পরিক সম্পর্ক বা ইন্টারঅ্যাকশন বিবেচনা করতে সাহায্য করে।

- **হাইপারপ্যারামিটার - Degree:** পলিনোমিয়াল কার্নেলের প্রধান হাইপারপ্যারামিটার হলো এর ডিগ্রি (Degree)। ডিগ্রির মান নির্ধারণ করে যে ডেটাকে কত উচ্চ-মাত্রার পলিনোমিয়াল স্পেসে রূপান্তর করা হবে। ডিগ্রির মান যত বেশি হবে, ডিসিশন বাউন্ডারি তত বেশি জটিল আকার ধারণ করবে।

গাউসিয়ান / আরবিএফ কার্নেল (Gaussian / RBF Kernel)

রেডিয়াল বেসিস ফাংশন (RBF) বা গাউসিয়ান কার্নেল হলো এসভিএম-এর সবচেয়ে জনপ্রিয় এবং শক্তিশালী কার্নেলগুলোর একটি। এটি ডেটা পয়েন্টগুলোকে একটি অসীম-মাত্রার (Infinite-dimensional) ফিচার স্পেসে ম্যাপিং করতে পারে, যার ফলে অত্যন্ত জটিল এবং বৃত্তাকার ডেটা বিন্যাসকেও নিখুঁতভাবে আলাদা করা সম্ভব হয়।

- **হাইপারপ্যারামিটার - Gamma:** আরবিএফ কার্নেলের আচরণ নিয়ন্ত্রণ করার প্রধান হাইপারপ্যারামিটার হলো গামা (γ)। গামার মান নির্ধারণ করে একটি একক ট্রেনিং পয়েন্টের প্রভাব ডিসিশন বাউন্ডারির ওপর কতদূর পর্যন্ত বিস্তৃত থাকবে।

অধ্যায়: হাইপারপ্যারামিটার টিউনিং (Hyperparameter Tuning)

সাপোর্ট ভেক্টর ক্লাসিফায়ারের প্যারামিটারসমূহ (Support Vector Classifier Parameters)

একটি সাপোর্ট ভেক্টর ক্লাসিফায়ার (SVC) মডেলের কার্যকারিতা এবং বাউন্ডারির আকৃতি মূলত ৪টি প্রধান হাইপারপ্যারামিটারের ওপর নির্ভর করে:

- **C:** এটি হলো সফট মার্জিনের পেনাল্টি প্যারামিটার, যা ক্লাসিফিকেশন ভুল এবং মার্জিনের আকারের মধ্যে ভারসাম্য বজায় রাখে।
- **Kernel:** এটি মডেলের জন্য উপযুক্ত কার্নেল টাইপ (যেমন: 'linear', 'poly', 'rbf') নির্বাচন করে জটিল ডেটা প্রসেস করার পথ নির্ধারণ করে।
- **Gamma:** এটি মূলত RBF কার্নেলের জন্য ব্যবহৃত প্যারামিটার, যা ডিসিশন বাউন্ডারির বক্রতা ও স্পর্শকাতরতা নিয়ন্ত্রণ করে।
- **Degree:** এটি পলিনোমিয়াল কার্নেলের জন্য প্রযোজ্য প্যারামিটার, যা রূপান্তরকারী পলিনোমিয়ালের সর্বোচ্চ ঘাত নির্ধারণ করে।

অধ্যায়: ব্যবহারিক সারাংশ (Practical Summary)

মডেল নির্বাচন (Model Selection)

- **Linear Kernel কখন ব্যবহার করবেন:** যখন ডেটাসেটে ফিচারের সংখ্যা নমুনার সংখ্যার তুলনায় অনেক বেশি থাকে (যেমন: টেক্সট ক্লাসিফিকেশন) অথবা ডেটা যখন স্বভাবগতভাবেই সরলরৈখিক বিন্যাসে থাকে।
- **Polynomial Kernel কখন ব্যবহার করবেন:** যখন ডেটার ফিচারগুলোর মধ্যে একটি সুনির্দিষ্ট এবং সীমিত মাত্রার ঘাতভিত্তিক সম্পর্ক বিদ্যমান থাকে।

- **RBF Kernel কখন ব্যবহার করবেন:** বাস্তব জীবনের অধিকাংশ জটিল, ওভারল্যাপড এবং সম্পূর্ণ নন-লিনিয়ার ডেটাসেটের জন্য RBF কার্নেলকে প্রথম পছন্দ হিসেবে বিবেচনা করা হয়।

হাইপারপ্যারামিটার নির্বাচন (Choosing Hyperparameters)

Large C বনাম Small C

- **Large C (উচ্চ পেনাল্টি):** মডেলটি কোনো ভুল ক্লাসিফিকেশন মেনে নিতে চায় না, ফলে মার্জিন ছোট হয়ে যায়। এটি ট্রেনিং ডেটাকে নিখুঁতভাবে ফিট করলেও মডেলটিতে ওভারফিটিং (Overfitting)-এর ঝুঁকি তৈরি হয়।
- **Small C (কম পেনাল্টি):** মডেলটি কিছু ভুল ক্লাসিফিকেশন উপেক্ষা করে একটি বড় মার্জিন তৈরি করে। এর ফলে মডেলটির সাধারণীকরণ ক্ষমতা বৃদ্ধি পায় এবং ওভারফিটিং রোধ হয়।

Large Gamma বনাম Small Gamma

- **Large Gamma:** গামার মান বড় হলে প্রতিটি ডেটা পয়েন্টের প্রভাবের ব্যাসার্ধ কমে যায়। ডিসিশন বাউন্ডারিটি কেবল তার খুব কাছের পয়েন্টগুলোর ওপর ভিত্তি করে তৈরি হয়, যা বাউন্ডারিকে অত্যন্ত আঁকাবাঁকা ও জটিল করে তোলে এবং ওভারফিটিং ঘটায়।
- **Small Gamma:** গামার মান ছোট হলে দূরবর্তী পয়েন্টগুলোর প্রভাবও বাউন্ডারি গণনায় অন্তর্ভুক্ত হয়। এর ফলে ডিসিশন বাউন্ডারিটি অনেক মসৃণ (Smooth) এবং বিস্তৃত হয়, যা মডেলের জেনারেলাইজেশন উন্নত করে।

ডিগ্রি নির্বাচন (Degree Selection)

পলিনোমিয়াল কার্নেলের ক্ষেত্রে ডিগ্রির মান নির্বাচনের সময় সতর্কতা অবলম্বন করতে হয়। কম ডিগ্রি (যেমন: 2 বা 3) মডেলকে সরল ও কার্যকর রাখে। ডিগ্রির মান অতিরিক্ত বেশি হলে মডেলটি কম্পিউটেশনালি অত্যন্ত ধীরগতির হয়ে পড়ে এবং ট্রেনিং ডেটার গোলমালের প্রতি অতিরিক্ত সংবেদনশীল হয়ে ওভারফিট করে ফেলে।

হাইপারপ্যারামিটার বিশ্লেষণ (Hyperparameters)

C

- **Default:** 1.0
- **কাজ কী:** এটি হলো রেগুলারাইজেশন (Regularization) বা পেনাল্টি প্যারামিটার। মডেলটি ট্রেনিং ডেটায় কোনো ভুল বা মিস-ক্লাসিফিকেশন করলে তাকে কতটা কঠোর গাণিতিক শাস্তি (Penalty) দেওয়া হবে, তা এটি নিয়ন্ত্রণ করে।
- **অন্যান্য অপশন:** যেকোনো ধনাত্মক ফ্লোট সংখ্যা (যেমন: 0.1, 10, 100)।
- **কখন কোনটি ব্যবহার করব:**
 - **C-এর মান অনেক বেশি হলে (যেমন: ১০০):** মডেল কোনো ভুল সহ্য করবে না (Hard Margin-এর মতো কাজ করবে)। এটি ট্রেনিং ডেটার প্রতিটি পয়েন্ট নিখুঁতভাবে মেলানোর চেষ্টা করবে, যা মডেলকে জটিল করে এবং **Overfitting (High Variance)**-এর ঝুঁকি বাড়ায়।
 - **C-এর মান অনেক কম হলে (যেমন: 0.1):** মডেল কিছুটা ভুল বা মার্জিন ওভারল্যাপ করার অনুমতি দেবে (**Soft Margin**)। এটি মডেলকে সরল রাখে এবং **Underfitting (High Bias)**-এর দিকে নিয়ে যেতে পারে। ডেটাতে নয়েজ বেশি থাকলে C-এর মান কমিয়ে দেওয়া উচিত।

kernel

- **Default:** 'rbf' (Radial Basis Function / Gaussian Kernel)
- **কাজ কী:** ইনপুট ডেটাকে কোন গাণিতিক ফাংশনের মাধ্যমে উচ্চতর মাত্রায় (Higher Dimension) রূপান্তর করা হবে, তা এটি ঠিক করে।
- **অন্যান্য অপশন:** 'linear', 'poly', 'sigmoid', অথবা কাস্টম কোনো ফাংশন।
- **কখন কোনটি ব্যবহার করব:**
 - **'linear':** আপনার ডেটা যদি অলরেডি সরলরেখায় বিভাজনযোগ্য (Linearly Separable) হয়, তবে এটি ব্যবহার করা উচিত। এটি সবচেয়ে ফাস্ট কাজ করে।
 - **'poly':** উপাত্তের মধ্যে যদি বহুপদী বা বাঁকা কোনো প্যাটার্ন থাকে।
 - **'rbf':** বাস্তব জীবনের বেশিরভাগ জটিল এবং অরৈখিক (Non-linear) ডেটার জন্য এটি ডিফল্ট এবং সেরা চয়েস। এটি ডেটাকে অসীম মাত্রায় কল্পনা করে বৃত্তাকার বা গোলকধাঁধার মতো সীমানা তৈরি করতে পারে।

gamma

- **Default: 'scale'**
- **কাজ কী:** এটি শুধুমাত্র 'rbf', 'poly', এবং 'sigmoid' কার্নেলের ক্ষেত্রে কাজ করে। একটি একক সাপোর্ট ভেক্টরের প্রভাব বা বিস্তার কত দূর পর্যন্ত ছড়াবে, তা এটি নির্ধারণ করে।
- **অন্যান্য অপশন:** 'auto' অথবা যেকোনো ধনাত্মক দশমিক সংখ্যা বা ফ্লোট মান (যেমন: 0.01, 1.0)।
- **কখন কোনটি ব্যবহার করব:**
 - মান অনেক বেশি হলে (High Gamma): সাপোর্ট ভেক্টরের প্রভাব শুধু তার খুব কাছের সীমানায় সীমাবদ্ধ থাকে। ফলে সিদ্ধান্ত রেখাটি অত্যন্ত আঁকাবাঁকা ও জটিল হয়, যা মডেলে Overfitting তৈরি করতে পারে।
 - মান অনেক কম হলে (Low Gamma): সাপোর্ট ভেক্টরের প্রভাব অনেক দূর পর্যন্ত বিস্তৃত হয়। এর ফলে সিদ্ধান্ত রেখাটি অনেক বেশি সোজা ও মসৃণ (Smooth) হয়ে যায়, যা ডেটার সূক্ষ্ম প্যাটার্ন মিস করে Underfitting ঘটাতে পারে।

epsilon (শুধুমাত্র SVR-এর জন্য)

- **Default: 0.1**
- **কাজ কী:** এটি সাপোর্ট ভেক্টর রিগ্রেশনের (SVR) একটি অত্যন্ত অনন্য প্যারামিটার। এটি হাইপারপ্লেনের উভয় পাশে একটি অদৃশ্য টিউব বা মার্জিন তৈরি করে। এই টিউবের ভেতরে থাকা কোনো ভুলের জন্য মডেল কোনো পেনাল্টি বা শাস্তি গণনা করে না।
- **অন্যান্য অপশন:** যেকোনো অ-ঋণাত্মক ফ্লোট সংখ্যা।
- **কখন কোনটি ব্যবহার করব:** যদি আপনি চান আপনার রিগ্রেশন মডেলটি ছোটখাটো নয়েজ বা ছোট ভুলগুলোকে সম্পূর্ণ ইগনোর করুক, তবে এর মান বাড়িয়ে দেওয়া যায়। আর যদি আপনি চান প্রতিটি ডেটা পয়েন্টের জন্য মডেল চরম নিখুঁত হোক, তবে এর মান কমিয়ে শূন্যের কাছাকাছি নিয়ে আসতে হবে।

degree

- **Default: 3**
- **কাজ কী:** এটি শুধুমাত্র তখনই কাজ করে যখন আপনি kernel='poly' (Polynomial) সিলেক্ট করেন। এটি বহুপদী সমীকরণের সর্বোচ্চ পাওয়ার বা ডিগ্রি নির্ধারণ করে।
- **অন্যান্য অপশন:** যেকোনো পূর্ণসংখ্যা (যেমন: 2, 4, 5)।

- **কখন কোনটি ব্যবহার করব:** যদি ডেটার বক্রতা বা প্যাটার্ন অনেক বেশি জটিল হয়, তবে ডিগ্রি বাড়িয়ে ৪ বা ৫ করা যায়। তবে ডিগ্রি অতিরিক্ত বাড়ালে কম্পিউটেশনাল কস্ট এবং ওভারফিটিংয়ের সম্ভাবনা মারাত্মকভাবে বেড়ে যায়।

coef0

- **Default:** 0.0
- **কাজ কী:** এটি শুধুমাত্র 'poly' এবং 'sigmoid' কার্ণেলের ক্ষেত্রে কাজ করে। এটি উচ্চ মাত্রার সমীকরণে একটি স্বাধীন ধ্রুবক বা বায়াস (Independent term) যোগ করে, যা উচ্চ মাত্রার ম্যাপিংকে কিছুটা শিফট বা নিয়ন্ত্রণ করতে সাহায্য করে।

shrinking

- **Default:** True
- **কাজ কী:** এটি একটি অস্টিমাইজেশন ট্রিক। ট্রেইনিং প্রক্রিয়ার শুরুতে যে ডেটা পয়েন্টগুলো সিদ্ধান্ত সীমানা থেকে অনেক দূরে থাকে এবং যেগুলো সাপোর্ট ভেক্টর হওয়ার কোনো সম্ভাবনাই নেই, সেগুলোকে ব্যাকএন্ডের বড় গাণিতিক হিসাব থেকে আগেই বাদ দিয়ে দেওয়া হয়। এর ফলে মডেল অনেক দ্রুত ট্রেইন হয়। এটি সাধারণত সর্বদা True রাখাই স্ট্যান্ডার্ড।

probability (SVC-এর জন্য)

- **Default:** 'deprecated' (পুরোনো সংস্করণে এটি False থাকত)
- **কাজ কী:** এটি True করা হলে মডেলটি শুধু সরাসরি ক্লাস (যেমন: +1 বা -1) প্রেডিক্ট না করে, প্রতিটি ক্লাসের নিখুঁত সম্ভাব্যতা বা প্রোবাবিলিটি (Probability Scores) হিসাব করতে পারে (যেমন: এই ডেটাটি পজিটিভ হওয়ার সম্ভাবনা ৮৫%)।
- **কখন ব্যবহার করব:** আপনার প্রজেক্টে যদি predict_proba() ফাংশন ব্যবহার করার দরকার পড়ে (যেমন: ROC-AUC কার্ভ আঁকার জন্য), তখন এটি True করতে হবে। তবে এটি অন করলে ব্যাকএন্ডে **Platt Scaling** নামের একটি ৫-ফোল্ড ক্রস-ভ্যালিডেশন লুপ চলে, যা মডেলের ট্রেইনিং টাইম মারাত্মকভাবে বাড়িয়ে দেয়।

tol

- **Default:** 0.001 (অর্থাৎ 10^{-3})
- **কাজ কী:** এটি হলো অস্টিমাইজেশন থামানোর জন্য সহনশীলতার মাত্রা বা Tolerance। ব্যাকএন্ডে লুপ চলার সময় ভুলের উন্নতির হার যদি এই মানের চেয়ে কম হয়ে যায়, তবে মডেল ধরে নেয় সে তার বেস্ট হাইপারপ্যারামেটার পেয়ে গেছে এবং ট্রেইনিং স্টপ করে দেয়।

cache_size

- **Default:** 200 (মেগাবাইট)
- **কাজ কী:** SVM ব্যাকএন্ডে প্রচুর ম্যাট্রিক্স ডট প্রোডাক্ট হিসাব করে। র‍্যামের (RAM) ঠিক কত মেগাবাইট জায়গা এই হিসাবের ক্যাশ মেমোরি হিসেবে লক করে রাখা হবে, তা এটি ঠিক করে। বড় ডেটাসেটে ট্রেনিং স্পিড বাড়াতে এটিকে বাড়িয়ে ৫০০ বা ১০০০ করা যেতে পারে।

class_weight (শুধুমাত্র SVC-এর জন্য)

- **Default:** None
- **কাজ কী:** প্রবল ইমব্যালেন্সড ডেটাসেটের ক্ষেত্রে ছোট ক্লাসকে অতিরিক্ত গাণিতিক গুরুত্ব দেওয়ার জন্য ব্যবহৃত হয়। class_weight='balanced' দিলে মডেল স্বয়ংক্রিয়ভাবে মাইনোরিটি ক্লাসের ভুলের জন্য পেনাল্টি বাড়িয়ে দেয়।

verbose

- **Default:** False
- **কাজ কী:** এটি True করে দিলে ব্যাকএন্ডে অপটিমাইজেশনের ক্যালোকেশনের লাইভ লগ বা টেক্সট টার্মিনালে প্রিন্ট করে দেখায়। সাধারণত এটি রিগ্রেশন বা ক্লাসিফিকেশনের সাধারণ প্রজেক্টে False-ই রাখা হয়।

max_iter

- **Default:** -1 (অর্থাৎ কোনো নির্দিষ্ট সীমা নেই, যতক্ষণ না অপটিমাইজেশন শেষ হচ্ছে চলতেই থাকবে)
- **কাজ কী:** ব্যাকএন্ডের কোয়ান্ট্রটিক প্রোগ্রামিং সলভার সর্বোচ্চ কতবার লুপ বা ইটারেশন চালাবে, তা বেঁধে দেওয়া। যদি মডেলটি বিশাল ডেটাসেটে আটকে গিয়ে অনন্তকাল ধরে চলতে থাকে, তবে এখানে একটি নির্দিষ্ট সংখ্যা (যেমন: ৫০০০) সেট করে ট্রেনিং ফোর্স-স্টপ করা যায়।

decision_function_shape (শুধুমাত্র SVC-এর জন্য)

- **Default:** 'ovr' (One-vs-Rest)
- **কাজ কী:** মাল্টি-ক্লাস ক্লাসিফিকেশনের ক্ষেত্রে (যেমন: ৩ বা তার বেশি ক্লাস থাকলে) মডেলটি ব্যাকএন্ডে কীভাবে কাজ করবে তা ঠিক করে। 'ovr' প্রতিটি ক্লাসের বিপরীতে বাকি সব ক্লাসকে ধরে আলাদা আলাদা বাইনারি মডেল বানায়। অন্য অপশনটি হলো 'ovo' (One-vs-One), যা প্রতি দুটি ক্লাসের জোড়া জোড়া নিয়ে আলাদা মডেল তৈরি করে (এটি কম্পিউটেশনালি অনেক ভারী)।

break_ties (শুধুমাত্র SVC-এর জন্য)

- **Default:** False
- **কাজ কী:** এটি শুধুমাত্র তখনই কাজ করে যখন `decision_function_shape='ovr'` থাকে এবং ক্লাসের সংখ্যা ২-এর বেশি হয়। যদি কোনো টেস্ট ডেটার ক্ষেত্রে দুটি ক্লাসের প্রোবাবিলিটি স্কোর হুবহু এক চলে আসে (টাই হয়), তখন True দেওয়া থাকলে মডেল তার ভেতরের গাণিতিক ডিসিশন ভ্যালুর সূক্ষ্ম ব্যবধান দেখে সেই টাই ভেঙে একটি চূড়ান্ত ক্লাস ঘোষণা করে।

random_state

- **Default:** None
- **কাজ কী:** যদিও SVM একটি ডিটারমিনিস্টিক (নির্দিষ্ট) মডেল, কিন্তু যখন `probability=True` করা হয়, তখন ব্যাকএন্ডের বুটস্ট্র্যাপ এবং প্ল্যাট স্কেলিং প্রক্রিয়ার র্যান্ডমেনেসকে লক করার জন্য একটি নির্দিষ্ট গাণিতিক বীজ (Seed) বা সংখ্যা (যেমন: 42) এখানে ব্যবহার করা হয়, যাতে প্রতিবার অবিকল একই রেজাল্ট আসে।

SVC() Attributes

support_vectors_

Default: Created after fit()

Decision Boundary নির্ধারণ করার জন্য ব্যবহৃত প্রকৃত Support Vector Data Point-গুলো সংরক্ষণ করে। শুধুমাত্র এই Point-গুলোর উপর ভিত্তি করেই SVM Decision Boundary তৈরি করে।

support_

Default: Created after fit()

Training Data-এর মধ্যে কোন কোন Sample Support Vector হিসেবে নির্বাচিত হয়েছে, তাদের Index সংরক্ষণ করে।

dual_coef_

Default: Created after fit()

প্রতিটি Support Vector-এর Dual Coefficient ($\alpha \times y$) সংরক্ষণ করে। Decision Function গণনার সময় এই Coefficient ব্যবহার করা হয়।

n_support_

Default: Created after fit()

প্রতিটি Class-এ কতটি Support Vector রয়েছে তা সংরক্ষণ করে। Binary Classification-এ এটি সাধারণত দুইটি সংখ্যা ধারণ করে।

coef_

Default: Created after fit() (*Only when kernel='linear'*)

Linear Kernel ব্যবহার করলে প্রতিটি Feature-এর Learned Weight (Coefficient) সংরক্ষণ করে। Non-linear Kernel-এর ক্ষেত্রে এই Attribute পাওয়া যায় না।

intercept_

Default: Created after fit()

Decision Function-এর Bias (Intercept) সংরক্ষণ করে।

classes_

Default: Created after fit()

Training Data-তে থাকা সকল Target Class-এর List সংরক্ষণ করে।

fit_status_

Default: Created after fit()

Model সফলভাবে Train হয়েছে কিনা তা নির্দেশ করে। 0 হলে Training সফল হয়েছে, অন্য মান হলে Training-এ সমস্যা হয়েছে।

shape_fit_

Default: Created after fit()

Training Data-এর Shape (n_samples, n_features) সংরক্ষণ করে।

probA_

Default: Created after fit() (*Only when probability=True*)

Probability Estimation-এর জন্য ব্যবহৃত Platt Scaling-এর Parameter A সংরক্ষণ করে।

probB_

Default: Created after fit() (*Only when probability=True*)

Probability Estimation-এর জন্য ব্যবহৃত Platt Scaling-এর Parameter B সংরক্ষণ করে।

class_weight_

Default: Created after fit() (*When class_weight is used*)

Training-এর সময় প্রতিটি Class-এর জন্য ব্যবহৃত Effective Weight সংরক্ষণ করে।

n_features_in_

Default: Created after fit()

Training-এর সময় Model মোট কতটি Input Feature ব্যবহার করেছে তা সংরক্ষণ করে।

feature_names_in_

Default: Created after fit() (*DataFrame input only*)

Training Data যদি Pandas DataFrame হয়, তাহলে Feature-গুলোর নাম সংরক্ষণ করে।

SVR() Attributes

support_vectors_

Default: Created after fit()

Regression Model-এর Decision Function নির্ধারণ করার জন্য ব্যবহৃত প্রকৃত Support Vector-গুলো সংরক্ষণ করে।

support_

Default: Created after fit()

Training Data-এর মধ্যে কোন কোন Sample Support Vector হিসেবে নির্বাচিত হয়েছে, তাদের Index সংরক্ষণ করে।

dual_coef_

Default: Created after fit()

প্রতিটি Support Vector-এর Dual Coefficient সংরক্ষণ করে। Prediction করার সময় এই মানগুলো ব্যবহার করা হয়।

coef_

Default: Created after fit() (*Only when kernel='linear'*)

Linear Kernel ব্যবহার করলে প্রতিটি Feature-এর Learned Weight সংরক্ষণ করে। অন্য Kernel-এর ক্ষেত্রে এই Attribute পাওয়া যায় না।

intercept_

Default: Created after fit()

Regression Function-এর Bias (Intercept) সংরক্ষণ করে।

fit_status_

Default: Created after fit()

Model সফলভাবে Train হয়েছে কিনা তা নির্দেশ করে। 0 হলে Training সফল হয়েছে।

shape_fit_

Default: Created after fit()

Training Data-এর Shape (n_samples, n_features) সংরক্ষণ করে।

n_features_in_

Default: Created after fit()

Training-এর সময় ব্যবহৃত মোট Input Feature-এর সংখ্যা সংরক্ষণ করে।

feature_names_in_

Default: Created after fit() (*DataFrame input only*)

Training Data যদি Pandas DataFrame হয়, তাহলে Feature-গুলোর নাম সংরক্ষণ করে।

Code:

```
import pandas as pd
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report

cancer = load_breast_cancer()
X = pd.DataFrame(cancer.data, columns=cancer.feature_names)
y = pd.Series(cancer.target)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

numeric_features = X.columns.tolist()
preprocessor = ColumnTransformer(
    transformers=[
        ('num', StandardScaler(), numeric_features)
    ]
)

svm_pipeline = Pipeline(
    steps=[
        ('preprocessor', preprocessor),
        ('model', SVC(random_state=42))
    ]
)

param_grid = {
    'model__C': [0.1, 1, 10],
    'model__kernel': ['linear', 'rbf'],
    'model__gamma': ['scale', 'auto']
}
```

```
grid_search = GridSearchCV(
    estimator=svm_pipeline,
    param_grid=param_grid,
    cv=5,
    scoring='accuracy',
    n_jobs=-1
)

grid_search.fit(X_train, y_train)

print("--- GridSearchCV Tuning Results ---")
print("Best Parameters Found:", grid_search.best_params_)
print(f"Best CV Accuracy: {grid_search.best_score_:.4f}\n")

best_svm_model = grid_search.best_estimator_
y_pred = best_svm_model.predict(X_test)

print("--- Final Model Performance on Test Set ---")
print(f"Test Accuracy: {accuracy_score(y_test, y_pred):.4f}\n")
print(classification_report(y_test, y_pred))
```

Code:

```
wine = load_wine()
X = pd.DataFrame(wine.data, columns=wine.feature_names)
y = pd.Series(wine.target)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)

numeric_features = X.columns.tolist()
preprocessor = ColumnTransformer(
    transformers=[
        ('num', StandardScaler(), numeric_features)
    ]
)

svm_pipeline = Pipeline(
    steps=[
        ('preprocessor', preprocessor),
        ('model', SVC(random_state=42))
    ]
)
```

```
grid_param = [
    {
        "model__kernel": ['linear'],
        "model__C": [0.01, 0.1, 1, 10, 50, 100]
    },
    {
        "model__kernel": ['rbf'],
        "model__C": [0.01, 0.1, 1, 100],
        "model__gamma": [0.01, 0.1, 5, 10, 'scale', 'auto']
    },
    {
        "model__kernel": ['poly'],
        "model__C": [0.01, 0.1, 1, 100],
        "model__degree": [2, 3]
    }
]
```

```
grid_search = GridSearchCV(
    estimator=svm_pipeline,
    param_grid=grid_param,
    cv=5,
    scoring='accuracy',
    n_jobs=-1
)

grid_search.fit(X_train, y_train)

print("--- GridSearchCV Tuning Results ---")
print("Best Parameters Found:", grid_search.best_params_)
print(f"Best CV Accuracy: {grid_search.best_score_:.4f}\n")

best_svm_model = grid_search.best_estimator_
y_pred = best_svm_model.predict(X_test)

print("--- Final Model Performance on Test Set ---")
print(f"Test Accuracy: {accuracy_score(y_test, y_pred):.4f}\n")
print(classification_report(y_test, y_pred, target_names=wine.target_names))
```

Code:

```
preprocessor = ColumnTransformer(  
    transformers=[  
        ('num', StandardScaler(), X_train.select_dtypes(include=np.number).columns)  
    ]  
)  
  
pipeline = Pipeline(steps=[  
    ('preprocessor', preprocessor),  
    ('classifier', SVC())  
)  
  
param_grid = {  
    'classifier__C': [0.1, 0.5, 1, 5, 10],  
    #'classifier__penalty': ['l1', 'l2'],  
    'classifier__kernel': ['rbf', 'linear', 'poly', 'sigmoid'],  
    #'classifier__epsilon': [0.1, 0.2, 0.5, 1, 2, 5, 10],  
    'classifier__shrinking': [True, False]  
}  
  
grid_search = GridSearchCV(  
    estimator = pipeline,  
    param_grid = param_grid,  
    scoring = 'accuracy',  
    cv = 5,  
    n_jobs = -1  
)
```

```
grid_search.fit(X_train, y_train)  
grid_search.best_params_  
grid_search.best_score_  
best_model = grid_search.best_estimator_  
y_pred = best_model.predict(X_test)
```

Q1: SVM-এ ফিচার স্কেলিং (Feature Scaling) করা কেন বাধ্যতামূলক, যেখানে র‍্যান্ডম ফরেস্ট এর প্রয়োজন হয় না?

- **Answer:** SVM মূলত একটি জ্যামিতিক দূরত্ব-ভিত্তিক (Distance-based) অ্যালগরিদম। এটি বিভিন্ন ক্লাসের মধ্যকার দূরত্ব বা মার্জিন সর্বোচ্চ করার জন্য সাপোর্ট ভেক্টরগুলোর ইউক্লিডিয়ান দূরত্ব (Euclidean Distance) গণনা করে। যদি কোনো একটি ফিচারের রেঞ্জ অনেক বড় হয় (যেমন: Income = ৫০,০০০) এবং অন্যটির রেঞ্জ ছোট হয় (যেমন: Age = ২), তবে দূরত্বের হিসাবে বড় রেঞ্জের ফিচারটি পুরো মডেলকে ডমিনেন্ট করবে এবং ছোট ফিচারের গুরুত্ব হারিয়ে যাবে। তাই সব ফিচারকে সমান গুরুত্ব দিতে স্কেলিং বাধ্যতামূলক। অন্যদিকে, র‍্যান্ডম ফরেস্ট ডিসিশন ট্রির রুলস মেনে নোড স্প্লিট করে, যা দূরত্বের ওপর নির্ভর করে না।

Q2: 'Kernel Trick' বলতে কী বোঝায়? এটি কি আসলেই ডেটার ডাইমেনশন বাড়িয়ে দেয়?

- **Answer:** যখন কোনো ডেটাসেটকে লিনিয়ার লাইনে আলাদা করা যায় না, তখন কার্নেল ট্রিক ব্যবহৃত হয়। তাত্ত্বিকভাবে এটি ডেটাকে একটি উচ্চতর মাত্রায় (Higher Dimension) ট্রান্সফর্ম করে যেন সেখানে একটি লিনিয়ার হাইপারপ্লেন দিয়ে ক্লাসগুলোকে আলাদা করা যায়। তবে এটি বাস্তবে বা ফিজিক্যালি ডেটার কোনো নতুন কলাম তৈরি করে না বা উচ্চ মাত্রায় ডেটা রিলোড করে না। এটি একটি গাণিতিক শর্টকাট ফাংশন, যা মূল লো-ডাইমেনশনে বসেই উচ্চ ডাইমেনশনের ডট প্রোডাক্টের $(x_i \cdot x_j)$ চূড়ান্ত ফলাফল সরাসরি গণনা করে দেয়। এর ফলে মেমোরি এবং কম্পিউটেশনাল সময় মারাত্মকভাবে বেঁচে যায়।

Q3: SVM-এর C প্যারামিটারটি কীভাবে Bias এবং Variance-এর ভারসাম্য (Trade-off) নিয়ন্ত্রণ করে?

- **Answer:** C প্যারামিটারটি মূলত রেগুলারাইজেশন এবং ট্রেইনিং এররের পেনাল্টি নিয়ন্ত্রণ করে:
 - **C -এর মান খুব বেশি হলে:** মডেল ট্রেইনিং সেটে কোনো ভুল করতে চায় না (Hard Margin)। এটি প্রতিটি নয়েজ পয়েন্টকে মেলানোর জন্য সিদ্ধান্ত সীমানাকে অনেক জটিল ও আঁকাবাঁকা করে তোলে। এর ফলে মডেলের **Bias কমে (Low Bias)** কিন্তু **Variance মারাত্মক বাড়ে (High Variance)**, যা ওভারফিটিং ঘটায়।
 - **C -এর মান খুব কম হলে:** মডেল কিছুটা ভুল বা মার্জিন ওভারল্যাপ করার ছাড় দেয় (Soft Margin)। সিদ্ধান্ত সীমানাটি খুব সরল বা মসৃণ হয়। এর ফলে মডেলের **Variance কমে (Low Variance)** কিন্তু **Bias বেড়ে যায় (High Bias)**, যা আন্ডারফিটিং ঘটাতে পারে।

Q4: SVM-এর প্রধান সীমাবদ্ধতা বা ডিসঅ্যাডভান্টেজগুলো কী কী?

- **Answer:** শক্তিশালী মডেল হলেও SVM-এর কিছু বড় সীমাবদ্ধতা রয়েছে:
 - **বৃহৎ ডেটাসেটে ধীরগতি:** ডেটার সংখ্যা (*Rows*) অনেক বেশি হলে (যেমন: ১ লাখের উপরে) এর ব্যাকএন্ডের কোয়াদ্রেটিক প্রোগ্রামিং হিসাব করতে প্রচুর সময় ও কম্পিউটেশনাল পাওয়ার লাগে।
 - **নয়েজি ডেটায় দুর্বলতা:** ডেটাসেটে যদি ক্লাসের সীমানাগুলো একে অপরের ভেতর অতিরিক্ত ঢুকে থাকে (*High Overlapping Noise*), তবে SVM ভালো মার্জিন তৈরি করতে পারে না।
 - **প্রোবাবিলিটি স্কেরের অভাব:** এটি লজিস্টিক রিগ্রেশনের মতো স্বয়ংক্রিয়ভাবে কোনো সম্ভাব্যতা স্কের (*predict_proba*) দেয় না। এটি অন করতে গেলে প্ল্যাট স্কেলিং মেথড ব্যবহার করতে হয় যা মডেলকে আরও ধীরগতির করে তোলে।

Q5: 'Support Vectors' বলতে কী বোঝায় এবং মডেলের প্রেডিকশনে এদের ভূমিকা কী?

- **Answer:** ট্রেনিং ডেটাসেটের যে নির্দিষ্ট বিন্দু বা স্যাম্পলগুলো সিদ্ধান্ত সীমানার (*Hyperplane*) সবচেয়ে কাছাকাছি অবস্থান করে এবং মার্জিনাল লাইন লক করতে সরাসরি অংশ নেয়, সেগুলোকে **Support Vectors** বলে। ট্রেনিং শেষ হওয়ার পর, SVM বাকি সব সাধারণ ডেটা পয়েন্ট মেমোরি থেকে সম্পূর্ণ ডিলিট করে দেয় এবং শুধুমাত্র এই সাপোর্ট ভেক্টরগুলোকে ধরে রাখে। নতুন কোনো অজানা টেস্ট ডেটা আসলে, মডেলটি কেবল এই সাপোর্ট ভেক্টরগুলোর সাপেক্ষে দূরত্ব মেপে তার ক্লাস নির্ধারণ করে।

সাপোর্ট ভেক্টর মেশিন (SVM) — সারসংক্ষেপ

১. মূল দর্শন ও জ্যামিতিক উপাদান (Core Philosophy & Geometry)

- হাইপারপ্লেন (Hyperplane):** SVM হলো একটি সুপারভাইজড লার্নিং অ্যালগরিদম, যার মূল লক্ষ্য হলো বহুমাত্রিক উপাত্ত ক্ষেত্রে (n -Dimensional Space) একটি লিনিয়ার সিদ্ধান্ত সীমানা বা হাইপারপ্লেন ($w^T x + b = 0$) তৈরি করা।
- জ্যামিতিক রূপান্তর:** মাত্রাভেদে এই বাউন্ডারিটি ভিন্ন রূপ নেয়; যেমন—দ্বিমাত্রিকে (2D) এটি একটি সরলরেখা, ত্রিমাত্রিকে (3D) একটি সমতল পৃষ্ঠ বা প্লেন, এবং তার অধিক মাত্রায় এটি একটি হাইপারপ্লেন।
- সাপোর্ট ভেক্টরস (Support Vectors):** সিদ্ধান্ত রেখার সবচেয়ে নিকটে এবং মার্জিনাল বাউন্ডারির ($w^T x + b = \pm 1$) ওপর অবস্থিত যে বিশেষ বিন্দুগুলো হাইপারপ্লেনের অবস্থান ও অভিযোজন (Orientation) সরাসরি নির্ধারণ করে, তাদের সাপোর্ট ভেক্টর বলে।

২. মার্জিন সর্বোচ্চকরণ ও গাণিতিক ভিত্তি (Margin Maximization & Mathematics)

- ম্যাক্সিমাম মার্জিন ক্লাসিফায়ার:** SVM কেবল ডেটা আলাদাই করে না, বরং দুই ক্লাসের মধ্যবর্তী সম্ভাব্য সবচেয়ে চওড়া রাস্তা বা সর্বোচ্চ মার্জিন (Maximum Margin) তৈরি করে।
- গাণিতিক সমীকরণ:** জ্যামিতিক প্রতিপাদন অনুযায়ী মোট মার্জিনের দূরত্ব হলো:

$$\text{Margin } (M) = \frac{2}{\|w\|}$$

- অপটিমাইজেশন লজিক:** গাণিতিক নিয়ম অনুযায়ী, ভগ্নাংশের হর $\|w\|$ যত হ্রাস পাবে, মোট মার্জিন (M) তত বৃদ্ধি পাবে। তাই ক্যালকুলাসের সুবিধার্থে SVM-এর মূল অপটিমাইজেশন সমস্যাটিকে একটি সমতুল্য সর্বনিম্নকরণ সমস্যা হিসেবে রূপ দেওয়া হয়:

$$\min \frac{1}{2} \|w\|^2 \quad \text{subject to} \quad y_i(w^T x_i + b) \geq 1$$

৩. লস ফাংশন এবং আউটলায়ার প্রতিরোধ ক্ষমতা (Loss Function & Robustness)

- হিঞ্জ লস (Hinge Loss):** লজিস্টিক রিগ্রেশনের ক্রমাগত পরিবর্তনশীল 'লগ-লস'-এর বিপরীতে SVM ব্যবহার করে হিঞ্জ লস:

$$\text{Hinge Loss} = \max(0, 1 - y_i(w^T x + b))$$

- পেনাল্টি মেকানিজম:** যদি কোনো ডেটা পয়েন্ট সঠিকভাবে ক্লাসিফাইড এবং মার্জিনের বাইরে থাকে ($y_i(w^T x + b) \geq 1$), তবে তার লস সরাসরি **০ (Zero)** হয়।

- **Robust to Outliers:** সিদ্ধান্ত সীমানাটি শুধুমাত্র অল্প কিছু সাপোর্ট ভেক্টরের ওপর নির্ভর করে। মার্জিনের বাইরে অবস্থিত অন্য হাজারটি বিন্দু পরিবর্তন বা অপসারণ করলেও হাইপারপ্লেন স্থানচ্যুত হয় না, যা একে আউটলায়ারের বিরুদ্ধে অত্যন্ত শক্তিশালী (Robust) করে তোলে।

৪. কার্নেল ট্রিক ও ডেটাসেটের প্রকারভেদ (Kernel Trick & Dataset Types)

- **লিনিয়ারলি সেপারেবল ডেটা:** ডেটা সোজা লাইনে ভাগ করা সম্ভব হলে কোনো ভুল বা ওভারল্যাপিং ছাড়া "Hard Margin" ব্যবহার করে নিখুঁত শ্রেণীবিভাগ করা যায়।
- **অরৈখিক উপাত্ত বিন্যাস (Non-Linear Data):** বাস্তব জীবনের জটিল ডেটা সোজা লাইনে আলাদা করা না গেলে SVM তার বিখ্যাত **Kernel Trick** (যেমন: Polynomial, RBF কার্নেল) ব্যবহার করে। এটি মূল ইনপুট স্পেসের ডেটাকে কোনো বাস্তব রূপান্তর ছাড়াই উচ্চ মাত্রার ফিচার স্পেসে (Higher Dimensional Space) গাণিতিক শর্টকাটের মাধ্যমে নিখুঁতভাবে পৃথক করে।

৫. অন্যান্য মডেলের তুলনায় SVM-এর অপরিহার্যতা (Why We Need SVM)

- **লজিস্টিক রিগ্রেশন বনাম SVM:** লজিস্টিক রিগ্রেশনে মার্জিনের ওপর কোনো সুনির্দিষ্ট জ্যামিতিক নিয়ন্ত্রণ থাকে না এবং এটি আউটলায়ার দ্বারা প্রভাবিত হয়। SVM মার্জিন সর্বোচ্চ করে এবং হিঞ্জ লসের মাধ্যমে অপ্রয়োজনীয় বিন্দুর প্রভাব মুক্ত রেখে লজিস্টিক রিগ্রেশনের চেয়ে অনেক বেশি স্থিতিশীল (Robust) সিদ্ধান্ত বাউন্ডারি দেয়।
- **KNN বনাম SVM:** KNN একটি অলস শিক্ষণ পদ্ধতি (Lazy Learner), যা টেস্ট টাইমে প্রতিটি ডেটার জন্য পুরো ডেটাসেটের দূরত্ব মাপে (ধীরগতির ও মেমোরি সাপেক্ষ)। পক্ষান্তরে, SVM ট্রেনিং শেষে শুধু অল্প কিছু সাপোর্ট ভেক্টর মেমোরিতে রাখে, ফলে এর প্রেডিকশন স্পিড অত্যন্ত ফাস্ট।
- **Decision Tree / Random Forest বনাম SVM:** বৃক্ষ-ভিত্তিক মডেলগুলো অক্ষ-বরাবর লম্বালম্বিভাবে উপাত্ত খণ্ডিত করে বৃত্তাকার বা জটিল প্যাটার্নে ওভারফিটিং তৈরি করে। কিন্তু SVM তার আরবিএফ (RBF) কার্নেল দিয়ে অতি সহজেই আঁকাবাঁকা বৃত্তাকার সিদ্ধান্ত সীমানা চিহ্নিত করতে পারে।

আনসুপারভাইজড লার্নিং

মেশিন লার্নিং (Machine Learning) বিজ্ঞানের যে শাখায় কোনো পূর্বনির্ধারিত লক্ষ্য-ভেরিয়েবল বা উত্তরসূচক লেবেল (Target Labels) ব্যতিরেকেই উপাত্তের ভেতরের অদৃশ্য বিন্যাস, গাণিতিক সম্পর্ক এবং অন্তর্নিহিত কাঠামো উন্মোচন করা হয়, তাকে আনসুপারভাইজড লার্নিং (Unsupervised Learning) বলা হয়।

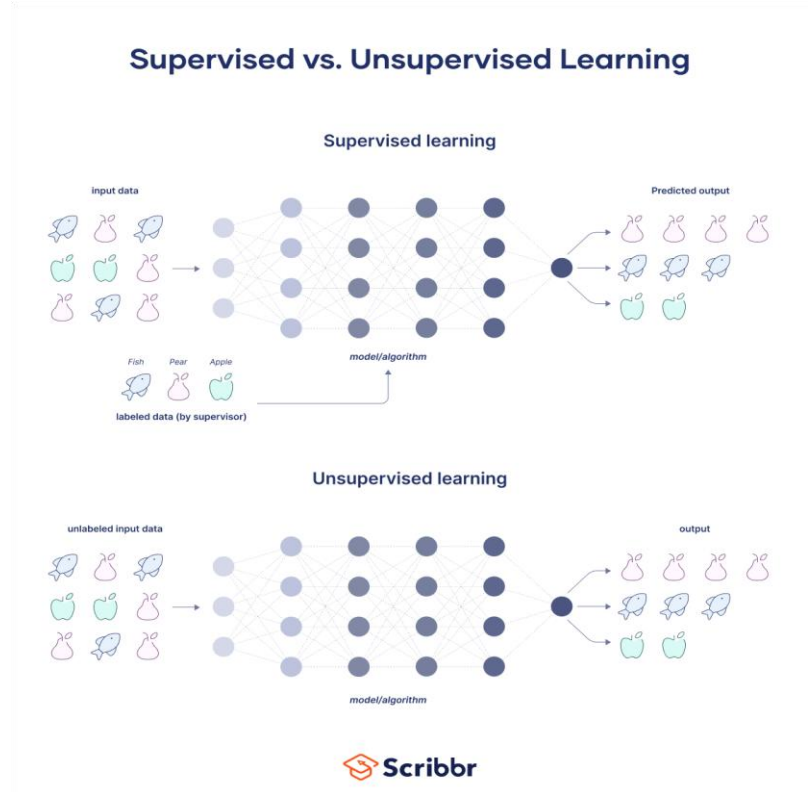
সুপারভাইজড লার্নিংয়ের মতো এখানে কোনো কৃত্রিম নির্দেশিকা বা "শিক্ষক" থাকে না। এই পদ্ধতিতে অ্যালগরিদমকে শুধুমাত্র ইনপুট ফিচার ভেক্টর সম্বলিত একটি লেবেলহীন ডেটাসেট প্রদান করা হয়।

$$\{x_i\}_{i=1}^N$$

গাণিতিক ও পরিসংখ্যানগত দৃষ্টিকোণ থেকে, আনসুপারভাইজড লার্নিংয়ের মূল উদ্দেশ্য হলো ইনপুট উপাত্তের সম্ভাব্যতা ঘনত্ব ফাংশন (Probability Density Function - PDF) অথবা উপাত্তের জ্যামিতিক বিস্তৃতি (Geometric Structure) বিশ্লেষণ করা।

বাস্তব উদাহরণ

- গ্রাহকদের কেনাকাটার ইতিহাস বিশ্লেষণ করে সমজাতীয় বৈশিষ্ট্যের ভিত্তিতে বিভিন্ন গ্রুপে বিভক্ত করা (Clustering)।
- নেটওয়ার্ক ট্রাফিকের স্বাভাবিক আচরণের বিন্যাস শিখে আকস্মিক অসঙ্গতি বা সাইবার আক্রমণ শনাক্ত করা (Anomaly Detection)।



সুপারভাইজড বনাম আনসুপারভাইজড লার্নিং (Supervised vs Unsupervised Learning)

মেশিন লার্নিংয়ে উপাত্তের গঠন, গাণিতিক অপটিমাইজেশন এবং ব্যবহারিক প্রয়োগের ভিত্তিতে এই দুই ধরনের লার্নিংয়ের মধ্যে গুরুত্বপূর্ণ পার্থক্য রয়েছে।

মূল্যায়নের ক্ষেত্র	সুপারভাইজড লার্নিং (Supervised Learning)	আনসুপারভাইজড লার্নিং (Unsupervised Learning)
উপাত্তের গাণিতিক রূপ (Data Structure)	লেবেলযুক্ত ডেটাসেট: $\{(x_i, y_i)\}_{i=1}^N$	লেবেলহীন ডেটাসেট: $\{x_i\}_{i=1}^N$
অ্যালগরিদমিক লক্ষ্য (Objective)	ইনপুট x থেকে আউটপুট y পূর্বাভাস দেওয়ার জন্য একটি গাণিতিক ফাংশন শেখা।	ডেটার ভেতরের অজানা বিন্যাস, ঘনত্ব এবং ফিচারগুলোর গোপন সম্পর্ক খুঁজে বের করা।
মডেলের নির্ভুলতা মূল্যায়ন (Evaluation)	MSE, Cross-Entropy ইত্যাদি Loss Function ব্যবহার করে সহজেই মূল্যায়ন করা যায়।	পূর্বনির্ধারিত সঠিক উত্তর না থাকায় নির্ভুলতা সরাসরি পরিমাপ করা কঠিন।
প্রধান ব্যবহার (Core Applications)	Regression এবং Classification।	Clustering, Dimensionality Reduction এবং Anomaly Detection।
কম্পিউটেশনাল জটিলতা (Computational Complexity)	Training তুলনামূলকভাবে ব্যয়বহুল হলেও Prediction দ্রুত হয়।	Distance ও Density বারবার গণনা করতে হওয়ায় Training ও Generalization উভয় ক্ষেত্রেই জটিলতা ও Memory Cost তুলনামূলকভাবে বেশি।

আনসুপারভাইজড লার্নিংয়ের প্রধান গাণিতিক রূপসমূহ (Core Problem Types)

বাস্তবমুখী গাণিতিক অপটিমাইজেশনের ওপর ভিত্তি করে আনসুপারভাইজড অ্যালগরিদম গুলোকে মূলত তিনটি প্রধান ভাগে বিভক্ত করা যায়।

ক্লাস্টারিং (Clustering)

একটি নির্দিষ্ট দূরত্ব পরিমাপক ম্যাট্রিক্সের (যেমন: Euclidean Distance বা Cosine Similarity) ওপর ভিত্তি করে সমধর্মী উপাত্ত বিন্দুগুলোকে স্বয়ংক্রিয়ভাবে এক একটি ক্লাস্টার বা গ্রুপে বিন্যস্ত করার পদ্ধতি।

ডাইমেনশনালিটি রিডাকশন (Dimensionality Reduction)

উপাত্ত ক্ষেত্রে বৈশিষ্ট্যের সংখ্যা বা ডাইমেনশন (D) যদি অত্যধিক বেশি হয়, তবে উপাত্তের ভেতরের মূল তথ্যগত বৈচিত্র্য (Variance) অক্ষুণ্ণ রেখে বৈশিষ্ট্যের সংখ্যা একটি সুনির্দিষ্ট নিম্ন মাত্রায় নামিয়ে আনার গাণিতিক কৌশল।

$$D_{\text{new}} < D$$

অ্যানোমালি ডিটেকশন (Anomaly Detection)

সমগ্র উপাত্ত ক্ষেত্রের স্বাভাবিক ঘনত্বের বিন্যাস থেকে বিচ্যুত বা বাউন্ডারির বাইরে অবস্থিত চরম Outlier বা ত্রুটিপূর্ণ নমুনাগুলোকে জ্যামিতিক দূরত্বের ভিত্তিতে পৃথক করার প্রক্রিয়া।

বাস্তব জীবনের প্রয়োগ ও সমস্যার ধরন

আনসুপারভাইজড লার্নিং অ্যালগরিদমগুলো কোনো পূর্বনির্ধারিত লেবেল ছাড়াই উপাত্তের অন্তর্নিহিত গঠন উন্মোচন করতে পারে। তাই বাস্তব জীবনের জটিল ও বহুমুখী সমস্যা সমাধানে এটি অত্যন্ত কার্যকর ভূমিকা পালন করে।

তাত্ত্বিক বিশ্লেষণ ও বাস্তব প্রয়োগের ভিত্তিতে আনসুপারভাইজড লার্নিংয়ের প্রধান সমস্যা ও প্রয়োগক্ষেত্রগুলোকে তিনটি ভাগে বিভক্ত করা যায়।

ক্লাস্টারিং (Clustering)

সমস্যার প্রকৃতি

এটি একটি নন-প্যারামেট্রিক (Non-parametric) পদ্ধতি, যার লক্ষ্য হলো কোনো লেবেলহীন ডেটাসেটের মধ্যে থাকা সদৃশ বৈশিষ্ট্যসম্পন্ন ডেটা পয়েন্টগুলোকে স্বয়ংক্রিয়ভাবে বিভিন্ন ক্লাস্টার (Cluster) বা গ্রুপে ভাগ করা।

সাধারণত **Euclidean Distance** অথবা **Cosine Similarity**-এর মতো Distance Metric ব্যবহার করে ডেটা পয়েন্টগুলোর মধ্যে সাদৃশ্য নির্ণয় করা হয়।

বাস্তব জীবনের প্রয়োগ

- গ্রাহক বিভাজন (Customer Segmentation): ই-কমার্স ও রিটেইল ব্যবসায় গ্রাহকদের কেনাকাটার ইতিহাস, পছন্দ এবং ব্যয়ের ধরন বিশ্লেষণ করে বিভিন্ন গ্রুপে ভাগ করা হয়, যেমন উচ্চ ব্যয়কারী, সাশ্রয়ী কিংবা নিষ্ক্রিয় গ্রাহক। এটি লক্ষ্যভিত্তিক মার্কেটিংয়ে সহায়তা করে।

- চিত্র বিভাজন (Image Segmentation): চিকিৎসাবিজ্ঞানে MRI অথবা X-ray ইমেজ বিশ্লেষণ করে সুস্থ টিস্যু এবং টিউমার বা ক্ষতিগ্রস্ত অংশকে স্বয়ংক্রিয়ভাবে আলাদা করতে ক্লাস্টারিং ব্যবহৃত হয়।
- সামাজিক নেটওয়ার্ক বিশ্লেষণ (Social Network Analysis): সামাজিক যোগাযোগমাধ্যমে ব্যবহারকারীদের পারস্পরিক যোগাযোগ, অভিন্ন আগ্রহ এবং সম্পর্কের ঘনত্ব বিশ্লেষণ করে স্বয়ংক্রিয়ভাবে বিভিন্ন কমিউনিটি বা গ্রুপ শনাক্ত করা হয়।

মাত্রিকতা হ্রাস (Dimensionality Reduction)

সমস্যার প্রকৃতি

বাস্তব জীবনের অনেক ডেটাসেটে বৈশিষ্ট্যের সংখ্যা (Features) অত্যন্ত বেশি থাকে। ফলে কম্পিউটেশনাল জটিলতা বৃদ্ধি পায় এবং নয়েজের পরিমাণও বেড়ে যায়। এই সমস্যাটিকে Curse of Dimensionality বলা হয়।

Dimensionality Reduction-এর মূল উদ্দেশ্য হলো উপাত্তের গুরুত্বপূর্ণ তথ্য (Information) ও বৈচিত্র্য (Variance) যতটা সম্ভব অক্ষুণ্ণ রেখে উচ্চমাত্রিক (High-dimensional) ডেটাকে নিম্নমাত্রিক (Low-dimensional) ডেটায় রূপান্তর করা।

বাস্তব জীবনের প্রয়োগ

- উপাত্ত দৃশ্যমানকরণ (Data Visualization): PCA বা UMAP-এর মতো অ্যালগরিদম ব্যবহার করে হাজার মাত্রার ডেটাকে 2D বা 3D-তে রূপান্তর করা হয়, যাতে মানুষ সহজে ডেটার গঠন পর্যবেক্ষণ করতে পারে।
- উপাত্ত সংকোচন ও মেমোরি সাশ্রয় (Data Compression): ইমেজ, অডিও বা টেক্সটের গুরুত্বপূর্ণ বৈশিষ্ট্য বজায় রেখে ফাইলের আকার কমাতে এটি ব্যবহৃত হয়। ফলে স্টোরেজের প্রয়োজন কমে এবং প্রসেসিং দ্রুত হয়।
- মডেলের কার্যদক্ষতা বৃদ্ধি (Feature Extraction): সুপারভাইজড লার্নিংয়ের আগে অপ্রয়োজনীয় ফিচার বাদ দিয়ে প্রয়োজনীয় বৈশিষ্ট্য নির্বাচন করা হয়, ফলে ট্রেনিং দ্রুত হয় এবং Overfitting-এর সম্ভাবনা কমে।

অ্যানোমালি ডিটেকশন (Anomaly Detection)

সমস্যার প্রকৃতি

এই ধরনের সমস্যায় ডেটাসেটের অধিকাংশ নমুনাই স্বাভাবিক (Normal), কিন্তু অস্বাভাবিক (Abnormal) বা ত্রুটিপূর্ণ নমুনা খুবই কম অথবা একেবারেই অনুপস্থিত থাকে।

অ্যালগরিদম প্রথমে স্বাভাবিক ডেটার ঘনত্ব (Density) ও গঠন (Structure) শিখে নেয়। এরপর নতুন কোনো ডেটা যদি সেই স্বাভাবিক প্যাটার্ন থেকে উল্লেখযোগ্যভাবে বিচ্যুত হয়, তাহলে সেটিকে **Anomaly** বা **Outlier** হিসেবে শনাক্ত করা হয়।

বাস্তব জীবনের প্রয়োগ

- ক্রেডিট কার্ড জালিয়াতি শনাক্তকরণ (Fraud Detection): গ্রাহকের স্বাভাবিক লেনদেনের ধরণ থেকে ভিন্ন কোনো সন্দেহজনক লেনদেন স্বয়ংক্রিয়ভাবে শনাক্ত করা হয়।
- শিল্প উৎপাদনে ত্রুটি নির্ণয় (Manufacturing Defect Detection): সেন্সর ডেটা বা পণ্যের ছবি বিশ্লেষণ করে ত্রুটিযুক্ত পণ্যকে স্বয়ংক্রিয়ভাবে আলাদা করা হয়।
- সাইবার নিরাপত্তা ও অনুপ্রবেশ শনাক্তকরণ (Intrusion Detection): নেটওয়ার্ক ট্রাফিকের স্বাভাবিক আচরণ বিশ্লেষণ করে হ্যাকিংয়ের চেষ্টা, ম্যালওয়্যার আক্রমণ অথবা অস্বাভাবিক নেটওয়ার্ক কার্যকলাপ শনাক্ত করা হয়।

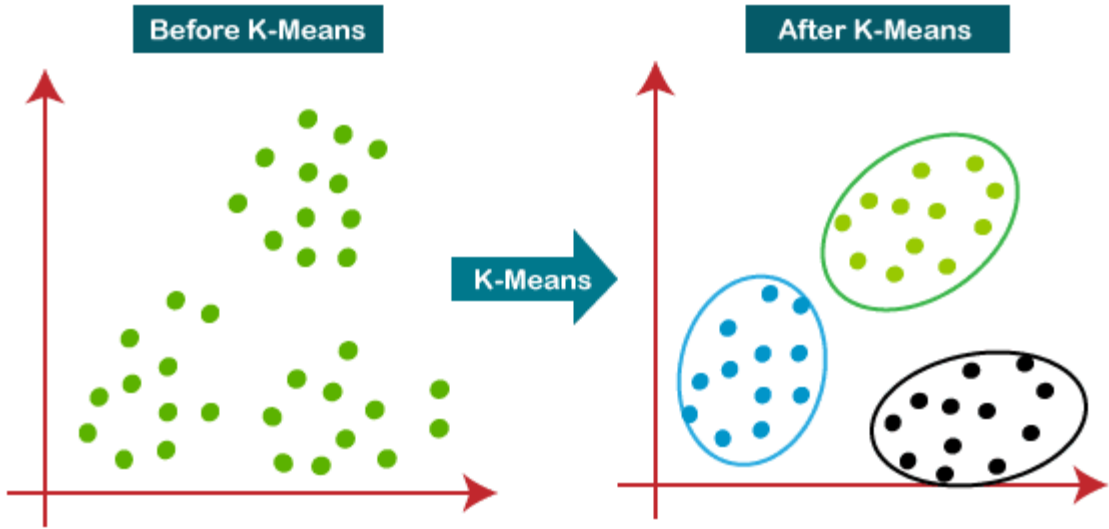
K-Means Clustering

K-Means হলো একটি অত্যন্ত জনপ্রিয়, নন-প্যারামেট্রিক (Non-parametric) এবং সেন্ট্রয়েড-ভিত্তিক (Centroid-based) হার্ড ক্লাস্টারিং (Hard Clustering) অ্যালগরিদম।

সহজ ভাষায়, এটি এমন একটি গাণিতিক পদ্ধতি যা কোনো পূর্বনির্ধারিত লেবেলবিহীন (Unlabeled) ডেটাসেটের ডেটা পয়েন্টগুলোর পারস্পরিক জ্যামিতিক দূরত্ব পরিমাপ করে তাদের স্বয়ংক্রিয়ভাবে K সংখ্যক পৃথক ক্লাস্টার বা গ্রুপে বিভক্ত করে।

এই অ্যালগরিদমে দুটি গুরুত্বপূর্ণ উপাদান রয়েছে।

- **K (Number of Clusters):** এটি একটি User-defined Hyperparameter, যা নির্ধারণ করে ডেটাসেটকে মোট কতটি ক্লাস্টারে ভাগ করা হবে।
- **Mean (Centroid):** প্রতিটি ক্লাস্টারের অন্তর্ভুক্ত সকল ডেটা পয়েন্টের গাণিতিক গড় (Arithmetic Mean), যাকে Centroid বলা হয়। এই Centroid-ই ক্লাস্টারের কেন্দ্রবিন্দু হিসেবে কাজ করে।



Classification of K-Means

মেশিন লার্নিংয়ের গাঠনিক বিন্যাস অনুযায়ী K-Means-কে নিম্নোক্ত শ্রেণিতে বিভক্ত করা যায়।

- এটি একটি **Unsupervised Learning Model**, কারণ এটি কোনো Target Label বা শিক্ষক ছাড়াই ডেটার অন্তর্নিহিত গঠন (Hidden Structure) আবিষ্কার করে।
- এটি একটি **Instance-based Model**, কারণ এটি কোনো গ্লোবাল গাণিতিক সমীকরণ শেখে না; বরং ডেটা পয়েন্টগুলোর অবস্থান ও দূরত্বের ভিত্তিতে ক্লাস্টার তৈরি করে।
- এটি একটি **Hard Clustering Model**, কারণ প্রতিটি ডেটা পয়েন্ট শুধুমাত্র একটি ক্লাস্টারের সদস্য হতে পারে। একই ডেটা পয়েন্ট একাধিক ক্লাস্টারের অন্তর্ভুক্ত হতে পারে না। এই বৈশিষ্ট্য K-Means-কে Soft Clustering অ্যালগরিদম (যেমন Gaussian Mixture Model) থেকে আলাদা করে।

Problem Solved by K-Means

K-Means মূলত ক্লাস্টারিং (Clustering) সমস্যার সমাধান করে।

জ্যামিতিক দৃষ্টিকোণ থেকে, এটি Feature Space-কে একাধিক অঞ্চলে (Regions) ভাগ করে, যাতে একই ক্লাস্টারের ডেটা পয়েন্টগুলো একে অপরের যতটা সম্ভব কাছাকাছি থাকে এবং ভিন্ন ক্লাস্টারের ডেটা পয়েন্টগুলোর মধ্যকার দূরত্ব যতটা সম্ভব বেশি হয়।

বাস্তব জীবনে K-Means নিম্নলিখিত সমস্যাগুলোর সমাধান করতে পারে।

- গ্রাহকের কেনাকাটার ধরণ, ব্যয়ের পরিমাণ এবং আচরণের ভিত্তিতে স্বয়ংক্রিয়ভাবে বিভিন্ন Customer Segment তৈরি করা।
- ডিজিটাল ছবির সমজাতীয় Pixel বা Color Value-গুলোকে একত্রিত করে Image Segmentation সম্পন্ন করা।

Importance of K-Means Clustering

বর্তমানে Linear Regression, Logistic Regression কিংবা Decision Tree-এর মতো শক্তিশালী Supervised Learning Model থাকা সত্ত্বেও K-Means-এর গুরুত্ব অত্যন্ত বেশি।

এর প্রধান কারণগুলো হলো:

- **লেবেলবিহীন ডেটা বিশ্লেষণ (Handling Unlabeled Data):** বাস্তব বিশ্বের অধিকাংশ ডেটার কোনো Label থাকে না। প্রতিটি ডেটাকে ম্যানুয়ালি Label করা সময়সাপেক্ষ এবং ব্যয়বহুল। K-Means কোনো Label ছাড়াই অর্থপূর্ণ গ্রুপ তৈরি করতে পারে।

- **উচ্চ কম্পিউটেশনাল দক্ষতা (High Computational Efficiency):** Hierarchical Clustering বা HDBSCAN-এর মতো অনেক ক্লাস্টারিং অ্যালগরিদমের তুলনায় K-Means অনেক দ্রুত কাজ করে এবং বৃহৎ ডেটাসেটও অল্প সময়ে প্রক্রিয়াকরণ করতে পারে।
- **সহজ ব্যাখ্যাযোগ্যতা (Interpretability):** প্রতিটি ক্লাস্টারের একটি নির্দিষ্ট Centroid থাকায় সহজেই বোঝানো যায় কেন নির্দিষ্ট ডেটা পয়েন্টগুলো একই ক্লাস্টারে রয়েছে। ফলে Business Analysis এবং Decision Making সহজ হয়।
- **Feature Engineering ও Data Preprocessing:** অনেক ক্ষেত্রে Supervised Learning Model ট্রেনিংয়ের আগে K-Means ব্যবহার করে Cluster Distance-এর মতো নতুন Feature তৈরি করা হয়, যা পরবর্তীতে মডেলের কার্যক্ষমতা এবং Accuracy বৃদ্ধি করতে সাহায্য করে।

Mathematical Foundation of K-Means Clustering

Feature Representation

K-Means অ্যালগরিদমে প্রতিটি ডেটা একটি Feature Vector হিসেবে প্রকাশ করা হয়। যদি কোনো ডেটাসেটে D টি Feature থাকে, তাহলে একটি ডেটা পয়েন্টকে লেখা যায়,

$$x_i = [x_{i1}, x_{i2}, x_{i3}, \dots, x_{iD}]$$

যেখানে,

- x_i = i -তম Data Point
- D = মোট Feature বা Dimension
- N = মোট Data Point-এর সংখ্যা

অর্থাৎ সম্পূর্ণ Dataset-কে প্রকাশ করা যায়,

$$X = \{x_1, x_2, \dots, x_N\}$$

Number of Clusters

K-Means-এ শুরুতেই একটি Hyperparameter নির্বাচন করতে হয়, যাকে K বলা হয়।

$$K = \text{Number of Clusters}$$

এটি নির্ধারণ করে Dataset-কে মোট কতটি Cluster-এ ভাগ করা হবে।

Centroid of a Cluster

প্রতিটি Cluster-এর একটি কেন্দ্রবিন্দু থাকে, যাকে **Centroid** বলা হয়।

Centroid হলো সেই Cluster-এর অন্তর্ভুক্ত সব Data Point-এর গাণিতিক গড় (Arithmetic Mean)।

$$c_j = \frac{1}{|A_j|} \sum_{x_i \in A_j} x_i$$

যেখানে,

- c_j = j -তম Cluster-এর Centroid
- A_j = j -তম Cluster
- $|A_j|$ = ঐ Cluster-এর মোট Data Point সংখ্যা

সহজভাবে,

$$\text{Centroid} = \frac{\text{Cluster-এর সব Data Point-এর যোগফল}}{\text{Cluster-এর মোট Data Point সংখ্যা}}$$

এই সমীকরণ প্রতিটি Iteration শেষে নতুন Centroid নির্ণয়ের জন্য ব্যবহৃত হয়।

Euclidean Distance

K-Means সাধারণত দুটি Data Point-এর মধ্যকার দূরত্ব নির্ণয়ের জন্য **Euclidean Distance** ব্যবহার করে।

$$d(x_i, c_j) = \sqrt{\sum_{k=1}^D (x_{ik} - c_{jk})^2}$$

যেখানে,

- x_i = একটি Data Point
- c_j = একটি Centroid
- D = মোট Feature

দ্বিমাত্রিক (2D) ক্ষেত্রে সমীকরণটি হয়,

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

এই Distance ব্যবহার করে প্রতিটি Data Point-কে সবচেয়ে নিকটবর্তী Centroid-এর Cluster-এ Assign করা হয়।

Objective Function (Within-Cluster Sum of Squares)

K-Means-এর মূল লক্ষ্য হলো প্রতিটি Cluster-এর ভেতরে থাকা Data Point-গুলোর Centroid থেকে মোট Squared Distance যতটা সম্ভব কমানো।

এই Objective Function-কে Within-Cluster Sum of Squares (WCSS) অথবা Sum of Squared Errors (SSE) বলা হয়।

$$J = \sum_{j=1}^K \sum_{x_i \in A_j} \|x_i - c_j\|^2$$

এখানে,

- J = Total Cost Function
- K = মোট Cluster সংখ্যা
- A_j = j -তম Cluster
- c_j = ঐ Cluster-এর Centroid

সহজভাবে,

$$\text{Total Cost} = \sum (\text{Point থেকে নিজের Centroid পর্যন্ত দূরত্ব})^2$$

K-Means Algorithm-এর প্রধান উদ্দেশ্য হলো,

$$\min J$$

অর্থাৎ Total Cost যতটা সম্ভব কমিয়ে আনা।

Why Does the Cost Always Decrease?

K-Means অ্যালগরিদমের Cost Function প্রতিটি Iteration-এ কখনো বৃদ্ধি পায় না; বরং এটি হয় কমে অথবা অপরিবর্তিত থাকে। এর কারণ হলো অ্যালগরিদমটি দুটি ধারাবাহিক অপ্টিমাইজেশন ধাপ অনুসরণ করে।

প্রথম ধাপে (Assignment Step), প্রতিটি Data Point-এর সঙ্গে সবগুলো Centroid-এর দূরত্ব গণনা করা হয় এবং পয়েন্টটিকে সবচেয়ে নিকটবর্তী Centroid-এর Cluster-এ স্থানান্তর করা হয়। যেহেতু প্রতিটি Data Point সর্বদা সর্বনিম্ন দূরত্বের Cluster-এ Assign করা হয়, তাই এই ধাপে মোট Squared Distance বা Cost কখনো বৃদ্ধি পায় না।

দ্বিতীয় ধাপে (Update Step), প্রতিটি Cluster-এর অন্তর্ভুক্ত সকল Data Point-এর গাণিতিক গড় (Arithmetic Mean) নির্ণয় করে নতুন Centroid গণনা করা হয়। গাণিতিকভাবে প্রমাণিত যে কোনো Cluster-এর Mean সেই অবস্থান, যেখানে ঐ Cluster-এর সকল Data Point থেকে Squared Distance-এর যোগফল সর্বনিম্ন হয়। ফলে নতুন Centroid নির্ধারণের পর মোট Cost আরও কমে যায় অথবা পূর্বের মতোই থাকে।

এই দুটি ধাপ পর্যায়ক্রমে পুনরাবৃত্তি হতে থাকে। প্রতিটি Iteration-এ Cost Function হয় কমে, নতুবা অপরিবর্তিত থাকে। যেহেতু Cost Function-এর একটি নিম্নসীমা (Lower Bound) রয়েছে এবং এটি কখনো ঋণাত্মক হতে পারে না, তাই একসময় অ্যালগরিদমটি এমন একটি অবস্থায় পৌঁছে যায় যেখানে আর কোনো পরিবর্তন ঘটে না। এই অবস্থাকেই **Convergence** বলা হয়।

Update Step

প্রতিটি Cluster-এর নতুন Mean নির্ণয় করে নতুন Centroid গণনা করা হয়।

এর ফলে নতুন Centroid Cluster-এর প্রকৃত কেন্দ্রের আরও কাছাকাছি চলে আসে এবং মোট Squared Distance আরও কমে যায়।

এই দুটি ধাপ পুনরাবৃত্তি হতে থাকে যতক্ষণ না আর কোনো পরিবর্তন ঘটে।

ফলে প্রতিটি Iteration-এ,

$$J_{\text{new}} \leq J_{\text{old}}$$

অর্থাৎ Cost Function হয় কমে, নতুবা অপরিবর্তিত থাকে। একসময় Algorithm একটি স্থিতিশীল অবস্থায় (Convergence) পৌঁছে যায়।

Determining the Optimal Number of Clusters

K-Means Algorithm চালানোর আগে উপযুক্ত K নির্বাচন করা অত্যন্ত গুরুত্বপূর্ণ।

বাস্তবে Optimal K নির্ধারণের জন্য বিভিন্ন পদ্ধতি ব্যবহৃত হয়, যেমন,

- Elbow Method
- Silhouette Score
- Gap Statistic
- Prediction Strength

এর মধ্যে **Elbow Method** এবং **Silhouette Score** সবচেয়ে বেশি ব্যবহৃত ও পাঠ্যবইয়ে আলোচিত পদ্ধতি।

K-Means অ্যালগরিদমের মূল উদ্দেশ্য হলো প্রতিটি ডেটা পয়েন্টকে তার নিকটবর্তী সেন্ট্রয়েডের (Centroid/Cluster Mean) সাথে যুক্ত করে ক্লাস্টারের ভেতরের বিশৃঙ্খলা বা ভুলের পরিমাণ সর্বনিম্ন করা। গাণিতিক ভাষায় একে Within-Cluster Sum of Squares (WCSS) বা Inertia বলা হয়।

ধরা যাক, আমাদের কাছে n সংখ্যক স্যাম্পল সংবলিত একটি ডেটাসেট $D = \{x_1, x_2, \dots, x_n\}$ আছে। আমরা এই ডেটাকে K সংখ্যক ক্লাস্টারে $S = \{S_1, S_2, \dots, S_K\}$ ভাগ করতে চাই। এর কস্ট ফাংশন (J) সমীকরণটি হলো:

$$J = \sum_{j=1}^K \sum_{x_i \in S_j} \|x_i - \mu_j\|^2$$

যেখানে:

- K : মোট ক্লাস্টারের সংখ্যা (User-defined Hyperparameter)।
- S_j : j -তম ক্লাস্টারের অন্তর্ভুক্ত ডেটা পয়েন্টগুলোর সেট।
- μ_j : j -তম ক্লাস্টারের সেন্ট্রয়েড বা গড় মান (Mean Vector)।
- $\|x_i - \mu_j\|^2$: ডেটা পয়েন্ট x_i এবং সেন্ট্রয়েড μ_j -এর মধ্যকার ইউক্লিডিয়ান দূরত্বের বর্গ (Squared Euclidean Distance)।

ব্যাকএন্ড ওয়ার্কফ্লো এবং অ্যালগরিদম

K-Means অ্যালগরিদমটি মূলত একটি ইটারেটিভ (Iterative) পদ্ধতি অনুসরণ করে গ্লোবাল মিনিমাম বা সর্বনিম্ন কস্টের দিকে এগিয়ে যায়। এর মূল ধাপগুলো নিচে দেওয়া হলো:

Input:

- Dataset X
- Number of Clusters K

Output:

- Final Clusters ($S_1, S_2, \dots, S_K$)
- Final Centroids ($\mu_1, \mu_2, \dots, \mu_K$)

Step 1: Centroid Initialization

ডেটাসেট থেকে র‍্যান্ডমভাবে (Randomly) K সংখ্যক ডেটা পয়েন্ট নির্বাচন করে সেগুলোকে প্রাথমিক সেন্ট্রয়েড হিসেবে নির্ধারণ করা হয়।

$$\mu_1, \mu_2, \dots, \mu_K$$

Step 2: Iterative Optimization

নিচের ধাপগুলো বারবার পুনরাবৃত্তি করা হয়, যতক্ষণ না অ্যালগরিদমটি স্থির (Converge) হয়।

A. Cluster Assignment (Expectation Step)

প্রতিটি ডেটা পয়েন্ট x_i -এর জন্য সকল Centroid-এর সাথে Euclidean Distance গণনা করা হয়।

এরপর x_i -কে যে Centroid-এর দূরত্ব সবচেয়ে কম, সেই Cluster-এ অন্তর্ভুক্ত করা হয়।

$$S_j = \{x_i: \|x_i - \mu_j\|^2 \leq \|x_i - \mu_l\|^2, \forall l\}$$

B. Centroid Update (Maximization Step)

প্রতিটি Cluster-এর অন্তর্ভুক্ত সকল ডেটা পয়েন্টের গাণিতিক গড় (Arithmetic Mean) নির্ণয় করে নতুন Centroid গণনা করা হয়।

$$\mu_j = \frac{1}{|S_j|} \sum_{x_i \in S_j} x_i$$

C. Convergence Check

নতুন Centroid এবং পূর্ববর্তী Centroid-এর মান তুলনা করা হয়।

যদি সকল Centroid-এর অবস্থান অপরিবর্তিত থাকে (অর্থাৎ নতুন ও পুরোনো Centroid একই হয়), তাহলে Iteration বন্ধ করা হয়। অন্যথায়, পুনরায় Cluster Assignment ধাপ থেকে একই প্রক্রিয়া শুরু হয়।

ধাপ ১: ডেটাসেট ও প্যারামিটার নির্ধারণ

ধরা যাক, আমাদের কাছে ২-ডাইমেনশনাল ($d = 2$) একটি ছোট ডেটাসেট আছে যেখানে $n = 4$ টি ডেটা পয়েন্ট রয়েছে:

- $A_1 = (1,1)$
- $A_2 = (2,1)$
- $A_3 = (4,3)$
- $A_4 = (5,4)$

আমরা চাই ডেটাগুলোকে $K = 2$ টি ক্লাস্টারে ভাগ করতে।

ধাপ ২: র্যান্ডম সেন্ট্রয়েড নির্বাচন (Initialization)

আমরা আন্দাজে ডেটাসেট থেকে প্রথম দুটি পয়েন্টকে প্রাথমিক সেন্ট্রয়েড হিসেবে ধরে নিলাম:

- সেন্ট্রয়েড ১ (μ_1) = (1,1)
- সেন্ট্রয়েড ২ (μ_2) = (2,1)

ইটারেশন ১ (Iteration 1)

অংশ A: দূরত্ব গণনা ও ক্লাস্টার অ্যাসাইনমেন্ট

আমরা প্রতিটি ডেটা পয়েন্ট $x_i = (x_{i1}, x_{i2})$ থেকে সেন্ট্রয়েড $\mu_j = (\mu_{j1}, \mu_{j2})$ এর ইউক্লিডিয়ান দূরত্বের বর্গ হিসাব করব। সূত্র: $\|x_i - \mu_j\|^2 = (x_{i1} - \mu_{j1})^2 + (x_{i2} - \mu_{j2})^2$

- পয়েন্ট $A_1(1,1)$ এর জন্য:
 - μ_1 থেকে দূরত্ব = $(1 - 1)^2 + (1 - 1)^2 = 0$
 - μ_2 থেকে দূরত্ব = $(1 - 2)^2 + (1 - 1)^2 = 1 + 0 = 1$
 - সিদ্ধান্ত: যেহেতু $0 < 1$, তাই $A_1 \rightarrow \text{Cluster 1}$
- পয়েন্ট $A_2(2,1)$ এর জন্য:
 - μ_1 থেকে দূরত্ব = $(2 - 1)^2 + (1 - 1)^2 = 1 + 0 = 1$
 - μ_2 থেকে দূরত্ব = $(2 - 2)^2 + (1 - 1)^2 = 0$
 - সিদ্ধান্ত: যেহেতু $0 < 1$, তাই $A_2 \rightarrow \text{Cluster 2}$
- পয়েন্ট $A_3(4,3)$ এর জন্য:

- μ_1 থেকে দূরত্ব $= (4 - 1)^2 + (3 - 1)^2 = 9 + 4 = 13$

- μ_2 থেকে দূরত্ব $= (4 - 2)^2 + (3 - 1)^2 = 4 + 4 = 8$

- সিদ্ধান্ত: যেহেতু $8 < 13$, তাই $A_3 \rightarrow \text{Cluster 2}$

• পয়েন্ট $A_4(5,4)$ এর জন্য:

- μ_1 থেকে দূরত্ব $= (5 - 1)^2 + (4 - 1)^2 = 16 + 9 = 25$

- μ_2 থেকে দূরত্ব $= (5 - 2)^2 + (4 - 1)^2 = 9 + 9 = 18$

- সিদ্ধান্ত: যেহেতু $18 < 25$, তাই $A_4 \rightarrow \text{Cluster 2}$

ইটারেশন ১ শেষে নতুন ক্লাস্টার গ্রুপ:

- $S_1 = \{A_1\}$

- $S_2 = \{A_2, A_3, A_4\}$

অংশ B: বর্তমান কস্ট বা WCSS হিসাব

$$J = \sum \|x_i - \mu_1\|^2 + \sum \|x_i - \mu_2\|^2$$

$$J = [0] + [0 + 8 + 18] = 26$$

অংশ C: নতুন সেন্ট্রয়েড গণনা (Centroid Update)

- নতুন সেন্ট্রয়েড μ_1 :

$$\mu_1 = \left(\frac{1}{1}, \frac{1}{1}\right) = (1,1)$$

- নতুন সেন্ট্রয়েড μ_2 :

$$\mu_2 = \left(\frac{2 + 4 + 5}{3}, \frac{1 + 3 + 4}{3}\right) = \left(\frac{11}{3}, \frac{8}{3}\right) \approx (3.67, 2.67)$$

ইটারেশন ২ (Iteration 2)

যেহেতু সেন্ট্রয়েডের মান পরিবর্তিত হয়েছে, তাই আমরা নতুন সেন্ট্রয়েড $\mu_1 = (1,1)$ এবং $\mu_2 = (3.67, 2.67)$ ব্যবহার করে আবার দূরত্ব হিসাব করব।

অংশ A: দূরত্ব গণনা ও ক্লাস্টার অ্যাসাইনমেন্ট

- পয়েন্ট $A_1(1,1)$ এর জন্য:

- μ_1 থেকে দূরত্ব $= 0$

- μ_2 থেকে দূরত্ব $= (1 - 3.67)^2 + (1 - 2.67)^2 = (-2.67)^2 + (-1.67)^2 \approx 7.13 + 2.79 = 9.92$
- সিদ্ধান্ত: $A_1 \rightarrow \text{Cluster 1}$
- পয়েন্ট $A_2(2,1)$ এর জন্য:
 - μ_1 থেকে দূরত্ব $= (2 - 1)^2 + (1 - 1)^2 = 1$
 - μ_2 থেকে দূরত্ব $= (2 - 3.67)^2 + (1 - 2.67)^2 = (-1.67)^2 + (-1.67)^2 \approx 2.79 + 2.79 = 5.58$
 - সিদ্ধান্ত: যেহেতু $1 < 5.58$, তাই A_2 ক্লাস্টার পরিবর্তন করে ১ নম্বর ক্লাস্টারে চলে গেল। ($A_2 \rightarrow \text{Cluster 1}$)
- পয়েন্ট $A_3(4,3)$ এর জন্য:
 - μ_1 থেকে দূরত্ব $= (4 - 1)^2 + (3 - 1)^2 = 13$
 - μ_2 থেকে দূরত্ব $= (4 - 3.67)^2 + (3 - 2.67)^2 = (0.33)^2 + (0.33)^2 \approx 0.11 + 0.11 = 0.22$
 - সিদ্ধান্ত: $A_3 \rightarrow \text{Cluster 2}$
- পয়েন্ট $A_4(5,4)$ এর জন্য:
 - μ_1 থেকে দূরত্ব $= 25$
 - μ_2 থেকে দূরত্ব $= (5 - 3.67)^2 + (4 - 2.67)^2 = (1.33)^2 + (1.33)^2 \approx 1.77 + 1.77 = 3.54$
 - সিদ্ধান্ত: $A_4 \rightarrow \text{Cluster 2}$

ইটারেশন ২ শেষে নতুন ক্লাস্টার গ্রুপ:

- $S_1 = \{A_1, A_2\}$
- $S_2 = \{A_3, A_4\}$

অংশ B: বর্তমান কস্ট বা WCSS হিসাব

$$J = [0 + 1] + [0.22 + 3.54] = 4.76$$

(দেখা যাচ্ছে কস্ট ২৬ থেকে কমে ৪.৭৬ এ নেমে এসেছে, অর্থাৎ মডেলটি সঠিক দিকে যাচ্ছে)

অংশ C: নতুন সেন্ট্রয়েড গণনা (Centroid Update)

- নতুন সেন্ট্রয়েড μ_1 :

$$\mu_1 = \left(\frac{1+2}{2}, \frac{1+1}{2} \right) = (1.5, 1)$$

- নতুন সেন্ট্রয়েড μ_2 :

$$\mu_2 = \left(\frac{4+5}{2}, \frac{3+4}{2} \right) = (4.5, 3.5)$$

ইটারেশন ৩ (Iteration 3)

এখন নতুন আপডেটেড সেন্ট্রয়েড হলো $\mu_1 = (1.5, 1.0)$ এবং $\mu_2 = (4.5, 3.5)$ ।

অংশ A: দূরত্ব গণনা ও ক্লাস্টার অ্যাসাইনমেন্ট

- পয়েন্ট $A_1(1,1)$: μ_1 থেকে দূরত্ব = 0.25 | μ_2 থেকে দূরত্ব = 18.5 $\rightarrow A_1 \rightarrow S_1$
- পয়েন্ট $A_2(2,1)$: μ_1 থেকে দূরত্ব = 0.25 | μ_2 থেকে দূরত্ব = 12.5 $\rightarrow A_2 \rightarrow S_1$
- পয়েন্ট $A_3(4,3)$: μ_1 থেকে দূরত্ব = 10.25 | μ_2 থেকে দূরত্ব = 0.5 $\rightarrow A_3 \rightarrow S_2$
- পয়েন্ট $A_4(5,4)$: μ_1 থেকে দূরত্ব = 21.25 | μ_2 থেকে দূরত্ব = 0.5 $\rightarrow A_4 \rightarrow S_2$

ইটারেশন ৩ শেষে ক্লাস্টার গ্রুপ:

- $S_1 = \{A_1, A_2\}$
- $S_2 = \{A_3, A_4\}$

Convergence Reached (স্থিরতা অর্জন): খেয়াল করে দেখুন, ইটারেশন ৩-এর ক্লাস্টার অ্যাসাইনমেন্ট এবং ইটারেশন ২-এর ক্লাস্টার অ্যাসাইনমেন্ট হুবহু একই এসেছে। এর মানে হলো ডেটা পয়েন্টগুলো আর তাদের দল পরিবর্তন করছে না। সেন্ট্রয়েড পুনর্গণনা করলেও মান একই থাকবে। অতএব, মডেলটি গ্লোবাল মিনিমাম বা সর্বনিম্ন কস্ট পয়েন্ট অর্জন করেছে এবং ট্রেনিং এখানেই সমাপ্ত।

চূড়ান্ত ফলাফল টেবিল (Final Model State)

ক্লাস্টার (Cluster)	আইডি অন্তর্ভুক্ত পয়েন্টসমূহ	ডেটা সেন্ট্রয়েড (Centroid)	ক্লাস্টার কস্ট (WCSS)
Cluster 1	$A_1(1,1), A_2(2,1)$	$\mu_1 = (1.5, 1.0)$	$0.25 + 0.25 = 0.50$
Cluster 2	$A_3(4,3), A_4(5,4)$	$\mu_2 = (4.5, 3.5)$	$0.50 + 0.50 = 1.00$

সর্বনিম্ন চূড়ান্ত কস্ট (Minimum Cost) $J = 0.50 + 1.00 = 1.50$

Parameters:

১. মোস্ট ইম্পোর্টেন্ট প্যারামিটারসমূহ (The Heavy Hitters)

মডেলের ক্লাস্টারিং পারফরম্যান্স, গতি এবং সেন্ট্রয়েডের সঠিক অবস্থান নির্ধারণে এই প্যারামিটারগুলো সবচেয়ে গুরুত্বপূর্ণ ভূমিকা পালন করে।

n_clusters

- **Default:** 8
- **প্যারামিটারটি কী কাজ করে:** এটি মূলত ডেটাসেটকে মোট কতটি ক্লাস্টারে ভাগ করা হবে এবং কতটি সেন্ট্রয়েডের স্থানাঙ্ক (Coordinates) তৈরি করা হবে তা নির্ধারণ করে।
- **অন্যান্য অপশনসমূহ এবং কখন ব্যবহার করবেন:** এটি যেকোনো ধনাত্মক পূর্ণসংখ্যা (Integer) গ্রহণ করে। বাস্তব প্রজেক্টে ব্যবসার লক্ষ্য বা ডেটার ধরনের ওপর ভিত্তি করে এটি সেট করা হয়। যদি সঠিক ক্লাস্টার সংখ্যা জানা না থাকে, তবে **Elbow Method** বা **Silhouette Analysis**-এর মতো গাণিতিক পদ্ধতি ব্যবহার করে সবচেয়ে নিখুঁত K বা ক্লাস্টার সংখ্যা খুঁজে বের করে এখানে ইনপুট দিতে হয়।

init

- **Default:** 'k-means++'
- **প্যারামিটারটি কী কাজ করে:** অ্যালগরিদমটি শুরু করার সময় প্রাথমিক সেন্ট্রয়েডের অবস্থান কীভাবে নির্বাচন করা হবে, এটি তা নিয়ন্ত্রণ করে। ডিফল্ট 'k-means++' পদ্ধতিটি ডেটা পয়েন্টগুলোর সামগ্রিক জড়তার (Inertia) ওপর ভিত্তি করে একটি প্রায়োগিক সম্ভাব্যতা বন্টন (Empirical probability distribution) ব্যবহার করে সেন্ট্রয়েডের বীজ (Seeds) বপন করে। এটি সাধারণ পদ্ধতির চেয়ে অনেক দ্রুত মডেলকে কনভার্জ (স্থির)

করতে সাহায্য করে। Scikit-Learn-এর এই ইমপ্লিমেন্টেশনটি প্রতিটি ধাপে বেশ কয়েকটি ট্রায়াল চালিয়ে সেরা সেন্ট্রয়েডটি বেছে নেয় (Greedy K-Means++)।

- **অন্যান্য অপশনসমূহ এবং কখন ব্যবহার করবেন:**

- 'random': এটি ডেটাসেট থেকে সম্পূর্ণ র্যান্ডমলি (আন্দাজে) K সংখ্যক সারি বা পর্যবেক্ষণ বেছে নিয়ে প্রাথমিক সেন্ট্রয়েড নির্ধারণ করে। এটি মূলত শিক্ষানবিস বা তাত্ত্বিক পরীক্ষার ক্ষেত্রে ব্যবহার করা হয়, তবে এটি ব্যবহার করলে মডেলটি প্রায়শই লোকাল মিনিমামে (Local Minima) আটকে যায়।
- array-like (ম্যাট্রিক্স): যদি আপনার আগে থেকেই সেন্ট্রয়েডের অবস্থান জানা থাকে, তবে তার স্থানাঙ্ক সরাসরি ($n_clusters$, $n_features$) শেপের একটি অ্যারে আকারে পাস করতে পারেন।
- callable (ফাংশন): কাস্টম কোনো শুরুকরণ কৌশল (Custom Initialization Strategy) তৈরি করতে চাইলে এখানে একটি পাইথন ফাংশন পাস করা যায়।

n_init

- **Default:** 'auto'
- **প্যারামিটারটি কী কাজ করে:** K-Means অ্যালগরিদমটি ভিন্ন ভিন্ন প্রাথমিক সেন্ট্রয়েড বীজ নিয়ে মোট কতবার নতুন করে রান (Run) করবে, এটি তা নির্ধারণ করে। একাধিকবার রান করার পর, যে রানটি সবচেয়ে সর্বনিম্ন জড়তা (Inertia/WCSS) প্রদান করে, মডেলটি সেটিকেই চূড়ান্ত ফলাফল হিসেবে গ্রহণ করে। এটি মডেলকে লোকাল মিনিমাম থেকে রক্ষা করতে অত্যন্ত সাহায্য করে।
- **অন্যান্য অপশনসমূহ এবং কখন ব্যবহার করবেন:** এটি যেকোনো পূর্ণসংখ্যা (যেমন: 5, 10, 20) গ্রহণ করতে পারে।
 - যখন এটি 'auto' থাকে, তখন এটি init-এর মানের ওপর ভিত্তি করে স্বয়ংক্রিয়ভাবে রানের সংখ্যা নির্ধারণ করে। যদি init='random' বা callable ফাংশন ব্যবহার করা হয়, তবে এটি **১০ বার** রান করে। আর যদি init='k-means++' বা কাস্টম অ্যারে দেওয়া হয়, তবে এটি মাত্র **১ বার** রান করে (কারণ k-means++ প্রথমবারেই চমৎকার সেন্ট্রয়েড খুঁজে পায়)।
 - উচ্চ-মাত্রার স্পার্স ডেটার (Sparse High-Dimensional Data) ক্ষেত্রে লোকাল মিনিমাম এড়াতে এখানে ম্যানুয়ালি বড় সংখ্যা (যেমন: 15 বা 20) ব্যবহার করা উচিত।

২. মিডিয়াম ইম্পর্টেন্ট প্যারামিটারসমূহ (Optimization & Execution)

মডেলের কনভার্জেন্সের গতিবেগ, কম্পিউটেশনাল সময় এবং মেমোরি অপ্টিমাইজেশনের জন্য এই প্যারামিটারগুলো ব্যবহার করা হয়।

max_iter

- **Default:** 300
- **প্যারামিটারটি কী কাজ করে:** এটি একটি একক রানের (Single Run) জন্য K-Means অ্যালগরিদমটি সর্বোচ্চ কতবার ইটারেশন বা চক্র (Centroid Update -> Assign Cluster) চালাবে তা নির্ধারণ করে দেয়।
- **অন্যান্য অপশনসমূহ এবং কখন ব্যবহার করবেন:** যেকোনো ধনাত্মক পূর্ণসংখ্যা পাস করা যায়। ডেটাসেট যদি অনেক বিশাল হয় এবং ৩০০ ইটারেশন হওয়ার আগেই যদি মডেল স্থির হয়ে যায়, তবে কোনো সমস্যা নেই। তবে ডেটা যদি খুব জটিল হয় এবং ৩০০ ইটারেশনের মধ্যেও সেন্ট্রয়েডগুলো স্থির হতে না পারে, তবে এই মান বাড়িয়ে 500 বা 1000 করা প্রয়োজন হতে পারে (যদিও বাস্তব ক্ষেত্রে ৩০০ ইটারেশনই যথেষ্ট)।

tol

- **Default:** 1e-4 (0.0001)
- **প্যারামিটারটি কী কাজ করে:** এটি হলো কনভার্জেন্স থ্রেশহোল্ড বা সহনশীলতার মাত্রা (Convergence Tolerance)। পরপর দুটি ইটারেশনে সেন্ট্রয়েডগুলোর অবস্থানের পার্থক্যের Frobenius Norm যদি এই tol-এর চেয়ে কম হয়, তবে অ্যালগরিদম ধরে নেয় যে মডেলটি পুরোপুরি কনভার্জ বা স্থির হয়ে গেছে এবং সর্বোচ্চ ইটারেশন (max_iter) পূর্ণ হওয়ার আগেই ট্রেনিং বন্ধ করে দেয়।
- **অন্যান্য অপশনসমূহ এবং কখন ব্যবহার করবেন:** এটি যেকোনো ফ্লোট (Float) সংখ্যা গ্রহণ করে। যদি ক্লাস্টারিং অত্যন্ত নিখুঁত এবং সুনির্দিষ্ট করতে হয়, তবে এই মান কমিয়ে 1e-5 বা 1e-6 করা যায় (এতে ট্রেনিং সময় একটু বেশি লাগবে)। আবার দ্রুত খসড়া ফলাফলের জন্য এটি বাড়িয়ে 1e-3 করা যেতে পারে।

algorithm

- **Default:** 'loyd'
- **প্যারামিটারটি কী কাজ করে:** ক্লাস্টারিং গাণিতিক সমস্যাটি সমাধান করার জন্য ব্যাকএন্ডে কোন ক্লাসিক্যাল বা পরিবর্তিত অ্যালগরিদম মেকানিজম ব্যবহার করা হবে তা এটি নিয়ন্ত্রণ করে। ডিফল্ট 'loyd' হলো চিরাচরিত Expectation-Maximization (EM) স্টাইল অ্যালগরিদম।

- **অন্যান্য অপশনসমূহ এবং কখন ব্যবহার করবেন:**

- 'elkan': এটি মূলত জ্যামিতিক ত্রিভুজ অসমতা (Triangle Inequality) ব্যবহার করে কাজ করে। যেসব ডেটাসেটে ক্লাস্টারগুলো খুব সুনির্দিষ্ট এবং পরিষ্কারভাবে আলাদা করা থাকে, সেখানে দূরত্ব গণনার অপ্ৰয়োজনীয় ধাপগুলো এড়িয়ে এটি লয়েডের চেয়ে অনেক দ্রুত কাজ করতে পারে। তবে এটি মেমোরির দিক থেকে বেশ ব্যয়বহুল, কারণ ব্যাকএন্ডে এটি (n_samples, n_clusters) শেপের একটি অতিরিক্ত অ্যারে মেমোরিতে ধরে রাখে। ডেটা সাইজ মাঝারি এবং ক্লাস্টার সুনির্দিষ্ট হলে এটি ব্যবহার করা দারুণ কার্যকর।

৩. লেস ইম্পর্টেন্ট প্যারামিটারসমূহ (System & Housekeeping)

এই প্যারামিটারগুলো মডেলের ক্লাস্টারিং পারফরম্যান্স বা একুরেসিতে কোনো পরিবর্তন আনে না, বরং এগুলো কোডের পুনরুৎপাদনযোগ্যতা (Reproducibility) এবং সিস্টেম রিসোর্স নিয়ন্ত্রণে ব্যবহৃত হয়।

random_state

- **Default:** None
- **প্যারামিটারটি কী কাজ করে:** সেন্ট্রয়েড শুরুকরণের (Centroid Initialization) সময় যে সুডোরাণ্ডম নম্বর জেনারেটর কাজ করে, এটি তার বীজ (Seed) নির্ধারণ করে।
- **অন্যান্য অপশনসমূহ এবং কখন ব্যবহার করবেন:** এখানে যেকোনো নির্দিষ্ট পূর্ণসংখ্যা (যেমন: 0, 42) ব্যবহার করা উচিত। অ্যাসাইনমেন্ট, একাডেমিক রিসার্চ বা 실험 ট্র্যাকিংয়ের সময় কোডটি যতবারই রান করা হোক না কেন, প্রতিবার যেন ছবছ একই প্রাথমিক সেন্ট্রয়েড এবং একই ক্লাস্টার ফলাফল পাওয়া যায়, তা নিশ্চিত করতে এটি লক করা বাধ্যতামূলক।

copy_x

- **Default:** True
- **প্যারামিটারটি কী কাজ করে:** দূরত্বের সংখ্যাসূচক নির্ভুলতা (Numerical Accuracy) বৃদ্ধির জন্য K-Means ব্যাকএন্ডে ডেটাকে প্রথমে সেন্ট্রালাইজ (Center) করে নেয়। এটি True থাকলে মূল ইনপুট ডেটাসেটটি অপরিবর্তিত থাকে এবং একটি কপি নিয়ে কাজ করা হয়।
- **অন্যান্য অপশনসমূহ এবং কখন ব্যবহার করবেন:** False দিলে মেমোরি সাশ্রয় হয় কারণ এটি সরাসরি মূল মেমোরির ডেটার ওপর অপারেশন চালায় এবং কাজ শেষে ডেটার গড় পুনরায় যোগ করে দেয়। তবে এর ফলে খুব সামান্য সংখ্যাসূচক মানের পার্থক্য (Small

Numerical Differences) তৈরি হতে পারে। ডেটার সাইজ যদি র‍্যামের তুলনায় অত্যন্ত বিশাল হয়, কেবল তখনই মেমোরি বাঁচাতে এটি False করা যেতে পারে।

verbose

- **Default: 0**
- **প্যারামিটারটি কী কাজ করে:** এটি ভার্সিটি মোড নিয়ন্ত্রণ করে, অর্থাৎ মডেল ট্রেনিং চলাকালে স্ক্রিনে ভেতরের বিস্তারিত লগ বা অগ্রগতি দেখাবে কি না।
- **অন্যান্য অপশনসমূহ এবং কখন ব্যবহার করবেন:** যেকোনো ধনাত্মক পূর্ণসংখ্যা দেওয়া যায়। verbose=1 বা তার বেশি দিলে প্রতি ইটারেশনে জড়তা বা কস্টের মান কত কমছে তা কনসোলে দেখা যায়, যা মূলত দীর্ঘ সময় ধরে চলা বড় মডেলগুলোর ডিবাগিংয়ে সাহায্য করে।

KMeans() Attributes

cluster_centers_

Default: Created after fit()

প্রতিটি Cluster-এর Final Centroid (Center Point)-এর Coordinates সংরক্ষণ করে। Training শেষে এই Centroid-গুলোর অবস্থানই প্রতিটি Cluster-এর কেন্দ্র নির্দেশ করে।

labels_

Default: Created after fit()

Training Data-এর প্রতিটি Sample কোন Cluster-এ Assigned হয়েছে, সেই Cluster Label সংরক্ষণ করে। Label সাধারণত 0 থেকে n_clusters - 1 পর্যন্ত হয়।

inertia_

Default: Created after fit()

Model-এর Final Within-Cluster Sum of Squares (WCSS) সংরক্ষণ করে। এটি প্রতিটি Data Point থেকে তার Assigned Centroid পর্যন্ত Squared Distance-এর যোগফল। ছোট inertia_ সাধারণত ভালো Clustering নির্দেশ করে এবং Elbow Method-এ এটি ব্যবহৃত হয়।

n_iter_

Default: Created after fit()

Model Converge হতে মোট কতটি Iteration লেগেছে তা সংরক্ষণ করে। যদি আগে Converge হয়ে যায়, তাহলে এটি max_iter-এর চেয়ে কম হতে পারে।

n_features_in_

Default: Created after fit()

Training-এর সময় Model মোট কতটি Input Feature ব্যবহার করেছে তা সংরক্ষণ করে।

feature_names_in_

Default: Created after fit() (*DataFrame input only*)

Training Data যদি Pandas DataFrame হয়, তাহলে Feature-গুলোর নাম সংরক্ষণ করে।

n_steps_

Default: Created after fit() (*Only when using BisectingKMeans, not standard KMeans*)

Standard KMeans-এ এই Attribute থাকে না। BisectingKMeans ব্যবহার করলে Clustering সম্পন্ন করতে মোট কতটি Splitting Step হয়েছে তা সংরক্ষণ করে।

KMeans() Methods

fit(X)

Training Data ব্যবহার করে K-Means Model Train করে। এটি Initial Centroid নির্বাচন করে, Iteratively Cluster Assign করে এবং Final Centroid নির্ধারণ করে।

predict(X)

Train করা Model ব্যবহার করে নতুন Data কোন Cluster-এর সবচেয়ে কাছাকাছি তা নির্ধারণ করে সেই Cluster Label Return করে।

fit_predict(X)

এক ধাপে Model Train করে এবং প্রতিটি Sample-এর Cluster Label Return করে। এটি fit() এবং predict() একসাথে করার সমতুল্য।

transform(X)

প্রতিটি Data Point থেকে প্রতিটি Cluster Centroid পর্যন্ত Distance গণনা করে Return করে। এটি Cluster Label নয়, Distance Matrix প্রদান করে।

fit_transform(X)

প্রথমে Model Train করে, তারপর প্রতিটি Sample থেকে প্রতিটি Cluster Centroid পর্যন্ত Distance Matrix Return করে। Return হওয়া Matrix-এর Shape (n_samples, n_clusters) হয়।

score(X)

Model-এর Negative Inertia (Negative WCSS) Return করে। Scikit-learn-এ বড় Score ভালো ধরা হয়, তাই এটি -inertia_ Return করে।

get_params(deep=True)

Model-এর বর্তমান সব Parameter Dictionary আকারে Return করে। Model Inspect করা বা GridSearchCV-তে ব্যবহার করার জন্য এটি উপকারী।

set_params(params)

এক বা একাধিক Parameter-এর মান পরিবর্তন করে। Parameter পরিবর্তনের পরে Model-কে পুনরায় fit() করতে হয়।

get_metadata_routing()

Scikit-learn-এর Metadata Routing Configuration Return করে। এটি মূলত Advanced Pipeline এবং Meta-estimator ব্যবহারের জন্য তৈরি হয়েছে এবং সাধারণ Machine Learning Workflow-তে খুব কম ব্যবহৃত হয়।

SciKit Learn Code:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

df = load_iris(as_frame=True).frame
df

sns.scatterplot(data=df, x='sepal length (cm)', y='sepal width (cm)', hue='target')
plt.title('Actual Species - Sepal')
plt.show()

sns.scatterplot(data=df, x='petal length (cm)', y='petal width (cm)', hue='target', palette='Set1')
plt.title('Actual Species - Petal')
plt.show()

X2 = df[['petal length (cm)', 'petal width (cm)']].copy()
X2

scaler = StandardScaler()

X2_scaled = scaler.fit_transform(X2)

X2_scaled = pd.DataFrame(X2_scaled, columns=X2.columns)
X2_scaled

wcss = []
silhouette_scores = []

for i in range(2, 21):
    kmeans = KMeans(n_clusters=i, random_state=42)
    kmeans.fit(X2_scaled)

    wcss.append(kmeans.inertia_)
    silhouette_scores.append(silhouette_score(X2_scaled, kmeans.labels_))
```

```
plt.plot(range(2, 21), wcss, marker='o')
plt.title('Elbow Method')
plt.xlabel('Number of Clusters')
plt.ylabel('WCSS')
plt.show()
```

```
plt.plot(range(2, 21), silhouette_scores, marker='x')
plt.title('Silhouette Analysis')
plt.xlabel('Number of Clusters')
plt.ylabel('Silhouette Score')
plt.show()
```

```
km = KMeans(n_clusters=3, init='k-means++', n_init=10, max_iter=100, tol=1e-3, algorithm='lloyd', random_state=42)

km.fit(X2_scaled)

label = km.labels_

X2['cluster'] = label
X2
```

```
X2.cluster.value_counts()
```

```
km.cluster_centers_
```

```
centers = scaler.inverse_transform(km.cluster_centers_)
centers
```

```
sns.scatterplot(data=X2, x='petal length (cm)', y='petal width (cm)', hue='cluster', palette='Set1')

plt.scatter(centers[:, 0], centers[:, 1], c='black', marker='X', s=200, label='Centroids')

plt.title('KMeans Clusters with Centroids')
plt.legend()
plt.show()
```

```
distance = km.transform(X2_scaled)
print(distance)
```

```
print(km.n_iter_)
```

```
print(km.inertia_)
```

```
print(km.score(X2_scaled))
```

```
print(km.get_params())

km.set_params(max_iter=300)

print(km.get_params()['max_iter'])

new_data = pd.DataFrame({
    'petal length (cm)': [1.5, 4.8, 6.2],
    'petal width (cm)': [0.2, 1.6, 2.3]
})

new_data_scaled = scaler.transform(new_data)

predicted_cluster = km.predict(new_data_scaled)

new_data['Predicted Cluster'] = predicted_cluster

print(new_data)

km2 = KMeans(n_clusters=3, random_state=42)

cluster = km2.fit_predict(X2_scaled)

print(cluster)

sns.scatterplot(data=df, x='petal length (cm)', y='petal width (cm)', hue='target', palette='Set1')

plt.title('Actual Species (Ground Truth)')
plt.show()

print(df.target.value_counts())
```

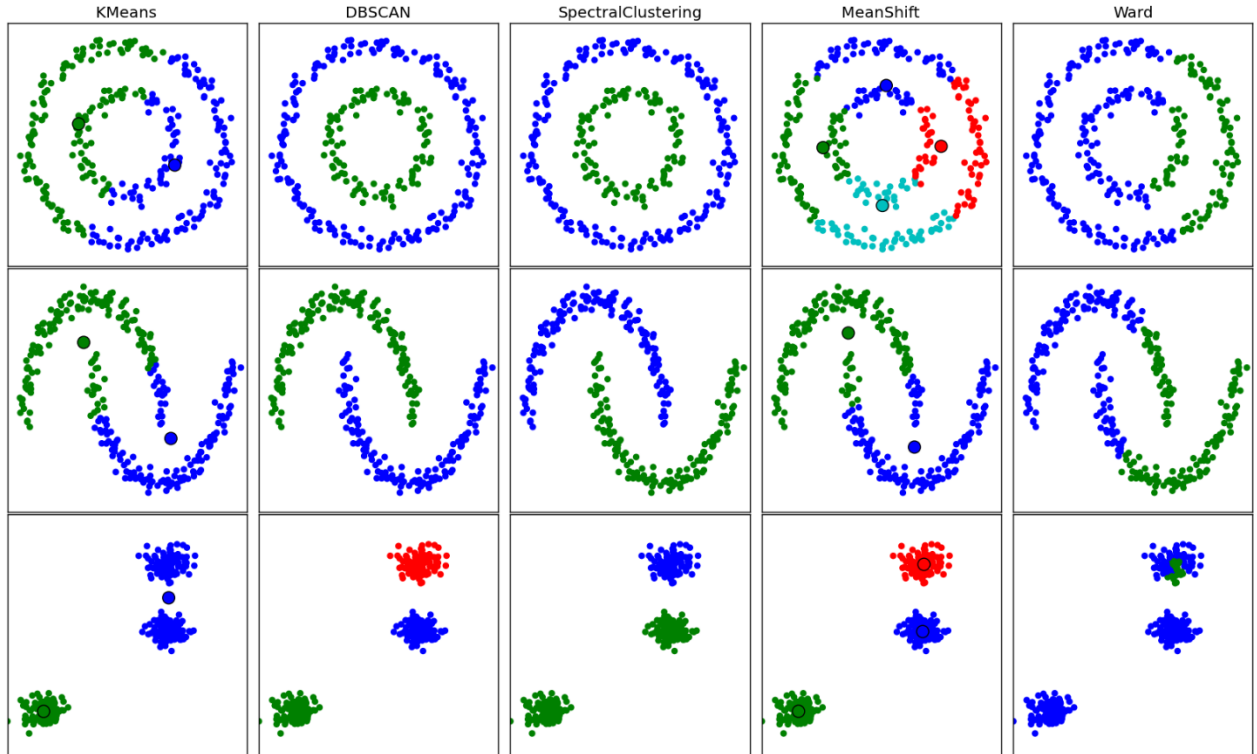
Limitations of K-Means Clustering

১ . আউটলায়ারের প্রতি চরম সংবেদনশীলতা (Extreme Sensitivity to Outliers)

- **সমস্যার প্রকৃতি:** K-Means অ্যালগরিদম প্রতিটি ক্লাস্টারের নতুন কেন্দ্রবিন্দু নির্ধারণের জন্য গাণিতিক গড় বা Arithmetic Mean ব্যবহার করে.
- **প্রভাব:** ডেটাসেটে যদি কোনো চরম Outlier বা ব্যতিক্রমী বড়/ছোট ডেটা পয়েন্ট থাকে, তবে তা Mean-কে নিজের দিকে মারাত্মকভাবে টেনে নিয়ে যায়. ফলে পুরো ক্লাস্টারের আসল জ্যামিতিক বাউন্ডারি বা আকৃতি বিচ্যুত (Distorted) হয়ে পড়ে.

২ . K-এর মান আগে থেকে নির্ধারণ করতে হয় (Pre-defined Number of Clusters)

- **সমস্যার প্রকৃতি:** K-Means চালানোর একদম শুরুতে ইউজারকে নিজের থেকে n_clusters বা K-এর মান নির্দিষ্ট করে দিতে হয়.
- **প্রভাব:** বাস্তব জীবনের অজানা ডেটাসেট দেখে মানুষের পক্ষে আগে থেকে জানা সম্ভব নয় যে ভেতরে কয়টি আসল গ্রুপ লুকিয়ে আছে. যদিও Elbow Method বা Silhouette Score ব্যবহার করে এটি অনুমান করা যায়, তবে তা কম্পিউটেশনালি সময়সাপেক্ষ এবং সবসময় নিখুঁত সমাধান দেয় না.



৩ . শুধুমাত্র বৃত্তাকার বা গোলকধাঁধার মতো ক্লাস্টারে সীমাবদ্ধ (Assumes Spherical Clusters)

- **সমস্যার প্রকৃতি:** K-Means সম্পূর্ণভাবে Euclidean Distance-এর ওপর ভিত্তি করে ডেটা পয়েন্টগুলোকে নিকটবর্তী সেন্ট্রয়েডে অ্যাসাইন করে।
- **প্রভাব:** এটি ধরে নেয় যে সব ক্লাস্টারের আকৃতি বৃত্তাকার বা গোলকাকার (Spherical) এবং তাদের ঘনত্ব (Density) সমান। উপাত্তের বিন্যাস যদি জটিল, লম্বাটে, বাঁকা (Elongated/Non-convex patterns) কিংবা একে অপরের ভেতরে জটিলভাবে শিকলের মতো ঢুকে থাকে, তবে K-Means সেখানে সম্পূর্ণ ব্যর্থ হয়।

৪ . লোকাল মিনিমামে আটকে যাওয়ার ঝুঁকি (Prone to Local Minima)

- **সমস্যার প্রকৃতি:** অ্যালগরিদমটি শুরুতে প্রাথমিক সেন্ট্রয়েডগুলো সম্পূর্ণ আন্দাজে বা র্যান্ডমভাবে বসানোর মাধ্যমে (Centroid Initialization) কাজ শুরু করে।
- **প্রভাব:** প্রাথমিক সেন্ট্রয়েড সিলেকশন যদি ভালো না হয়, তবে মডেলটি গ্লোবাল মিনিমাম (Global Minimum) বা সর্বোত্তম বৈশ্বিক কস্ট পয়েন্টে পৌঁছাতে না পেরে মাঝপথেই লোকাল মিনিমামে (Local Minima) আটকে যায় এবং ভুল ক্লাস্টারিং করে। যদিও Scikit-Learn ডিফল্টভাবে k-means++ এবং n_init ব্যবহার করে এই সমস্যা কমানোর চেষ্টা করে।

৫ . ফিচার স্কেলিং বাধ্যতামূলক (High Dependency on Feature Scaling)

- **সমস্যার প্রকৃতি:** K-Means একটি পিওর দূরত্ব-ভিত্তিক (Distance-based) অ্যালগরিদম।
- **প্রভাব:** যদি ডেটাসেটের ফিচারগুলোর স্কেল অসমান হয় (যেমন: Salary কলামের মান 0 থেকে 1,000,000 এবং Age কলামের মান 0 থেকে 100), তবে বড় স্কেলের ফিচারটি পুরো জ্যামিতিক দূরত্বকে ডমিনেন্ট (Dominant) করে ফেলবে। ফলে ছোট ফিচারের গুরুত্ব সম্পূর্ণ হারিয়ে যায়। তাই StandardScaler প্রয়োগ না করলে এর আউটপুট অর্থহীন হয়ে পড়ে।

৬ . বৃহৎ ডাইমেনশনে কার্যকারিতা হ্রাস (Curse of Dimensionality)

- **সমস্যার প্রকৃতি:** উপাত্ত ক্ষেত্রে ফিচারের সংখ্যা বা ডাইমেনশন (D) যখন বহুগুণ বেড়ে যায়, তখন উচ্চ-মাত্রিক স্পেসে প্রতিটি পয়েন্টের মধ্যবর্তী ইউক্লিডিয়ান দূরত্ব প্রায় সমান হতে শুরু করে।
- **প্রভাব:** ডাইমেনশন অত্যধিক বেশি হলে K-Means আর "সবচেয়ে কাছের সেন্ট্রয়েড" সঠিকভাবে আলাদা করতে পারে না। এতে ক্লাস্টারিং এক্যুরেসি মারাত্মকভাবে কমে যায় এবং কম্পিউটেশনাল মেমোরি কস্ট বহুগুণ বৃদ্ধি পায়।

প্রশ্ন ১: K-Means কীভাবে আউটলায়ার (Outliers) হ্যান্ডেল করে এবং কস্ট ফাংশনের ওপর এর প্রভাব কী?

- **উত্তর:** K-Means অ্যালগরিদম আউটলায়ার বা ব্যতিক্রমী ডেটা পয়েন্টের প্রতি অত্যন্ত সংবেদনশীল (Sensitive), কারণ এটি সেন্ট্রয়েড আপডেট করার জন্য গাণিতিক গড় (Mean) ব্যবহার করে।
- **প্রভাব:** যেহেতু কস্ট ফাংশনটি প্রতিটি পয়েন্ট থেকে সেন্ট্রয়েডের ইউক্লিডিয়ান দূরত্বের বর্গকে সর্বনিম্ন করার চেষ্টা করে ($J = \sum \sum \|x_i - \mu_j\|^2$), তাই একটিমাত্র বিশাল দূরত্বের আউটলায়ার পয়েন্ট পুরো কস্ট ফাংশনকে ডমিনেট বা প্রভাবিত করতে পারে। এর ফলে সেন্ট্রয়েডটি আক্ষরিক অর্থেই আউটলায়ার পয়েন্টটির দিকে মারাত্মকভাবে সরে যায় এবং ক্লাস্টারের আসল জ্যামিতিক সীমানা নষ্ট হয়।

প্রশ্ন ২: Scikit-Learn-এর KMeans-এ n_init প্যারামিটারটির উদ্দেশ্য কী এবং এটি কীভাবে লোকাল মিনিমা রোধ করে?

- **উত্তর:** বাস্তব ক্ষেত্রে K-Means অ্যালগরিদমটি অত্যন্ত দ্রুত কাজ করলেও এটি প্রাথমিক সেন্ট্রয়েডগুলো শুরুতে ঠিক কোথায় বসানো হচ্ছে তার ওপর ভিত্তি করে প্রায়ই লোকাল মিনিমাতে (Local Minima) আটকে যায়।
- **মেকানিজম:** n_init প্যারামিটারটি নির্ধারণ করে যে অ্যালগরিদমটি সম্পূর্ণ ভিন্ন ভিন্ন র্যান্ডম সেন্ট্রয়েড বীজ (Centroid seeds) নিয়ে মোট কতবার নতুন করে রান করবে। Scikit-Learn এই ধারাবাহিক রানগুলো সম্পন্ন করার পর যে রানের আউটপুট সবচেয়ে সর্বনিম্ন জড়তা বা কস্ট (inertia_) প্রদান করে, সেটিকেই চূড়ান্ত ক্লাস্টার মডেল হিসেবে বেছে নেয়। এর ফলে মডেলটির গ্লোবাল মিনিমাম (Global Minimum) বা সবচেয়ে সঠিক সমাধান খুঁজে পাওয়ার সম্ভাবনা বহুগুণ বেড়ে যায়।

প্রশ্ন ৩: K-Means ক্লাস্টারিং চালানোর আগে ফিচার স্কেলিং (Feature Scaling) করা কেন বাধ্যতামূলক?

- **উত্তর:** K-Means একটি দূরত্ব-ভিত্তিক (Distance-based) অ্যালগরিদম, যা ক্লাস্টারের সীমানা নির্ধারণের সময় প্রতিটি ফিচার বা ডাইমেনশনের দূরত্বকে সমান গুরুত্ব দেয়।
- **মেকানিজম:** যদি ডেটাসেটের ফিচারগুলোর স্কেল বা রেঞ্জ অসমান হয় (যেমন: Salary কলামের মান ০ থেকে ১,০০০,০০০ এবং Age কলামের মান ০ থেকে ১০০), তবে দূরত্ব গণনার পুরো প্রক্রিয়াটি সম্পূর্ণভাবে বড় স্কেলের ফিচারটি (Salary) দ্বারা নিয়ন্ত্রিত হবে এবং ছোট স্কেলের ফিচারের (Age) গুরুত্ব হারিয়ে যাবে। StandardScaler-এর মতো উপাদান ব্যবহার করে ফিচার স্কেলিং করলে সমস্ত ফিচারের ভ্যারিয়েন্স একই স্কেলে চলে আসে, যা প্রতিটি ফিচারকে ক্লাস্টার গঠনে সমানভাবে অবদান রাখার সুযোগ করে দেয়।

প্রশ্ন ৪: KMeans-এর ক্ষেত্রে লয়েড (Lloyd) এবং এলকান (Elkan) অ্যালগরিদমের মধ্যে মূল পার্থক্য কী?

- **উত্তর:** লয়েড এবং এলকান উভয় অ্যালগরিদমই ব্যাকঅ্যাঙ্গে একই K-Means গাণিতিক সমস্যার সমাধান করে, তবে তাদের কাজ করার অভ্যন্তরীণ কৌশল ভিন্ন।
- **লয়েড অ্যালগরিদম (Lloyd's):** এটি হলো চিরাচরিত Expectation-Maximization স্টাইলের অ্যালগরিদম, যা প্রতি ইটারেশনে প্রতিটি ডেটা পয়েন্ট থেকে প্রতিটি সেন্ট্রয়েডের দূরত্ব ম্যানুয়ালি গণনা করে।
- **এলকান অ্যালগরিদম (Elkan's):** এটি মূলত জ্যামিতিক ত্রিভুজ অসমতা (Triangle Inequality) ব্যবহার করে কাজ করে। যেসব ডেটাসেটে ক্লাস্টারগুলো খুব সুনির্দিষ্টভাবে আলাদা থাকে, সেখানে এটি দূরত্বের একটি উচ্চ ও নিম্ন সীমা ট্র্যাক করে অপয়োজনীয় দূরত্ব গণনা এড়িয়ে চলে। ফলে এটি লয়েডের চেয়ে অনেক দ্রুত কাজ করতে পারে, তবে ব্যাকঅ্যাঙ্গে একটি অতিরিক্ত দূরত্ব অ্যারে মেমোরিতে ধরে রাখার কারণে এটি অত্যন্ত র‍্যাম বা মেমোরি-সংবেদনশীল (Memory intensive)।

প্রশ্ন ৫: মডেলটি সম্পূর্ণ স্থির (Converge) হওয়ার আগেই যদি অ্যালগরিদমটি বন্ধ হয়ে যায়, তবে সেন্ট্রয়েড ও ক্লাস্টার অ্যাসাইনমেন্টের কী ঘটে?

- **উত্তর:** যদি KMeans অ্যালগরিদমটি পুরোপুরি স্থির বা কনভার্জ হওয়ার আগেই নির্ধারিত সর্বোচ্চ ইটারেশন সীমা (max_iter) স্পর্শ করে বা সহনশীলতার মাত্রা (tol) অতিক্রমের কারণে বন্ধ হয়ে যায়, তবে এর অভ্যন্তরীণ গাণিতিক অবস্থা সামঞ্জস্যহীন (Inconsistent) হয়ে পড়ে।
- **প্রভাব:** এমন পরিস্থিতিতে মেমোরিতে থাকা সেন্ট্রয়েডের স্থানাঙ্কগুলো (cluster_centers_) ওই ক্লাস্টারের অন্তর্ভুক্ত ডেটা পয়েন্টগুলোর প্রকৃত গাণিতিক গড়ের সাথে মিলবে না। তবে প্রডাকশনে কাজ সচল রাখার জন্য, এস্টিমেটরটি একদম শেষ ইটারেশনের পর পুনরায় ডেটা পয়েন্টগুলোর লেবেল রি-অ্যাসাইন করে দেয়, যাতে চূড়ান্ত labels_ অ্যাট্রিবিউটটি মূল ডেটার ওপর চালানো .predict() মেথডের ফলাফলের সাথে পুরোপুরি সামঞ্জস্যপূর্ণ থাকে।

K-Means ক্লাস্টারিংয়ের চূড়ান্ত সারসংক্ষেপ

পেশাদারী ইন্টারভিউ এবং পরীক্ষার কুইক রিভিশনের সুবিধার্থে পুরো অধ্যায়ের মূল গাণিতিক ও কাঠামোগত বিষয়গুলো নিচে সংক্ষেপে টেবিল এবং কী-পয়েন্টের মাধ্যমে উপস্থাপন করা হলো:

১. কুইক রিভিশন ম্যাট্রিক্স (Quick Revision Matrix)

ক্যাটাগরি	মূল আলোচ্য বিষয় ও গাণিতিক রূপ
মূল সমীকরণ (Core Formula)	$J = \sum_{j=1}^K \sum_{x_i \in S_j} \ x_i - \mu_j\ ^2$ (Within-Cluster Sum of Squares)
অ্যালগরিদমের ধরন	Unsupervised, Iterative Partitional Clustering, Eager Learner
জটিলতা (Complexity)	গড় সময় জটিলতা: $O(K \cdot n \cdot T)$; যেখানে T = Iterations
শুরুকরণ কৌশল (Init)	k-means++ (দ্রুত কনভার্জেন্স ও লোকাল মিনিমা এড়ানোর জন্য ডিফল্ট কৌশল)
স্থিরতার শর্ত (Convergence)	সর্বোচ্চ ইটারেশন সীমা (max_iter=300) অথবা সেন্ট্রয়েডের নড়াচড়া থ্রেশহোল্ডের নিচে নামা (tol=1e-4)
মূল মেথডসমূহ (API)	.fit(), .predict(), .transform(), .fit_predict(), .fit_transform()
প্রধান অ্যাক্সেসবিউটস	cluster_centers_ (সেন্ট্রয়েড), labels_ (ক্লাস্টার ইনডেক্স), inertia_ (WCSS কস্ট)

২. মূল জাস্টিফিকেশন ও টেকনিক্যাল ইনসাইটস (Core Structural Insights)

- ফিচার স্কেলিং বাধ্যতামূলক:** K-Means যেহেতু জ্যামিতিক ইউক্লিডিয়ান দূরত্বের ওপর ভিত্তি করে দল গঠন করে, তাই অসমান স্কেলের ফিচার থাকলে বড় স্কেলের ফিচারটি পুরো দূরত্বকে ডমিনেট করে ফেলে। এই কারণে মডেল ফিট করার পূর্বে StandardScaler ব্যবহার করা অপরিহার্য।
- লোকাল মিনিমা ও রিস্টার্ট লজিক:** অ্যালগরিদমটি শুরুতে র্যান্ডম সেন্ট্রয়েড নির্বাচনের ওপর অত্যন্ত নির্ভরশীল হওয়ায় প্রায়ই লোকাল মিনিমাতে আটকে যায়। এই সমস্যা দূর করতে n_init='auto' প্যারামিটার ব্যবহার করে ভিন্ন ভিন্ন সেন্ট্রয়েড বীজ নিয়ে ব্যাকঅ্যান্ডে একাধিকবার লুপ চালানো হয় এবং সর্বনিম্ন ইনোশিয়া রানটি বেছে নেওয়া হয়।
- এলবো মেথড (Elbow Method):** ক্লাস্টারের সংখ্যা বা K -এর মান আগে থেকে জানা না থাকলে, প্রতিবার K -এর মান বাড়িয়ে ইনোশিয়া বা WCSS-এর ড্রপ রেট প্লট করা হয়।

গ্রাফের রেখাটি যে বিন্দুতে এসে হুট করে বাটির কনুইয়ের মতো (Elbow) বেঁকে যায় এবং কস্ট কমার গতি ধীর হয়ে যায়, সেটিকেই আদর্শ ক্লাস্টার সংখ্যা হিসেবে বিবেচনা করা হয়।

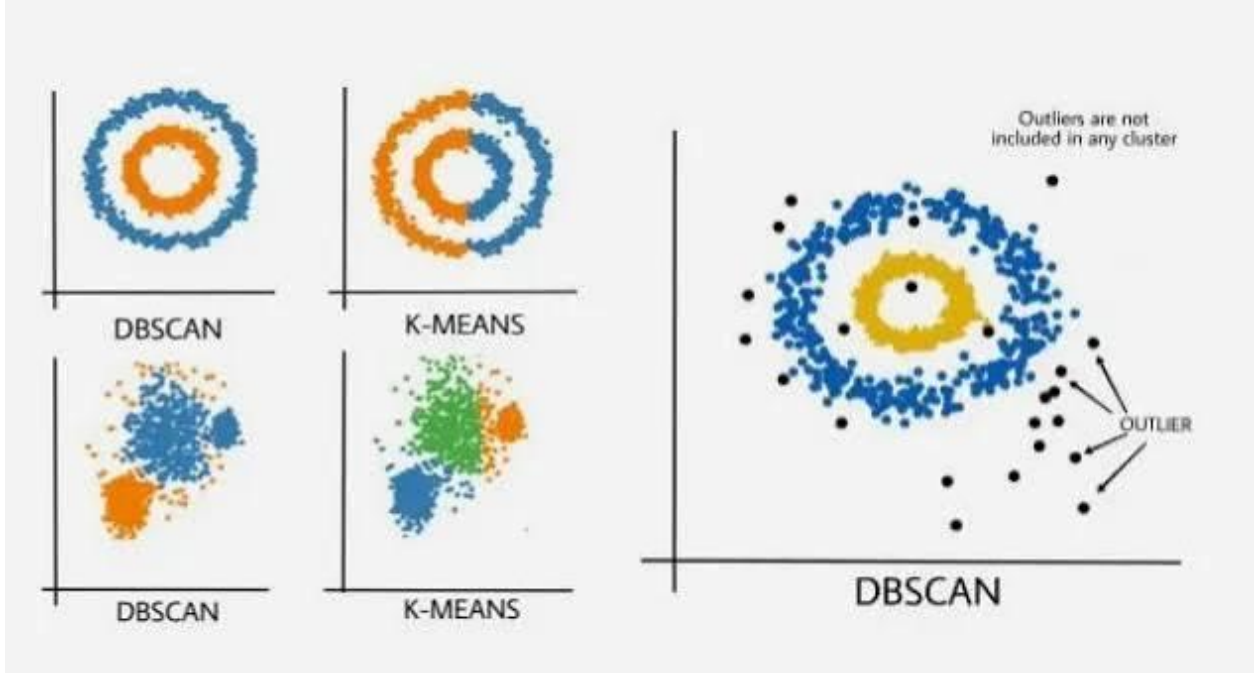
- **পাইপলাইন ইন্টিগ্রেশন:** প্রডাকশন কোডে ডেটা লিক (Data Leakage) সম্পূর্ণ বন্ধ করতে এবং ফিচার স্কেলিং ও মডেলিং মেকানিজমকে লিক-প্রুফ রাখতে সর্বদা ColumnTransformer এবং Pipeline ক্রমানুসারে সাজিয়ে GridSearchCV-এর ভেতর দিয়ে রান করা উচিত।

DBSCAN Clustering

DBSCAN-এর পূর্ণরূপ হলো Density-Based Spatial Clustering of Applications with Noise।

একটি Density-Based বা ঘনত্ব-ভিত্তিক আনসুপারভাইজড মেশিন লার্নিং অ্যালগরিদম। K-Means-এর মতো এটি বৃত্তাকার ক্লাস্টার তৈরি করে না বা আগে থেকে ক্লাস্টারের সংখ্যা (K) জানতে চায় না। বরং, ডেটার ঘনত্ব (Density) মেপে এটি স্বয়ংক্রিয়ভাবে ক্লাস্টার তৈরি করে।

- **Density-Based (DB):** এটি ডেটার উচ্চ-ঘনত্ব বিশিষ্ট এলাকাগুলোকে ক্লাস্টার হিসেবে চিহ্নিত করে।
- **Spatial (S):** এটি বহুমাত্রিক স্পেসে ডেটা পয়েন্টগুলোর পারস্পরিক দূরত্বের ওপর ভিত্তি করে কাজ করে।
- **Clustering (C):** সমধর্মী ডেটাগুলোকে একসাথে গ্রুপ বা দলভুক্ত করে।
- **Applications with Noise (AN):** বাস্তব জীবনের অ্যাপ্লিকেশনে ডেটার মধ্যে যে নয়েজ (Noise) বা অপ্রয়োজনীয় Outlier থাকে, এটি সেগুলোকে ক্লাস্টারে জোর করে না ঢুকিয়ে আলাদাভাবে সনাক্ত করতে পারে।



DBSCAN মূলত ২টি প্যারামিটারের সাহায্যে ডেটার ঘনত্ব পরিমাপ করে:

১. **Epsilon (ϵ):** একটি নির্দিষ্ট ডেটা পয়েন্টের চারপাশের খোঁজাখুঁজির সর্বোচ্চ ব্যাসার্ধ বা Radius।

২. **MinPts (Minimum Points):** একটি ক্লাস্টার শুরু করার জন্য ওই ব্যাসার্ধের ভেতর ন্যূনতম কয়টি ডেটা পয়েন্ট থাকতে হবে তার সংখ্যা।

Why We Need DBSCAN Instead of K-Means

বাস্তব জীবনের জটিল ডেটাসেটে **K-Means Clustering**-এর মূল সীমাবদ্ধতাগুলো দূর করার জন্যই **DBSCAN**-এর প্রয়োজন হয়। নিচে প্রধান কারণগুলো আলোচনা করা হলো:

১. জটিল এবং আঁকাবাঁকা আকৃতির ক্লাস্টার হ্যান্ডলিং (Arbitrary Shaped Clusters)

- **K-Means-এর সমস্যা:** K-Means দূরত্বের গাণিতিক গড় (Mean) ব্যবহার করে বলে এটি সবসময় ধরে নেয় ক্লাস্টারগুলো বৃত্তাকার বা গোলকাকার (Spherical Clusters)। ডেটা যদি অর্ধচন্দ্রাকৃতি, কনসেন্ট্রিক বৃত্ত (একটির ভেতর আরেকটি রিং) বা আঁকাবাঁকা লাইনের মতো হয়, তবে K-Means সেখানে ভুল ক্লাস্টারিং করে।
- **DBSCAN-এর সুবিধা:** যেহেতু DBSCAN দূরত্বের চেয়ে ডেটার ঘনত্বের (Density) ওপর ভিত্তি করে কাজ করে, তাই ডেটার আকৃতি যত জটিল বা আঁকাবাঁকা হোক না কেন, এটি খুব সহজেই আসল প্যাটার্ন সনাক্ত করতে পারে।

২. আউটলায়ার এবং নয়েজ থেকে মুক্তি (Robustness to Noise & Outliers)

- **K-Means-এর সমস্যা:** K-Means-এর ডিসিশন বাউন্ডারি অত্যন্ত সংবেদনশীল। এটি ডেটাসেটে থাকা প্রতিটি সিঙ্গেল আউটলায়ার বা নয়েজ পয়েন্টকে জোর করে কোনো না কোনো ক্লাস্টারে ঢুকিয়ে নেয়। এর ফলে ক্লাস্টারের সেন্ট্রয়েড বা কেন্দ্রবিন্দুটি আসল অবস্থান থেকে বিচ্যুত (Drift) হয়ে যায়।
- **DBSCAN-এর সুবিধা:** DBSCAN-এর নামের ভেতরেই 'Noise' শব্দটি আছে। এটি কম ঘনত্বের এলাকার বিচ্ছিন্ন পয়েন্টগুলোকে সরাসরি **Noise** বা আউটলায়ার হিসেবে ফ্ল্যাগ (Flag) করে রেখে দেয় এবং ক্লাস্টারের ভেতরে জায়গা দেয় না। ফলে আসল ক্লাস্টারগুলো একদম নিখুঁত থাকে।

৩. K-এর মান আগে থেকে জানার ঝামেলা নেই (No Need to Specify 'K')

- **K-Means-এর সমস্যা:** K-Means রান করার একদম শুরুতে ইউজারকে $n_clusters$ বা K -এর মান নির্দিষ্ট করে দিতে হয়। বাস্তব জীবনের অজানা বড় ডেটাসেটে কয়টি গ্রুপ আছে তা আগে থেকে অনুমান করা অত্যন্ত কঠিন এবং ভুল K সিলেক্ট করলে পুরো আউটপুট নষ্ট হয়ে যায়।

- **DBSCAN-এর সুবিধা:** DBSCAN-এ আগে থেকে কয়টি ক্লাস্টার তৈরি হবে তা বলে দিতে হয় না। এটি ডেটার ডেনসিটি বা ঘনত্ব বিশ্লেষণ করে স্বয়ংক্রিয়ভাবে নিজেই খুঁজে বের করে যে সেখানে ঠিক কয়টি ক্লাস্টার হওয়া উচিত।

8. সেন্ট্রয়েড ইনিশিয়ালাইজেশনের ওপর নির্ভরশীল নয় (No Initial Centroid Dependency)

- **K-Means-এর সমস্যা:** K-Means-এর পারফরম্যান্স শুরুতে র্যান্ডমভাবে সিলেক্ট করা সেন্ট্রয়েডের ওপর নির্ভর করে। ইনিশিয়াল সেন্ট্রয়েড সিলেকশন খারাপ হলে মডেলটি লোকাল মিনিমামে (Local Minima) আটকে যায় এবং প্রতিবার রান করলে ভিন্ন ভিন্ন ফলাফল দিতে পারে।
- **DBSCAN-এর সুবিধা:** DBSCAN-এ কোনো সেন্ট্রয়েড বা কেন্দ্রবিন্দুর ধারণা নেই। এটি ডেটা পয়েন্টগুলোর ভেতরের ডেনসিটি লিংক ধরে অগ্রসর হয়, তাই এটি সবসময় স্থিতিশীল (Stable) এবং ডিটারমিনিস্টিক ফলাফল দেয়।

Core Parameters of DBSCAN & Handling Sparse Density

DBSCAN অ্যালগরিদমের পুরো মেকানিজম এবং ক্লাস্টার গঠনের নিখুঁততা সম্পূর্ণভাবে দুটি হাইপারপ্যারামিটারের ওপর নির্ভর করে। নিচে এদের গাণিতিক ও ব্যবহারিক রূপ আলোচনা করা হলো:

১ . Epsilon (ϵ / Eps)

- **প্যারামিটারটি কী কাজ করে:** এটি হলো একটি নির্দিষ্ট ডেটা পয়েন্টের চারপাশে প্রতিবেশীদের খুঁজে বের করার সর্বোচ্চ জ্যামিতিক ব্যাসার্ধ বা Radius । সহজ ভাষায়, কোনো পয়েন্টের চারপাশের কতটুকু দূরত্বকে তার "Neighborhood" বা এলাকা ধরা হবে, তা ϵ নির্ধারণ করে।
- **প্রভাব (Effect of Tuning):**
 - ϵ খুব ছোট হলে (Small ϵ): খোঁজাখুঁজির এলাকা ছোট হয়ে যায়। ফলে ডেটা পয়েন্টগুলো একে অপরের খুব কাছে থাকা সত্ত্বেও প্রয়োজনীয় ঘনত্ব না পেয়ে বিচ্ছিন্ন হয়ে যায়। এতে অনেক আসল ক্লাস্টার তৈরি হতে পারে না এবং ডেটার একটি বড় অংশ Noise হিসেবে বাদ পড়ে যায় (Underfitting/High Bias)।
 - ϵ খুব বড় হলে (Large ϵ): এলাকা অনেক বড় হয়ে যায়। এর ফলে দুটি আলাদা ও ভিন্ন ভিন্ন ক্লাস্টারও একটি বড় ব্যাসার্ধের কারণে জোড়া লেগে একটি একক ক্লাস্টার হয়ে যেতে পারে (Overfitting/High Variance)।

২ . Minimum Points (MinPts)

- **প্যারামিটারটি কী কাজ করে:** এটি নির্ধারণ করে যে, ϵ ব্যাসার্ধের ভেতরের এলাকায় (Neighborhood) নিজেসহ ন্যূনতম কতটি ডেটা পয়েন্ট উপস্থিত থাকলে তবেই একটি নতুন ক্লাস্টার বা ঘন অঞ্চল (Dense Region) গঠন করা শুরু হবে।
- **প্রভাব (Effect of Tuning):**
 - **MinPts খুব ছোট হলে (যেমন: $\text{MinPts} \leq 2$):** মডেলটি অত্যন্ত নমনীয় হয়ে যায়। এর ফলে সামান্য ৩-৪টি নয়েজ বা আউটলায়ার পয়েন্টও কাছাকাছি থাকলে তারা নিজেদের একটি স্বতন্ত্র ক্লাস্টার বানিয়ে ফেলে।
 - **MinPts খুব বড় হলে:** মডেলটি কঠোর বা Strict হয়ে যায়। তখন শুধুমাত্র চরম ঘন (Extremely Dense) অঞ্চল ছাড়া অন্য কোথাও ক্লাস্টার তৈরি হতে পারে না, এবং মাঝারি ঘনত্বের এলাকাগুলোও নয়েজ হিসেবে বাদ পড়ে যায়।
- **Rule of Thumb:** সাধারণ ডেটাসেটের ক্ষেত্রে $\text{MinPts} = 2 \times \text{Dimension}$ সূত্রটি ব্যবহার করা হয়। ডেটাতে নয়েজ বেশি থাকলে MinPts-এর মান আরও বাড়িয়ে দেওয়া উচিত।

Data Point Classifications in DBSCAN

DBSCAN অ্যালগরিদমে `model.fit()` হওয়ার পর, এটি ইনপুট করা ϵ (Epsilon) এবং *MinPts* (Minimum Points)-এর ওপর ভিত্তি করে ডেটাসেটের প্রতিটি ডেটা পয়েন্টকে ৩টি সুনির্দিষ্ট ভাগে ক্লাসিফাই করে।

১ . Core Points (মূল বিন্দু)

- **সংজ্ঞা (Definition):** কোনো একটি ডেটা পয়েন্টের চারপাশে যদি ϵ ব্যাসার্ধের (Epsilon Radius) একটি বৃত্ত আঁকা হয় এবং সেই বৃত্তের ভেতর নিজেসহ ন্যূনতম *MinPts* সংখ্যক ডেটা পয়েন্ট থাকে, তবে তাকে **Core Point** বলা হয়।
- **গাণিতিক শর্ত (Mathematical Condition):**
$$\text{Neighborhood Points}(P) \geq \text{MinPts}$$
 - এই পয়েন্টগুলো ক্লাস্টারের একদম কেন্দ্র বা ভেতরের ঘন অঞ্চল (Dense Interior) গঠন করে। একটি ক্লাস্টারে একাধিক বা শত শত Core Point থাকতে পারে এবং তারা একে অপরের সাথে ঘনত্বের শিকল দিয়ে যুক্ত থাকে।

২ . Border Points (সীমান্ত বিন্দু)

- **সংজ্ঞা (Definition):** একটি ডেটা পয়েন্টের নিজস্ব ϵ ব্যাসার্ধের বৃত্তের ভেতর যদি $MinPts$ -এর চেয়ে কম সংখ্যক পয়েন্ট থাকে, কিন্তু পয়েন্টটি যদি অন্য কোনো একটি Core Point-এর এলাকার ভেতরে অবস্থান করে, তবে তাকে **Border Point** বলা হয়।
- **গাণিতিক শর্ত (Mathematical Condition):**
 $Neighborhood\ Points(P) < MinPts$ AND $P \in Neighborhood\ of\ a\ Core\ Point$
- এরা নিজেরা কোনো ক্লাস্টার শুরু করার মতো যথেষ্ট ঘন নয়, কিন্তু কোনো একটি শক্তিশালী Core Point-এর প্রতিবেশী হওয়ায় এরা ক্লাস্টারের অংশ হিসেবে গণ্য হয়। এই পয়েন্টগুলো মূলত একটি ক্লাস্টারের একদম বাইরের সীমানা বা ডালপালা (Outer Edges) তৈরি করে।

৩ . Noise Points / Outliers (নয়েজ বা বিচ্ছিন্ন বিন্দু)

- **সংজ্ঞা (Definition):** যে ডেটা পয়েন্টটি নিজে কোনো Core Point-ও নয়, আবার অন্য কোনো Core Point-এর সীমানার বা এলাকার ভেতরেও পড়ে না (অর্থাৎ সে কোনো Core Point-এর প্রতিবেশী নয়), তাকে **Noise Point** বলা হয়।
- **গাণিতিক শর্ত (Mathematical Condition):**
 $Neighborhood\ Points(P) < MinPts$ AND $P \notin Neighborhood\ of\ any\ Core\ Point$
- **নোটে রাখার মতো বৈশিষ্ট্য (Key Concept):** এই পয়েন্টগুলো ডেটাসেটের অত্যন্ত কম ঘনত্বের (Sparse Density) বা ছড়ানো-ছিটানো এলাকায় একা অবস্থান করে। DBSCAN-এর সবচেয়ে চমৎকার ব্যাপার হলো, এটি এই Noise Point গুলোকে কোনো ক্লাস্টারে জোর করে জায়গা না দিয়ে সরাসরি -1 লেবেল দিয়ে আউটলায়ার হিসেবে আলাদা করে দেয়।

ধরা যাক, আমরা প্যারামিটার সেট করেছি: $\epsilon = 2\text{ cm}$ এবং $MinPts = 4$ । এবার নিচের ৩টি পয়েন্টের আচরণ লক্ষ্য করি:

- **Point A:** তার ২ সেমি ব্যাসার্ধের বৃত্তের ভেতর মোট ৫টি পয়েন্ট আছে। যেহেতু $5 \geq 4$, তাই **Point A একটি Core Point**।
- **Point B:** তার ২ সেমি ব্যাসার্ধের বৃত্তের ভেতর মাত্র ৩টি পয়েন্ট আছে ($3 < 4$, তাই সে Core হতে পারল না)। কিন্তু, সে উপরের **Point A (Core Point)**-এর বৃত্তের ভেতরে বসা আছে। সুতরাং, **Point B একটি Border Point**।

- **Point C:** তার ২ সেমি বৃত্তের ভেতর মাত্র ১টি পয়েন্ট আছে (সে নিজে)। এবং আশেপাশে ২ সেমির মধ্যে কোনো Core Point-ই নেই। সুতরাং, **Point C একটি Noise Point।**

ঘনত্ব-সংযুক্ত বিন্দু এবং ক্লাস্টার গঠনের মেকানিজম (Density-Connected Points & Cluster Formation)

মেশিন লার্নিং শাস্ত্রে DBSCAN অ্যালগরিদমের ব্যাকগ্রাউন্ড ওয়ার্কফ্লো এবং জ্যামিতিক ডিসিশন বাউন্ডারি বোঝার জন্য সবচেয়ে গুরুত্বপূর্ণ মেকানিজম হলো বিন্দুগুলোর পারস্পরিক ঘনত্ব-গম্যতা (Density-Reachability) এবং ঘনত্ব-সংযুক্ততা (Density-Connectedness)। বাস্তব জীবনের জটিল উপাত্ত ক্ষেত্রে দুটি অত্যন্ত দূরবর্তী বিন্দু সরাসরি একে অপরের সীমানার মধ্যে না থাকা সত্ত্বেও কীভাবে একই ক্লাস্টারের (Same Cluster) অন্তর্ভুক্ত হয়, তা মূলত একটি অবিচ্ছিন্ন গাণিতিক শিকল বা চেইনের মাধ্যমে নির্ধারিত হয়।

নিচে এই মেকানিজমের গাঠনিক উপাদান এবং গাণিতিক নীতি বিস্তারিতভাবে আলোচনা করা হলো:

১. সরাসরি ঘনত্ব-গম্যতা (Directly Density-Reachable)

কোনো একটি উপাত্ত বিন্দু P যদি অন্য একটি মূল বিন্দু বা Core Point Q -এর ϵ ব্যাসার্ধের (Epsilon Neighborhood) ভেতরে সরাসরি অবস্থান করে, তবে গাণিতিক ভাষায় বলা হয় যে, P বিন্দুটি Q থেকে সরাসরি ঘনত্ব-গম্য বা Directly Density-Reachable।

- **নীতি:** এই শর্তটি পূরণের জন্য একটি বিন্দুকে অবশ্যই Core Point হতে হবে, যার ওপর অন্য বিন্দুটি নির্ভরশীল। তবে নির্ভরশীল বিন্দুটি নিজে Core Point বা Border Point যেকোনো একটি হলেই চলে।
- **কাঠামোগত উদাহরণ:** ধরা যাক, B একটি Core Point এবং A তার সীমানার ভেতরে থাকা একটি সীমান্ত বিন্দু বা Border Point। এই জ্যামিতিক বিন্যাসে A বিন্দুটি B থেকে সরাসরি ঘনত্ব-গম্য (Directly Density-Reachable)।

২. ধারাবাহিক ঘনত্ব-গম্যতা (Density-Reachable)

যদি কোনো একটি বিন্দু P থেকে অন্য একটি দূরবর্তী বিন্দু Q -তে পৌঁছানোর জন্য মাঝখানে একঝাঁক মূল বিন্দুর একটি ধারাবাহিক শিকল (Chain of Core Points) বিদ্যমান থাকে, যেখানে প্রতি জোড়া পরপর বিন্দু একে অপরের থেকে Directly Density-Reachable, তবে তাকে ধারাবাহিক ঘনত্ব-গম্য বা Density-Reachable বলা হয়।

- **নীতি:** এটি মূলত একটি ট্রানজিটিভ প্রোপার্টি (Transitive Property)। যদি X থেকে Y -তে যাওয়া যায় এবং Y থেকে Z -এ যাওয়া যায়, তবে পরোক্ষভাবে X থেকে Z -এ পৌঁছানো সম্ভব।
- **কাঠামোগত উদাহরণ:** ধরা যাক, B, C , এবং D হলো তিনটি স্বতন্ত্র মূল বিন্দু বা Core Points, যারা পরস্পরের ব্যাসার্ধের সাথে ওভারল্যাপ করে একটি চেইন গঠন করেছে ($B \leftrightarrow C \leftrightarrow D$)। যেহেতু B থেকে C এবং C থেকে D -তে ক্রমান্বয়ে পৌঁছানো যাচ্ছে, তাই দূরবর্তী হওয়া সত্ত্বেও B এবং D একে অপরের সাপেক্ষে ঘনত্ব-গম্য (Density-Reachable)।

৩. ঘনত্ব-সংযুক্ততা (Density-Connected)

এটিই ক্লাস্টার গঠনের চূড়ান্ত গাণিতিক শর্ত। যদি উপাত্ত ক্ষেত্রের দুটি বিন্দু A এবং F একে অপরের সাথে সরাসরি যুক্ত না থাকে, এমনকি তারা যদি একে অপরের থেকে মাইলের পর মাইল দূরে অবস্থান করে, তবুও তারা যদি অন্য কোনো একটি সাধারণ মূল বিন্দু চেইনের (Common Core Point Chain) সাথে যুক্ত বা Density-Reachable থাকে, তবে গাণিতিকভাবে A এবং F -কে ঘনত্ব-সংযুক্ত বা Density-Connected বলা হয়।

DBSCAN-এর স্বর্ণসূত্র (The Golden Rule of DBSCAN):

উপাত্ত সেটের যে বিন্দুগুলো একে অপরের সাথে জ্যামিতিকভাবে ঘনত্ব-সংযুক্ত (Density-Connected) অবস্থায় থাকে, অ্যালগরিদমটি তাদের সবাইকে একটি একক ক্লাস্টারের (Same Cluster) অন্তর্ভুক্ত করে।

The Workflow Chain

DBSCAN-এর ব্যাকএন্ডে পুরো প্রসেসটি নিচের চেইনের মাধ্যমে ধাপে ধাপে সম্পন্ন হয়:

$$A \text{ (Border)} \leftrightarrow B \text{ (Core)} \leftrightarrow C \text{ (Core)} \leftrightarrow D \text{ (Core)} \leftrightarrow F \text{ (Border)}$$

১. শুরুতে B নামক Core Point-এর খোঁজাখুঁজি প্রক্রিয়ায় তার Border Point হিসেবে A -কে সনাক্ত করা হয়।
২. এরপর B -এর ঘনত্ব বা লিঙ্কিং ধরে অ্যালগরিদমটি সামনে অগ্রসর হয়ে পরবর্তী Core Point C -কে খুঁজে পায়।
৩. C -এর ব্যাসার্ধ ধরে চেইনটি আরও বিস্তৃত হয়ে D নামক আরেকটি Core Point-এর সাথে যুক্ত হয়।
৪. চূড়ান্ত ধাপে D তার নিজস্ব সীমানার শেষ প্রান্তে থাকা Border Point F -কে ক্লাস্টারে টেনে নেয়।

চূড়ান্ত সিদ্ধান্ত:

এই জ্যামিতিক বিন্যাসে A এবং D সরাসরি কানেক্টেড নয়, কিংবা A এবং F -এর মধ্যে কোনো সরাসরি যোগাযোগ নেই। কিন্তু ব্যাকঅ্যাঙ্গে $B \rightarrow C \rightarrow D$ নামক কোর বিন্দুগুলোর একটি অবিচ্ছিন্ন ডেনসিটি সেতু থাকার কারণে পুরো শিকলটি একসাথে লক হয়ে যায়। এই ঘনত্ব-সংযুক্ততার (Density-Connectedness) কারণেই A, B, C, D , এবং F বিন্দুগুলো কোনো বিভাজন ছাড়াই একটি একক এবং সুনির্দিষ্ট ক্লাস্টারের অংশ হিসেবে আত্মপ্রকাশ করে।

ডিবিএসক্যান অ্যালগরিদমের অভ্যন্তরীণ কার্যপ্রক্রিয়া (Full Backend Workflow)

সুপারভাইজড লার্নিং বা সেন্ট্রয়েড-ভিত্তিক ক্লাস্টারিংয়ের মতো DBSCAN কোনো লুপ বা গাণিতিক সমীকরণের মাধ্যমে কেন্দ্রবিন্দু খোঁজার চেষ্টা করে না। এটি সম্পূর্ণ ডেটাসেটের ঘনত্ব এবং বিন্দুগুলোর মধ্যকার জ্যামিতিক লিঙ্কিং বিশ্লেষণ করে ধাপে ধাপে অগ্রসর হয়।

ব্যাকএন্ডে পুরো প্রসেসটি নিচের ধাপগুলোর মাধ্যমে সম্পন্ন হয়:

ধাপ ১: উপাত্ত বিন্দু নির্বাচন এবং লেবেলহীন ঘোষণা (Initialization)

অ্যালগরিদমের শুরুতেই ডেটাসেটের প্রতিটি বিন্দুকে 'Unvisited' বা লেবেলহীন হিসেবে চিহ্নিত করা হয়। এরপর সম্পূর্ণ উপাত্ত ক্ষেত্র থেকে র্যান্ডমলি (এলোমেলোভাবে) যেকোনো একটি প্রাথমিক বিন্দু P বেছে নেওয়া হয়।

ধাপ ২: এলাকার ঘনত্ব পরিমাপ (Neighborhood Query)

নির্বাচিত বিন্দু P -কে কেন্দ্র করে আগে থেকে নির্ধারিত ব্যাসার্ধ ϵ (Epsilon) অনুযায়ী একটি জ্যামিতিক বৃত্ত বা এলাকা কল্পনা করা হয়। এই সীমানার ভেতর মোট কতটি প্রতিবেশী বিন্দু (Neighborhood Points) অবস্থান করছে তা গণনা করা হয়।

ধাপ ৩: বিন্দুর শ্রেণীবিভাগ এবং ক্লাস্টার সূচনা (Core Point Check)

- যদি বৃত্তের ভেতর নিজের সহ বিন্দুর সংখ্যা $MinPts$ -এর চেয়ে কম হয়, তবে সাময়িকভাবে P -কে **Noise** বা আউটলায়ার হিসেবে লেবেল করা হয় এবং অ্যালগরিদমটি পুনরায় ধাপ ১-এ ফিরে গিয়ে নতুন একটি আনভিজিটেড বিন্দু বেছে নেয়।
- আর যদি বিন্দুর সংখ্যা $MinPts$ বা তার বেশি হয়, তবে P -কে একটি **Core Point** ঘোষণা করা হয় এবং এই বিন্দুটিকে কেন্দ্র করে একটি নতুন ক্লাস্টার (Cluster) গঠন করার প্রক্রিয়া শুরু হয়।

ধাপ ৪: চেইন রিয়্যাকশন ও ক্লাস্টার সম্প্রসারণ (Density-Reachable Expansion)

যখন একটি নতুন ক্লাস্টার তৈরি হয়, তখন P -এর ϵ ব্যাসার্ধের ভেতরে থাকা সমস্ত প্রতিবেশী বিন্দুকে একটি অস্থায়ী 'Queue' বা কার্য তালিকায় যুক্ত করা হয়। এরপর এই তালিকা ধরে চেইন রিয়্যাকশন শুরু হয়:

- তালিকা থেকে প্রতিটি বিন্দুকে একে একে তুলে আনা হয় এবং তার চারপাশে আবার ϵ ব্যাসার্ধের এলাকা পরীক্ষা করা হয়।
- যদি সেই নতুন বিন্দুটিও একটি Core Point হিসেবে প্রমাণিত হয়, তবে তার নিজস্ব প্রতিবেশীদেরও এই ক্লাস্টারের কার্য তালিকায় (Queue) যুক্ত করা হয়। এভাবে ঘনত্বের সেতু তৈরি করে ক্লাস্টারটি চারদিকে ছড়াতে থাকে।

- যদি কোনো বিন্দু Core Point না হয়ে অন্য কোনো Core Point-এর সীমানায় থাকে, তবে তাকে **Border Point** হিসেবে ক্লাস্টারে অন্তর্ভুক্ত করা হয়, কিন্তু তার চারপাশে নতুন করে আর কোনো প্রতিবেশীর খোঁজ করা হয় না (চেইন রিয়্যাকশন এখানে থেমে যায়)।

ধাপ ৫: ক্লাস্টার সমাপ্তি (Convergence of a Single Cluster)

যখন বর্তমান কার্য তালিকার (Queue) সমস্ত বিন্দুর খোঁজাখুঁজি শেষ হয়ে যায় এবং ঘনত্ব-সংযুক্ত (Density-Connected) হওয়ার মতো নতুন কোনো বিন্দু আর অবশিষ্ট থাকে না, তখন অ্যালগরিদমটি বর্তমান ক্লাস্টারটির বৃদ্ধি বন্ধ করে দেয় এবং সেটিকে একটি সম্পূর্ণ ক্লাস্টার হিসেবে মেমোরিতে লক করে ফেলে।

ধাপ ৬: পুনরাবৃত্তি এবং চূড়ান্ত নয়েজ নির্ধারণ (Iteration & Final Output)

এরপর অ্যালগরিদমটি ডেটাসেটের অবশিষ্ট 'Unvisited' বিন্দুগুলোর দিকে নজর দেয় এবং **ধাপ ১ থেকে ৫** পর্যন্ত পুরো প্রক্রিয়াটি বারবার পুনরাবৃত্তি করতে থাকে।

- এই প্রক্রিয়ায় যদি কোনো বিন্দু পূর্বে 'Noise' হিসেবে লেবেল হয়ে থাকে, কিন্তু পরবর্তীতে সম্প্রসারণের সময় সেটি কোনো ক্লাস্টারের Border Point হিসেবে ধরা পড়ে, তবে তার আগের ভুল লেবেলটি মুছে তাকে ওই ক্লাস্টারের অংশ বানিয়ে দেওয়া হয়।
- সম্পূর্ণ ডেটাসেটের প্রতিটি বিন্দু ভিজিট করা শেষ হলে, যে বিন্দুগুলো কোনো ক্লাস্টারের Core বা Border Point হতে পারেনি, সেগুলোকে চূড়ান্তভাবে **Noise / Outlier (Label: -1)** ঘোষণা করে মেমোরি থেকে আলাদা করে দেওয়া হয় এবং ট্রেনিং প্রক্রিয়া সমাপ্ত হয়।

১. প্রতিবেশ বা এলাকার দূরত্ব পরিমাপের সূত্র (Euclidean Distance Formula)

ধাপ ২ (Neighborhood Query)-এ একটি বিন্দুকে কেন্দ্র করে চারপাশে যে Epsilon (ϵ) ব্যাসার্ধের এলাকা বা বৃত্ত কল্পনা করা হয়, তার জন্য ব্যাকএন্ডে ইউক্লিডিয়ান দূরত্ব (Euclidean Distance)-এর জ্যামিতিক সূত্র ব্যবহার করা হয়।

ধরা যাক, আমাদের নির্বাচিত প্রাথমিক বিন্দুটি হলো $P(x_1, y_1)$ এবং ডেটাসেটের অন্য যেকোনো একটি বিন্দু হলো $Q(x_2, y_2)$ । তাহলে তাদের মধ্যকার দূরত্ব (d) নির্ণয়ের সূত্র:

$$d(P, Q) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

DBSCAN-এর জ্যামিতিক শর্ত: যদি $d(P, Q) \leq \epsilon$ হয়, তবেই কেবল Q বিন্দুটি P -এর প্রতিবেশ বা Neighborhood ($N_\epsilon(P)$)-এর অংশ হিসেবে গণ্য হবে।

২. এলাকার ঘনত্ব এবং বিন্দুর শ্রেণীবিভাগ পরিমাপের সূত্র (Density Set-Inclusion Formula)

ধাপ ৩ (Core Point Check) এবং **ধাপ ৪ (Cluster Expansion)**-এ কোনো বিন্দু Core Point নাকি Noise হবে, তা নির্ধারণের জন্য সেট থিওরি বা সেট-ইনক্লুশন (Set-Inclusion) এবং কার্ডিনালিটি (Count/Cardinality)-এর গাণিতিক সূত্র ব্যবহার করা হয়।

ধরা যাক, P বিন্দুর ϵ ব্যাসার্ধের ভেতরের প্রতিবেশীদের সেটটি হলো $N_\epsilon(P)$ । এই সেটের গাণিতিক রূপ:

$$N_\epsilon(P) = \{Q \in X \mid d(P, Q) \leq \epsilon\}$$

এখন, এই সেটের মোট বিন্দুর সংখ্যা বা কার্ডিনালিটিকে $|N_\epsilon(P)|$ দ্বারা প্রকাশ করা হয়।

বিন্দুর শ্রেণীবিভাগের গাণিতিক শর্তসমূহ:

- **Core Point (মূল বিন্দু) হওয়ার শর্ত:**

$$|N_\epsilon(P)| \geq \text{min_samples}$$

(অর্থাৎ, এলাকার মোট বিন্দুর সংখ্যা যদি min_samples বা তার বেশি হয়)

- **Noise Point (নয়েজ বা আউটলায়ার) হওয়ার শর্ত:**

$$|N_\epsilon(P)| < \text{min_samples} \quad \text{AND} \quad P \notin N_\epsilon(C)$$

(এখানে C হলো ডেটাসেটের যেকোনো একটি Core Point। অর্থাৎ, নিজস্ব এলাকায় বিন্দু কম থাকবে এবং সে অন্য কোনো কোর বিন্দুর প্রতিবেশে বা এলাকাতেও থাকবে না)

- **Border Point (সীমান্ত বিন্দু) হওয়ার শর্ত:**

$$|N_\epsilon(P)| < \text{min_samples} \quad \text{AND} \quad P \in N_\epsilon(C)$$

(অর্থাৎ, নিজস্ব এলাকায় বিন্দু কম থাকবে, কিন্তু সে অন্য একটি কোর বিন্দু C-এর এলাকার ভেতরে অবস্থান করবে)

ডিবিএসক্যান অ্যালগরিদমের সম্পূর্ণ গাণিতিক প্রয়োগ (Hand-Calculated Numerical Example)

ডিবিএসক্যান (DBSCAN) অ্যালগরিদমের অভ্যন্তরীণ কার্যপ্রক্রিয়া এবং এর দুটি গাণিতিক সূত্র (Euclidean Distance ও Density Set-Inclusion) কীভাবে ব্যাকএন্ডে কাজ করে, তা একটি ছোট হাতে-গণনাযোগ্য ডেটাসেটের মাধ্যমে শুরু থেকে শেষ পর্যন্ত ধাপে ধাপে দেখানো হলো।

১. ডেটাসেট ও হাইপারপ্যারামিটার নির্ধারণ (Dataset & Parameters)

ধরা যাক, একটি দ্বিমাত্রিক স্থানে (2D Space) আমাদের কাছে $N = 4$ টি ডেটা পয়েন্ট রয়েছে:

- $A_1 = (1,1)$
- $A_2 = (2,1)$
- $A_3 = (2,2)$
- $A_4 = (6,5)$

ডোমেইন নলেজের ভিত্তিতে প্যারামিটার নির্বাচন:

- **Epsilon (ϵ)** = 2 (প্রতিবেশ খোঁজার সর্বোচ্চ জ্যামিতিক ব্যাসার্ধ)
- **min_samples** = 3 (Core Point হওয়ার সর্বনিম্ন ঘনত্ব থ্রেশহোল্ড)

২. প্রতিবেশীদের দূরত্ব গণনা ও শ্রেণীবিভাগ (Distance Calculation & Classification)

অ্যালগরিদমটি প্রতিটি বিন্দুর জন্য ইউক্লিডিয়ান দূরত্ব সূত্র ব্যবহার করে তার প্রতিবেশ সেট $N_\epsilon(P)$ এবং বিন্দুর সংখ্যা $|N_\epsilon(P)|$ গণনা করবে।

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

ক. বিন্দু $A_1(1,1)$ এর জন্য প্রতিবেশ অনুসন্ধান:

- $d(A_1, A_1) = \sqrt{(1-1)^2 + (1-1)^2} = 0 \leq 2 \Rightarrow A_1 \in N_\epsilon(A_1)$
- $d(A_1, A_2) = \sqrt{(2-1)^2 + (1-1)^2} = \sqrt{1+0} = 1 \leq 2 \Rightarrow A_2 \in N_\epsilon(A_1)$
- $d(A_1, A_3) = \sqrt{(2-1)^2 + (2-1)^2} = \sqrt{1+1} = \sqrt{2} \approx 1.41 \leq 2 \Rightarrow A_3 \in N_\epsilon(A_1)$
- $d(A_1, A_4) = \sqrt{(6-1)^2 + (5-1)^2} = \sqrt{25+16} = \sqrt{41} \approx 6.40 > 2 \Rightarrow A_4 \notin N_\epsilon(A_1)$
- **ফলাফল:** A_1 এর প্রতিবেশ সেট, $N_\epsilon(A_1) = \{A_1, A_2, A_3\}$
- **ঘনত্ব চেক:** $|N_\epsilon(A_1)| = 3$ । যেহেতু $3 \geq \text{min_samples}(3)$, সেহেতু গাণিতিক শর্ত অনুযায়ী:

$A_1 \rightarrow \text{Core Point}$

খ. বিন্দু $A_2(2, 1)$ এর জন্য প্রতিবেশ অনুসন্ধান:

- $d(A_2, A_1) = \sqrt{(1-2)^2 + (1-1)^2} = 1 \leq 2 \Rightarrow A_1 \in N_\epsilon(A_2)$
- $d(A_2, A_2) = \sqrt{(2-2)^2 + (1-1)^2} = 0 \leq 2 \Rightarrow A_2 \in N_\epsilon(A_2)$
- $d(A_2, A_3) = \sqrt{(2-2)^2 + (2-1)^2} = 1 \leq 2 \Rightarrow A_3 \in N_\epsilon(A_2)$
- $d(A_2, A_4) = \sqrt{(6-2)^2 + (5-1)^2} = \sqrt{16+16} = \sqrt{32} \approx 5.66 > 2 \Rightarrow A_4 \notin N_\epsilon(A_2)$
- **ফলাফল:** A_2 এর প্রতিবেশ সেট, $N_\epsilon(A_2) = \{A_1, A_2, A_3\}$
- **ঘনত্ব চেক:** $|N_\epsilon(A_2)| = 3$ | যেহেতু $3 \geq 3$, সেহেতু:

$A_2 \rightarrow$ Core Point

গ. বিন্দু $A_3(2, 2)$ এর জন্য প্রতিবেশ অনুসন্ধান:

- $d(A_3, A_1) = \sqrt{2} \approx 1.41 \leq 2 \Rightarrow A_1 \in N_\epsilon(A_3)$
- $d(A_3, A_2) = 1 \leq 2 \Rightarrow A_2 \in N_\epsilon(A_3)$
- $d(A_3, A_3) = 0 \leq 2 \Rightarrow A_3 \in N_\epsilon(A_3)$
- $d(A_3, A_4) = \sqrt{(6-2)^2 + (5-2)^2} = \sqrt{16+9} = \sqrt{25} = 5 > 2 \Rightarrow A_4 \notin N_\epsilon(A_3)$
- **ফলাফল:** A_3 এর প্রতিবেশ সেট, $N_\epsilon(A_3) = \{A_1, A_2, A_3\}$
- **ঘনত্ব চেক:** $|N_\epsilon(A_3)| = 3$ | যেহেতু $3 \geq 3$, সেহেতু:

$A_3 \rightarrow$ Core Point

ঘ. বিন্দু $A_4(6, 5)$ এর জন্য প্রতিবেশ অনুসন্ধান:

- $d(A_4, A_1) \approx 6.40 > 2 \Rightarrow A_1 \notin N_\epsilon(A_4)$
- $d(A_4, A_2) \approx 5.66 > 2 \Rightarrow A_2 \notin N_\epsilon(A_4)$
- $d(A_4, A_3) = 5 > 2 \Rightarrow A_3 \notin N_\epsilon(A_4)$
- $d(A_4, A_4) = 0 \leq 2 \Rightarrow A_4 \in N_\epsilon(A_4)$
- **ফলাফল:** A_4 এর প্রতিবেশ সেট, $N_\epsilon(A_4) = \{A_4\}$
- **ঘনত্ব চেক:** $|N_\epsilon(A_4)| = 1$ | যেহেতু $1 < 3$, সেহেতু সে Core Point হতে পারল না। আবার, A_4 অন্য কোনো Core Point (A_1, A_2, A_3)-এর প্রতিবেশ সেটের ভেতরেও অবস্থান করে না। সুতরাং, গাণিতিক শর্ত অনুযায়ী:

$A_4 \rightarrow$ Noise Point

৩. ব্যাকএন্ড ওয়ার্কফ্লো এবং ক্লাস্টার গঠন (Cluster Formation Process)

এবার ডিবিএসক্যানের অভ্যন্তরীণ লজিক চেইনের মাধ্যমে চূড়ান্ত আউটপুট তৈরি হবে:

- ইটারেশন ১:** অ্যালগরিদমটি প্রথমে র্যান্ডমলি A_1 বিন্দুটি ভিজিট করে এবং দেখে এটি একটি **Core Point**। তাৎক্ষণিকভাবে একটি নতুন ক্লাস্টার **Cluster 0** তৈরি হয়।
- সম্প্রসারণ (Expansion):** A_1 এর প্রতিবেশীদের (A_2, A_3) একটি অস্থায়ী Queue-তে যুক্ত করা হয়।
- ডেনসিটি চেইন লিঙ্কিং:** * Queue থেকে A_2 বের করা হয়। যেহেতু A_2 নিজেও একটি Core Point, তাই তার প্রতিবেশীদের দিয়ে ক্লাস্টার সম্প্রসারণের চেষ্টা করা হয় (কিন্তু নতুন কোনো বিন্দু পাওয়া যায় না)। A_2 সফলভাবে **Cluster 0**-এ অন্তর্ভুক্ত হয়。
 - এরপর Queue থেকে A_3 বের করা হয়। A_3 নিজেও Core Point প্রমাণিত হওয়ায় সেও **Cluster 0**-এর অংশ হয়ে যায়।
- ক্লাস্টার সমাপ্তি:** Queue খালি হয়ে যায় এবং A_1, A_2, A_3 বিন্দুত্রয় জ্যামিতিকভাবে **Density-Connected** হওয়ায় প্রথম ক্লাস্টারের কাজ সম্পন্ন হয়।
- ইটারেশন ২:** অবশিষ্ট আনভিজিটেড বিন্দু A_4 ভিজিট করা হয়। যেহেতু এটি কোনো কোর বা বর্ডার বিন্দু নয়, তাই এটিকে চূড়ান্তভাবে **Noise** লেবেল দেওয়া হয়।

বিন্দু আইডি (Point)	স্থানাঙ্ক (Coordinates)	বিন্দুর ধরণ (Type)	চূড়ান্ত ক্লাস্টার লেবেল (labels_)
A_1	(1,1)	Core Point	0 (First Cluster)
A_2	(2,1)	Core Point	0 (First Cluster)
A_3	(2,2)	Core Point	0 (First Cluster)
A_4	(6,5)	Noise Point	-1 (Noise / Outlier)

(Hyperparameter Tuning)

sklearn.cluster.DBSCAN ক্লাসে মডেলের ক্লাস্টারিং পারফরম্যান্স, গতি এবং মেমোরি অপ্টিমাইজেশন নিয়ন্ত্রণ করার জন্য বেশ কয়েকটি গুরুত্বপূর্ণ হাইপারপ্যারামিটার রয়েছে। নিচে সবচেয়ে গুরুত্বপূর্ণ (Most Important) থেকে কম গুরুত্বপূর্ণ (Less Important) ক্রমানুসারে প্রতিটি প্যারামিটার বইয়ের ফরম্যাটে বিস্তারিত আলোচনা করা হলো:

১. eps (Epsilon)

- **Default:** 0.5
- **প্যারামিটারটি কী কাজ করে:** এটি হলো দুটি উপাত্ত বিন্দুর মধ্যকার প্রতিবেশ সনাক্ত করার সর্বোচ্চ জ্যামিতিক ব্যাসার্ধ বা দূরত্ব। কোনো একটি বিন্দু থেকে অন্য বিন্দুর দূরত্ব যদি এই eps-এর চেয়ে কম বা সমান হয়, তবেই তারা একে অপরের প্রতিবেশী (Neighbor) হিসেবে গণ্য হবে। এটি ডিবিএসক্যান অ্যালগরিদমের সবচেয়ে সংবেদনশীল এবং গুরুত্বপূর্ণ প্যারামিটার, যা ক্লাস্টারের সংখ্যা এবং আকৃতি নির্ধারণ করে। এর মান সাধারণত যত ছোট নেওয়া হয়, ক্লাস্টারের সংখ্যা তত বৃদ্ধি পায়।
- **অন্যান্য অপশন ও ব্যবহার:** এখানে যেকোনো ধনাত্মক দশমিক সংখ্যা বা ফ্লোট (Float) মান ইনপুট দেওয়া যায়।
 - ডেটা পয়েন্টগুলো যদি খুব ঘন বা গাদাগাদি অবস্থায় থাকে, তবে সূক্ষ্ম ক্লাস্টার খোঁজার জন্য eps-এর মান কমিয়ে দেওয়া উচিত।
 - আর ডেটা যদি ছড়ানো-ছিটানো (Sparse) হয়, তবে বড় ডেনসিটি চেইন তৈরি করতে এর মান বাড়িয়ে দেওয়া প্রয়োজন। সঠিক মান খুঁজে পেতে অনেক সময় K-Distance Graph বা Elbow Method-এর সাহায্য নেওয়া হয়।

২. min_samples

- **Default:** 5
- **প্যারামিটারটি কী কাজ করে:** এটি নির্ধারণ করে যে একটি নির্দিষ্ট বিন্দুর ϵ ব্যাসার্ধের ভেতর নিজেসহ নূন্যতম কতটি উপাত্ত বিন্দু (অথবা টোটাল ওয়েট) উপস্থিত থাকলে সেই বিন্দুটিকে একটি **Core Point** হিসেবে ঘোষণা করা হবে। এর মান যত বেশি বা হাই সেট করা হবে, ডিবিএসক্যান তত বেশি ঘন (Denser) ক্লাস্টার খুঁজে বের করবে। আর এর মান কম হলে তুলনামূলক ছড়ানো বা কম ঘনত্বের (Sparse) ক্লাস্টারও সনাক্ত করা সম্ভব হয়।
- **অন্যান্য অপশন ও ব্যবহার:** এটি যেকোনো ধনাত্মক পূর্ণসংখ্যা (Integer) গ্রহণ করে।

- ডেটাসেটে যদি নয়েজ বা আউটলাইয়ারের পরিমাণ অনেক বেশি থাকে, তবে নয়েজ পয়েন্টগুলো যেন ভুল করে নিজেদের ক্লাস্টার বানাতে না পারে, তা আটকাতে min_samples-এর মান বাড়িয়ে দেওয়া (যেমন: 10 বা তার বেশি) উচিত।
- সাধারণ নিয়ম অনুযায়ী (Rule of Thumb), এর মান নূন্যতম ফিচারের সংখ্যার চেয়ে বেশি বা দ্বিগুণ ($\text{MinPts} \geq \text{Features} + 1$) রাখা স্ট্যান্ডার্ড।

৩. metric

- **Default:** 'euclidean'
- **প্যারামিটারটি কী কাজ করে:** ফিচার অ্যারের উপাত্ত বিন্দুগুলোর মধ্যকার পারস্পরিক জ্যামিতিক দূরত্ব পরিমাপ করার জন্য ব্যাকএন্ডে কোন গাণিতিক সূত্র ব্যবহার করা হবে, তা এই প্যারামিটারটি ঠিক করে।
- **অন্যান্য অপশন ও ব্যবহার:** এটি স্ট্রিং (String) হিসেবে অনেকগুলো অপশন সাপোর্ট করে (যেমন: 'manhattan', 'cosine', 'minkowski' ইত্যাদি) অথবা কোনো কাস্টম ফাংশনও গ্রহণ করতে পারে।
 - 'euclidean': সাধারণ এবং কন্টিনিউয়াস নিউমেরিক্যাল ডেটার জন্য এটি ডিফল্ট এবং সেরা চয়েস।
 - 'manhattan': হাই-ডাইমেনশনাল ডেটা কিংবা গ্রিড-ভিত্তিক পথের দূরত্ব মাপার জন্য এটি বেশ কার্যকর।
 - 'precomputed': আপনার কাছে যদি আগে থেকেই একটি স্কয়ার ডিস্টেন্স ম্যাট্রিক্স (Distance Matrix) বা স্পার্স গ্রাফ তৈরি করা থাকে, তখন এক্সপ্রেসন হিসেবে ইনপুট ম্যাট্রিক্স পাস করার জন্য এই অপশনটি ব্যবহার করতে হবে।

৪. p

- **Default:** None (যা গাণিতিকভাবে $p=2$ বা Euclidean distance-এর সমতুল্য)
- **প্যারামিটারটি কী কাজ করে:** এটি শুধুমাত্র তখনই কার্যকর হয় যখন `metric='minkowski'` ব্যবহার করা হয়। এটি মিনকোভস্কি মেট্রিকের পাওয়ার প্যারামিটার (Power Parameter) হিসেবে দূরত্বের ঘাত নির্ধারণ করে।
- **অন্যান্য অপশন ও ব্যবহার:** এখানে যেকোনো পূর্ণসংখ্যা বা দশমিক সংখ্যা ইনপুট দেওয়া যায়।

- $p=1$: এটি দূরত্বের হিসাবকে সরাসরি ম্যানহাটন দূরত্ব (Manhattan Distance) রূপান্তর করে ফেলে।
- $p=2$: এটি ইউক্লিডিয়ান দূরত্ব (Euclidean Distance) হিসেবে কাজ করে।
- যদি ফিচারের বৈশিষ্ট্য অনুযায়ী কাস্টম কোনো ঘাতের দূরত্ব পরিমাপ করতে হয়, কেবল তখনই এই মান পরিবর্তন করার প্রয়োজন পড়ে।

৫. algorithm

- **Default:** 'auto'
- **প্যারামিটারটি কী কাজ করে:** প্রতিটি বিন্দুর জন্য তার নিকটবর্তী প্রতিবেশী বা Nearest Neighbors খুঁজে বের করার সময় ব্যাকএন্ডে কোন সার্চ অ্যালগরিদম আর্কিটেকচার ব্যবহার করা হবে, তা এটি নির্ধারণ করে। এটি ক্লাস্টারিংয়ের এক্যুরেসিতে কোনো পরিবর্তন আনে না, শুধুমাত্র প্রসেসিং স্পিড নিয়ন্ত্রণ করে।
- **অন্যান্য অপশন ও ব্যবহার:** এর অপশনগুলো হলো: 'ball_tree', 'kd_tree', এবং 'brute'।
 - 'auto': সায়েন্টিফিক লাইব্রেরিটি .fit() মেথডে পাস করা ডেটার সাইজ এবং ডাইমেনশন দেখে নিজেই সবচেয়ে দ্রুততম ও সেরা অ্যালগরিদমটি স্বয়ংক্রিয়ভাবে বেছে নেয়।
 - 'kd_tree': কম ডাইমেনশন বা কম ফিচার বিশিষ্ট বড় ডেটাসেটে দ্রুত কোয়েরি করার জন্য উপযোগী।
 - 'ball_tree': হাই-ডাইমেনশনাল ডেটাসেটে যেখানে কেডি-ট্রি দুর্বল পারফর্ম করে, সেখানে এটি ভালো গতি দেয়।
 - 'brute': একদম ছোট ডেটাসেটের জন্য ব্রুট-ফোর্স (সব বিন্দুর সাথে সবার দূরত্ব সরাসরি হিসাব করা) পদ্ধতিটি ব্যবহার করা যেতে পারে।

৬. n_jobs

- **Default:** None (যার অর্থ কম্পিউটারের মাত্র ১টি প্রসেসর কোর প্রসেসিংয়ে অংশ নেবে)
- **প্যারামিটারটি কী কাজ করে:** Nearest Neighbors কোয়েরি এবং দূরত্ব গণনার বড় ম্যাথমেটিক্যাল ক্যালকুলেশনগুলো সম্পন্ন করার সময় সিপিইউ (CPU)-এর কতগুলো কোর সমান্তরালভাবে বা প্যারালালি (Parallel Jobs) কাজ করবে, তা এটি নিয়ন্ত্রণ করে।
- **অন্যান্য অপশন ও ব্যবহার:** এখানে নির্দিষ্ট প্রসেসর সংখ্যা (পূর্ণসংখ্যা) দেওয়া যায়।

- বড় ডেটাসেটে কম্পিউটেশন স্পিড বহুগুণ বৃদ্ধি করার জন্য এই প্যারামিটারের মান -1 ব্যবহার করা উচিত। `n_jobs=-1` দিলে সিস্টেমের সমস্ত অ্যাভেইলেবল সিপিইউ কোর একসাথে প্যারালাল প্রসেসিং শুরু করে, যা মডেল ট্রেনিংয়ের সময় বহুগুণ বাঁচিয়ে দেয়।

৭. leaf_size

- **Default:** 30
- **প্যারামিটারটি কী কাজ করে:** এটি শুধুমাত্র তখনই প্রভাব ফেলে যখন আগের algorithm প্যারামিটারে 'ball_tree' অথবা 'kd_tree' সক্রিয় থাকে। এটি ইন্টারনাল ট্রির লিফ নোডের সাইজ নির্ধারণ করে দেয়।
- **অন্যান্য অপশন ও ব্যবহার:** যেকোনো পূর্ণসংখ্যা গ্রহণ করে।
 - এই মানটি পরিবর্তনের মাধ্যমে ট্রি কনস্ট্রাকশন স্পিড, কোয়েরি স্পিড এবং মেমোরি স্টোরেজ কস্ট অপ্টিমাইজ করা যায়। সাধারণ মেশিন লার্নিং ওয়ার্কফ্লোতে এর ডিফল্ট মান ৩০-ই সবচেয়ে সুষম পারফরম্যান্স দেয়, তাই অ্যাডভান্সড অপ্টিমাইজেশন ছাড়া এটি পরিবর্তন করার প্রয়োজন হয় না।

৮. metric_params

- **Default:** None
- **প্যারামিটারটি কী কাজ করে:** দূরত্বের জন্য ব্যবহৃত কাস্টম বা জটিল কোনো ডিস্টেন্স ফাংশনে যদি অতিরিক্ত কোনো কি-ওয়ার্ড আর্গুমেন্ট (Keyword Arguments) বা প্যারামিটার পাস করার প্রয়োজন হয়, তবে সেটিকে ডিকশনারি (Dictionary) আকারে পাঠানোর জন্য এটি ব্যবহৃত হয়।
- **অন্যান্য অপশন ও ব্যবহার:** এটি একটি পাইথন ডিকশনারি {} গ্রহণ করে। সাধারণ স্ট্যান্ডার্ড মেট্রিক ব্যবহারের সময় এটি সম্পূর্ণ অব্যবহৃত থাকে; শুধুমাত্র অ্যাডভান্সড কাস্টম ডিস্টেন্স ম্যাপিংয়ের সময় এর ব্যবহার দেখা যায়।

(Attributes & Methods)

sklearn.cluster.DBSCAN মডেলটি .fit() করার পর, অ্যালগরিদমটি তার অভ্যন্তরীণ লার্নিং এবং ক্লাস্টার বিন্যাসের সমস্ত তথ্য বেশ কিছু গুরুত্বপূর্ণ অ্যাট্রিবিউটের (Attributes) মধ্যে সংরক্ষণ করে। নিচে গুরুত্বপূর্ণ অ্যাট্রিবিউটসমূহ এবং মডেলের কোর মেথডগুলোর (Methods) কার্যকারিতা বইয়ের ফরম্যাটে বিস্তারিত আলোচনা করা হলো:

মডেল অ্যাট্রিবিউটসমূহ (DBSCAN Attributes)

মডেল ফিট হওয়ার পর ব্যাকএন্ডের লার্নিং ডেটা অ্যানালাইসিস করার জন্য এই অ্যাট্রিবিউটগুলো ব্যবহার করা হয়। এদের প্রতিটির শেষে একটি আন্ডারস্কোর () থাকে।

labels_

- **Default:** Created after fit()
- **অ্যাট্রিবিউটটি কী কাজ করে:** এটি হলো সবচেয়ে গুরুত্বপূর্ণ অ্যাট্রিবিউট, যা ইনপুট করা মূল ডেটাসেটের প্রতিটি উপাত্ত বিন্দুর জন্য নির্ধারিত ক্লাস্টার আইডির একটি অ্যারে (Array) প্রদান করে। ডিবিএসক্যান যে বিন্দুগুলোকে সফলভাবে ক্লাস্টারে গ্রুপ করতে পারে, সেগুলোকে অ-ঋণাত্মক পূর্ণসংখ্যা বা নন-নেগেটিভ ইন্টিজার (যেমন: 0, 1, 2 ইত্যাদি) লেবেল দেয়। আর যে বিন্দুগুলো কম ঘনত্বের কারণে কোনো ক্লাস্টারের অংশ হতে পারে না, সেগুলোকে সরাসরি -1 লেবেল দিয়ে নয়েজ বা আউটলায়ার (Noise/Outliers) হিসেবে চিহ্নিত করে।

core_sample_indices_

- **Default:** Created after fit()
- **অ্যাট্রিবিউটটি কী কাজ করে:** এটি ডেটাসেটে খুঁজে পাওয়া সমস্ত **Core Points**-এর সূচক বা ইণ্ডেক্স নম্বরগুলোর (Indices) একটি অ্যারে ধারণ করে। এর মাধ্যমে জানা যায় মূল ডেটাসেটের ঠিক কত নম্বর লাইনের সারিগুলো (Rows) কোর স্যাম্পল হিসেবে প্রমাণিত হয়েছে।

components_

- **Default:** Created after fit()
- **অ্যাট্রিবিউটটি কী কাজ করে:** এটি ট্রেনিংয়ের সময় খুঁজে পাওয়া প্রতিটি কোর স্যাম্পলের (Core Samples) মূল ফিচারের মানগুলোর একটি বাস্তব কপি মেমোরিতে ধরে রাখে। এর শেপ বা আকৃতি হয় (n_core_samples, n_features)।

n_features_in_

- **Default:** Created after fit()
- **অ্যাট্রিবিউটটি কী কাজ করে:** মডেলটি ফিট করার সময় ইনপুট ম্যাট্রিক্সে (X) মোট কতটি ফিচার বা কলাম দেখা হয়েছিল, তার সুনির্দিষ্ট সংখ্যা এটি সংরক্ষণ করে।

feature_names_in_

- **Default:** Created after fit() (Only for DataFrame input)
- **অ্যাট্রিবিউটটি কী কাজ করে:** ট্রেনিংয়ের সময় ইনপুট ডেটা হিসেবে যদি কলামের নামযুক্ত প্যান্ডাস ডেটাফ্রেম (Pandas DataFrame) ব্যবহার করা হয়, তবে সব কলামের অফিশিয়াল নামের একটি অ্যারে এটি সংরক্ষণ করে।

কোর মেথডসমূহ (DBSCAN Methods)

ডিবিএসক্যান অবজেক্টের ওপর অপারেশন চালানো এবং পাইপলাইনে ডেটা প্রসেস করার জন্য নিচের মেথডগুলো ব্যবহৃত হয়।

fit(X, y=None, sample_weight=None)

- **মেথডটি কী কাজ করে:** এটি ইনপুট ফিচার অ্যারে বা ডিস্টেন্স ম্যাট্রিক্স ব্যবহার করে ডিবিএসক্যানের মূল ক্লাস্টারিং প্রক্রিয়া সম্পন্ন করে। এটি ডেটার ঘনত্ব পরিমাপ করে কোর, বর্ডার ও নয়েজ পয়েন্ট আলাদা করে ব্যাকএন্ড মেকানিজম সম্পন্ন করে।
- **অন্যান্য প্যারামিটার লজিক:** * y: এটি এখানে সম্পূর্ণ ইগনোর করা হয় (Ignored)। শুধুমাত্র Scikit-Learn-এর অফিশিয়াল এপিআই ডিজাইন এবং কনভেনশন বজায় রাখার জন্য প্যারামিটারটি উপস্থিত থাকে।
 - sample_weight: এটি প্রতিটি ডেটা স্যাম্পলের জন্য কাস্টম ওয়েট বা গুরুত্ব নির্ধারণ করতে ব্যবহৃত হয়। কোনো স্যাম্পলের ওয়েট যদি min_samples-এর চেয়ে বেশি হয়, তবে সে একাই একটি কোর স্যাম্পল হতে পারে। অন্যদিকে, কোনো স্যাম্পলের ওয়েট ঋণাত্মক হলে তা তার প্রতিবেশীদের কোর স্যাম্পল হতে বাধা দিতে পারে। এর ডিফল্ট মান থাকে 1।

fit_predict(X, y=None, sample_weight=None)

- **মেথডটি কী কাজ করে:** এটি অত্যন্ত জনপ্রিয় একটি শর্টকাট মেথড। এটি এক লাইনের কোডেই মডেলকে ডেটার ওপর ফিট করে এবং একই সাথে প্রতিটি বিন্দুর জন্য নির্ধারিত ক্লাস্টার লেবেলের অ্যারেটি সরাসরি রিটার্ন (Return) করে।

- **ব্যবহারের প্রসঙ্গ:** কোডিংয়ের সময় আলাদা করে `.fit(X)` এবং পরবর্তীতে `.labels_` কল না করে সরাসরি লেবেল অ্যারেটি মেমোরিতে নিয়ে আসার জন্য এটি ব্যবহার করা হয়। এটি মূলত `fit(X).labels_`-এর সমতুল্য।

get_params(deep=True)

- **মেথডটি কী কাজ করে:** এস্টিমেটরের বর্তমান কনফিগারেশনে কোন কোন হাইপারপ্যারামিটার (যেমন: `eps`, `min_samples` ইত্যাদি) কী মান সেট করা আছে, তার একটি সুনির্দিষ্ট ম্যাপিং ডিকশনারি রিটার্ন করে। এটি গ্রিড সার্চ বা মডেল ট্র্যাকিংয়ের সময় বেশ উপযোগী।

set_params(params)

- **মেথডটি কী কাজ করে:** এটি একটি কাস্টম মেথড যার সাহায্যে অবজেক্ট তৈরি করার পরেও যেকোনো হাইপারপ্যারামিটারের মান সরাসরি ওভাররাইট বা পরিবর্তন করা যায়। বিশেষ করে জটিল Pipeline-এর ভেতর কোনো নির্দিষ্ট উপাদানের প্যারামিটার আপডেট করতে এটি কাজ করে।

Code:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler
from sklearn.datasets import make_moons
from sklearn.cluster import DBSCAN
from sklearn.cluster import KMeans

X, y = make_moons(n_samples=300, noise=0.06, random_state=42)

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

dbscan = DBSCAN(eps=0.3, min_samples=3)
dbscan.fit(X_scaled)
label_dbscan = dbscan.labels_

sns.scatterplot(x=X_scaled[:, 0], y=X_scaled[:, 1], hue=label_dbscan, palette='viridis')
plt.title('DBSCAN Clustering Results')
plt.xlabel('Feature 1 (Scaled)')
plt.ylabel('Feature 2 (Scaled)')
plt.legend(title='Cluster')
plt.show()

kmeans = KMeans(n_clusters=2, init="k-means++", max_iter=300, random_state=42)
kmeans.fit(X_scaled)
label_kmeans = kmeans.labels_

sns.scatterplot(x=X_scaled[:, 0], y=X_scaled[:, 1], hue=label_kmeans, palette='viridis')
plt.title('K-Means Clustering Results')
plt.xlabel('Feature 1 (Scaled)')
plt.ylabel('Feature 2 (Scaled)')
plt.legend(title='Cluster')
plt.show()
```

প্রশ্ন ১: কে-মিন্স (K-Means)-এর তুলনায় জটিল ও অরৈখিক (Non-linear) আকৃতির উপাত্ত শ্রেণীবিভাগে ডিবিএসক্যান (DBSCAN) কেন অনন্য কার্যকারিতা প্রদর্শন করে?

- **উত্তর:** K-Means অ্যালগরিদমটি সেন্ট্রয়েড-ভিত্তিক হওয়ার কারণে এটি ধরে নেয় ক্লাস্টারগুলো সবসময় বৃত্তাকার বা গোলকাকার (Spherical). ফলে আঁকাবাঁকা বা চাঁদের মতো (make_moons) জটিল ডেটার বিন্যাস থাকলে K-Means ক্লাসগুলোকে মাঝখান থেকে সোজা লাইনে কেটে ফেলে এবং ভুল ক্লাস্টারিং করে.
- বিপরীতে, DBSCAN কোনো নির্দিষ্ট কেন্দ্রবিন্দু খোঁজার চেষ্টা করে না; এটি সম্পূর্ণভাবে ডেটার উচ্চ-ঘনত্ব বিশিষ্ট এলাকা অনুসরণ করে অগ্রসর হয়. চাঁদের মতো বাঁকা লাইনের প্রতিটি বিন্দু পরস্পরের খুব কাছাকাছি থাকায় তারা একটি অবিচ্ছিন্ন ডেনসিটি শিকল বা সেতু তৈরি করে. এই ঘনত্ব-সংযুক্ততার (Density-Connectedness) কারণে জ্যামিতিক আকৃতি যত জটিল বা আঁকাবাঁকাই হোক না কেন, DBSCAN নিখুঁতভাবে আসল ক্লাস্টার সনাক্ত করতে পারে.

প্রশ্ন ২: ডিবিএসক্যান (DBSCAN) অ্যালগরিদমে Epsilon (ϵ) প্যারামিটারটি খুব ছোট বা খুব বড় সেট করলে মডেলের আউটপুটে কী প্রভাব পড়ে?

- **উত্তর:** eps হলো একটি নির্দিষ্ট ডেটা পয়েন্ট থেকে তার প্রতিবেশীদের খুঁজে বের করার সর্বোচ্চ জ্যামিতিক ব্যাসার্ধ বা খোঁজাখুঁজির এলাকা.
 - ϵ **অত্যধিক ছোট হলে (Too Small):** খোঁজাখুঁজির ব্যাসার্ধ ছোট হওয়ার কারণে ডেটা পয়েন্টগুলো একে অপরের খুব কাছে থাকা সত্ত্বেও প্রয়োজনীয় ঘনত্ব (MinPts) পূর্ণ করতে পারে না. এর ফলে অনেক আসল ক্লাস্টার তৈরি হতে ব্যর্থ হয় এবং ডেটার একটি বিশাল অংশ **Noise (Label: -1)** হিসেবে বাদ পড়ে যায় (Underfitting/High Bias).
 - ϵ **অত্যধিক বড় হলে (Too Large):** এলাকা অনেক বড় হয়ে যাওয়ার কারণে সম্পূর্ণ আলাদা এবং ভিন্ন ভিন্ন দুটি ক্লাস্টারও একটি কমন ব্যাসার্ধের ভেতর চলে আসে. এর ফলে একাধিক ক্লাস্টার জোড়া লেগে একটি একক বৃহৎ ক্লাস্টারে পরিণত হয় (Overfitting/High Variance).

প্রশ্ন ৩: ডিবিএসক্যান (DBSCAN)-এ একটি সীমান্ত বিন্দু (Border Point) এবং নয়েজ বিন্দু (Noise Point)-এর মধ্যকার গাণিতিক ও কাঠামোগত পার্থক্য কী?

- **উত্তর:** 1. **Border Point (সীমান্ত বিন্দু):** এই বিন্দুর নিজস্ব ϵ ব্যাসার্ধের বৃত্তের ভেতর প্রয়োজনীয় MinPts-এর চেয়ে কম সংখ্যক পয়েন্ট থাকে (তাই সে নিজে Core Point হতে পারে না). কিন্তু, পয়েন্টটি যদি অন্য কোনো একটি Core Point-এর এলাকার সীমানার ভেতরে অবস্থান করে, তবে তাকে **Border Point** বলে. এরা ক্লাস্টারের একদম বাইরের সীমানা বা ডালপালা তৈরি করে.

2. **Noise Point (নয়েজ বিন্দু):** এই বিন্দুর নিজস্ব ব্যাসার্ধের ভেতর $MinPts$ -এর চেয়ে কম পয়েন্ট থাকে এবং এটি আশেপাশের কোনো Core Point-এর এলাকার ভেতরেও পড়ে না (অর্থাৎ সে সম্পূর্ণ বিচ্ছিন্ন). DBSCAN এদেরকে কোনো ক্লাস্টারে জায়গা না দিয়ে সরাসরি -1 লেবেল দিয়ে আউটলায়ার হিসেবে মেমোরি থেকে আলাদা করে দেয়.

প্রশ্ন ৪: ডিবিএসক্যানে ঘনত্ব-সংযুক্ত বিন্দু (Density-Connected Points) বলতে কী বোঝায় এবং এটি ক্লাস্টার গঠনে কীভাবে ভূমিকা রাখে?

- **উত্তর:** যদি উপাত্ত ক্ষেত্রের দুটি বিন্দু A এবং F একে অপরের সাথে সরাসরি কানেক্টেড না থাকে, এমনকি তারা যদি পরস্পর থেকে অনেক দূরে অবস্থান করে, তবুও তারা দুজনেই যদি অন্য কোনো একটি সাধারণ মূল বিন্দু চেইনের (Common Core Point Chain) সাথে যুক্ত বা Density-Reachable থাকে, তবে গাণিতিকভাবে A এবং F -কে **Density-Connected** বলা হয়.
- **ভূমিকা:** ডিবিএসক্যানের মূল নিয়মই হলো—যে পয়েন্টগুলো একে অপরের সাথে ঘনত্ব-সংযুক্ত অবস্থায় থাকে, অ্যালগরিদমটি তাদের সবাইকে একটি একক ক্লাস্টারের (Same Cluster) অন্তর্ভুক্ত করে. চেইন রিয়াকশনের মতো এক কোর বিন্দু অন্য কোর বিন্দুর হাত ধরে পুরো সেতুটি লক করে ফেলে, যার ফলে দূরবর্তী বিন্দুগুলোও একই গ্রুপের অংশ হিসেবে আত্মপ্রকাশ করে.

প্রশ্ন ৫: ভিন্ন ভিন্ন বা পরিবর্তনশীল ঘনত্ব (Variable/Multi-Density) যুক্ত ডেটাসেটে ডিবিএসক্যান (DBSCAN) কেন দুর্বল পারফর্ম করে এবং এর সমাধান কী?

- **উত্তর:** DBSCAN-এর একটি বড় সীমাবদ্ধতা হলো, এটি সম্পূর্ণ ডেটাসেটের ওপর একটি **Global ϵ** এবং **Global $MinPts$** ব্যবহার করে কাজ করে. অর্থাৎ, পুরো উপাত্ত ক্ষেত্রের সব জায়গায় ঘনত্বের থ্রেশহোল্ড সমান থাকে.
- **দুর্বলতার কারণ:** ডেটাসেটে যদি Variable Density (কোথাও খুব ঘন এবং কোথাও খুব ছড়ানো ডেটা) থাকে, তবে ছড়ানো বা কম ঘনত্বের (Sparse Density) এলাকাগুলোকে ধরার জন্য ϵ -এর মান বড় করলে ঘন এলাকার একাধিক আলাদা ক্লাস্টার একসাথে ব্লেন্ড বা মার্জ (Merge) হয়ে যায়. আবার ঘন এলাকাকে ধরার জন্য ϵ ছোট করলে, ছড়ানো এলাকার ডেটাগুলো কোনো ক্লাস্টার না পেয়ে সম্পূর্ণ **Noise** হিসেবে বাদ পড়ে যায়.
- **সমাধান:** ডোমেইন নলেজের ভিত্তিতে ডেটাতে এমন Variable Density দেখা গেলে সাধারণ DBSCAN ব্যবহার না করে এর উন্নত সংস্করণ **HDBSCAN (Hierarchical DBSCAN)** অথবা **OPTICS** অ্যালগরিদম ব্যবহার করা উচিত, যা স্বয়ংক্রিয়ভাবে এলাকার ঘনত্ব অনুযায়ী পরিবর্তনশীল ব্যাসার্ধ হিসাব করতে পারে.

ডিবিএসক্যান ক্লাস্টারিং সারসংক্ষেপ

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) হলো একটি শক্তিশালী আনসুপারভাইজড মেশিন লার্নিং অ্যালগরিদম, যা উপাত্তের জ্যামিতিক ঘনত্ব (Density) বিশ্লেষণ করে স্বয়ংক্রিয়ভাবে ক্লাস্টার তৈরি করে।

১. মূল প্যারামিটারসমূহ (Core Parameters)

- **Epsilon (ϵ):** প্রতিবেশীদের খুঁজে বের করার সর্বোচ্চ জ্যামিতিক ব্যাসার্ধ বা দূরত্ব।
- **MinSamples (MinPts):** ক্লাস্টার গঠনের জন্য ওই ব্যাসার্ধের ভেতর নিজেকেসহ নূন্যতম প্রয়োজনীয় বিন্দুর সংখ্যা।

২. বিন্দুর শ্রেণীবিভাগ (Data Point Classifications)

DBSCAN প্যারামিটারের ওপর ভিত্তি করে পুরো ডেটাসেটকে ৩টি সুনির্দিষ্ট বিন্দুতে ভাগ করে:

- **Core Points:** ব্যাসার্ধের ভেতর *MinPts* বা তার বেশি প্রতিবেশী থাকা ঘন বিন্দু।
- **Border Points:** নিজে Core Point নয়, কিন্তু কোনো Core Point-এর এলাকার ভেতরে থাকা সীমান্ত বিন্দু।
- **Noise Points:** ব্যাসার্ধের ভেতর প্রয়োজনীয় ঘনত্ব না পাওয়া এবং সম্পূর্ণ বিচ্ছিন্ন আউটলায়ার বিন্দু (Label: -1)।

৩. ক্লাস্টার গঠনের নীতি (Density-Connected Rules)

- **Directly Density-Reachable:** একটি বিন্দু অন্য একটি Core Point-এর সীমানায় সরাসরি অবস্থান করলে।
- **Density-Reachable:** কোর বিন্দুগুলোর একটি অবিচ্ছিন্ন ধারাবাহিক শিকল বা চেইনের মাধ্যমে যুক্ত থাকলে।
- **Density-Connected:** দুটি দূরবর্তী বিন্দু সরাসরি যুক্ত না থাকলেও যদি একই কমন কোর বিন্দু চেইনের সাথে যুক্ত থাকে। **গাণিতিক নিয়ম অনুযায়ী, ঘনত্ব-সংযুক্ত বিন্দুগুলো সবসময় একই ক্লাস্টারের (Same Cluster) অন্তর্ভুক্ত হয়।**

৪. K-Means-এর পরিবর্তে কেন DBSCAN শ্রেষ্ঠ?

- **Arbitrary Shapes:** K-Means যেখানে শুধু বৃত্তাকার ক্লাস্টার তৈরি করতে পারে, DBSCAN সেখানে চাঁদের মতো (make_moons) যেকোনো জটিল বা আঁকাবাঁকা আকৃতির ক্লাস্টার নিখুঁতভাবে সনাক্ত করতে পারে।

- **No Pre-defined K:** K-Means-এর মতো ডিবিএসক্যানে রান করার আগে থেকে ক্লাস্টারের সংখ্যা (K) বলে দিতে হয় না।
- **Robust to Outliers:** এটি আউটলায়ারগুলোকে জোর করে ক্লাস্টারে না ঢুকিয়ে সরাসরি **Noise** হিসেবে আলাদা করে দেয়, ফলে আসল ক্লাস্টারের জ্যামিতিক সীমানা সুরক্ষিত থাকে।

Hierarchical DBSCAN (HDBSCAN)

HDBSCAN-এর পূর্ণরূপ হলো **Hierarchical Density-Based Spatial Clustering of Applications with Noise**। এটি হলো প্রথাগত DBSCAN অ্যালগরিদমের একটি অত্যন্ত শক্তিশালী এবং আধুনিক বর্ধিত রূপ (Enhanced Version)।

আমরা জেনেছি যে, সাধারণ DBSCAN-এর একটি বড় সীমাবদ্ধতা হলো এটি সম্পূর্ণ উপাত্ত ক্ষেত্রের ওপর একটিমাত্র গ্লোবাল ব্যাসার্ধ বা **Global Epsilon (ϵ)** ব্যবহার করে কাজ করে, যার ফলে ভিন্ন ভিন্ন বা পরিবর্তনশীল ঘনত্ব যুক্ত ডেটাসেটে (Variable/Multi-Density Data) এটি ব্যর্থ হয়। HDBSCAN মূলত এই ডেনসিটি বা ঘনত্বের তারতম্যজনিত সমস্যাটি দূর করার জন্যই তৈরি করা হয়েছে।

১. এটি কীভাবে কাজ করে? (How it Works)

HDBSCAN মূলত **Density-Based Clustering**-কে **Hierarchical Clustering**-এর সাথে যুক্ত করে কাজ করে। এর প্রধান মেকানিজমটি নিচে সংক্ষেপে আলোচনা করা হলো:

- **পরিবর্তনশীল ব্যাসার্ধ (Variable Epsilon):** HDBSCAN-এ ইউজারকে কোনো সুনির্দিষ্ট ϵ বা Epsilon মান সেট করে দিতে হয় না। অ্যালগরিদমটি প্রতিটি এলাকার ডেটার বিন্যাস এবং ঠাসাঠাসি অবস্থা দেখে স্বয়ংক্রিয়ভাবে পরিবর্তনশীল ব্যাসার্ধ (Variable ϵ) হিসাব করে নেয়।
- **ডেন্ড্রোগ্রাম বা ট্রি তৈরি (Single Linkage Tree):** এটি প্রথমে ডেটার ঘনত্ব অনুযায়ী একটি ক্রমানুসারী বৃক্ষ বা ডেন্ড্রোগ্রাম (Dendrogram) তৈরি করে, যা দেখায় কীভাবে বিন্দুগুলো ঘনত্বের ওপর ভিত্তি করে ছোট থেকে বড় গ্রুপে জোড়া লাগছে।
- **ক্লাস্টার ঘনীভূতকরণ (Condensing the Tree):** তৈরি হওয়া জটিল ট্রি-টি থেকে এটি ছোট বা অর্থহীন ডালপালাগুলোকে ছেঁটে ফেলে এবং শুধুমাত্র মেমোরিতে টিকে থাকার মতো স্থিতিশীল ও সুনির্দিষ্ট ঘনত্বের ক্লাস্টারগুলোকে চূড়ান্ত আউটপুট হিসেবে ধরে রাখে।

২. কোর প্যারামিটার (Core Parameter)

DBSCAN-এ যেখানে ২টি প্যারামিটার টিউন করতে হতো, HDBSCAN-এ মূলত ১টি প্রধান প্যারামিটারের দিকে ফোকাস করলেই চলে:

- **min_cluster_size:** এটি নির্ধারণ করে যে একটি ক্লাস্টার টিকে থাকার জন্য নূন্যতম কতটি ডেটা পয়েন্টের উপস্থিতি প্রয়োজন। মানটি ছোট হলে অনেক ছোট ছোট ক্লাস্টার তৈরি হবে, আর বড় হলে শুধুমাত্র বড় বড় গ্রুপগুলোই টিকে থাকবে।

৩. সাধারণ DBSCAN বনাম HDBSCAN (Why it is Better)

মূল্যায়নের ক্ষেত্র	সাধারণ DBSCAN	HDBSCAN
Epsilon (ϵ) প্যারামিটার	ম্যানুয়ালি সেট করতে হয় এবং এটি পুরো ডেটায় ফিক্সড থাকে।	কোনো নির্দিষ্ট মানের প্রয়োজন নেই, এটি স্বয়ংক্রিয় ও পরিবর্তনশীল।
ভিন্ন ঘনত্ব ব্যবস্থাপনা	Multi-Density বা ছড়ানো ডেটা (Sparse Data) থাকলে ক্লাস্টার মিস করে।	ডেটার ঘনত্ব যেমনই হোক না কেন, এটি যেকোনো পরিবর্তনশীল ঘনত্ব নিখুঁতভাবে হ্যান্ডেল করে।
টিউনিং জটিলতা	সঠিক eps খুঁজে পাওয়া বেশ কঠিন এবং সময়সাপেক্ষ।	টিউন করা অত্যন্ত সহজ, শুধু min_cluster_size লক করলেই চলে।
আউটলায়ার সনাক্তকরণ	আউটলায়ারকে Noise (-1) হিসেবে আলাদা করতে পারে।	অনেক বেশি নিখুঁতভাবে এবং স্থিতিশীল উপায়ে নয়েজ ফিল্টার করতে পারে।

এইচডিবিএসক্যান (HDBSCAN) অ্যালগরিদমের অভ্যন্তরীণ কার্যপ্রক্রিয়া

HDBSCAN মূলত ৫টি প্রধান গাণিতিক ও জ্যামিতিক ধাপের মাধ্যমে কাজ করে। এটি প্রথাগত স্পেস বা দূরত্বকে একটি বিশেষ "ঘনত্ব-ভিত্তিক দূরত্ব" রূপান্তর করে একটি ক্রমানুসারী বৃক্ষ (Hierarchy Tree) তৈরি করে এবং সেখান থেকে ক্লাস্টার নিষ্কাশন করে।

তাত্ত্বিক প্রক্রিয়াটি নিচে বইয়ের ফরম্যাটে ধাপে ধাপে আলোচনা করা হলো:

ধাপ ১: স্পেস রূপান্তর এবং মিউচুয়াল রিচ্যাভিলিটি ডিস্টেন্স (Transforming the Space)

কম ঘনত্ব বা স্পার্স ডেনসিটি (Sparse Density) এলাকার নয়েজ পয়েন্টগুলো যেন ক্লাস্টার গঠনে ব্যাঘাত ঘটাতে না পারে, সেজন্য HDBSCAN প্রথমে সাধারণ ইউক্লিডিয়ান দূরত্বকে **Mutual Reachability Distance**-এ রূপান্তর করে।

ধরা যাক, আমাদের কাছে দুটি বিন্দু A এবং B আছে। এদের মধ্যকার পরিবর্তিত দূরত্ব $d_{mr}(A, B)$ নির্ণয়ের গাণিতিক সূত্র:

$$d_{mr}(A, B) = \max(\{core_k(A), core_k(B), d(A, B)\})$$

- এখানে $core_k(P)$ হলো P বিন্দু থেকে তার k -তম প্রতিবেশীর প্রকৃত দূরত্ব (যা বিন্দুর নিজস্ব ঘনত্ব নির্দেশ করে)।
- $d(A, B)$ হলো তাদের মধ্যকার সাধারণ ইউক্লিডিয়ান দূরত্ব।
- লজিক:** এই সূত্রের কারণে ঘন এলাকার বিন্দুগুলোর দূরত্ব অপরিবর্তিত থাকে, কিন্তু কম ঘনত্বের ছড়ানো বা নয়েজ বিন্দুগুলোর দূরত্ব গাণিতিকভাবে অনেক বেড়ে যায়। ফলে নয়েজ পয়েন্টগুলো স্বয়ংক্রিয়ভাবে ক্লাস্টার থেকে দূরে ঠেলে দেওয়া হয়।

ধাপ ২: মিনিমাম স্প্যানিং ট্রি গঠন (Graph Construction via Minimum Spanning Tree)

স্পেস রূপান্তরিত হওয়ার পর, ডেটাসেটের প্রতিটি বিন্দুকে এক একটি নোড (Node) এবং তাদের মধ্যকার মিউচুয়াল রিচ্যাভিলিটি ডিস্টেন্সকে এজ (Edge) বা সংযোগকারী রেখা ধরে একটি সম্পূর্ণ গ্রাফ কল্লনা করা হয়।

- এরপর প্রিমস (Prim's) বা ক্রসকলস (Kruskal's) অ্যালগরিদম ব্যবহার করে সেই গ্রাফ থেকে একটি **Minimum Spanning Tree (MST)** তৈরি করা হয়।
- এই ট্রি-টি নিশ্চিত করে যে, সবচেয়ে কম দূরত্ব বা সর্বোচ্চ ঘনত্বের লিঙ্কিংগুলো আগে কানেক্টেড হবে।

ধাপ ৩: ক্রমানুসারী ক্লাস্টার বৃক্ষ তৈরি (Building the Cluster Hierarchy)

মিনিমাম স্প্যানিং ট্রি (MST) তৈরি হওয়ার পর, সেটিকে একটি ক্রমানুসারী বৃক্ষ বা **Dendrogram**-এ রূপান্তর করা হয়।

- অ্যালগরিদমটি দূরত্বের মান আস্তে আস্তে কমাতে থাকে (অথবা ঘনত্বের থ্রেশহোল্ড বাড়াতে থাকে)।
- দূরত্বের বাধন যত শক্ত হতে থাকে, MST-এর এজ বা রেখাগুলো এক এক করে কাটতে শুরু করে। এর ফলে মূল ট্রি-টি ভেঙে ছোট ছোট ডালে বিভক্ত হতে থাকে, যা দেখায় কীভাবে একটি বড় গ্রুপ ভেঙে একাধিক ছোট উপ-ক্লাস্টারে রূপান্তরিত হচ্ছে।

ধাপ ৪: বৃক্ষ ঘনীভূতকরণ এবং অপয়োজনীয় ডাল ছাঁটাই (Condensing the Cluster Tree)

ধাপ ৩-এ তৈরি হওয়া ডেনড্রোগ্রাম বা ট্রি-টি অত্যন্ত জটিল এবং বিশাল আকৃতির হয়, যেখানে মাত্র ১-২টি বিন্দুর বিচ্ছিন্ন হওয়াকেও একেকটি নতুন ডাল বা ক্লাস্টার হিসেবে দেখানো হয়। বাস্তবসম্মত ক্লাস্টার পেতে HDBSCAN এই ট্রি-কে ঘনীভূত (Condense) করে।

- এই ধাপে ইউজারের দেওয়া একমাত্র প্যারামিটার **min_cluster_size** কাজ করে।
- ট্রি-এর কোনো ডাল বা স্প্লিট (Split) হওয়ার সময় যদি দেখা যায় যে, একটি ডালে **min_cluster_size**-এর চেয়ে কম সংখ্যক বিন্দু রয়েছে, তবে অ্যালগরিদমটি সেটিকে নতুন ক্লাস্টার বলে গণ্য করে না। বরং সেটিকে "ক্লাস্টার থেকে বিন্দু ঝরে পড়া" (Points shedding from cluster) হিসেবে বিবেচনা করে ডালটি কেটে ফেলে।
- আর যদি উভয় ডালে বিন্দুর সংখ্যা **min_cluster_size**-এর চেয়ে বেশি থাকে, কেবল তখনই সেটিকে একটি সত্যিকারের ক্লাস্টার বিভাজন বা স্প্লিট হিসেবে মেমোরিতে রাখা হয়। এর ফলে একটি অত্যন্ত পরিষ্কার ও ছোট **Condensed Tree** পাওয়া যায়।

ধাপ ৫: ক্লাস্টার নিষ্কাশন এবং স্থায়িত্ব পরিমাপ (Extracting the Clusters via Stability)

Condensed Tree পাওয়ার পর, চূড়ান্ত ধাপে কোন ডালগুলোকে আমরা আসল ক্লাস্টার হিসেবে বেছে নেব—তা নির্ধারণ করতে **Cluster Stability** পরিমাপ করা হয়।

- গাণিতিকভাবে, প্রতিটি ক্লাস্টারের জন্য $\lambda_{\text{stability}}$ (**Lambda Stability**) হিসাব করা হয়। একটি ক্লাস্টার ঘনত্বের থ্রেশহোল্ডের কত বড় রেঞ্জ জুড়ে নিজেকে টিকিয়ে রাখতে পেরেছে (অর্থাৎ কত দীর্ঘ সময় ধরে সে ভাঙেনি), তা এই স্ট্যাবিলিটি নির্দেশ করে।
- সমীকরণ:

$$\text{Stability}(\text{Cluster}) = \sum_{p \in \text{Cluster}} (\lambda_p - \lambda_{\text{birth}})$$

- **অপ্টিমাইজেশন লজিক:** HDBSCAN নিচ থেকে ওপরের দিকে ট্রি ট্রান্সার্স করে এবং যে ডালগুলোর সম্মিলিত স্ট্যাবিলিটি সবচেয়ে বেশি, সেগুলোকে চূড়ান্ত ক্লাস্টার হিসেবে লুফে নেয়। যদি কোনো প্যারেন্ট ক্লাস্টারের স্থায়িত্ব তার চাইল্ড ক্লাস্টারগুলোর যোগফলের চেয়ে বেশি হয়, তবে চাইল্ডগুলোকে বাতিল করে প্যারেন্টকেই ক্লাস্টার ধরা হয়; অন্যথায় চাইল্ডগুলোকে ক্লাস্টার হিসেবে ঘোষণা করা হয়।

এইচডিবিএসক্যান (HDBSCAN) অ্যালগরিদমের গাণিতিক ভিত্তি

HDBSCAN অ্যালগরিদমের পুরো আর্কিটেকচার এবং এর পরিবর্তনশীল ঘনত্ব (Variable Density) সনাক্ত করার ক্ষমতা মূলত ৩টি কোর গাণিতিক ধারণার ওপর ভিত্তি করে দাঁড়িয়ে আছে। নিচে এদের গাণিতিক সমীকরণ এবং থিওরি বিস্তারিত আলোচনা করা হলো:

১. কোর ডিস্টেন্স (Core Distance)

কোনো একটি উপাত্ত বিন্দু P তার চারপাশের এলাকা কতটা ঘন—তা পরিমাপ করার জন্য **Core Distance** ব্যবহার করা হয়।

- **গাণিতিক সংজ্ঞা:** ইউজার কর্তৃক ইনপুট দেওয়া min_samples বা k প্যারামিটারের ওপর ভিত্তি করে, বিন্দু P থেকে তার k -তম নিকটবর্তী প্রতিবেশীর (Neighbor) প্রকৃত ইউক্লিডিয়ান দূরত্বই হলো ওই বিন্দুর Core Distance।

- **সমীকরণ:**

$$\text{core}_k(P) = d(P, N_k(P))$$

(এখানে $N_k(P)$ হলো P বিন্দুর k -তম প্রতিবেশ এবং d হলো সাধারণ Euclidean Distance সূত্র).

- **লজিক:** বিন্দুটি যদি কোনো অত্যন্ত ঘন বা ঠাসাঠাসি এলাকায় থাকে, তবে তার প্রতিবেশীরা খুব কাছেই অবস্থান করবে, ফলে তার $\text{core}_k(P)$ এর মান হবে খুবই ছোট। আর বিন্দুটি যদি কোনো ছড়ানো-ছিটানো (Sparse Density) বা নয়েজ এলাকায় থাকে, তবে তার k -তম প্রতিবেশীকে খুঁজে পেতে অনেক দূরে যেতে হবে, ফলে তার $\text{core}_k(P)$ এর মান হবে অনেক বড়।

২. মিউচুয়াল রিচ্যাভিলিটি ডিস্টেন্স (Mutual Reachability Distance)

HDBSCAN-এর সবচেয়ে শক্তিশালী জ্যামিতিক হাতিয়ার হলো এই **Mutual Reachability Distance** (d_{mr})। সাধারণ ইউক্লিডিয়ান স্পেসে ছড়ানো ডেটা বা নয়েজ পয়েন্টগুলো যেন ক্লাস্টার

গঠনে বিভ্রান্তি তৈরি করতে না পারে, সেজন্য দূরত্ব পরিমাপের প্রথাগত ট্র্যাকিংকে এই সূত্রের মাধ্যমে পরিবর্তন (Transform) করা হয়।

দুটি বিন্দু A এবং B -এর মধ্যকার পরিবর্তিত ঘনত্ব-ভিত্তিক দূরত্ব নির্ণয়ের সূত্র:

$$d_{mr}(A, B) = \max(\{core_k(A), core_k(B), d(A, B)\})$$

- **সূত্রের কার্যপ্রণালী:** 1. **ক্ষেত্র ১ (উভয় বিন্দু ঘন এলাকায় থাকলে):** যদি A এবং B দুটি বিন্দুই অত্যন্ত ঘন অঞ্চলের ভেতরে থাকে, তবে তাদের নিজস্ব $core_distance$ হবে খুবই ছোট। তখন এই সূত্রের $\max$ মেকানিজম অনুযায়ী তাদের মধ্যকার আসল ইউক্লিডিয়ান দূরত্ব $d(A, B)$ -ই বজায় থাকবে। 2. **ক্ষেত্র ২ (যেকোনো একটি বিন্দু নয়েজ বা স্পার্স এলাকায় থাকলে):** যদি যেকোনো একটি বিন্দু (ধরা যাক B) ক্লাস্টার থেকে দূরে কোনো ছড়ানো এলাকায় একা বসে থাকে, তবে তার $core_distance$ বা $core_k(B)$ এর মান হবে বিশাল বড়। তখন সূত্রের $\max$ ফিল্টারটি আসল দূরত্বকে ইগনোর করে সরাসরি সেই বড় $core_k(B)$ মানটিকে আউটপুট হিসেবে দেবে।
- **গাণিতিক ইনসাইট:** এই স্পেস ট্রান্সফর্মেশনের কারণে ছড়ানো এলাকার বা নয়েজ পয়েন্টগুলোর দূরত্ব গাণিতিকভাবে বহুগুণ বেড়ে যায়, যা তাদেরকে মূল ক্লাস্টারের ঘনত্বের চেইন থেকে স্বয়ংক্রিয়ভাবে দূরে ঠেলে দেয়।

৩. ক্লাস্টার স্থায়িত্ব পরিমাপ (Cluster Stability - λ)

ডেনড্রোগ্রাম (Dendrogram) বা ক্রমানুসারী বৃক্ষ থেকে অপ্রয়োজনীয় ডালপালা ছেঁটে ফেলার পর, কোন ডালগুলোকে আমরা চূড়ান্ত এবং আসল ক্লাস্টার হিসেবে মেমোরিতে লক করব, তা নির্ধারণ করতে **Cluster Stability** হিসাব করা হয়।

এখানে দূরত্ব বা ডিস্টেন্স (d)-এর বিপরীত রূপ হিসেবে **ঘনত্ব বা ল্যাম্বডা (λ)** ব্যবহার করা হয়:

$$\lambda = \frac{1}{d}$$

- **গাণিতিক মেকানিজম:** * λ_{birth} : যে ঘনত্বে বা ল্যাম্বডা মানে এসে একটি ডাল বা ক্লাস্টার প্রথম জন্ম নেয় (প্যারেন্ট ক্লাস্টার থেকে ভেঙে আলাদা হয়)।
 - λ_{death} : যে ঘনত্বে এসে ওই ক্লাস্টারটি আবার ভেঙে দুটি ছোট উপ-ক্লাস্টারে বিভক্ত হয়ে যায়।
 - λ_p : ক্লাস্টারের ভেতরে থাকা কোনো একটি নির্দিষ্ট বিন্দু p ঠিক কোন ঘনত্বে এসে ক্লাস্টার থেকে ছিটকে বা ঝরে পড়ে (Shedding)।

এই উপাদানগুলোর ওপর ভিত্তি করে একটি নির্দিষ্ট ক্লাস্টারের চূড়ান্ত স্থায়িত্ব বা **Stability** পরিমাপের সমীকরণ:

$$\text{Stability}(\text{Cluster}) = \sum_{p \in \text{Cluster}} (\lambda_p - \lambda_{\text{birth}})$$

- **অপ্টিমাইজেশন লজিক (Excess of Mass):** HDBSCAN প্রতিটি ডালের জন্য এই স্ট্যাবিলিটি মানটি যোগ করে। যদি কোনো প্যারেন্ট ক্লাস্টারের নিজস্ব স্থায়িত্বের মান, তার থেকে জন্ম নেওয়া চাইল্ড ক্লাস্টারগুলোর সম্মিলিত যোগফলের চেয়ে বেশি হয়:

$$\text{Stability}(\text{Parent}) > \text{Stability}(\text{Child}_1) + \text{Stability}(\text{Child}_2)$$

তবে চাইল্ডগুলোকে অর্থহীন নয়জ ধরে কেটে ফেলা হয় এবং প্যারেন্টকেই চূড়ান্ত ক্লাস্টার হিসেবে রেখে দেওয়া হয়। অন্যথায় চাইল্ড ক্লাস্টার দুটি টিকে থাকে এবং প্যারেন্টকে বাতিল করা হয়।

এইচডিবিএসক্যান গাণিতিক প্রয়োগ

HDBSCAN-এর গাণিতিক ভিত্তি (Core Distance, Mutual Reachability Distance এবং Stability) কীভাবে প্র্যাক্টিক্যাল উপাত্ত ক্ষেত্রে কাজ করে, তা একটি ছোট হাতে-গণনাযোগ্য উদাহরণের মাধ্যমে ধাপে ধাপে দেখানো হলো।

১. ডেটাসেট ও হাইপারপ্যারামিটার নির্ধারণ (Dataset & Parameters)

ধরা যাক, একটি একমাত্রিক স্থানে (1D Space) আমাদের কাছে $N = 4$ টি ডেটা পয়েন্ট রয়েছে:

- $x_1 = 1$
- $x_2 = 2$
- $x_3 = 2.5$
- $x_4 = 7$ (এটি একটি ছড়ানো বিন্দু বা সম্ভাব্য নয়জ)

প্যারামিটার নির্বাচন:

- **min_samples (k) = 2** (কোর ডিস্টেন্সের জন্য ২-তম প্রতিবেশ বিবেচনা করা হবে)
- **min_cluster_size = 2** (একটি ক্লাস্টার টিকে থাকার সর্বনিম্ন সাইজ)

২. কোর ডিস্টেন্স গণনা (Core Distance Calculation)

প্রতিটি বিন্দুর নিজস্ব অবস্থান থেকে তার k -তম (অর্থাৎ ২-তম) নিকটবর্তী প্রতিবেশীর দূরত্বই হলো তার কোর ডিস্টেন্স। (মনে রাখবেন, ১-তম নিকটবর্তী প্রতিবেশী হচ্ছে বিন্দুটি নিজেই, যার দূরত্ব ০)।

- $\text{core}_2(x_1)$: $x_1(1)$ থেকে বাকিদের দূরত্ব হলো: x_2 এর দূরত্ব $|2 - 1| = 1$, x_3 এর দূরত্ব $|2.5 - 1| = 1.5$ । সুতরাং, ২-তম নিকটবর্তী প্রতিবেশী x_2 , যার দূরত্ব 1।

$$\text{core}_2(x_1) = 1.0$$

- $\text{core}_2(x_2)$: $x_2(2)$ থেকে বাকিদের দূরত্ব: x_3 এর দূরত্ব 0.5, x_1 এর দূরত্ব 1। ২-তম প্রতিবেশী x_1 , যার দূরত্ব 1।

$$\text{core}_2(x_2) = 1.0$$

- $\text{core}_2(x_3)$: $x_3(2.5)$ থেকে বাকিদের দূরত্ব: x_2 এর দূরত্ব 0.5, x_1 এর দূরত্ব 1.5। ২-তম প্রতিবেশী x_1 , যার দূরত্ব 1.5।

$$\text{core}_2(x_3) = 1.5$$

- $\text{core}_2(x_4)$: $x_4(7)$ থেকে বাকিদের দূরত্ব: x_3 এর দূরত্ব 4.5, x_2 এর দূরত্ব 5। ২-তম প্রতিবেশী x_2 , যার দূরত্ব 5। (এটি স্পার্স বা ছড়ানো এলাকায় থাকায় এর কোর ডিস্টেন্স অনেক বড় এসেছে)।

$$\text{core}_2(x_4) = 5.0$$

৩. মিউচুয়াল রিচ্যাবিলিটি ডিস্টেন্স ম্যাট্রিক্স তৈরি (Mutual Reachability Matrix)

এবার আমরা পরিবর্তিত দূরত্ব সূত্র ব্যবহার করে বিন্দুগুলোর মধ্যকার d_{mr} গণনা করব:

$$d_{mr}(A, B) = \max(\{\text{core}_k(A), \text{core}_k(B), d(A, B)\})$$

- $d_{mr}(x_1, x_2)$: $\max(\{\text{core}_2(x_1) = 1.0, \text{core}_2(x_2) = 1.0, d(x_1, x_2) = 1.0\}) = 1.0$
- $d_{mr}(x_2, x_3)$: $\max(\{\text{core}_2(x_2) = 1.0, \text{core}_2(x_3) = 1.5, d(x_2, x_3) = 0.5\}) = 1.5$
- $d_{mr}(x_1, x_3)$: $\max(\{\text{core}_2(x_1) = 1.0, \text{core}_2(x_3) = 1.5, d(x_1, x_3) = 1.5\}) = 1.5$
- $d_{mr}(x_3, x_4)$: $\max(\{\text{core}_2(x_3) = 1.5, \text{core}_2(x_4) = 5.0, d(x_3, x_4) = 4.5\}) = 5.0$

৪. মিনিমাম স্প্যানিং ট্রি এবং ডেন্ড্রোগ্রাম বিভাজন (MST & Hierarchy)

d_{mr} মানের ওপর ভিত্তি করে সর্বনিম্ন সংযোগ রেখাগুলো আগে জুড়লে যে **Minimum Spanning Tree (MST)** পাওয়া যায়, তার বিন্যাস:

$$(x_1) \overset{1.0}{\longleftrightarrow} (x_2) \overset{1.5}{\longleftrightarrow} (x_3) \overset{5.0}{\longleftrightarrow} (x_4)$$

এবার আমরা ল্যাম্বডা বা ঘনত্বের থ্রেশহোল্ড ($\lambda = \frac{1}{d}$) বাড়িয়ে ট্রি-টি কাটতে শুরু করব:

- $\lambda = 0$ থেকে 0.2 এর পূর্ব পর্যন্ত ($d > 5.0$): সবগুলো বিন্দু $\{x_1, x_2, x_3, x_4\}$ একটি একক বৃহৎ প্যারেন্ট ক্লাস্টারে যুক্ত থাকে।

- $\lambda = 0.2$ তে ($d = 5.0$): 5.0 ওজনের এজ বা সংযোগটি কেটে যায়। এর ফলে x_4 বিন্দুটি মূল ট্রি থেকে সম্পূর্ণ আলাদা হয়ে যায়।
 - **ছাঁটাই লজিক (Condensation):** আলাদা হওয়া নতুন ডালে মাত্র ১টি বিন্দু (x_4) রয়েছে। যেহেতু আমাদের শর্ত $\text{min_cluster_size} = 2$, তাই এটিকে নতুন ক্লাস্টার বলা যাবে না। এটি হলো "ক্লাস্টার থেকে x_4 বিন্দুর ঝরে পড়া"। বিন্দু x_4 এর ঝরে পড়ার ঘনত্ব $\lambda_{x_4} = 0.2$ ।
- $\lambda = 0.67$ তে ($d = 1.5$): এবার 1.5 ওজনের এজটি কেটে যায় এবং x_3 বিন্দুটি আলাদা হয়ে যায়। এটিও min_cluster_size পূর্ণ না করায় ঝরে পড়া বিন্দু হিসেবে গণ্য হবে। $\lambda_{x_3} = 0.67$ ।
- $\lambda = 1.0$ তে ($d = 1.0$): শেষ প্রান্তে থাকা এজটি কেটে x_1 এবং x_2 আলাদা হয়ে যায় এবং ক্লাস্টারটি সম্পূর্ণ বিলুপ্ত হয়। $\lambda_{x_1} = 1.0$ এবং $\lambda_{x_2} = 1.0$ ।

৫. ক্লাস্টার স্থায়িত্ব পরিমাপ (Cluster Stability Optimization)

যেহেতু মাঝপথে কোনো বৈধ স্প্লিট বা বিভাজন (যেখানে উভয় ডালে নূন্যতম ২টি করে বিন্দু থাকে) ঘটেনি, তাই আমাদের মূল প্যারেন্ট ক্লাস্টারটিই একমাত্র প্রধান ডাল হিসেবে টিকে থাকে। এই ক্লাস্টারটির জন্ম হয়েছিল একদম শুরুতে ($\lambda_{\text{birth}} = 0$)।

এবার প্রতিটি বিন্দুর ঝরে পড়ার ঘনত্ব (λ_p) ব্যবহার করে ক্লাস্টারের স্থায়িত্ব বা Stability গণনা করি:

$$\text{Stability} = \sum_p (\lambda_p - \lambda_{\text{birth}})$$

$$\text{Stability} = (1.0 - 0) + (1.0 - 0) + (0.67 - 0) + (0.2 - 0) = 1.0 + 1.0 + 0.67 + 0.2 = 2.87$$

চূড়ান্ত সিদ্ধান্ত: এই ২.৮৭ উচ্চ স্থায়িত্ব স্কোর থাকার কারণে $\{x_1, x_2, x_3\}$ বিন্দুগুলো একসাথে একটি সুনির্দিষ্ট বৈধ ক্লাস্টার গঠন করে। অন্যদিকে, x_4 বিন্দুটির স্থায়িত্বের একদম শুরুতে ($\lambda = 0.2$) ঝরে পড়ার কারণে এবং ক্লাস্টারের মূল ঘনত্বের সাথে সামঞ্জস্য না থাকায় সে কোনো গ্রুপ পায় না।

(Final Labels)

- $x_1, x_2, x_3 \rightarrow \text{Cluster 0}$
- $x_4 \rightarrow -1 \text{ (Noise / Outlier)}$

এইচডিবিএসক্যান (HDBSCAN) হাইপারপ্যারামিটার বিশ্লেষণ

HDBSCAN অ্যালগরিদমের সবচেয়ে বড় সুবিধা হলো এটি প্রথাগত DBSCAN-এর মতো Epsilon (ϵ) প্যারামিটারের ওপর নির্ভরশীল নয়, যার কারণে টিউনিং প্রসেস অনেক সহজ হয়ে যায়। পাইথনের `hdbscan` বা `sklearn.cluster` লাইব্রেরিতে মডেলের কার্যকারিতা ও গতি নিয়ন্ত্রণ করার জন্য নিচের প্যারামিটারগুলো সবচেয়ে গুরুত্বপূর্ণ (Most Important) থেকে কম গুরুত্বপূর্ণ (Less Important) ক্রমানুসারে আলোচনা করা হলো:

১. `min_cluster_size`

- **Default:** 5
- **প্যারামিটারটি কী কাজ করে:** এটি হলো HDBSCAN-এর সবচেয়ে প্রধান এবং সংবেদনশীল হাইপারপ্যারামিটার। এটি নির্ধারণ করে যে, একটি ডালপালা বা গ্রুপকে একটি সত্যিকারের স্বাধীন ক্লাস্টার (True Cluster) হিসেবে মেমোরিতে টিকিয়ে রাখার জন্য তার ভেতর ন্যূনতম কতটি ডেটা পয়েন্টের উপস্থিতি প্রয়োজন।
- **অন্যান্য অপশন ও ব্যবহার:** এটি যেকোনো ধনাত্মক পূর্ণসংখ্যা (Integer) গ্রহণ করে।
 - **মান খুব ছোট হলে (যেমন: ৩ বা ৪):** ট্রি ছাঁটাইয়ের সময় অ্যালগরিদমটি খুব নরম বা নমনীয় আচরণ করবে, যার ফলে ডেটাসেটে অনেক ছোট ছোট উপ-ক্লাস্টার (Micro-clusters) তৈরি হবে।
 - **মান খুব বড় হলে (যেমন: ৫০ বা ১০০):** ট্রি-র ছোটখাটো সব ডালপালা কেটে বাদ দেওয়া হবে, যার ফলে শুধুমাত্র বড় এবং প্রধান গ্রুপগুলোই টিকে থাকবে। ডোমেইন নলেজের ওপর ভিত্তি করে আপনার প্রজেক্টে ন্যূনতম কত বড় গ্রুপ দরকার, সেই অনুযায়ী এটি ফিক্সড করতে হবে।

২. `min_samples`

- **Default:** None (যা স্বয়ংক্রিয়ভাবে `min_cluster_size`-এর মানের সমান ধরে নেওয়া হয়)
- **প্যারামিটারটি কী কাজ করে:** এটি মূলত ঘনত্ব পরিমাপের জন্য **কোর ডিস্টেন্স (Core Distance)** গণনায় ব্যবহৃত হয়। এটি নির্ধারণ করে যে, কোনো একটি বিন্দুর চারপাশের এলাকা কতটা ঘন—তা পরিমাপের সময় তার কততম নিকটবর্তী প্রতিবেশীকে (Neighbor) বিবেচনা করা হবে। এটি সরাসরি মডেলের নয়েজ সিলেকশন (Noise Selection) এবং ক্লাস্টারের রক্ষণশীলতা নিয়ন্ত্রণ করে।
- **অন্যান্য অপশন ও ব্যবহার:** যেকোনো পূর্ণসংখ্যা ইনপুট দেওয়া যায়।

- যদি আপনার ডেটাসেটে প্রচুর পরিমাণে ছড়ানো নয়েজ বা আউটলায়ার থাকে, তবে min_samples-এর মান বাড়িয়ে দেওয়া উচিত। মান যত বড় করবেন, কোর ডিস্টেন্সের মান তত বড় হবে এবং ছড়ানো পয়েন্টগুলো সহজেই **Noise (-1)** হিসেবে বাদ পড়ে যাবে, যা ক্লাস্টার বাউন্ডারিকে আরও সুনির্দিষ্ট ও রক্ষণশীল (Conservative) করে তোলে।
- আর ডেটা যদি খুব ক্লিন হয়, তবে এটিকে None বা ছোট মান দিয়ে রাখা ভালো।

৩. metric

- **Default:** 'euclidean'
- **প্যারামিটারটি কী কাজ করে:** মিউচুয়াল রিচ্যাবিলিটি ডিস্টেন্স (d_{mr}) এবং মিনিমাম স্প্যানিং ট্রি গঠন করার সময় বিন্দুগুলোর মধ্যকার পারস্পরিক দূরত্ব কোন গাণিতিক সূত্রের সাহায্যে পরিমাপ করা হবে, তা এটি নির্ধারণ করে।
- **অন্যান্য অপশন ও ব্যবহার:** এটি বিভিন্ন ডিস্টেন্স স্ট্রিং সাপোর্ট করে, যেমন: 'manhattan', 'cosine', 'chebyshev' ইত্যাদি।
 - সাধারণ ফিজিক্যাল স্পেস বা কন্টিনিউয়াস নিউমেরিক্যাল ডেটার জন্য ডিফল্ট 'euclidean'-ই সেরা।
 - শহরের রাস্তা বা গ্রিড-ভিত্তিক পথের দূরত্ব ট্র্যাকিংয়ের জন্য ডোমেইন নলেজ অনুযায়ী 'manhattan' ব্যবহার করা সুসংগত।
 - টেক্সট মাইনিং বা এনএলপি (NLP) প্রজেক্টের ক্ষেত্রে ভেক্টরের অ্যাঙ্গেল বা কোণ মাপার জন্য 'cosine' মেট্রিক সিলেক্ট করা আবশ্যিক।

৪. alpha

- **Default:** 1.0
- **প্যারামিটারটি কী কাজ করে:** এটি ডেন্ড্রোগ্রাম বা সিঙ্গেল লিঙ্কেজ ট্রি (Single Linkage Tree) তৈরি করার সময় গাণিতিক দূরত্বের কঠোরতা বা স্কেলিং নিয়ন্ত্রণ করে। এটি মূলত দূরত্বের বাধনকে কিছুটা শিথিল বা শক্ত করতে ব্যবহৃত হয়।
- **অন্যান্য অপশন ও ব্যবহার:** যেকোনো ধনাত্মক ফ্লোট (Float) সংখ্যা।
 - সাধারণ প্রজেক্টে এর ডিফল্ট মান ১.০-ই সবচেয়ে সুষম আউটপুট দেয়, তাই অ্যাডভান্সড থিওরিটিক্যাল অপ্টিমাইজেশন ছাড়া এটি পরিবর্তন করার সাধারণত কোনো প্রয়োজন হয় না।

৫. algorithm

- **Default:** 'best'
- **প্যারামিটারটি কী কাজ করে:** ব্যাকঅ্যান্ডে প্রতিবেশীদের খোঁজার জন্য এবং মিনিমাম স্প্যানিং ট্রি তৈরির গাণিতিক ক্যালকুলেশনগুলো দ্রুত সম্পন্ন করতে কোন ইন্টারনাল সার্চ আর্কিটেকচার মেকানিজম ব্যবহার করা হবে, তা এটি ঠিক করে।
- **অন্যান্য অপশন ও ব্যবহার:** এর অপশনগুলো হলো: 'generic', 'prims_kdtree', 'prims_balltree', এবং 'boruvka_kdtree'।
 - 'best': লাইব্রেরিটি ইনপুট ডেটার সাইজ, ডাইমেনশন এবং মেমোরি কস্টের ওপর ভিত্তি করে ব্যাকঅ্যান্ডের জন্য সবচেয়ে সুপার-ফাস্ট ও উপযুক্ত অ্যালগরিদমটি স্বয়ংক্রিয়ভাবে বেছে নেয়।
 - ছোট ও মাঝারি ডেটার জন্য প্রিমস বা বোরুভকা কেডি-ট্রি চমৎকার স্পিড অপ্টিমাইজেশন দেয়।

৬. leaf_size

- **Default:** 40
- **প্যারামিটারটি কী কাজ করে:** কেডি-ট্রি বা বল-ট্রি তৈরি করার সময় ইন্টারনাল ট্রির লিফ নোডের সাইজ কত হবে, তা এটি নিয়ন্ত্রণ করে। এটি মেমোরি স্টোরেজ এবং কুয়েরি করার প্রসেসিং টাইমের স্পিডকে প্রভাবিত করে।
- **অন্যান্য অপশন ও ব্যবহার:** যেকোনো পূর্ণসংখ্যা। সাধারণ ডেটা প্রসেসিং ওয়ার্কফ্লোতে এর ডিফল্ট মানই সবচেয়ে স্টেবল পারফরম্যান্স দেয়, তাই এটি টিউন করার প্রয়োজন পড়ে না।

Principal Component Analysis

মেশিন লার্নিং এবং ডেটা প্রিপ্রসেসিংয়ের ক্ষেত্রে ফিচার সিলেকশন (Feature Selection) এবং ফিচার এক্সট্রাকশন (Feature Extraction) দুটি অত্যন্ত গুরুত্বপূর্ণ অথচ ভিন্নধর্মী পদ্ধতি, যা মূলত ডেটাসেটের ডাইমেনশনালিটি বা কলামের সংখ্যা কমানোর জন্য ব্যবহৃত হয়। **ফিচার সিলেকশন** হলো এমন একটি প্রক্রিয়া যেখানে মূল ডেটাসেটে বিদ্যমান ফিচার বা বৈশিষ্ট্যগুলোর মধ্য থেকে কোনো পরিবর্তন ছাড়াই শুধুমাত্র সবচেয়ে কার্যকরী এবং প্রাসঙ্গিক ফিচারগুলোকে বেছে নেওয়া হয় এবং অপ্রয়োজনীয় বা পুনরাবৃত্তিমূলক ফিচারগুলোকে বাদ দেওয়া হয়। যেমন—একটি বাড়ির দাম নির্ধারণের মডেল তৈরি করার সময় যদি ডেটাসেটে 'শোবার ঘরের সংখ্যা', 'ডাইনিং রুমের আয়তন' এবং 'বাড়ির মালিকের প্রিয় রঙ'—এই তিনটি ফিচার থাকে, তবে ফিচার সিলেকশন পদ্ধতিটি শুধুমাত্র প্রথম দুটি প্রাসঙ্গিক বৈশিষ্ট্যকে রেখে তৃতীয়টি সম্পূর্ণ বর্জন করবে। এখানে মূল ডেটার কোনো রূপান্তর ঘটে না।

অন্যদিকে, **ফিচার এক্সট্রাকশন** পদ্ধতিতে মূল ফিচারগুলোর ডেটাকে গাণিতিক বা অ্যালগরিদমিক রূপান্তরের মাধ্যমে সম্পূর্ণ নতুন এবং কম সংখ্যক একগুচ্ছ ফিচারে পরিণত করা হয়, যা মূল ডেটার প্রায় সমস্ত গুরুত্বপূর্ণ তথ্য বা বৈচিত্র্য ধারণ করতে পারে। এই প্রক্রিয়ায় আদি ফিচারগুলোর নিজস্ব রূপ বা পরিচয় বজায় থাকে না। উদাহরণস্বরূপ—কোনো রোগীর স্বাস্থ্য পরীক্ষার ডেটাসেটে যদি 'সিস্টোলিক রক্তচাপ' এবং 'ডায়াস্টোলিক রক্তচাপ' নামক দুটি ভিন্ন ফিচার থাকে, তবে প্রিন্সিপাল কম্পোনেন্ট অ্যানালাইসিস (PCA) এর মতো ফিচার এক্সট্রাকশন পদ্ধতি ব্যবহার করে এই দুটিকে একত্রিত করে 'সামগ্রিক রক্তচাপের তীব্রতা' নামক সম্পূর্ণ নতুন একটি সমন্বিত ফিচার তৈরি করা যেতে পারে। সংক্ষেপে বলা যায়, ফিচার সিলেকশন যেখানে বিদ্যমান বৈশিষ্ট্যগুলোর মধ্য থেকে ফিল্টারিং বা বাছাইকরণের কাজ করে, ফিচার এক্সট্রাকশন সেখানে মূল ডেটার তথ্যকে ঘনীভূত করে সম্পূর্ণ নতুন বৈশিষ্ট্যের জন্ম দেয়।

মেশিন লার্নিং মডেল তৈরিতে ফিচার সিলেকশন এবং ফিচার এক্সট্রাকশনের জন্য ব্যবহৃত প্রধান মডেল ও অ্যালগরিদমগুলোর সংক্ষিপ্ত রূপ নিচে দেওয়া হলো:

১. ফিচার সিলেকশন (Feature Selection) মডেলসমূহ

এই পদ্ধতিতে মূল ফিচারগুলোর মধ্য থেকে সেরা কলামগুলো বাছাই করা হয়:

- **ফিল্টার মেথড (Filter Methods):** মূল ডেটার পরিসংখ্যানগত বৈশিষ্ট্য যাচাই করে ফিচার বাছাই করা হয়। যেমন: Chi-Square (χ^2) টেস্ট এবং ANOVA (Analysis of Variance)।

- **র‍্যাপার মেথড (Wrapper Methods):** একটি নির্দিষ্ট প্রেডিক্টিভ মডেলকে বারবার ব্যবহার করে ফিচারের কার্যকারিতা পরীক্ষা করা হয়। যেমন: RFE (Recursive Feature Elimination)।
- **এম্বেডেড মেথড (Embedded Methods):** মডেলের নিজস্ব ট্রেনিং প্রক্রিয়ার ভেতরেই স্বয়ংক্রিয়ভাবে ফিচার বাছাই সম্পন্ন হয়। যেমন: LASSO রিগ্রেশন (L1 Regularization) এবং র‍্যান্ডম ফরেস্ট (Random Forest) ফিচার ইম্পোর্টেন্স।

২. ফিচার এক্সট্রাকশন (Feature Extraction) মডেলসমূহ

এই পদ্ধতিতে গাণিতিক রূপান্তরের মাধ্যমে মূল ফিচারগুলোকে একত্রিত করে সম্পূর্ণ নতুন ফিচার তৈরি করা হয়:

- **PCA (Principal Component Analysis):** এটি একটি আনসুপারভাইজড লিনিয়ার মডেল, যা ডেটার সর্বোচ্চ ভ্যারিয়েন্স বা বৈচিত্র্য অক্ষ (Principal Components) বরাবর প্রজেক্ট করে ডাইমেনশন কমায়।
- **LDA (Linear Discriminant Analysis):** এটি একটি সুপারভাইজড লিনিয়ার মডেল, যা বিভিন্ন ক্লাসের মধ্যকার পার্থক্য (Separability) সর্বোচ্চ করার মাধ্যমে ডাইমেনশন কমায়।
- **অটোএনকোডার (Autoencoders):** এটি একটি ডিপ লার্নিং নিউরাল নেটওয়ার্ক আর্কিটেকচার, যার 'এনকোডার' অংশটি ইনপুট ডেটাকে একটি কম ডাইমেনশনের ঘন বা সংকুচিত অবস্থায় (Bottleneck layer) রূপান্তর করে।
- **সিএনএন (CNN - Convolutional Neural Networks):** ইমেজ ডেটার ক্ষেত্রে সিএনএন-এর শুরুর লেয়ারগুলো স্বয়ংক্রিয়ভাবে ছবির এজ (Edge), টেক্সচার ইত্যাদি এক্সট্রাক্ট করে নতুন ফিচার ম্যাপ তৈরি করে।

প্রিন্সিপাল কম্পোনেন্ট অ্যানালাইসিস (PCA) এর পরিচিতি

মেশিন লার্নিং এবং ডেটা সায়েন্সে যখন আমরা বিশাল কোনো ডেটাসেট নিয়ে কাজ করি, তখন অনেক সময় ফিচারের সংখ্যা অনেক বেশি থাকে। অতিরিক্ত ফিচারের কারণে মডেল জটিল হয়ে যায় এবং পারফরম্যান্স কমে যায়। এই সমস্যা সমাধানের জন্য ডাইমেনশনালিটি রিডাকশন বা ফিচারের সংখ্যা কমানোর প্রয়োজন হয়। আর এই কাজের জন্য সবচেয়ে জনপ্রিয় এবং শক্তিশালী একটি আনসুপারভাইজড অ্যালগরিদম হলো **Principal Component Analysis (PCA)**। নিচে এর মূল বিষয়গুলো সহজভাবে আলোচনা করা হলো:

Curse of Dimensionality (মাত্রার অভিশাপ)

ডেটাসেটে ফিচারের (কলাম) সংখ্যা মাত্রাতিরিক্তভাবে বৃদ্ধি পাওয়ার কারণে যে গাণিতিক ও কম্পিউটেশনাল জটিলতা তৈরি হয়, তাকে মেশিন লার্নিংয়ের ভাষায় **Curse of Dimensionality** বা মাত্রার অভিশাপ বলা হয়।

- **ধারণা:** ডেটার মাত্রা (Dimension) যত বাড়ে, ডেটা পয়েন্টগুলোর মধ্যবর্তী খালি জায়গা (Sparsity) তত গুণিতক হারে বৃদ্ধি পায়।
- **ফলাফল:** এর ফলে ডেটা পয়েন্টগুলো ফিচার স্পেসে একে অপরের থেকে অনেক দূরে ছিটকে যায় এবং তাদের মধ্যে মিল বা অমিল খুঁজে পাওয়া কঠিন হয়ে পড়ে।

অতিরিক্ত ফিচার কেন সমস্যা তৈরি করে? (Why so much features create problems?)

ডেটাসেটে ফিচারের সংখ্যা অনেক বেশি হলে মূলত নিচের সমস্যাগুলো দেখা দেয়:

- **ওভারফিটিং (Overfitting):** অতিরিক্ত ফিচারের কারণে মডেল গুরুত্বপূর্ণ প্যাটার্ন শেখার পাশাপাশি ডেটার অপ্রয়োজনীয় নয়েজ (Noise) বা ভুল তথ্যও মুখস্থ করে ফেলে। ফলে ট্রেনিং ডেটায় চমৎকার ফলাফল দিলেও নতুন কোনো টেস্ট ডেটায় মডেলটি সম্পূর্ণ ব্যর্থ হয়।
- **কম্পিউটেশনাল খরচ বৃদ্ধি:** ফিচারের সংখ্যা যত বেশি হবে, মডেল ট্রেন করতে তত বেশি মেমোরি (RAM) এবং প্রসেসিং টাইমের প্রয়োজন হয়।
- **দূরত্ব পরিমাপের জটিলতা (Distance Distortion):** KNN বা SVM-এর মতো দূরত্ব-ভিত্তিক অ্যালগরিদমগুলোতে মাত্রা বাড়ার সাথে সাথে পয়েন্টগুলোর মধ্যকার দূরত্বের পার্থক্য কমে যায়, ফলে মডেল সঠিক সিদ্ধান্ত নিতে পারে না।

PCA কী? (What is PCA?)

Principal Component Analysis (PCA) হলো একটি লিনিয়ার ডাইমেনশনালিটি রিডাকশন টেকনিক, যা কোনো ডেটাসেটের মূল তথ্য বা বৈশিষ্ট্যগুলোকে অক্ষুণ্ণ রেখে ফিচারের সংখ্যা (Dimensions) কমিয়ে আনে। এটি ডেটার মূল ফিচারগুলোকে সরাসরি বাদ দেয় না, বরং

ফিচারগুলোর লিনিয়ার কম্বিনেশনের মাধ্যমে সম্পূর্ণ নতুন কিছু ফিচার তৈরি করে, যাদের **Principal Components (PCs)** বলা হয়। এই নতুন কম্পোনেন্টগুলো একে অপরের থেকে সম্পূর্ণ স্বাধীন থাকে।

ভ্যারিয়েন্স (Variance)

মেশিন লার্নিং ও স্ট্যাটিস্টিক্সে ভ্যারিয়েন্স বা ভেদাঙ্ক বলতে বোঝায় ডেটা পয়েন্টগুলো তাদের গড় মান (Mean) থেকে কতটা ছড়ানো-ছিটানো বা বিন্যস্ত অবস্থায় রয়েছে।

- **PCA-তে ভূমিকা:** PCA অ্যালগরিদমে ভ্যারিয়েন্সকে "তথ্যের পরিমাণ" হিসেবে বিবেচনা করা হয়। যে অক্ষ বা ডাইমেনশনে ডেটার ছড়াছড়ি বা ভ্যারিয়েন্স সবচেয়ে বেশি, সেখানে তথ্যের পরিমাণও সবচেয়ে বেশি থাকে। PCA এমনভাবে নতুন অক্ষ তৈরি করে যাতে প্রথম প্রিন্সিপাল কম্পোনেন্টটি (PC1) ডেটার সর্বোচ্চ ভ্যারিয়েন্স ধরে রাখতে পারে।

কোরিলেশন (Correlation)

কোরিলেশন বা সহসম্বন্ধ হলো দুটি ফিচারের মধ্যকার পারস্পরিক গাণিতিক সম্পর্ক। অর্থাৎ, একটি ফিচারের মান বাড়লে বা কমলে অন্য ফিচারটির মান কীভাবে পরিবর্তিত হচ্ছে, তা কোরিলেশন দিয়ে মাপা হয়।

- **PCA-তে ভূমিকা:** বাস্তব জীবনের ডেটাসেটে অনেক ফিচার একে অপরের সাথে গভীরভাবে সম্পর্কিত থাকে (যাকে Multicollinearity বলা হয়)। যেমন—বাড়ির আয়তন বাড়লে রুমের সংখ্যাও সাধারণত বাড়ে। PCA এই কোরিলেটেড বা ওভারল্যাপিং ফিচারগুলোর ভেতরের তথ্যকে একত্রিত করে সম্পূর্ণ নতুন এবং আন-কোরিলেটেড (Uncorrelated) ফিচার তৈরি করে, যা মডেলের জটিলতা কমায়।

PCA-এর মূল লক্ষ্য (Main Goal of PCA)

PCA-এর প্রধান লক্ষ্যগুলোকে নিচে সংক্ষেপে তুলে ধরা হলো:

- **তথ্য সংরক্ষণ করে মাত্রা কমানো:** ডেটাসেটের সবচেয়ে গুরুত্বপূর্ণ তথ্য ও ভ্যারিয়েন্সকে অক্ষুণ্ণ রেখে ফিচারের সংখ্যা বা ডাইমেনশন সর্বোচ্চ পরিমাণে কমিয়ে আনা।
- **মাল্টিকোলিনিয়ারিটি দূর করা:** ফিচারের মধ্যকার পারস্পরিক কোরিলেশন দূর করে সম্পূর্ণ স্বাধীন ও আন-কোরিলেটেড প্রিন্সিপাল কম্পোনেন্ট তৈরি করা।
- **ডেটা ভিজ্যুয়ালাইজেশন:** ৪ বা তার বেশি উচ্চ-মাত্রার (High-Dimensional) ডেটাকে সহজে বোঝার জন্য ২D বা ৩D গ্রাফে রূপান্তর করা, যা মানুষের পক্ষে ভিজ্যুয়ালাইজ করা সম্ভব।

- **মডেলের গতি ও দক্ষতা বাড়ানো:** অপ্রয়োজনীয় নয়েজ দূর করার মাধ্যমে মেশিন লার্নিং মডেলের কম্পিউটেশনাল সময় বাঁচানো এবং ওভারফিটিং প্রতিরোধ করা।

মেশিন লার্নিং এবং ডেটা সায়েন্সে **Curse of Dimensionality** বা মাত্রার অভিশাপ বোঝার জন্য ডাইমেনশন (ফিচার) বাড়ার সাথে সাথে ডেটার জ্যামিতিক রূপ এবং ঘনত্বের কেমন পরিবর্তন হয়, তা ধাপে ধাপে জানা প্রয়োজন।

নিচে ১D থেকে শুরু করে মাত্রা বাড়লে কীভাবে ডেটা স্পেসে দূরত্ব এবং ফাঁকা জায়গা বাড়ে, তা বিস্তারিত আলোচনা করা হলো:

১D থেকে ৩D এবং উচ্চ মাত্রায় রূপান্তর (The Dimensional Shift)

ধরা যাক, আমাদের কাছে একটি নির্দিষ্ট সংখ্যক ডেটা পয়েন্ট আছে এবং আমরা সেগুলোকে ১টি, ২টি, ৩টি এবং আরও বেশি ফিচারের বক্সে সাজাবো।

১D (এক মাত্রা — ১টি ফিচার):

- **জ্যামিতিক রূপ:** এটি একটি সরলরেখা (Line)।
- **ডেটার অবস্থা:** ধরুন, আপনার কাছে ১০টি ডেটা পয়েন্ট আছে এবং সরলরেখাটির দৈর্ঘ্য ১০ ইউনিট। আপনি যদি পয়েন্টগুলোকে সাজান, তবে প্রতিটি পয়েন্ট গড়ে ১ ইউনিট দূরে দূরে বসবে। ডেটা পয়েন্টগুলো খুব কাছাকাছি থাকবে এবং তাদের মধ্যকার দূরত্ব বা ঘনত্ব খুব সহজে পরিমাপ করা যাবে।

২D (দ্বিমাত্রিক — ২টি ফিচার):

- **জ্যামিতিক রূপ:** এটি একটি দ্বিমাত্রিক সমতল পৃষ্ঠ বা গ্রিড (Square/Plane)।
- **ডেটার অবস্থা:** এখন দৈর্ঘ্য এবং প্রস্থ উভয়ই ১০ ইউনিট, অর্থাৎ মোট এলাকা $10 \times 10 = 100$ বর্গইউনিট। সেই একই ১০টি ডেটা পয়েন্টকে যদি আপনি এই ১০০টি বক্সের মধ্যে ছড়িয়ে দেন, তবে বক্সগুলোর বেশিরভাগই খালি থাকবে। পয়েন্টগুলোর নিজেদের মধ্যকার গড় দূরত্ব ১D-এর চেয়ে অনেক বেড়ে যাবে।

৩D (ত্রিমাত্রিক — ৩টি ফিচার):

- **জ্যামিতিক রূপ:** এটি একটি ত্রিমাত্রিক ঘনক বা কিউব (Cube/Space)।
- **ডেটার অবস্থা:** এখন আয়তন হলো $10 \times 10 \times 10 = 1000$ ঘনইউনিট। সেই একই ১০টি ডেটা পয়েন্টকে ১০০০টি ছোট কিউবের মধ্যে ছেড়ে দিলে পয়েন্টগুলো চরমভাবে একে অপরের থেকে দূরে ছিটকে যাবে। স্পেসের ৯৯% এলাকাই সম্পূর্ণ ফাঁকা (Sparse) থাকবে।

উচ্চ মাত্রা (High Dimension — যেমন ১০০টি ফিচার):

- যখন ডাইমেনশন বা ফিচারের সংখ্যা ১০০ বা ১০০০ হয়ে যায়, তখন স্পেসের মোট আয়তন বা খালি জায়গার পরিমাণ গাণিতিকভাবে অসীমমুখী হয়। ডেটা পয়েন্টের সংখ্যা যদি কয়েক হাজারও হয়, তবুও এই বিশাল হাইপার-স্পেসে তারা একে অপরের থেকে কোটি কোটি মাইল দূরে এক একটি বিচ্ছিন্ন দ্বীপের মতো অবস্থান করে।

উচ্চ মাত্রায় (High Dimension) ঠিক কী কী সমস্যা ঘটে?

ডাইমেনশন এভাবে বাড়তে থাকলে প্রধান ৪টি গাণিতিক জটিলতা তৈরি হয়:

১. ডেটার চরম শূন্যতা (Data Sparsity)

ডাইমেনশন বাড়ার সাথে সাথে স্পেসের আয়তন সূচকীয় হারে (Exponentially) বাড়ে, কিন্তু সেই তুলনায় আমাদের ডেটা স্যাম্পলের সংখ্যা বাড়ে না। ফলে পুরো ডেটা স্পেসটি ফাঁকা হয়ে যায়। মডেলের পক্ষে এই বিশাল ফাঁকা জায়গা থেকে কোনো সঠিক প্যাটার্ন বা ট্রেন্ড খুঁজে পাওয়া অসম্ভব হয়ে পড়ে।

২. দূরত্বের সমতা বা বৈচিত্র্যহীনতা (Distance Concentration Effect)

এটি মাত্রার অভিশাপের সবচেয়ে মারাত্মক সমস্যা।

- ১D বা ২D-তে:** কোনো একটি পয়েন্টের খুব কাছে কিছু বন্ধু (Nearest Neighbors) থাকে এবং দূরে কিছু শত্রু থাকে।
- উচ্চ মাত্রায়:** Euclidean Distance বা দূরত্ব পরিমাপের সূত্র অনুযায়ী, ডাইমেনশন যত বাড়ে, প্রতিটি বিন্দুর মধ্যকার পরম দূরত্ব (Absolute Distance) তত বাড়ে। কিন্তু সমস্যা হলো, সবচেয়ে কাছের বিন্দুর দূরত্ব ($Dist_{min}$) এবং সবচেয়ে দূরের বিন্দুর দূরত্বের ($Dist_{max}$) মধ্যকার পার্থক্য প্রায় শূন্য হয়ে যায়।
- ফলাফল:** ফিচার স্পেসে সব ডেটা পয়েন্ট তখন একে অপরের থেকে প্রায় সমান দূরত্বে অবস্থান করে। ফলে KNN বা SVM-এর মতো অ্যালগরিদমগুলো কোনটা আসল "কাছের প্রতিবেশী (Nearest Neighbor)" আর কোনটা "দূরের", তা আর বৈজ্ঞানিকভাবে আলাদা করতে পারে না।

৩. ওভারফিটিং এবং হিউজ ডেটার প্রয়োজনীয়তা (Hughes Phenomenon)

তাত্ত্বিকভাবে, উচ্চ মাত্রার ফিচার স্পেসে একটি লিনিয়ার মডেলকে নিখুঁতভাবে ট্রেনিং করতে হলে ফিচারের সংখ্যার তুলনায় সূচকীয় হারে (Exponentially) বেশি ডেটা স্যাম্পল (Rows) প্রয়োজন। বাস্তব জীবনে এত বিশাল ডেটা পাওয়া সম্ভব হয় না। পর্যাপ্ত ডেটা না থাকায় মডেলটি ডেটার মূল লজিক শেখার পরিবর্তে ট্রেনিং ডেটার নয়েজ (Noise) মুখস্থ করে ফেলে এবং ওভারফিট করে।

৪. কম্পিউটেশনাল বিস্ফোরণ (Computational Explosion)

ফিচারের সংখ্যা যত বেশি হবে, প্রতিটি ডেটা পয়েন্টের দূরত্ব এবং ম্যাট্রিক্স মাল্টিপ্লিকেশন হিসাব করতে কম্পিউটারের প্রসেসরের তত বেশি গাণিতিক লুপ চালাতে হয়। এতে মডেল ট্রেনিং এবং প্রেডিকশন টাইম (Latency) মারাত্মকভাবে বেড়ে যায় এবং সিস্টেমের মেমোরি ক্র্যাশ করতে পারে।

OLS vs Gradient Descent এবং PCA-এর ইন্টারভিউ টিপস (Interview & Exam Focus)

Q: লিনিয়ার রিগ্রেশনের OLS পদ্ধতি কি Curse of Dimensionality-তে আক্রান্ত হয়? A: হ্যাঁ, প্রবলভাবে আক্রান্ত হয়। OLS-এর মূল সূত্র হলো $\theta = (X^T X)^{-1} X^T y$ । যখন ফিচারের সংখ্যা (m) স্যাম্পল সংখ্যার (n) চেয়ে বেশি হয়ে যায়, তখন $X^T X$ ম্যাট্রিক্সটি Non-invertible বা Singular হয়ে যায় (অর্থাৎ এর Determinant শূন্য হয়)। ফলে ওয়ান-শটে ওজনের মান বের করা গাণিতিকভাবে অসম্ভব হয়ে পড়ে (Multicollinearity সমস্যা)।

Q: এই মাত্রার অভিশাপ থেকে মুক্তির উপায় কী? A: প্রধান দুটি উপায় হলো:

1. **Feature Selection:** ডোমেন নলেজ বা লজিক ব্যবহার করে অপ্রয়োজনীয় ফিচার ড্রপ করা।
2. **Dimensionality Reduction (যেমন PCA):** উচ্চ মাত্রার ছড়ানো-ছিটানো ডেটার ভ্যারিয়েন্সকে অক্ষুণ্ণ রেখে গাণিতিক রূপান্তরের মাধ্যমে কম মাত্রার কম্পোনেন্টে (PC1, PC2) রূপান্তর করা, যাতে তথ্যের ক্ষতি ছাড়াই মাত্রার অভিশাপ দূর হয়।

PCA এর পরিচিতি

প্রিন্সিপাল কম্পোনেন্ট অ্যানালাইসিস বা PCA হলো একটি আনসুপারভাইজড মেশিন লার্নিং অ্যালগরিদম, যা বৃহৎ ডেটাসেটের ডাইমেনশন বা ফিচারের সংখ্যা কমাতে ব্যবহৃত হয়। ডেটাসেটে যখন অনেক বেশি ভ্যারিয়েবল বা ফিচার থাকে, তখন ডেটা ভিজ্যুয়ালাইজ করা কঠিন হয়ে পড়ে, কম্পিউটেশনাল খরচ বৃদ্ধি পায় এবং মডেলটি ওভারফিটিংয়ের ঝুঁকিতে থাকে। PCA মূল উচ্চ-মাত্রিক ফিচারগুলোকে প্রিন্সিপাল কম্পোনেন্ট নামক একগুচ্ছ লম্ব অক্ষ বা অর্থোগোনাল অ্যাক্সিসে রূপান্তরিত করে এই সমস্যার সমাধান করে, যা ডেটার সর্বোচ্চ ভ্যারিয়েন্স বা তথ্যকে ধরে রেখে ফিচারের সংখ্যা কমিয়ে আনে।

মূল বিষয়সমূহ

PCA এর গাণিতিক ও তাত্ত্বিক ভিত্তি মূলত ভ্যারিয়েন্স, কোভ্যারিয়েন্স এবং জ্যামিতিক রূপান্তরের ওপর নির্ভর করে গড়ে উঠেছে।

- **ভ্যারিয়েন্স:** পরিসংখ্যানের ভাষায় ভ্যারিয়েন্স বা ভেদাঙ্ক বলতে বোঝায় ডেটা পয়েন্টগুলো তাদের গড় মান থেকে কতটা ছড়ানো-ছিটানো অবস্থায় রয়েছে। PCA ভ্যারিয়েন্সকে তথ্যের পরিমাণ হিসেবে বিবেচনা করে; যে অক্ষের ভ্যারিয়েন্স বা বিস্তৃতি সবচেয়ে বেশি, সেখানে তথ্যের পরিমাণও সবচেয়ে বেশি থাকে।
- **কোভ্যারিয়েন্স এবং কোরিলেশন:** বাস্তব জীবনের ডেটাসেটে প্রায়শই এমন অনেক ফিচার থাকে যা একে অপরের সাথে গভীরভাবে সম্পর্কিত এবং একই ধরনের তথ্য বহন করে। PCA এই ফিচারগুলোর মধ্যকার পারস্পরিক কোরিলেশন মূল্যায়নের জন্য একটি কোভ্যারিয়েন্স ম্যাট্রিক্স তৈরি করে, যা তথ্যের পুনরাবৃত্তি বা অপ্রয়োজনীয়তা দূর করতে সাহায্য করে।
- **প্রিন্সিপাল কম্পোনেন্ট:** এগুলো হলো অ্যালগরিদম দ্বারা তৈরি সম্পূর্ণ নতুন এবং পারস্পরিক সম্পর্কহীন ভ্যারিয়েবল। এর মধ্যে প্রথম প্রিন্সিপাল কম্পোনেন্টটি (PC1) ডেটাসেটের সর্বোচ্চ সম্ভাব্য ভ্যারিয়েন্স বা তথ্য ধরে রাখে, দ্বিতীয় প্রিন্সিপাল কম্পোনেন্টটি (PC2) পরবর্তী সর্বোচ্চ ভ্যারিয়েন্স ধরে রাখে এবং এভাবে বাকিগুলো কাজ করে।
- **আইগেনভ্যালু এবং আইগেনভেক্টর:** গাণিতিকভাবে PCA কোভ্যারিয়েন্স ম্যাট্রিক্সের আইগেনভেক্টর এবং আইগেনভ্যালু গণনা করে। আইগেনভেক্টরগুলো নতুন অক্ষ বা প্রিন্সিপাল কম্পোনেন্টগুলোর দিক নির্ধারণ করে, অন্যদিকে আইগেনভ্যালুগুলো সেই অক্ষের মাত্রা বা ভ্যারিয়েন্সের পরিমাণ নির্দেশ করে।

ফিচার সিলেকশন বনাম ফিচার এক্সট্রাকশন

PCA হলো একটি ফিচার এক্সট্রাকশন টেকনিক, এটি কোনো ফিচার সিলেকশন টেকনিক নয়।

- ফিচার সিলেকশন: এই পদ্ধতিতে মূল ডেটাসেটের কলামগুলোর গুরুত্বের ওপর ভিত্তি করে কিছু সুনির্দিষ্ট কলাম রেখে দেওয়া হয় অথবা বাদ দেওয়া হয়। উদাহরণস্বরূপ, ১০টি মূল কলামের মধ্যে ৩টি বাদ দিয়ে বাকি ৭টি কলামকে অবিকল আগের মতোই রেখে দেওয়া ফিচার সিলেকশনের কাজ।
- ফিচার এক্সট্রাকশন: এই পদ্ধতিটি মূল কলামগুলোকে সরাসরি বাদ দেয় না। বরং, এটি সমস্ত মূল ফিচারের একটি লিনিয়ার কম্বিনেশন বা রৈখিক সংমিশ্রণ নিয়ে সম্পূর্ণ নতুন কিছু ভ্যারিয়েবল তৈরি করে। PCA সব উচ্চ-মাত্রিক কলামের তথ্যকে সম্পূর্ণ নতুন একটি নিম্ন-মাত্রিক স্পেসে ম্যাপ করে, যার অর্থ হলো এর ফলে তৈরি হওয়া প্রিন্সিপাল কম্পোনেন্টগুলো মূলত আগের সব ফিচারের একটি নতুন মিশ্রণ।

PCA এর বৈশিষ্ট্যসমূহ

PCA এর বেশ কিছু অনন্য গাণিতিক ও আচরণগত বৈশিষ্ট্য রয়েছে, যা একটি ডেটাসেটের রূপান্তর প্রক্রিয়াকে নিয়ন্ত্রণ করে।

- আনসুপারভাইজড প্রকৃতি: PCA গাণিতিক হিসাব-নিকাশের সময় ডেটাসেটের টার্গেট ভ্যারিয়েবল বা লেবেলের ওপর নির্ভর করে না; এটি সম্পূর্ণভাবে ইনপুট ফিচারগুলোর বিন্যাস ও ছড়াছড়ির ওপর ভিত্তি করে কাজ করে।
- অর্থোগোনালিটি: তৈরি হওয়া সব প্রিন্সিপাল কম্পোনেন্ট একে অপরের সাথে নিখুঁতভাবে লম্ব বা ৯০ ডিগ্রি কোণে অবস্থান করে। এটি নিশ্চিত করে যে কম্পোনেন্টগুলো একে অপরের থেকে সম্পূর্ণ স্বাধীন বা সম্পর্কহীন, যা ডেটাসেট থেকে মাল্টিকোলিনিয়ারিটির সমস্যা পুরোপুরি দূর করে।
- তথ্য ঘনীকরণ: প্রিন্সিপাল কম্পোনেন্টগুলোকে তাদের ধরে রাখা ভ্যারিয়েন্সের পরিমাণের ওপর ভিত্তি করে ক্রমানুসারে সাজানো হয়। এর ফলে ব্যবহারকারী খুব কম তথ্য ধারণকারী শেষের কম্পোনেন্টগুলোকে নয়েজ হিসেবে বাদ দিয়ে দিতে পারেন এবং ডেটার মূল তথ্যের সিংহভাগ অক্ষুণ্ণ রাখতে পারেন।
- স্কেলের প্রতি সংবেদনশীলতা: যেহেতু PCA সর্বোচ্চ ভ্যারিয়েন্স বা বিস্তৃতির দিক বরাবর ডেটাকে প্রজেক্ট করে, তাই যেসব ফিচারের সংখ্যাগত স্কেল বা মান বড়, সেগুলো অ্যালগরিদমকে প্রভাবিত করে। এই কারণে PCA প্রয়োগ করার আগে ডেটাকে স্ট্যান্ডার্ডাইজড করা (অর্থাৎ গড় শূন্য এবং ভ্যারিয়েন্স এক-এ আনা) বাধ্যতামূলক।

কেন ব্যবহার করা হয়

ডেটা সায়েন্টিস্টরা ডেটা প্রিপেয়ার করার প্রক্রিয়াকে সহজ করতে এবং মডেলের কার্যক্ষমতা বাড়াতে PCA ব্যবহার করেন।

- মাত্রার অভিষাপ দূরীকরণ: উচ্চ মাত্রার ফিচার স্পেসে ডেটা পয়েন্টগুলো অত্যন্ত ফাঁকা ফাঁকা অবস্থায় ছড়িয়ে পড়ে, যা KNN বা SVM-এর মতো দূরত্ব-ভিত্তিক মডেলগুলোর জন্য সঠিক প্যাটার্ন খুঁজে পাওয়া কঠিন করে তোলে। PCA এই ছড়ানো-ছিটানো ডেটাকে একটি সংক্ষিপ্ত ও সুসম ফিচার স্পেসে ঘনীভূত করে নিয়ে আসে।
- ওভারফিটিং প্রতিরোধ: ডেটাসেটের অপ্রয়োজনীয় ফিচার দূর করে এবং কম ভ্যারিয়েন্স যুক্ত নয়েজ ফিল্টার করার মাধ্যমে PCA মেশিন লার্নিং মডেলের ডিসিশন বাউন্ডারিকে সহজ করে তোলে, যা মডেলের নতুন ডেটার ওপর সঠিক সিদ্ধান্ত নেওয়ার ক্ষমতা বাড়ায়।
- কম্পিউটেশনাল খরচ কমানো: কম ফিচার বা কলামের ওপর মডেল ট্রেন করলে প্রসেসিং টাইম উল্লেখযোগ্যভাবে কমে যায়, র‍্যামের ব্যবহার হ্রাস পায় এবং রিয়েল-টাইম প্রোডাকশনে মডেল অত্যন্ত দ্রুত প্রেডিকশন দিতে পারে।
- ডেটা ভিজ্যুয়ালাইজেশন সক্ষমতা: মানুষের পক্ষে সর্বোচ্চ তিনটি মাত্রা বা 3D পর্যন্ত ভিজ্যুয়ালাইজ করা সম্ভব। PCA ডজন ডজন বা শত শত ফিচার যুক্ত একটি জটিল ডেটাসেটকে মাত্র ২টি বা ৩টি প্রিন্সিপাল কম্পোনেন্টে কম্প্রেস করতে পারে, যার ফলে অ্যানালিস্টরা সহজেই গ্রাফ তৈরি করে ডেটার ভেতরের ক্লাস্টার বা প্যাটার্ন খালি চোখে দেখতে পারেন।

PCA-তে কীভাবে ফিচার বা প্রিন্সিপাল কম্পোনেন্ট সিলেক্ট করা হয়

প্রিন্সিপাল কম্পোনেন্ট অ্যানালাইসিস বা PCA-তে ফিচার সিলেক্ট করার প্রক্রিয়াটি সাধারণ ফিচার সিলেকশনের চেয়ে আলাদা। এখানে মূল ডেটাসেটের কলামগুলোকে সরাসরি বাদ দেওয়া হয় না। বরং, PCA প্রথমে সবগুলো মূল ফিচারের ভেতরের তথ্য বা ভ্যারিয়েন্স ব্যবহার করে একগুচ্ছ নতুন ফিচার বা অক্ষ তৈরি করে, যাদের আমরা PC1, PC2, PC3 এভাবে PCn পর্যন্ত নামকরণ করি।

পরবর্তী ধাপে, ডেটার আসল তথ্যের যেন কোনো বড় ক্ষতি না হয়, তা নিশ্চিত করে এই নতুন কম্পোনেন্টগুলোর মধ্য থেকে সেরা কয়েকটি কম্পোনেন্টকে বেছে নেওয়া হয় এবং বাকিগুলোকে বাদ দেওয়া হয়। নিচে কোনো গাণিতিক সমীকরণ ছাড়া এর সম্পূর্ণ তাত্ত্বিক ধারণাটি আলোচনা করা হলো:

PC1, PC2, PC3PCn যেভাবে তৈরি এবং সাজানো হয়

PCA অ্যালগরিদমটি তার তৈরি করা নতুন কম্পোনেন্টগুলোকে একটি সুনির্দিষ্ট নিয়মে সাজায়, যা নিচে ধাপে ধাপে ব্যাখ্যা করা হলো:

- PC1 (প্রথম প্রিন্সিপাল কম্পোনেন্ট): এটি হলো নতুন তৈরি হওয়া অক্ষগুলোর মধ্যে সবচেয়ে শক্তিশালী এবং গুরুত্বপূর্ণ অক্ষ। পুরো ডেটাসেটের মধ্যে যে দিক বরাবর ডেটা সবচেয়ে বেশি ছড়ানো-ছিটানো থাকে, অর্থাৎ যেখানে তথ্যের পরিমাণ বা ভ্যারিয়েন্স সবচেয়ে বেশি, PC1 ঠিক সেই দিক বরাবর তৈরি হয়। এটি একা জোড়াতালি ছাড়াই ডেটার সবচেয়ে বড় অংশের রহস্য উন্মোচন করে।
- PC2 (দ্বিতীয় প্রিন্সিপাল কম্পোনেন্ট): এটি এমন একটি দিক বরাবর তৈরি হয় যা প্রথম কম্পোনেন্ট বা PC1 এর সাথে নিখুঁতভাবে লম্ব বা ৯০ ডিগ্রি কোণে অবস্থান করে। লম্বভাবে থাকার কারণে PC1 যে তথ্য অলরেডি কভার করে ফেলেছে, PC2 তার পুনরাবৃত্তি করে না। PC1 এর তথ্য বাদ দেওয়ার পর ডেটাসেটে যে পরবর্তী সর্বোচ্চ ভ্যারিয়েন্স বা তথ্য অবশিষ্ট থাকে, PC2 সেটুকুকে নিজের ভেতর ধারণ করে।
- PC3 থেকে PCn (পরবর্তী প্রিন্সিপাল কম্পোনেন্টসমূহ): এই প্রক্রিয়াকরণটি ধারাবাহিকভাবে চলতেই থাকে। PC3 তৈরি হয় PC1 এবং PC2 উভয়ের সাথে লম্বভাবে এবং এটি বাকি থাকা তথ্যের মধ্যে পরবর্তী সর্বোচ্চ অংশটুকু লুফে নেয়। এভাবে আপনার মূল ডেটাসেটে যদি সর্বমোট ১০০টি ফিচার বা কলাম থাকে, তবে PCA ক্রমানুসারে PC1 থেকে শুরু করে PC100 (PCn) পর্যন্ত ঠিক ১০০টি নতুন কম্পোনেন্ট তৈরি করবে।
- তথ্যের ক্রমহ্রাসমান ধারা: এই পুরো শৃঙ্খলের মূল সৌন্দর্য হলো, PC1 থেকে আপনি যত সামনের দিকে (PC2, PC3... PCn) এগোতে থাকবেন, কম্পোনেন্টগুলোর তথ্য বা ভ্যারিয়েন্স ধারণ করার ক্ষমতা তত দ্রুত কমতে থাকবে। শেষের দিকের

কম্পোনেন্টগুলোতে (যেমন PC95 থেকে PC100) প্রায় কোনো আসল তথ্যই থাকে না, সেগুলো মূলত ডেটার অপ্ৰয়োজনীয় নয়জ বা কুয়াশা মাত্র।

কম্পোনেন্ট বা ফিচার সিলেক্ট করার তাত্ত্বিক পদ্ধতিসমূহ

যেহেতু PC1 থেকে PCn পর্যন্ত ক্রমান্বয়ে তথ্যের পরিমাণ কমতে থাকে, তাই আমাদের লক্ষ্য থাকে একটা নির্দিষ্ট সীমানার পর বাকি সব অপ্ৰয়োজনীয় কম্পোনেন্টগুলোকে কেটে বাদ দিয়ে দেওয়া। এই সিলেকশন মূলত ৩টি তাত্ত্বিক ধারণার ওপর ভিত্তি করে করা হয়:

- স্ক্রি প্লট তত্ত্ব (Scree Plot Theory): এটি একটি চাক্ষুষ বা ভিজ্যুয়াল পদ্ধতি। একটি গ্রাফের এক্স-অক্ষ কম্পোনেন্টগুলোর নাম (PC1, PC2...) এবং ওয়াই-অক্ষ তাদের তথ্যের পরিমাণ (Variance) বসিয়ে একটি রেখাচিত্র আঁকা হয়। গ্রাফটি দেখতে অনেকটা ওপর থেকে নিচে নেমে যাওয়া খাড়া পাহাড়ের ঢালের মতো হয়, যা হুট করে এক জায়গায় এসে সমতলের মতো সোজা হয়ে যায় (যাকে কনসেপচুয়ালি কনুই বা Elbow বলা হয়)। এই কনুই বা বাঁকটি যে পয়েন্টে তৈরি হয়, ঠিক তার আগের কম্পোনেন্ট পর্যন্ত রেখে দিয়ে বাকি ডানদিকের সব কম্পোনেন্টকে নয়জ হিসেবে বর্জন করা হয়।
- কিউমুলেটিভ ভ্যারিয়েন্স থ্রেশহোল্ড (Cumulative Variance Threshold): এই তত্ত্বটি একটি নির্দিষ্ট লক্ষ্যমাত্রা নির্ধারণের ওপর কাজ করে। ডেটা সায়েন্টিস্টরা সাধারণত সিদ্ধান্ত নেন যে তারা মূল ডেটার অন্তত ৮০%, ৯০% বা ৯৫% তথ্য অক্ষুণ্ণ রাখবেন। এরপর PC1 থেকে শুরু করে একে একে পরবর্তী কম্পোনেন্টগুলোর তথ্য যোগ করা হতে থাকে। যোগফলটি যখনই আমাদের কাঙ্ক্ষিত লক্ষ্যমাত্রায় (যেমন ৯০%) পৌঁছে যায়, ঠিক তখনই ফিচার নেওয়ার প্রক্রিয়াটি থামিয়ে দেওয়া হয়। ধরা যাক, ১০০টি ফিচারের মধ্যে প্রথম ১৫টি PC যোগ করেই ৯০% তথ্য পাওয়া গেছে, তবে ওই ১৫টি কম্পোনেন্টকেই ফাইনাল ফিচার হিসেবে সিলেক্ট করা হবে এবং বাকি ৮৫টি ফেলে দেওয়া হবে।
- কাইসারের নিয়ম (Kaiser's Criterion): এই তত্ত্বের মূল কথা হলো, একটি নতুন প্রিন্সিপাল কম্পোনেন্টকে আমাদের চূড়ান্ত ফিচার হিসেবে টিকে থাকতে হলে তার ভেতরের তথ্যের পরিমাণ অন্তত গড় তথ্যের চেয়ে বেশি হতে হবে। যে কম্পোনেন্টটি অন্তত একটি গড়পড়তা মূল ফিচারের সমান তথ্যও নিজের ভেতর ধরে রাখতে পারে না, সেটিকে সিস্টেমে রাখার কোনো যৌক্তিকতা নেই। এই নিয়ম অনুযায়ী, গড় তথ্যের চেয়ে কম স্কোর পাওয়া শেষের সব কম্পোনেন্টকে স্বয়ংক্রিয়ভাবে ফিচার লিস্ট থেকে ছাঁটাই করা হয়।

PCA যেভাবে কাজ করে (Working of PCA)

একটি উচ্চ-মাত্রিক ডেটাসেটকে পারস্পরিক সম্পর্কহীন প্রিন্সিপাল কম্পোনেন্টে রূপান্তরিত করার জন্য PCA অ্যালগরিদমটি ধাপে ধাপে একটি সুনির্দিষ্ট প্রক্রিয়ার মধ্য দিয়ে কাজ করে। এর মূল লক্ষ্য হলো ডেটার সর্বোচ্চ ভ্যারিয়েন্স বা বিস্তৃতির দিকগুলো খুঁজে বের করা এবং ডেটাসেটকে সেই নতুন স্থানাঙ্ক বা কোঅর্ডিনেটসের ওপর ঘুরিয়ে দেওয়া।

ধাপ ১: স্ট্যান্ডার্ডাইজেশন (Standardization)

PCA-এর কাজ শুরু করার প্রথম পদক্ষেপ হলো এটি নিশ্চিত করা যেন মূল ডেটাসেটের সব ফিচার তাদের পরিমাপের একক নির্বিশেষে সমানভাবে অবদান রাখতে পারে।

- ভ্যারিয়েন্সের আধিপত্য: ডেটাসেটে যদি এমন কোনো ফিচার থাকে যার সংখ্যাগত স্কেল হাজার বা লক্ষের ঘরে (যেমন- বেতন) এবং অন্য একটি ফিচারের স্কেল যদি একক ঘরে হয় (যেমন- বয়স), তবে বড় সংখ্যার ফিচারটির ভ্যারিয়েন্স স্বাভাবিকভাবেই অনেক বেশি দেখাবে।
- সমাধান: PCA সঠিকভাবে কাজ করার জন্য ডেটাকে স্ট্যান্ডার্ডাইজড করা প্রয়োজন। এই প্রক্রিয়ায় প্রতিটি কলামের গড় মানকে শিফট করে একদম শূন্য (Zero) করে দেওয়া হয় এবং ডেটাকে এমনভাবে স্কেল করা হয় যাতে প্রতিটি কলামের ভ্যারিয়েন্সের মান এক (One) হয়ে যায়। এর ফলে সব ফিচার সম্পূর্ণ সমান একটি অবস্থানে চলে আসে।

ধাপ ২: কোভ্যারিয়েন্স ম্যাট্রিক্স তৈরি (Covariance Matrix Construction)

ডেটা সেন্টারড এবং স্কেল করার পর, অ্যালগরিদমটি বিভিন্ন কলামের মধ্যকার পারস্পরিক সম্পর্কগুলো খোঁজার চেষ্টা করে।

- তথ্যের পুনরাবৃত্তি শনাক্তকরণ: এর মূল উদ্দেশ্য হলো ভ্যারিয়েবলগুলো একে অপরের সাথে কীভাবে পরিবর্তিত হচ্ছে তা দেখা। PCA একটি বর্গাকার ম্যাট্রিক্স তৈরি করে যা কোভ্যারিয়েন্স ম্যাট্রিক্স নামে পরিচিত। এটি সম্ভাব্য প্রতিটি ফিচারের জোড়ার মধ্যকার পারস্পরিক কোরিলেশন ম্যাপ করে।
- ডায়াগোনাল বনাম অফ-ডায়াগোনাল: এই ম্যাট্রিক্সের ডায়াগোনাল বা কোণাকুণি উপাদানগুলো প্রতিটি একক ফিচারের নিজস্ব ভ্যারিয়েন্স নির্দেশ করে, আর অফ-ডায়াগোনাল উপাদানগুলো ভিন্ন ভিন্ন ফিচারের মধ্যকার কোভ্যারিয়েন্স দেখায়। অফ-ডায়াগোনাল উপাদানের মান অনেক বেশি হওয়ার অর্থ হলো ফিচার দুটি রিডান্ডেন্ট বা অতিরিক্ত, অর্থাৎ তারা প্রায় একই ধরনের তথ্য বহন করছে।

ধাপ ৩: প্রিন্সিপাল কম্পোনেন্ট গণনা (Eigen Decomposition)

এটি PCA-এর সবচেয়ে গুরুত্বপূর্ণ তাত্ত্বিক ধাপ যেখানে ডেটার নতুন স্ট্রাকচারাল অক্ষ বা ডাইমেনশনগুলো আবিষ্কৃত হয়।

- দিক বা অক্ষ খুঁজে নেওয়া: অ্যালগরিদমটি কোভ্যারিয়েন্স ম্যাট্রিক্সকে ভেঙে তার আইগেনভেক্টর এবং আইগেনভ্যালুগুলো খুঁজে বের করে।
- দিক হিসেবে আইগেনভেক্টর: আইগেনভেক্টরগুলো নতুন অক্ষের দিক বা ডিরেকশন নির্দেশ করে। প্রথম আইগেনভেক্টরটি ডেটা ক্লাউডের সর্বোচ্চ ভ্যারিয়েন্স বা বিস্তৃতির দিক বরাবর সরাসরি নিশানা করে একটি বেস্ট-ফিট লাইন তৈরি করে। এই দিকটিই পরবর্তীতে PC1 হিসেবে আত্মপ্রকাশ করে।
- গুরুত্বের স্কের হিসেবে আইগেনভ্যালু: প্রতিটি আইগেনভেক্টরের একটি নির্দিষ্ট আইগেনভ্যালু থাকে। এই আইগেনভ্যালুটি একটি স্কের হিসেবে কাজ করে যা বলে দেয় ওই নির্দিষ্ট দিক বরাবর ঠিক কতটুকু ভ্যারিয়েন্স বা তথ্য মডেলটি ধরে রাখতে পেরেছে।

ধাপ ৪: অর্থোগোনালিটি বা লম্বের শর্ত আরোপ (PC2 থেকে PCn)

তথ্যের প্রধান অক্ষ বা PC1 নির্ধারণ করার পর, PCA একটি কঠোর জ্যামিতিক শর্তের অধীনে বাকি কম্পোনেন্টগুলো তৈরি করে।

- শূন্য কোরিলেশনের শর্ত: অ্যালগরিদমটি দ্বিতীয় দিকটি (PC2) এমনভাবে নির্ধারণ করে যাতে এটি PC1 এর সাথে নিখুঁতভাবে লম্ব বা ৯০ ডিগ্রি কোণে অবস্থান করে।
- অবশিষ্ট ভ্যারিয়েন্সের সর্বোচ্চকরণ: PC1 এর সাথে লম্ব থাকার পাশাপাশি, PC2 নিজেকে ডেটা ক্লাউডের মধ্যে অবশিষ্ট থাকা পরবর্তী সর্বোচ্চ বিস্তৃতি বা ভ্যারিয়েন্সের দিক বরাবর বিন্যস্ত করে।
- ধারাবাহিক লুপ: এই প্রক্রিয়াটি ক্রমানুসারে PC3, PC4 থেকে শুরু করে PCn পর্যন্ত চলতেই থাকে। যেহেতু প্রতিটি নতুন কম্পোনেন্টকে তার পূর্ববর্তী সব কম্পোনেন্টের সাথে লম্ব হতে হয়, তাই চূড়ান্ত অক্ষগুলোর নিজেদের মধ্যে কোনো পারস্পরিক কোরিলেশন বা সম্পর্ক থাকে না, যা মাল্টিকোলিনিয়ারিটির সমস্যাকে পুরোপুরি নিশ্চিহ্ন করে দেয়।

ধাপ ৫: নতুন স্পেসে ডেটা প্রজেক্ট করা বা রূপান্তর (Transformation)

কাজের সর্বশেষ ধাপে মূল ডেটা পয়েন্টগুলোকে নতুনভাবে প্রতিষ্ঠিত প্রিন্সিপাল কম্পোনেন্টের স্থানাঙ্কের ওপর ঘুরিয়ে দেওয়া বা প্রজেক্ট করা হয়।

- স্থানাঙ্ক পরিবর্তন: মূল ডেটা পয়েন্টগুলোকে গাণিতিকভাবে নতুন আইগেনভেক্টরের ওপর প্রজেক্ট বা প্রতিস্থাপন করা হয়। এর ফলে মূল ফিচারের কলামগুলো বাতিল হয়ে যায় এবং

ডেটার প্রতিটি সারি এখন PC1, PC2 ইত্যাদি নতুন অক্ষের কোঅর্ডিনেট ভ্যালু দ্বারা সংজ্ঞায়িত হয়।

- ডাইমেনশন সিলেকশন: পূর্বে আলোচিত ফিচার সিলেকশনের তত্ত্বগুলো (যেমন স্ক্রি প্লট বা কিউমুলেটিভ ভ্যারিয়েন্স থ্রেশহোল্ড) ব্যবহার করে শুধুমাত্র সর্বোচ্চ পারফর্ম করা প্রিন্সিপাল কম্পোনেন্টগুলোকে রেখে দেওয়া হয়। কম আইগেনভ্যালু বিশিষ্ট শেষের দিকের দুর্বল কম্পোনেন্টগুলোকে স্থায়ীভাবে ফেলে দেওয়া হয়। এর ফলে একটি সংকুচিত, পরিচ্ছন্ন এবং অত্যন্ত অপ্টিমাইজড ডেটাসেট তৈরি হয়, যা মেশিন লার্নিং মডেলের জন্য সম্পূর্ণ প্রস্তুত।

PCA এর সিমুলেশন: একটি সম্পূর্ণ তাত্ত্বিক উদাহরণ (Simulations of PCA: A Complete Theoretical Example)

বাস্তবে PCA কীভাবে কাজ করে তা বোঝার জন্য, আসুন গাড়ি পারফরম্যান্স ট্র্যাকিংয়ের একটি কাল্পনিক ডেটাসেটের সাহায্যে পুরো বিষয়টি ভিজ্যুয়ালাইজ বা কল্পনা করি।

ডেটাসেট (উচ্চ-মাত্রিক স্পেস)

ধরা যাক, আমাদের কাছে ৫০০টি ভিন্ন গাড়ির তথ্য সম্বলিত একটি ডেটাসেট আছে। প্রতিটি গাড়ির জন্য আমরা মূলত ২টি প্রধান ফিচার ট্র্যাক করছি:

- ইঞ্জিনের আকার (Engine Size): যা লিটার বা সিসি (cc) এককে মাপা হয়।
- হর্সপাওয়ার (Horsepower): ইঞ্জিনের কাঁচা শক্তির আউটপুট।

এই দুটি ভ্যারিয়েবলকে একটি সাধারণ দ্বিমাত্রিক গ্রিডে প্লট করা হলো, যেখানে আনুভূমিক অক্ষটি (Horizontal Axis) ইঞ্জিনের আকার এবং উল্লম্ব অক্ষটি (Vertical Axis) হর্সপাওয়ার নির্দেশ করে।

ধাপ ১: ডেটা ক্লাউডকে সেন্টারিং করা

যখন মূল পয়েন্টগুলোকে গ্রাফে বসানো হয়, তখন ডেটা ক্লাউডটি তার মূল সংখ্যাগত গড়ের ওপর ভিত্তি করে গ্রিডের যেকোনো একপাশে অবস্থান করে। PCA প্রথমে এই পুরো স্পেসটিকে শিফট বা স্থানান্তরিত করে।

- অক্ষ পরিবর্তন: অ্যালগরিদমটি প্রথমে ৫০০টি গাড়ির গড় ইঞ্জিনের আকার এবং গড় হর্সপাওয়ার হিসাব করে।
- অরিজিন বা মূলবিন্দু প্রতিষ্ঠা: এরপর এটি পুরো ডেটা ক্লাউডটিকে এমনভাবে সরিয়ে নেয় যাতে এই গড় পয়েন্টটি ঠিক গ্রাফের কেন্দ্র বা (০,০) স্থানান্ত্রে বসে। গাড়ির ডেটার মূল আকৃতি বা বিন্যাসের কোনো পরিবর্তন হয় না; এটি শুধু এমনভাবে স্থানান্তরিত হয় যেন ডেটা ক্লাউডের কেন্দ্রটিই গ্রাফের মূল কেন্দ্রবিন্দুতে পরিণত হয়।

ধাপ ২: প্রথম অক্ষ বা PC1 অঙ্কন

ডেটা ক্লাউডটি কেন্দ্রের সাথে মিলে যাওয়ার পর, অ্যালগরিদমটি সর্বোচ্চ বিস্তৃতি বা ভ্যারিয়েন্সের দিকটি খুঁজতে শুরু করে।

- বেস্ট-ফিট লাইন: PCA মূলবিন্দু (০,০) দিয়ে অতিক্রমকারী একটি সোজা রেখাকে ততক্ষণ পর্যন্ত ঘুরাতে থাকে, যতক্ষণ না এটি ডেটা ক্লাউডের সবচেয়ে লম্বা বা বিস্তৃত অংশের সাথে নিখুঁতভাবে মিলে যায়।

- ভ্যারিয়েন্স ক্যাপচার: আমাদের এই গাড়ির ডেটাসেটে, ইঞ্জিনের আকার যত বাড়ে, হর্সপাওয়ারও সাধারণত তত বৃদ্ধি পায়। ফলে বেস্ট-ফিট লাইনটি কোণাকুণিভাবে ওপরের দিকে হেলে থাকবে। এই কোণাকুণি দিকটিই হলো সেই পথ যেখানে ডেটা পয়েন্টগুলো সবচেয়ে বেশি ছড়িয়ে আছে।
- PC1 চূড়ান্তকরণ: এই কোণাকুণি রেখাটিই প্রথম প্রিন্সিপাল কম্পোনেন্ট (PC1) হিসেবে নির্ধারিত হয়। অ্যালগরিদমটি এই লাইনের দিক নির্দেশকারী একটি ওজন (আইগেনভেক্টর) এবং এটি কত বিশাল পরিমাণ ভ্যারিয়েন্স ধরে রেখেছে তার একটি স্কোর (আইগেনভ্যালু) বরাদ্দ করে।

ধাপ ৩: দ্বিতীয় অক্ষ বা PC2 অঙ্কন

প্রধান কোণাকুণি ট্রেন্ডটি ক্যাপচার করার পর, PCA একটি কঠোর জ্যামিতিক নিয়মের অধীনে পরবর্তী অক্ষটি তৈরি করে।

- লম্বের শর্ত: অ্যালগরিদমটি মূলবিন্দু দিয়ে আরেকটি রেখা টানে যা PC1 এর সাথে নিখুঁতভাবে নব্বই ডিগ্রি (৯০°) কোণে অবস্থান করে।
- অবশিষ্ট বৈচিত্র্য ক্যাপচার: যেহেতু এই অক্ষটি লম্ব, তাই এটি PC1 এর সাথে সম্পূর্ণ সম্পর্কহীন বা আন-কোরিলেটেড। আমাদের গাড়ির সিমুলেশনে PC2 কোণাকুণি নিচের দিকে নির্দেশ করবে। এটি সেই ছোটখাটো বৈচিত্র্যগুলোকে ক্যাপচার করে যা PC1 মিস করেছিল—যেমন কোনো একটি সুনির্দিষ্ট গাড়ির হর্সপাওয়ার তার ইঞ্জিনের আকারের তুলনায় সামান্য কম বা বেশি হওয়া।

ধাপ ৪: রোটেশন এবং ডাইমেনশনালিটি রিডাকশন

সিমুলেশনের শেষ ধাপে ব্যবহারকারীর দেখার দৃষ্টিকোণ পরিবর্তন করা হয় এবং স্পেসটিকে সংকুচিত করা হয়।

- গ্রাফ ঘোরানো: অ্যালগরিদমটি পুরো গ্রাফটিকে এমনভাবে ঘুরিয়ে দেয় যাতে PC1 সমতল আনুভূমিক অক্ষে পরিণত হয় এবং PC2 উল্লম্ব অক্ষে পরিণত হয়। ইঞ্জিনের আকার এবং হর্সপাওয়ারের মূল কলামগুলো এবার বাতিল হয়ে যায়।
- দুর্বল কম্পোনেন্ট বাদ দেওয়া: যদি সিমুলেশনে দেখা যায় যে PC1 পুরো ডেটার ৯৫% ভ্যারিয়েন্স ধারণ করেছে এবং PC2 মাত্র ৫% ধারণ করেছে, তবে আমরা PC2 কে সম্পূর্ণ বাদ দিয়ে দিতে পারি। এর ফলে দ্বিমাত্রিক ডেটা ক্লাউডটি কোনো বড় তথ্যের ক্ষতি ছাড়াই মাত্র ১টি একমাত্রিক লাইনে (PC1) চ্যাপ্টা বা সংকুচিত হয়ে যায়, যা সফলভাবে PCA রূপান্তর সম্পন্ন করে।

যখন PCA খুব ভালো কাজ করে (When PCA Works Well)

ডেটার সুনির্দিষ্ট কিছু কাঠামোগত শর্তের অধীনে PCA সবচেয়ে সেরা ফলাফল দেয়।

- **লিনিয়ার সম্পর্ক:** PCA সম্পূর্ণভাবে লিনিয়ার বা রৈখিক রূপান্তরের ওপর ভিত্তি করে কাজ করে। যদি আপনার মূল ফিচারগুলো একটি ধ্রুবক বা সরলরৈখিক হারে একে অপরের সাথে পরিবর্তিত হয়, তবে PCA খুব সহজেই ভ্যারিয়েন্সের আসল দিকগুলো চিহ্নিত করতে পারে।
- **উচ্চ কোরিলেশন এবং তথ্যের পুনরাবৃত্তি:** যখন একটি ডেটাসেটে ডজন ডজন ফিচার থাকে যা একে অপরের সাথে গভীরভাবে সম্পর্কিত (মাল্টিকোলিনিয়ারিটি), তখন PCA সেই পুনরাবৃত্তিমূলক তথ্যগুলোকে মাত্র কয়েকটি স্বাধীন প্রিন্সিপাল কম্পোনেন্টে সংকুচিত করতে চমৎকার কাজ করে।
- **কন্টিনিউয়াস নিউমেরিক ডেটা:** যেহেতু এই অ্যালগরিদমটি ভ্যারিয়েন্স এবং গড় গণনার ওপর সম্পূর্ণ নির্ভরশীল, তাই এটি কন্টিনিউয়াস নিউমেরিক ডেটার ক্ষেত্রে অসাধারণ কাজ করে যেখানে স্পেশাল দূরত্বগুলোর একটি বাস্তব অর্থ থাকে।
- **উচ্চ শূন্যতা এবং নয়েজ:** ডেটাসেটে যদি অনেক কম-ভ্যারিয়েন্স যুক্ত এবং নয়েজি বা অর্থহীন কলাম থাকে, তবে PCA অত্যন্ত কার্যকর। এটি আসল তথ্যগুলোকে ওপরের দিকের কম্পোনেন্টগুলোতে ঘনীভূত করে এবং শেষের কম্পোনেন্টগুলোতে অপ্রয়োজনীয় নয়েজ জমা করে, যা পরে ফেলে দেওয়া যায়।

যখন PCA ব্যর্থ হয় (When PCA Fails)

নিচের পরিস্থিতিগুলোতে PCA দুর্বল ফলাফল দেয় অথবা তথ্যকে বিকৃত করে ফেলে।

- **নন-লিনিয়ার ডেটা স্ট্রাকচার:** যদি ফিচারগুলোর মধ্যকার সম্পর্ক কোনো জটিল বা নন-লিনিয়ার প্যাটার্ন তৈরি করে—যেমন একটি নিখুঁত বৃত্ত, তরঙ্গ বা সর্পিলাকৃতি, তবে PCA সম্পূর্ণ ব্যর্থ হবে। যেহেতু এটি শুধুমাত্র সর্বোচ্চ ভ্যারিয়েন্সের সরলরেখা খোঁজে, তাই এটি একটি বৃত্তের ঠিক মাঝখান দিয়ে একটি সোজা দাগ টেনে দেবে, যা ডেটার আসল জ্যামিতিক কাঠামোকে সম্পূর্ণ মিস করবে।
- **মিশ্র ডেটা টাইপ (ক্যাটাগরিক্যাল ভ্যারিয়েবল):** PCA মূলত ক্যাটাগরিক্যাল কলামের (যেমন- হ্যাঁ/না, লাল/নীল/সবুজ) জন্য ডিজাইন করা হয়নি। বিচ্ছিন্ন ক্যাটাগরির মধ্য দিয়ে গড়, ভ্যারিয়েন্স হিসাব করা বা একটি কন্টিনিউয়াস লম্ব অক্ষ টানা ধারণাগতভাবে ভুল এবং এটি স্থানাঙ্কের রূপান্তরকে নষ্ট করে ফেলে।
- **স্কেল না করা বা আন-স্ট্যান্ডার্ডাইজড ডেটা:** অ্যালগরিদমটি চালানোর আগে ডেটা স্ট্যান্ডার্ডাইজেশন না করলে PCA পুরোপুরি ব্যর্থ হয়। বিশাল সংখ্যার একটি ফিচার শুধুমাত্র তার স্কেলের কারণে ভ্যারিয়েন্সের প্রধান উৎস হিসেবে গণ্য হবে, যা আসল তথ্য নির্বিশেষে প্রিন্সিপাল কম্পোনেন্টগুলোকে নিজের দিকে টেনে নেবে।

- আউটলায়ারের হস্তক্ষেপ: যেহেতু PCA ভ্যারিয়েন্স ব্যবহার করে (যা গড় থেকে বর্গের পার্থক্যের ওপর নির্ভর করে), তাই চরম আউটলায়ার বা ব্যতিক্রমী বড়/ছোট মানগুলো বেস্ট-ফিট লাইনটিকে নিজের দিকে তীব্রভাবে আকর্ষণ করে। মাত্র কয়েকটি মারাত্মক আউটলায়ার পুরো PC1 কে ডেটা ক্লাউডের আসল ঘন বিন্যাস থেকে দূরে সরিয়ে দিতে পারে।

PCA-এর হাইপারপ্যারামিটারসমূহ (Scikit-Learn Hyperparameters)

sklearn.decomposition.PCA মডেলটির ভেতরের কার্যপ্রক্রিয়া নিয়ন্ত্রণ করতে, ওভারফিটিং রোধ করতে এবং কম্পিউটেশনাল সময় বাঁচাতে নিচের প্যারামিটারগুলো ব্যবহার করা হয়। গুরুত্বের ওপর ভিত্তি করে এদের ক্রমানুসারে আলোচনা করা হলো:

n_components

- **Default:** None
- **ব্যবহার এবং কার্যপদ্ধতি:** এটি র্যান্ডম ফরেষ্ট বা ডিসিশন ট্রির মেকানিজমের মতো একটি মডেলের জটিলতা নিয়ন্ত্রণের প্রধান ব্রেক হিসেবে কাজ করে। এটি নির্ধারণ করে PCA রূপান্তরের পর আপনি চূড়ান্তভাবে সর্বমোট কতটি প্রিন্সিপাল কম্পোনেন্ট বা ফিচার রেখে দিতে চান। যদি 'None' রাখা হয়, তবে এটি মূল ফিচারের সংখ্যার সমান (অর্থাৎ PC_1 থেকে PC_n পর্যন্ত সবকটি) কম্পোনেন্ট তৈরি করে। ডাইমেনশনালিটি রিডাকশন বা মাত্রার অভিশাপ দূর করার জন্য বাস্তব প্রজেক্টে এই প্যারামিটারটি সবচেয়ে বেশি টিউন করা হয়।
- **অন্যান্য অপশন:**
 - যেকোনো পূর্ণসংখ্যা (int): যেমন n_components=2 বা 3 দিলে মডেলটি শুধুমাত্র শীর্ষ ২টি বা ৩টি প্রধান কম্পোনেন্ট রেখে বাকিগুলো ফেলে দেয়, যা ডেটা ভিজুয়ালাইজেশনের জন্য চমৎকার কাজ করে।
 - শূন্য থেকে একের মধ্যকার ভগ্নাংশ (float): যেমন n_components=0.95 দিলে মডেলটি কিউমুলেটিভ ভ্যারিয়েন্স থ্রেশহোল্ড তত্ত্ব ব্যবহার করে স্বয়ংক্রিয়ভাবে ততগুলো কম্পোনেন্ট বেছে নেয় যা মূল ডেটার অন্তত ৯৫% তথ্য বা ভ্যারিয়েন্স ধরে রাখতে সক্ষম।

svd_solver

- **Default:** 'auto'
- **ব্যবহার এবং কার্যপদ্ধতি:** এটি ব্যাকএন্ডে আইগেন ডিকম্পজিশন বা Singular Value Decomposition (SVD) হিসাব করার সুনির্দিষ্ট গাণিতিক অ্যালগরিদম নির্বাচন করে। ডিফল্ট 'auto' থাকলে Scikit-Learn ডেটাসেটের আকার এবং ফিচারের সংখ্যার ওপর ভিত্তি করে স্বয়ংক্রিয়ভাবে সবচেয়ে দ্রুতগতির ও সঠিক Solver টি বেছে নেয়। এটি মডেলের এক্যুরেসিতে বড় পরিবর্তন না আনলেও কম্পিউটেশনাল সময় ও মেমোরি খরচে বিশাল প্রভাব ফেলে।
- **অন্যান্য অপশন:**

- 'full': এটি OLS সলিউশনের মতো একবারে সম্পূর্ণ প্রথাগত SVD গণনা করে। ডেটাসেট ছোট বা মাঝারি হলে এবং নিখুঁত গাণিতিক ফলাফলের প্রয়োজন হলে এটি ব্যবহৃত হয়।
- 'arpack': এটি একটি ইটারেটিভ পদ্ধতি যা লুপের মাধ্যমে কাজ করে। যখন ডেটাসেট মাঝারি আকৃতির হয় এবং আপনি খুব কম সংখ্যক কম্পোনেন্ট ($n_components \ll n_features$) এক্সট্রাক্ট করতে চান, তখন এটি দ্রুত কনভার্স করে।
- 'randomized': এটি একটি ট্রান্সফ্রেমড বা র্যান্ডমাইজড SVD অ্যালগরিদম ব্যবহার করে। যখন ডেটাসেট বিশাল আকৃতির হয় (যেমন- লাখ লাখ রো এবং হাজার হাজার কলাম) এবং আপনি অল্প কয়েকটি মেইন কম্পোনেন্ট চান, তখন এটি নিখুঁত মান খোঁজার বদলে দ্রুত একটি আনুমানিক সঠিক ফলাফল দেয়, যা প্রসেসিং টাইম বহুগুণ বাঁচায়।

whiten

- **Default:** False
- **ব্যবহার এবং কার্যপদ্ধতি:** এটিকে ডেটার ওজনের ওপর এক ধরনের অতিরিক্ত রেগুলারাইজেশন বা ব্রেক বলা যায়। এটি প্রতিটি প্রিন্সিপাল কম্পোনেন্টের আউটপুটকে তাদের নিজস্ব আইগেনভ্যালু বা স্কেরের বর্গমূল দিয়ে ভাগ করে দেয়। এর ফলে প্রতিটি কম্পোনেন্টের নিজস্ব বৈচিত্র্যের মান ফিক্সড হয়ে যায়। এটি ডাউনস্ট্রিম মডেলের (যেমন লিনিয়ার মডেল বা সাপোর্ট ভেক্টর মেশিন) স্ট্যাবিলিটি বাড়াতে সাহায্য করে।
- **অন্যান্য অপশন:**
 - True: যখন PCA করার পর এই নতুন ফিচারগুলোকে সরাসরি কোনো নিউরাল নেটওয়ার্ক বা SVM মডেলে ইনপুট হিসেবে পাঠানো হয়, তখন এটি ব্যবহার করা হয়। এটি নিশ্চিত করে যে প্রতিটি ফিচার যেন নিখুঁতভাবে একই স্কেলে থাকে, যা ডাউনস্ট্রিম অ্যালগরিদমের অপটিমাইজেশন প্রক্রিয়াকে আরও দ্রুত ও নির্ভুল করে।

random_state

- **Default:** None
- **ব্যবহার এবং কার্যপদ্ধতি:** এটি কোডের আউটপুটকে প্রতিবার একই রকম বা রিপ্ৰোডিউসিবল (Reproducible) রাখার জন্য ব্যবহৃত হয়। এটি মূলত তখনই কাজ করে যখন `svd_solver='randomized'` বা `'arpack'` মোডে চলে, কারণ এই অ্যালগরিদমগুলো শুরুতে কিছু র্যান্ডম অপারেশন চালায়।

- **অন্যান্য অপশন:**

- যেকোনো ফ্লোয়িং পূর্ণসংখ্যা (যেমন: 0, 42): অ্যাসাইনমেন্ট, পরীক্ষা বা টিম প্রজেক্টে ট্র্যাকিং করার সময় এটি ব্যবহার করা বাধ্যতামূলক। এটি নিশ্চিত করে যে কোডটি যতবারই রান করা হোক না কেন, প্রতিবার অবিকল একই নতুন অক্ষ এবং একই কোঅর্ডিনেটসের মান পাওয়া যাবে।

copy

- **Default:** True

- **ব্যবহার এবং কার্যপদ্ধতি:** এটি মেমোরি সাশ্রয় ও ডেটা সুরক্ষার একটি ব্যাকএন্ড মেকানিজম। ডিফল্ট True থাকলে মডেলটি মূল ইনপুট ডেটার একটি কপি তৈরি করে তার ওপর স্ট্যান্ডার্ডাইজেশন এবং অক্ষ পরিবর্তনের কাজগুলো চালায়, যার ফলে আপনার মূল ডেটাসেটটি মেমোরিতে সম্পূর্ণ অপরিবর্তিত ও সুরক্ষিত থাকে।

- **অন্যান্য অপশন:**

- False: যখন ডেটাসেটের আকার এত বিশাল যে মেমোরিতে অতিরিক্ত কপি তৈরি করলে র‍্যাম ক্র্যাশ করার ঝুঁকি থাকে, তখন এটি ব্যবহার করা হয়। এটি সরাসরি মূল ডেটার ওপর ওভাররাইট (In-place transformation) করে কাজ করে, যা মেমোরি বাঁচালেও মূল ডেটা পরিবর্তন করে ফেলে।

tol

- **Default:** 0.0

- **ব্যবহার এবং কার্যপদ্ধতি:** এটি মূলত `svd_solver='arpack'` ব্যবহারের সময় ইটারেশন বা লুপ থামানোর একটি টলারেন্স থ্রেশহোল্ড বা কনভার্জেন্স সীমা হিসেবে কাজ করে। ডিফল্ট 0.0 থাকলে এটি সিস্টেমের নিখুঁত সীমানা পর্যন্ত গণনা চালিয়ে যায়।

- **অন্যান্য অপশন:**

- যেকোনো ছোট ফ্লোট সংখ্যা (যেমন: $1e-4$): যখন অনেক বড় ডেটাসেটে 'arpack' সলভারটি নিখুঁত মান খুঁজতে গিয়ে অতিরিক্ত সময় নিচ্ছে, তখন টলারেন্স সামান্য বাড়িয়ে দিয়ে দ্রুত ট্রেনিং শেষ করার জন্য এটি টিউন করা হয়।

iterated_power

- **Default:** 'auto'

- **ব্যবহার এবং কার্যপদ্ধতি:** এটি শুধুমাত্র তখনই কাজ করে যখন `svd_solver='randomized'` সেট করা থাকে। এটি র‍্যান্ডমাইজড SVD অ্যালগরিদমের

পাওয়ার ইটারেশনের সংখ্যা বা লুপের সংখ্যা নিয়ন্ত্রণ করে আরও নিখুঁত আইগেনভেক্টর খুঁজে পেতে সাহায্য করে।

- **অন্যান্য অপশন:**

- যেকোনো পজিটিভ পূর্ণসংখ্যা: টেক্সট মাইনিং বা এনএলপি (NLP) এর মতো অতি বিশাল এবং অত্যন্ত নয়েজি স্পার্স ডেটাসেটে আনুমানিক ফাস্ট রেজাল্টের এক্যুরেসী আরও কিছুটা বাড়াতে এই মানটি ম্যানুয়ালি সেট করা হয় (যেমন ৪ বা ৭)।

n_oversamples

- **Default:** 10

- **ব্যবহার এবং কার্যপদ্ধতি:** এটিও শুধুমাত্র `svd_solver='randomized'` মোডের জন্য প্রযোজ্য। র্যান্ডমাইজড SVD হিসাব করার সময় তথ্যের কোনো বড় ক্ষতি যেন না হয়, সেজন্য এটি লক্ষ্যমাত্রার চেয়ে অতিরিক্ত কিছু ওভারস্যাম্পল নিয়ে কাজ করে জ্যামিতিক স্থায়িত্ব বাড়ায়।

- **অন্যান্য অপশন:**

- যেকোনো পূর্ণসংখ্যা: যখন অত্যন্ত সংবেদনশীল ডেটা (যেমন মেডিকেল ইমেজ) নিয়ে কাজ করা হয় এবং র্যান্ডমাইজড সলভারের অভ্যন্তরীণ ট্রাঙ্কেশন এরর সর্বনিম্ন করতে চাওয়া হয়, তখন এই ওভারস্যাম্পলের মান বাড়িয়ে দেওয়া হয়।

power_iteration_normalizer

- **Default:** 'auto'

- **ব্যবহার এবং কার্যপদ্ধতি:** এটি শুধুমাত্র `svd_solver='randomized'` মোডের একটি অভ্যন্তরীণ গাণিতিক নরমালিসেশন টেকনিক। এটি পাওয়ার ইটারেশনের প্রতিটি ধাপে ম্যাট্রিক্সের সংখ্যাগত স্থিতিশীলতা বজায় রাখে, যাতে কোনো গাণিতিক ওভারফ্লো না ঘটে।

- **অন্যান্য অপশন:**

- 'QR', 'LU', 'none': অতি সুনির্দিষ্ট গাণিতিক এক্সপেরিমেন্ট বা গবেষক লেভেলের কাজে ম্যাট্রিক্সের Numerical Stability নিখুঁতভাবে পরীক্ষা করার জন্য এগুলো ম্যানুয়ালি পরিবর্তন করা হয়, সাধারণ প্রজেক্টে ডিফল্ট রাখাই স্ট্যান্ডার্ড।

PCA-এর মূল অ্যাট্রিবিউটসমূহ (Scikit-Learn Attributes)

মডেল ডট ফিট (model.fit(X_train)) করার পর, অ্যালগরিদমটি ব্যাকএন্ডের গাণিতিক হিসাব-নিকাশ শেষে মেমোরিতে যে সুনির্দিষ্ট আউটপুট বা ফলাফলগুলো জমা করে রাখে, সেগুলোকে অ্যাট্রিবিউট বলা হয়। Scikit-Learn-এর নিয়ম অনুযায়ী এদের প্রতিটির শেষে একটি আন্ডারস্কোর () থাকে। গুরুত্বের ওপর ভিত্তি করে এদের ক্রমানুসারে আলোচনা করা হলো:

components_

- **আকৃতি (Shape):** (n_components, n_features)
- **ব্যবহার এবং কার্যপদ্ধতি:** এটি হলো PCA-এর সবচেয়ে গুরুত্বপূর্ণ এবং মূল আউটপুট। এটি ডেটাসেটের ফিচার স্পেসের প্রধান অক্ষ বা প্রিন্সিপাল কম্পোনেন্টগুলোর দিক নির্দেশ করে, যেখানে ডেটার ভ্যারিয়েন্স বা বিস্তৃতি সবচেয়ে বেশি থাকে। গাণিতিক ভাষায়, এগুলো হলো সেন্ট্রালাইজড ইনপুট ডেটার আইগেনভেক্টর বা রাইট সিঙ্গুলার ভেক্টর (Right Singular Vectors)। এই কম্পোনেন্টগুলো তাদের ধরে রাখা তথ্যের পরিমাণ অনুযায়ী বড় থেকে ছোট ক্রমে (explained_variance_ এর ওপর ভিত্তি করে) সাজানো থাকে। এর সাহায্যে আমরা বুঝতে পারি যে মূল ফিচারগুলোর কোন কোন অংশ মিলে নতুন অক্ষগুলো তৈরি হয়েছে।

explained_variance_ratio_

- **আকৃতি (Shape):** (n_components,)
- **ব্যবহার এবং কার্যপদ্ধতি:** বাস্তব জীবনের প্রজেক্টে কতটি ফিচার ফাইনালি সিলেক্ট করা হবে, তা নির্ধারণ করার জন্য এই অ্যাট্রিবিউটটি সবচেয়ে বেশি ব্যবহৃত হয়। এটি প্রতিটি প্রিন্সিপাল কম্পোনেন্ট মূল ডেটাসেটের সর্বমোট তথ্যের কত শতাংশ (Percentage) নিজের ভেতর ধরে রাখতে পেরেছে তা শতকরা আকারে প্রকাশ করে। যদি n_components প্যারামিটারটি সেট করা না থাকে, তবে সবকটি কম্পোনেন্টের রেশিও এখানে জমা হয় এবং তাদের সমষ্টি সবসময় ঠিক 1.0 (বা ১০০%) হয়। যেমন- আউটপুট যদি হয় [0.75, 0.15], এর অর্থ হলো প্রথম কম্পোনেন্টটি একাই ৭৫% তথ্য এবং দ্বিতীয়টি ১৫% তথ্য ধরে রেখেছে।

explained_variance_

- **আকৃতি (Shape):** (n_components,)
- **ব্যবহার এবং কর্পোরেট গুরুত্ব:** এটি প্রতিটি নির্বাচিত প্রিন্সিপাল কম্পোনেন্ট দ্বারা ব্যাখ্যা করা প্রকৃত ভ্যারিয়েন্স বা তথ্যের পরিমাণকে নির্দেশ করে। এই মানগুলো মূলত কোভ্যারিয়েন্স ম্যাট্রিক্সের সবচেয়ে বড় আইগেনভ্যালুগুলোর (Eigenvalues) সমান। ইন্টারভিউ বা একাডেমিক পরীক্ষায় কাইসারের নিয়ম (Kaiser's Criterion) প্রয়োগ করার

সময় এই অ্যাট্রিবিউটটি সরাসরি চেক করা হয়, কারণ এই ভ্যালু দেখেই সিদ্ধান্ত নেওয়া হয় যে কোন অক্ষটির গাণিতিক শক্তি গড় তথ্যের চেয়ে বেশি।

mean_

- **আকৃতি (Shape):** (n_features,)
- **ব্যবহার এবং কার্যপদ্ধতি:** এটি ট্রেনিং সেট থেকে সংগৃহীত প্রতিটি মূল ফিচারের বা কলামের নিজস্ব গড় মান (Empirical Mean) সংরক্ষণ করে। গাণিতিক ফর্মে এটি $X.mean(axis=0)$ এর সমান। PCA ব্যাকএন্ডে ডেটা সেন্টারিং বা স্ট্যান্ডার্ডাইজেশন করার সময় যে গড় মানগুলো হিসাব করেছিল, তা এখানে জমা থাকে। পরবর্তীতে নতুন কোনো টেস্ট ডেটা আসলে সেটিকে একই মাপে শিফট করার জন্য মডেল এই অ্যাট্রিবিউটের সাহায্য নেয়।

singular_values_

- **আকৃতি (Shape):** (n_components,)
- **ব্যবহার এবং কার্যপদ্ধতি:** এটি প্রতিটি নির্বাচিত প্রিন্সিপাল কম্পোনেন্টের সাথে সম্পর্কিত সুনির্দিষ্ট সিঙ্গুলার ভ্যালুগুলো (Singular Values) ধারণ করে। জ্যামিতিকভাবে এই মানগুলো নিম্ন-মাত্রার স্পেসে ভ্যারিয়েবলগুলোর ২-নর্মস (2-norms) বা দূরত্বের সমান। ব্যাকএন্ডে সলভারগুলো ডেটার ম্যাট্রিক্স ডিকম্পজিশন করার সময় সরাসরি এই সিঙ্গুলার মানগুলো মেমোরিতে লক করে রাখে।

n_components_

- **আকৃতি (Type):** int (পূর্ণসংখ্যা)
- **ব্যবহার এবং কার্যপদ্ধতি:** এটি মডেল দ্বারা চূড়ান্তভাবে অনুমিত বা নির্বাচিত প্রিন্সিপাল কম্পোনেন্টের প্রকৃত সংখ্যাটি জানায়। যখন আমরা প্যারামিটারে কোনো ভগ্নাংশ বা বিশেষ কমান্ড (যেমন `n_components='mle'`) দিই, তখন মডেল ইনপুট ডেটা বিশ্লেষণ করে নিজে থেকে যে কয়টি সেরা ফিচার বেছে নেয়, তার সংখ্যাটি এখানে দেখা যায়। অন্যথায়, এটি সরাসরি আমাদের দেওয়া `n_components` প্যারামিটারের মানের সমান হয়।

feature_names_in_

- **আকৃতি (Shape):** (n_features_in_,)
- **ব্যবহার এবং কার্যপদ্ধতি:** যদি মডেলটি ট্রেন করার সময় ইনপুট হিসেবে কোনো Pandas DataFrame ব্যবহার করা হয় এবং তার কলামের নামগুলো স্ট্রিং (Text) টেক্সট আকারে থাকে, তবে এটি সেই মূল ফিচার বা কলামগুলোর অফিশিয়াল নামের তালিকাটি

ছব্ব সংরক্ষণ করে। এর ফলে মডেলটি পরবর্তীতে কোন কোন কলামের ওপর কাজ করেছিল তা সহজেই অডিট করা যায়।

n_features_in_

- **আকৃতি (Type):** int
- **ব্যবহার এবং কার্যপদ্ধতি:** মডেলটি ডট ফিট (fit) করার সময় সর্বমোট কতটি ইনপুট ফিচার বা কলামের মুখোমুখি হয়েছিল, এটি সরাসরি সেই সংখ্যাটি মেমোরিতে ধরে রাখে। ডাউনস্ট্রিম পাইপলাইনে ডেটার আকৃতি ঠিক আছে কি না তা যাচাই করতে এটি ব্যবহৃত হয়।

n_samples_

- **আকৃতি (Type):** int
- **ব্যবহার এবং কার্যপদ্ধতি:** ট্রেনিং ডেটাসেটে সর্বমোট কতগুলো রো (Rows) বা ডেটা স্যাম্পল উপস্থিত ছিল, এটি সেই সংখ্যাটি প্রকাশ করে। অভ্যন্তরীণ ডিগ্রি অফ ফ্রিডম এবং ভ্যারিয়েন্সের গড় হিসাব করার কাজে ব্যাকএন্ডে এটি ব্যবহৃত হয়।

noise_variance_

- **আকৃতি (Type):** float (দশমিক সংখ্যা)
- **ব্যবহার এবং কার্যপদ্ধতি:** এটি প্রবাবিলিস্টিক পিসিএ (Probabilistic PCA) মডেলের ওপর ভিত্তি করে ডেটাসেটের আনুমানিক নয়েজ কোভ্যারিয়েন্স বা অপ্ৰয়োজনীয় আবর্জনার পরিমাণ হিসাব করে। গাণিতিকভাবে এটি কোভ্যারিয়েন্স ম্যাট্রিক্সের ক্ষুদ্রতম আইগেনভ্যালুগুলোর গড় মানের সমান। ডেটার সামগ্রিক কোভ্যারিয়েন্স নিখুঁত করতে এবং স্যাম্পল স্কেরিংয়ের গভীর গাণিতিক মূল্যায়নে এটি ব্যবহৃত হয়।

Code:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# ডেটাসেট ও প্রিপ্রসেসিং মডিউল ইম্পোর্ট
from sklearn.datasets import load_breast_cancer
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

# মডেল ট্রেনিং এবং মূল্যায়নের মডিউল ইম্পোর্ট
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

# ১. ডেটা লোড এবং ফিচার প্রস্তুতকরণ (Data Loading & Preprocessing)
# শ্বন ক্যান্সার (Breast Cancer) ডেটাসেটটি Pandas DataFrame হিসেবে লোড করা হচ্ছে
data = load_breast_cancer(as_frame=True)
df = data.frame

# ডেটাসেটের আকৃতি (Rows, Columns) যাচাই করা হচ্ছে
print(f"Dataset Shape: {df.shape}")

# ইনপুট ফিচার (X) এবং টার্গেট ভ্যারিয়েবল (y) আলাদা করা হচ্ছে
X = df.drop('target', axis=1)
y = df['target']

# ২. ফিচার স্ট্যান্ডার্ডাইজেশন (Feature Standardization)
# PCA প্রয়োগ করার আগে সব ফিচারের স্কেল সমান করা বাধ্যতামূলক (Mean=0, Variance=1)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# আউটপুট স্ক্রিনে বৈজ্ঞানিক (e-05) নোটেশন বন্ধ করে সাধারণ দশমিক দেখানোর ব্যবস্থা
np.set_printoptions(suppress=True)

# ৩. পূর্ণাঙ্গ PCA মডেলিং ও অপ্টিমাল কম্পোনেন্ট খোঁজা (Full PCA & Component Selection)
# n_components=None রাখা হয়েছে যেন মূল ৩০টি ফিচারের বিপরীতে ৩০টি কম্পোনেন্টই তৈরি হয়
pca_full = PCA(n_components=None)
X_pca_full = pca_full.fit_transform(X_scaled)
```

```

# প্রতিটি প্রিন্সিপাল কম্পোনেন্ট কত শতাংশ তথ্য (Variance) ধরে রেখেছে তা দেখা হচ্ছে
explained_variance = pca_full.explained_variance_ratio_
print("\nIndividual Explained Variance Ratio:")
print(explained_variance)

# কম্পোনেন্টগুলোর তথ্যের ক্রমপুঞ্জিত যোগফল বা Cumulative Sum হিসাব করা হচ্ছে
cumulative_variance = np.cumsum(explained_variance)
print("\nCumulative Explained Variance Ratio:")
print(cumulative_variance)

# ৪. স্ক্রি প্লট বা কিউমুলেটিভ ভ্যারিয়েন্স গ্রাফ (Data Visualization)
# কতটি কম্পোনেন্ট নিলে কত শতাংশ তথ্য সংরক্ষিত থাকবে তা গ্রাফে দেখা হচ্ছে
plt.figure(figsize=(8, 5))
plt.plot(cumulative_variance, marker='o', linestyle='--', color='b')
plt.title("PCA Cumulative Explained Variance")
plt.xlabel("Number of Principal Components")
plt.ylabel("Cumulative Variance Ratio")
plt.grid(True)
plt.show()

```

```

# ৫. ডেটা বিভাজন (Train-Test Split)
# মূল ডেটাসেটকে ৮০% ট্রেনিং এবং ২০% টেস্ট সেটে বিভক্ত করা হচ্ছে
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)

# ৬. পাইপলাইন ১: PCA ছাড়া মডেলিং (Pipeline WITHOUT PCA)
# প্রথমে ডেটা স্কেল হবে এবং সরাসরি লজিস্টিক রিগ্রেশনে ইনপুট যাবে
pipe_no_pca = Pipeline([
    ("scaler", StandardScaler()),
    ("clf", LogisticRegression(max_iter=500))
])

# মডেল ট্রেনিং এবং একুরেসী পরীক্ষা
pipe_no_pca.fit(X_train, y_train)
pred_no_pca = pipe_no_pca.predict(X_test)
acc_no_pca = accuracy_score(y_test, pred_no_pca)
print(f"Accuracy without PCA (All 30 features): {acc_no_pca:.4f}")

```



```
# ৭. পাইপলাইন ২: PCA সহ মডেলিং (Pipeline WITH PCA)
# ৩০টি কলাম থেকে মাত্র ১০টি মেইন প্রিন্সিপাল কম্পোনেন্ট (PC) এক্সট্রাক্ট করা হচ্ছে
pipe_with_pca = Pipeline([
    ("scaler", StandardScaler()),
    ("pca", PCA(n_components=10)),
    ("clf", LogisticRegression(max_iter=500))
])

# মডেল ট্রেনিং এবং এক্যুরেসী পরীক্ষা
pipe_with_pca.fit(X_train, y_train)
pred_with_pca = pipe_with_pca.predict(X_test)
acc_with_pca = accuracy_score(y_test, pred_with_pca)
print(f"Accuracy with PCA (Only 10 components): {acc_with_pca:.4f}")
```

প্রশ্ন ১: যদি আমার ডেটাসেটের ৩০টি ফিচারের মধ্যে প্রতিটি ফিচারই একে অপরের সাথে সম্পূর্ণ আন-কোরিলেটেড (Uncorrelated) বা স্বাধীন থাকে, তবে সেখানে PCA ব্যবহার করলে কি ডাইমেনশন কমানো সম্ভব? যৌক্তিক কারণ ব্যাখ্যা করো।

- **উত্তর:** তাত্ত্বিকভাবে, এই পরিস্থিতিতে PCA ব্যবহার করে কোনো লাভ হবে না এবং ডাইমেনশন কমানো সম্ভব হবে না।
- **ব্যাখ্যা:** PCA-এর মূল শক্তি হলো ডেটার ভেতরের কোরিলেশন বা তথ্যের পুনরাবৃত্তিকে (Redundancy) কাজে লাগিয়ে মাত্রা কমানো। যদি সব ফিচার আগে থেকেই আন-কোরিলেটেড থাকে, তবে কোভ্যারিয়েন্স ম্যাট্রিক্সের অফ-ডায়াগোনাল উপাদানগুলো সব শূন্য হবে। এর ফলে PCA কোনো নতুন জাদুকরী অক্ষ তৈরি করতে পারবে না; মূল ফিচারগুলোই এক একটি প্রিন্সিপাল কম্পোনেন্ট হিসেবে আত্মপ্রকাশ করবে। যেহেতু কোনো কম্পোনেন্টই অন্য কম্পোনেন্টের তথ্য শেয়ার করছে না, তাই তথ্যের বড় ক্ষতি না করে কোনো কম্পোনেন্টকে ফেলে দেওয়া সম্ভব হবে না।

প্রশ্ন ২: কোনো একটি ডেটাসেটে PCA প্রয়োগ করার পর দেখা গেল প্রথম ৩টি প্রিন্সিপাল কম্পোনেন্টের (PC_1, PC_2, PC_3) আইগেনভ্যালু যথাক্রমে ৩.৫, ২.৮ এবং ০.৮। কাইসারের নিয়ম (Kaiser's Criterion) অনুযায়ী আপনি কোন কোন কম্পোনেন্টকে চূড়ান্ত ফিচার হিসেবে সিলেক্ট করবেন এবং কেন?

- **উত্তর:** কাইসারের নিয়ম অনুযায়ী শুধুমাত্র PC_1 এবং PC_2 কে সিলেক্ট করা হবে, এবং PC_3 কে বাদ দেওয়া হবে।
- **ব্যাখ্যা:** কাইসারের নিয়মের মূল কথা হলো, একটি স্ট্যান্ডার্ডাইজড ডেটাসেটে প্রতিটি মূল ফিচারের গড় ভ্যারিয়েন্স বা আইগেনভ্যালু হয় ১.০। তাই কোনো প্রিন্সিপাল কম্পোনেন্টকে ফিচার হিসেবে টিকে থাকতে হলে তার আইগেনভ্যালুর মান অবশ্যই ১.০ এর চেয়ে বেশি হতে হবে। এখানে PC_1 (৩.৫) এবং PC_2 (২.৮) এর মান ১.০ এর চেয়ে বড় হওয়ায় এরা পাস করেছে, কিন্তু PC_3 (০.৮) এর মান ১.০ এর চেয়ে কম হওয়ায় এটি একটি গড়পড়তা ফিচারের সমান তথ্যও ধারণ করতে পারছে না। তাই এটিকে নয়েজ হিসেবে ছাঁটাই করা হবে।

প্রশ্ন ৩: উচ্চ মাত্রার ডেটাসেটে (High-Dimensional Space) ডেটা পয়েন্টগুলোর পরম দূরত্ব (Absolute Distance) বাড়া সত্ত্বেও কেন KNN অ্যালগরিদমটি তার কার্যক্ষমতা হারিয়ে ফেলে? এটার জ্যামিতিক কারণ কী?

- **উত্তর:** এর মূল জ্যামিতিক কারণ হলো দূরত্বের বৈচিত্র্যহীনতা বা Distance Concentration Effect।
- **ব্যাখ্যা:** মাত্রা বাড়ার সাথে সাথে Euclidean Distance-এর গাণিতিক নিয়ম অনুযায়ী সব বিন্দুর মধ্যকার পরম দূরত্ব অবশ্যই বাড়ে। কিন্তু সমস্যা হলো, সব বিন্দুর দূরত্ব একসাথে

বাড়ার কারণে সবচেয়ে কাছের বিন্দুর দূরত্ব ($Dist_{min}$) এবং সবচেয়ে দূরের বিন্দুর দূরত্বের ($Dist_{max}$) মধ্যকার ব্যবধান বা অনুপাত প্রায় শূন্যের কাছাকাছি চলে আসে। ফিচার স্পেসে সব ডেটা পয়েন্ট তখন একে অপরের থেকে প্রায় সমান দূরে অবস্থান করে। যেহেতু কোনো বিন্দুর মধ্যে দূরত্বের কোনো আপেক্ষিক পার্থক্য থাকে না, তাই KNN অ্যালগরিদমটি আলাদা করতে পারে না যে কে আসল প্রতিবেশী আর কে দূরের শত্রু।

প্রশ্ন ৪: লজিস্টিক রিগ্রেশনের coefficients (সহগ) দেখে আমরা সহজেই বুঝতে পারি কোন ফিচারটি মডেলের জন্য কতটুকু গুরুত্বপূর্ণ। কিন্তু PCA করার পর লজিস্টিক রিগ্রেশনের সহগ দেখে মূল ফিচারের গুরুত্ব ব্যাখ্যা করা যায় না কেন? একে কী ধরনের মডেল বলা হয়?

- **উত্তর:** কারণ PCA করার পর মডেলটি তার "ব্যাখ্যাযোগ্যতা" বা Interpretability হারিয়ে একটি জটিল ব্ল্যাক-বক্স (Black-Box) মডেলে পরিণত হয়।
- **ব্যাখ্যা:** লজিস্টিক রিগ্রেশনে মূল কলামগুলো (যেমন- বয়স, ওজন) সরাসরি ব্যবহৃত হয়, তাই সহগ দেখে সরাসরি বলা যায় ওজনের গুরুত্ব কতটুকু। কিন্তু PCA হলো একটি ফিচার এক্সট্রাকশন টেকনিক; এটি মূল ৩০টি কলামকে রেন্ডম বা লিনিয়ার কম্বিনেশন করে সম্পূর্ণ নতুন ১০টি PC তৈরি করে। যখন আপনি লজিস্টিক রিগ্রেশনকে এই নতুন PC গুলো ইনপুট হিসেবে দেবেন, তখন মডেলের সহগগুলো শুধু দেখাবে যে কোন PC-এর গুরুত্ব কতটুকু। যেহেতু একটি PC মূলত ৩০টি ফিচারেরই একটা খিচুড়ি বা মিশ্রণ, তাই সেখান থেকে আলাদা করে মূল একক ফিচারের (যেমন- বয়স) অবদান কতটুকু ছিল, তা ট্র্যাক করা গাণিতিকভাবে অসম্ভব হয়ে পড়ে।

প্রশ্ন ৫: আপনার ডেটাসেটে একটি অত্যন্ত গুরুত্বপূর্ণ কলাম আছে যার আউটলায়ার বা চরম ব্যতিক্রমী মানগুলো (Outliers) ডেটার আসল বৈশিষ্ট্য প্রকাশ করে। এই ডেটাসেটে স্ট্যান্ডার্ডাইজেশন না করে সরাসরি PCA চালালে কী বিপর্যয় ঘটবে?

- **উত্তর:** স্ট্যান্ডার্ডাইজেশন না করলে PCA সম্পূর্ণ ভুল এবং বিভ্রান্তিকর অক্ষ (PC) তৈরি করবে, যা ডেটার আসল তথ্যকে বিকৃত করে ফেলবে।
- **ব্যাখ্যা:** PCA সম্পূর্ণভাবে ভ্যারিয়েন্স বা বিস্তৃতির ওপর ভিত্তি করে কাজ করে। যদি ডেটা স্কেল না করা হয়, তবে যে ফিচারের সংখ্যাগত মান বড় (যেমন- কারও বেতন ১,০০,০০০ টাকা বনাম কারও বয়স ২৫ বছর), তার ভ্যারিয়েন্স স্বাভাবিকভাবেই অনেক বড় দেখাবে। এর ওপর যদি চরম আউটলায়ার থাকে, তবে সেই আউটলায়ারের বর্গের পার্থক্যের কারণে ভ্যারিয়েন্স আকাশচুম্বী হবে। PCA তখন অক্ষের মতো ধরে নেবে যে এই কলামেই পৃথিবীর সব তথ্য লুকিয়ে আছে এবং সে প্রথম অক্ষটিকে (PC_1) ওই আউটলায়ারের দিক বরাবর টেনে নিয়ে যাবে। ফলে আসল গুরুত্বপূর্ণ কিন্তু ছোট স্কেলের ফিচারগুলোর তথ্য পুরোপুরি হারিয়ে যাবে এবং মডেলটি আউটলায়ার দ্বারা তীব্রভাবে বিচ্যুত হবে।

প্রিন্সিপাল কম্পোনেন্ট অ্যানালাইসিস (PCA) এর সম্পূর্ণ সারসংক্ষেপ

বিশাল এবং জটিল ডেটাসেটের ডাইমেনশন বা ফিচারের সংখ্যা কমিয়ে সেটিকে সহজ ও কার্যকর করার গাণিতিক প্রক্রিয়াই হলো ডাইমেনশনালিটি রিডাকশন বা মাত্রার সংকোচন। এই কাজের জন্য বিশ্বজুড়ে সবচেয়ে জনপ্রিয় আনসুপারভাইজড অ্যালগরিদম হলো **Principal Component Analysis (PCA)**। পুরো অধ্যায়ের মূল শিক্ষণীয় বিষয়গুলো নিচে সুনির্দিষ্ট পয়েন্ট আকারে সামারি করা হলো:

মাত্রার অভিশাপ (Curse of Dimensionality)

ডেটাসেটে ফিচারের সংখ্যা মাত্রাতিরিক্তভাবে বৃদ্ধি পাওয়ার কারণে যে গাণিতিক ও কম্পিউটেশনাল জটিলতা তৈরি হয়, তাকে মাত্রার অভিশাপ বলা হয়।

- **ডেটার শূন্যতা (Sparsity):** ডাইমেনশন বাড়ার সাথে সাথে ফিচার স্পেসের আয়তন সূচকীয় হারে বাড়ে, যার ফলে ডেটা পয়েন্টগুলো একে অপরের থেকে চরম দূরে ছিটকে যায় এবং স্পেসটি ফাঁকা হয়ে পড়ে।
- **দূরত্বের সমতা (Distance Concentration Effect):** উচ্চ মাত্রায় সব বিন্দুর মধ্যকার পরম দূরত্ব বৃদ্ধি পায় ঠিকই, কিন্তু সবচেয়ে কাছের বিন্দু এবং সবচেয়ে দূরের বিন্দুর দূরত্বের আপেক্ষিক পার্থক্য প্রায় শূন্য হয়ে যায়। ফলে KNN বা SVM-এর মতো দূরত্ব-ভিত্তিক মডেলগুলো তাদের কার্যক্ষমতা পুরোপুরি হারিয়ে ফেলে।
- **মডেলের ক্ষতি:** অতিরিক্ত ফিচারের কারণে কম্পিউটেশনাল খরচ ও সময় বহুগুণ বেড়ে যায় এবং মডেল গুরুত্বপূর্ণ লজিক শেখার বদলে ডেটার নয়েজ (Noise) মুখস্থ করে ওভারফিটিং (Overfitting) ঘটায়।

PCA এর মূল তাত্ত্বিক ধারণা ও বৈশিষ্ট্য

PCA কোনো ফিচার সিলেকশন পদ্ধতি নয়, এটি একটি ফিচার এক্সট্রাকশন (Feature Extraction) টেকনিক।

- **মেকানিজম:** এটি মূল ফিচারগুলোকে সরাসরি ফেলে দেয় না, বরং তাদের লিনিয়ার কম্বিনেশন বা রৈখিক সংমিশ্রণ ঘটিয়ে সম্পূর্ণ নতুন একগুচ্ছ ভ্যারিয়েবল তৈরি করে, যাদের **Principal Components (PCs)** বলা হয়।
- **ভ্যারিয়েন্স এবং তথ্য:** PCA-তে ভ্যারিয়েন্স বা ডেটার বিস্তৃতিকে "আসল তথ্য" হিসেবে গণ্য করা হয়। প্রথম কম্পোনেন্ট (PC_1) ডেটার সর্বোচ্চ ভ্যারিয়েন্স ধরে রাখে, এবং পরবর্তী কম্পোনেন্টগুলো ($PC_2, PC_3 \dots PC_n$) পর্যায়ক্রমে অবশিষ্ট সর্বোচ্চ তথ্য লুফে নেয়।
- **অর্থোগোনালিটি (Orthogonality):** তৈরি হওয়া সবকটি PC একে অপরের সাথে নিখুঁতভাবে লম্ব বা ৯০ ডিগ্রি কোণে অবস্থান করে। এর ফলে তাদের নিজেদের মধ্যে

কোনো কোরিলেশন থাকে না, যা ডেটাসেট থেকে মাল্টিকোলিনিয়ারিটি (Multicollinearity) সম্পূর্ণ দূর করে।

- **আনসুপারভাইজড ও স্কেল সংবেদনশীল:** PCA টার্গেট লেবেলের সাহায্য ছাড়াই শুধু ইনপুট ফিচারের বিন্যাস দেখে কাজ করে। এটি স্কেলের প্রতি অত্যন্ত সংবেদনশীল হওয়ায় এর ওপর কাজ করার আগে **StandardScaler** দিয়ে ডেটা স্কেল করা বাধ্যতামূলক।

PCA যেভাবে কাজ করে (Step-by-step Working)

1. **স্ট্যান্ডার্ডাইজেশন:** প্রতিটি ফিচারের গড় মানকে ০ এবং ভ্যারিয়েন্সকে ১-এ এনে সব কলামকে সমান গাণিতিক মর্যাদা দেওয়া হয়।
2. **কোভ্যারিয়েন্স ম্যাট্রিক্স:** ফিচারগুলো একে অপরের সাথে কীভাবে পরিবর্তিত হচ্ছে এবং তাদের মধ্যে তথ্যের কোনো পুনরাবৃত্তি আছে কি না, তা বের করতে এই স্কয়ার ম্যাট্রিক্স তৈরি করা হয়।
3. **আইগেন ডিকম্পজিশন:** কোভ্যারিয়েন্স ম্যাট্রিক্সকে ভেঙে আইগেনভেক্টর (যা নতুন অক্ষ বা PC-এর দিক দেখায়) এবং আইগেনভ্যালু (যা ওই অক্ষের ভ্যারিয়েন্স বা শক্তির স্কেল জানায়) বের করা হয়।
4. **ডেটা প্রজেকশন:** মূল ডেটা পয়েন্টগুলোকে অক্ষ বরাবর রোটेट বা ঘুরিয়ে দিয়ে নতুন প্রিন্সিপাল কম্পোনেন্টের কোঅর্ডিনেটসে রূপান্তর করা হয়।
5. **ফিচার সিলেকশন:** স্ক্রি প্লট (Scree Plot) এর কনুই বা বাঁক দেখে কিংবা কিউমুলেটিভ ভ্যারিয়েন্স থ্রেশহোল্ড (যেমন অন্তত ৯০% তথ্য ধরে রাখা) নীতি ব্যবহার করে শীর্ষ কয়েকটি PC রেখে বাকিগুলো চিরতরে ফেলে দেওয়া হয়।

Scikit-Learn-এর ভারী প্যারামিটার ও অ্যাট্রিবিউটসমূহ

মোস্ট ইম্পর্টেন্ট প্যারামিটারসমূহ:

- **n_components:** চূড়ান্তভাবে কতটি কম্পোনেন্ট রাখতে চান তা নির্ধারণ করে (যেমন নির্দিষ্ট সংখ্যা বা 0.95 ভগ্নাংশ)।
- **svd_solver:** ব্যাকএন্ডের গাণিতিক সলভার ঠিক করে। বড় ডেটাসেটে দ্রুত কাজের জন্য randomized এবং ছোট ডেটায় নিখুঁত মানের জন্য full ব্যবহৃত হয়।

মোস্ট ইম্পর্টেন্ট অ্যাট্রিবিউটসমূহ (Trailing Underscore):

- **components_:** এক্সট্রাক্ট হওয়া প্রিন্সিপাল কম্পোনেন্ট বা আইগেনভেক্টরগুলোর দিক অ্যারে আকারে দেখায়।

- `explained_variance_ratio_`: প্রতিটি কম্পোনেন্ট মূল তথ্যের কত শতাংশ নিজের ভেতর ধরে রেখেছে তার শতকরা অনুপাত জানায়।
- `explained_variance_`: কোভ্যারিয়েন্স ম্যাট্রিক্স থেকে প্রাপ্ত আসল আইগেনভ্যালু বা ভ্যারিয়েন্সের মান ধারণ করে।

PCA-এর শক্তি ও সীমাবদ্ধতা (When it Works & Fails)

এটি চমৎকার কাজ করে যখন:

- ডেটাসেটের ফিচারগুলোর মধ্যে শক্তিশালী সরলরৈখিক বা লিনিয়ার সম্পর্ক থাকে।
- ফিচারের সংখ্যা অনেক বেশি এবং তাদের মধ্যে তীব্র কোরিলেশন বা রিডান্ডেন্সি বিদ্যমান থাকে।

এটি ব্যর্থ বা বিকৃত ফলাফল দেয় যখন:

- ফিচারগুলোর সম্পর্ক নন-লিনিয়ার বা আঁকাবাঁকা (যেমন বৃত্তাকার বা সর্পিলাকার) হয়, কারণ PCA শুধু সোজা অক্ষ বরাবর ভ্যারিয়েন্স খোঁজে।
- ডেটাসেটে প্রচুর ক্যাটাগরিক্যাল ভ্যারিয়েবল থাকে কিংবা ডেটা ট্রেন করার আগে স্ট্যান্ডার্ডাইজেশন বাদ দেওয়া হয়।
- ডেটাতে চরম আউটলায়ার (Outliers) থাকে, যা বেস্ট-ফিট লাইনকে নিজের দিকে টেনে নিয়ে আসল তথ্যকে বিচ্যুত করে ফেলে।

Naive Bayes

Naive Bayes হলো একটি সহজ কিন্তু শক্তিশালী সুপারভাইজড মেশিন লার্নিং অ্যালগরিদম যা মূলত ক্লাসিফিকেশন বা শ্রেণিবিভাগের কাজে ব্যবহৃত হয়। এটি Bayes' Theorem বা বেইজ উপপাদ্যের গাণিতিক নীতির ওপর ভিত্তি করে তৈরি।

এই অ্যালগরিদমটিকে "Naive" বা "সরল" বলা হয় কারণ এটি একটি অত্যন্ত শক্তিশালী কিন্তু সরল ধারণা ধরে নেয়: প্রতিটি ইনপুট ফিচার একে অপরের থেকে সম্পূর্ণ স্বাধীন বা ইন্ডিপেন্ডেন্ট। অর্থাৎ, একটি ফিচারের উপস্থিতি অন্য কোনো ফিচারের উপস্থিতিতে প্রভাবিত করে না। বাস্তবে এমন স্বাধীনতা খুব কমই পাওয়া যায়, কিন্তু এই সহজ ধারণার কারণেই মডেলটি অত্যন্ত দ্রুত কাজ করতে পারে।

এর কাজের মূল ভিত্তি হলো প্রোবাবিলিটি বা সম্ভাবনা। যখনই আমাদের সামনে কোনো নতুন ডেটা পয়েন্ট আসে, এটি Bayes' Theorem ব্যবহার করে প্রতিটি সম্ভাব্য ক্লাসের জন্য সম্ভাবনা হিসাব করে। যে ক্লাসের সম্ভাবনা সবচেয়ে বেশি হয়, মডেলটি সেই ক্লাসটিকেই আউটপুট হিসেবে প্রেডিক্ট করে। এই অ্যালগরিদমটি বিশেষভাবে টেক্সট ক্লাসিফিকেশন, ইমেইল স্প্যাম ডিটেকশন এবং সেন্টিমেন্ট অ্যানালাইসিসের মতো বড় ও জটিল ডেটাসেটে অসাধারণ ফলাফল দেয়।

Naive Bayes-এর প্রকারভেদ ও ব্যবহারের উদাহরণ

Naive Bayes অ্যালগরিদমটি মূলত ডেটার ডিস্ট্রিবিউশন বা বণ্টনের ওপর ভিত্তি করে কয়েক ধরনের হয়। নিচে এদের বিস্তারিত দেওয়া হলো:

Naive Bayes টাইপ	ডেটার প্রকৃতি	ডেটাসেটের উদাহরণ
Gaussian NB	কন্টিনিউয়াস ডেটা যা নরমাল ডিস্ট্রিবিউশন মেনে চলে	আইরিস (Iris) ফ্লাওয়ার ডেটাসেট (যেখানে পাপড়ির দৈর্ঘ্য ও প্রস্থ কন্টিনিউয়াস মান)
Multinomial NB	ডিসক্রিট কাউন্ট (যেমন: শব্দের সংখ্যা)	টেক্সট ক্লাসিফিকেশন বা স্প্যাম ডিটেকশন (ইমেইলে কোন শব্দ কতবার আছে)
Complement NB	ইমব্যালেন্সড ডেটাসেট	টেক্সট ক্লাসিফিকেশন (যখন একটি ক্লাসের ডেটা অন্য ক্লাসের চেয়ে অনেক বেশি থাকে)
Bernoulli NB	বাইনারি বা বুলিয়ান (০ অথবা ১) ডেটা	স্প্যাম ফিল্টারিং (একটি শব্দ ইমেইলে আছে কি নেই—শুধু উপস্থিতি বা অনুপস্থিতি)
Categorical NB	ক্যাটাগরিক্যাল ডেটা	শপিং ডেটাসেট (যেখানে বৈশিষ্ট্যগুলো নির্দিষ্ট ক্যাটাগরি যেমন: রং: লাল/নীল/সবুজ)

বিস্তারিত ব্যাখ্যা:

১. Gaussian Naive Bayes (GaussianNB)

এটি ধরে নেয় যে ফিচারগুলো গাউসিয়ান (নরমাল) ডিস্ট্রিবিউশন অনুসরণ করে। যখন ডেটার মানগুলো কন্টিনিউয়াস হয়, তখন এটি সবচেয়ে ভালো কাজ করে। উদাহরণস্বরূপ, আইরিস ফ্লাওয়ার ডেটাসেটে ফুলের বৈশিষ্ট্যগুলো সংখ্যাসূচক হয়।

২. Multinomial Naive Bayes (MultinomialNB)

এটি মাল্টিনোমিয়াল ডিস্ট্রিবিউশনের ডেটার জন্য ব্যবহৃত হয়, যা বিশেষ করে টেক্সট ক্লাসিফিকেশনে জনপ্রিয়। উদাহরণস্বরূপ, একটি ডকুমেন্টে কোন শব্দ কতবার এসেছে (word count), তার ভিত্তিতে ইমেইল স্প্যাম কি না তা নির্ণয় করা হয়।

৩. Complement Naive Bayes (ComplementNB)

এটি স্ট্যান্ডার্ড মাল্টিনোমিয়াল Naive Bayes-এর একটি উন্নত সংস্করণ, যা ইমব্যালেন্সড ডেটাসেটে খুব স্থিতিশীল ফলাফল দেয়। টেক্সট ক্লাসিফিকেশনের ক্ষেত্রে এটি প্রায়ই সাধারণ মাল্টিনোমিয়াল মডেলের চেয়ে অনেক ভালো পারফর্ম করে।

৪. Bernoulli Naive Bayes (BernoulliNB)

এই মডেলে প্রতিটি ফিচারকে বাইনারি (০ অথবা ১) হিসেবে ধরে নেওয়া হয়। টেক্সট ক্লাসিফিকেশনে এটি ওয়ার্ড কাউন্টের বদলে ওয়ার্ড অকারেন্স (শব্দটি আছে কি নেই) ব্যবহার করে। এটি ছোট ইমেইল বা ডকুমেন্টের ক্ষেত্রে ভালো কাজ করতে পারে।

৫. Categorical Naive Bayes (CategoricalNB)

যখন ডেটা পুরোপুরি ক্যাটাগরিক্যাল হয় (যেমন: লিঙ্গ: পুরুষ/মহিলা, পেশা: ছাত্র/চাকরিজীবী), তখন এই মডেলটি ব্যবহৃত হয়। এটি প্রতিটি ক্যাটাগরির জন্য আলাদা ডিস্ট্রিবিউশন হিসাব করে।

গাণিতিক সূত্র ও পরিভাষা (Mathematical Formula and Terms)

Naive Bayes-এর ভিত্তি হলো **Bayes' Theorem**, যা মূলত একটি শর্তাধীন সম্ভাবনা (Conditional Probability) নির্ণয়ের উপায়।

১. মূল সমীকরণ (The Core Formula)

যদি y একটি ক্লাস এবং X একটি ফিচার ভেক্টর (যা $x_1, x_2, \dots, x_n$ ফিচার নিয়ে গঠিত) হয়, তবে বেইজ থিওরেম অনুযায়ী:

$$P(y|X) = \frac{P(X|y) \cdot P(y)}{P(X)}$$

এখানে:

- $P(y|X)$ (**Posterior Probability**): ফিচার X দেওয়া থাকলে y ক্লাস হওয়ার সম্ভাবনা।
- $P(X|y)$ (**Likelihood**): y ক্লাসটির অধীনে ফিচার X -এর সম্ভাবনা।
- $P(y)$ (**Prior Probability**): কোনো ফিচার না দেখেই y ক্লাস হওয়ার প্রাথমিক সম্ভাবনা।
- $P(X)$ (**Evidence**): ফিচারটির নিজস্ব সম্ভাবনা।

২. Naive Assumption (শর্তাধীন স্বাধীনতা)

Naive Bayes ধরে নেয় যে, ক্লাসের মান জানা থাকলে প্রতিটি ফিচার একে অপরের থেকে স্বাধীন (Conditional Independence)। ফলে সমীকরণটি সরল হয়ে দাঁড়ায়:

$$P(y|x_1, \dots, x_n) = \frac{P(y) \prod_{i=1}^n P(x_i|y)}{P(x_1, \dots, x_n)}$$

যেহেতু $P(x_1, \dots, x_n)$ একটি ধ্রুবক (Constant), তাই ক্লাসিফিকেশনের নিয়মটি হয়:

$$y = \arg \max_y P(y) \prod_{i=1}^n P(x_i|y)$$

৩. গুরুত্বপূর্ণ পরিভাষা

- MAP (Maximum A Posteriori)**: মডেলটি $P(y|X)$ কে সর্বোচ্চ করার জন্য যে অনুমান বা এস্টিমেশন ব্যবহার করে, তাকে MAP এস্টিমেশন বলা হয়।
- Laplace Smoothing**: যদি কোনো ফিচার ট্রেনিং সেটে না থাকে, তবে তার সম্ভাবনা ০ হয়ে যায়। এই সমস্যা দূর করার জন্য সম্ভাবনা গণনায় একটি ছোট মান যোগ করা হয়, একে Laplace Smoothing বলে।
- Likelihood**: প্রতিটি নির্দিষ্ট ক্লাসের জন্য ফিচারের ডিস্ট্রিবিউশন কেমন হবে, তা এটি নির্দেশ করে।

১. ডেটাসেট (Dataset)

আমরা ৫ জন শিক্ষার্থীর তথ্য নিয়ে একটি ছোট ডেটাসেট তৈরি করি:

ID	Study Hours (x1)	Attendance (x2)	Pass (y)
1	2	60	No
2	3	70	No
3	5	80	Yes
4	6	90	Yes
5	4	75	Yes

- **টার্গেট:** Pass (Yes/No)
- **ফিচার:** Study Hours, Attendance

২. ট্রেনিং ধাপ (Training Phase)

ট্রেনিংয়ের কাজ হলো প্রতিটি ক্লাসের জন্য ফিচারগুলোর Prior Probability এবং Likelihood (Mean ও Variance) বের করা।

ক. Prior Probability $P(y)$

- মোট ডেটা $N = 5$
- $P(Yes) = 3/5 = 0.6$
- $P(No) = 2/5 = 0.4$

খ. Likelihood (Mean ও Variance নির্ণয়)

আমাদের প্রতিটি ফিচারের জন্য Mean (μ) এবং Variance (σ^2) বের করতে হবে।

Pass = Yes (ID 3, 4, 5):

- Study Hours: Mean $\mu = (5 + 6 + 4)/3 = 5$, Var $\sigma^2 \approx 1$
- Attendance: Mean $\mu = (80 + 90 + 75)/3 = 81.67$, Var $\sigma^2 \approx 58.33$

Pass = No (ID 1, 2):

- Study Hours: Mean $\mu = (2 + 3)/2 = 2.5$, Var $\sigma^2 = 0.5$
- Attendance: Mean $\mu = (60 + 70)/2 = 65$, Var $\sigma^2 = 50$

৩. প্রেডিকশন ধাপ (Prediction Phase)

ধরুন, একটি নতুন শিক্ষার্থীর ডেটা আসলো: Study Hours = 4, Attendance = 78।

ধাপ ১: গাউসিয়ান PDF ব্যবহার করে সম্ভাবনা বের করা আমরা প্রতিটি ফিচারের জন্য সম্ভাবনা বের করব:

$$P(x_i|y) = \frac{1}{\sqrt{2\pi\sigma_y^2}} \cdot \exp\left(-\frac{(x_i-\mu_y)^2}{2\sigma_y^2}\right)$$

- **Pass = Yes এর জন্য:**

- $P(\text{Study} = 4|\text{Yes})$ ও $P(\text{Atten} = 78|\text{Yes})$ গণনা করি।
- Likelihood $L_{\text{Yes}} = P(\text{Study} = 4|\text{Yes}) \times P(\text{Atten} = 78|\text{Yes}) \times P(\text{Yes})$

- **Pass = No এর জন্য:**

- $P(\text{Study} = 4|\text{No})$ ও $P(\text{Atten} = 78|\text{No})$ গণনা করি।
- Likelihood $L_{\text{No}} = P(\text{Study} = 4|\text{No}) \times P(\text{Atten} = 78|\text{No}) \times P(\text{No})$

ধাপ ২: চূড়ান্ত সিদ্ধান্ত (MAP Estimation) যেহেতু আমাদের ফিচারগুলো স্বাধীন ধরা হয়েছে, আমরা উপরের Likelihood গুলো তুলনা করব:

- যদি $L_{\text{Yes}} > L_{\text{No}}$ হয়, তবে প্রেডিকশন হবে "Yes"।
- যদি $L_{\text{No}} > L_{\text{Yes}}$ হয়, তবে প্রেডিকশন হবে "No"।

১. ট্রেনিং থেকে প্রাপ্ত প্যারামিটার (আমাদের আগের হিসাব অনুযায়ী):

- **Pass = Yes:** $\mu_{study} = 5, \sigma_{study}^2 = 1; \mu_{atten} = 81.67, \sigma_{atten}^2 = 58.33$
- **Pass = No:** $\mu_{study} = 2.5, \sigma_{study}^2 = 0.5; \mu_{atten} = 65, \sigma_{atten}^2 = 50$
- **Prior:** $P(Yes) = 0.6, P(No) = 0.4$

২. সম্ভাবনা গণনা (Gaussian PDF)

$$\text{সূত্র: } P(x|y) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$$

ক. Pass = Yes এর জন্য সম্ভাবনা:

$$১. \text{ Study Hours (4): } P(4|Yes) = \frac{1}{\sqrt{2\pi(1)}} \exp\left(-\frac{(4-5)^2}{2(1)}\right) \approx 0.3989 \cdot e^{-0.5} \approx \mathbf{0.2419}$$

$$২. \text{ Attendance (78): } P(78|Yes) = \frac{1}{\sqrt{2\pi(58.33)}} \exp\left(-\frac{(78-81.67)^2}{2(58.33)}\right) \approx 0.0523 \cdot e^{-0.116} \approx \mathbf{0.0466}$$

$$\text{Likelihood } (L_{Yes}): 0.2419 \times 0.0466 \times 0.6 \approx \mathbf{0.00676}$$

খ. Pass = No এর জন্য সম্ভাবনা:

$$১. \text{ Study Hours (4): } P(4|No) = \frac{1}{\sqrt{2\pi(0.5)}} \exp\left(-\frac{(4-2.5)^2}{2(0.5)}\right) \approx 0.5642 \cdot e^{-2.25} \approx \mathbf{0.0594}$$

$$২. \text{ Attendance (78): } P(78|No) = \frac{1}{\sqrt{2\pi(50)}} \exp\left(-\frac{(78-65)^2}{2(50)}\right) \approx 0.0564 \cdot e^{-1.69} \approx \mathbf{0.0105}$$

$$\text{Likelihood } (L_{No}): 0.0594 \times 0.0105 \times 0.4 \approx \mathbf{0.00025}$$

৩. চূড়ান্ত সিদ্ধান্ত (Comparison)

এখন আমরা প্রাপ্ত Likelihood দুটির তুলনা করব:

- $L_{Yes} = 0.00676$
- $L_{No} = 0.00025$

যেহেতু $L_{Yes} > L_{No}$ ($0.00676 > 0.00025$), তাই মডেলটি প্রেডিক্ট করবে যে শিক্ষার্থীটি **"Pass = Yes"**।

Over Iris Dataset:

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, classification_report

iris = load_iris()
X = iris.data
y = iris.target

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
                                                    random_state=42)

gnb = GaussianNB()

gnb.fit(X_train, y_train)

y_pred = gnb.predict(X_test)

print(f"Accuracy Score: {accuracy_score(y_test, y_pred):.4f}")
print("\nClassification Report:\n")
print(classification_report(y_test, y_pred, target_names=iris.target_names))
```

Over News Groups:

```
from sklearn.datasets import fetch_20newsgroups
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.pipeline import make_pipeline

data = fetch_20newsgroups(subset='train', categories=['alt.atheism', 'comp
    .graphics'])
X_train = data.data
y_train = data.target

model = make_pipeline(CountVectorizer(), MultinomialNB())

model.fit(X_train, y_train)

sample_text = ["This is about computer graphics and software rendering"]
prediction = model.predict(sample_text)

print(f"Predicted category index: {prediction[0]}")
print(f"Predicted category name: {data.target_names[prediction[0]]}")
```

Naive Bayes: Hyperparameter Hierarchy

Naive Bayes মডেলগুলো মূলত তাদের গাণিতিক ডিস্ট্রিবিউশন অনুমানের ওপর ভিত্তি করে কাজ করে, তাই এতে অন্যান্য অ্যালগরিদমের (যেমন: Random Forest) তুলনায় টিউন করার মতো প্যারামিটার কিছুটা কম।

গুরুত্ব	প্যারামিটার	কেন গুরুত্বপূর্ণ?	কোন মডেলে প্রযোজ্য?
সবচেয়ে বেশি	alpha	এটি Laplace/Lidstone Smoothing নিয়ন্ত্রণ করে। এটি নিশ্চিত করে যে কোনো ফিচার অনুপস্থিত থাকলেও মডেল যেন ০-এর কারণে পুরো গুণফল ০ না করে ফেলে।	Multinomial, Bernoulli, Complement, Categorical
সবচেয়ে বেশি	var_smoothing	এটি ছোট ভ্যারিয়েন্সের মানগুলোকে স্টেবল করে গাণিতিক ত্রুটি রোধ করে। মডেলের স্ট্যাবিলিটির জন্য এটি অত্যন্ত কার্যকর।	Gaussian
মাঝারি	fit_prior	এটি মডেলকে শেখায় যে ডেটাসেটে ক্লাসের অনুপাত (Prior) বিবেচনা করতে হবে কি না। False দিলে সব ক্লাসকে সমান গুরুত্ব দেওয়া হয়।	সব মডেল
মাঝারি	binarize	এটি ফিচারগুলোকে নির্দিষ্ট সীমার ভিত্তিতে বাইনারি (০ বা ১) মান দেয়। টেক্সট ক্লাসিফিকেশনে এটি অত্যন্ত প্রভাবশালী।	Bernoulli
কম	min_categories	প্রতিটি ফিচারের ন্যূনতম ক্যাটাগরি সংখ্যা নির্ধারণ করে, যা ডেটার সঠিক ডিস্ট্রিবিউশন এস্টিমেশনে সাহায্য করে।	Categorical

বিস্তারিত বিশ্লেষণ

১. alpha (Smoothing Factor)

এটি সবচেয়ে গুরুত্বপূর্ণ প্যারামিটার কারণ এটি মডেলের **জেনারেলাইজেশন ক্ষমতা** নিয়ন্ত্রণ করে।

- যদি $\alpha=1.0$ হয়, তবে এটি Laplace Smoothing হিসেবে কাজ করে এবং জিরো-প্রোবাবিলিটি সমস্যা দূর করে।
- যদি আপনার ডেটাসেটে এমন কোনো শব্দ বা ফিচার থাকে যা ট্রেনিং ডেটায় নেই কিন্তু টেস্ট ডেটায় আছে, তবে alpha সেই ফিচারকে একটি ক্ষুদ্র সম্ভাবনা প্রদান করে মডেলকে অকেজো হওয়া থেকে বাঁচায়।

২. var_smoothing (Variance Smoothing)

Gaussian Naive Bayes-এর জন্য এটিই সবচেয়ে বড় ব্রেক।

- অনেক সময় ফিচারগুলোর ভ্যারিয়েন্স খুব কম থাকে, যা ক্যালকুলেশনের সময় জিরো বা নেগেটিভ মান আসার ঝুঁকি তৈরি করে।
- var_smoothing এই ভ্যারিয়েন্সের সাথে সর্বোচ্চ ভ্যারিয়েন্সের একটি অংশ যোগ করে মডেলকে গাণিতিকভাবে স্থিতিশীল রাখে।

৩. binarize (Threshold for Binary Conversion)

BernoulliNB-এর ক্ষেত্রে এটি একটি গেম-চেঞ্জার।

- যদি আপনার ইনপুট ডেটা কন্টিনিউয়াস হয়, তবে binarize মানটি একটি নির্দিষ্ট থ্রেশহোল্ড হিসেবে কাজ করে যা ডেটাকে ০ বা ১ এ রূপান্তর করে দেয়। এটি মডেলের সিদ্ধান্ত গ্রহণ প্রক্রিয়ায় সরাসরি প্রভাব ফেলে।

৪. fit_prior (Class Prior)

এটি মডেলের সিদ্ধান্তের ওপর ভিত্তি হিসেবে কাজ করে। ডিফল্ট True রাখা ভালো কারণ এটি ডেটার প্রকৃত ক্লাসের অনুপাত শিখতে পারে। তবে যদি আপনার ডেটাসেট প্রবলভাবে ইমব্যালেন্সড হয় এবং আপনি চান মডেল পক্ষপাতমুক্ত সিদ্ধান্ত নিক, তবে fit_prior=False ব্যবহার করে সব ক্লাসকে সমান অগ্রাধিকার দেওয়া যেতে পারে।

Naive Bayes প্রশ্ন ও উত্তর

প্রশ্ন ১: Naive Bayes-কে কেন 'Naive' বা 'সরল' বলা হয়?

উত্তর: Naive Bayes-কে 'Naive' বলা হয় কারণ এটি একটি শক্তিশালী কিন্তু সরল ধারণা ধরে নেয় যে, একটি ডেটাসেটের প্রতিটি ফিচার একে অপরের থেকে সম্পূর্ণ স্বাধীন (Conditional Independence)। বাস্তবে এমন স্বাধীনতা খুব কমই পাওয়া যায়, কিন্তু এই ধারণার কারণেই মডেলটি অত্যন্ত দ্রুত কাজ করতে পারে।

প্রশ্ন ২: Naive Bayes কেন টেক্সট ক্লাসিফিকেশন বা স্প্যাম ডিটেকশনের জন্য সবচেয়ে জনপ্রিয়?

উত্তর: টেক্সট ডেটায় সাধারণত ফিচারের সংখ্যা (Vocabulary size) অনেক বেশি থাকে। Naive Bayes ফিচারের স্বাধীনতার ধারণা ব্যবহার করে জটিল এই সমস্যাকে প্রতিটি ফিচারের জন্য আলাদা ওয়ান-ডাইমেনশনাল ডিস্ট্রিবিউশনে ভাগ করে ফেলে, যা কম্পিউটেশনাল খরচ কমায় এবং 'Curse of Dimensionality' সমস্যা সমাধানে সাহায্য করে।

প্রশ্ন ৩: 'Zero Frequency' সমস্যা বলতে কী বোঝায় এবং এটি কীভাবে সমাধান করা যায়?

উত্তর: যদি কোনো ফিচার বা শব্দ ট্রেনিং সেটে না থাকে, তবে তার সম্ভাবনা (Likelihood) ০ হয়ে যায়, যার ফলে পুরো গুণফল ০ হয়ে যায়। একে 'Zero Frequency' সমস্যা বলে। এটি সমাধানের জন্য Laplace Smoothing বা Lidstone Smoothing ব্যবহার করা হয়, যেখানে সম্ভাবনা গণনায় একটি ছোট মান (যেমন ১) যোগ করা হয় যাতে ফলাফল কখনো ০ না হয়।

প্রশ্ন ৪: লজিস্টিক রিগ্রেশন এবং Naive Bayes-এর মধ্যে মূল পার্থক্য কী?

উত্তর: লজিস্টিক রিগ্রেশন একটি 'Discriminative' মডেল, যা সরাসরি ক্লাসের সীমানা (Decision Boundary) শেখার চেষ্টা করে। অন্যদিকে Naive Bayes একটি 'Generative' মডেল, যা প্রতিটি ক্লাসের জন্য ডেটার ডিস্ট্রিবিউশন কেমন হবে তা শেখে এবং বেইজ থিওরেম ব্যবহার করে সম্ভাবনা নির্ণয় করে।

প্রশ্ন ৫: Naive Bayes কি আউটলায়ারের (Outliers) প্রতি সংবেদনশীল?

উত্তর: Naive Bayes আউটলায়ারের প্রতি লিনিয়ার রিগ্রেশন বা KNN-এর মতো অতটা সংবেদনশীল নয়। কারণ এটি পুরো ডেটাসেটের গড় (Mean) ও ভেদাঙ্ক (Variance) বা ফ্রিকোয়েন্সির ওপর ভিত্তি করে সম্ভাবনা নির্ণয় করে, কোনো নির্দিষ্ট দূরত্বের (Distance) ওপর নয়।

প্রশ্ন ৬: কখন Gaussian Naive Bayes বনাম Multinomial Naive Bayes ব্যবহার করবেন?

উত্তর: যখন ডেটার ফিচারগুলো কন্টিনিউয়াস (Continuous) এবং নরমাল ডিস্ট্রিবিউশন মেনে চলে, তখন GaussianNB ব্যবহার করা হয় (যেমন: ফুলের দৈর্ঘ্য/প্রস্থ)। আর যখন ডেটাগুলো ডিসক্রিট কাউন্ট (যেমন: শব্দের সংখ্যা) বা মাল্টিনোমিয়াল ডিস্ট্রিবিউশন মেনে চলে, তখন MultinomialNB ব্যবহার করা হয় (যেমন: টেক্সট ক্লাসিফিকেশন)।

- **মৌলিক ধারণা:** Naive Bayes অ্যালগরিদমগুলো সুপারভাইজড লার্নিংয়ের একটি সেট যা বেইজ থিওরেম এবং প্রতিটি ফিচারের মধ্যে শর্তাধীন স্বাধীনতার (conditional independence) সরল অনুমানের ওপর ভিত্তি করে কাজ করে।
- **গাণিতিক ভিত্তি:** এটি ক্লাস ভ্যারিয়েবল এবং ডিপেন্ডেন্ট ফিচার ভেক্টরের সম্পর্কের ওপর ভিত্তি করে কাজ করে। ইনপুট সাপেক্ষে ক্লাসটি ধ্রুবক হওয়ায় এটি Maximum A Posteriori (MAP) এস্টিমেশন ব্যবহার করে প্রেডিকশন রুল নির্ধারণ করে।
- **ব্যবহারিক সুবিধা:** এগুলোর অনুমান সরল হওয়া সত্ত্বেও বাস্তব জীবনে, বিশেষ করে ডকুমেন্ট ক্লাসিফিকেশন এবং স্প্যাম ফিল্টারিংয়ে অত্যন্ত কার্যকর। এগুলো ট্রেনিং ডেটার অল্প পরিমাণের ওপর ভিত্তি করেই প্যারামিটার নির্ধারণ করতে পারে এবং ক্লাসিফিকেশনের গতি অত্যন্ত দ্রুত।
- **ডাইমেনশনালিটি:** প্রতিটি ডিস্ট্রিবিউশনকে আলাদাভাবে এক-মাত্রিক হিসেবে অনুমান করার ফলে এগুলো 'কার্স অফ ডাইমেনশনালিটি' (curse of dimensionality) সংক্রান্ত সমস্যা কমাতে সাহায্য করে।
- **সীমাবদ্ধতা:** এটি একটি ভালো ক্লাসিফায়ার হলেও 'বাজে এস্টিমেটর' হিসেবে পরিচিত, তাই predict_proba থেকে প্রাপ্ত প্রোবাবিলিটি আউটপুটগুলোকে খুব গুরুত্বের সাথে নেওয়া উচিত নয়।

বিভিন্ন ধরন:

- **GaussianNB:** ফিচারগুলো গাউসিয়ান ডিস্ট্রিবিউশন অনুসরণ করে বলে ধরে নেয়।
- **MultinomialNB:** মাল্টিনোমিয়ালি ডিস্ট্রিবিউটেড ডেটার জন্য ব্যবহৃত হয়, যা টেক্সট ক্লাসিফিকেশনের একটি ক্লাসিক ভেরিয়েন্ট।
- **ComplementNB:** স্ট্যান্ডার্ড মাল্টিনোমিয়াল Naive Bayes-এর একটি অ্যাডাপ্টেশন যা ইমব্যালেন্সড ডেটাসেটের জন্য উপযোগী।
- **BernoulliNB:** বাইনারি বা বুলিয়ান ডেটার ক্ষেত্রে ব্যবহৃত হয়।
- **CategoricalNB:** ক্যাটাগরিক্যাল ডিস্ট্রিবিউশন অনুসরণকারী ডেটার ক্ষেত্রে প্রযোজ্য।
- **ইনক্রিমেন্টাল লার্নিং:** বড় স্কেলের ডেটার জন্য partial_fit মেথড ব্যবহার করা যায় যা পুরো ডেটাসেট মেমোরিতে ধরে না এমন সমস্যার সমাধান করে।

L1 (Lasso) এবং L2 (Ridge) Regularization-এর মধ্যে পার্থক্য

Linear Regression-এ Model এমনভাবে Coefficient (Weight) শিখে যাতে Prediction Error বা Mean Squared Error (MSE) সর্বনিম্ন হয়। কিন্তু অনেক সময় Model কিছু Feature-এর জন্য অত্যন্ত বড় Weight শিখে ফেলে, যার ফলে Overfitting এবং Multicollinearity-এর মতো সমস্যা দেখা দেয়। এই সমস্যা কমানোর জন্য Regularization ব্যবহার করা হয়। Regularization-এর দুটি জনপ্রিয় পদ্ধতি হলো L1 (Lasso) এবং L2 (Ridge)। উভয়ের উদ্দেশ্য Weight নিয়ন্ত্রণ করা হলেও তাদের কাজ করার পদ্ধতি ভিন্ন।

L1 (Lasso) Regularization

L1 Regularization-এ Loss Function-এর সাথে সকল Weight-এর Absolute Value ($|w|$) যোগ করা হয়। ফলে Model চেষ্টা করে অপ্রয়োজনীয় Feature-এর Weight একেবারে 0 করে দিতে। অর্থাৎ যেসব Feature Prediction-এর জন্য তেমন গুরুত্বপূর্ণ নয়, সেগুলো Model নিজেই বাদ দেয়। এজন্য L1 Regularization-কে Feature Selection Technique-ও বলা হয়।

Formula হলো:

$$\text{Loss} = \text{MSE} + \lambda \sum_{i=1}^n |w_i|$$

এখানে,

- **MSE** = Mean Squared Error
- **λ (Lambda)** = Regularization Parameter
- **w** = Model-এর Coefficient বা Weight

এই Formula-তে দেখা যাচ্ছে, প্রতিটি Weight-এর Absolute Value যোগ করা হচ্ছে। ফলে যদি কোনো Feature Prediction-এর জন্য খুব বেশি গুরুত্বপূর্ণ না হয়, তাহলে Model তার Weight ধীরে ধীরে 0 করে দেয়। অর্থাৎ অপ্রয়োজনীয় Feature Model থেকে বাদ পড়ে যায়। এজন্য L1 Regularization-কে **Feature Selection Technique** বলা হয়।

ধরো একটি Model Train করার পরে Weight পাওয়া গেল:

Age = 18

Salary = 32

Balance = 25

Gender = 6

City = 4

L1 Regularization প্রয়োগ করার পরে ফলাফল হতে পারে:

Age = 18

Salary = 0

Balance = 22

Gender = 0

City = 3

এখানে Salary এবং Gender-এর Weight 0 হয়ে গেছে। অর্থাৎ Model সিদ্ধান্ত নিয়েছে এই Feature দুটি Prediction-এর জন্য প্রয়োজনীয় নয়। তাই ভবিষ্যতে Prediction করার সময় এই Feature দুটি আর ব্যবহার হবে না।

যেসব Dataset-এ অনেক Feature থাকে এবং ধারণা করা হয় যে সব Feature গুরুত্বপূর্ণ নয়, সেখানে L1 (Lasso) সবচেয়ে কার্যকর।

L2 (Ridge) Regularization

L2 Regularization-এ Loss Function-এর সাথে সকল Weight-এর Square (w^2) যোগ করা হয়। এর ফলে বড় Weight-এর জন্য বেশি Penalty হয় এবং Model সব Weight ছোট করার চেষ্টা করে। তবে L2 সাধারণত কোনো Weight-কে 0 করে না। অর্থাৎ সব Feature Model-এ থাকে, শুধু তাদের গুরুত্ব (Weight) কমে যায়।

L2 Regularization-এ মূল Loss Function-এর সাথে সকল Weight-এর **Square (w^2)** যোগ করা হয়। এর Formula হলো:

$$\text{Loss} = \text{MSE} + \lambda \sum_{i=1}^n w_i^2$$

এখানে,

- **MSE** = Mean Squared Error
- **λ (Lambda)** = Regularization Parameter
- **w** = Model-এর Coefficient বা Weight

এই Formula-তে প্রতিটি Weight-এর Square যোগ করা হয়। ফলে Weight যত বড় হবে, Penalty তত বেশি হবে। তাই Model সব Weight ছোট করার চেষ্টা করে। তবে L2 সাধারণত কোনো Weight-কে 0 করে না। অর্থাৎ সব Feature Model-এ থাকে, শুধু তাদের গুরুত্ব (Weight) কমে যায়।

ধরো একই Model-এর Weight ছিল:

Age = 18

Salary = 32

Balance = 25

Gender = 6

City = 4

L2 Regularization প্রয়োগ করার পরে ফলাফল হতে পারে:

Age = 11

Salary = 19

Balance = 15

Gender = 3

City = 2

এখানে লক্ষ্য করো, কোনো Weight 0 হয়নি। বরং সবগুলোই আগের তুলনায় ছোট হয়েছে। অর্থাৎ Model সব Feature ব্যবহার করছে, কিন্তু কোনো Feature-কে অতিরিক্ত গুরুত্ব দিচ্ছে না।

যেসব Dataset-এ সব Feature গুরুত্বপূর্ণ, কিন্তু Feature-গুলোর মধ্যে Multicollinearity রয়েছে অথবা Overfitting কমাতে হয়, সেখানে L2 (Ridge) সবচেয়ে উপযোগী।

L1 (Lasso) এবং L2 (Ridge)-এর মূল পার্থক্য

L1 Regularization Weight-এর Absolute Value ($|w|$) ব্যবহার করে এবং অপ্রয়োজনীয় Feature-এর Weight 0 করে দিয়ে Feature Selection করে। অন্যদিকে L2 Regularization Weight-এর Square (w^2) ব্যবহার করে এবং সব Weight ছোট করে, কিন্তু সাধারণত কোনো Weight-কে 0 করে না। তাই L1 ব্যবহার করা হয় যখন Feature Selection প্রয়োজন হয়, আর L2 ব্যবহার করা হয় যখন সব Feature রাখা দরকার কিন্তু তাদের Weight নিয়ন্ত্রণ করতে হয়।

সংক্ষেপে বলা যায়, L1 (Lasso) অপ্রয়োজনীয় Feature বাদ দেয়, আর L2 (Ridge) সব Feature রেখে তাদের Weight কমিয়ে Model-কে আরও স্থিতিশীল করে।

Linear Regression, Ridge Regression, Lasso Regression এবং Elastic Net-এর মধ্যে পার্থক্য

Linear Regression, Ridge Regression, Lasso Regression এবং Elastic Net চারটিই Regression Algorithm এবং Continuous Value Prediction-এর জন্য ব্যবহৃত হয়। এদের মূল Model Equation একই হলেও Loss Function এবং Regularization-এর কারণে কাজের ধরনে পার্থক্য দেখা যায়। কোন Model কখন ব্যবহার করতে হবে, তা Dataset-এর বৈশিষ্ট্য এবং সমস্যার উপর নির্ভর করে।

Linear Regression

Linear Regression হলো সবচেয়ে মৌলিক Regression Algorithm। এটি কোনো ধরনের Regularization ব্যবহার করে না। Model শুধুমাত্র Prediction Error কমানোর চেষ্টা করে। যদি Dataset পরিষ্কার হয় এবং Overfitting বা Multicollinearity-এর সমস্যা না থাকে, তাহলে এটি খুব ভালো ফলাফল দেয়। কিন্তু Dataset বড় বা জটিল হলে অনেক সময় Coefficient (Weight) অনেক বড় হয়ে যায়, যার ফলে Model Overfit করতে পারে।

Formula

$$\text{Loss} = \text{MSE}$$

যেখানে,

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

ধরো Train করার পরে Coefficient পাওয়া গেল:

Age = 25

Salary = 40

Balance = 18

Gender = 8

City = 5

এখানে কোনো ধরনের Penalty নেই। তাই কোনো কোনো Weight অনেক বড় হতে পারে।

কখন ব্যবহার করবেন

- Dataset ছোট হলে।
- Overfitting না থাকলে।

- Feature-গুলোর মধ্যে Multicollinearity না থাকলে।
- সাধারণ Regression Problem-এর জন্য।

Ridge Regression

Ridge Regression হলো Linear Regression-এর একটি উন্নত সংস্করণ যেখানে **L2 Regularization** ব্যবহার করা হয়। এখানে Loss Function-এর সাথে প্রতিটি Weight-এর Square যোগ করা হয়। ফলে বড় Weight-এর জন্য বেশি Penalty হয় এবং Model সব Weight ছোট করার চেষ্টা করে। তবে Ridge সাধারণত কোনো Weight-কে Zero করে না। অর্থাৎ সব Feature Model-এ থেকে যায়।

Formula

$$\text{Loss} = \text{MSE} + \lambda \sum_{i=1}^n w_i^2$$

যেখানে,

- MSE = Mean Squared Error
- λ = Regularization Parameter
- w_i = প্রতিটি Feature-এর Weight

ধরো Linear Regression-এর Weight ছিল:

Age = 25

Salary = 40

Balance = 18

Gender = 8

City = 5

Ridge করার পরে হতে পারে:

Age = 15

Salary = 22

Balance = 11

Gender = 4

City = 3

এখানে দেখা যাচ্ছে সব Weight ছোট হয়েছে, কিন্তু কোনো Weight Zero হয়নি।

কখন ব্যবহার করবেন

- Feature-গুলোর মধ্যে Multicollinearity থাকলে।

- সব Feature গুরুত্বপূর্ণ হলে।
- Overfitting কমাতে চাইলে।
- সব Feature রেখে Stable Model তৈরি করতে চাইলে।

Lasso Regression

Lasso Regression হলো Linear Regression-এর একটি উন্নত সংস্করণ যেখানে **L1 Regularization** ব্যবহার করা হয়। এখানে Loss Function-এর সাথে প্রতিটি Weight-এর Absolute Value যোগ করা হয়। এর ফলে Model অপয়োজনীয় Feature-এর Weight একেবারে Zero করে দিতে পারে। তাই Lasso স্বয়ংক্রিয়ভাবে Feature Selection করতে পারে।

Formula

$$\text{Loss} = \text{MSE} + \lambda \sum_{i=1}^n |w_i|$$

যেখানে,

- MSE = Mean Squared Error
- λ = Regularization Parameter
- w_i = প্রতিটি Feature-এর Weight

ধরো Linear Regression-এর Weight ছিল:

Age = 25

Salary = 40

Balance = 18

Gender = 8

City = 5

Lasso করার পরে হতে পারে:

Age = 21

Salary = 0

Balance = 15

Gender = 0

City = 3

এখানে Salary এবং Gender-এর Weight Zero হয়ে গেছে। অর্থাৎ Model সিদ্ধান্ত নিয়েছে এই Feature দুটি Prediction-এর জন্য গুরুত্বপূর্ণ নয়।

কখন ব্যবহার করবেন

- Dataset-এ অনেক Feature থাকলে।
- অনেক Feature অপয়োজনীয় হলে।
- Automatic Feature Selection প্রয়োজন হলে।
- High-dimensional Dataset-এর ক্ষেত্রে।

Elastic Net

Elastic Net হলো Ridge এবং Lasso-এর সমন্বিত সংস্করণ। এতে L1 এবং L2 উভয় ধরনের Regularization ব্যবহার করা হয়। ফলে কিছু Feature-এর Weight Zero হয়ে যায় এবং বাকি Weight-গুলো ছোট হয়ে যায়। এটি Feature Selection এবং Multicollinearity উভয় সমস্যাই একসাথে সমাধান করতে পারে।

Formula

$$\text{Loss} = \text{MSE} + \lambda_1 \sum_{i=1}^n |w_i| + \lambda_2 \sum_{i=1}^n w_i^2$$

যেখানে,

- MSE = Mean Squared Error
- λ_1 = L1 Regularization-এর Penalty
- λ_2 = L2 Regularization-এর Penalty
- w_i = প্রতিটি Feature-এর Weight

ধরো Linear Regression-এর Weight ছিল:

Age = 25

Salary = 40

Balance = 18

Gender = 8

City = 5

Elastic Net করার পরে হতে পারে:

Age = 18

Salary = 20

Balance = 12

Gender = 0

City = 2

এখানে Gender-এর Weight Zero হয়ে গেছে এবং বাকি সব Weight ছোট হয়েছে। অর্থাৎ Model একই সঙ্গে Feature Selection এবং Weight Shrinkage করেছে।

কখন ব্যবহার করবেন

- Dataset-এ অনেক Feature থাকলে।
- অনেক Feature একে অপরের সাথে Correlated হলে।
- একই সঙ্গে Feature Selection এবং Overfitting নিয়ন্ত্রণ করতে চাইলে।
- বড় ও জটিল Dataset-এর ক্ষেত্রে।

চারটি Model-এর মূল পার্থক্য

Linear Regression কোনো Regularization ব্যবহার করে না। Ridge Regression L2 Regularization ব্যবহার করে সব Feature রেখে তাদের Weight ছোট করে। Lasso Regression L1 Regularization ব্যবহার করে অপ্রয়োজনীয় Feature-এর Weight Zero করে Feature Selection করে। অন্যদিকে Elastic Net L1 এবং L2 উভয় Regularization ব্যবহার করে, ফলে এটি একই সঙ্গে Feature Selection এবং Weight Shrinkage করতে পারে।

- **Linear Regression:** কোনো Regularization নেই।
- **Ridge Regression:** সব Feature রাখে, শুধু Weight ছোট করে।
- **Lasso Regression:** অপ্রয়োজনীয় Feature বাদ দেয় (Weight = 0)।
- **Elastic Net:** কিছু Feature বাদ দেয় এবং বাকি Weight-ও ছোট করে।

Gradient Boosting Algorithm

শুরুতেই বুঝি: Ensemble Learning আসলে কী?

ধরুন, আপনি একটা কঠিন সিদ্ধান্ত নিতে যাচ্ছেন — একটা শেয়ারে বিনিয়োগ করবেন কিনা। আপনি একজন বিশেষজ্ঞের মতামত না নিয়ে ১০০ জন বিশেষজ্ঞের মতামত নিয়ে সিদ্ধান্ত নিলে, সাধারণত সিদ্ধান্তটা বেশি নির্ভরযোগ্য হয়। এটাই হলো **Ensemble Learning**-এর মূল দর্শন — একাধিক "সিদ্ধান্ত নেওয়ার মডেল" একত্রিত করে একটা অনেক বেশি শক্তিশালী ও নির্ভুল ফলাফল তৈরি করা।

এখন এই ১০০ জন বিশেষজ্ঞকে একত্রিত করার দুইটা সম্পূর্ণ ভিন্ন পদ্ধতি আছে:

- **পদ্ধতি ১ (Bagging — Random Forest যেভাবে কাজ করে):** ১০০ জন বিশেষজ্ঞকে একসাথে, আলাদা আলাদা ঘরে বসিয়ে দিন। কেউ কারো মতামত জানবে না। সবাই স্বাধীনভাবে (Independently) নিজের সিদ্ধান্ত দেবে, শেষে সবার ভোট গড় করে চূড়ান্ত সিদ্ধান্ত নেওয়া হবে।
- **পদ্ধতি ২ (Boosting — Gradient Boosting যেভাবে কাজ করে):** এখানে বিশেষজ্ঞরা একসাথে বসেন না, বরং একজনের পর একজন কাজ করেন। প্রথম বিশেষজ্ঞ একটা মোটামুটি অনুমান করলেন। দ্বিতীয় বিশেষজ্ঞ প্রথমজনের অনুমানটা দেখে বললেন, "আপনি এখানে এখানে ভুল করেছেন, আমি শুধু সেই ভুলটুকু ঠিক করার চেষ্টা করছি।" তৃতীয়জন আবার প্রথম দুইজন মিলে যে ভুলটুকু রয়ে গেছে, শুধু সেটাই ঠিক করার চেষ্টা করেন। এভাবে একের পর এক, প্রতিজন আগের সবার সম্মিলিত ভুল সংশোধন করতে থাকেন।

Gradient Boosting হলো ঠিক দ্বিতীয় পদ্ধতিটাই — একটা Sequential (ধারাবাহিক) Ensemble পদ্ধতি, যেখানে প্রতিটি নতুন মডেল আগের সব মডেলের সম্মিলিত ভুল (Error/Residual) সংশোধন করার চেষ্টা করে।

নামের মধ্যে লুকানো দুটি শব্দের রহস্য

"Boosting" শব্দটার মানে: দুর্বল জিনিসকে ধাপে ধাপে "বুস্ট" করে শক্তিশালী করা। এখানে "দুর্বল" বলতে বোঝানো হয়েছে এমন একটা ছোট মডেলকে, যেটা একা একা ভালো প্রেডিকশন দিতে পারে না — কিন্তু অনেকগুলো একসাথে সিরিজে বসালে দারুণ কাজ করে। ঠিক যেমন একজন সদ্য শেখা শিক্ষার্থী একটা বড় পরীক্ষায় ভালো করতে পারে না, কিন্তু ১০০ জন সদ্য শেখা শিক্ষার্থী পালা করে করে একে অপরের ভুল ঠিক করতে করতে একসাথে খুব ভালো ফলাফল দিতে পারে।

"Gradient" শব্দটার মানে: এটা এসেছে গণিতের একটা বিখ্যাত কৌশল **Gradient Descent** থেকে। Gradient Descent-এ আমরা কোনো একটা Loss (ভুলের পরিমাণ) কমানোর জন্য, সেই Loss-এর "ঢাল" (Gradient) বা সবচেয়ে খাড়া নিচের দিকের রাস্তা অনুসরণ করে ধীরে ধীরে এগোই — যেমন পাহাড় থেকে নামার সময় আপনি সবসময় সবচেয়ে খাড়া নিচের দিকের রাস্তা খুঁজে খুঁজে নামেন যাতে দ্রুত নিচে পৌঁছান।

সাধারণ Gradient Descent-এ আমরা মডেলের **সংখ্যা** (Parameter, যেমন w, b) পরিবর্তন করি Loss কমানোর জন্য। কিন্তু Gradient Boosting-এ আমরা সংখ্যা পরিবর্তন করি না — বরং প্রতিটি ধাপে একটা **সম্পূর্ণ নতুন গাছ** (Decision Tree) যোগ করি, যেটা ওই মুহূর্তে Loss কমানোর সবচেয়ে ভালো দিকটা নির্দেশ করে। তাই একে

বলা হয় "Function Space-এ Gradient Descent" — কারণ এখানে আমরা সংখ্যার জায়গায় একটা সম্পূর্ণ ফাংশন (গাছ) যোগ করছি।

একটা গল্পের মাধ্যমে বোঝা যাক (Intuition)

ধরুন আপনি একজন ছাত্রের বার্ষিক পরীক্ষার নম্বর অনুমান করার চেষ্টা করছেন। ছাত্রটার প্রকৃত নম্বর হলো ১০০-এর মধ্যে ৮৮, কিন্তু আপনি এটা আগে থেকে জানেন না — আপনাকে ধাপে ধাপে অনুমান করে এই সংখ্যার কাছে পৌঁছাতে হবে।

ধাপ ১ (প্রথম, সবচেয়ে সহজ মডেল): আপনার হাতে কোনো তথ্য নেই, তাই আপনি পুরো ক্লাসের গড় নম্বর অনুমান করে বললেন — ৬৫। এখন হিসাব করুন, ভুলের পরিমাণ (Error) কত? প্রকৃত (৮৮) — অনুমান (৬৫) = ২৩। মানে আপনি এখনো ২৩ নম্বর কম বলেছেন।

ধাপ ২ (দ্বিতীয় মডেল – শুধু ভুলটুকু শেখে): এখন আরেকটা ছোট মডেল তৈরি হলো, যার একমাত্র কাজ হলো এই "২৩" সংখ্যাটাকেই প্রেডিক্ট করার চেষ্টা করা — লক্ষ্য করুন, এই মডেল কিন্তু ছাত্রের প্রকৃত নম্বর (৮৮) শেখার চেষ্টা করছে না, সে শুধু প্রথম মডেলের ভুলটুকু শেখার চেষ্টা করছে। ধরা যাক এই দ্বিতীয় মডেল অনুমান করলো ১৪।

এখন মোট প্রেডিকশন = ৬৫ + ১৪ = ৭৯। আগে ছিল ৬৫, এখন হলো ৭৯ — যা প্রকৃত মান ৮৮-এর অনেক কাছাকাছি চলে এসেছে। কিন্তু এখনও একটু পার্থক্য আছে: ৮৮ - ৭৯ = ৯।

ধাপ ৩ (তৃতীয় মডেল – আবার নতুন ভুলটুকু শেখে): এখন তৃতীয় একটা মডেল এই নতুন "৯" পার্থক্যটুকু প্রেডিক্ট করার চেষ্টা করবে। এভাবে একের পর এক ছোট ছোট মডেল যোগ হতে হতে, চূড়ান্ত প্রেডিকশন ধীরে ধীরে প্রকৃত মান ৮৮-এর একদম কাছাকাছি পৌঁছে যাবে।

সবচেয়ে গুরুত্বপূর্ণ কথাটা মনে রাখুন: প্রতিটি নতুন গাছ কখনোই আসল ডেটা (ছাত্রের নম্বর ৮৮) থেকে সরাসরি শেখে না। সে শুধু শেখে — "আগের সব গাছ মিলে ঠিক কতটুকু ভুল করেছে" (একে বলে Residual)। এই জিনিসটাই Gradient Boosting-কে অন্য সব Algorithm থেকে আলাদা করে তোলে।

মূল চারটি স্তম্ভ (Core Structure) — বিস্তারিত ব্যাখ্যা

১. Weak Learner (দুর্বল শিক্ষার্থী) – এরা কেন ইচ্ছাকৃতভাবেই "দুর্বল" রাখা হয়?

প্রতিটি ছোট মডেল হিসেবে সাধারণত একটা খুবই ছোট ও অগভীর (Shallow) Decision Tree ব্যবহার করা হয় — যার গভীরতা (max_depth) সাধারণত মাত্র ৩ থেকে ৮ রাখা হয়।

এখানে একটা গুরুত্বপূর্ণ প্রশ্ন আসতে পারে — কেন ইচ্ছাকৃতভাবে দুর্বল মডেল ব্যবহার করা হয়, শক্তিশালী মডেল কেন নয়? এর উত্তর হলো: যদি প্রথম গাছটাই খুব শক্তিশালী ও গভীর হয়, তাহলে সে একাই পুরো ডেটার প্যাটার্ন (এমনকি Noise/ভুল তথ্যও) মুখস্থ করে ফেলবে, এবং পরের কোনো গাছের শেখার মতো তেমন কোনো Residual অবশিষ্ট থাকবে না। ফলে মডেল খুব দ্রুত Overfit করে ফেলবে — অর্থাৎ Training ডেটায় নিখুঁত কিন্তু নতুন ডেটায় ভয়াবহ খারাপ ফল দেবে।

তাই ইচ্ছা করেই প্রতিটি গাছকে "দুর্বল" রাখা হয়, যাতে একটা গাছ শুধু সামান্য কিছু শেখে, বাকিটুকু পরের গাছের জন্য রেখে দেয় — এভাবে ধীরে ধীরে, নিয়ন্ত্রিতভাবে শেখাটাই মডেলকে স্থিতিশীল ও Generalizable (নতুন ডেটাতেও ভালো কাজ করার উপযোগী) করে তোলে।

২. Sequential Learning (ধারাবাহিক শিক্ষণ) – কেন গাছগুলো একসাথে তৈরি করা যায় না?

Random Forest-এ প্রতিটা গাছ একে অপরের থেকে সম্পূর্ণ স্বাধীন — তাই কম্পিউটারের একাধিক Core একসাথে ব্যবহার করে সবগুলো গাছ একই সময়ে (Parallel-ভাবে) তৈরি করা সম্ভব, যা দ্রুতগতির।

কিন্তু Gradient Boosting-এ এটা করা একদমই অসম্ভব, কারণ:

- দ্বিতীয় গাছ তৈরি করার আগে অবশ্যই জানতে হবে **প্রথম গাছটা ঠিক কী প্রেডিক্ট করেছে** এবং তার Residual কত।
- তৃতীয় গাছ তৈরি করার আগে জানতে হবে **প্রথম + দ্বিতীয় গাছ মিলে** এখন কী প্রেডিক্ট করছে।

অর্থাৎ প্রতিটি গাছ তার আগের গাছের ফলাফলের ওপর সম্পূর্ণ নির্ভরশীল — একটা গাছ সম্পূর্ণ শেষ না হওয়া পর্যন্ত পরেরটা শুরুই করা যায় না। এই কারণেই এটা এক ধরনের "ওয়ান-বাই-ওয়ান লাইনে দাঁড়ানো" প্রক্রিয়া, আর এই কারণেই এটা Random Forest-এর তুলনায় ধীরগতির।

৩. Additive Model (সংযোজনমূলক মডেল) – চূড়ান্ত উত্তর কীভাবে তৈরি হয়?

চূড়ান্ত প্রেডিকশন কখনোই কোনো একটা গাছের একক ফলাফল নয় — এটা সবগুলো গাছের প্রেডিকশনের একটা যোগফল। সূত্রটা এরকম:

$$F(x) = F_0(x) + \eta \cdot h_1(x) + \eta \cdot h_2(x) + \eta \cdot h_3(x) + \dots + \eta \cdot h_m(x)$$

এটাকে সহজভাবে বুঝতে চাইলে একটা লেয়ার-কেক ভাবুন:

- $F_0(x)$ হলো একদম প্রথম, ভিত্তি স্তর — সাধারণত পুরো ডেটার গড় মান (আমাদের গল্পে যেটা ছিল ৬৫)।
- $h_1, h_2, h_3, \dots$ — এগুলো একের পর এক তৈরি হওয়া গাছ, প্রতিটাই আগের সম্মিলিত ভুলের একটা অংশ ঠিক করছে।
- η (Eta / Learning Rate) — একটা ছোট সংখ্যা (যেমন ০.১), যেটা প্রতিটা গাছের অবদানকে খানিকটা "সংযত" করে যোগ করে।

η কেন গুরুত্বপূর্ণ, একটা উদাহরণ দিয়ে বুঝি: ধরুন দ্বিতীয় গাছ প্রেডিক্ট করলো Residual = ১৪। যদি $\eta = ১.০$ (সম্পূর্ণ ওজনে) হতো, তাহলে পুরো ১৪-ই একবারে যোগ হতো — অনেকটা "এক লাফে অনেকদূর" যাওয়ার মতো, যা মডেলকে অস্থির করে দিতে পারে। কিন্তু $\eta = ০.১$ হলে মাত্র $১৪ \times ০.১ = ১.৪$ যোগ হবে — অর্থাৎ প্রতিটা ধাপে সামান্য একটু করে, সতর্কভাবে এগোনো হচ্ছে। এটা অনেকটা গাড়ি চালানোর মতো — একবারে ফুল স্পিডে এক্সেলারেটর না চেপে, ধীরে ধীরে গতি বাড়ানোই নিরাপদ ও নিয়ন্ত্রিত।

৪. Gradient Descent Analogy — এটাই যে "গ্র্যাডিয়েন্ট" নামের আসল কারণ

সাধারণ Machine Learning মডেলে (যেমন Linear Regression) আমরা Gradient Descent ব্যবহার করে মডেলের ওজন/Parameter (w, b) ধীরে ধীরে পরিবর্তন করি, যাতে Loss কমতে থাকে। প্রতিবার আমরা Loss-এর

"Negative Gradient" (সবচেয়ে খাড়া নিচের দিকের ঢাল) অনুসরণ করি — এটাই সবচেয়ে দ্রুত Loss কমানোর দিক।

Gradient Boosting একই দর্শন অনুসরণ করে, কিন্তু একটা গুরুত্বপূর্ণ পার্থক্য নিয়ে:

- সাধারণ Gradient Descent: প্রতি ধাপে **একটা সংখ্যা** (Parameter) আপডেট হয়।
- Gradient Boosting: প্রতি ধাপে **একটা সম্পূর্ণ নতুন Decision Tree** তৈরি হয়ে যোগ হয়, যেটা আসলে সেই মুহূর্তের Negative Gradient-কেই প্রতিনিধিত্ব করে।

এই কারণেই একে বলা হয় **"Function Space-এ Gradient Descent"** — কারণ আমরা এখানে সংখ্যার জগতে নয়, বরং ফাংশনের (গাছের) জগতে ধাপে ধাপে এগোচ্ছি।

Gradient Boosting Algorithm

গাণিতিক ভিত্তি – ব্যাকএন্ডে আসলে কী ঘটে, ধাপে ধাপে

Part 1-এ আমরা গল্পের মাধ্যমে বুঝেছি Gradient Boosting কীভাবে কাজ করে। এখন সেই একই জিনিস গাণিতিকভাবে, প্রতিটি ধাপ আলাদা করে বুঝি।

ধাপ ১: প্রাথমিক প্রেডিকশন (Initial Prediction) — যাত্রা শুরুর বিন্দু

মডেল শুরু হয় একটা অত্যন্ত সাধারণ, একক Constant মান দিয়ে — এমন একটা মান যা পুরো Training ডেটাসেটের ওপর Loss Function-কে সর্বনিম্ন করে।

$$F_0(x) = \underset{\gamma}{\operatorname{argmin}} \sum_i L(y_i, \gamma)$$

এই সূত্রটা আসলে বলছে: "এমন একটা γ (গামা) সংখ্যা খুঁজে বের করো, যেটা বসালে সব ডেটাপয়েন্টের মিলিত Loss সবচেয়ে কম হয়।"

- **Regression সমস্যা (Loss = MSE হলে):** গাণিতিকভাবে প্রমাণিত যে, MSE-কে সর্বনিম্ন করা γ -এর মান হলো টার্গেটের **গড় (Mean)**। এই কারণেই আমরা Part 1-এ ৬৫ (গড় নম্বর) দিয়ে শুরু করেছিলাম — এটা কোনো এলোমেলো অনুমান ছিল না, বরং গাণিতিকভাবে সবচেয়ে ভালো "প্রথম অনুমান"।
- **Classification সমস্যা:** এখানে Mean কাজ করে না (কারণ Class লেবেল সংখ্যা নয়), তাই এখানে ব্যবহার করা হয় **Log-Odds**-এর একটা প্রাথমিক মান।

ধাপ ২: Pseudo-Residual গণনা করা – "কতটুকু ভুল হলো" বের করা

প্রতিটি Iteration-এর শুরুতে, বর্তমান মডেলের প্রেডিকশন এবং প্রকৃত মানের মধ্যে একটা **Pseudo-Residual** বের করা হয়। এটা আসলে Loss Function-এর Negative Gradient:

$$r_{im} = -[\partial L(y_i, F(x_i)) / \partial F(x_i)]$$

এই সূত্রটা দেখে ভয় পাওয়ার কিছু নেই — এর ভেতরের অর্থটা খুব সহজ। এটা মূলত বলছে: "Loss যদি একটু পরিবর্তন হয়, তাহলে সেটা কোন দিকে সবচেয়ে বেশি কমবে, সেই দিকটা বের করো।"

সবচেয়ে সুন্দর ব্যাপার হলো: Regression-এ যদি $\text{Loss} = \text{MSE}$ (Mean Squared Error) ব্যবহার করা হয়, তাহলে এই জটিল ডেরিভেটিভ সূত্রটা সরলীকৃত হয়ে দাঁড়ায় শুধুমাত্র:

$$\text{Residual} = \text{Actual (y)} - \text{Predicted (F(x))}$$

অর্থাৎ প্রকৃত মান বিয়োগ অনুমিত মান — এত সহজ! এই কারণেই Part 1-এর উদাহরণে আমরা সরাসরি $(৮৮ - ৬৫ = ২৩)$ বিয়োগ করেই Residual বের করেছিলাম — এটাই আসলে Gradient গণনার সরলীকৃত রূপ।

ধাপ ৩: নতুন Weak Learner গাছ ট্রেন করা – ভুলের প্যাটার্ন শেখা

এরপর একটা নতুন, ছোট Decision Tree (h_m) ট্রেন করা হয়। এই গাছের ইনপুট হলো একই Feature-গুলো (যেমন Study Hours), কিন্তু টার্গেট হলো আসল y নয় — বরং ধাপ ২-এ পাওয়া Residual-গুলো।

এখানে একটা জিনিস স্পষ্ট করে বোঝা দরকার: এই গাছ কখনো শেখে না "কত ঘন্টা পড়লে কত নম্বর পাওয়া যায়"। সে শেখে "কত ঘন্টা পড়লে আগের মডেল কতটুকু ভুল করেছে" — অর্থাৎ সে ভুলের প্যাটার্ন শেখে, আসল উত্তরের প্যাটার্ন নয়।

ধাপ ৪: মডেল আপডেট করা – Learning Rate দিয়ে সংযত করে যোগ

নতুন তৈরি হওয়া গাছটাকে সরাসরি পুরো ওজনে যোগ না করে, Learning Rate (η) দিয়ে গুণ করে যোগ করা হয়:

$$F_m(x) = F_{m-1}(x) + \eta \cdot h_m(x)$$

এই "সংযত করে যোগ করা"টাই Gradient Boosting-এর সবচেয়ে গুরুত্বপূর্ণ সুরক্ষা কৌশল — এর ফলে মডেল একবারে বিশাল লাফ না দিয়ে ধীরে ধীরে, নিয়ন্ত্রিতভাবে উন্নত হয়, যা Overfitting-এর ঝুঁকি অনেকটাই কমিয়ে দেয়।

ধাপ ৫: পুনরাবৃত্তি (Iteration) — একই প্রক্রিয়া বহুবার চলতে থাকে

ধাপ ২ থেকে ৪ পর্যন্ত পুরো প্রক্রিয়াটা $n_{\text{estimators}}$ -এ নির্ধারিত সংখ্যক বার (যেমন ১০০ বা ৫০০ বার) পুনরাবৃত্তি হয়। প্রতিটা নতুন Iteration-এ Residual আগের চেয়ে ছোট হতে থাকে — যেন প্রতিটা নতুন বিশেষজ্ঞ আগের চেয়ে ছোট একটা ফাঁকফোকর পূরণ করছেন, আর মডেল ধীরে ধীরে প্রকৃত উত্তরের একদম কাছে পৌঁছে যাচ্ছে।

সংখ্যাসূচক উদাহরণ – সম্পূর্ণ ধাপে ধাপে হিসাব (Regression)

ধরা যাক আমাদের কাছে ৪ জন ছাত্রের ডেটা আছে:

ID	Study Hours	প্রকৃত Marks (y)
1	1	40
2	3	55
3	5	85

4	7	100
---	---	-----

ধাপ ১ – F_0 বের করা: $\text{Mean} = (40+55+85+100)/4 = 280/4 = 65$ । তাই প্রতিটি ছাত্রের জন্য প্রাথমিক অনুমান = ৬৫।

ID	Study Hours	প্রকৃত Marks	F_0	Residual ($y - F_0$)
1	1	40	65	-25
2	3	55	65	-10
3	5	85	65	+20
4	7	100	65	+35

ধাপ ২ – প্রথম গাছ h_1 তৈরি: এই গাছ Study Hours দেখে Residual (-25, -10, +20, +35) প্রেডিক্ট করার চেষ্টা করবে। ধরা যাক গাছটি একটা শর্ত দিয়ে ভাগ করলো: **Study Hours ≤ 4 কিনা?**

- বাম দিকে (Hours ≤ 4 , ID 1 ও 2): Residual-এর গড় = $(-25 + -10)/2 = -17.5$
- ডান দিকে (Hours > 4 , ID 3 ও 4): Residual-এর গড় = $(20 + 35)/2 = +27.5$

তাই $h_1(x)$ হবে -17.5 অথবা +27.5, Study Hours-এর মান অনুযায়ী।

ধাপ ৩ – মডেল আপডেট ($\eta = 0.1$ ধরে):

$$F_1(x) = F_0(x) + 0.1 \times h_1(x)$$

- ID 1: $F_1 = 65 + 0.1 \times (-17.5) = 65 - 1.75 = 63.25$ (প্রকৃত মান ৪০-এর দিকে একটু এগোলো)
- ID 4: $F_1 = 65 + 0.1 \times (27.5) = 65 + 2.75 = 67.75$ (প্রকৃত মান ১০০-এর দিকে একটু এগোলো)

লক্ষ্য করুন — খুব সামান্য একটু এগোলো, একেবারে লাফ দিয়ে ১০০-তে পৌঁছানি। এটাই Learning Rate-এর "সংযত" রাখার কাজ।

ধাপ ৪ – নতুন Residual ও চক্র চলতে থাকা: এখন নতুন Residual = $(y - F_1)$ বের হবে, যা আগের চেয়ে ছোট। দ্বিতীয় গাছ h_2 এই নতুন, ছোট Residual শেখার চেষ্টা করবে। এভাবে ধরুন ১০০ বার এই চক্র চললে, প্রতিবার Residual ক্রমশ ছোট হতে হতে শেষে মডেলের প্রেডিকশন প্রকৃত মান (৪০, ৫৫, ৮৫, ১০০)-এর একদম কাছাকাছি পৌঁছে যাবে।

এই ধীরে ধীরে এগিয়ে যাওয়াটাই Gradient Boosting-এর মূল শক্তি – এবং একইসাথে এটাই এর ধীরগতির কারণ।

গুরুত্বপূর্ণ Hyperparameter – বিস্তারিত ও ব্যবহারিক ব্যাখ্যা

n_estimators — Default: 100

কাজ: মোট কতগুলো গাছ তৈরি হবে তা নির্ধারণ করে।

Default কখন রাখব: ছোট/মাঝারি ডেটাসেটে ১০০ ভালো শুরুর বিন্দু।

ঝুঁকি: বেশি রাখলে (যেমন ১০,০০০) Training ডেটা প্রায় মুখস্থ হয়ে যায় (Overfitting) এবং সময়ও অনেক বেড়ে যায়।

কখন পরিবর্তন করব: learning_rate ছোট রাখলে (যেমন 0.01) n_estimators বাড়াতে হয়, কারণ প্রতিটা গাছ খুবই সামান্য অবদান রাখছে। বাস্তবে early_stopping বা Validation Curve দিয়ে সর্বোত্তম মান বের করাই সবচেয়ে নিরাপদ।

learning_rate — Default: 0.1

কাজ: প্রতিটা নতুন গাছের অবদানকে কতটা সংকুচিত করে যোগ করা হবে।

Default কখন রাখব: ০.১ বেশিরভাগ সাধারণ সমস্যায় ভারসাম্যপূর্ণ ফলাফল দেয়।

ঝুঁকি: বড় রাখলে (যেমন ১.০) মডেল Optimal সমাধান পার হয়ে যেতে পারে (Overshoot) এবং ফলাফল অস্থির হয়। খুব ছোট রাখলে (যেমন ০.০০১) শিখতে অনেক বেশি সময় ও গাছ লাগবে।

কখন পরিবর্তন করব: বাস্তবে সাধারণত learning_rate ছোট (০.০১-০.১) রেখে n_estimators বাড়ানো হয় — এই কম্বিনেশন সবচেয়ে স্থিতিশীল ফল দেয়, তবে ট্রেনিং সময় বেশি লাগে।

max_depth — Default: 3

কাজ: প্রতিটা Weak Learner গাছের সর্বোচ্চ গভীরতা।

Default কখন রাখব: যেহেতু দর্শনটাই "দুর্বল" মডেল একত্রিত করা, তাই ৩-৮ রেঞ্জের রাখা হয়।

ঝুঁকি: বড় রাখলে (যেমন ২০) প্রতিটা গাছ নিজেই শক্তিশালী হয়ে Noise শিখে ফেলবে, Overfitting বাড়বে।

কখন পরিবর্তন করব: Feature-দের মধ্যে জটিল Interaction থাকলে সামান্য গভীর (৫-৮) গাছ ব্যবহার করা যায়, তবে সাধারণত ৩-৫ রেঞ্জের সবচেয়ে ভালো Generalization পাওয়া যায়।

subsample — Default: 1.0

কাজ: প্রতিটা গাছ ট্রেন করার সময় Training ডেটার কত অংশ Randomly ব্যবহার হবে।

Default কখন রাখব: ১.০ মানে সম্পূর্ণ ডেটাসেট, ছোট ডেটাসেটে যথেষ্ট।

কখন পরিবর্তন করব: ১.০-এর কম (যেমন ০.৮) সেট করলে প্রতিটা গাছ ভিন্ন Random অংশ দেখে, যা Randomness যোগ করে Overfitting কমায় — একে বলা হয় **Stochastic Gradient Boosting**।

Parameter	Default	সংক্ষিপ্ত কাজ
loss	'log_loss' / 'squared_error'	কোন Loss Function দিয়ে Gradient হিসাব হবে
min_samples_split	2	Node ভাঙার জন্য ন্যূনতম Sample সংখ্যা — বাড়ালে Overfitting কমে
min_samples_leaf	1	প্রতিটা Leaf-এ ন্যূনতম Sample সংখ্যা — বাড়ালে মডেল স্মুথ হয়
max_features	None	প্রতিটা Split-এ সর্বোচ্চ কতগুলো Feature বিবেচনা হবে

গুরুত্বপূর্ণ Attribute (ট্রেনিং শেষে যা দেখা যায়)

- **feature_importances_** — প্রতিটা Feature সামগ্রিকভাবে কতটুকু Loss কমাতে সাহায্য করেছে, যোগফল সবসময় ১.০।
- **train_score_** — প্রতিটা Stage-এ Training Loss কেমন ছিল, এটা প্লট করলে Learning Curve দেখা যায় এবং বোঝা যায় কোথায় Overfitting শুরু হয়েছে।
- **estimators_** — ট্রেনিং হওয়া প্রতিটা গাছের অ্যারে, আলাদা করে পরীক্ষা করা যায়।
- **n_estimators_** — বাস্তবে কতগুলো গাছ ব্যবহৃত হয়েছে (Early Stopping থাকলে নির্ধারিত সংখ্যার চেয়ে কম হতে পারে)।

কেন এত জনপ্রিয়? (Advantages)

- **উচ্চ নির্ভুলতা:** প্রতিটা গাছ আগের ভুল সংশোধনে মনোযোগী, তাই একক Decision Tree বা Random Forest-এর চেয়েও নিখুঁত ফল দেয়।
- **নমনীয় Loss Function:** MSE (Regression), Log-Loss (Classification), Huber Loss (Outlier-Robust)-এর মতো বিভিন্ন Loss ব্যবহার করে কাস্টমাইজ করা যায়।
- **Feature Importance:** feature_importances_ থেকে সহজে বের করা যায়, Feature Selection-এ সাহায্য করে।
- **প্রতিযোগিতায় প্রমাণিত:** Kaggle-এ XGBoost, LightGBM, CatBoost-এর মতো Gradient Boosting লাইব্রেরি প্রায়ই Deep Learning-কেও পেছনে ফেলে দেয়।

সীমাবদ্ধতা (Disadvantages)

- **ধীর ট্রেনিং:** গাছগুলো Sequential হওয়ায় Parallelize করা কঠিন, তাই Random Forest-এর চেয়ে বেশি সময় লাগে।
- **Overfitting-প্রবণ:** n_estimators বেশি ও learning_rate যথাযথ না হলে মডেল Noise মুখস্থ করে ফেলতে পারে।
- **Hyperparameter-সংবেদনশীল:** learning_rate, n_estimators, max_depth-এর সঠিক সমন্বয় দরকার।
- **Outlier-সংবেদনশীল:** প্রতিটা গাছ Residual শেখে, তাই চরম Outlier থাকলে মডেল ভুল দিকে ঝুঁকে যেতে পারে।

কোন কোন ক্ষেত্রে ব্যবহার হয়?

- **ফাইন্যান্স ও ব্যাংকিং:** ক্রেডিট স্কোরিং, Fraud Detection, Algorithmic Trading Signal তৈরি।

- **ডিজিটাল মার্কেটিং:** Customer Lifetime Value অনুমান, CTR প্রেডিকশন (Google, Facebook-এর মতো প্ল্যাটফর্মে)।
- **স্বাস্থ্যসেবা:** জটিল বায়োমেডিকেল ডেটা থেকে রোগের ঝুঁকি স্কোরিং।
- **প্রতিযোগিতামূলক ডেটা সায়েন্স:** Kaggle-এ Structured/Tabular ডেটার জন্য সবচেয়ে শক্তিশালী ও জনপ্রিয় পছন্দ।

প্রশ্নোত্তর (দ্রুত ঝালাই)

প্রশ্ন: Random Forest এবং Gradient Boosting-এর মূল পার্থক্য কী?

উত্তর: Random Forest স্বাধীন গাছ Parallel-ভাবে তৈরি করে ভোট/গড় নেয় (Bagging)। Gradient Boosting গাছগুলো নির্ভরশীলভাবে Sequentially তৈরি করে, প্রতিটা নতুন গাছ আগের ভুল সংশোধন করে। ফলে Gradient Boosting বেশি Accurate কিন্তু ধীরগতির ও Overfitting-প্রবণ, Random Forest তুলনামূলক দ্রুত ও স্থিতিশীল।

প্রশ্ন: learning_rate ছোট রাখলে কেন n_estimators বাড়াতে হয়?

উত্তর: learning_rate ছোট মানে প্রতিটা গাছের অবদান সামান্য। তাই প্রকৃত মানের কাছে পৌঁছাতে অনেক বেশি ধাপ/গাছ দরকার হয় — এটাই তাদের মধ্যকার Trade-off, যা ট্রেনিং সময় বাড়ালেও ফলাফল বেশি স্থিতিশীল করে।

Quick Revision (সংক্ষিপ্ত সারাংশ)

- **Type:** Sequential Ensemble (Boosting) — Weak Learner একটার পর একটা আগের ভুল সংশোধন করে।
- **Base Learner:** সাধারণত Shallow Decision Tree (max_depth 3–8)।
- **Core Idea:** প্রতিটা নতুন গাছ Residual/Negative Gradient শেখে, Learning Rate দিয়ে গুণ করে যোগ হয় — $F(x) = F_0(x) + \eta \cdot \sum h_m(x)$ ।
- **সুবিধা:** উচ্চ Accuracy, নমনীয় Loss Function, Feature Importance।
- **সীমাবদ্ধতা:** ধীর ট্রেনিং, Overfitting-প্রবণ, Hyperparameter-সংবেদনশীল, Outlier-সংবেদনশীল।

HistGradientBoosting

HistGradientBoosting হলো Scikit-Learn লাইব্রেরির একটি আধুনিক ও অত্যন্ত দ্রুতগতির Gradient Boosting বাস্তবায়ন। এটি মূলত Microsoft-এর তৈরি LightGBM লাইব্রেরির মূল কৌশল থেকে অনুপ্রাণিত। এর সবচেয়ে বড় লক্ষ্য হলো — ঐতিহ্যবাহী GradientBoostingClassifier/Regressor যেসব বড় ডেটাসেটে (কয়েক লক্ষ বা তার বেশি Row) অত্যন্ত ধীরগতির হয়ে পড়ে, সেসব ক্ষেত্রে উল্লেখযোগ্যভাবে দ্রুত ও কার্যকরভাবে ট্রেনিং সম্পন্ন করা।

উদাহরণ (Intuition) — কেন এটি এত দ্রুত?

সাধারণ Gradient Boosting-এ প্রতিটি Split খোঁজার সময় অ্যালগরিদমকে একটি Numerical ফিচারের প্রতিটি সম্ভাব্য Unique মান আলাদাভাবে পরীক্ষা করতে হয়। ধরা যাক, একটি কলামে 'Age' ফিচারের ১০ লক্ষ Unique মান আছে — তাহলে প্রতিটি Split খুঁজতে অ্যালগরিদমকে ১০ লক্ষ বার ভিন্ন ভিন্ন Threshold (যেমন Age > 23.5, Age > 23.6, Age > 23.7...) পরীক্ষা করতে হবে, যা কম্পিউটেশনালি অত্যন্ত ব্যয়বহুল ও সময়সাপেক্ষ।

HistGradientBoosting এই সমস্যার একটি চমৎকার সমাধান দেয়। কল্পনা করুন, বয়সের মতো একটি ধারাবাহিক ফিচারকে সরাসরি প্রতিটি Unique মান হিসেবে না দেখে, একে মাত্র ২৫৫টি নির্দিষ্ট 'বাক্স' বা Bin-এ ভাগ করে ফেলা হচ্ছে — যেমন 0-5 বছর একটি বাক্সে, 6-10 বছর আরেকটি বাক্সে, এবং এভাবে চলতে থাকে। এরপর অ্যালগরিদমকে আর প্রতিটি Unique মান নয়, বরং শুধুমাত্র এই সীমিত সংখ্যক Bin-এর সীমানাগুলোতে Split খুঁজলেই চলে। ফলে ১০ লক্ষ সম্ভাব্য Threshold-এর জায়গায় মাত্র ২৫৫টি সম্ভাব্য Threshold পরীক্ষা করলেই চলে, যা গণনার পরিমাণ কয়েক হাজার গুণ কমিয়ে দেয়।

মূল কার্যপ্রণালী (How it Works) — বিস্তারিত

১. Histogram-Based Binning

ট্রেনিং শুরু হওয়ার আগেই, প্রতিটি Continuous (ধারাবাহিক সংখ্যাসূচক) ফিচারকে সর্বোচ্চ ২৫৫টি Discrete Bin-এ রূপান্তর করা হয় (max_bins হাইপারপ্যারামিটার দিয়ে নিয়ন্ত্রিত হয়)। এই

রূপান্তর প্রক্রিয়াটি একবারই করা হয়, পুরো ট্রেনিং শুরু হওয়ার আগে, যাতে প্রতিটি গাছ তৈরির সময় বারবার এই কাজ পুনরাবৃত্তি করতে না হয়।

২. দ্রুত Split-Finding

প্রতিটি Split খোঁজার সময় অ্যালগরিদমকে আর প্রতিটি Unique ভ্যালু নয়, বরং মাত্র সীমিত সংখ্যক Bin-Boundary পরীক্ষা করলেই চলে। এর ফলে Time Complexity ব্যবহারিকভাবে ডেটার আকারের ওপর অনেক কম নির্ভরশীল হয়ে পড়ে, যা বড় ডেটাসেটে বিশাল গতি এনে দেয়।

৩. Histogram Subtraction Trick

একটি চমৎকার অপ্টিমাইজেশন কৌশল হলো — যদি একটি Parent Node-এর সম্পূর্ণ Histogram (প্রতিটি Bin-এ কতগুলো Sample এবং তাদের Gradient/Hessian-এর যোগফল) ইতিমধ্যে জানা থাকে, এবং একটি Child Node-এর Histogram গণনা করা হয়ে যায়, তাহলে অন্য Child-এর Histogram নতুন করে গণনা না করে সহজেই বিয়োগ করে (Parent – Known Child) বের করা যায়। এতে প্রতিটি Split-এ অর্ধেক গণনা সময় বেঁচে যায়।

৪. Native Missing Value Support

এটি স্বয়ংক্রিয়ভাবে Missing Value (NaN) হ্যান্ডেল করতে পারে — প্রতিটি Split-এর সময় অ্যালগরিদম নিজেই শেখে যে Missing Value থাকা Sample-গুলো বাম নাকি ডান Child-এ পাঠানো উচিত, যার ফলে আলাদাভাবে Imputation (যেমন Mean বা Median দিয়ে পূরণ) করার প্রয়োজন হয় না।

৫. Native Categorical Support

categorical_features প্যারামিটার ব্যবহার করে সরাসরি Categorical কলাম (যেমন 'City' বা 'Product Type') মডেলে পাস করা যায়, One-Hot Encoding-এর প্রয়োজন ছাড়াই। এতে High-Cardinality ক্যাটাগরিক্যাল ডেটার ক্ষেত্রে (অনেক ইউনিক ক্যাটাগরি থাকলে) কলাম সংখ্যা বহুগুণ বেড়ে যাওয়া (Column Explosion) এড়ানো যায়।

সাধারণ Gradient Boosting বনাম HistGradientBoosting

বিষয়	GradientBoosting (ঐতিহ্যবাহী)	HistGradientBoosting
Split খোঁজার পদ্ধতি	প্রতিটি Unique মান পরীক্ষা করে	সীমিত সংখ্যক (সর্বোচ্চ ২৫৫) Bin/Histogram ব্যবহার করে
বড় ডেটাসেটে গতি	তুলনামূলক ধীর	উল্লেখযোগ্যভাবে দ্রুত (১০০০+ Sample-এ স্পষ্টভাবে কার্যকর)
Missing Value	আগে থেকে Impute করতে হয়	স্বয়ংক্রিয়ভাবে হ্যান্ডেল করে
Categorical ফিচার	One-Hot Encoding প্রয়োজন	সরাসরি সাপোর্ট করে (categorical_features)
মেমোরি ব্যবহার	বেশি	কম (Bin-ভিত্তিক Compact Storage)
অনুপ্রেরণা	মূল Boosting গবেষণাপত্র	Microsoft LightGBM

গাণিতিক ভিত্তি — Histogram-ভিত্তিক Split Gain

অন্তর্নিহিত মূল গাণিতিক নীতি সাধারণ Gradient Boosting-এর মতোই — প্রতিটি নতুন গাছ পূর্ববর্তী মডেলের Negative Gradient (Residual) শেখে এবং Learning Rate দিয়ে গুণ করে যোগ হয়। পার্থক্যটা শুধু কম্পিউটেশনাল ব্যাকএন্ডে। প্রতিটি Bin-এর জন্য সেই Bin-এ পড়া সব Sample-এর Gradient (g) এবং Hessian (h অর্থাৎ দ্বিতীয় Derivative)-এর যোগফল জমা রাখা হয়। এরপর প্রতিটি সম্ভাব্য Bin-Split-এর জন্য Gain নির্ণয় করা হয় নিচের সূত্রে, যেখানে L এবং R যথাক্রমে বাম ও ডান Split-এর ফলে সৃষ্ট গ্রুপ:

Formula:

$$Gain = \frac{1}{2} \left[\frac{(\sum g_L)^2}{\sum h_L + \lambda} + \frac{(\sum g_R)^2}{\sum h_R + \lambda} - \frac{(\sum g)^2}{\sum h + \lambda} \right] - \gamma$$

যে Split সর্বোচ্চ Gain প্রদান করে, সেটিকেই সর্বোত্তম হিসেবে নির্বাচন করা হয়। এখানে λ (Lambda) একটি Regularization টার্ম যা অতিরিক্ত জটিল Split রোধ করে, এবং γ (Gamma) একটি ন্যূনতম Threshold যা নির্দিষ্ট করে যে একটি Split তখনই গ্রহণযোগ্য হবে যখন তার Gain এই মানের চেয়ে বেশি হবে। এই Histogram-ভিত্তিক গাণিতিক পদ্ধতি XGBoost এবং LightGBM-এর মূল কৌশলের সাথে ছবছ সামঞ্জস্যপূর্ণ, যে কারণে HistGradientBoosting-এর Performance প্রায়ই এই জনপ্রিয় লাইব্রেরিগুলোর কাছাকাছি চলে আসে।

গুরুত্বপূর্ণ হাইপারপ্যারামিটার — বিস্তারিত ব্যাখ্যা

max_iter — Default: 100

কাজ কী: মোট Boosting Iteration বা গাছের সংখ্যা নির্ধারণ করে, এটি সাধারণ GradientBoosting-এর n_estimators-এর সমতুল্য। কখন Default রাখব: বেশিরভাগ ক্ষেত্রে ১০০ একটি ভালো শুরু, তবে early_stopping='auto' চালু থাকলে অ্যালগরিদম নিজেই প্রয়োজন অনুযায়ী থেমে যাবে। কখন পরিবর্তন করব: বড় ডেটাসেটে ও ছোট learning_rate ব্যবহার করলে max_iter বাড়ানো প্রয়োজন হতে পারে।

max_bins — Default: 255

কাজ কী: প্রতিটি Numerical ফিচার সর্বোচ্চ কতগুলো Bin-এ ভাগ হবে তা নিয়ন্ত্রণ করে (সর্বোচ্চ সীমা ২৫৫, কারণ এর বেশি হলে প্রতিটি Bin-এর Index একটি সিঙ্গেল বাইটে (uint8) সংরক্ষণ করা সম্ভব হয় না, যা মেমোরি সাশ্রয়ের একটি বড় কৌশল)। কখন Default রাখব: ২৫৫ প্রায় সবসময় যথেষ্ট নিখুঁত Resolution দেয়। কখন পরিবর্তন করব: যদি ডেটাসেট অত্যন্ত ছোট হয় বা মেমোরি সীমাবদ্ধতা থাকে, তবে max_bins কমিয়ে (যেমন ৩২ বা ৬৪) মেমোরি ব্যবহার আরও কমানো যায়, যদিও এতে সামান্য Precision কমতে পারে।

max_leaf_nodes — Default: 31

কাজ কী: প্রতিটি গাছে সর্বোচ্চ কয়টি Leaf Node থাকতে পারবে তা নিয়ন্ত্রণ করে — এটিই মূলত গাছের জটিলতা নির্ধারণ করে (HistGradientBoosting max_depth-এর বদলে সাধারণত max_leaf_nodes দিয়েই গাছের আকার নিয়ন্ত্রণ করে)। কখন পরিবর্তন করব: মান বাড়ালে

(যেমন ৬৩ বা ১২৭) গাছ জটিলতর হয়ে বেশি Pattern ধরতে পারে, কিন্তু Overfitting-এর ঝুঁকিও বাড়ে।

`l2_regularization` — Default: 0

কাজ কী: প্রতিটি Leaf-এর প্রেডিকশন মানের ওপর একটি L2 Penalty আরোপ করে, যা Leaf-এর মানগুলোকে অতিরিক্ত চরম হয়ে যাওয়া থেকে বিরত রাখে এবং Overfitting কমাতে সাহায্য করে। কখন পরিবর্তন করব: মডেল Overfit করছে বলে সন্দেহ হলে এই মান বাড়িয়ে (যেমন ০.১ থেকে ১.০) দেখা যেতে পারে।

`early_stopping` — Default: 'auto'

কাজ কী: Validation ডেটার Score ক্রমাগত কয়েকটি Iteration ধরে উন্নত না হলে ট্রেনিং স্বয়ংক্রিয়ভাবে থামিয়ে দেয়, যা অপয়োজনীয় অতিরিক্ত গাছ তৈরি এবং Overfitting প্রতিরোধ করে ও সময় বাঁচায়।

Parameter	Default	সংক্ষিপ্ত কাজ
learning_rate	0.1	প্রতিটি গাছের অবদানের হার নিয়ন্ত্রণ করে (GradientBoosting-এর মতোই)।
categorical_features	None	কোন কলামগুলো Categorical হিসেবে বিবেচিত হবে তা সরাসরি নির্দিষ্ট করা যায়।
min_samples_leaf	20	প্রতিটি Leaf-এ ন্যূনতম Sample সংখ্যা — HistGradientBoosting-এ এই Default অন্যান্য Tree মডেলের চেয়ে বড় রাখা হয়েছে যাতে Overfitting এমনিতেই নিয়ন্ত্রিত থাকে।

গুরুত্বপূর্ণ অ্যাট্রিবিউট

- `n_iter_` : Early Stopping-এর কারণে প্রকৃতপক্ষে কতগুলো Iteration সম্পন্ন হয়েছে তার প্রকৃত সংখ্যা।

- `train_score_ / validation_score_` : প্রতিটি Iteration-এ Training ও Validation ডেটার Score-এর ইতিহাস, যা দিয়ে মডেলের অগ্রগতি ও Overfitting-এর মুহূর্ত পর্যবেক্ষণ করা যায়।
- `is_categorical_` : কোন ফিচারগুলো Categorical হিসেবে শনাক্ত হয়েছে তার একটি Boolean অ্যারে।

কোথায় ব্যবহার করা উচিত?

- যখন ডেটাসেট বড় (Row সংখ্যা কয়েক হাজার থেকে লক্ষাধিক পর্যন্ত) এবং সাধারণ GradientBoostingClassifier অত্যন্ত ধীর মনে হয়।
- যখন ডেটায় Missing Value বা Categorical কলাম প্রচুর এবং সেগুলো আলাদাভাবে Preprocess করার মতো সময় বা সুযোগ নেই।
- যখন XGBoost/LightGBM ইনস্টল করার সুযোগ বা প্রয়োজন নেই, কিন্তু Scikit-Learn-এর মধ্যেই তুলনীয় গতি ও Accuracy প্রয়োজন।

প্রশ্ন: HistGradientBoosting কি সবসময় সাধারণ GradientBoosting-এর চেয়ে ভালো?

উত্তর: গতির দিক থেকে বড় ডেটাসেটে HistGradientBoosting প্রায় সবসময়ই দ্রুততর, কিন্তু Accuracy-এর দিক থেকে দুটির ফলাফল সাধারণত কাছাকাছি থাকে। ছোট ডেটাসেটে (কয়েকশ বা কয়েক হাজার Row) সাধারণ GradientBoosting যথেষ্ট দ্রুত এবং ব্যবহার করাও সহজ, তাই সেক্ষেত্রে HistGradientBoosting ব্যবহারের বাড়তি সুবিধা খুব একটা পাওয়া যায় না।

Quick Revision

- Type: Gradient Boosting-এর Histogram-ভিত্তিক দ্রুত সংস্করণ, LightGBM দ্বারা অনুপ্রাণিত।
- মূল কৌশল: Continuous ফিচারকে সর্বোচ্চ ২৫৫টি Bin-এ ভাগ করে Split-Finding-কে বহুগুণ দ্রুততর করা।
- বাড়তি সুবিধা: Native Missing Value ও Categorical Feature সাপোর্ট, কোনো আলাদা Preprocessing ছাড়াই।

- সেৱা ব্যৱহাৰক্ষেত্ৰ: বড় আকাৰৰ Tabular ডেটাসেট, যেখানে সাধাৰণ GradientBoostingClassifier ধীৰগতিৰ হয় পড়ে।

Agglomerative Clustering

Agglomerative Clustering হলো একটি Unsupervised Machine Learning Algorithm এবং Hierarchical Clustering-এর সবচেয়ে জনপ্রিয় ও ব্যাপকভাবে ব্যবহৃত ধরন, যা 'Bottom-Up' বা 'নিচ থেকে ওপরে' পদ্ধতিতে কাজ করে। Supervised Learning-এর মতো এখানে কোনো লেবেল বা টার্গেট ভ্যারিয়েবল আগে থেকে দেওয়া থাকে না — অ্যালগরিদম সম্পূর্ণ স্বাধীনভাবে ডেটার মধ্যে থাকা স্বাভাবিক গ্রুপ বা Cluster খুঁজে বের করে, শুধুমাত্র পয়েন্টগুলোর মধ্যকার দূরত্ব বা সাদৃশ্য বিশ্লেষণ করে।

উদাহরণ (Intuition)

কল্পনা করুন, একটি ক্লাসরুমে ৬ জন ছাত্রছাত্রী রয়েছে এবং আপনি তাদের বসার আসন থেকে দূরত্বের ভিত্তিতে বন্ধু-দল ভাগ করতে চান। শুরুতে প্রতিজন ছাত্রকে একটি করে আলাদা দল হিসেবে ধরা হয় — অর্থাৎ ৬ জন থাকলে শুরুতে ৬টি দল। এরপর যে দুইজন ছাত্র একে অপরের সবচেয়ে কাছাকাছি বসে, তাদের একত্রিত করে একটি নতুন দল বানানো হয়, ফলে এখন দলের সংখ্যা কমে ৫টি হয়ে যায়। এই প্রক্রিয়া বারবার চলতে থাকে — প্রতিবার সবচেয়ে কাছাকাছি অবস্থিত দুটি দলকে একত্রিত করা হয় — যতক্ষণ না শেষ পর্যন্ত সবাই মিলে একটি বড় দলে পরিণত হয়। এই সম্পূর্ণ একত্রীকরণ প্রক্রিয়াটি ধাপে ধাপে লিখে রাখলে একটি গাছের মতো কাঠামো তৈরি হয়, যাকে বলা হয় Dendrogram। এই Dendrogram-এর যেকোনো একটি উচ্চতায় গাছটি 'কেটে' দিলেই আমরা কাঙ্ক্ষিত সংখ্যক Cluster পেয়ে যাই — এটিই Agglomerative (অর্থাৎ 'একত্রীকরণ') Clustering-এর মূল ধারণা।

মূল গাঠনিক প্রক্রিয়া (How it Works) — ধাপে ধাপে

ধাপ ১ — প্রাথমিক অবস্থা

প্রতিটি ডেটা পয়েন্টকে একটি করে সম্পূর্ণ স্বতন্ত্র Cluster হিসেবে ধরা হয়। n সংখ্যক Sample থাকলে শুরুতে ঠিক n সংখ্যক Cluster থাকে।

ধাপ ২ — দূরত্ব গণনা

প্রতিটি সম্ভাব্য Cluster জোড়ার মধ্যে দূরত্ব (Distance) গণনা করা হয় (সাধারণত Euclidean Distance ব্যবহার করা হয়) এবং এই দূরত্বগুলো একটি Distance Matrix-এ সংরক্ষণ করা হয়।

ধাপ ৩ — সবচেয়ে কাছের জোড়া একত্রিত করা

Distance Matrix থেকে সবচেয়ে কম দূরত্বে থাকা দুটি Cluster খুঁজে বের করে তাদের একত্রিত করে একটি নতুন, বড় Cluster তৈরি করা হয়।

ধাপ ৪ — Distance Matrix আপডেট ও পুনরাবৃত্তি

নতুন তৈরি হওয়া Cluster-এর সাথে বাকি সব Cluster-এর দূরত্ব পুনরায় গণনা করা হয় (Linkage পদ্ধতি অনুযায়ী), এবং আবার সবচেয়ে কাছের জোড়া একত্রিত করা হয়। এই প্রক্রিয়া বারবার চলতে থাকে, প্রতিবার Cluster-এর সংখ্যা একটি করে কমতে থাকে, যতক্ষণ না সবগুলো ডেটা একটি একক Cluster-এ পরিণত হয়।

ধাপ ৫ — ফলাফল: Dendrogram

পুরো একত্রীকরণ প্রক্রিয়াটি একটি গাছের মতো চিত্র বা Dendrogram-এ প্রদর্শিত হয়, যেখানে X-অক্ষ প্রতিটি ডেটা পয়েন্ট এবং Y-অক্ষ একত্রীকরণের সময়কার দূরত্ব দেখানো হয়। একটি নির্দিষ্ট উচ্চতায় (Distance Threshold) এই গাছটি অনুভূমিকভাবে কেটে দিলে, ঠিক যতগুলো উল্লম্ব রেখা কাটা হবে, ততগুলো Cluster পাওয়া যায়।

সংখ্যাসূচক উদাহরণ — সম্পূর্ণ ধাপে ধাপে প্রক্রিয়া (1D ডেটা, Single Linkage)

ধরা যাক, আমাদের কাছে একটি অক্ষ (1D) পাঁচটি ডেটা পয়েন্ট আছে: $A=2$, $B=3$, $C=9$, $D=10$, $E=25$ । আমরা Single Linkage (দুই Cluster-এর সবচেয়ে কাছের দুই পয়েন্টের দূরত্ব) ব্যবহার করে ধাপে ধাপে Agglomerative Clustering করবো।

ধাপ	সবচেয়ে কাছের জোড়া	দূরত্ব	একত্রীকরণের পর অবস্থা
১	A(2) ও B(3)	1	{A,B}, {C}, {D}, {E}
২	C(9) ও D(10)	1	{A,B}, {C,D}, {E}
৩	{A,B} ও {C,D}	6 (B থেকে C-এর দূরত্ব)	{A,B,C,D}, {E}
৪	{A,B,C,D} ও {E}	15 (D থেকে E-এর দূরত্ব)	{A,B,C,D,E}

লক্ষ করুন, প্রতিটি ধাপে দূরত্ব ক্রমশ বাড়ছে ($1 \rightarrow 1 \rightarrow 6 \rightarrow 15$), যা স্বাভাবিক — কারণ প্রথমে সবচেয়ে কাছাকাছি পয়েন্টগুলো একত্রিত হয়, শেষে সবচেয়ে দূরের গ্রুপগুলো একত্রিত হয়। যদি আমরা Dendrogram-কে Distance = 6-এর ঠিক ওপরে কাটি (অর্থাৎ ৬-এর বেশি দূরত্বে আর কোনো একত্রীকরণ অনুমোদন না করি), তাহলে আমরা পাবো দুইটি Cluster: {A, B, C, D} এবং {E} — যা স্বজ্ঞাতভাবেও সঠিক, কারণ E (25) বাকি চারটি পয়েন্ট থেকে অনেক দূরে অবস্থিত।

Linkage Criteria — কীভাবে দুই Cluster-এর দূরত্ব মাপা হয়?

যখন একটি Cluster-এ একাধিক পয়েন্ট থাকে (যেমন উপরের {A,B} Cluster), তখন প্রশ্ন ওঠে — এই Cluster-এর সাথে আরেকটি Cluster-এর দূরত্ব ঠিক কীভাবে মাপা হবে? এর জন্য একাধিক পদ্ধতি রয়েছে, যাকে বলা হয় Linkage।

Linkage	সংজ্ঞা	বৈশিষ্ট্য
Single	দুই Cluster-এর সবচেয়ে কাছের দুই পয়েন্টের দূরত্ব	লম্বা, শিকল-আকৃতির (Chain-like) Cluster তৈরি করে; Noise বা Outlier-এ সংবেদনশীল
Complete	দুই Cluster-এর সবচেয়ে দূরের দুই পয়েন্টের দূরত্ব	কমপ্যাক্ট ও প্রায় সমান আকারের Cluster তৈরি করে

Linkage	সংজ্ঞা	বৈশিষ্ট্য
Average	দুই Cluster-এর মধ্যে সব সম্ভাব্য জোড়া পয়েন্টের দূরত্বের গড়	Single ও Complete-এর মাঝামাঝি একটি ভারসাম্যপূর্ণ ফলাফল দেয়
Ward	একত্রিত করলে Cluster-এর ভেতরের Variance কতটা বাড়বে তার পরিমাপ (Minimum Variance পদ্ধতি)	Scikit-Learn-এর ডিফল্ট; সাধারণত সবচেয়ে সুসম ও গোলাকার Cluster দেয়

গাণিতিক ভিত্তি — Ward's Method

Ward Linkage-এ দুটি Cluster A এবং B একত্রিত করলে Cluster-এর ভেতরের Variance বা 'বিশৃঙ্খলা' ঠিক কতটুকু বৃদ্ধি পাবে, তা নিচের সূত্রে গণনা করা হয়। এখানে n_A , n_B হলো প্রতিটি Cluster-এর সদস্য সংখ্যা এবং c_A , c_B হলো তাদের Centroid (কেন্দ্রবিন্দু):

Formula:

$$\Delta(A, B) = \frac{n_A n_B}{n_A + n_B} \|c_A - c_B\|^2$$

প্রতিটি ধাপে অ্যালগরিদম এমন একটি জোড়া বেছে নেয়, যাদের একত্রিত করলে এই Δ মান (অর্থাৎ, Cluster-ভেতরের Variance বৃদ্ধি) সবচেয়ে কম হয়। এই কারণেই Ward Linkage K-Means-এর মতোই তুলনামূলক গোলাকার ও ভারসাম্যপূর্ণ আকারের Cluster তৈরি করে, এবং এই বৈশিষ্ট্যের কারণেই এটি Scikit-Learn-এ ডিফল্ট Linkage হিসেবে নির্ধারিত হয়েছে।

K-Means বনাম Agglomerative Clustering

বিষয়	K-Means	Agglomerative Clustering
পদ্ধতি	Centroid-ভিত্তিক, Iterative (বারবার Centroid সরিয়ে অপটিমাইজ করা)	Hierarchical, Bottom-Up একত্রীকরণ

বিষয়	K-Means	Agglomerative Clustering
Cluster সংখ্যা আগে থেকে জানা লাগে?	হ্যাঁ, k নির্দিষ্ট করে দিতেই হয়	না, Dendrogram তৈরির পর প্রয়োজন অনুযায়ী সিদ্ধান্ত নেওয়া যায়
Cluster-এর আকার	সাধারণত গোলাকার (Convex)	Linkage অনুযায়ী যেকোনো আকৃতি হতে পারে (বিশেষত Single Linkage-এ)
বড় ডেটাসেটে গতি	দ্রুত, সহজেই Scalable	ধীর ($O(n^2)$ থেকে $O(n^3)$ Time Complexity)
ফলাফলের ব্যাখ্যা	শুধু চূড়ান্ত Cluster লেবেল পাওয়া যায়	Dendrogram-এর মাধ্যমে সম্পূর্ণ শ্রেণিবিন্যাস কাঠামো দেখা যায়
Randomness	Initial Centroid র্যান্ডম, ফলে প্রতিবার সামান্য ভিন্ন ফলাফল হতে পারে	সম্পূর্ণ Deterministic — একই ডেটায় সবসময় একই ফলাফল

গুরুত্বপূর্ণ হাইপারপ্যারামিটার — বিস্তারিত ব্যাখ্যা

n_clusters — Default: 2

কাজ কী: চূড়ান্তভাবে কতগুলো Cluster রাখা হবে তা নির্ধারণ করে। কখন Default রাখব: যদি আপনি আগে থেকেই জানেন যে ডেটায় ঠিক কতগুলো প্রাকৃতিক গ্রুপ থাকতে পারে (যেমন কাস্টমার সেগমেন্টেশনে ৩টি সেগমেন্ট চান), তাহলে সরাসরি সেই সংখ্যা বসিয়ে দিন। কখন পরিবর্তন করব: যদি সঠিক সংখ্যা না জানা থাকে, তাহলে n_clusters=None রেখে তার বদলে distance_threshold ব্যবহার করা ভালো — এতে Dendrogram থেকে স্বয়ংক্রিয়ভাবে উপযুক্ত সংখ্যক Cluster নির্ধারিত হয়।

linkage — Default: 'ward'

কাজ কী: দুই Cluster-এর মধ্যে দূরত্ব মাপার পদ্ধতি নির্ধারণ করে ('ward', 'complete', 'average', 'single')। কখন Default রাখব: বেশিরভাগ সাধারণ সমস্যায় 'ward' সবচেয়ে সুষম ও

ব্যবহারযোগ্য ফলাফল দেয়। কখন পরিবর্তন করব: যদি আপনার ডেটায় Cluster-গুলোর আকৃতি অস্বাভাবিক বা প্রসারিত (Non-convex) হওয়ার সম্ভাবনা থাকে (যেমন চাঁদের আকৃতির ডেটা), তবে 'single' Linkage ব্যবহার করে দেখা যেতে পারে, যদিও এটি Outlier-এর প্রতি অনেক বেশি সংবেদনশীল।

distance_threshold — Default: None

কাজ কী: নির্দিষ্ট করা হলে, এই দূরত্বের বেশি হলে আর কোনো একত্রীকরণ ঘটবে না — ফলে n_clusters স্বয়ংক্রিয়ভাবে নির্ধারিত হয়ে যায়। কখন ব্যবহার করব: যখন Cluster-এর সঠিক সংখ্যা আগে থেকে জানা নেই, তখন এটি n_clusters=None-এর সাথে একত্রে ব্যবহার করে Dendrogram বিশ্লেষণের মাধ্যমে স্বাভাবিক Cluster সংখ্যা খুঁজে বের করা যায়।

Parameter	Default	সংক্ষিপ্ত কাজ
metric	'euclidean'	দূরত্ব মাপার মূল Distance Metric (মনে রাখতে হবে, 'ward' শুধুমাত্র 'euclidean'-এর সাথেই কাজ করে)।
compute_full_tree	'auto'	সম্পূর্ণ Dendrogram তৈরি করা হবে কি না তা নিয়ন্ত্রণ করে, n_clusters ছোট হলে এটি গণনা সময় বাঁচাতে পারে।
compute_distances	False	True করলে প্রতিটি একত্রীকরণের দূরত্ব সংরক্ষণ করা হয় (distances_ অ্যাট্রিবিউটে), যা Dendrogram Plot করতে প্রয়োজন হয়।

গুরুত্বপূর্ণ অ্যাট্রিবিউট

- labels_ : প্রতিটি ডেটা পয়েন্ট চূড়ান্তভাবে কোন Cluster-এ পড়েছে তার একটি অ্যারে।
- n_clusters_ : প্রকৃতপক্ষে যতগুলো Cluster তৈরি হয়েছে তার সংখ্যা।
- children_ : কোন কোন Cluster ধাপে ধাপে একত্রিত হয়েছিল তার সম্পূর্ণ ইতিহাস — এই তথ্য দিয়েই Dendrogram আঁকা হয়।

- `distances_` : প্রতিটি একত্রীকরণের সময়কার দূরত্ব বা Linkage মান (`compute_distances=True` থাকলে)।

কোন কোন সেক্টরে ব্যবহার হয়?

- জীববিজ্ঞান ও বায়োইনফরমেটিক্স: জিন এক্সপ্রেশন ডেটা বিশ্লেষণ করে জীবের বিবর্তনীয় সম্পর্ক (Phylogenetic Tree) নির্ণয় করা — এখানে Dendrogram-এর প্রাকৃতিক শ্রেণিবিন্যাস কাঠামো অত্যন্ত মূল্যবান।
- মার্কেটিং: গ্রাহকদের ক্রয় আচরণের ভিত্তিতে ধাপে ধাপে Segment তৈরি করা, যেখানে ব্যবসায়িক প্রয়োজন অনুযায়ী পরে সিদ্ধান্ত নেওয়া যায় কয়টি Segment ব্যবহার করা হবে।
- নথি ও পাঠ্য বিশ্লেষণ (Document Clustering): সংবাদ নিবন্ধ বা গবেষণাপত্রকে বিষয়ভিত্তিক Hierarchical গ্রুপে ভাগ করা।
- ইমেজ প্রসেসিং: একই রকম রঙ বা বৈশিষ্ট্যের ছবিগুলোকে ধাপে ধাপে গ্রুপবদ্ধ করা।

প্রশ্ন: Dendrogram থেকে সঠিক Cluster সংখ্যা কীভাবে নির্ধারণ করব?

উত্তর: Dendrogram-এ সবচেয়ে লম্বা উল্লম্ব রেখা (যেটি কোনো অনুভূমিক রেখা দ্বারা বাধাপ্রাপ্ত হয়নি) খুঁজে বের করে সেই জায়গায় গাছটি কাটা হয়। একটি লম্বা উল্লম্ব রেখার অর্থ হলো, দুটি Cluster একত্রিত করতে অনেক বেশি দূরত্ব অতিক্রম করতে হয়েছে — অর্থাৎ তারা স্বাভাবিকভাবেই একে অপরের থেকে অনেকটা আলাদা। এই সবচেয়ে লম্বা 'ফাঁকা' জায়গায় গাছ কাটলেই সাধারণত সবচেয়ে অর্থবহ (Meaningful) সংখ্যক Cluster পাওয়া যায়।

Quick Revision

- Type: Unsupervised, Hierarchical (Bottom-Up) Clustering Algorithm।
- প্রক্রিয়া: প্রতিটি পয়েন্ট নিজে একটি Cluster থেকে শুরু করে ধাপে ধাপে সবচেয়ে কাছের জোড়া একত্রিত করে একটি বড় গাছ (Dendrogram) তৈরি করা।
- Linkage: Ward (Default), Complete, Average, Single — প্রতিটি ভিন্ন আকৃতির Cluster তৈরি করে।

- সুবিধা: Cluster সংখ্যা আগে থেকে জানার প্রয়োজন নেই, Dendrogram-এর মাধ্যমে সম্পূর্ণ কাঠামো দেখা যায়, সম্পূর্ণ Deterministic।
- সীমাবদ্ধতা: বড় ডেটাসেটে ধীরগতি ($O(n^2)$ থেকে $O(n^3)$ Complexity), একবার একত্রিত হলে তা আর বিভক্ত করা যায় না (Greedy ও Irreversible)।

t-SNE (t-Distributed Stochastic Neighbor Embedding)

t-SNE হলো একটি Unsupervised, Non-linear Dimensionality Reduction অ্যালগরিদম, যা মূলত High-Dimensional ডেটাকে (যেমন ৫০ বা ১০০টি ফিচারবিশিষ্ট ডেটা) মাত্র ২ বা ৩ মাত্রায় (2D/3D) নামিয়ে এনে দৃশ্যমান বা Visualize করার জন্য ব্যবহৃত হয়। PCA-এর মতো এটি একটি সরলরৈখিক (Linear) পদ্ধতি নয় — বরং এটি ডেটার মধ্যে থাকা স্থানীয় (Local) কাঠামো, অর্থাৎ কোন পয়েন্টগুলো একে অপরের 'প্রতিবেশী', সেই সম্পর্কটুকু যতটা সম্ভব অক্ষুণ্ণ রেখে নতুন নিম্ন-মাত্রিক স্পেসে স্থানান্তর করার চেষ্টা করে।

উদাহরণ (Intuition)

কল্পনা করুন, আপনার কাছে হাতে লেখা সংখ্যার (MNIST Digit) হাজার হাজার ছবি আছে, যেখানে প্রতিটি ছবি ৭৮৪টি Pixel-ভ্যালু বা ৭৮৪ মাত্রার একটি ডেটা পয়েন্ট হিসেবে প্রতিনিধিত্ব করে। মানুষের চোখে ৭৮৪ মাত্রার কোনো প্যাটার্ন সরাসরি দেখা সম্পূর্ণ অসম্ভব। t-SNE এই বিশাল মাত্রার ডেটাকে এমনভাবে মাত্র ২ মাত্রায় নামিয়ে আনে যে, যেসব সংখ্যা দেখতে একই রকম (যেমন সব '৭'-এর ছবি) সেগুলো নতুন 2D স্পেসেও একে অপরের কাছাকাছি একটি গুচ্ছে জড়ো হয়, এবং সম্পূর্ণভিন্ন সংখ্যাগুলো (যেমন '৭' বনাম '৩') একে অপরের থেকে দূরে অবস্থান করে। ফলাফল হিসেবে একটি সাধারণ 2D স্ক্যাটার প্লটেই বিভিন্ন সংখ্যার প্রাকৃতিক ও স্বতন্ত্র ক্লাস্টার সরাসরি চোখে দেখা সম্ভব হয়ে ওঠে, যা মডেলের আচরণ বোঝার জন্য অত্যন্ত সহায়ক।

মূল কার্যপ্রণালী (How it Works) — ধাপে ধাপে

ধাপ ১ — High-Dimensional স্পেসে সাদৃশ্য নির্ণয়

প্রথমে High-Dimensional স্পেসের প্রতিটি পয়েন্ট জোড়ার মধ্যে দূরত্বকে একটি Gaussian (Normal) সম্ভাবনা বিতরণে রূপান্তর করা হয়। এর অর্থ হলো — যদি দুটি পয়েন্ট কাছাকাছি থাকে, তাদের একে অপরকে 'প্রতিবেশী' হিসেবে বেছে নেওয়ার সম্ভাবনা বেশি ধরা হয়; দূরে থাকলে সেই সম্ভাবনা কম ধরা হয়।

ধাপ ২ — Low-Dimensional স্পেসে র্যান্ডম প্রারম্ভ

লক্ষ্যবস্তু 2D বা 3D স্পেসে প্রতিটি পয়েন্টকে প্রাথমিকভাবে সম্পূর্ণ র্যান্ডমভাবে (অথবা PCA ব্যবহার করে আরও স্মার্টভাবে) স্থাপন করা হয়।

ধাপ ৩ — Low-Dimensional সাদৃশ্য নির্ণয়

এই নতুন Low-Dimensional স্পেসে পয়েন্টগুলোর মধ্যে সাদৃশ্য গণনা করা হয়, তবে এবার Gaussian Distribution-এর বদলে একটি Student's t-Distribution ব্যবহার করা হয় — এই কারণেই অ্যালগরিদমের নামে 't' অক্ষরটি যুক্ত হয়েছে।

ধাপ ৪ — দুই বিতরণের পার্থক্য কমানো

High-Dimensional এবং Low-Dimensional স্পেসের সাদৃশ্য বিতরণ দুটির মধ্যে পার্থক্য একটি বিশেষ পরিমাপ (KL Divergence) দিয়ে গণনা করা হয়। এরপর Gradient Descent ব্যবহার করে ধীরে ধীরে Low-Dimensional পয়েন্টগুলোর অবস্থান এমনভাবে পরিবর্তন করা হয়, যাতে এই পার্থক্যটুকু ক্রমাগত কমতে থাকে।

ধাপ ৫ — Iteration

এই সম্পূর্ণ প্রক্রিয়া বহুবার (সাধারণত কয়েকশ Iteration, $n_iter=1000$ বা তার বেশি) পুনরাবৃত্তি হয়, যতক্ষণ না KL Divergence স্থিতিশীল হয়ে যায় এবং একটি স্থায়ী ও অর্থবহ Layout তৈরি হয়ে যায়।

কেন t-Distribution ব্যবহার করা হয়? — Crowding Problem

একটি স্বাভাবিক প্রশ্ন হলো, Low-Dimensional স্পেসে Gaussian Distribution ব্যবহার না করে t-Distribution কেন ব্যবহার করা হয়? এর কারণ হলো একটি সমস্যা, যাকে বলা হয় Crowding Problem। High-Dimensional স্পেসে একটি পয়েন্টের অনেক 'মাঝারি দূরত্বের' প্রতিবেশী থাকতে পারে, কিন্তু যখন এই সম্পর্কগুলোকে জোর করে মাত্র ২ মাত্রায় গুঁজে দেওয়া হয়, তখন এই মাঝারি দূরত্বের পয়েন্টগুলো একে অপরের ওপর ভিড় করে ফেলার প্রবণতা তৈরি হয়। t-Distribution-এর একটি বিশেষ বৈশিষ্ট্য হলো এর 'ভারী লেজ' (Heavy Tail) — অর্থাৎ দূরের পয়েন্টগুলোর জন্য এটি Gaussian-এর তুলনায় বেশি সম্ভাবনা দেয়। এর ফলে t-SNE স্বাভাবিকভাবেই মাঝারি-দূরত্বের পয়েন্টগুলোকে একে অপরের থেকে আরও বেশি দূরত্বে ঠেলে

দেয়, যা এই Crowding সমস্যা সমাধান করে এবং একটি পরিষ্কার, সুস্পষ্ট ক্লাস্টার Visualization তৈরি করে।

গাণিতিক ভিত্তি – বিস্তারিত

High-Dimensional স্পেসে দুটি পয়েন্ট x_i ও x_j -এর মধ্যে শর্তাধীন সাদৃশ্য (Conditional Similarity) নিচের সূত্রে গণনা করা হয়, যেখানে σ_i হলো পয়েন্ট x_i -এর চারপাশের Gaussian-এর প্রস্থ (এটি perplexity প্যারামিটার দ্বারা নিয়ন্ত্রিত হয়):

Formula:

$$p_{j|i} = \frac{\exp\left(-\frac{\|x_i - x_j\|^2}{2\sigma_i^2}\right)}{\sum_{k \neq i} \exp\left(-\frac{\|x_i - x_k\|^2}{2\sigma_i^2}\right)}$$

এরপর একটি Symmetric সাদৃশ্য p_{ij} তৈরি করা হয় $p_{j|i}$ এবং $p_{i|j}$ -এর গড় নিয়ে, যা পুরো High-Dimensional সাদৃশ্য বিতরণ P তৈরি করে।

Low-Dimensional স্পেসে (y_i, y_j) পয়েন্টগুলোর মধ্যে সাদৃশ্য t-Distribution দিয়ে গণনা করা হয়:

Formula:

$$q_{ij} = \frac{(1 + \|y_i - y_j\|^2)^{-1}}{\sum_{k \neq i} (1 + \|y_k - y_l\|^2)^{-1}}$$

চূড়ান্ত লক্ষ্য থাকে দুই বিতরণ P (High-Dimensional) এবং Q (Low-Dimensional)-এর মধ্যকার Kullback-Leibler (KL) Divergence সর্বনিম্ন করা, যা মূলত দুই বিতরণ কতটা 'ভিন্ন' তার একটি পরিমাপ:

Formula:

$$KL(P \parallel Q) = \sum_i \sum_j p_{ij} \log \left(\frac{p_{ij}}{q_{ij}} \right)$$

Gradient Descent ব্যবহার করে প্রতিটি y_i পয়েন্টের অবস্থান এমনভাবে আপডেট করা হয়, যাতে এই KL Divergence প্রতিটি ধাপে কমতে থাকে — এভাবেই ধীরে ধীরে একটি স্থিতিশীল, অর্থবহ 2D Layout তৈরি হয়ে যায়।

Perplexity — সবচেয়ে গুরুত্বপূর্ণ প্যারামিটার

Perplexity হলো t-SNE-এর সবচেয়ে গুরুত্বপূর্ণ এবং সবচেয়ে বেশি প্রভাবশালী হাইপারপ্যারামিটার। একে সহজ ভাষায় বলা যায় — প্রতিটি পয়েন্টের 'কার্যকর প্রতিবেশী সংখ্যা' (Effective Number of Neighbors)-এর একটি আনুমানিক অনুমান। এটি σ_i (প্রতিটি পয়েন্টের চারপাশের Gaussian-এর প্রস্থ)-এর মান নির্ধারণে ব্যবহৃত হয়।

- Perplexity ছোট রাখলে (যেমন ৫): t-SNE খুবই স্থানীয় (Local) কাঠামোর ওপর মনোযোগ দেয়, ফলে অনেকগুলো ছোট ছোট, বিচ্ছিন্ন ক্লাস্টার তৈরি হতে পারে, যার কিছু হয়তো Noise থেকে তৈরি হওয়া কৃত্রিম গঠন।
- Perplexity বড় রাখলে (যেমন ৫০): t-SNE বৃহত্তর কাঠামো বিবেচনা করে, ফলে বড় ও মসৃণ ক্লাস্টার তৈরি হয়, কিন্তু কখনো কখনো সূক্ষ্ম স্থানীয় পার্থক্য হারিয়ে যেতে পারে।
- সাধারণ নির্দেশিকা: Perplexity সাধারণত ৫ থেকে ৫০-এর মধ্যে রাখা হয় এবং এটি সবসময় ডেটাসেটের মোট Sample সংখ্যার চেয়ে ছোট হতে হবে।

PCA বনাম t-SNE

বিষয়	PCA	t-SNE
পদ্ধতির ধরন	Linear Dimensionality Reduction	Non-linear Dimensionality Reduction
লক্ষ্য	সর্বোচ্চ Variance (তথ্য) ধরে রাখা	স্থানীয় (Local) প্রতিবেশী সম্পর্ক অক্ষুণ্ণ রাখা
গতি	দ্রুত, বড় ডেটাসেটে সহজেই Scalable	তুলনামূলক ধীর, Computationally ব্যয়বহুল
Deterministic?	হ্যাঁ, একই ডেটায় সবসময় একই ফলাফল	না, random_state পরিবর্তনে ভিন্ন Layout তৈরি হতে পারে
নতুন ডেটা যোগ করা	সহজ (transform() মেথড রয়েছে)	কঠিন — নতুন ডেটার জন্য সাধারণত সম্পূর্ণ পুনরায় Fit করতে হয়
মূল ব্যবহার	Feature Reduction, Noise কমানো, Preprocessing Step হিসেবে	শুধুমাত্র Visualization / Exploratory Data Analysis

গুরুত্বপূর্ণ হাইপারপ্যারামিটার (TSNE) — বিস্তারিত

Parameter	Default	কাজ
n_components	2	চূড়ান্তভাবে কত মাত্রায় (Dimension) ডেটা নামানো হবে — সাধারণত Visualization-এর জন্য 2 বা 3।
perplexity	30	প্রতিটি পয়েন্টের কার্যকর প্রতিবেশী সংখ্যা আনুমানিক নির্দেশ করে। ছোট ডেটাসেটে কম (5–15), বড় ডেটাসেটে বেশি (30–50) মান ব্যবহার হয়।
learning_rate	'auto' (সাধারণত 10–1000)	Gradient Descent-এর প্রতিটি ধাপে পয়েন্টগুলো কতটা সরবে তা নিয়ন্ত্রণ করে; খুব ছোট হলে সবাই

Parameter	Default	কাজ
		একটি বিন্দুতে জমা হয়ে যেতে পারে, খুব বড় হলে পয়েন্টগুলো এলোমেলোভাবে ছড়িয়ে যেতে পারে।
n_iter	1000	মোট Optimization Iteration সংখ্যা — কম হলে Layout স্থিতিশীল হওয়ার আগেই থেমে যেতে পারে, যা অসম্পূর্ণ বা বিভ্রান্তিকর ফলাফল দিতে পারে।
metric	'euclidean'	High-Dimensional স্পেসে দূরত্ব মাপার পদ্ধতি।
init	'pca'	পয়েন্টগুলোর প্রাথমিক অবস্থান কীভাবে নির্ধারিত হবে — 'random' বা 'pca'; PCA-ভিত্তিক Init সাধারণত বেশি স্থিতিশীল ও পুনরুৎপাদনযোগ্য ফলাফল দেয়।

গুরুত্বপূর্ণ অ্যাট্রিবিউট

- embedding_ : চূড়ান্তভাবে গণনা করা Low-Dimensional (সাধারণত 2D) কো-অর্ডিনেট, যা সরাসরি স্ক্যাটার Plot করা যায়।
- kl_divergence_ : Optimization শেষে High ও Low-Dimensional বিতরণের মধ্যে চূড়ান্ত KL Divergence মান — এই মান যত কম হবে, Embedding তত বেশি নির্ভরযোগ্য বলে বিবেচিত হয়।
- n_iter_ : প্রকৃতপক্ষে কতগুলো Iteration সম্পন্ন হয়েছে।

গুরুত্বপূর্ণ সতর্কতা — ভুল ব্যাখ্যা এড়াতে

- t-SNE Plot-এ দুটি Cluster-এর মধ্যকার দূরত্ব বা প্রতিটি Cluster-এর আকার নিয়ে সরাসরি কোনো সিদ্ধান্তে আসা উচিত নয় — এটি শুধুমাত্র দেখায় কোন পয়েন্টগুলো একে অপরের 'প্রতিবেশী', প্রকৃত দূরত্বের অনুপাত বা সামগ্রিক কাঠামো (Global Structure) এটি নির্ভরযোগ্যভাবে সংরক্ষণ করে না।

- Perplexity পরিবর্তন করলে সম্পূর্ণ ভিন্ন Layout তৈরি হতে পারে, তাই একটি একক Plot দেখে চূড়ান্ত সিদ্ধান্তে না গিয়ে একাধিক Perplexity মান দিয়ে পরীক্ষা করে দেখা একটি ভালো অভ্যাস।
- ফিচার সংখ্যা অত্যন্ত বেশি হলে (যেমন কয়েকশ), প্রথমে PCA ব্যবহার করে ৫০ মাত্রায় নামিয়ে, তারপর সেই ফলাফলের ওপর t-SNE প্রয়োগ করা একটি প্রচলিত ও কার্যকর Best Practice — এতে Noise কমে এবং t-SNE-এর গণনার সময়ও কমে যায়।

কোথায় ব্যবহার হয়?

- জিনোমিক্স ও বায়োইনফরমেটিক্স: Single-Cell RNA Sequencing ডেটায় হাজার হাজার কোষের মধ্যে বিভিন্ন প্রকারভেদ (Cell Type) ভিজ্যুয়ালাইজ করা।
- কম্পিউটার ভিশন: Neural Network-এর শেষ স্তরের Embedding ভিজ্যুয়ালাইজ করে যাচাই করা যে মডেল বিভিন্ন Class-কে ঠিকভাবে আলাদা করতে শিখেছে কি না।
- Natural Language Processing: Word Embedding (Word2Vec, BERT) ভিজ্যুয়ালাইজ করে শব্দগুলোর মধ্যকার অর্থগত সাদৃশ্য চোখে দেখার সুযোগ তৈরি করা।

প্রশ্ন: t-SNE কি Clustering Algorithm?

উত্তর: না, t-SNE নিজে কোনো Clustering Algorithm নয় — এটি একটি Dimensionality Reduction ও Visualization টুল, যা ডেটাকে মানুষের চোখে দেখার উপযোগী করে তোলে। এতে চোখে দেখে ক্লাস্টার শনাক্ত করা গেলেও, প্রকৃত Cluster লেবেল পেতে হলে t-SNE-এর আউটপুটের ওপর আলাদাভাবে K-Means বা DBSCAN-এর মতো একটি প্রকৃত Clustering Algorithm প্রয়োগ করতে হয়, অথবা visually inspection-এর মাধ্যমে মানুষ নিজে সিদ্ধান্ত নেয়।

Quick Revision

- Type: Unsupervised, Non-linear Dimensionality Reduction — মূলত Visualization-এর জন্য ব্যবহৃত।
- মূল কৌশল: High-Dimensional স্পেসের প্রতিবেশী সম্পর্ক (Gaussian Similarity) Low-Dimensional স্পেসে (t-Distribution Similarity) যতটা সম্ভব বজায় রাখা, KL Divergence কমিয়ে।

- গুরুত্বপূর্ণ প্যারামিটার: perplexity — কার্যকর প্রতিবেশী সংখ্যা নিয়ন্ত্রণ করে, সাধারণত 5-50।
- সীমাবদ্ধতা: ধীরগতি, Deterministic নয়, নতুন ডেটার জন্য পুনরায় Fit প্রয়োজন, Global দূরত্ব নির্ভরযোগ্য নয়।

Isolation Forest

Isolation Forest হলো একটি Unsupervised, Tree-ভিত্তিক Anomaly বা Outlier Detection Algorithm। অন্যান্য বেশিরভাগ Outlier Detection পদ্ধতি প্রথমে ডেটার মধ্যে 'স্বাভাবিক' বা Normal অংশের একটি বিস্তারিত প্রোফাইল তৈরি করে (যেমন Density বা Distance পরিমাপ করে), তারপর যা সেই প্রোফাইলের সাথে না মেলে তাকে Outlier হিসেবে চিহ্নিত করে। Isolation Forest সম্পূর্ণ ভিন্ন এবং অভিনব একটি দর্শনে কাজ করে — এটি সরাসরি ধরে নেয় যে, Anomaly বা ব্যতিক্রমী পয়েন্টগুলো সংখ্যায় কম (Few) এবং অন্যান্য স্বাভাবিক ডেটা থেকে ভিন্ন (Different), তাই তাদের 'বিচ্ছিন্ন' (Isolate) করে ফেলা সাধারণ পয়েন্টের তুলনায় অনেক সহজ ও দ্রুত।

উদাহরণ (Intuition)

কল্পনা করুন, একটি ঘরে ১০০ জন সাধারণ উচ্চতার (৫ থেকে ৫.৮ ফুট) মানুষ এবং মাত্র ১ জন অস্বাভাবিক লম্বা (৭ ফুট) মানুষ আছেন। যদি আপনি এলোমেলোভাবে প্রশ্ন করে (যেমন: 'উচ্চতা কি ৬ ফুটের বেশি?') মানুষদের ভাগ করতে থাকেন, তাহলে ওই অস্বাভাবিক লম্বা মানুষটি মাত্র ১টি প্রশ্নেই বাকি সবার থেকে আলাদা হয়ে যাবে, কারণ সে সংখ্যাগরিষ্ঠ থেকে অনেক দূরে অবস্থান করছে এবং তাকে আলাদা করার মতো র‍্যান্ডম প্রশ্নের সংখ্যা অনেক বেশি। কিন্তু সাধারণ উচ্চতার মানুষদের একে অপরের থেকে সম্পূর্ণ আলাদা করতে আরও অনেক বেশি প্রশ্নের প্রয়োজন হবে, কারণ তারা একে অপরের এত কাছাকাছি যে তাদের আলাদা করতে বহু ধাপে ভাগ করতে হয়। Isolation Forest ঠিক এই নীতিতেই কাজ করে — যে পয়েন্ট যত কম প্রশ্নে (Split) আলাদা হয়ে যায়, তাকে তত বেশি Anomalous ধরা হয়।

মূল কার্যপ্রণালী (How it Works) — ধাপে ধাপে

১. Random Partitioning

বহুসংখ্যক Isolation Tree (iTree) তৈরি করা হয়। প্রতিটি গাছ তৈরির সময়, প্রতিটি Node-এ একটি র‍্যান্ডম ফিচার বেছে নেওয়া হয় এবং সেই ফিচারের Minimum ও Maximum মানের মধ্যে একটি সম্পূর্ণ র‍্যান্ডম Split Value বেছে নিয়ে ডেটাকে দুই ভাগে ভাগ করা হয়। এখানে লক্ষণীয়

যে, সাধারণ Decision Tree-এর মতো Gini Impurity বা Entropy গণনা করে 'সেরা' Split খোঁজা হয় না — সম্পূর্ণভাবে Random-ভাবে ভাগ করা হয়, যা এই অ্যালগরিদমকে অত্যন্ত দ্রুতগতির করে তোলে।

২. পুনরাবৃত্তিমূলক বিভাজন

এই র্যান্ডম Split প্রতিটি নতুন Sub-node-এ আবার পুনরাবৃত্তি হতে থাকে, যতক্ষণ না প্রতিটি ডেটা পয়েন্ট সম্পূর্ণভাবে একটি একক Leaf-এ বিচ্ছিন্ন (Isolated) হয়ে যায়, অথবা গাছটি একটি পূর্বনির্ধারিত সর্বোচ্চ গভীরতায় পৌঁছে যায়।

৩. Path Length পরিমাপ

প্রতিটি ডেটা পয়েন্টের জন্য গণনা করা হয়, Root Node থেকে শুরু করে সেই পয়েন্ট সম্পূর্ণভাবে বিচ্ছিন্ন (Isolate) হতে ঠিক কতগুলো Split বা কতটুকু Path Length প্রয়োজন হলো — একে বলা হয় $h(x)$ ।

৪. Ensemble Averaging

একটি মাত্র গাছের ফলাফলের ওপর নির্ভর করা নির্ভরযোগ্য নয়, কারণ একটি নির্দিষ্ট Random Split-এর কারণে ফলাফল পরিবর্তিত হতে পারে। তাই এই সম্পূর্ণ প্রক্রিয়া বহুসংখ্যক গাছ ($n_estimators$, সাধারণত ১০০টি) জুড়ে পুনরাবৃত্তি হয় এবং প্রতিটি পয়েন্টের সবগুলো গাছ জুড়ে গড় Path Length $E(h(x))$ বের করা হয়।

৫. Anomaly Score নির্ধারণ

যেসব পয়েন্টের গড় Path Length তুলনামূলকভাবে ছোট (অর্থাৎ তারা দ্রুত বিচ্ছিন্ন হয়ে যায়), তাদের Anomaly হিসেবে চিহ্নিত করা হয়। বিপরীতভাবে, যেসব পয়েন্টের Path Length বড় (বিচ্ছিন্ন হতে অনেক বেশি Split প্রয়োজন হয়), তাদের স্বাভাবিক বা Normal বলে গণ্য করা হয়, কারণ তারা অন্যান্য অনেক পয়েন্টের সাথে মিলেমিশে থাকে।

সংখ্যাসূচক উদাহরণ — একটি ছোট Isolation Tree তৈরি হওয়ার প্রক্রিয়া

ধরা যাক, আমাদের কাছে একটি 1D ফিচার (Transaction Amount) এর ছয়টি মান আছে: 10, 12, 11, 13, 9, 500। এখানে 500 একটি স্পষ্ট Anomaly (হয়তো একটি জালিয়াতি লেনদেন), বাকি পাঁচটি মান স্বাভাবিক লেনদেনের পরিসরে (৯ থেকে ১৩) রয়েছে।

Split ধাপ	র‍্যান্ডম Threshold	ফলাফল
Split ১ (Root)	Amount > 250	বামে: {10,12,11,13,9} (৫টি পয়েন্ট) — ডানে: {500} (মাত্র ১টি পয়েন্ট, তাৎক্ষণিকভাবে বিচ্ছিন্ন!)
Split ২ (বাম শাখায়)	Amount > 11.5	বামে: {10,11,9} — ডানে: {12,13}
Split ৩ (আরও ভাঙতে থাকবে)	...	প্রতিটি স্বাভাবিক পয়েন্ট সম্পূর্ণ বিচ্ছিন্ন হতে আরও ২-৩টি Split প্রয়োজন হবে

লক্ষ করুন, 500 মানটি মাত্র ১টি Split-এই (Path Length $h(x) = 1$) সম্পূর্ণভাবে বাকি সবার থেকে বিচ্ছিন্ন হয়ে গেছে, কারণ এটি বাকি ডেটার তুলনায় সংখ্যাগতভাবে অনেক দূরে অবস্থিত। অন্যদিকে, 10, 11, 12, 13, 9 — এই মানগুলো একে অপরের অত্যন্ত কাছাকাছি থাকায় তাদের সম্পূর্ণ আলাদা করতে অনেক বেশি Split (Path Length ৩ থেকে ৪ বা তার বেশি) প্রয়োজন হবে। এই সংক্ষিপ্ত Path Length-ই 500-কে একটি স্পষ্ট Anomaly হিসেবে চিহ্নিত করে দেয়।

গাণিতিক ভিত্তি — Anomaly Score

একটি পয়েন্ট x -এর Anomaly Score $s(x, n)$ নিচের সূত্রে গণনা করা হয়, যেখানে $E(h(x))$ হলো সবগুলো গাছ জুড়ে x -এর গড় Path Length, এবং $c(n)$ হলো n সংখ্যক Sample-এর একটি সাধারণ Binary Search Tree-তে গড় Path Length-এর একটি Normalization ফ্যাক্টর, যা Score-কে ০ থেকে ১-এর মধ্যে আনতে ব্যবহৃত হয়:

Formula:

$$s(x, n) = 2^{-\frac{E(h(x))}{c(n)}}$$

এখানে $c(n)$ নিচের সূত্রে গণনা করা হয় ($H(i)$ হলো Harmonic Number, যা প্রায় $\ln(i) + 0.5772$ (Euler's Constant)-এর কাছাকাছি):

Formula:

$$c(n) = 2H(n-1) - \frac{2(n-1)}{n}$$

Score-এর ব্যাখ্যা

- $s(x, n) \rightarrow 1$ -এর কাছাকাছি হলে সেই পয়েন্টটি অত্যন্ত সম্ভাব্য একটি Anomaly, কারণ এর Path Length খুবই ছোট (খুব কম Split-এই বিচ্ছিন্ন হয়ে গেছে)।
- $s(x, n) \rightarrow 0.5$ -এর কাছাকাছি হলে সেই পয়েন্টটি সম্পূর্ণ ডেটাসেটের একটি স্বাভাবিক বা Normal অংশ হিসেবে বিবেচিত হয়, কারণ এর Path Length গড় Path Length-এর কাছাকাছি।
- $s(x, n) 0.5$ -এর চেয়েও ছোট হলে, সেই পয়েন্ট নিশ্চিতভাবে Normal ডেটার একদম কেন্দ্রে বা ঘন এলাকায় অবস্থিত, যেখানে তাকে আলাদা করা সবচেয়ে কঠিন।

গুরুত্বপূর্ণ হাইপারপ্যারামিটার — বিস্তারিত ব্যাখ্যা

n_estimators — Default: 100

কাজ কী: মোট কতগুলো Isolation Tree তৈরি হবে তা নির্ধারণ করে। কখন Default রাখব: ১০০টি গাছ সাধারণত একটি স্থিতিশীল ও নির্ভরযোগ্য গড় Path Length পেতে যথেষ্ট। কখন পরিবর্তন করব: যদি ফলাফল Run-থেকে-Run অস্থির মনে হয়, তবে n_estimators বাড়িয়ে (যেমন ২০০-৩০০) স্থিতিশীলতা বাড়ানো যায়, যদিও এতে গণনার সময় সামান্য বাড়বে।

max_samples — Default: 'auto' (min(256, n))

কাজ কী: প্রতিটি গাছ তৈরির সময় সম্পূর্ণ ডেটাসেট থেকে Randomly কতগুলো Sample ব্যবহার করা হবে তা নির্ধারণ করে। কখন Default রাখব: গবেষণায় দেখা গেছে যে মাত্র ২৫৬টি Sample-ই সাধারণত একটি নির্ভরযোগ্য Isolation Tree তৈরি করতে যথেষ্ট, তাই এই ছোট Default মান

দ্রুতগতির জন্য বিশেষভাবে বেছে নেওয়া হয়েছে। কখন পরিবর্তন করব: ডেটাসেট অত্যন্ত ছোট হলে এই মান সম্পূর্ণ ডেটাসেটের সমান বা কাছাকাছি রাখা যেতে পারে।

contamination — Default: 'auto'

কাজ কী: ডেটাসেটে আনুমানিক কত শতাংশ Anomaly আছে তা নির্দিষ্ট করে, যা predict() মেথডে Normal (1) বা Anomaly (-1) নির্ধারণের Decision Threshold ঠিক করতে ব্যবহৃত হয়।
কখন পরিবর্তন করব: যদি ব্যবসায়িক প্রেক্ষাপট থেকে আনুমানিক জানা থাকে যে ডেটার প্রায় ২% Fraud বা Anomaly, তবে contamination=0.02 সেট করলে Threshold সেই অনুপাতে সামঞ্জস্য হবে।

Parameter	Default	সংক্ষিপ্ত কাজ
max_features	1.0	প্রতিটি Split-এর সময় সর্বোচ্চ কতগুলো ফিচার বিবেচনা করা হবে।
bootstrap	False	Sample গুলো Replacement সহ (Bootstrap) নাকি ছাড়া নেওয়া হবে তা নির্ধারণ করে।
random_state	None	ফলাফল পুনরুৎপাদনযোগ্য (Reproducible) রাখার জন্য একটি ফিক্সড সংখ্যা দেওয়া উচিত (যেমন 42)।

গুরুত্বপূর্ণ অ্যাট্রিবিউট ও মেথড

- predict(X) : প্রতিটি পয়েন্টকে Normal (1) নাকি Anomaly (-1) হিসেবে চিহ্নিত করে।
- decision_function(X) : প্রতিটি পয়েন্টের জন্য একটি Continuous Score দেয় — যত ঋণাত্মক (Negative), তত বেশি Anomalous, যা Ranking-এর জন্য ব্যবহার করা যায়।
- score_samples(X) : প্রতিটি পয়েন্টের র -অ্যানোমালি স্কোর (Raw Anomaly Score) প্রদান করে।
- estimators_ : ট্রেনিং হওয়া সবগুলো পৃথক Isolation Tree-এর অ্যারে।

কোথায় ব্যবহার হয়?

- ফাইন্যান্স ও ব্যাংকিং: ক্রেডিট কার্ড লেনদেনে জালিয়াতি (Fraud) শনাক্তকরণ — অস্বাভাবিক পরিমাণ বা অস্বাভাবিক লোকেশনের লেনদেন দ্রুত চিহ্নিত করা।
- নেটওয়ার্ক সিকিউরিটি: Intrusion Detection — অস্বাভাবিক নেটওয়ার্ক ট্রাফিক প্যাটার্ন বা Cyber Attack শনাক্ত করা।
- শিল্প ও উৎপাদন (Manufacturing): সেন্সর ডেটায় যন্ত্রের অস্বাভাবিক আচরণ শনাক্ত করে যন্ত্র বিকল হওয়ার আগেই Predictive Maintenance করা।
- স্বাস্থ্যসেবা: রোগীর ভাইটাল সাইনে (ECG, Blood Pressure) অস্বাভাবিক প্যাটার্ন প্রাথমিকভাবে ফ্ল্যাগ করে চিকিৎসকের নজরে আনা।

প্রশ্ন: Isolation Forest-এ কেন Gini বা Entropy-এর মতো Impurity পরিমাপের প্রয়োজন হয় না?

উত্তর: সাধারণ Decision Tree-এর লক্ষ্য থাকে ডেটাকে Class অনুযায়ী নিখুঁতভাবে আলাদা করা, তাই সেখানে Gini বা Entropy দিয়ে 'সেরা' Split খুঁজে বের করতে হয়। কিন্তু Isolation Forest-এর লক্ষ্য সম্পূর্ণ ভিন্ন — এটি শুধু জানতে চায় একটি পয়েন্টকে বিচ্ছিন্ন করতে কতগুলো Random Split প্রয়োজন হয়। যেহেতু Anomaly পয়েন্টগুলো এমনিতেই সংখ্যায় কম ও বাকিদের থেকে ভিন্ন, তাই সম্পূর্ণ Random Split ব্যবহার করলেও তারা স্বাভাবিকভাবেই দ্রুত বিচ্ছিন্ন হয়ে যায় — এই কারণে জটিল Impurity গণনার প্রয়োজন পড়ে না, যা অ্যালগরিদমকে অনেক দ্রুতগতির করে তোলে।

Quick Revision

- Type: Unsupervised, Tree-ভিত্তিক Anomaly Detection Algorithm।
- মূল ধারণা: Anomaly পয়েন্ট সংখ্যায় কম ও ভিন্ন হওয়ায় Random Split-এর মাধ্যমে দ্রুত (কম Path Length-এ) বিচ্ছিন্ন হয়ে যায়।
- Score: গড় Path Length থেকে গণনা করা Anomaly Score 1-এর কাছাকাছি হলে Outlier, 0.5-এর কাছাকাছি হলে Normal।

- সুবিধা: High-Dimensional ডেটায় Scalable, Distance বা Density গণনার প্রয়োজন নেই বলে অত্যন্ত দ্রুতগতির।

One-Class SVM

One-Class SVM (Support Vector Machine) হলো একটি Unsupervised Anomaly বা Novelty Detection Algorithm, যা ঐতিহ্যবাহী Support Vector Machine-এর মূল ধারণার ওপর ভিত্তি করে তৈরি। সাধারণ SVM যেখানে দুই বা ততোধিক Class-কে (যেমন 'Spam' বনাম 'Not Spam') আলাদা করার জন্য একটি Decision Boundary শেখে, সেখানে One-Class SVM শুধুমাত্র একটি Class ('Normal' ডেটা) নিয়ে কাজ করে এবং সেই Normal ডেটাকে ঘিরে একটি নিখুঁত ও যতটা সম্ভব টাইট সীমানা (Boundary) তৈরি করার চেষ্টা করে।

উদাহরণ (Intuition)

কল্পনা করুন, আপনি শুধুমাত্র সুস্থ মানুষদের হার্টবিট সাইন ডেটা (Heart Rate, Blood Pressure) দিয়ে একটি মডেল ট্রেন করছেন — অসুস্থ রোগীর কোনো ডেটা আপনার কাছে নেই, অথবা থাকলেও এতটাই কম যে তা দিয়ে একটি ভালো Classifier ট্রেন করা সম্ভব নয়। One-Class SVM এই সুস্থ মানুষদের ডেটার চারপাশে একটি টাইট, নিখুঁত সীমানা (Boundary) আঁকে, ঠিক যেন একটি অদৃশ্য 'বেড়া' দিয়ে সব স্বাভাবিক ডেটা পয়েন্টকে ঘিরে ফেলা হচ্ছে। ভবিষ্যতে নতুন কোনো রোগীর ডেটা যদি এই বেড়ার ভেতরে পড়ে, তবে তাকে 'স্বাভাবিক' বা Normal ধরা হয়; আর বেড়ার বাইরে পড়লে তাকে 'অস্বাভাবিক' বা Anomaly/Novelty হিসেবে চিহ্নিত করা হয়।

মূল কার্যপ্রণালী (How it Works) — ধাপে ধাপে

১. Kernel Trick ব্যবহার

সাধারণ SVM-এর মতোই One-Class SVM ডেটাকে একটি উচ্চ-মাত্রিক (Higher-Dimensional) Feature Space-এ ম্যাপ করে, সাধারণত RBF (Radial Basis Function) বা Gaussian Kernel ব্যবহার করে। এর মাধ্যমে মূল স্পেসে জটিল ও Non-linear আকৃতির ডেটার জন্যও একটি কার্যকর সীমানা আঁকা সম্ভব হয়, যা সরাসরি মূল স্পেসে সম্ভব হতো না।

২. Origin থেকে সর্বোচ্চ দূরত্বে Hyperplane

এই উচ্চ-মাত্রিক স্পেসে অ্যালগরিদম এমন একটি Hyperplane (একটি সমতল বা সীমানা) খোঁজে, যা অধিকাংশ Training ডেটা পয়েন্টকে Origin (কেন্দ্রবিন্দু বা মূলবিন্দু) থেকে সর্বোচ্চ

সম্ভব দূরত্বে আলাদা করে রাখে। এর অর্থ হলো, ডেটাকে Origin থেকে যতটা সম্ভব দূরে ঠেলে দিয়ে তাদের চারপাশে একটি যতটা সম্ভব টাইট সীমানা তৈরি করা হয়, যাতে সীমানার বাইরের এলাকা যতটা সম্ভব বড় থাকে — এই বড় বাইরের এলাকাটাই ভবিষ্যতে Anomaly শনাক্ত করার জন্য ব্যবহৃত হবে।

৩. Soft Margin (nu Parameter)

বাস্তব জীবনের ডেটায় প্রায়ই সব Training পয়েন্টকে নিখুঁতভাবে সীমানার ভেতরে রাখা সম্ভব হয় না (কারণ Training ডেটাতেও সামান্য Noise থাকতে পারে)। তাই nu প্যারামিটার দিয়ে নির্দিষ্ট করা হয় যে Training ডেটার সর্বোচ্চ কত শতাংশ পয়েন্টকে সীমানার বাইরে (Outlier হিসেবে) থাকতে দেওয়া 'অনুমোদিত' বা গ্রহণযোগ্য — এই নমনীয়তাকে বলা হয় Soft Margin।

8. Decision Function

একবার Boundary শেখা হয়ে গেলে, নতুন যেকোনো পয়েন্টের জন্য অ্যালগরিদম গণনা করে সে এই শেখা সীমানার ভেতরে (Positive Side, অর্থাৎ Normal) নাকি বাইরে (Negative Side, অর্থাৎ Anomaly) অবস্থান করছে।

গাণিতিক ভিত্তি — বিস্তারিত

One-Class SVM (Schölkopf-এর প্রস্তাবিত ν -SVM Formulation অনুযায়ী) নিচের Optimization সমস্যাটি সমাধান করে। এখানে w হলো Hyperplane-এর দিক নির্দেশক ভেক্টর, ρ (Rho) হলো Origin থেকে Hyperplane-এর দূরত্ব, ξ_i (X_i) হলো Slack Variable যা কিছু পয়েন্টকে সীমানার ভেতরে না থেকে একটু 'বাইরে' থাকার অনুমতি দেয়, এবং ν (ν) $\in (0, 1]$ হলো Outlier Fraction নিয়ন্ত্রণকারী প্যারামিটার:

Formula:

$$\min \frac{1}{2} \|w\|^2 + \frac{1}{\nu n} \sum_{i=1}^n \xi_i - \rho$$

Subject to:

$$w \cdot \phi(x_i) \geq \rho - \xi_i$$

$$\xi_i \geq 0$$

এখানে $\phi(x)$ হলো Kernel Function দ্বারা মূল ডেটাকে উচ্চ-মাত্রিক স্পেসে Mapping করার একটি ফাংশন। এই Optimization সমস্যার লক্ষ্য হলো $\|w\|$ (Hyperplane-এর জটিলতা) সর্বনিম্ন রাখা এবং একই সাথে ρ (Origin থেকে দূরত্ব) সর্বোচ্চ রাখা, যাতে যতটা সম্ভব বেশি ডেটা Origin থেকে দূরে, একটি টাইট গ্রুপে থাকে। Optimization সমাধান হলে চূড়ান্ত Decision Function নিম্নরূপ হয়:

Formula:

$$f(x) = \text{sign}(w \cdot \phi(x) - \rho)$$

$f(x)$ -এর মান ধনাত্মক (Positive) হলে পয়েন্টটি সীমানার ভেতরে (Normal) বিবেচিত হয়; ঋণাত্মক (Negative) হলে পয়েন্টটি সীমানার বাইরে (Anomaly) বিবেচিত হয়।

গুরুত্বপূর্ণ হাইপারপ্যারামিটার — বিস্তারিত ব্যাখ্যা

kernel — Default: 'rbf'

কাজ কী: কোন Kernel Function ব্যবহার করে ডেটাকে উচ্চ-মাত্রিক স্পেসে ম্যাপ করা হবে — 'linear', 'poly', 'rbf', 'sigmoid' এর মধ্যে থেকে বেছে নেওয়া যায়। কখন Default রাখব: RBF Kernel সবচেয়ে বেশি ব্যবহৃত হয় কারণ এটি Non-linear ও জটিল আকৃতির সীমানা আঁকতে সক্ষম, যা বেশিরভাগ বাস্তব ডেটাসেটের জন্য প্রয়োজন হয়। কখন পরিবর্তন করব: যদি ডেটা স্পষ্টভাবে Linear সম্পর্কযুক্ত হয়, তবে 'linear' Kernel ব্যবহার করলে দ্রুততর এবং সহজে ব্যাখ্যাযোগ্য ফলাফল পাওয়া যেতে পারে।

nu — Default: 0.5

কাজ কী: এটি দুটি জিনিস একসাথে নিয়ন্ত্রণ করে — Training ডেটার সর্বোচ্চ কত ভগ্নাংশ Outlier হিসেবে থাকতে পারবে তার একটি Upper Bound, এবং Support Vector সংখ্যার একটি Lower Bound। মান ০ থেকে ১-এর মধ্যে থাকে। কখন Default রাখব: $nu=0.5$ মানে প্রায় অর্ধেক ডেটাকে Support Vector ধরা হচ্ছে, যা অনেক বাস্তব সমস্যার জন্য অতিরিক্ত নমনীয়। কখন পরিবর্তন করব: যদি জানা থাকে ডেটাসেটের প্রকৃত Anomaly অনুপাত কম (যেমন মাত্র ৫%), তবে $nu=0.05$ সেট করলে মডেল অনেক বেশি 'কড়া' বা টাইট সীমানা তৈরি করবে, যা বাস্তব পরিস্থিতির সাথে বেশি সামঞ্জস্যপূর্ণ হবে।

gamma — Default: 'scale'

কাজ কী: RBF Kernel-এ প্রতিটি একক Training পয়েন্টের প্রভাব কতদূর পর্যন্ত বিস্তৃত হবে তা নিয়ন্ত্রণ করে। কখন Default রাখব: 'scale' মান ফিচারের Variance অনুযায়ী স্বয়ংক্রিয়ভাবে একটি যুক্তিসঙ্গত gamma নির্ধারণ করে, যা বেশিরভাগ ক্ষেত্রে ভালো কাজ করে। ঝুঁকি কী: gamma অতিরিক্ত বড় রাখলে প্রতিটি পয়েন্টের প্রভাব খুবই সংকীর্ণ এলাকায় সীমাবদ্ধ থাকে, ফলে সীমানা অত্যন্ত জটিল ও Training ডেটা-নির্দিষ্ট (Overfit) হয়ে যায়। gamma অতিরিক্ত ছোট রাখলে সীমানা অতিরিক্ত মসৃণ ও সাধারণ (Underfit) হয়ে পড়ে, যা সূক্ষ্ম Anomaly ধরতে ব্যর্থ হতে পারে।

Parameter	Default	সংক্ষিপ্ত কাজ
degree	3	যদি kernel='poly' হয়, তবে Polynomial Function-এর ডিগ্রি নির্ধারণ করে।
tol	1e-3	Optimization থামানোর জন্য Convergence-এর সহনশীলতা (Tolerance)।
shrinking	True	একটি Optimization Heuristic যা ট্রেনিং প্রক্রিয়াকে দ্রুততর করতে সাহায্য করে।

গুরুত্বপূর্ণ অ্যাট্রিবিউট ও মেথড

- predict(X) : প্রতিটি পয়েন্টকে Normal (1) নাকি Anomaly (-1) হিসেবে চিহ্নিত করে।

- `decision_function(X)` : Hyperplane থেকে প্রতিটি পয়েন্টের স্বাক্ষরযুক্ত (Signed) দূরত্ব প্রদান করে, যা Ranking-এর জন্য উপযোগী।
- `support_vectors_` : Boundary নির্ধারণে সরাসরি ভূমিকা রাখা Support Vector পয়েন্টগুলোর অ্যারে — শুধুমাত্র এই পয়েন্টগুলোই আসলে সীমানা নির্ধারণ করে, বাকি সব পয়েন্ট উপেক্ষা করা যায়।
- `support_` : Support Vector হিসেবে চিহ্নিত ডেটা পয়েন্টগুলোর Index।
- `n_support_` : মোট কতগুলো Support Vector ব্যবহৃত হয়েছে তার সংখ্যা।

ফিচার স্কেলিং কেন বাধ্যতামূলক?

One-Class SVM সম্পূর্ণভাবে Kernel Function-এর মাধ্যমে দূরত্ব গণনার ওপর নির্ভর করে কাজ করে। যদি ডেটার একটি ফিচারের স্কেল অন্য ফিচারের তুলনায় অনেক বড় হয় (যেমন Salary ০ থেকে ১০,০০,০০০ বনাম Age ০ থেকে ১০০), তাহলে বড় স্কেলের ফিচারটি পুরো দূরত্ব গণনাকে সম্পূর্ণভাবে প্রাধান্য দিয়ে ফেলবে এবং ছোট স্কেলের ফিচারের প্রভাব প্রায় হারিয়ে যাবে। তাই One-Class SVM প্রয়োগের আগে StandardScaler বা MinMaxScaler দিয়ে সব ফিচারকে একই স্কেলে আনা একেবারে বাধ্যতামূলক একটি ধাপ।

Isolation Forest বনাম One-Class SVM (বিস্তারিত তুলনা)

বিষয়	Isolation Forest	One-Class SVM
ভিত্তি	Tree-ভিত্তিক Random Partitioning	Kernel-ভিত্তিক Boundary/Hyperplane Learning
বড় ডেটাসেটে গতি	দ্রুত ও Scalable (Linear-এর কাছাকাছি Complexity)	ধীরগতির, বিশেষত n বড় হলে ($O(n^2)$ বা তার বেশি Complexity)
High-Dimensional ডেটা	সাধারণত ভালো কাজ করে	Kernel ও gamma যথাযথভাবে টিউন না করলে দুর্বল ফলাফল দিতে পারে

বিষয়	Isolation Forest	One-Class SVM
ফিচার স্কেলিং প্রয়োজন?	না (Tree-ভিত্তিক বলে Scale-Independent)	হ্যাঁ, বাধ্যতামূলক (Distance/Kernel-ভিত্তিক বলে)
Decision Boundary-এর ধরন	পরোক্ষভাবে Path Length দিয়ে নির্ধারিত	সরাসরি একটি সুনির্দিষ্ট গাণিতিক সীমানা (Hyperplane)
Hyperparameter সংবেদনশীলতা	তুলনামূলক কম সংবেদনশীল	nu ও gamma-এর সঠিক মানের ওপর ফলাফল অত্যন্ত নির্ভরশীল

কোথায় ব্যবহার হয়?

- Novelty Detection: যখন Training ডেটাতে শুধুমাত্র 'স্বাভাবিক' Class-এর ডেটা থাকে এবং ভবিষ্যতে সম্পূর্ণ নতুন বা অস্বাভাবিক (Novel) নমুনা শনাক্ত করতে হয়, যেমন উৎপাদন লাইনে সম্পূর্ণ নতুন ধরনের ত্রুটিযুক্ত পণ্য শনাক্তকরণ।
- স্বাস্থ্যসেবা: শুধুমাত্র সুস্থ রোগীর ডেটা দিয়ে ট্রেনিং করে অস্বাভাবিক মেডিকেল রিপোর্ট বা ECG সিগন্যাল শনাক্ত করা।
- সাইবার সিকিউরিটি: স্বাভাবিক ব্যবহারকারীর আচরণ শেখার পর অস্বাভাবিক Login Pattern বা Access Pattern শনাক্ত করা।
- গুণমান নিয়ন্ত্রণ (Quality Control): শিল্প উৎপাদনে শুধুমাত্র ভালো পণ্যের নমুনা দিয়ে ট্রেনিং করে ত্রুটিপূর্ণ পণ্য আলাদা করা।

প্রশ্ন: One-Class SVM এবং সাধারণ (Two-Class) SVM-এর মধ্যে মূল পার্থক্য কী?

উত্তর: সাধারণ SVM দুই বা ততোধিক Class-এর ডেটা থেকে শেখে এবং তাদের মধ্যে একটি Decision Boundary আঁকে যা একটি Class-কে অন্য Class থেকে আলাদা করে। কিন্তু One-Class SVM-এ শুধুমাত্র একটি Class-এর (Normal) ডেটা দেওয়া থাকে, তাই এটি দুই Class-এর মধ্যে সীমানা আঁকে না — বরং একটিমাত্র Class-এর ডেটাকে Origin থেকে দূরে ঠেলে দিয়ে তার চারপাশে একটি সীমানা তৈরি করে। মূল গাণিতিক Optimization একই SVM পরিবারের হলেও, লক্ষ্য এবং প্রয়োগ সম্পূর্ণ ভিন্ন।

Quick Revision

- Type: Unsupervised, Kernel-ভিত্তিক Anomaly/Novelty Detection Algorithm, ঐতিহ্যবাহী SVM-এর সম্প্রসারণ।
- মূল ধারণা: Normal ডেটাকে Origin থেকে সর্বোচ্চ দূরত্বে বিচ্ছিন্ন করে এমন একটি Hyperplane/Boundary খোঁজা।
- গুরুত্বপূর্ণ প্যারামিটার: nu (Outlier Fraction নিয়ন্ত্রণ করে) এবং gamma (RBF Kernel-এর সীমানার জটিলতা নিয়ন্ত্রণ করে)।
- প্রয়োজনীয়তা: ফিচার স্কেলিং (StandardScaler) বাধ্যতামূলক।
- সীমাবদ্ধতা: বড় ডেটাসেটে ধীরগতি, Hyperparameter-এর প্রতি অত্যন্ত সংবেদনশীল।

সামগ্রিক তুলনা (Overall Comparison)

অ্যালগরিদম	ক্যাটাগরি	মূল সমস্যা	মূল কৌশল
Gradient Boosting	Supervised (Ensemble)	Classification / Regression	Sequential Weak Learner, Residual শেখা
HistGradientBoosting	Supervised (Ensemble)	Classification / Regression	Histogram-Binning দিয়ে দ্রুত Boosting
Agglomerative Clustering	Unsupervised	Clustering	Bottom-Up Hierarchical একত্রীকরণ
t-SNE	Unsupervised	Dimensionality Reduction (Visualization)	Local Neighbor সাদৃশ্য সংরক্ষণ
Isolation Forest	Unsupervised	Anomaly Detection	Random Partitioning, Path Length
One-Class SVM	Unsupervised	Anomaly / Novelty Detection	Kernel-ভিত্তিক Boundary Learning